

The CNA Bible

CNA 0.1.0-alpha.1 Edition

The Core Framework, the Graphics Machine, the Ecosystem, the Platforms,
and the Practice

A comprehensive, revision-bounded technical account of CNA — a
C++23 reimplementation of the XNA 4.0 programming model —
and the foundation, content, renderer, and verification projects around it.

Grounded in pinned source code, tests, project records,
and retained verification artifacts.

CNA edition: `v0.1.0-alpha.1`

`1bb2145d99ed572dd4eb15009c34e2e5f410fcf0`

2026-08-20

Robert Vokac

Preface

CNA is a C++23 reimplementation of the Microsoft XNA 4.0 programming model. This is the 0.1.0-**alpha.1** edition, pinned to the immutable `v0.1.0-alpha.1` tag at `1bb2145d99ed572dd4eb15009c34e2e5f410fcf0`. The tag is CNA's first Semantic Versioning release and remains explicitly pre-1.0: the project may change public interfaces between prereleases. It does not turn every source-visible route into a stable or runtime-proven contract.

The release separates platform integration, graphics, and audio into independent build axes. SDL3 remains the default platform and audio implementation, but SDL2, headless and terminal platform routes and a null-audio route are represented separately. A build may contain one or several renderer families; 50 public renderer identities map to 46 implementation families, and a multi-renderer build chooses one at runtime before its first graphics device is created. CNA is neither a game nor an editor-centered engine. It is a compatibility framework for native C++ programs written in the shape of `Microsoft::Xna::Framework`.

The surrounding repositories provide capabilities CNA should not duplicate. `sharp-runtime` supplies selected .NET-shaped types; `easy-gl` and `meta-gl` provide the OpenGL/OpenGL ES layer used by five public identities; `free-direct` and `free-api` implement a narrow legacy compatibility route; `xna4-spec` makes the XNA API surface searchable; and the sample and example repositories exercise the framework from a user's perspective. Chapter 2 maps those relationships without treating every adjacent repository as part of CNA itself.

This book serves two roles. It is a technical monograph about CNA's architecture and engineering trade-offs, and a reference for people building, porting, testing, or extending CNA programs. It follows public API contracts into their pinned implementations, then separates what source inspection establishes from what a build, execution, renderer trace, independent oracle, or pixel comparison establishes. Bugs, partial routes, stale project documents, and unsupported configurations remain visible because they affect engineering decisions.

The book is not the project's live API documentation. Software changes faster than an edition can. Treat the text as a revision-bounded map: use it to locate the relevant subsystem and its constraints, then check the current repository before relying on a detail in new code. The full pin set and observed checkout state appear in Appendix D.

The release also introduces an experimental native C API. Appendix I documents its handle, result, version and coverage boundaries, including the exact-tag renderer identity mismatch that blocks the final library build. Its C ABI version is independent of the CNA product version; neither number is a substitute for the other.

The organization follows the path of a call through the system. Parts I and II introduce the ecosystem, build, framework lifecycle, math conventions, and module boundaries. Parts III and IV trace the public graphics API into renderer implementations and native APIs. Parts V and VI cover asset resolution, XNB, CNJ, glTF, models, animation, and conformance evidence. Part VII covers input, audio, media, devices, storage, networking, and related XNA services. The remaining parts cover foundation libraries, platforms, verification, and porting practice.

Appendix G is the authoritative evidence key. In short, API presence, successful compilation,

execution, renderer engagement, behavioral comparison, and pixel comparison are different observations. A green result is reported at the narrowest level the retained evidence supports. This convention lets a reader distinguish a working portable path from a source-visible implementation, a historical run, or a test that skipped before reaching its intended boundary.

The production process, including its use of AI-agent assistance, is described in the front matter that follows. The authority for technical claims remains the identified source and evidence, not the tool that drafted a sentence.

Reading Paths

The book is designed for selective reading. Chapters state their own evidence boundaries, while Appendix G defines the shared vocabulary. The routes below avoid repeating material and identify a useful first destination for common readers.

New CNA user

Read Chapters 1, 4, and 5. Continue with the game lifecycle (Chapter 6), SpriteBatch (Chapter 13), and content resolution (Chapter 33). Chapter 20 explains renderer choice; Chapter 79 lists material gaps at the edition pin.

XNA or FNA porter

Start with the compatibility boundary in Chapters 1 and 3, then use the migration workflow in Chapter 74. Consult Chapters 33–36 for assets, Chapter 17 for effects and custom shaders, and Appendix E when strict XNA compatibility matters.

Renderer implementer

Read Chapters 12, 19, 20, and 18. Appendix B summarizes the family-level contract; Chapters 71 and 70 explain engagement and oracle requirements.

Runtime renderer user

Read Chapters 4, 20, and 19. The selection chapter separates compiled families, configured default, runtime preference, fallback and the first-device latch; Appendix B supplies the family inventory and platform gates.

Native C consumer

Begin with Appendix I, then use Chapters 4 and 9 for build and link topology. Read Chapter 73 before interpreting the C API workflow and release-gate inventory as executed evidence.

Framework contributor

Begin with the lifecycle and module boundaries in Chapters 6, 9, and 10. Chapters 68 and 72 cover test registration and mechanical policy gates.

Content or glTF contributor

Read Chapters 33–37, then Chapters 39–43. Appendix H keeps importer, runtime, renderer, and oracle evidence in separate columns.

Verification contributor

Read Parts XI and Appendix G first. Chapter 69 defines reproducible counts; Chapter 70 distinguishes authority and tolerance; Chapter 73 records what automation did and did not exercise at the pin.

The shortest practical route

For a first program, the minimum sequence is: select a renderer when configuring CNA, build CNA as the repository documents, derive a class from `Game`, let `GraphicsDeviceManager` create the graphics device, load assets through `ContentManager`, and draw through an XNA-shaped

API such as `SpriteBatch`. Chapter 5 contains the complete small program and commands. Portable code stays within the `Microsoft::Xna::Framework` surface and treats CNAEXT-marked APIs as explicit extensions.

When a result differs by renderer, first identify the public selector, implementation family, native or translation API, and whether the intended route engaged. Then reduce the case to a state transition, readback, or pixel oracle. A successful process exit alone cannot localize a renderer-specific failure.

Production and Evidence Method

This edition was assembled and revised with extensive AI-agent assistance operating against pinned repositories, source code, tests, project records, and generated verification artifacts. Agents helped inventory APIs, compare claims across chapters, inspect implementation paths, draft and edit prose, generate structured diagrams, run validators, and challenge the resulting manuscript. AI output was not treated as technical authority. The cited repository state, external specification or implementation, test oracle, and retained artifact determine each claim.

The edition is deliberately revision-bounded. CNA behavior is described at `v0.1.0-alpha.1 @ 1bb2145d99ed572dd4eb15009c34e2e5f410fcf0`, while sibling behavior is described at the pins in Appendix D; executed results additionally name their configuration and host. Automated source scans, builds, PDF checks, and adversarial review passes reduce drift but cannot make a changing codebase timeless. Later editions should correct stale statements when the repositories or their evidence change. Responsibility for the published selection, wording, and claims remains with the named human author and editor.

Evidence labels used in this edition

Label	Meaning
Source-proven	A reachable implementation, fallback, refusal, or absence was established from the pinned source. No successful build or execution is implied.
Compile-proven	Names, signatures, includes, templates, and required link closure were exercised in the named configuration.
Runtime-observed	The named executable path ran and produced the recorded state, value, file, trace, or lifecycle event.
Renderer-engaged	The intended renderer family and native or translation route were shown to execute, rather than a substitute or early skip path.
Oracle-compared	The observation was compared with an identified authority under a stated normalization or tolerance policy.
Pixel-verified	Selected pixels or a complete frame were compared at the stated readback boundary, renderer, host, and tolerance.
Manually observed	A human inspected the result, but no retained discriminating oracle supplied an automated verdict.
Historically recorded	A dated project artifact reports the result; this edition did not rerun it.
Blocked / unsupported / not attempted	Three distinct outcomes: an external precondition prevented the attempt, the named path has no implementation, or no attempt was made.

These labels are coordinates, not a single quality ladder. A compile probe can answer an API question that a screenshot cannot; a hostile-input test and a pixel oracle exercise different contracts. Appendix G gives the full evidence vector and preferred authority for each kind of claim.

Repository paths and revisions serve as the primary references, so the book does not maintain a separate bibliography database. External links are few and reviewed. Source locations describe the pin and may move later; names, revision identifiers, and the repository map provide the

durable reproduction route.

PDF accessibility boundary

The source supplies a consistent heading hierarchy, document language and metadata, bookmarks, selectable text, Unicode mappings, and descriptions for figures. The current pdfLaTeX pipeline does not produce a fully tagged PDF, so this edition does not claim PDF/UA conformance. Moving to a tagged-PDF toolchain remains publishing work for a later release and requires a separate structural and assistive-technology review.

Contents

Preface	iii
Reading Paths	v
Production and Evidence Method	vii
I CNA and Its Ecosystem	1
1 CNA and XNA 4.0	3
1.1 The one-sentence version	3
1.2 Why XNA, and why now	4
1.3 Design goals, in the project’s own words	4
1.4 Verification is a vector, not a leaderboard	5
1.5 What CNA is not	5
1.6 Auditability as a framework property	6
1.7 The first tagged release and the continuing phase workflow	7
1.8 CNA’s software license	7
1.9 Reading map for the rest of this book	7
2 Ecosystem and Vocabulary	9
2.1 The dependency graph	9
2.2 The sibling build graph	9
2.3 sharp-runtime: the floor CNA stands on	10
2.4 easy-gl: OpenGL without a windowing opinion	11
2.5 free-direct: DirectX 3 scoped to two legacy games	12
2.6 xna4-spec: a ground truth to audit against	12
2.7 cna-samples and cna-examples: two different kinds of proof	12
2.8 Repositories mentioned but out of this book’s scope	13
3 Language, Conventions, and CNAEXT	15
3.1 The prime directive: match FNA, not taste	15
3.2 Two namespaces, two jobs	15
3.3 The CNAEXT tag: a compiler-enforced honesty check	16
3.4 C# properties, translated consistently	16
3.5 SharpRuntime type aliases: never a raw C++ primitive in the XNA surface	17
3.6 Events, as <code>System::EventHandler<T></code>	18
3.7 Interfaces as abstract base classes, <code>IDisposable</code> as a pattern	18
3.8 Visibility is chosen, not defaulted	18
3.9 Physical modules own the namespace-shaped include surface	18
3.10 No backward-compatibility shims	19
3.11 Documentation: full Doxygen, or nothing	19
3.12 Missing dependencies: stub correctly, never invent	19
3.13 The per-file porting checklist, in miniature	20

3.14	Why this chapter earns its place in a “bible”	20
II	Architecture, Building, and the Framework	21
4	Configuring and Building CNA	23
4.1	Prerequisites	23
4.2	The default build: Linux, the OpenGL ES 3 renderer	23
4.3	The full CMake option reference	24
4.4	Generated product-version API	26
4.5	Windows, natively and cross-compiled	26
4.6	Building for Android and the Web	27
4.7	System-installed SDL, if you prefer it	28
4.8	Linux system dependencies beyond SDL	28
4.9	Verifying a build	28
5	Your First CNA Game	31
5.1	The whole program	31
5.2	Reading it top to bottom	32
5.3	A second program: making it move	33
5.4	A third program: bouncing off the edges, with sound	34
5.5	A fourth program: rectangle collision	36
5.6	What happens if you change the renderer	37
6	The Game Lifecycle	39
6.1	Game: base class, not framework-owned entry point	39
6.2	GameTime: three fields, one privileged writer	45
6.3	GameComponent and DrawableGameComponent: the ordering contract	45
6.4	GameComponentCollection: an event-raising collection	47
6.5	GameServiceContainer: type-keyed lookup without C# reflection	47
6.6	GameWindow: one concrete class where FNA has a hierarchy	48
6.7	GraphicsDeviceManager’s role in the loop	49
6.8	Supporting cast: LaunchParameters, TitleContainer, TitleLocation, Frame- workDispatcher	51
6.9	One catch clause for the framework boundary	51
6.10	What this buys a porting engineer	52
7	Math Types and Coordinate Conventions	53
7.1	Mental model	53
7.2	Common value surface	53
7.3	Coordinate and composition contract	53
7.4	Vectors	54
7.5	Matrix	55
7.6	Quaternion	56
7.7	Planes, rays, and bounding volumes	56
7.8	Integer geometry and Color	57
7.9	MathHelper and degenerate inputs	58
7.10	Curves	58
7.11	Compiled world/view/projection example	59
7.12	Summary	60
8	Geometry, Color, Curves, and C++ Value Layout	61

8.1	The value-type resemblance has a hard limit	61
8.2	GPU streams are a second, plain-data representation	62
8.3	Planes and half-spaces	62
8.4	Bounds encode several non-obvious conventions	63
8.5	Rectangle, Point, and edge ownership	63
8.6	Color is packed, named, and polymorphic	63
8.7	Curves: ordered keys and explicit extrapolation	63
8.8	Exceptions are part of the port	64
8.9	What the math tests prove	64
9	The Module Monorepo	65
9.1	Location is ownership	65
9.2	Fifteen framework modules and one optional ABI module	65
9.3	The module factory and one public include root	66
9.4	Two deliberate construction exceptions	67
9.5	Renderer families are physical modules too	67
9.6	Examples moved to their owners	68
9.7	The migration was checked as a no-loss transformation	68
9.8	How to read a source path in this book	68
10	Cycles, Umbrellas, Boundaries, and Enforcement	69
10.1	Three deliberate archive cycles	69
10.2	The three umbrella targets	69
10.3	Narrow aliases are the architectural surface	70
10.4	Fourteen module probes	70
10.5	The Ninja caveat: registered does not mean executed	71
10.6	Include reachability and header self-containment	71
10.7	Source partition and legacy-root guards	71
10.8	How to change a boundary safely	72
11	Dependencies, Tests, Examples, and Developer Tools	73
11.1	Five acquisition mechanisms	73
11.2	Submodules: initialize, do not guess	73
11.3	The unusual SDL configure-time build	74
11.4	System-package gates can change source composition	74
11.5	Sibling repositories are part of the workspace contract	74
11.6	Renderer dependencies are intentionally asymmetric	75
11.7	Build-tree C++ consumption and the native C package	75
11.8	The CnaTests corpus	76
11.9	Examples are executable documentation, not one evidence tier	76
11.10	Presets are named environments, not universal promises	76
11.11	Developer tools and what they decide	76
III	The Graphics Machine	79
12	GraphicsDevice and the Rendering Pipeline	81
12.1	Construction and native lifetime	81
12.2	Public state	82
12.3	Clear, present, and reset	83
12.4	Render targets	84
12.5	Vertex and index bindings	85

12.6 Draw calls	86
12.7 Backbuffer readback	87
12.8 Resources and disposal	87
12.9 Adapters and format queries	88
12.10 CNA extensions and diagnostics	88
12.11 Compiled example: direct two-triangle rendering	89
12.12 Summary	89
13 SpriteBatch and 2D Rendering	91
13.1 Begin, Draw, End	91
13.2 Draw overloads and rotation origin	92
13.3 Vector positions are quantized before they reach a renderer	93
13.4 The D3D9 half-pixel offset and a mutation-testing trap	93
13.5 SpriteSortMode and SpriteEffects	94
13.6 Historical SpriteEffects layout defect	94
13.7 Begin() leaves GraphicsDevice.BlendState active	94
13.8 SpriteFont	95
13.9 SpriteFont evidence by renderer	96
13.10 Two corrected SDL_RENDERER defects	96
13.11 Flush timing: sorting without draw-call coalescing	96
13.12 DepthStencilState at Begin	97
13.13 Per-glyph advance math in DrawString	97
13.14 Exception-type mismatch for nested Begin()	98
13.15 Where this leaves a porting engineer	99
14 Textures and Render Targets	101
14.1 Texture: a shared Color gate with one renderer-specific branch	101
14.2 Texture3D and TextureCube: current structure and history	101
14.3 Texture2D	102
14.4 Texture3D and TextureCube	104
14.5 Render-target divergences	107
14.6 SurfaceFormat, without flattening the Skia exception	109
14.7 Buffers: VertexBuffer and IndexBuffer	109
14.8 Vertex layout: fixed by stride, except for EasyGL raw declarations	110
15 Stock Effects and Draw Parameters	113
15.1 Effect: stock and compiled modes	113
15.2 Stock parameters are not compiled-effect reflection	114
15.3 GpuDrawParams: the shared stock-effect draw packet	115
15.4 BasicEffect	115
15.5 AlphaTestEffect	116
15.6 DualTextureEffect	117
15.7 EnvironmentMapEffect	118
15.8 SkinnedEffect	120
15.9 Closing the preferPerPixelLighting divergence	121
15.10 The pattern behind these bugs	121
16 State Objects and Value Semantics	123
16.1 By-value state semantics	123
16.2 The value surface is broader than the binding bridge	124
16.3 BlendState	125
16.4 DepthStencilState	125

16.5	RasterizerState	127
16.6	SamplerState	128
16.7	The recurring device-default copy defect	130
17	Shaders: Four Answers to XNA's .fx	131
17.1	XNA's compiled-effect format	131
17.2	The public compiled-effect boundary	131
17.3	Answer one: reimplement the stock semantics	132
17.4	Answer two: recompile Microsoft's sources on D3D9	132
17.5	Answer three: execute compiled Effect Framework bytecode	133
17.6	Answer four: bypass .fx with ShaderEffect	133
17.7	The honest summary	139
18	Vertex Streams and Capabilities	141
18.1	Public vertex objects are not upload layouts	141
18.2	VertexDeclaration is the byte contract	141
18.3	Buffers, index widths, and dynamic updates	142
18.4	Bindings carry offset and input rate	142
18.5	Draw validation has a deliberate order	143
18.6	Thirteen capability questions	143
18.7	The default polarity is permissive	144
18.8	Unsupported 3D policy is separate from capability	144
18.9	Four interface-default failure shapes	144
18.10A	portable draw claim	145
IV	The Renderers	147
19	The Renderer Contract	149
19.1	Contract shape and default behavior	149
19.2	Presentation and input are family contracts	152
19.3	Resource and draw defaults	155
19.4	Targets, clears, viewport, and scissor	156
19.5	Capability answers and operational evidence	157
19.6	Selection and factory boundary	158
19.7	Implications for users and implementers	159
20	Renderer Selection and Identity	161
20.1	Identity, family, and native API are different axes	161
20.2	The complete registry, grouped by implementation	162
20.3	One renderer remains the default build mode	162
20.4	Opt-in multi-renderer builds	162
20.5	Runtime choice, precedence, and latch	163
20.6	Names that no longer select anything	163
20.7	Platform gates are part of identity	164
20.8	How the selected identity reaches C++	164
20.9	Family selection is not BGFX driver selection	165
20.10	Mechanical checks and the remaining CI trap	165
20.11A	selection checklist	165
21	OpenGL I: The Independent Compatibility Ladder	167
21.1	Four identities, four implementation families	167

21.2	OPENGL1: a modern contract on a fixed pipeline	167
21.3	OPENGL2: the GLSL 1.10 bridge	168
21.4	OPENGLS1: fixed function on the mobile branch	169
21.5	TINYGL: fixed function as a CPU rasterizer	169
21.6	The same contract does not imply the same proof	169
21.7	Choosing among the compatibility identities	170
22	OpenGL II: Programmable, Web, and Portable Profiles	171
22.1	Documentation status at the pin	171
22.2	Confirmed fixed since the original bug list	171
22.3	Confirmed pre-existing gaps	171
22.4	D3D9 oracle cross-comparison	173
22.5	Viewport and scissor	174
22.6	GetData() and CPU shadows	175
22.7	Clear is immediate GL state, but Target is not colour-only	176
22.8	Test baseline	176
22.9	Xvfb screenshot evidence	176
22.10	OPENGL4: modern core without the EasyGL family	177
22.11	PORTABLEGL: OpenGL-shaped software, not a GPU driver	177
22.12A	ten-identity decision map	178
23	Native Modern GPU APIs	179
23.1	Historical Vulkan campaign baseline	179
23.2	BlendState: native mapping and historical defect	180
23.3	Stencil testing: another discarded-parameter defect	180
23.4	OcclusionQuery: current contract and render-pass limit	180
23.5	Remaining limitation	181
23.6	Two corrected render-target black-frame defects	181
23.7	Clear is aspect-selective and issue-ordered	182
23.8	A confirmed non-gap: device-feature gating	182
23.9	Notable feature-matrix specifics	182
23.10	Render-target mipmaps, MSAA, and viewport	182
23.11	How a swapchain gets built: format, color space, and present mode	183
23.12	Two frames in flight, and the deferred-present trick	185
23.13	The 512-live-descriptor-set ceiling	187
23.14	Coverage gap	188
23.15	METAL: a narrow implementation with explicit refusals	188
23.16	IGL: one CNA identity over two IGL backends	188
23.17	Comparing the five evidence tiers	189
23.18	Legacy WebGPU Chapter Material	189
23.19	What “experimental” concretely means here	189
23.20	The native 2D baseline	189
23.21A	bug a manual screenshot review missed, and a pixel assertion caught	189
23.22	3D depth testing and the gap it exposed	190
23.23	Render targets, environment mapping, instancing, and mip generation	191
23.24	Three former limitations that are no longer true	194
23.25	Clear ordering and usage are consistent across target types	195
23.26	What remains open	195
23.27	Verification methodology, and how it differs from D3D9’s	196
23.28	Legacy SDL GPU Chapter Material	196
23.29	Why this renderer costs nothing new	196

23.30	Precompiled bytecode only, in a fixed resource-binding order	196
23.31	SkinnedEffect's storage-buffer workaround and push-uniform limit	197
23.32	Custom ShaderEffect runtime compilation without a development package	197
23.33	Pipeline state is snapshotted at draw time, except where it is deliberately not	198
23.34	Deferred rendering still preserves the game's draw order	199
23.35	Present timing belongs to the device manager, not a test-only renderer poke	200
23.36	Two violations exposed by validation mode	200
23.37	A pre-release render-target use-after-free	201
23.38	MRT as a shader capability	202
23.39	Usage selects per-resource first-use load/store behavior	202
23.40	Forced swapchain failure preserves the queued frame	204
23.41	What remains open	204
23.42	Clear state is per segment and issue-ordered	205
23.43	Verification methodology	205
24	BGFX, Magnum, LLGL, and Diligent	207
24.1	Clear owns an ordered, aspect-selective view	207
24.2	Recorded BGFX campaign	208
24.3	Choosing bgfx's underlying native renderer	208
24.4	DepthBias: an upstream BGFX limit	209
24.5	Render-target readback crashes: one root cause, five fixes	210
24.6	View IDs: a free-list-backed pool, not a hardcoded slot	210
24.7	Sampling a render target: a wrong-handle-type cast, since fixed	210
24.8	Corrected silent-black-mesh defect	211
24.9	OcclusionQuery: a real implementation, correctness not provable here	211
24.10	Instancing: exact transport, with a native-profile shader fault	212
24.11	Texture3D and TextureCube readback: copy first, then read	213
24.12	Viewport uses ordered views; scissor remains per draw	214
24.13	Current qualification	214
24.14	MAGNUM: a wrapper with no offline shader generator	214
24.15	LLGL: a broad library behind a deliberately narrower CNA path	215
24.16	DILIGENT: one HLSL program, several device types	215
24.17	Wrapper-selection checklist	215
25	Abstraction Layers II: Sokol, Wicked, and FNA3D	217
25.1	SOKOL: two selectors, only one mature cross-product	217
25.2	WICKED: an experimental Vulkan adapter	218
25.3	FNA3D: the XNA shader-program path	218
25.4	Six shader strategies across the wrapper chapters	219
25.5	Selection and verification	219
26	DirectX 1–3 and free-direct	221
26.1	DIRECTX1: before Direct3D existed	221
26.2	DIRECTX2: fixed function without execute buffers	221
26.3	DIRECTX3: a small port, not a renamed free-direct	222
26.4	Why the first three rungs matter	222
26.5	FREEDIRECT: a separate portable 2D branch	222
26.6	An unanticipated architectural constraint	222
26.7	A capability beyond SDL_RENDERER	222
26.8	Corrected FREEDIRECT defects	223
26.9	RenderTarget2D: the same shadow-backbuffer trick	223
26.10	Public depth/stencil clear	224

26.11	Known limitations at the pin	225
26.12	The transform math is sound, but public input takes another route	225
26.13	The fixed 32-bit surface decision	226
27	The Retro Ladder: DirectX 5–8 and Glide	229
27.1	DIRECTX5: the stable immediate-mode baseline	229
27.2	DIRECTX6: a small delta with stencil support	229
27.3	DIRECTX7: the flattened device model	230
27.4	DIRECTX8: a new device generation, deliberately no Shader Model 1	230
27.5	GLIDE: a contemporary fixed-function branch	230
27.6	What the ladder teaches	231
27.7	A practical retro verification pass	231
28	Modern Direct3D	233
28.1	D3D9: zero-tolerance XNA oracle comparisons	233
28.2	Three D3D11 resource-lifetime groups	236
28.3	DIRECTX10: a bounded MVP with one unsafe default	236
28.4	D3D11: native Windows implementation	237
28.5	MRT finalization across all bound targets	240
28.6	D3D12: shared foundations and split verification	240
28.7	One public <code>ShaderEffect</code> API, two Direct3D contracts	242
28.8	Persistent Direct3D resources implement the shared usage contract	244
28.9	Cube-target MSAA, mip generation, and public finalization	245
28.10	<code>HeadlessEXT</code> : routine off-screen D3D12 coverage	246
29	Windows 2D: Direct2D and GDI	247
29.1	DIRECT2D: a different Windows delivery stack	247
29.2	GDI: Win32 presentation over the software renderer	248
29.3	Choosing and testing the Windows 2D paths	248
30	2D and Vector Rasterizers	249
30.1	Scope, by design	249
30.2	Recorded SDL Renderer audit and later repairs	251
30.3	One blocked sampler decision, and one resource gap that later closed	252
30.4	Native and emulated behavior	253
30.5	Reading the feature matrix for this renderer	254
30.6	Xvfb screenshot evidence	254
30.7	Pixel readback and presentation mode	255
30.8	SKIA: a bounded contract written down before expansion	256
30.9	BLEND2D: CPU vectors presented as an SDL texture	256
30.10	OPENVG: a vector API on a historical GL substrate	257
30.11	NANOVG: immediate vectors without off-screen storage	257
30.12	Five ways to prove a 2D renderer	257
31	Web and DOM Renderers	259
31.1	CANVAS: immediate raster operations	259
31.2	HTML_DOM: sprites as pooled CSS objects	260
31.3	SVG_DOM: retained vector structure	260
31.4	PIXIJS: browser library objects and render textures	260
31.5	The removed ASCII identity	261
31.6	Comparison by object model	261
31.7	Deployment checklist	261

32 Non-GPU and Diagnostic Renderers	263
32.1 HEADLESS: behavior without an image	263
32.2 SOFTWARE: bounded CPU rasterization	263
32.3 STUB: the smallest terminal implementation	264
32.4 Choosing the right non-GPU test double	264
32.5 Capability discipline	265
 V The Content Pipeline	 267
33 ContentManager and Asset Resolution	269
33.1 The resolution algorithm	269
33.2 RootDirectory and containment	270
33.3 Direct and managed loading	271
33.4 Caching and ownership	271
33.5 Failure and concurrency boundaries	272
33.6 Provider, ResourceContentManager, and TitleContainer	272
33.7 Content manifest	273
33.8 Audio and video readers	273
33.9 Porting implications	274
33.10 Summary	274
 34 The XNB Container	 275
34.1 The ten-byte header	275
34.2 Compression is four-valued	275
34.3 LZX preserves state across blocks	276
34.4 The reader table starts inside the decoded stream	276
34.5 Limits and enforcement points	276
34.6 Length checks close the container boundary	277
34.7 What the container does not prove	277
 35 XNB Type Readers and Object Graphs	 279
35.1 Two registration systems	279
35.2 One object-graph session	280
35.3 ContentTypeReader and type erasure	281
35.4 Built-in reader families	281
35.5 Registration and platform inventory	284
35.6 Rules for custom readers	284
35.7 Summary	285
 36 CNJ: CNA's Own Content Format	 287
36.1 A small envelope, validated separately	287
36.2 Version ceilings belong to types	288
36.3 Built-in type vocabulary	288
36.4 sourceFile is a capability, not a convention	288
36.5 Two JSON implementations coexist	288
36.6 Custom loaders receive raw JSON	289
36.7 Binary sidecars remain format-specific	289
36.8 Production and provenance	289
36.9 The format in one rule	289
 37 Content and Asset Boundaries	 291

37.1	Limits are useful only where they are enforced	291
37.2	Top-level names and internal references have different trust rules	291
37.3	Object graphs are bounded on both recursive axes	292
37.4	Fuzzing reaches the public loader	292
37.5	Differential oracles reduce self-consistency risk	292
37.6	Error normalization and its limits	292
37.7	Current asymmetries to keep visible	293
VI	3D Content: Models, glTF, and Conformance	295
38	Models, Meshes, and Drawing	297
38.1	The runtime hierarchy	297
38.2	Two ways to draw a loaded model	297
38.3	The draw contract: a model is more than a bag of pointers	298
38.4	Four model asset routes, not two	300
38.5	Lifetime and copies: value syntax, one mutable graph	301
38.6	Skinning: two independent systems, not one	302
38.7	A worked example: naming an arbitrary root bone	305
38.8	Collections are read views, but not uniform ones	306
39	From glTF File to Model: the Import Core	309
39.1	The parser boundary	309
39.2	Scene graph first, meshes second	310
39.3	Groups are an ownership boundary	310
39.4	What extraction preserves	311
39.5	Failure and lifetime boundaries	311
40	GPU Packing and the Stride ABI	313
40.1	The ten glTF layouts	313
40.2	Why ordinary C++ structs were unsafe	313
40.3	Indices: source width is not destination width	314
40.4	All seven topology modes have an explicit route	315
40.5	Tangents and winding	315
40.6	Hard limits carried by the bytes	315
41	Skinning and Animation across Two Systems	317
41.1	The ordinary Model path	317
41.2	The Avatar path	317
41.3	Scene indices are not palette indices	318
41.4	The coordinate-space failure that produced D8	318
41.5	Clip extraction	319
41.6	Scaling and capacity	319
41.7	Practical update order	320
42	The CNJ Model Toolchain	321
42.1	A renderer-free build tool	321
42.2	The Model envelope is version 2	321
42.3	Binary sidecars and bounded reads	322
42.4	Offline and runtime parity	322
42.5	Occlusion through DualTextureEffect	323
42.6	Choosing a route	323

43 glTF Conformance and Oracles	325
43.1 Pinned authority and deterministic corpus	325
43.2 The ladder, including its L0 gate	325
43.3 Pixel evidence is renderer-specific	326
43.4 The append-only defect ledger	326
43.5 Current capability and loss reporting	327
43.6 What the milestone name means	327
 VII Input, Audio, Media, Devices, Networking, and Services	 329
44 The Input System	331
44.1 Architecture: event-driven, not poll-driven	331
44.2 Keyboard	331
44.3 Mouse	332
44.4 GamePad	334
44.5 Touch and gestures	336
44.6 TextInputEXT and CNA-only CNAEXT extensions	337
44.7 Focus loss and held input	340
44.8 Platform-specific boundaries	340
44.9 Status	341
 45 The Audio System	 343
45.1 Audio implementation is a separate axis	343
45.2 Mixer architecture and lifetime	343
45.3 SoundEffect and instances	343
45.4 Dynamic streaming	344
45.5 XACT objects	345
45.6 Content formats and historical corrections	345
45.7 Deliberate and open limits	346
45.8 Microphone	346
45.9 Evidence status	347
 46 The Media System	 349
46.1 Song contract	349
46.2 MediaLibrary and metadata	350
46.3 Playback and queue ownership	350
46.4 Visualization	351
46.5 Video contract	351
46.6 Platform limitations	351
46.7 Evidence status	352
 47 Sensors and Microsoft.Devices	 353
47.1 Sensor model and delivery	353
47.2 Concurrency contract	354
47.3 Compass and motion math	354
47.4 Vibration	355
47.5 Known limitation and evidence status	355
 48 Host Integration through CNA.Devices	 357
48.1 The option removes the types	357
48.2 The SDL wrapper map	357

48.3	Camera: poll, do not wait for a callback	358
48.4	Platform support is per class	358
48.5	Overlap with CNA::Input	359
48.6	Lifetime and test boundary	359
49	GamerServices and Local Identity	361
49.1	Local identities and object lifetime	361
49.2	Persistence contract	362
49.3	Achievements	362
49.4	Leaderboards	362
49.5	Guide overlays and stubs	363
49.6	Presence, privileges, and profile	363
49.7	Dispatcher, component, and the historical hang	363
49.8	Evidence and user implications	364
50	Storage and Persistence Boundaries	365
50.1	StorageDevice contract	365
50.2	Storage root resolution	365
50.3	StorageContainer contract	366
50.4	Open path-containment defects	366
50.5	File sharing is not enforced	367
50.6	Exception boundary and evidence	367
51	Networking and Session Semantics	369
51.1	Session types and ownership	369
51.2	SystemLink transport	370
51.3	Host migration	370
51.4	Delivery guarantees	370
51.5	Compatibility limits and asymmetries	371
51.6	Asynchronous lifecycle constraint	371
51.7	Lifetime hardening	371
51.8	Evidence status	372
52	Avatar: Faithful Absence and CNAEXT Rendering	373
52.1	Base API contract	373
52.2	SkinnedModelEXT contract	374
52.3	Animation and palette computation	374
52.4	Lighting, appearance, and wardrobe	375
52.5	Content pipeline	376
52.6	Open garment interpenetration	376
52.7	Renderer and evidence status	376
VIII	sharp-runtime: The Foundation Layer	377
53	Why sharp-runtime Exists	379
53.1	Scale	379
53.2	Namespace structure	379
53.3	Object and IDisposable: deliberately not two roots	380
53.4	EventHandler and MulticastAction: solving a problem C# doesn't have	380
53.5	Value types used by later chapters	381
53.6	Concurrency primitives	381

53.7	Permanent scope boundaries: what will never be ported, and why	383
53.8	The build is part of the public architecture	384
53.9	Local and tracked CI gates have different scope	384
54	Components and CNA Integration	387
54.1	Directories, components, and targets are different counts	387
54.2	Header names predict location, not the link closure	388
54.3	Standalone selection and embedded selection differ	388
54.4	CNA's sibling contract	388
54.5	The linking helper makes closures explicit	389
54.6	Tests enforce component ownership	389
54.7	Upgrade rules for the seam	390
55	The Object Model, Honestly	391
55.1	Object is opt-in, not universal	391
55.2	String is an API vocabulary, not storage	392
55.3	Exceptions join the C++ hierarchy	392
55.4	Lifetime is RAII first	393
55.5	Delegates have three representations	393
55.6	Type is identity without reflection	393
56	Namespaces, Types, and Include Paths	395
56.1	Two public roots, no umbrella	395
56.2	Namespaces do not equal components one-for-one	395
56.3	CNA's dependency subset	396
56.4	String and StringBuilder: static utility versus mutable buffer	396
56.5	Stream: a lightweight subset, honest about it	397
56.6	UTF8Encoding: conformance validation and fallback evidence	399
56.7	Generic continuations exist; generic combinators do not	401
56.8	Reading a dependency from source	401
57	Parity and Permanent Deviations	403
57.1	Three outcomes, not one generic stub	403
57.2	Reflection: correctly a stub, not a gap	403
57.3	Garbage collection: inert compatibility answers, by design	403
57.4	Delegates: a three-tier model, corrected once in the project's own history	404
57.5	Serialization, P/Invoke, and cryptography: three different reasons for the same conclusion	405
57.6	IDisposable and the exception hierarchy: parity by pattern, not by mechanism	405
57.7	Naming is a compatibility surface	407
57.8	Two further, narrower accepted boundaries	407
57.9	A documented deviation still needs verification	407
58	Verification and Audit in sharp-runtime	409
58.1	Three test locations have three jobs	409
58.2	The executable boundary tests the dependency boundary	409
58.3	The measured gate is not green	410
58.4	Negative consumers prove absence	410
58.5	Audit is a tracked data set	410
58.6	A completed task list triggered more skepticism	411
58.7	Sanitizers answer a different class of question	411
58.8	Reproducibility has an external seam	412

IX Sibling Foundation Libraries	413
59 easy-gl and meta-gl: Profile, Loader, and Dispatch	415
59.1 The ownership split	415
59.2 Device initialization delegates context truth downward	415
59.3 Capability gating by feature, not version	416
59.4 Meta-gl's typed call surface and invalid-input contract	417
59.5 Design corrections and one breaking change	417
59.6 Two tiers of context-loss recovery	419
59.7 Testing without a GPU	421
59.8 Two known, honestly stated limitations	421
59.9 The public history was rewritten without changing the trees	421
60 free-direct and free-api	423
60.1 A game-driven boundary	423
60.2 Three DirectX subsystems, no Direct3D	423
60.3 The two libraries form a one-way dependency	424
60.4 CNA is the odd consumer	424
60.5 The <code>fopen</code> macro is a real containment cost	425
60.6 FREEDIRECT is not DIRECTX3	425
60.7 Verification boundaries	425
X Cross-Platform Engineering	427
61 The Cross-Platform Contract	429
61.1 The five-axis model	429
61.2 The platform contract is implemented	429
61.3 Implemented, host-limited, and reserved choices	430
61.4 Platform capabilities and renderer needs	431
61.5 Audio remains orthogonal	431
61.6 Target taxonomy is still useful, but different	431
61.7 A platform claim is an evidence vector	431
62 Windows I: Cross-Compiling from Linux	433
62.1 Two MinGW toolchains	433
62.2 Vendored SDL is built for the target, not reused from the host	433
62.3 Runtime staging is part of a successful build	434
62.4 CTest must know that PE files are not host executables	434
62.5 Native MSVC is a separate, unproven lane	435
62.6 The i686 wall	435
63 Windows II: Wine and Engagement Gates	437
63.1 Six prefixes isolate incompatible runtime stacks	437
63.2 Which translation runtime owns each renderer	438
63.3 An engagement gate turns logs into a verdict	438
63.4 Environment-specific workarounds are part of the evidence	439
63.5 D3D12 and Proton require matched ownership	439
63.6 The D3D9 oracle controls both sides	439
63.7 What translated execution cannot prove	439
64 Web I: C++23 to WebAssembly	441

64.1	The toolchain changes the execution contract	441
64.2	Assets must cross the virtual-filesystem boundary	441
64.3	Networking is a different topology, not a transparent port	442
64.4	What a successful build establishes	442
64.5	A reproducible build record	442
64.6	The portability lesson	443
65	Web II: the Main-Loop Problem	445
65.1	The throw that is not a C++ exception	445
65.2	Reducing the failure before explaining it	445
65.3	The repair and its boundary	446
65.4	The Web loop is not the native loop	446
65.5	How to test a host-owned loop	446
66	Web III: Six Renderers, Four Kinds of Evidence	449
66.1	An evidence ladder	449
66.2	HTML_DOM: automated browser and pixel evidence	449
66.3	SVG_DOM: browser harness plus bundle CI	450
66.4	WEBGL2 and WebGL1: translation without browser execution	450
66.5	CANVAS: startup failure before renderer evidence	450
66.6	PIXIJS: new source route, browser proof still required	450
66.7	The matrix	451
66.8	A common browser-verification shape	451
67	Android and Apple Platform Evidence	453
67.1	Android application structure	453
67.2	Recorded Android execution	453
67.3	Lifecycle coverage is explicit but partial	454
67.4	Assets expose a deeper integration gap	454
67.5	macOS: native routes are declared	454
67.6	iOS: final-link and simulator route; tvOS remains a non-claim	455
XI	Testing and Verification	457
68	The CnaTests Architecture	459
68.1	One discovered GoogleTest binary	459
68.2	One configure, one renderer set	459
68.3	Standalone examples are a separate corpus	460
68.4	Why some tests need another process	460
68.5	Compilation failure can be the oracle	460
68.6	Discovery, selection, execution, verdict	460
69	Counting Discipline	463
69.1	Four quantities that must not be exchanged	463
69.2	How the static counts are reproduced	463
69.3	Why there is no all-renderer total	464
69.4	Identity counts can be settled	464
69.5	Counts that require execution	464
69.6	A reporting grammar	464
70	Oracles: What Does “Correct” Mean?	467

70.1	The captured-XNA image corpus	467
70.2	A policy table is part of the oracle	468
70.3	Project-owned goldens answer a weaker question	468
70.4	Point samples and the skip boundary	468
70.5	FNA as a numeric reference	468
70.6	Independent oracles inside the corpus	469
70.7	Choose authority before tolerance	469
71	Verification in Hostile Environments	471
71.1	The Wine wrapper as a verdict pipeline	471
71.2	Discovery needs a controlled escape hatch	471
71.3	Proton solves a different reproducibility problem	472
71.4	Virtual displays are infrastructure, not evidence	472
71.5	Browsers need an out-of-process verdict	472
71.6	Hostility is any plausible false green	472
72	Discipline as a Test	475
72.1	DirectX eras as executable boundaries	475
72.2	Link closures make module boundaries observable	475
72.3	Documents can be machine-checked contracts	476
72.4	Can the green test go red?	476
72.5	Automation is narrower than apparatus	476
72.6	A gate-quality checklist	477
73	CI Execution and Coverage	479
73.1	Twenty-one workflows, nineteen automatic triggers	479
73.2	New architecture gates	479
73.3	A live configuration defect changes the matrix	480
73.4	Windows jobs deliberately run a narrower corpus	480
73.5	Sanitizers and validation are targeted	481
73.6	Fuzzing, properties, and mutation are not continuous campaigns	481
73.7	There is no code-coverage measurement	481
XII	Porting, Practice, and the Road Ahead	483
74	Migrating an XNA or FNA Game to CNA	485
74.1	Freeze a behavioral baseline first	485
74.2	Translate the API shape mechanically	486
74.3	Inventory content by actual reader route	486
74.4	Treat custom shaders as renderer contracts	486
74.5	Audit vertex layouts and model assumptions	486
74.6	Select a renderer by evidence, not by name	487
74.7	Make service differences explicit	487
74.8	Build a verification ladder for the port	487
74.9	Troubleshoot from the boundary inward	488
75	Case Study: Speedy Blupi and Planet Blupi	489
75.1	Call-site-driven scope	489
75.2	A pixel-format mismatch that never became a visible bug	489
75.3	Color keying: implementation broader than observed use	489
75.4	GetDC as a gameplay dependency	490

75.5	A performance audit adds exact costs to the same call sites	490
75.6	Interfaces never exercised at all	490
75.7	DirectSound structure-layout hazard	491
75.8	Dead and unexercised paths	491
75.9	Native-CNA port feasibility	491
75.10	Porting implications	493
76	Auditing xna4-spec Compatibility	495
76.1	What the repository contains	495
76.2	Where it sits in CNA's authority hierarchy	495
76.3	A reproducible surface audit	495
76.4	Auditing Ray	496
76.5	Auditing Viewport	496
76.6	What inventories cannot prove	497
77	cna-samples and cna-examples	499
77.1	cna-samples: ports with provenance	499
77.2	Read each missing file as a dated case record	499
77.3	Why official samples remain valuable	500
77.4	cna-examples: one navigable CNA-native application	500
77.5	A screenshot sweep needs semantic companions	500
77.6	The pinned documentation uses obsolete renderer names	500
77.7	Use both repositories as triangulation	501
78	Project Practice and Durable Task Tracking	503
78.1	Documents have roles, not equal authority	503
78.2	Subsystem plans preserve causality	503
78.3	Integration records need immutable inputs	504
78.4	Executable ledgers resist prose drift	504
78.5	SemVer and phase tracking coexist	504
78.6	sharp-runtime's database makes scope measurable	505
78.7	Separate observations, inferences, and decisions	505
78.8	Handoff is part of implementation	505
79	Roadmap, Open Gaps, and How to Re-Audit This Book	507
79.1	Triage correctness before expanding breadth	507
79.2	Complete capability truth before adding renderers	507
79.3	Close the arbitrary-effect boundary deliberately	508
79.4	Harden content symmetrically	508
79.5	Complete glTF semantics beyond importer coverage	508
79.6	Turn platform claims into retained artifacts	508
79.7	Repair the verification apparatus itself	509
79.8	Extend packaging beyond the native C boundary	509
79.9	Reconcile documentation from generated facts	509
79.10	How to re-audit this edition	510
A	Core Framework and Graphics API Reference	511
A.1	Math and Geometry Types	511
A.2	Game Framework and Content	512
A.3	Graphics Core: Resources and Batching	513
A.4	Model and Stock Effects	513
A.5	GraphicsDevice and State Objects	514

A.6	Effect Base, Parameters, and Queries	515
A.7	Reading this appendix safely	515
B	Renderer Registry, Capabilities, and Evidence	517
B.1	Canonical public identities	517
B.2	Four implementation questions across all 46 families	518
B.3	Capability answers require operation-level proof	519
C	Glossary of XNA and CNA Terms	521
D	Repository and Module Map	525
D.1	Pinned repository set	525
D.2	Physical CNA framework modules	526
D.3	Renderer module tree	527
D.4	How to extend this map without making it stale	528
E	CNAEXT: Marker and Engine-Layer Catalog	529
E.1	Framework and math bridges	529
E.2	Graphics and content extensions	530
E.3	Input, audio, media, and host services	531
E.4	How the strict check proves itself	532
F	Ecosystem and Platform Reference	533
F.1	Input: Keyboard, Mouse, GamePad, and Touch	533
F.2	Audio and Media	534
F.3	Devices and Sensors	534
F.4	Networking and GamerServices	535
F.5	Storage	536
F.6	SharpRuntime Core Types Used Everywhere	536
F.7	Reading this appendix safely	537
G	Verification Tiers and Oracles	539
G.1	Canonical claim labels	539
G.2	Evidence forms are not interchangeable	540
G.3	Oracle authority depends on the question	541
G.4	Renderer and platform claims need a complete vector	541
G.5	Automation at the pinned revision	542
G.6	A minimum claim recipe	542
H	glTF Importer, Runtime, and Test-Evidence Matrix	545
H.1	Containers, scenes, and geometry	545
H.2	Materials, scenes, and animation	546
H.3	Route and oracle boundaries	547
I	Experimental Native C API	549
I.1	Two version systems, deliberately independent	549
I.2	Results, diagnostics, and handles	550
I.3	Surface and coverage boundary	550
I.4	Renderer and platform consequences	551
I.5	Verification inventory, not a universal pass claim	551
	Index	553

Part I

CNA and Its Ecosystem

Chapter 1

CNA and XNA 4.0

1.1 The one-sentence version

CNA is a native C++ reimplementation of the *XNA 4.0* programming model, compiled with a C++23 toolchain floor but written in a disciplined C++17/20 idiom. It reimplements the game framework Microsoft shipped for the Xbox 360 and Windows from roughly 2006 to 2013. CNA’s default platform and audio implementations use SDL3, but those are independent configuration axes. Rendering is delegated to one of 50 public identities mapped to 46 implementation families; an opt-in build may compile several families and select one at runtime.

That description contains three separate engineering problems which organize the rest of the book:

CNA is a *translation target*, not a loader or binary compatibility layer. It cannot run an existing C# assembly. Porting translates the source to C++; even a property access becomes a `getXProperty()` or `setXProperty(value)` call. Names and observable contracts can remain XNA-shaped while the source language and ownership model cannot.

1. **Reproduce an API.** XNA’s public surface — `Microsoft.Xna.Framework` and its sub-namespaces — has to exist in C++, with the same class names, method signatures, namespaces, and observable behavior, so that code written against real XNA (or against FNA, the open-source XNA reimplementation in C# that this project treats as its primary behavioral reference) has somewhere to go.
2. **Reproduce a runtime.** XNA is a managed-code API. It assumes things a bare C++ program does not have for free: garbage-collected `System.Object`, events and delegates, exceptions with a .NET-shaped hierarchy, `TimeSpan`, collections. CNA does not reimplement the CLR — instead it depends on a sibling project, `sharp-runtime`, which provides a deliberately partial C++ subset of the .NET base class library, just deep enough to make the XNA API’s vocabulary make sense in C++.
3. **Reproduce a GPU.** XNA only had to run against Direct3D 9 (Xbox 360’s own GPU API is Direct3D-shaped too). CNA has to make the same `GraphicsDevice` / `SpriteBatch` / `Effect` surface available while pixels are produced by SDL’s 2D renderer, OpenGL, Vulkan, BGFX, WebGPU, Direct3D 9/11/12 under Wine, an HTML `<canvas>` under Emscripten, or a from-scratch DirectDraw reimplementation. Identical API presence does not imply identical behavior or evidence; Part IV records those distinctions.

From the project’s own README (2026-07-11 snapshot): “CNA is a C++ reimplementation of the XNA 4.0 programming model, built on SDL3 and a pluggable graphics renderer layer. It is a framework/runtime and abstraction layer—not a game—designed to preserve XNA-style APIs (`Microsoft::Xna::Framework`) while using modern C++ inter-

nals.”

1.2 Why XNA, and why now

XNA Game Studio was Microsoft’s managed-code (C#) game development framework, built around a small, coherent **Game** / **GraphicsDevice** / **SpriteBatch** / **Effect** vocabulary that a whole generation of independent and student developers learned game programming on. Microsoft discontinued active XNA development years ago; the community response was **FNA**, a faithful open-source reimplement of the XNA 4.0 API in C#. The pinned local reference selects its SDL3 platform implementation by default and retains SDL2 as an explicit compatibility choice through `FNA_PLATFORM_BACKEND=SDL2`; describing FNA simply as an SDL2 runtime is therefore historical shorthand, not its current default.

CNA asks a different question than FNA does. FNA asks: *how do we keep running C# XNA code forever, on modern platforms, without Microsoft?* CNA asks: *what if the code itself did not have to be C#?* A native C++ implementation of the same API gives a team that likes the XNA/MonoGame mental model — a thin, explicit, immediate-mode-ish 2D/3D API without an editor or a scene graph imposed on you — a path that does not require a managed runtime or GC pauses, and that a build system and toolchain of your choosing can own end to end. CNA’s own README states this directly: it is meant to “provide a native C++ path for teams that like the XNA/MonoGame model but need non-managed runtime/toolchain control.”

This is also, not incidentally, why CNA is useful as a target for *porting* existing XNA/FNA games rather than only for writing new ones from scratch — a use case this book returns to in this book’s migration and case-study chapters, using the real ports of *Speedy Blupi* and *Planet Blupi* (via the `free-direct` sibling project) as worked examples.

“Why now” is not merely an argument from intent. At the pin, CNA has fifteen physical framework modules, an optional native C API source module (whose alpha.1 final target is compile-blocked), 50 public renderer identities over 46 implementation families, an XNB reader stack, its own CNJ format, a shared runtime/offline glTF importer, and implemented Media types backed by file parsing, FFmpeg and `SDL3_mixer`. Those facts establish breadth; they do not establish uniform parity. Later chapters therefore report each route’s behavior, host and oracle separately instead of reviving a fragile percentage of “types present.”

1.3 Design goals, in the project’s own words

CNA’s own README states six goals directly, worth quoting in full since every one of them recurs as a concrete design decision somewhere later in this book, not just as an aspiration: “recreate the XNA developer experience in native C++”; “provide a native C++ path for teams that like the XNA/MonoGame model but need non-managed runtime/toolchain control”; “mirror core XNA namespaces and API patterns while implementing them incrementally”; “decouple gameplay-facing API from renderer implementation details”; “enable one high-level API surface across different rendering technologies”; and “keep SDL/OpenGL/Vulkan-level concerns behind framework abstractions.” The middle four of these six are really one idea stated four ways — the renderer-abstraction goal — and Chapter 19 is where that idea becomes a concrete `IGraphicsRenderer` contract rather than a design principle; the first two are the “why native C++, why not just use FNA” argument already made above.

1.4 Verification is a vector, not a leaderboard

There is no honest one-number answer to “how tested is CNA?” A source tree can contain a test definition without registering it in a selected build; CTest can register a command that the host cannot execute; a renderer can return success without consuming effect state; and a pixel can be correct on one translated runtime without proving another host.

The book therefore records a claim as a vector: public identity and implementation family, configuration, execution host, driver or translation layer, oracle, and unavailable branches. Examples deliberately occupy different tiers:

- the 50-identity / 46-family registry is strong structural evidence, not rendering proof;
- the Direct3D 9 corpus compares 39 scenes with real XNA output, but runs through Wine and DXVK rather than native period hardware;
- HTML_DOM has HTTP-served Chromium execution and compositor samples, while a successful build of another Web identity proves less;
- recorded Vulkan and BGFX pass fractions belong to their named campaigns and configured populations, not to a universal current suite total.

Appendix B gives the bounded renderer matrix. Chapters 69 and 70 define the counting and oracle vocabulary used throughout the edition.

1.5 What CNA is not

It is worth being precise about the negative space, because CNA’s own documentation is:

- **Not a game.** There is no bundled playable demo executable shipped as the project’s flagship artifact — the README states this is a deliberate priority of framework/runtime development over a shipping game demo. What ships instead is a test suite (thousands of GoogleTest cases across the unit and GPU pixel-test suites) and a set of minimal verification executables like `hello-triangle-sdl`.
- **Not an XNA content builder.** CNA has a substantial runtime XNB reader, but no MGCB-style import/process/write toolchain and no automatic reader bootstrap. A game must register the built-in readers before loading XNB; CNJ, loose files, and glTF are separate routes. Chapters 34–36 define those boundaries.
- **Not universally compiled-shader compatible.** CNA has distinct answers to XNA effects: renderer-owned stock effects; compiled XNA/FNA D3D9 Effect Framework bytecode accepted by the public `Effect` constructor on qualified renderers; and renderer-specific custom source or program forms through `ShaderEffect`. `.fx` or HLSL source, DXBC and MGSX are not interchangeable with that compiled Effect Framework format. No one route works identically on all 46 families; Chapter 17 separates those contracts.
- **Not binary-compatible with real Xbox Live.** The `GamerServices` namespace (Chapter 49) reimplements the public API shape with local/synthetic semantics, matching how FNA itself already handles this namespace — there is no real Xbox Live behind it, by design.
- **Not a drop-in Microsoft DirectX runtime.** The `free-direct` sibling implements a game-driven subset of DirectDraw, DirectSound, and DirectPlay over SDL3 and free-api. Its default DirectPlay transport is in-process; ENet networking is opt-in and speaks

to the same reimplementations, not Microsoft’s historical service. Chapter 60 gives the per-method status census.

1.6 Auditability as a framework property

The single most distinctive thing about how this project is run — and the reason this book can make specific, checkable claims instead of vague ones — is its verification methodology. Three mechanisms recur throughout the codebase and throughout this book:

Differential testing against a real reference implementation. `tools/fna-reference/` runs a genuine, live `FNA.dll` and compares CNA’s behavior against it call-for-call, rather than against a written specification of what FNA is supposed to do. Disputed behavior that differential testing cannot settle is checked against real XNA 4.0 running in a Windows 7 virtual machine.

A compile-time purity check. A dedicated build define, `CNA_STRICT_XNA_API`, turns the otherwise-inert `CNAEXT` marker macro into a real `[[deprecated]]` attribute, checked under `-Werror=deprecated-declarations` by two small standalone CMake targets built specifically for this purpose. Because every CNA extension beyond the real XNA 4.0 API must be wrapped in the `CNAEXT` marker macro (Chapter 3 explains the convention, and the self-verifying pair of targets that checks it, in full), this gives the project a mechanical — not just documented — guarantee that the public `Microsoft::Xna::Framework` surface has not silently grown project-specific extensions.

Pixel-level oracle corpora, not just “it compiles.” The D3D9 renderer in particular ships an oracle corpus that diffs CNA’s own render output against *the real XNA 4.0 runtime’s own render* of the same scene. The checked-in corpus contains 39 declarative scenes and the same number of matching reference PNGs. Its registered D3D9 gate runs the full corpus through the absolute per-channel comparator at tolerance zero; that wiring is an enforceable contract, while a pass count still requires an identified run. Chapter 28 covers what that verification pipeline (MinGW-w64 cross-compilation, Wine, DXVK, a real GPU) actually looks like.

Why “differential testing against FNA” is not by itself enough — a real finding. CNA’s own `CHECKLIST.md` names FNA as authoritative for *behavior* but the original Microsoft XNA 4.0 reference assemblies as authoritative for *API surface*, and explains why with a genuine near-miss: FNA is itself a reimplementations, and its own `src/Media/Song.cs` quietly omits three public XNA 4.0 members — `Song.Album`, `Song.Artist`, and `Song.Genre` — along with `Song.ToString()`. Because CNA’s `Media` namespace (Chapter 46) was itself audited against FNA across eight consecutive adversarial review passes, every one of those eight passes came back clean — CNA had faithfully reproduced FNA’s own incompleteness, not merely failed to find a bug that was there to find. The gap was only caught on a ninth pass, once the audit method itself changed to a member-level diff against the real Microsoft reference XML rather than another comparison against FNA. The fix (tracked as `MEDIA-174` through `MEDIA-180` in `plan_media.md`) added non-owning `Album*` / `Artist*` / `Genre*` back-pointers to `Song`, populated by `MediaLibrary` and `nullptr` for a song outside any library — mirroring an ownership pattern the surrounding `Media` types already used. The methodological lesson the project drew from this, stated directly in its own checklist, is the one this chapter opened with: “FNA doesn’t have it either” is not a justification, because that is exactly the reasoning that let this specific gap survive eight reviews.

This matters for how you should read the rest of this book. Where a claim in the chapters ahead is stated as a fact about CNA’s behavior, it is because the project’s own test suite, documentation, or source comments state it as a verified fact, generally with a specific mechanism (a CTest binary, a doc file, a task number) behind it — and where the project’s own documentation flags something as a dated snapshot, an estimate, or a known open gap, this book preserves that qualification rather than erasing it for a cleaner sentence.

1.7 The first tagged release and the continuing phase workflow

CNA developed continuously from 2025-02-22 without conventional releases while numbered phases and tasks carried engineering state. On 2026-08-20 the annotated `v0.1.0-alpha.1` tag established the first Semantic Versioning release. The product version is assembled from CMake’s numeric `0.1.0` project version and the separate `alpha.1` prerelease field. CMake prints the complete version during configuration, `cmake/Version.cmake` generates `CNA/Version.hpp`, Doxygen uses the same product version, and the changelog and release procedure name the tag spelling with its leading `v`.

The leading zero and alpha qualifier matter. The release policy permits compatibility-breaking changes before 1.0 when the minor or prerelease version advances; `alpha.1` should not be read as a stable 1.0-quality API promise. The native C ABI has its own experimental version, 0.7.0, described in Appendix I; product and ABI versions answer different questions.

SemVer did not replace the project’s strictly incrementing **Phase** counter. Phases in `CHECKLIST.md` and the per-subsystem `plan_*.md` files still name bounded work, split it into numbered tasks, and retain closure evidence. They are an engineering workflow; tags identify publishable source states. Chapter 78 covers how the two now coexist.

This phase-based, task-tracked structure supplies valuable historical evidence, but a closing count remains a snapshot of one configuration. This edition preserves such a number only with its campaign and verdict channel; current source definitions, CTest registrations, executed cases, and oracle scenes are never silently combined into one total.

1.8 CNA’s software license

CNA is licensed under the Microsoft Public License (Ms-PL), the same permissive-but-not-MIT license FNA itself uses — a deliberate choice, not a coincidence: portions of CNA are directly derived from or based on FNA’s own source (most visibly the D3D9 renderer’s vendored Stock Effects HLSL sources, discussed in Part IV), and matching FNA’s license keeps that derivation legally uncomplicated. Ms-PL permits commercial use, modification, and redistribution, but — unlike MIT or the 2-clause BSD license — includes an explicit patent grant limited to contributions and a reciprocal “if you sue over patents, your license ends” clause; it is not a copyleft license, so a game built against CNA is not obligated to publish its own source.

1.9 Reading map for the rest of this book

Chapter 2 draws the full dependency graph between CNA and its sibling repositories. Chapter 3 is the “house style” chapter — the concrete rules (API mirroring, namespace discipline, the `CNAEXT` tag, the `getXProperty` / `setXProperty` convention, the porting checklist) that every other chapter’s code examples silently assume. Part II gets you from a clean checkout to a running triangle and then through the core framework types. Parts III and IV are the graphics machine, first as an API and then as 46 implementation factories serving 50 public renderer identities.

Chapter 2

Ecosystem and Vocabulary

CNA does not sit alone in the `openeggbert` GitHub organization. It sits at the center of a small, deliberately layered set of sibling repositories, most of which expect to be checked out as literal sibling directories on disk — `../sharp-runtime`, `../easy-gl`, `../cna-samples`, and so on — because CNake’s own build system looks for them there. This chapter is the map; later chapters (Part [VIII](#) for `sharp-runtime`, Part [IX](#) for `easy-gl` and `free-direct`) are the territory.

2.1 The dependency graph

Repository	Role	Consumed by
<code>sharp-runtime</code>	C++ subset of the .NET BCL	CNA (foundation layer)
<code>easy-gl</code>	Toolkit-independent OpenGL/ES wrapper	CNA’s five EasyGL-backed profiles
<code>free-direct</code>	DirectDraw/DirectSound/DirectPlay subset	CNA’s FREEDIRECT renderer
<code>xna4-spec</code>	Machine-readable XNA 4.0 API spec (XML)	Compatibility audits of CNA
<code>cna-samples</code>	Ports of official XNA 4.0 samples	Exercises CNA end-to-end
<code>cna-examples</code>	In-app demo catalog for CNA	Exercises CNA end-to-end

Every one of these is a genuinely independent, separately buildable, separately tested project — none of them is a vendored copy or a submodule of CNA. `sharp-runtime` in particular describes itself as serving “as a foundation for higher-level frameworks (e.g. CNA)” and lists CNA as its own downstream consumer, not the other way around; you could, in principle, use `sharp-runtime` or `easy-gl` in a project that has nothing to do with XNA at all.

2.2 The sibling build graph

The dependency graph above is not just a narrative claim — it is exactly what CNA’s own `CMakeLists.txt` and `cmake/RendererSelection.cmake` do at configure time, checked directly rather than assumed. Every sibling dependency follows the identical pattern: an `if(NOT EXISTS ../CMakeLists.txt)` guard that fails configuration outright, with a real, copy-pasteable fix in the error message itself, rather than failing confusingly deep in a missing-header compile error later:

```
1 CNA: Missing sibling repository 'sharp-runtime' at
2 ${CMAKE_CURRENT_SOURCE_DIR}/../sharp-runtime -- this is a separate
  git
3 checkout expected next to this repo's own directory, not a git
4 submodule (git submodule update --init will not fetch it). Fix:
5 cd ${CMAKE_CURRENT_SOURCE_DIR}/.. &&
6 git clone https://github.com/openeggbert/sharp-runtime.git
```

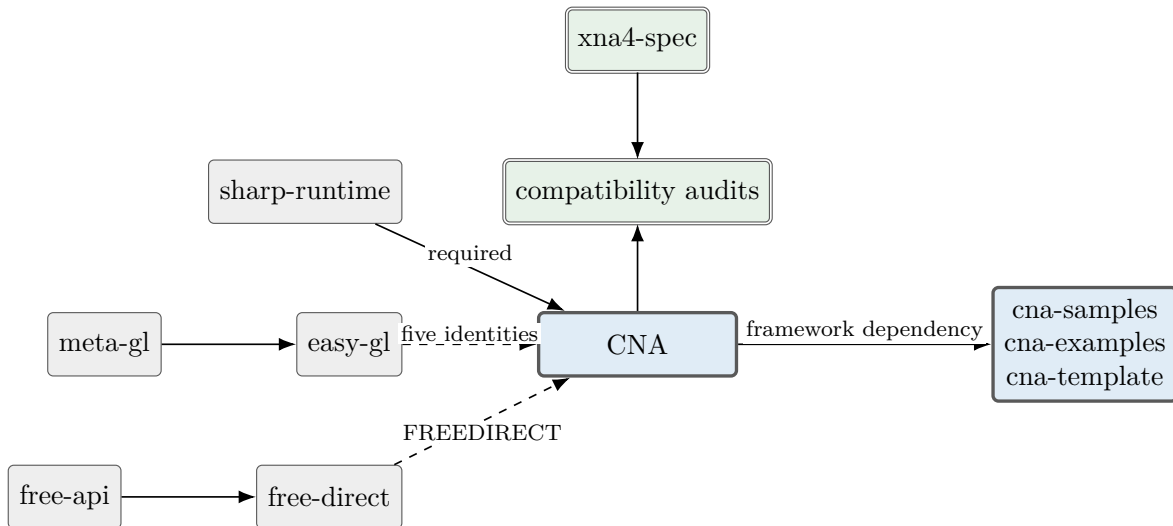


Figure 2.1: Repository relationships relevant to CNA at the edition pins. Arrows point from a provider or evidence input to its consumer; dashed edges are renderer-conditional. The graph is not an organization-wide repository inventory.

`sharp-runtime` is unconditionally required (`add_subdirectory(..sharp-runtime SHARP_RUNTIME)` runs regardless of renderer choice); `easy-gl` and `free-direct` are each pulled in only when a matching `-DCNA_GRAPHICS_RENDERER` value is selected, so a plain `SDL_RENDERER` or `VULKAN` build never even checks whether those two sibling directories exist.

A real, second-level dependency this book’s dependency-graph table doesn’t show. `free-direct` is not a leaf in this graph — its own `CMakeLists.txt` pulls in a *third* sibling repository, `free-api`, an experimental, small WinAPI-compatibility-subset project (targeting roughly the Win32 API surface as it existed around 1998) built on SDL3 the same way every other project in this ecosystem is, per `free-api`’s own README. `RendererSelection.cmake`’s own comment explains why this does not need a separate SDL configuration flag: `free-direct` resolves `SDL3::SDL3 / SDL3_image::SDL3_image / SDL3_mixer::SDL3_mixer` from CNA’s own already-vendored targets, set up before renderer selection runs, so the whole three-level chain (`CNA → free-direct → free-api`) shares one SDL3 build rather than each level vendoring its own. The same comment names two real, independent consumers of `free-direct` beyond CNA itself — `free-eggbert` and `planetblupi` — confirming this sibling chain is genuinely reused ecosystem infrastructure, not machinery built only for CNA’s own `FREEDIRECT` renderer.

2.3 sharp-runtime: the floor CNA stands on

XNA’s public API vocabulary assumes .NET underneath it: `byte`, `TimeSpan`, `events`, `IDisposable`, `System.Object.GetType()`. A C++ reimplementing of XNA has two choices — invent ad-hoc C++ substitutes for all of that inline, scattered through the XNA porting work itself, or build the .NET-shaped foundation once, as its own project, and depend on it. CNA’s own `CLAUDE.md` porting rules choose the second option explicitly: “If CNA needs anything from .NET that is not yet in `sharp-runtime` — a type alias, a class, an interface, an exception type, a utility — add it to `sharp-runtime` first, then use it from CNA. Do not inline .NET concepts directly into CNA headers as raw C++ types or ad-hoc workarounds.”

`sharp-runtime` is explicit that it is *not* attempting a CLR: no reflection, no GC, no delegates

beyond `std::function`, no serialization infrastructure, no P/Invoke, no full cryptography/TLS stack. What it does provide is documented as a “pragmatic subset designed for use in native C++ applications,” tracked class-by-class in a SQLite database (`plan.sqlite3`) whose exact blob exists in the pinned commit despite also matching a current ignore rule, with an honest, small state machine per .NET type: `todo`, `ported`, `ignore` / `ignored`, or `tobedecided` for genuinely ambiguous cases that need a human architecture call rather than a guess. Chapter 53 and Chapter 56 cover `sharp-runtime`’s own architecture, its `System::*` namespace layout, and its component-isolated test apparatus in depth; Chapter 57 covers the specific question of how strictly CNA insists on that .NET shape versus where it deliberately diverges.

2.4 easy-gl: OpenGL without a windowing opinion

The EasyGL renderer family — five public OpenGL/OpenGL ES profiles sharing one implementation factory — is built on `easy-gl`, a small, toolkit-independent C++20 wrapper over OpenGL and OpenGL ES. Its defining architectural choice is what it refuses to own: no window creation, no GL context creation, no event loop. The host application (CNA, in this case) owns the window, creates and activates the GL context, and hands `easy-gl` a `GetProcAddress`-style loader callback; `easy-gl` owns everything from there down — shader compilation and linking, buffers, vertex arrays, textures, draw calls, and capability queries that let calling code gate OpenGL-only features (like polygon-mode fill flags) away from OpenGL ES targets at runtime via `device.supports(...)` / `device.require(...)`. As of the snapshot this book draws from, `easy-gl`’s own README describes its scope as still a deliberately compact, evolving vertical slice — initialization, capability detection, and basic draw flow, exercised by its own `hello-triangle-sdl` example and two dedicated smoke-test suites (`easy-gl-smoke-tests`, `easy-gl-resource-smoke-tests`) — with CNA’s own EasyGL implementation having grown well beyond that slice on top of it, per Chapter 22. Chapter 59 covers `easy-gl`’s public API surface (`easygl::Device`, `easygl::Shader`, `easygl::Program`, `easygl::Buffer`, `easygl::VertexArray`, `easygl::Texture`, `easygl::Feature`) in full.

A live instance of this book’s own recurring pattern, caught in a sibling repo rather than in CNA itself. `easy-gl` carries its own `MIGRATION.md` (added 2026-07-19, alongside a new 845-line `PLAN.md`), a nine-row “breaking change summary” table framed entirely in future tense — `Framebuffer::attach_texture_2d` “now takes `const Texture&`”, `Program::uniform_block_index` “returns `std::optional<unsigned int>`”, and six more signature changes, each described as “planned for the next release”. Reading the real, current headers directly instead of trusting that framing shows every single one of the nine has already landed: checking a sample of five directly against five real `include/easygl/` headers — `Framebuffer.hpp`, `Program.hpp`, and three others — finds `attach_texture_2d` is already taking `const Texture&`, `uniform_block_index` already returning the `optional`, and the remaining three checked signatures (`ProgramPipeline`’s two `Program`-typed parameters, `TransformFeedback::set_varyings`, `Sync::native_handle()`’s `GLsync` return type) all already in their described post-migration form. `MIGRATION.md` is not wrong about what changed — it is simply stale about *when*, describing already-shipped API surface as still upcoming. This is the identical shape of finding this book has made repeatedly about CNA’s own tracked documents (a snapshot that outlived the moment it was accurate), just observed here in a sibling repository this book does not otherwise audit — worth naming because it is independent confirmation that the failure mode is a property of fast-moving source-controlled documentation generally, not something specific to this project’s own writing habits.

2.5 free-direct: DirectX 3 scoped to two legacy games

Where **easy-gl** is a general-purpose OpenGL wrapper, **free-direct** is the opposite kind of project: a deliberately *narrow*, game-driven reimplementation of a subset of DirectX 3 (2D) — DirectDraw, DirectSound, and DirectPlay — built on SDL3 internally, whose explicit goal is not DirectX compatibility in general but running two specific legacy Windows games (*Speedy Blupi* and *Planet Blupi*) without their original OS dependencies. Its own README states the non-goals as plainly as the goals: no full DirectX 3 compatibility, no hardware-accurate emulation, no Direct3D pipeline, and no expansion of APIs the target games do not actually call. This project backs CNA’s **FREEDIRECT** graphics renderer (Chapter 26) and is covered as a project in its own right, including its real (not stubbed) ENet-backed DirectPlay networking, in Chapter 60 and in the Speedy Blupi/Planet Blupi porting case study in Chapter 75. **free-direct**’s own test suite is deliberately split by transport: the default **cctest** run (no label filter) exercises **LoopbackDirectPlayTransport**’s synchronous semantics and is not expected to pass **directplay_tests** unmodified against a real, asynchronous ENet build (1 of 8 tests fails that way, by design, not as a regression); a separate **-L enet** label opts into the ENet-backed transport tests specifically, verified 1/1 passing.

2.6 xna4-spec: a ground truth to audit against

A recurring failure mode for any reimplementation project is drift between what the README claims and what the code actually does — and a subtler one is drift between what the README claims and what the *original* API actually specified, from memory rather than from a real reference. **xna4-spec** exists to remove the second failure mode. It is a complete, machine-readable XML transcription of the official XNA 4.0 documentation — every class, struct, enum, interface, and delegate across all 19 XNA namespaces, 544 types in total, with their original property/method/constructor/field/event descriptions — converted from Microsoft’s own learn.microsoft.com XNA 4.0 reference pages. Its own README states its purpose directly: “Auditing C++ reimplementations of XNA 4.0 (comparing what is implemented vs. what is missing).” CNA’s release prose repeats a dated 227-of-245 public-FNA-type presence audit (92.7%). That denominator came from diffing FNA’s public type list against CNA headers, not from **xna4-spec**, and alpha.1 does not ship a generator that re-derives it as a tag invariant. Treat it as a historically recorded, methodology-limited presence snapshot—not behavioral coverage and not a fresh count made by this edition. Chapter 76 walks through using **xna4-spec** to run this kind of audit yourself, worked through in full against a real class (**Ray**) rather than described only in the abstract.

2.7 cna-samples and cna-examples: two different kinds of proof

Two sibling repositories exist purely to give CNA something real to run, and they answer two different questions.

cna-samples is a C++ porting catalog for the official Microsoft XNA Game Studio 4.0 sample archives, licensed Ms-PL to match its Microsoft-derived origin. Its question is: *does CNA run real, historical XNA sample code?* At the pinned revision its plan accounts for 153 archive entries: 63 completed ports, 23 tracked placeholders, and 67 deliberate exclusions. Those are catalog statuses, not fresh execution results; Chapter 77 explains why each dated placeholder must be re-audited before it is counted as a current CNA gap.

cna-examples is original, MIT-licensed CNA-specific work — explicitly *not* a port of Microsoft’s sample collection, precisely so it can carry a permissive license **cna-samples** cannot.

Conceptually inspired by JavaFX's **Ensemble** sample browser, it is a single cross-platform application (desktop, Web, mobile) that lets a reader navigate $\text{Area} \rightarrow \text{Category} \rightarrow \text{Demo}$ (Input, Audio, Devices, Net, Media, 2D/3D Graphics, ...) and run a live demonstration of each area, all inside one binary. Its question is: *does CNA have a coherent, in-app way to show off everything it can do?* As of the snapshot this book draws from it is an early skeleton — the application boots and navigates $\text{Area} \rightarrow \text{Category}$, but individual demo screens are not yet implemented, and only the EasyGL-backed renderer family is targeted so far.

2.8 Repositories mentioned but out of this book's scope

The **openegbert** organization hosts a considerably larger set of repositories beyond the six above — additional CNA-adjacent projects (**cna-template**, **cna-extended**, **cna-craft**), other graphics libraries (**meta-gl**, **easy-3d**), actual shipped games and tools built on this stack (**mesh-craft**, **galaxy-eggbert**, **free-eggbert**, **planetblupi**, **mobile-eggbert**), and a long tail of per-project marketing/documentation sites (the ***.openegbert.com** repositories). This book's scope is deliberately the six repositories in the table above, because they are the ones CNA's own build system, README, and **CLAUDE.md** porting rules directly name as dependencies or verification partners. Appendix D gives a fuller repository map for readers who want to go further.

Chapter 3

Language, Conventions, and CNAEXT

Every worked example in the rest of this book silently assumes a set of conventions that CNA enforces on itself through its `CLAUDE.md` porting rules and `CHECKLIST.md`. This chapter makes those conventions explicit, once, so that later chapters can simply use them.

3.1 The prime directive: match FNA, not taste

CNA’s porting rules name a single authoritative behavioral reference: a local FNA source tree. The instruction is blunt: “Do not treat old CNA code or AI-generated stubs as authoritative if they conflict with the FNA reference API.” Concretely, this means:

- Class names, struct names, enum names, method names, operator names, and constant names must match XNA/FNA *exactly*. A method is not renamed to be “more C++-like” if that would diverge from the reference.
- Behavior is matched over personal preference: packed value layouts (`Color` stores its channels as `AABBGGRR`), clamping behavior, integer cast behavior, operator behavior, method overloads, default values, and exception behavior where practical, all follow FNA even when a different choice would look cleaner in isolation.
- C++ member declaration order mirrors the original C# source order where practical, specifically to keep diff-based review tractable against the reference.

This is the single idea that makes the rest of the house style legible: every convention below exists to answer the same question — *how do you represent a C# concept in C++ without losing the ability to diff against the original?*

3.2 Two namespaces, two jobs

CNA code lives in exactly one of two namespace families, and the rule for which one is absolute:

Namespace	Contains
<code>Microsoft::Xna::Framework::*</code>	Real XNA 4.0 types, matching the reference exactly
<code>CNA::*</code>	Project-specific extensions, helpers, internal backends

Original XNA types are never moved into the CNA namespace, and the reverse is equally firm: anything that is not part of the real XNA 4.0 API must not silently live inside `Microsoft::Xna::Framework` unmarked. That is what the CNAEXT marker exists for.

3.3 The CNAEXT tag: a compiler-enforced honesty check

CNAEXT is an empty marker macro, defined in `modules/core/include/CNA/CNAHelper.hpp`, whose only job is to visibly tag a declaration inside the XNA namespace tree as *not* part of the original XNA 4.0 API — a rumble-and-gyro extension on `GamePad`, a `GetTypeName()` override, a raw joystick API, a haptics subsystem. Two concrete examples from the real codebase:

```

1 // A real XNA 4.0 override, unmarked:
2 [[nodiscard]] bool Equals(const Color& other) const;
3
4 // A CNA-only addition living in the same class, explicitly marked:
5 CNAEXT [[nodiscard]] const std::string& GetTypeName() const override
  ;

```

What makes this more than a documentation convention is a compiler-enforced check described in Chapter 1, precisely: `CNAHelper.hpp`'s own definition of `CNAEXT` is conditional on a second, distinct macro, `CNA_STRICT_XNA_API` — in a normal build `CNAEXT` expands to nothing at all (it is documentation only); only when `CNA_STRICT_XNA_API` is defined does it expand to `[[deprecated("CNAEXT: not part of the XNA 4.0 API surface")]]`. That define is never set for the normal CNA library target — it is scoped to exactly two small standalone CMake targets in `cmake/Harnesses.cmake`, each compiled with `-Werror=deprecated-declarations` on top of it, so a stray `CNAEXT`-tagged call anywhere those targets' translation units can see turns into a hard build failure rather than a warning that could be scrolled past.

A worked example: the check that verifies the check. A positive-only test — “the real API still compiles clean” — cannot by itself prove the mechanism would actually catch a real leak; it is equally consistent with a check that always passes. CNA's own `tools/devices/StrictXnaApiSurfaceCheck.cpp` (compiled as the `cna_strict_xna_api_check` target, run as the `StrictXnaApiSurfaceCheck_Compile_Run` CTest) is that positive direction, and its own header comment names every `CNAEXT`-tagged `Accelerometer / Sensors` member it deliberately avoids calling. The negative direction — proving a real leak genuinely fails — is a second, separate file, `tools/devices/StrictXnaApiSurfaceLeakCheck.cpp`, whose entire body is one deliberate violation:

```

1 Microsoft::Devices::Sensors::Accelerometer accelerometer;
2 accelerometer.InjectSyntheticSensorUpdate(1.0f, 0.0f, 0.0f); //
  CNAEXT -- must fail to compile here.

```

This compiles into an `EXCLUDE_FROM_ALL` target (`cna_strict_xna_api_leak_check`), so it never participates in an ordinary build. It is invoked by exactly one CTest: `StrictXnaApiSurfaceLeakCheck_MustFailToCompile`, whose `WILL_FAIL TRUE` property inverts the usual pass/fail meaning — the test *passes* only if the build command it wraps *fails*. If a future change to the `CNAEXT` macro or to `CNA_STRICT_XNA_API`'s wiring ever let this deliberately-broken file compile, this specific test would flip from passing to failing — catching a regression in the safety net itself, not merely in the API surface the net is meant to protect.

3.4 C# properties, translated consistently

C# auto-properties (`public byte R { get; set; }`) become a fixed pair of C++ methods, never a public field, unless the surrounding type has already established a field-style convention:

```

1 // C#:  public byte R { get; set; }

```

```

2
3 // C++:
4 [[nodiscard]] bytecs getRProperty() const;
5 void setRProperty(bytecs value);

```

Not every property setter is a bare field write, and the convention does not ask one to pretend otherwise: a C# property with real validation logic in its setter keeps that logic, translated directly, rather than being flattened into a public field for convenience. `SkinnedEffect::WeightsPerVertex` (Chapter 15) is a real instance of this, read directly from `SkinnedEffect.cpp`:

```

1 void SkinnedEffect::setWeightsPerVertexProperty(int v)
2 {
3     if (v != 1 && v != 2 && v != 4)
4         throw std::out_of_range("WeightsPerVertex must be 1, 2, or
5                                 4.");
6     weightsPerVertex_ = v;
7     dirtyFlags_ |= DirtyShaderIndex;
8 }

```

Two things this one setter does at once, both faithfully carried over from what a validated C# property would do: it rejects an invalid value outright (a real XNA/FNA `SkinnedEffect` genuinely only supports 1, 2, or 4 bone weights per vertex — there is no silent clamping to the nearest valid value), and it marks a dirty-shader-selection flag as a side effect of the assignment, since a changed weight count means a different GPU shader variant must be selected on the next `Apply()`. Neither behavior would survive a mechanical “just make it a public field” translation.

3.5 SharpRuntime type aliases: never a raw C++ primitive in the XNA surface

Wherever C# source uses a .NET primitive type name, CNA code uses the corresponding `sharp-runtime` alias rather than a raw C++ fundamental type, specifically so that the XNA-facing surface keeps a visible, greppable link back to its .NET origin:

C# type	sharp-runtime alias	Underlying C++ type
byte	bytecs / Byte	uint8_t
sbyte	sbytecs / SByte	int8_t
short	shortcs / Int16	int16_t
ushort	ushortcs / UInt16	uint16_t
int	intcs / Int32	int32_t
uint	uintcs / UInt32	uint32_t
long	longcs / Int64	int64_t
ulong	ulongcs / UInt64	uint64_t
float	Single	float
string	String	std::string
char	charcs	char16_t

If a needed alias does not yet exist, the rule is to add a minimal stub to `sharp-runtime` first — never to reach for a raw C++ type directly in the XNA API surface as a shortcut. This is the same missing-dependency discipline covered in §3.12 below.

3.6 Events, as `System::EventHandler<T>`

C# events and delegates are modeled uniformly through `sharp-runtime`'s `System::EventHandler<T>`, never through an ad-hoc project-specific callback type:

```

1 // Declaration, matching C# "public event EventHandler<T> Name;"
2 System::EventHandler<ExitingEventArgs> Exiting;
3
4 // Subscription
5 Exiting += [](System::Object* sender, const ExitingEventArgs& e) {
6     /* ... */ };
7
8 // Raising
9 Exiting.Raise(this, args); // or Exiting.Invoke(this, args);

```

3.7 Interfaces as abstract base classes, `IDisposable` as a pattern

C# interface relationships become C++ abstract base classes — for example `Color` implementing `IEquatable<Color>` and `IPackedVector` in C# becomes `struct Color : public Graphics::PackedVector::IPackedVectorT<UInt32>` in CNA. Where an exact mapping is not practical, the rule is to implement equivalent behavior and document the intentional deviation in the pull request description — explicitly *not* as a source comment, keeping the deviation visible to reviewers without cluttering the header for every future reader.

`IDisposable` follows a fixed pattern backed by `sharp-runtime`'s `System::IDisposable`:

```

1 class Foo : public System::IDisposable {
2 public:
3     void Dispose() override;
4 protected:
5     void Dispose(bool disposing); // only when the pattern
6                                     requires it
7 };

```

with an `isDisposed_` guard checked before any operation, throwing `std::runtime_error` on use-after-dispose.

3.8 Visibility is chosen, not defaulted

C# `internal` members do not automatically become C++ `public` just because C++ lacks an `internal` keyword — they become `private`, `protected`, live in a `detail/internal` namespace, or are omitted from the port entirely. The house style calls out a concrete anti-example: a C# `internal` `DebugDisplayString` helper should not become a public C++ API method just because it was easy to leave `public`.

3.9 Physical modules own the namespace-shaped include surface

Each physical module owns its declarations under `modules/<owner>/include/` and its implementation under `modules/<owner>/src/`. The public include subtree mirrors the namespace, while implementation paths may use a shorter subsystem directory:

```

1 modules/graphics/include/Microsoft/Xna/Framework/Graphics/Texture2D.
  hpp
2 modules/graphics/src/Xna/Texture2D.cpp

```

Non-template implementations do not live in headers. At the pin, that subtree contains 125 .hpp files: 107 directly in the `Microsoft::Xna::Framework::Graphics` directory and 18 in its `PackedVector` child. The directory mirrors both namespace levels; calling all 125 files the flat “Graphics namespace” would erase that real nested boundary.

3.10 No backward-compatibility shims

If correcting an API breaks an old demo, the demo is fixed — CNA does not carry a convenience alias just to keep outdated call sites compiling. The house style gives its own canonical wrong/right pair:

```

1 // Wrong: an old-CNA-era shortcut kept around for compatibility
2 inline const Color CornflowerBlue(/* ... */);
3
4 // Correct: the real, single source of truth
5 Color::CornflowerBlue

```

3.11 Documentation: full Doxygen, or nothing

Every public method, constructor, property accessor, operator, and constant in every .hpp file must carry a Doxygen block — bare /// line comments are never acceptable on a public declaration (they remain acceptable only for brief inline notes inside a method body). The intent of the original C# XML doc comments (<summary>, <param>, <returns>) is preserved, often verbatim from FNA where the wording already fits, but never marked with a comment like “taken from FNA” — provenance belongs in the porting record, not scattered through the header.

3.12 Missing dependencies: stub correctly, never invent

When a file being ported references a type that does not exist yet, the rule is layered: if the missing type belongs to the .NET runtime (`System.*` or a primitive alias), it is added to `sharp-runtime` first, following the same minimal-stub discipline described in Chapter 2. Otherwise, a minimal, *correctly named* stub is added directly in CNA, in the correct final namespace, sufficient only to compile — never a large unrelated system built just to satisfy one missing dependency. Every stub introduced this way is reported in the task or PR description, so it is never silently forgotten as “real” once the build goes green.

A worked example: a real missing-dependency gap, from stopgap to proper fix. CNA’s own checked-out history has a concrete instance of exactly the situation this rule exists for. At one point, CNA’s checkout would not build at all because two content-type readers called `System::IO::BinaryReader::ReadChar()` and `ReadDecimal()` — and neither method existed yet in `sharp-runtime`. Rather than inventing an ad-hoc replacement inline in CNA (the exact anti-pattern this section warns against), the documented fix at the time was a diagnostic-quality stopgap patch, explicitly labeled as temporary and preserved in this book’s own `tools/cna-screenshot-infra/` materials. `sharp-runtime`

has since implemented both methods for real, in its own `BinaryReader.cpp`, matching .NET’s actual documented behavior precisely rather than approximately: `ReadChar()` decodes one UTF-8 code point from the underlying stream (not a raw byte), throwing `System::FormatException` for an invalid lead byte, an invalid continuation byte, *or* a code point above `0xFFFF` that would require a UTF-16 surrogate pair — the last case chosen deliberately to match real .NET’s own `BinaryReader.ReadChar()`, whose internal `Decoder.GetChars` call targets a one-char buffer and throws for exactly the same reason, rather than silently truncating to one half of a surrogate pair. This is the missing-dependency rule’s full lifecycle in one real example: a genuine gap, a documented interim workaround, and a proper fix landing in `sharp-runtime` once someone did the work — not a stub that quietly became permanent.

3.13 The per-file porting checklist, in miniature

`CHECKLIST.md` governs the full requirements for porting one FNA source file to CNA; the house rules single out its non-negotiable minimum:

- An SPDX MS-PL license identifier at the top of both the `.hpp` and the `.cpp`.
- `#include "CNA/CNAHelper.hpp"` in the header, whenever CNAEXT is used anywhere in that file.
- Every method body checked line-by-line against the FNA equivalent, with every intentional deviation documented in a source comment (not silently absorbed).
- Every concrete class inheriting `System::Object` overrides `GetTypeName()`, tagged CNAEXT, returning the fully-qualified .NET name (e.g. `"Microsoft.Xna.Framework.Game"`).
- A unit test for every public method, operator, and constant — with out-ref overloads tested separately from their value-returning counterparts, and static factory methods (`CreateFromPoints`, `CreateMerged`, `CreateFromSphere`, ...) each getting a dedicated test.

A file is “done in one pass” — the house style explicitly rejects a “make-and-forget-partially” workflow where checklist items are deferred to a later, unscheduled pass.

3.14 Why this chapter earns its place in a “bible”

None of the conventions above are unique or clever in isolation — type-alias tables and Doxygen mandates exist in a thousand C++ style guides. What is unusual is how tightly they are all in service of one measurable goal: keeping a large, multi-renderer, multi-platform C++ codebase auditable against a real external reference (FNA, real XNA 4.0, `xna4-spec`) at every layer — namespace, visibility, member order, type alias, and a compiler flag that turns “did we leak a non-XNA type into the XNA surface” into a build error instead of a hoped-for code-review catch. The rest of this book takes these conventions as read; when a later chapter says a class “matches FNA exactly,” this is the mechanism that claim rests on.

Part II

Architecture, Building, and the Framework

Chapter 4

Configuring and Building CNA

4.1 Prerequisites

CNA targets C++23 and CMake 3.20+. On Linux, the minimum toolchain is GCC 12+ or Clang 15+; two sibling directories must be available to CMake alongside the CNA checkout itself: `../sharp-runtime` (always required) and `../easy-gl` (only required for the shared EasyGL implementation selected by `OPENGLES2`, `OPENGLES3`, `OPENGL33`, `WEBGL1`, or `WEBGL2`). `SDL3`, `SDL3_image`, and `SDL3_mixer` are built from vendored Git submodules by default, so no system SDL packages are required on a first build.

The root project version is 0.1.0 and `CNA_VERSION_PRERELEASE` is `alpha.1`; a configure of this edition prints `CNA: version 0.1.0-alpha.1`.

On Windows, any of MSVC 2022 (v17.8+), `clang-cl`, or MinGW-w64 (native or cross-compiled from Linux) works; only `../sharp-runtime` is needed as a sibling dependency, and the same vendored-submodule SDL story applies.

4.1.1 Initializing submodules

Before the first build, CNA’s vendored `SDL3/SDL3_image/SDL3_mixer` submodules must be populated:

```
1 git submodule update --init
```

A GitHub “Download ZIP” archive does *not* include submodule contents. Building from such an archive leaves `third_party/SDL` empty, and CMake aborts at configure time with a clear, specific error pointing at `cmake/ThirdPartySDL.cmake` rather than failing confusingly deep in a missing-header compile error. The two ways around this are cloning with Git and running the command above, or configuring with `-DCNA_USE_SYSTEM_SDL=ON` to use system-installed SDL3 packages instead of the vendored submodules.

4.2 The default build: Linux, the OpenGL ES 3 renderer

```
1 git submodule update --init
2 cmake -S . -B build -DCNA_GRAPHICS_RENDERER=OPENGLES3
3 cmake --build build --target CNA CnaTests
4 ctest --test-dir build --output-on-failure
```

`OPENGLES3` is CNA’s Linux default and is implemented by the mature shared EasyGL family (Chapter 22), so it is the natural first build. Swapping to `SDL_RENDERER` is a one-flag change:

```
1 cmake -S . -B build-sdlrenderer -DCNA_GRAPHICS_RENDERER=SDL_RENDERER
2 cmake --build build-sdlrenderer --target CNA CnaTests
```

This one-flag swap is the point of the renderer abstraction covered in Chapter 19 — the same CNA / CnaTests build targets exist regardless of which identity in Table 4.1 you select.

CNA_GRAPHICS_RENDERER value	Platform constraint
GL families	OPENGLES1, OPENGLES2, OPENGLES3, OPENGL1, OPENGL2, OPENGL33, OPENGL4, WEBGL1, WEBGL2, PORTABLEGL
Native and abstraction APIs	BGFX, VULKAN, WEBGPU, MAGNUM, WICKED, SOKOL, DILIGENT, LLGL, IGL, METAL, FNA3D, SDL_GPU
DirectX and Windows families	DIRECTX1, DIRECTX2, DIRECTX3, DIRECTX5, DIRECTX6, DIRECTX7, DIRECTX8, DIRECTX9, DIRECTX10, DIRECTX11, DIRECTX12, DIRECT2D, GDI, GLIDE, FREEDIRECT
2D, web, CPU, and diagnostic	SDL_RENDERER, CANVAS, HTML_DOM, SVG_DOM, PIXIJS, SKIA, BLEND2D, OPENVG, NANOVG, HEADLESS, SOFTWARE, TINYGL, STUB

Table 4.1: All 50 public renderer identities, grouped into 46 implementation families. Windows, browser, macOS, dependency, and architecture gates are enforced during configuration; the exact per-identity matrix belongs to Chapter 20.

4.3 The full CMake option reference

-DCNA_GRAPHICS_RENDERER is the option most build invocations in this book set explicitly, but CNA’s top-level CMakeLists.txt declares several more, every one of them read directly from that file rather than assumed from memory:

Option	Default	Effect
CNA_USE_CCACHE	ON	Auto-detects <code>ccache</code> and wires it in as the C/C++ compiler launcher for both CNA and <code>sharp-runtime</code> , unless a launcher is already set.
CNA_BUILD_TESTS	ON	Builds the <code>CnaTests</code> GoogleTest target and registers CTest entries.
CNA_BUILD_EXAMPLES	ON	Builds examples admitted by the selected modules, renderer and host.
CNA_GRAPHICS_RENDERERS	<i>(empty)</i>	Opt-in semicolon-separated renderer set. Empty preserves single-renderer mode; otherwise <code>CNA_GRAPHICS_RENDERER</code> names its required default.
CNA_PLATFORM	SDL3	Chooses window, event, input, timing and system-service implementation: <code>SDL3</code> , <code>SDL2</code> , <code>HEADLESS</code> , or POSIX-only <code>TERMINAL</code> .
CNA_AUDIO_PLATFORM	SDL3	Independently chooses <code>SDL3</code> , <code>SDL2</code> , or deterministic <code>NULL</code> playback/capture infrastructure.
CNA_BUILD_C_API	OFF	Adds the experimental C17 shared-library target <code>CNA::CApi</code> . At alpha.1 that target is compile-blocked by a stale 49-identity C renderer map; see Appendix I.
CNA_CNAEXT	OFF	Enables an opt-in, project-specific extended render pipeline (see below) — unrelated to the <code>CNAEXT</code> marker macro from Chapter 3.
CNA_DEVICES	OFF	Enables CNA-specific device/sensor extensions beyond XNA 4.0 (battery, camera, clipboard, and similar).
CNA_ENABLE_NET	ON	Builds <code>Net</code> and <code>GamerServices</code> networking; pulls in the vendored ENet dependency when on.
CNA_SHARP_RUNTIME_ROOT	<code>../sharp-runtime</code>	Overrides the required Sharp Runtime source tree. The release records a verified revision but does not enforce that revision in CMake.
CNA_USE_SYSTEM_SDL	OFF	Uses <code>find_package</code> for system <code>SDL3</code> packages instead of the vendored sub-modules (see above).
CNA_TEST_DISPLAY	<code>":0"</code>	The <code>DISPLAY</code> value baked into every GPU/window-creating CTest's environment; override to point the suite at a virtual display (e.g. <code>Xvfb</code>) instead of a real desktop.
CNA_SANITIZE	<i>(empty)</i>	Comma-separated sanitizer list (e.g. <code>address,undefined</code>) added to both compile and link flags project-wide; diagnostics only, never for a shipped build.

These switches are separately declared, but some combinations are intentionally refused. For example, a terminal platform admits only CPU or diagnostic renderers; SDL2-only mode has an explicit consistency gate; and a renderer may add its own SDK or architecture constraint. One combination lacks such a gate: the C API references Net and GamerServices unconditionally, but `CNA_BUILD_C_API=ON` with `CNA_ENABLE_NET=OFF` configures successfully and then fails on missing GamerServices headers. Keep networking enabled when reproducing the separate NanoVG identity failure recorded in Appendix I.

4.4 Generated product-version API

Configuration generates `CNA/Version.hpp` under the build include tree. It exposes `CNA_VERSION_MAJOR`, `_MINOR`, `_PATCH`, `_PRERELEASE`, and `_STRING`, plus `constexpr` accessors. The tag spelling has a leading `v`; the API's product string does not:

```

1  #include <CNA/Version.hpp>
2  #include <iostream>
3
4  int main()
5  {
6      std::cout << CNA::getVersionString() << '\n'; // 0.1.0-alpha.1
7      return CNA::isPreReleaseVersion() ? 0 : 1;
8  }
```

This listing was compile-checked as C++23 against the header generated by an exact-tag HEADLESS/HEADLESS/NULL configure; the executable returned `0.1.0-alpha.1`. It is a compile/runtime observation of the version header, not a renderer test. The C API's independent ABI version remains 0.7.0.

Two similar spellings at different gates. `CNA_CNAEXT` (a CMake option(), off by default) and `CNAEXT` (the marker macro from §3.3) are independent — confusing them is an easy real mistake to make when skimming the source. The marker macro tags an individual declaration as “not part of stock XNA 4.0” and is present in every build regardless of any flag. `CNA_CNAEXT` the CMake option instead gates an entire `CNA::Graphics` engine layer in `modules/graphics-ext`: PBR material data, CRT/depth/ASCII post-process effects, and `RenderPipelineSettings`. The last of these is only a settings bag, not a description of the whole extension module. Building with the option's OFF default leaves the guarded declarations and implementations out. Separately, `CNA::CnaExt` is a link umbrella over `graphics-ext` and `devices-ext`; Appendix E maps all three mechanisms.

4.5 Windows, natively and cross-compiled

On real Windows, `SDL_RENDERER` is selected automatically when no renderer is specified, and SDL is built from the vendored submodule with no pre-built binaries or `CMAKE_PREFIX_PATH` configuration needed:

```

1  git submodule update --init
2  cmake -S . -B build-win -DCNA_GRAPHICS_RENDERER=SDL_RENDERER
3  cmake --build build-win --target CNA CnaTests
```

The same target can be reached without a Windows machine, by cross-compiling from Linux with MinGW-w64 — the toolchain every Windows-targeting CTest binary in this book's Part IV chapters (`DIRECTX9/DIRECTX11/DIRECTX12`, and the plain `SDL_RENDERER` Windows target) is actually verified through:

```

1  sudo apt install mingw-w64
2
3  git submodule update --init
4  cmake -S . -B build-windows \
5      -DCMAKE_TOOLCHAIN_FILE=cmake/toolchains/mingw-w64.cmake \
6      -DCNA_GRAPHICS_RENDERER=SDL_RENDERER
7  cmake --build build-windows --target CNA CnaTests

```

Chapter 28 and this book’s cross-platform verification chapter cover what happens after the cross-compiled `.exe` exists: running it under Wine, with DXVK (Direct3D 9/11) or vkd3d-proton (Direct3D 12) translating the Direct3D calls to Vulkan on a real GPU, which is how this project verifies Windows-only renderers without ever touching a Windows machine.

4.6 Building for Android and the Web

Neither of these two targets gets its own `cmake/toolchains/` file the way `mingw-w64.cmake` does for Windows — both instead rely on toolchain infrastructure that ships with the NDK and the Emscripten SDK respectively, invoked directly.

4.6.1 Android (NDK)

Grounded in the project’s own `docs/devices-build.md`, a real cross-compile invocation looks like this:

```

1  cmake -S . -B cmake-build-android -G Ninja \
2      -DCMAKE_TOOLCHAIN_FILE="$HOME/Android/Sdk/ndk/30.0.14904198/build/
3      cmake/android.toolchain.cmake" \
4      -DANDROID_ABI=arm64-v8a \
5      -DANDROID_PLATFORM=android-24 \
6      -DCNA_BUILD_TESTS=OFF
7  cmake --build cmake-build-android --target CNA -j"$(nproc)"

```

Three details worth knowing before running this yourself, each one a real, documented finding rather than a generic Android-CMake tip: the exact NDK version path (`30.0.14904198`) is whatever happens to be installed in a given environment, not a fixed requirement — check `ls ~/Android/Sdk/ndk/` first rather than assuming either that it exists or that this exact version is what you have. `-DCNA_BUILD_TESTS=OFF` is not optional on this path today: `googletest` was never configured for the NDK toolchain, so only the `CNA` static library itself cross-compiles, not `CnaTests`. And this specific invocation is a **compile-only** verification — no APK packaging and no emulator/device run happen as part of it; Chapter 67 covers the further steps (the NDK’s own `llvm-nm`, not the host’s plain `nm`, is what actually confirms platform-conditional code compiled in) that turn a successful compile into a running, on-device verification.

4.6.2 Web (Emscripten)

The `CANVAS` renderer (Chapter 31) is Emscripten-only, hard-gated at configure time the same way the Direct3D renderers are hard-gated to Windows. Building it follows the standard Emscripten CMake pattern — `emcmake` wraps the configure step, `emmake` (or a plain `cmake -build`, once configured) wraps the build — applied to the same `-DCNA_GRAPHICS_RENDERER` flag every other renderer uses:

```

1  emcmake cmake -S . -B build-web -DCNA_GRAPHICS_RENDERER=CANVAS

```

```
2 cmake --build build-web
3 node build-web/CnaTests.js
```

This is a real, verified path, not a hopeful sketch: the project’s own tracking records a genuine `emcmake / emcc 6.0.2` configure-and-build succeeding, with the resulting `CnaTests.js` linking cleanly and a renderer-agnostic GTest suite (35 of 35 `Rectangle`-family tests) genuinely passing under `node`. Getting there surfaced a real, pre-existing bug outside CNA’s own tree entirely: `sharp-runtime`’s `FileSystemWatcher.hpp` declared three `inotify`-specific fields unconditionally, even though they were only ever *used* under `#ifdef __linux__` — a guard `emcc` does not define, so the fields existed but nothing initialized them, a genuine latent bug independent of Emscripten that this cross-compile attempt is what actually surfaced. Fixed directly in `sharp-runtime`, project-owner approved. What this build does *not* yet prove is anything about a real browser: this project’s own dev loop has no DOM and no `CanvasRenderingContext2D` at all, so `node`-based verification and real-browser verification remain two genuinely different claims, covered in full in Chapter 31.

4.7 System-installed SDL, if you prefer it

Rather than the vendored submodules, system `SDL3`/`SDL3_image`/`SDL3_mixer` packages can be used instead:

```
1 cmake -S . -B build -DCNA_USE_SYSTEM_SDL=ON -DCNA_GRAPHICS_RENDERER=
  SDL_RENDERER
```

This calls `find_package(SDL3 REQUIRED)`, `find_package(SDL3_image REQUIRED)`, and `find_package(SDL3_mixer REQUIRED)`, and requires those development packages to already be installed system-wide.

4.8 Linux system dependencies beyond SDL

Two native dependency routes deserve explicit configuration:

- **FFmpeg** (`libavcodec-dev libavformat-dev libavutil-dev libswresample-dev`) is required for `VideoPlayer` (video decoding). `libswscale-dev` is notably *not* required even though FFmpeg’s own video pipeline usually pairs with it — CNA implements `YUV→RGBA` conversion internally, so only the runtime `libswscale8` library needs to be present, not its headers.
- **Draco** decoding is on by default through CNA’s immutable `third_party/draco` gitlink at Draco 1.5.7. A packager may request a system package with `CNA_USE_SYSTEM_DRACO=ON`; `CNA_ENABLE_DRACO=OFF` deliberately builds the refusal path. The default therefore requires that submodule to be initialized, and does not vary with the host’s installed Draco version.

4.9 Verifying a build

CNA intentionally does not ship a bundled playable game demo as its primary verification artifact — the project prioritizes framework/runtime development over a shipping demo executable. Two commands stand in for “does this build actually work”:

```
1 ctest --test-dir build --output-on-failure
2 cmake --build build --target cna_demo_2d
```

`ctest` runs the GoogleTest-based **CnaTests** suite (thousands of unit tests, plus, on renderers where a display is available, GPU pixel tests); `cna_demo_2d` is the shared 2D integration program used by the platform and renderer infrastructure.

On CI coverage. At the tag, 19 workflows have an automatic push or pull-request trigger and the Direct3D/GDI workflows are manual-only. The intended general unfiltered job and two Input legs carry a retired renderer value and stop during configuration; other automatic jobs exercise narrower Input, Devices, Web, Metal, or arithmetic populations. Chapter 73 traces each route from trigger through verdict. Treat a green badge as evidence for those engaged routes, not as a universal project-wide test count or proof that every renderer ran.

Chapter 5

Your First CNA Game

This chapter develops a compact CNA program from the XNA-style skeleton in CNA's README. It is a complete translation unit, but the surrounding CMake target and named assets remain project inputs. Later chapters supply the detailed framework contracts.

5.1 The whole program

```
1  #include <memory>
2
3  #include "Microsoft/Xna/Framework/Game.hpp"
4  #include "Microsoft/Xna/Framework/Color.hpp"
5  #include "Microsoft/Xna/Framework/GraphicsDeviceManager.hpp"
6  #include "Microsoft/Xna/Framework/Graphics/SpriteBatch.hpp"
7  #include "Microsoft/Xna/Framework/Graphics/Texture2D.hpp"
8
9  using namespace Microsoft::Xna::Framework;
10 using namespace Microsoft::Xna::Framework::Graphics;
11
12 class MyGame final : public Game {
13 public:
14     MyGame()
15         : graphics_(this)
16     {
17     }
18
19 protected:
20     void LoadContent() override
21     {
22         spriteBatch_ = std::make_unique<SpriteBatch>(
23             getGraphicsDeviceProperty());
24         logo_ = std::make_unique<Texture2D>("assets/logo.png",
25             getGraphicsDeviceProperty());
26     }
27
28     void Update(GameTime& gameTime) override
29     {
30         // Update game state here.
31         Game::Update(gameTime);
32     }
33
34     void Draw(const GameTime& gameTime) override
35     {
36         auto& device = getGraphicsDeviceProperty();
37         device.Clear(Color::CornflowerBlue);
38     }
39 }
```

```

37         spriteBatch_>Begin();
38         spriteBatch_>Draw(*logo_, 100.0f, 80.0f);
39         spriteBatch_>End();
40         Game::Draw(gameTime);
41     }
42
43     private:
44         GraphicsDeviceManager graphics_;
45         std::unique_ptr<SpriteBatch> spriteBatch_;
46         std::unique_ptr<Texture2D> logo_;
47 };
48
49 int main()
50 {
51     MyGame game;
52     game.Run();
53     return 0;
54 }

```

5.2 Reading it top to bottom

The includes are the public surface. Each header follows the XNA namespace path; no CNA-specific include is required just to boot a game. This is the API-mirroring goal from Chapter 3 made visible: a developer who has written XNA or FNA code before can read this file without learning a single CNA-specific concept first.

MyGame inherits Game, and owns a GraphicsDeviceManager constructed against itself. This is the standard XNA source-level pattern, but CNA's C++ ownership differs from FNA's: the `Game` base already default-constructs its own `GraphicsDevice` before the derived constructor creates this manager. The manager registers the lifecycle services and, during `Game::DoInitialize()`, applies the preferred settings by resetting that same game-owned object; it does not construct or own a second device. Chapter 6 covers that boot sequence in full, and Chapter 12 covers what the resulting `GraphicsDevice` exposes.

LoadContent() is where GPU-backed resources are created, not the constructor. By the time `LoadContent()` runs, the graphics renderer has already been constructed by the base `Game` and then configured by `GraphicsDeviceManager`, so it is safe to construct a `SpriteBatch` against `getGraphicsDeviceProperty()` and to load a `Texture2D` from a file path. A derived constructor can technically already see CNA's default renderer, but it precedes the manager's preferred-setting reset and is the wrong lifecycle point for content. Defer resources to `LoadContent()` for the stable, configured device rather than relying on the old, incorrect claim that no renderer exists yet. This ordering constraint — and CNA's raw-asset-plus-JSON-descriptor content model that makes "assets/logo.png" a literal, directly loadable file path rather than a compiled .xnb content-pipeline output — is covered fully in Chapter 33.

Update(GameTime&) and Draw(const GameTime&) are the two halves of every frame, and note both the constness difference and the base calls. `Update` takes a mutable `GameTime&` (some XNA game loops advance simulation-only clocks here); `Draw` takes a `const GameTime&`, since rendering should not mutate timing state. Calling `Game::Update(gameTime)` runs registered updateable components and the framework dispatcher; calling `Game::Draw(gameTime)` runs registered drawable components. Omitting the first call silently stops dynamic audio, media, and touch polling. Chapter 6 covers the full lifecycle and the ordering choices this pair of overrides plugs into.

The draw call is the canonical `SpriteBatch` triad: `Begin()` / `Draw(...)` / `End()`. `GraphicsDevice::Clear` takes a named XNA color constant (`Color::CornflowerBlue` — traditionally XNA’s default clear color in generated project templates, preserved here rather than swapped for something “more CNA”). Application `Draw()` does not call `GraphicsDevice::Present()` itself: after the override returns, `Game::EndDraw()` delegates to the manager, which presents exactly once regardless of which public renderer identity from Part IV is active. Chapter 13 covers the full `SpriteBatch` API, including the overloads of `Draw` beyond the two-float-position form used here.

`main()` is exactly two lines of game-specific code. Construct the game, call `Run()`. Everything about window creation, renderer initialization, the timing loop, and shutdown lives inside `Game::Run()` — covered in Chapter 6 — not in application code. This is deliberate: it is the same shape of XNA `Program.cs`’s `Main` method, preserved so that porting an existing XNA game’s entry point is close to a mechanical translation.

5.3 A second program: making it move

The first program leaves `Update` empty. The next fragment extends the same skeleton with arrow-key movement at a frame-rate-independent speed. It shows only the changed members and additional includes:

```

1  #include "Microsoft/Xna/Framework/Input/Keyboard.hpp"
2  #include "Microsoft/Xna/Framework/Vector2.hpp"
3
4  using namespace Microsoft::Xna::Framework::Input;
5
6  class MyGame final : public Game {
7      // ... constructor and LoadContent() unchanged ...
8
9      void Update(GameTime& gameTime) override
10     {
11         const KeyboardState keys = Keyboard::GetState();
12         const float seconds =
13             static_cast<float>(gameTime.getElapsedGameTimeProperty()
14                               .getTotalSecondsProperty());
15         const float speed = 200.0f; // pixels per second
16
17         if (keys.IsKeyDown(Keys::Left)) position_.X -= speed *
18             seconds;
19         if (keys.IsKeyDown(Keys::Right)) position_.X += speed *
20             seconds;
21         if (keys.IsKeyDown(Keys::Up)) position_.Y -= speed *
22             seconds;
23         if (keys.IsKeyDown(Keys::Down)) position_.Y += speed *
24             seconds;
25
26         Game::Update(gameTime);
27     }
28
29     void Draw(const GameTime& gameTime) override
30     {
31         auto& device = getGraphicsDeviceProperty();
32         device.Clear(Color::CornflowerBlue);
33
34         spriteBatch_ -> Begin();
35         spriteBatch_ -> Draw(*logo_, position_, Color::White);
36     }
37 }
```

```

31         spriteBatch_ ->End();
32         Game::Draw(gameTime);
33     }
34
35     private:
36         GraphicsDeviceManager graphics_;
37         std::unique_ptr<SpriteBatch> spriteBatch_;
38         std::unique_ptr<Texture2D> logo_;
39         Vector2 position_{100.0f, 80.0f}; // new: replaces the two
           hardcoded floats
40     };

```

Reading polled input is a static call, not an object you own. Keyboard has no constructor to call and nothing to store between frames — `Keyboard::GetState()` is a static method returning a fresh, immutable `KeyboardState` snapshot every time it is called, and `Update` calls it once per frame. This is the same shape Chapter 44 covers for `Mouse` and `GamePad`.

Frame-rate independence comes from `GameTime`, not a hardcoded per-frame step. Multiplying a pixels-per-second constant by `gameTime.getElapsedGameTimeProperty().getTotalSecondsProperty()` — the declared `TimeSpan` accessor pair, not an invented shortcut — means the logo moves at the same real-world speed whether the game is running at 30 FPS or 240 FPS. A version that instead moved by a flat constant every `Update` call would move four times faster on a display with a four times higher refresh rate, which is the class of bug frame-rate-independent movement exists to avoid.

Why this program uses a different `Draw` overload than the first one. The first program in this chapter calls `spriteBatch_ -> Draw(*logo_, 100.0f, 80.0f)` — but that specific two-float-position overload, reading `modules/graphics/include/Microsoft/Xna/Framework/Graphics/SpriteBatch.hpp` directly, is tagged `CNAEXT`: a CNA-only convenience overload, not part of XNA 4.0 (Chapter 3 covers what that tag means). This second program instead calls `Draw(*logo_, position_, Color::White)`, the unmarked XNA 4.0 overload that takes a `Vector2` position and an explicit tint color — worth calling out directly here, since a reader who wants code that ports cleanly to XNA/FNA should reach for this overload as the default, not the convenience one the first, minimal example used to stay as short as possible.

5.4 A third program: bouncing off the edges, with sound

The second program does not constrain the sprite to the viewport. This fragment clamps it to the backbuffer bounds and plays a sound when a boundary is reached:

```

1  class MyGame final : public Game {
2      // ... constructor unchanged ...
3
4      void LoadContent() override
5      {
6          spriteBatch_ = std::make_unique<SpriteBatch>(
7              getGraphicsDeviceProperty());
8          logo_ = std::make_unique<Texture2D>("assets/logo.png",
9              getGraphicsDeviceProperty());
10         bounceSound_ = std::make_unique<SoundEffect>("assets/bounce.
           wav");
11     }
12 }

```

```

11     void Update(GameTime& gameTime) override
12     {
13         const KeyboardState keys = Keyboard::GetState();
14         const float seconds =
15             static_cast<float>(gameTime.getElapsedGameTimeProperty()
16                               .getTotalSecondsProperty());
17         const float speed = 200.0f;
18
19         if (keys.IsKeyDown(Keys::Left))    position_.X -= speed *
20             seconds;
21         if (keys.IsKeyDown(Keys::Right)) position_.X += speed *
22             seconds;
23         if (keys.IsKeyDown(Keys::Up))      position_.Y -= speed *
24             seconds;
25         if (keys.IsKeyDown(Keys::Down)) position_.Y += speed *
26             seconds;
27
28         const Viewport& viewport = getGraphicsDeviceProperty().
29             getViewportProperty();
30         const float maxX = static_cast<float>(viewport.
31             getWidthProperty()) - logo_->getWidthProperty();
32         const float maxY = static_cast<float>(viewport.
33             getHeightProperty()) - logo_->getHeightProperty();
34
35         bool hitEdge = false;
36         if (position_.X < 0.0f) { position_.X = 0.0f; hitEdge =
37             true; }
38         if (position_.X > maxX) { position_.X = maxX; hitEdge =
39             true; }
40         if (position_.Y < 0.0f) { position_.Y = 0.0f; hitEdge =
41             true; }
42         if (position_.Y > maxY) { position_.Y = maxY; hitEdge =
43             true; }
44
45         if (hitEdge) bounceSound_->Play();
46         Game::Update(gameTime);
47     }
48
49 private:
50     // ... other members unchanged ...
51     std::unique_ptr<SoundEffect> bounceSound_;
52 };

```

The bounds check clamps position, it does not just detect it. Setting `position_.X` back to `0.0f` or `maxX` in the same branch that detects the violation matters: without it, holding a direction key against the edge would let `position_` keep accumulating past the boundary every frame, and the sprite would need to travel back through the entire out-of-bounds distance before visibly re-entering the screen. Clamping in the same statement that detects the crossing keeps the visible sprite exactly at the edge, every frame, for as long as the key is held.

The viewport, not a hardcoded screen size, is what `maxX` / `maxY` are computed against. `Viewport::Width` / `Height` (Chapter 12) reflect whatever the back buffer's actual current size is — reading them here, rather than hardcoding the numbers the game happened to launch with, is what keeps this bounds check correct if the window is resized or the game runs under a different `PresentationMode` (Chapter 6).

`SoundEffect`'s file-path constructor is a CNAEXT convenience, the same shape as

Texture2D's. Real XNA has no such constructor either — a real XNA game always loads a **SoundEffect** through the compiled content pipeline. **Play()** here uses the simplest zero-argument overload (default volume, pitch, and pan); a real game that wants to vary bounce loudness by impact speed would reach for the three-argument **Play(volume, pitch, pan)** overload instead — covered in full in this book's audio chapter.

5.5 A fourth program: rectangle collision

The third program reacted to the screen's own edges; this section adds the other kind of reaction a first game almost always needs next — two sprites reacting to *each other*. A second, static texture (a coin) sits on screen; when the moving logo's own bounding rectangle overlaps it, the coin is collected and a counter increments. One more addition on top of the same running example:

```

1  class MyGame final : public Game {
2      // ... constructor unchanged ...
3
4      void LoadContent() override
5      {
6          spriteBatch_ = std::make_unique<SpriteBatch>(
7              getGraphicsDeviceProperty());
8          logo_ = std::make_unique<Texture2D>("assets/logo.png",
9              getGraphicsDeviceProperty());
10         bounceSound_ = std::make_unique<SoundEffect>("assets/bounce.
11             wav");
12         coin_ = std::make_unique<Texture2D>("assets/coin.png",
13             getGraphicsDeviceProperty());
14     }
15
16     void Update(GameTime& gameTime) override
17     {
18         // ... movement and edge-clamping from the third program,
19         // unchanged ...
20
21         if (!collected_)
22         {
23             const Rectangle logoBounds(static_cast<int>(position_.X),
24                                     ,
25                                     static_cast<int>(position_.Y
26                                     ),
27                                     logo_>getWidthProperty(),
28                                     logo_>getHeightProperty());
29             const Rectangle coinBounds(static_cast<int>(
30                 coinPosition_.X),
31                                     ,
32                                     static_cast<int>(
33                     coinPosition_.Y),
34                                     coin_>getWidthProperty(),
35                                     coin_>getHeightProperty());
36
37             if (logoBounds.Intersects(coinBounds))
38             {
39                 collected_ = true;
40                 ++score_;
41                 bounceSound_>Play();
42             }
43         }
44     }
45 }

```

```

34     Game::Update(gameTime);
35 }
36
37 void Draw(const GameTime& gameTime) override
38 {
39     auto& device = getGraphicsDeviceProperty();
40     device.Clear(Color::CornflowerBlue);
41
42     spriteBatch->Begin();
43     spriteBatch->Draw(*logo_, position_, Color::White);
44     if (!collected_) spriteBatch->Draw(*coin_, coinPosition_,
45         Color::White);
46     spriteBatch->End();
47     Game::Draw(gameTime);
48 }
49 private:
50     // ... other members unchanged ...
51     std::unique_ptr<Texture2D> coin_;
52     Vector2 coinPosition_{300.0f, 200.0f};
53     bool collected_ = false;
54     int score_ = 0;
55 };

```

The bounding rectangles are built fresh every frame, from the same position/size data already driving movement and drawing — not stored and kept in sync by hand. Rectangle’s constructor (Chapter 7) takes an integer `x / y / width / height`, so `position_`’s float components need an explicit `static_cast<int>` — the same pattern `Texture2D::Width / Height` already supply directly for the size half of each rectangle. Building both rectangles inline in `Update()`, from the exact position fields `Draw()` also reads, is what keeps the collision check and the visible sprites from ever silently disagreeing about where anything actually is.

Touching edges do not intersect. `RectangleTests.cpp` checks two 100×100 rectangles placed at $(0,0)$ and $(100,0)$ — sharing an edge exactly, with zero pixels of true overlap — return `false` from `Intersects()`, as do separated rectangles. Only non-zero-area overlap counts. A game that places a coin exactly at the logo’s own width, expecting the two to register as “touching” the instant they meet, will see the check silently never fire; leaving at least a few pixels of deliberate overlap margin between two sprites meant to collide avoids this class of bug entirely.

5.6 What happens if you change the renderer

Nothing in the file above mentions a graphics renderer at all — that is the entire point of Chapter 19’s abstraction layer. Recompiling this exact, unmodified source file with `-DCNA_GRAPHICS_RENDERER=VULKAN` instead of `OPENGLES3` produces a program that opens a Vulkan-backed window, uploads `assets/logo.png` through the Vulkan texture path, and issues the same `SpriteBatch` draw call through Vulkan command buffers instead of OpenGL calls — with the game-facing source unchanged. Part IV is dedicated to what actually differs underneath that unchanged surface, renderer by renderer.

Chapter 6

The Game Lifecycle

Chapter 5 showed `Update` / `Draw` as two overridden methods without asking what drives them. This chapter opens up `Game` itself — the class every CNA game inherits from — and the small constellation of supporting types (`GameTime`, `GameComponent`, `GameWindow`, `GameServiceContainer`) that make the loop work.

6.1 Game: base class, not framework-owned entry point

`Game` inherits `System::Object` and `System::IDisposable`, and its own Doxygen brief describes it precisely: “base class that provides the XNA-style game loop, services, window, content and components.” Four events — `Activated`, `Deactivated`, `Disposed`, `Exiting` — are all typed `System::EventHandler<System::EventArgs>`. This is worth pausing on for `Exiting` specifically: CNA does declare a dedicated `ExitingEventArgs` class (matching real XNA’s public API surface), but `Game::Exiting` itself is typed against plain `EventArgs`, not `EventHandler<ExitingEventArgs>` — and grepping the tree confirms `ExitingEventArgs` is otherwise unreferenced outside its own translation unit. This looks like an inconsistency until you check real XNA itself, which has exactly the same quirk: `Game.Exiting` is declared `EventHandler<EventArgs>` in the real assembly too, despite a dedicated `ExitingEventArgs` type existing. CNA preserves this faithfully rather than “fixing” it — a small, concrete example of Chapter 3’s “match FNA, not taste” rule in action.

6.1.1 Properties, and one that is deliberately not settable from outside

`Game` exposes its state through the project’s uniform `GetXProperty()` / `setXProperty()` convention: `Components`, `Content`, `GraphicsDevice` (get-only, sourced from the graphics device service — there is no `setGraphicsDeviceProperty`), `InactiveSleepTime`, `IsFixedTimeStep`, `IsMouseVisible`, `LaunchParameters`, `TargetElapsedTime`, `Services`, `Window`. One property is more restricted than the rest: `IsActive` is get-only publicly, with its setter kept private —

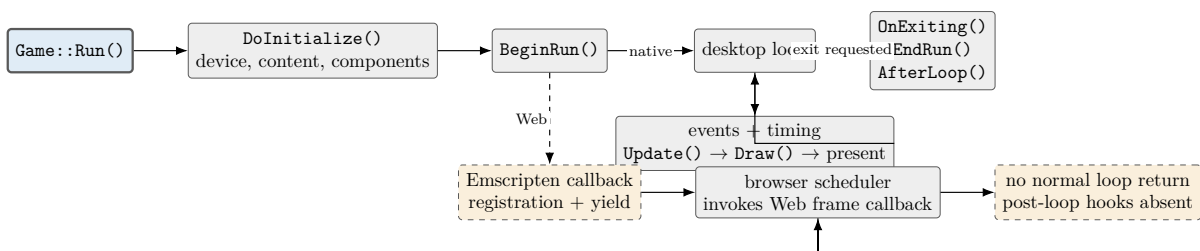


Figure 6.1: Desktop and Emscripten lifecycle control at 1bb2145d. The shared public methods do not imply identical scheduler or shutdown semantics.

“only `Game` itself may change the active state,” matching FNA’s own `internal set`. C++ has no exact equivalent of C#’s assembly-internal visibility, so a private setter reachable only from within `Game`’s own `OnActivated` / `OnDeactivated` handlers is the closest faithful translation.

6.1.2 The override points

A concrete game overrides a fixed set of protected virtuals: `Initialize()`, `LoadContent()`, `UnloadContent()`, `Update(GameTime&)`, `Draw(const GameTime&)`, `BeginRun()` / `EndRun()`, and `BeginDraw()` / `EndDraw()` — the last of which matters more than it looks: `BeginDraw()` returning `false` skips both `Draw` and `EndDraw` for that frame entirely, the mechanism `GraphicsDeviceManager` (§6.7) uses to suppress rendering when no device is available yet. A game can also override `ShowMissingRequirementMessage(const std::exception&)` and the protected `Dispose(bool disposing)`.

`UnloadContent()` is a live part of explicit shutdown. During initialization, `Game` subscribes that virtual to the registered graphics-device service’s `DeviceDisposing` event. `GraphicsDeviceManager::Dispose(bool)` raises the event even though the ordinary game-attached manager does not own the game-valued device it points at; only deletion of the device remains ownership-gated. Thus the public `Game::Dispose()` route reaches the game’s override exactly once through the service. The component-level contract is separate: explicit game disposal first disposes each `IDisposable` component, and `DrawableGameComponent::Dispose(bool)` invokes that component’s own virtual. Normal C++ destruction still does not execute either explicit callback path. Chapter 12 traces the manager ordering and ownership boundary.

6.1.3 `Exit()` requests a stop; `Exiting` describes the completed loop

The names of `Exit()` and `Exiting` invite a plausible but wrong mental model: that calling the former immediately raises the latter. In the desktop implementation, `Exit()` does only two things: it sets the public `RunApplication` loop flag to `false` and sets `suppressDraw_`. The current tick therefore skips its draw if an `Update()` called `Exit()`, and the enclosing `while (RunApplication)` loop then ends. Only after that loop has finished does `RunLoop()` call `OnExiting()`, which raises the event. It is a loop-boundary notification, not an immediate cancellation callback.

For an ordinary first desktop `Run()`, the source gives the following order:

1. `DoInitialize()` runs once when necessary; it creates/configures the registered graphics service, initializes gamepad support, calls `Initialize()`, and categorizes the current components.
2. `BeginRun()` executes, followed by `BeforeLoop()`. The latter sets `IsActive` true, so a newly inactive game raises `Activated` here.
3. `RunLoop()` repeatedly calls `Tick()` while its loop flag remains true. `Exit()` or an SDL quit event clears that flag, and the loop then raises `Exiting`.
4. Only after `Exiting` returns does `Run()` call `EndRun()` and `AfterLoop()`.

This order matters to an override which has both an `Exiting` handler and an `EndRun()` override: the handler runs first. Calling `Exit()` before `Run()` does not bypass the surrounding lifecycle; the initialization and begin hooks still run, the loop has zero ticks, and desktop `RunLoop()` still raises `Exiting`. Nor does `Run()` reset `RunApplication`; a second call after exit reaches the same zero-tick exit boundary unless application code sets that public CNA extension true again. `RunOneFrame()`, by contrast, performs its one-time initialization and one `Tick()` only. It does

not call `BeginRun()`, `BeforeLoop()`, `EndRun()`, `AfterLoop()`, or `OnExiting()`. Because only `BeforeLoop()` activates the game, `IsActive` remains false throughout `RunOneFrame()` — an easy trap for a test harness or editor host which uses that entry point. Browser builds use an Emscripten callback rather than the blocking desktop loop, but likewise raise `Exiting` only when that callback observes the false loop flag and cancels the browser loop.

6.1.4 Explicit disposal, destructor cleanup, and the gap between them

`Game::Dispose()` is not merely a spelling of the C++ destructor. Its protected `Dispose(true)` path first calls `Dispose()` on every current component that implements `IDisposable`. It then disposes the content manager, the cached graphics-device service when one has been resolved, and the SDL gamepad subsystem. It sets the private disposed flag only after that sequence. The public wrapper subsequently raises `Game::Disposed`. Consequently, explicit disposal is the route that makes a `DrawableGameComponent`'s virtual `Dispose(bool)` call its `UnloadContent()` hook. Later in the same sequence, disposing the resolved graphics service raises `DeviceDisposing`; the subscription installed by `Game::Initialize()` then calls `Game::UnloadContent()` before the manager's own `Disposed` event. These are two distinct virtual calls on two distinct objects.

The destructor instead calls `Dispose(false)`, then shuts down audio, and ordinary member destruction happens afterward. The false path deliberately skips the component loop, explicit `ContentManager::Dispose()`, graphics-service disposal, and gamepad-subsystem shutdown. The member destructors still run, so this is not a claim that all native storage survives a normal C++ scope exit; it is a precise warning that the explicit lifecycle callbacks and their ordering do not run there. In particular, `ContentManager`'s destructor is defaulted rather than a call to its public `Dispose()`, and the current `SdlInputBridge::ShutdownGamepadSubsystem()` call site is only the `disposing == true` branch. A typical scoped game which only calls `Run()` therefore relies on wider SDL shutdown for that gamepad-subsystem counterpart, whereas an explicitly disposed game executes it immediately. The same C++ destructor rule matters to a drawable component: it has no destructor which calls its override, and the later `GameComponent` base destructor's virtual call resolves to `GameComponent::Dispose(false)`, not `DrawableGameComponent::Dispose(false)`. Destruction alone therefore does not dispatch the drawable's `UnloadContent()` hook either.

Two event-safety details are visible directly in the short implementation. First, the disposed flag is written only after all the protected true-path cleanup succeeds. An exception from a component, content manager, or graphics service aborts the remaining cleanup, leaves the flag false, and prevents the outer `Disposed` notification. Second, the public wrapper raises `Disposed` even when the protected method returns early because the object was already disposed. Thus a second ordinary `Dispose()` raises the event again; a handler which calls `Dispose()` recursively re-enters that wrapper. This wrapper shape also matches the current FNA `Game` source, but it is still a reason for a handler to release only its own state and never dispose the notifying game again. No located CNA `Game` test exercises normal destruction, handler exceptions, or this re-entrancy boundary. Repeated public disposal *is* covered: the regression test proves that the manager's one-shot guard prevents a second `DeviceDisposing` raise and hence a second game `UnloadContent()`, even though the public `Game::Disposed` notification is raised again.

6.1.5 The timing loop, ported wholesale from FNA

`Game`'s private implementation carries over FNA's adaptive-sleep-precision timing loop directly: a 128-entry ring buffer of `previousSleepTimes_` (masked with `SLEEP_TIME_MASK` rather than a modulo), a running `worstCaseSleepPrecision_` estimate updated by `UpdateEstimatedSleepPrecision()`, and `AdvanceElapsedTime()` driving the actual frame

spacing against `TargetElapsedTime`, capped by a static `MaxElapsedTime`. Two pairs of component-list vectors — `updateableComponents_ / currentlyUpdatingComponents_` and `drawableComponents_ / currentlyDrawingComponents_` — exist specifically to guard against a component mutating the component collection (adding or removing itself, or another component) *during* its own `Update / Draw` call, a re-entrancy hazard FNA’s own implementation guards against the same way.

One platform-conditional block is CNA-specific rather than ported: under `#if defined(_EMSCRIPTEN_)`, `Game` gains a private `EmscriptenLoopState` struct and an `EmscriptenMainLoopCallback()`, because a blocking `while` loop is not a viable main-loop shape in a browser — Emscripten instead requires yielding control back to the browser’s own event loop via a registered callback. It is not merely another scheduler for the same `Tick()` body; it replaces that body and therefore changes observable contracts:

Aspect	Desktop <code>Tick()</code>	Emscripten callback
Clock and pacing	High-resolution <code>SDL_GetPerformanceCounter</code> ; adaptive delay/yield pacing	Millisecond <code>SDL_GetTicks</code> ; browser-driven cadence, no sleep- /spin ring
Step policy	Honors <code>IsFixedTimeStep</code>	Always drains fixed target-sized steps; ignores the property
Lag handling	500ms clamp and five-tick <code>IsRunningSlowly</code> hysteresis	250ms clamp; <code>IsRunningSlowly</code> forced false every step
Control methods	<code>SuppressDraw()</code> is consumed; <code>ResetElapsedTime()</code> affects variable-step mode	Neither field is consulted, so both calls are no-ops
Draw cadence	One draw attempt per tick	Draw only when at least one update ran
Lifetime	Loop returns, then <code>EndRun()</code> and <code>AfterLoop()</code> run	Registered infinite loop does not re- turn; those hooks never run
State ownership	Per-game timing members	One static loop-state slot, hence one running <code>Game</code> per process

The callback still cancels itself and raises `Exiting` after observing `RunApplication == false`. Chapter 64 covers the more dangerous object-lifetime consequence of Emscripten’s non-returning registration and WASM exception mode.

6.1.6 Two global debug keys hidden in the ordinary event pump

`PollEvents()` sends every SDL event through the input bridge first, then reserves two non-repeated key-downs: F9 calls the renderer’s `DebugSimulateContextLoss()`, and F10 calls `DebugRestoreContext()`. The input state still sees both keys because that bridge ran first; a game binding F9/F10 therefore gets its own key state *and* this framework action. FNA’s own `Game` installs no equivalent global hotkeys.

Most CNA products inherit empty hook bodies, so the reservation is invisible. EasyGL and D3D9 make it consequential. On desktop EasyGL either key performs a complete synchronous GL loss-and-recreate cycle; in a browser F9 requests asynchronous loss and F10 separately requests restore. D3D9 instead uses F9 for one `DeviceLost` transition and F10 for the real `DeviceResetting/DeviceReset` recovery sequence. The complete product and evidence matrix is §19.5.1.

6.1.7 Two small, explicitly-marked CNA extensions

Two CNAEXT-tagged additions live directly on `Game`: a public `RunApplication` flag (documented as “internal loop flag matching the FNA/XNA `Game` implementation shape”), and a pair of com-

patibility helpers, `getTargetFPSProperty()` and `getTargetMsFrameTimeProperty()`, plus a static `fpsToMillisecondsPerFrame(intcs)` — documented as existing specifically because “existing CNA examples” already depend on an FPS-based way to talk about frame timing, alongside the real XNA `TargetElapsedTime` `TimeSpan`.

6.1.8 What happens when a frame runs long

The timing loop described above is a real, working catch-up mechanism, not just a sleep-/measure pair — and its concrete behavior under load is worth tracing through with real numbers rather than left as an abstract description, since it directly determines how many times a game’s own `Update()` override runs per displayed frame. `Game::Tick()`, read directly from `Game.cpp`, accumulates real elapsed wall-clock time into `accumulatedElapsedTime_` every call, then — under `IsFixedTimeStep` (the default) — drains that accumulator in whole `TargetElapsedTime_` steps, calling `Update()` once per step, *before* a single `Draw()` for the frame:

```

1  gameTime_.setElapsedGameTimeProperty(TargetElapsedTime_);
2  int stepCount = 0;
3  while (TimeSpanGreaterOrEqual(accumulatedElapsedTime_,
    TargetElapsedTime_))
4  {
5      gameTime_.setTotalGameTimeProperty(gameTime_.
        getTotalGameTimeProperty() + TargetElapsedTime_);
6      accumulatedElapsedTime_ = accumulatedElapsedTime_ -
        TargetElapsedTime_;
7      ++stepCount;
8      Update(gameTime_);
9  }
```

With the real default `TargetElapsedTime` of 166667 ticks (16.6667ms, i.e. 60Hz) and a single real frame that happens to take 50ms — a stutter from, say, a garbage-collector-free but still real hitch: a texture streaming in, a burst of physics work, an OS scheduling gap — the accumulator holds roughly three whole steps’ worth of time. `Tick()` therefore calls `Update()` three times in a row, each with `gameTime.ElapsedGameTime` pinned to exactly one `TargetElapsedTime` (16.6667ms), *before* `Draw()` runs once for that call to `Tick()`. Immediately before drawing, however, the loop rewrites `ElapsedGameTime` to `TargetElapsedTime * stepCount`; in this example the single `Draw()` observes about 50ms, while each of the three updates observed 16.6667ms. This update/draw asymmetry is intentional catch-up accounting, not an unstable clock.

This is the mechanism, not an edge case: a game’s `Update()` must be safe to call multiple times between draws, and must never assume “one `Update()` call happened since the last `Draw()`” — exactly the invariant Chapter 5’s own `Update / Draw` split already depends on without stating it this precisely.

6.1.9 IsRunningSlowly: five-tick hysteresis

`GameTime::IsRunningSlowly` looks, from its own name and Chapter 5’s brief mention, like it might flip true the instant a single `Tick()` needed more than one `Update()` step. Reading `Game::Tick()` directly shows real, deliberate hysteresis instead, tracked by a private `updateFrameLag_` counter that persists across calls:

```

1  updateFrameLag_ += std::max(0, stepCount - 1);    // this Tick()'s
    own lag contribution
2
```

```

3  if (gameTime_.getIsRunningSlowlyProperty()) {
4      if (updateFrameLag_ == 0) gameTime_.setIsRunningSlowlyProperty(
        false);
5  } else if (updateFrameLag_ >= 5) {
6      gameTime_.setIsRunningSlowlyProperty(true);
7  }
8
9  if (stepCount == 1 && updateFrameLag_ > 0) --updateFrameLag_; //
    recover by one, one step at a time

```

Every `stepCount` beyond the first adds to `updateFrameLag_`; the earlier three-`Update()` frame above contributes 2. `IsRunningSlowly` only turns `true` once `updateFrameLag_` reaches 5 — several consecutive lagging ticks, not one — and, once `true`, only clears once `updateFrameLag_` returns all the way to 0, one unit per ordinary (`stepCount == 1`) tick. A single 50ms stutter followed immediately by normal frame times never sets `IsRunningSlowly` at all; a genuinely sustained slowdown does, and takes several good frames afterward to clear rather than resetting instantly on the first frame that recovers. A game using `IsRunningSlowly` to decide whether to skip an optional visual effect this frame is reading a smoothed, several-tick signal, not a single-frame one — worth knowing before assuming it reacts as fast as the frame time itself does. This specific hysteresis mechanism has no dedicated regression test anywhere in the project (`GameTimeTests.cpp` only exercises `GameTime`'s own plain getter/setter, never `Game::Tick()`'s multi-tick accumulation), so its correctness rests entirely on this direct reading of `Game.cpp` matching FNA's own algorithm, not on a passing test asserting it.

6.1.10 `ResetElapsedTime()` under fixed timestep

`Game::ResetElapsedTime()` exists, per its own XNA-inherited contract, to discard a large, misleading elapsed-time spike — the classic case being right after a blocking load screen, where the next `Update()` would otherwise see an artificially huge `ElapsedGameTime` covering the entire load. Reading its real implementation shows a one-line guard easy to miss:

```

1  void Game::ResetElapsedTime()
2  {
3      if (!IsFixedTimeStep_)
4      {
5          forceElapsedTimeToZero_ = true;
6      }
7  }

```

Under the default `IsFixedTimeStep = true`, calling `ResetElapsedTime()` does nothing at all — no field is set, no flag is consulted anywhere in `Tick()`'s fixed-step branch. This is not a CNA gap; it matches real XNA's own documented behavior (a fixed-timestep game's `ElapsedGameTime` is always exactly `TargetElapsedTime` by construction, so there is no spike for `ResetElapsedTime()` to discard in the first place), but it is easy to call defensively after a load screen and assume it did something, when — for the large majority of CNA games, which leave `IsFixedTimeStep` at its default — it silently did not.

6.1.11 The base `Update` and `Draw` calls are framework work, not optional ceremony

The default `Game::Update(GameTime&)` first snapshots and updates enabled components, then ends with `FrameworkDispatcher::Update()`. A derived override which never calls `Game::Update(gameTime)` therefore disables more than automatic components: it silently stops dynamic-stream buffer dispatch, microphone checks, media-player transitions, and

touch-panel updates. The safe shape is to perform game-specific work and call the base exactly once per update step. If a port intentionally replaces the component loop, it must still arrange one dispatcher call itself; calling both paths would pump those services twice.

The parallel rule for `Game::Draw(gameTime)` is simpler but still important: the base method draws the visible registered components in ascending `DrawOrder`. An override that never calls it suppresses every `DrawableGameComponent`. Presentation is different again: the outer tick calls `EndDraw()` after the override returns, and the registered manager presents there. Application `Draw()` code should not call `Present()` itself.

6.2 GameTime: three fields, one privileged writer

`GameTime` is deliberately small: `TotalGameTime` and `ElapsedGameTime` (both `TimeSpan`, publicly get-only), and `IsRunningSlowly` (bool, publicly get-only). All three setters are `private`, with `friend class Game` — no code outside `Game` itself may mutate a `GameTime` once constructed, which is exactly the invariant the two-line `Update` / `Draw` signatures from Chapter 5 rely on: a `GameTime` handed to your `Update` override is a value your code can read freely but never falsify.

6.3 GameComponent and DrawableGameComponent: the update/draw ordering contract

`GameComponent` implements `IGameComponent`, `IUpdateable`, and `System::IComparable<GameComponent>` in one base: a component knows its owning `Game` (get-only), can be individually `Enabled`, has an `UpdateOrder`, and exposes `CompareTo`. That method returns `other.UpdateOrder - this.UpdateOrder`, inverted relative to the usual compare-to sign convention; this is an FNA quirk preserved by CNA. The game itself does not depend on that sign: its insertion comparator explicitly sorts updateables by ascending `UpdateOrder`, with equal-order components retaining their existing relative order. `DrawableGameComponent` layers `IDrawable` on top, adding `GraphicsDevice` (get-only), `DrawOrder`, and `Visible`, and — notably — overrides `Initialize()` and calls `LoadContent()` there. In the ordinary game loop that occurs after `Game::DoInitialize()` has configured the already-existing device, so it is a suitable first content-load point. The private `OnDeviceCreated` method exists but is not subscribed to `IGraphicsDeviceService`; the component does *not* automatically reload content on a later manager `DeviceCreated` event. Component authors must therefore treat this as one initial-load convenience, not a complete device-lifecycle subscription.

`IGameComponent` itself is minimal — one pure-virtual `Initialize()` — and `IUpdateable` / `IDrawable` are the interface pair that `GameComponent` / `DrawableGameComponent` implement concretely: get/set enabled-or-visible state, an ordering value, a changed-event pair, and the actual `Update` / `Draw` method.

6.3.1 A self-contained DrawableGameComponent

The ordering contract described above is easiest to see in a small, real `DrawableGameComponent` — an FPS counter, a natural first component to reach for since it needs its own `Update` (accumulate elapsed time, recompute the average) and its own `Draw` (render the current value), independent of the main game's own overrides:

```
1 class FpsCounterComponent final : public DrawableGameComponent {
2 public:
```

```

3      explicit FpsCounterComponent(Game& game) : DrawableGameComponent
         (game) {}
4
5  protected:
6      void LoadContent() override
7      {
8          font_ = std::make_unique<SpriteFont>(/* ... */);
9          spriteBatch_ = std::make_unique<SpriteBatch>(
              getGraphicsDeviceProperty());
10     }
11
12     void Update(GameTime& gameTime) override
13     {
14         elapsed_ = elapsed_ + gameTime.getElapsedGameTimeProperty();
15         // TimeSpan has no operator+=
16         ++frames_;
17         if (elapsed_.getTotalSecondsProperty() >= 1.0) {
18             fps_ = frames_;
19             frames_ = 0;
20             elapsed_ = TimeSpan::Zero;
21         }
22     }
23
24     void Draw(const GameTime& gameTime) override
25     {
26         (void)gameTime;
27         spriteBatch_>Begin();
28         spriteBatch_>DrawString(*font_, "FPS: " + std::to_string(
29             fps_), Vector2(10, 10), Color::Yellow);
30         spriteBatch_>End();
31     }
32
33 private:
34     std::unique_ptr<SpriteFont> font_;
35     std::unique_ptr<SpriteBatch> spriteBatch_;
36     TimeSpan elapsed_ = TimeSpan::Zero;
37     int frames_ = 0;
38     int fps_ = 0;
39 };
40
41 // In MyGame:
42 class MyGame final : public Game {
43     // ...
44     MyGame() : graphics_(this), fpsCounter_(this)
45     {
46         fpsCounter_.setDrawOrderProperty(1000); // draw last, on top
47         // of everything else
48         getComponentsProperty().Add(&fpsCounter_);
49     }
50
51     ~MyGame() override
52     {
53         // Derived members are still alive here; remove the raw
54         // registration first.
55         getComponentsProperty().Remove(&fpsCounter_);
56     }
57
58 private:
59     GraphicsDeviceManager graphics_;

```



```

55     FpsCounterComponent fpsCounter_;
56 };

```

Two details worth being deliberate about, both real rather than incidental to this example. First, `GameComponentCollection::Add(IGameComponent*)` takes a raw, non-owning pointer. A derived class's members are actually destroyed *before* its `Game` base and the base's `Components` member, so merely making `fpsCounter_` a plain member does not make it outlive the collection. The destructor above is the safe RAII pairing: while the derived destructor body still has a live base and component, `Remove()` raises `ComponentRemoved`; `Game` then removes the component from its update/draw lists and removes the order-change event tokens it installed. Use the same remove-before-destroy rule for a separately owned or heap-allocated component.

This is more than collection tidiness. `Game::Dispose()` disposes components but does not remove them or their order-change tokens, and the collection's implicit C++ destructor does not call `Clear()` (so it raises no removal events). The component setters have no disposed guard. A component which survives an explicitly disposed game can still emit an order event into that game; a component which survives the game's destruction retains a raw callback to the now-dead game and can cause a native use-after-free on its next order change. Removing it while both objects are alive closes both routes. Second, `setDrawOrderProperty(1000)` is what guarantees the FPS counter renders *after* whatever the game's own `Draw` override draws first — `DrawableGameComponent` does not know or care what order its owner's own `Draw` override runs relative to `Game`'s own component-draw loop, so an on-top HUD element needs an explicit, high `DrawOrder` value to guarantee it, not just registration order.

6.4 GameComponentCollection: an event-raising collection, plus a real C++ iterator surface

`GameComponentCollection` raises `ComponentAdded` / `ComponentRemoved` events (typed `EventHandler<GameComponentCollectionEventArgs>`) around the usual collection operations (`Add`, `Insert`, `Remove`, `RemoveAt`, `Clear`, `IndexOf`, `operator[]`). Because C#'s `IEnumerable<IGameComponent>` has no direct C++ equivalent, the collection adds an explicitly CNAEXT-tagged set of `size_type` / `iterator` / `const_iterator` aliases and `begin()` / `end()` overloads — a small, representative example of how CNA extends an XNA collection type just enough to be usable with range-based `for` and the standard library, without touching its XNA-facing surface.

6.5 GameServiceContainer: type-keyed lookup without C# reflection

C#'s `GameServiceContainer` is a dictionary keyed by `System.Type`. C++ has no run-time reflection to build an equivalent dictionary directly, so CNA's `AddService<TService>` / `GetService<TService>` / `RemoveService<TService>` template methods key a `std::unordered_map<std::type_index, void*>` by `typeid`, delegating to non-template overloads underneath. Copy construction and copy assignment are explicitly deleted ("services are registered by pointer identity"), while move construction and move assignment are defaulted — a small but precise statement about ownership: a `GameServiceContainer` can be relocated, but never silently duplicated into a second set of service pointers.

6.5.1 Registering and consuming a service

GameServiceContainer’s own test suite (GameServiceContainerTests.cpp) is a good template for the pattern a real game uses to share a subsystem — say, a save-game manager — across otherwise-unrelated components without threading a pointer through every constructor:

```

1  struct ISaveGameService {
2      virtual void Save(const std::string& slot) = 0;
3      virtual ~ISaveGameService() = default;
4  };
5
6  class SaveGameManager final : public ISaveGameService {
7  public:
8      void Save(const std::string& slot) override { /* ... */ }
9  };
10
11 // In Game::Initialize(), or anywhere with access to
   getServicesProperty():
12 SaveGameManager saveManager;
13 getServicesProperty().AddService<ISaveGameService>(&saveManager);
14
15 // Anywhere else that only has a GameServiceContainer&:
16 if (auto* svc = services.GetService<ISaveGameService>()) {
17     svc->Save("slot1");
18 }

```

Two behaviors worth knowing before relying on this, both read directly from the container’s own implementation rather than assumed: `AddService<T>(nullptr)` throws `std::invalid_argument` rather than silently registering an empty slot, and calling `AddService<T>` a second time for a type that already has a registered service also throws `std::invalid_argument` — there is no “last write wins” overwrite behavior. A service must be explicitly `RemoveService<T>()`’d before a replacement can be registered under the same type.

6.6 GameWindow: one concrete class where FNA has a hierarchy

CNA’s own class comment for `GameWindow` states the deviation from FNA plainly: “FNA defines `GameWindow` as abstract with per-platform subclasses; CNA collapses that hierarchy into one concrete SDL-backed class.” Where FNA needs a `WinFormsGameWindow`, an SDL2-backed window subclass, and so on, CNA needs exactly one, because SDL3 already is the cross-platform windowing abstraction — there is no per-platform subclassing left to do. `GameWindow` exposes the expected XNA surface (`AllowUserResizing`, `ClientBounds`, `CurrentOrientation`, `Handle`, `ScreenDeviceName`, `Title`, and the `ClientSizeChanged` / `OrientationChanged` / `ScreenDeviceNameChanged` events), plus a small set of explicitly-named EXT-suffixed CNA extensions: `GetNativeSdlWindowEXT()` (documented as “never for use in the strict XNA-facing API surface itself,” but needed so e.g. `CNA::Devices::DisplayInfo` can query SDL3 content-scale/safe-area state), `IsBorderlessEXTProperty`, and `MinimizeEXT()` / `RestoreEXT()` (“CNA extension — XNA has no minimize/restore API of its own”). The EXT suffix convention here is a sibling of CNAEXT: both mark non-XNA additions, but EXT specifically marks methods bolted onto an otherwise XNA-shaped class, kept visually distinct from the XNA members around them.

Mutating window properties is not uniformly best-effort. SDL failures in setters such as title,

size, border, resizability, orientation, minimize, and restore become `std::runtime_error`. The deliberate exception is a read: if `SDL_GetWindowSize()` transiently fails during the browser backend's asynchronous startup, `ClientBounds` returns its last-known rectangle and lets a later resize event correct it rather than unwinding the game loop.

`GetNativeSdlWindowEXT()` is a borrowed pointer, not an ownership transfer. In an ordinary windowed game it views the device-created SDL window; in a headless configuration it can be null. Never destroy it from game code or retain it across explicit graphics-device disposal: the wrapper does not own it and is not notified when a separately owned device tears it down. The complete renderer availability, attached-window, input-routing, and teardown contract is in Chapter 19, §19.1.3.

6.7 GraphicsDeviceManager's role in the loop

`GraphicsDeviceManager` — already seen configuring the game-owned device in Chapter 5 — implements `IGraphicsDeviceManager`'s three-method contract (`BeginDraw()` → `bool`, `CreateDevice()`, and `EndDraw()`), which is exactly what `Game::BeginDraw()` / `EndDraw()` delegate to: if `GraphicsDeviceManager::BeginDraw()` returns `false` (no device available yet), `Game` skips `Draw` / `EndDraw` for that frame rather than crashing on a null device. Its five events (`DeviceCreated`, `DeviceDisposing`, `DeviceReset`, `DeviceResetting`, `PreparingDeviceSettings`) retain the FNA-shaped service surface, but its selection virtuals are currently not a real selection pipeline: no manager path invokes `FindBestDevice`, `RankDevices`, or `CanResetDevice`; the defaults respectively build a new default record, do nothing, and answer only whether a device pointer exists. A subclass override of `RankDevices` cannot therefore choose a higher-resolution adapter in today's CNA. The detailed event ordering, manager/device distinction, and the one-operation nature of `PreparingDeviceSettings` are documented in Chapter 12.

Construction deliberately registers the manager's two services without calling `ApplyChanges()`: `Game::DoInitialize()` soon calls `CreateDevice()`, which is the single initial configuration pass. That pass applies graphics profile, presentation mode, a real in-place `GraphicsDevice::Reset()`, and viewport refresh in that order. Only after the settle-in reset does the manager subscribe to the device's own `DeviceResetting`/`DeviceReset` events, so startup reports `DeviceCreated` rather than a synthetic reset pair. A later `ApplyChanges()` follows the same reset path; the device raises its pair and the manager forwards it exactly once instead of raising a duplicate pair itself. Forwarded reset and disposing events report the manager as sender, matching FNA.

One CNA-specific extension lives directly alongside this class rather than inside it: a `CNAEXT` enum class `PresentationMode` with five values — `Letterbox`, `Overscan`, `Stretch`, `NativeBackBuffer`, `FixedHeightDynamicWidth` — expressing the game's requested policy for its virtual back buffer and actual display when the two don't match in aspect ratio, exposed through `getPreferredPresentationModeProperty()` / `setPreferredPresentationModeProperty()`. Real XNA never had to solve this problem in the same way, since the Xbox 360 and a Windows desktop of that era rarely presented CNA's variety of target displays (a browser canvas, an Android device in either orientation, an arbitrary desktop window); this is a genuinely new subsystem, clearly marked as such rather than folded silently into the XNA-facing API. The enum is a manager-side request, not a portable promise that every renderer has a matching scale, letterbox, or crop pass; Chapter 19 gives the current product matrix.

A game that wants a fixed, letterboxed virtual resolution — one of the more common real Getting-Started decisions, since it means every downstream draw call can assume one known

coordinate space regardless of the player’s actual window or display size — sets this before the device is created, typically in the constructor right alongside `graphics_(this)`:

```

1  MyGame::MyGame()
2      : graphics_(this)
3  {
4      graphics_.setPreferredBackBufferWidthProperty(1280);
5      graphics_.setPreferredBackBufferHeightProperty(720);
6      graphics_.setPreferredPresentationModeProperty(PresentationMode
7          ::Letterbox);
8  }
```

On a renderer that implements Letterbox, a player resizing the window to a wider or taller aspect ratio than 1280:720 sees black bars rather than a stretched or cropped game, while draw calls remain in the 1280×720 logical coordinate space. The manager delivers this request before resetting virtual resolution; the GraphicsDevice path and the renderer-specific execution matrix are documented in Chapters 12 and 19.

6.7.1 Vertical retrace is a presentation preference, not the fixed timestep

`SynchronizeWithVerticalRetrace` defaults to `true`. During device selection, the manager translates that Boolean into `PresentationParameters.PresentationInterval`: `true` means `One`, `false` means `Immediate`. `ApplyChanges()` then resets CNA’s already-existing device and forwards the integer 1 or 0 to the renderer. This is independent of `Game::IsFixedTimeStep`. The fixed timestep controls how many `Update()` calls the accumulator schedules; VSync can separately block `Present()` at the display boundary. Turning one off does not turn the other off.

XNA’s third public choice, `PresentInterval::Two`, cannot be expressed through this Boolean property. A port that genuinely needs half-refresh presentation must set the interval on explicit `PresentationParameters`, for example in a `PreparingDeviceSettings` handler, and must still check the renderer result described in §19.2.4. That matrix matters: SDL Renderer and EasyGL submit the numeric 2 but cannot guarantee driver acceptance; D3D9 has a literal native interval 2 but it is fullscreen- and capability-dependent. Several other products reduce every positive value to ordinary VSync.

The Vulkan renderer consumes its present-mode choice only when its swapchain is first constructed and inherits the runtime `SetSwapInterval()` no-op. CNA constructs a `Game`’s device before the derived game can configure its manager, so the ordinary request to disable the manager’s vertical-retrace property, followed by `ApplyChanges()`, leaves Vulkan on the default FIFO VSync mode. The public property still reports the requested `Immediate`; it is not an observation of the active swapchain mode.

6.7.2 Format and fullscreen preferences: three fields, three kinds of truth

The manager’s backbuffer format starts as `Color`, its depth/stencil format as `Depth24`, and fullscreen as `false`. FNA’s manager uses the same defaults. A bare presentation object differs: its depth format begins as `None`. During device selection the manager copies all three preferences into the reset parameters, just as it does size and interval.

They do not have one uniform meaning after `ApplyChanges()`. CNA first stores all three values. It then asks SDL to change fullscreen state and window size, and finally calls the renderer’s `UpdatePresentationFormatEXT()` hook with the requested color format, depth/stencil

format, and fullscreen Boolean. Only D3D9 overrides that hook. The other thirteen products may therefore report a successful preference round trip while keeping fixed, driver-selected, or nonexistent native attachments.

Fullscreen is a separate layer from those attachments. An SDL-backed product can change its real window before its empty renderer hook is reached. A failed SDL fullscreen request, however, is cleared and treated as non-fatal. Thus a true public fullscreen property is requested state, not proof that the window manager or browser accepted the transition. D3D11 and D3D12 add another distinction: SDL may enlarge their window, but CNA never asks DXGI for exclusive mode. The complete color/depth/window matrix is §19.2.5.

6.8 Supporting cast: LaunchParameters, TitleContainer, TitleLocation, FrameworkDispatcher

Four smaller types round out the framework layer. `LaunchParameters` parses process `argv` into key/value pairs and — a CNA-specific representational choice — inherits publicly from `std::unordered_map<std::string, std::string>` rather than wrapping one, so a `LaunchParameters` instance *is* a standard associative container as well as an XNA-shaped one. `TitleContainer` is a static-only class providing `OpenStream(path) -> std::unique_ptr<System::IO::Stream>`, the entry point for reading title content files portably. `TitleLocation` resolves and caches the base path content is resolved relative to, exposing it under two accessor names — both `getPathProperty()` and a bare `Path()` return the same value, the latter kept specifically “to match XNA property name” alongside CNA’s own `getXProperty` convention. They are an independent direct-file facility, not the stream layer below `ContentManager`; their exact path, lifetime, Android, and test contract is §33.6.1. `FrameworkDispatcher`, also static-only, drives dynamic audio streams, microphones, media, and touch through one `Update()` entry point. The base `Game::Update()` calls it after components on every update step. Its dynamic-stream list is protected by a mutex, but callbacks execute after a snapshot has been copied and the lock released: a `BufferNeeded` handler may dispose its own stream without deadlocking on the same non-recursive mutex. This locking is a CNA thread-safety addition beyond FNA’s ordinary single-threaded assumption.

6.9 One catch clause for the framework boundary

The framework core does not translate failures into one XNA exception hierarchy. The pinned `modules/runtime` implementation has 30 explicit throw sites and all use raw standard C++ types: `std::out_of_range`, `std::runtime_error`, `std::invalid_argument`, `std::logic_error`, or `std::bad_alloc`. Elsewhere in CNA, some public exceptions derive from `std::runtime_error`, while audio, network, sensor, and project exceptions derive from `System::Exception`.

The hierarchy detail that matters at a porting boundary is that `System::Exception` : `std::exception`, not `std::runtime_error`. Consequently a `catch (const std::runtime_error&)` misses the `System::*` family, and a `catch (const System::Exception&)` misses raw standard failures. Only `catch (const std::exception&)` spans both families. Catch narrowly when recovery depends on the exact contract; use `std::exception` for a top-level log-and-exit boundary.

6.10 What this buys a porting engineer

None of the types in this chapter are exotic on their own. What matters, if you are porting an existing XNA or FNA game (this book’s migration chapter returns to this in practical detail), is that the entire timing/component/service/window layer a real XNA game’s **Game**-derived class depends on is present, member-for-member, under the same names. A **Game** subclass that only used the public, documented XNA surface — not FNA’s own internal/assembly-private details — should port to CNA by translating syntax, not by redesigning its structure.

Chapter 7

Math Types and Coordinate Conventions

CNA’s math module carries the values that connect game code, content, effects, and renderers. Its names follow XNA, but correct use depends on conventions that method names do not reveal: row-vector matrix composition, a right-handed world basis, packed-color layout, interval boundaries, and several incomplete geometric paths. This chapter explains those contracts. Appendix A is the compact API inventory; the module headers remain the signature authority.

7.1 Mental model

Except for `BoundingBox`, the core types are C++ value types with public components or fields. Copying a `Vector3`, `Matrix`, `Rectangle`, or `Color` copies its value; no framework identity or heap ownership is involved. `BoundingBox` instead owns derived planes and corners behind its matrix property, so it behaves more like a small object than a plain aggregate.

Most arithmetic families provide two spellings:

- a value-returning form, such as `Vector3::Normalize(value)`;
- an output-reference form with a trailing `result&`, mirroring XNA’s `ref/out` overloads.

They are separate public declarations. Do not assume the output-reference form is alias-safe: some implementations write fields while still reading the input. The live `Plane::Transform` defect later in this chapter is the consequential example.

7.2 Common value surface

The vectors, matrices, quaternions, planes, rays, integer geometry, bounding volumes, and `CurveKey` expose equality and hashing; most also expose `ToString`. Hash return types are not uniform: the port uses `int`, `intcs`, or `std::size_t` according to the type. Code that abstracts over these values should use `auto` or its own normalized hash type.

Several types retain private debugger-display helpers, and the floating-point vectors, `Matrix`, and `Quaternion` define `CheckForNaNs`. No external call site or debugger visualizer was located at the pin. Vector checks are compiled unconditionally; matrix and quaternion checks disappear under `NDEBUG`. This difference has no current runtime effect because none of the helpers is called.

7.3 Coordinate and composition contract

Four rules carry into every graphics chapter:

1. `Vector3::Forward` is $(0, 0, -1)$ and `Vector3::Backward` is $(0, 0, 1)$: CNA follows XNA's right-handed world basis.
2. CNA uses row-vector transforms. A position is conceptually multiplied on the left: $p' = pM$.
3. Matrix products read in application order. With row vectors, `rotation * translation` rotates first and translates second.
4. Perspective clip space needs four components. Preserve clip-space W until clipping and the perspective divide have completed.

This explains the standard world/view/projection product:

$$p_{\text{clip}} = p_{\text{object}} M_{\text{world}} M_{\text{view}} M_{\text{projection}}.$$

The public depth convention is $[0, 1]$. A renderer may translate that convention for its native API, but application matrices and effects remain XNA-shaped.

7.4 Vectors

`Vector2`, `Vector3`, and `Vector4` provide public components, zero/one and unit constants, arithmetic, dot products, distances, normalization, reflection, clamping, and the XNA interpolation family. `Vector3` adds directions and `Cross`; `Vector4` adds lower-dimensional transform inputs. Operations such as `Lerp` are not clamped, so an amount outside $[0, 1]$ extrapolates.

7.4.1 Operation groups

Group	Public families and boundary
Arithmetic	Add, Subtract, component/scalar Multiply and Divide, Negate, operators, Min, Max, and Clamp.
Measurement	Length, LengthSquared, Distance, DistanceSquared, and Dot; <code>Vector3</code> adds <code>Cross</code> .
Interpolation	Lerp, SmoothStep, Barycentric, CatmullRom, and Hermite. Amounts are caller-controlled.
Transforms	Matrix and quaternion Transform, plus array/range forms. <code>TransformNormal</code> omits translation and exists for <code>Vector2/Vector3</code> .

Normalizing a zero vector divides by zero; CNA does not substitute a fallback direction. Validate input when non-finite output would escape into gameplay or rendering.

```

1 // Illustrative: move toward a target without overshooting it.
2 Vector2 toTarget = target - position;
3 const float distance = toTarget.Length();
4 if (distance > 0.0001f)
5 {
6     toTarget.Normalize();
7     position = position + toTarget *
8         std::min(speed * deltaSeconds, distance);
9 }
```

Cross-product order sets the normal's sign. For a triangle (a, b, c) , `Cross(b - a, c - a)` reverses if b and c are exchanged; the same winding choice later controls face culling.

7.4.2 Why Vector4 matters

`Vector3::Transform(position, projection)` returns three components and therefore cannot carry the projected W needed by a software rasterizer or a manual clipping stage. `Vector4` accepts a `Vector3` input directly:

```
1  const Matrix combined = world * view * projection;
2  const Vector4 clip = Vector4::Transform(position, combined);
3  if (clip.W != 0.0f)
4  {
5      const Vector3 ndc(
6          clip.X / clip.W, clip.Y / clip.W, clip.Z / clip.W);
7  }
```

The Software renderer uses this four-component route before clipping and division. The snippet is illustrative; production code must also handle the clip volume and the sign and magnitude of W .

7.5 Matrix

`Matrix` stores M11 through M44. Translation occupies the fourth row, and direction properties expose the first three rows according to the right-handed basis. The principal factory groups are translation, scale, rotations, quaternion conversion, look-at/world, orthographic and perspective projections, billboards, planar shadow, and reflection.

7.5.1 Factory and arithmetic groups

Purpose	Representative methods
Object transform	CreateTranslation, CreateScale, CreateRotationX/Y/Z, CreateFromAxisAngle, CreateFromQuaternion, CreateFromYawPitchRoll.
Camera/projection	CreateLookAt, CreateWorld, CreateOrthographic, CreateOrthographicOffCenter, CreatePerspective*.
Special geometry	CreateBillboard, CreateConstrainedBillboard, CreateShadow, CreateReflection.
Algebra	Add, Subtract, matrix/scalar Multiply and Divide, Negate, Invert, Transpose, and component-wise Lerp.

`Matrix::Lerp` interpolates all sixteen fields. It is not a substitute for decomposing a transform and interpolating its rotation with `Quaternion::Slerp`. Likewise, `Invert` does not report a singular input; callers that may receive degenerate transforms need an independent validity check.

`Decompose(scale, rotation, translation)` returns `false` if any recovered axis scale is near zero and sets rotation to identity. Scale and translation remain useful, but no unique rotation can be extracted from the degenerate matrix.

7.5.2 The ToColumnMajor bridge

`CNAEXT Matrix::ToColumnMajor(float[16])` copies the sixteen fields in declaration order. It does not numerically transpose the matrix. The bridge relies on the receiving shader API interpreting those bytes as column-major storage: GLSL column i corresponds to HLSL row i . The dedicated identity test cannot distinguish a copy from a transpose, so renderer code and non-identity tests remain the evidence for this convention.

7.6 Quaternion

Quaternion provides axis-angle, rotation-matrix, and yaw/pitch/roll construction; normalization, conjugate and inverse; arithmetic; `Lerp`; and `Slerp`.

7.6.1 Composition and interpolation

Quaternion multiplication is easy to read backwards: `q1 * q2` represents `q2`'s rotation followed by `q1`'s. The explicitly named `Quaternion::Concatenate(a, b)` instead promises *a* followed by *b*. Use the named form when operation order should be obvious at the call site.

`Lerp` chooses the short arc, linearly mixes components, and normalizes the result. `Slerp` follows the sphere and gives uniform angular progress. For small turns the difference is usually negligible; a long slow rotation can expose `Lerp`'s ease-in/ease-out rate.

```
1 Quaternion facing = Quaternion::CreateFromYawPitchRoll(
2     0.0f, 0.0f, 0.0f);
3 const Quaternion target = Quaternion::CreateFromYawPitchRoll(
4     MathHelper::PiOver2, 0.0f, 0.0f);
5
6 facing = Quaternion::Slerp(
7     facing, target, std::min(1.0f, turnSpeed * deltaSeconds));
8 const Matrix orientation = Matrix::CreateFromQuaternion(facing);
```

The retained executable `tools/cna-screenshot-infra/quaternion_vs_matrix_demo.cpp` compares a Z-axis matrix rotation with the corresponding axis-angle quaternion route. At the pinned build used for the artifact, the matrix components, transformed point, and rendered image agreed.

7.7 Planes, rays, and bounding volumes

`Plane` stores a normal and distance; `DotCoordinate` includes the distance term and classifies a point's side, whereas `DotNormal` tests a direction only. `Ray` stores an origin and direction and returns `std::optional<float>` for intersection distance. `BoundingBox`, `BoundingSphere`, and `BoundingFrustum` expose containment and intersection families.

7.7.1 Operational summary

Type	Useful operations	Boundary
Plane	point/direction dot tests, normalization, volume intersection, matrix or quaternion transform	Matrix transform is defective at the pin; see below.
Ray	box, sphere, plane, and frustum intersection	Tolerances differ by shape; a zero direction is not a useful point query.
BoundingBox	create from points/sphere, merge, corners, containment/intersection	Touching and empty-input behavior follows individual implementations and exception types.
BoundingSphere	create from points/box, merge, transform, containment/intersection	A non-uniform transform remains a conservative sphere, not an ellipsoid.
BoundingFrustum	matrix-derived planes/corners, containment/intersection	Ray intersection currently checks origin containment rather than computing entry distance.

7.7.2 Current geometry defects

Limitation. Plane matrix transform. The intended inverse-transpose path calls `Matrix::Transpose(transformedMatrix, transformedMatrix)`. That output-reference overload is not alias-safe: it overwrites fields that later reads still need. Identity and diagonal scale hide the error; a translation or other nonsymmetric inverse exposes it. Avoid `Plane::Transform(plane, matrix)` for nontrivial matrices at 1bb2145d. The quaternion overload does not use this path.

Limitation. Frustum/ray intersection. The pinned implementation classifies only the ray origin. An origin outside the frustum returns `nullopt`, even when its direction enters the volume; an origin inside returns zero. The remaining `Intersects` branch throws `NotImplementedException`. This is not a general ray/frustum intersection algorithm.

Ray/box, ray/plane, and ray/sphere paths also use different parallel and behind-origin tolerances. A zero-direction ray inside a box currently returns `nullopt`; validate and normalize picking directions before querying several shapes.

```

1 // Illustrative: choose the nearest successfully reported hit.
2 const SceneObject* nearest = nullptr;
3 float nearestDistance = std::numeric_limits<float>::max();
4 for (const SceneObject& candidate : candidates)
5 {
6     if (const auto hit = pickRay.Intersects(candidate.GetBoundingBox());
7         hit && *hit < nearestDistance)
8     {
9         nearestDistance = *hit;
10        nearest = &candidate;
11    }
12 }
```

7.7.3 Transforming a bounding sphere

`BoundingSphere::Transform` transforms the center and scales the radius by the largest length of the matrix's three basis rows. Under a non-uniform scale (2,3,4), a radius of 2 becomes 8. The result encloses the transformed ellipsoid, which is safe for broad-phase culling but looser than an ellipsoid or oriented bound.

7.8 Integer geometry and Color

`Point` is the integer coordinate pair. `Rectangle` adds bounds, containment, intersection/union, offset, and inflation. Rectangle right and bottom edges are exclusive for point containment.

`Rectangle::IsEmpty` means exactly `X == Y == Width == Height == 0`. A zero-area rectangle at another position is not empty by that property. Use `Width > 0 && Height > 0` when the question is whether a region has area.

`Color` stores byte channels and a packed `UInt32`. Its numeric packed form is `AABBGRR`; on little-endian systems the four packed bytes appear as `RGBA`. That statement does not describe the layout of the polymorphic C++ object itself, so renderer uploads must use the supported conversion path instead of reinterpreting an array of `Color`.

`Color::FromNonPremultiplied` scales straight RGB channels by alpha for CNA's default premultiplied-alpha blend path:

```
1  const Color straight(200, 80, 40, 128);
2  const Color premultiplied =
3      Color::FromNonPremultiplied(200, 80, 40, 128);
```

The type can hold either convention; the selected `BlendState` determines how the channels are interpreted. Named color constants are generated API surface, not evidence about blend behavior.

7.9 MathHelper and degenerate inputs

`MathHelper` collects angle constants, unit conversion, clamping, interpolation, distance, wrapping, and epsilon comparison. `Lerp` is unclamped; `SmoothStep` clamps its amount before applying the cubic curve. `WrapAngle` maps to a principal range around zero.

Most lower-level math routines follow floating-point arithmetic instead of throwing on degenerate inputs. Zero-vector normalization, singular inversion, coincident look-at inputs, and invalid projection ranges can therefore yield infinities or NaNs. This policy is not uniform: some factory methods validate selected range conditions while adjacent operations do not. Validate at the boundary where bad values can affect gameplay, content, or a native API.

7.10 Curves

`Curve` evaluates an ordered `CurveKeyCollection`. Keys contain position, value, incoming/outgoing tangents, and smooth/step continuity. `Add` inserts by ascending position. `ComputeTangents` supports flat, linear, and smooth modes. Before and after the key span, `Constant`, `Linear`, `Cycle`, `CycleOffset`, and `Oscillate` choose clamping, extrapolation, wrapping, accumulated wrapping, or ping-pong behavior.

```
1  Curve height;
2  height.getKeysProperty().Add(CurveKey(0.0f, 10.0f));
3  height.getKeysProperty().Add(CurveKey(2.0f, 40.0f));
4  height.getKeysProperty().Add(CurveKey(4.0f, 10.0f));
5  height.ComputeTangents(CurveTangent::Smooth);
6  height.setPreLoopProperty(CurveLoopType::Constant);
7  height.setPostLoopProperty(CurveLoopType::Constant);
8
9  const float cameraHeight = height.Evaluate(elapsedSeconds);
```

This compiles against the public shape when the math headers and namespace aliases used by the application are present; it is an illustrative fragment rather than a complete program.

7.10.1 Current curve deviations

Three source-level findings matter for nontrivial key positions:

- Step evaluation compares the absolute query position with `1.0f`, not with the next key's position. Step intervals away from the canonical `[0, 1]` span can switch to the next value too early or too late.
- Post-loop linear extrapolation uses the *first* key's outgoing tangent while extending from the last key.

- Smooth tangent computation uses materially different near-zero thresholds for incoming and outgoing spans, so duplicate or nearly duplicate positions can produce asymmetric tangents.

Both XNB and CNJ readers can construct curves. Their file-format and resolution contracts live in Chapters 35 and 36; successful in-memory evaluation does not establish either loading route.

7.11 Compiled world/view/projection example

The retained source `tools/cna-screenshot-infra/math_rotation_demo.cpp` constructs a complete game, builds a world/view/projection chain, applies `BasicEffect`, and draws a colored triangle. Its central sequence is:

```

1  const float angle = MathHelper::ToRadians(30.0f);
2  const Matrix world = Matrix::CreateRotationZ(angle) *
3      Matrix::CreateTranslation(Vector3::Zero);
4  const Matrix view = Matrix::CreateLookAt(
5      Vector3(0.0f, 0.0f, 3.0f), Vector3::Zero, Vector3::Up);
6  const Matrix projection = Matrix::CreatePerspectiveFieldOfView(
7      MathHelper::PiOver4, 1.0f, 0.1f, 100.0f);
8
9  BasicEffect fx(dev);
10 fx.VertexColorEnabled = true;
11 fx.World = world;
12 fx.View = view;
13 fx.Projection = projection;
14 fx.Apply();

```

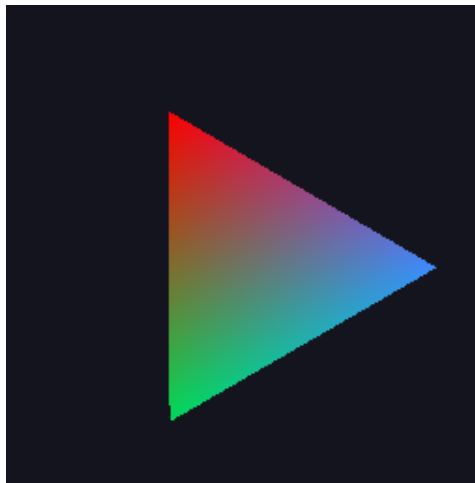


Figure 7.1: Software-renderer output from the retained matrix demo. The image verifies the composed rotation, view, projection, effect application, draw, and pixel path on the pinned configuration.

Evidence. The image is a pixel artifact for one complete path, not a conformance oracle for every math operation. The math test suite supplies analytic expectations at several tolerances, but this edition located no captured XNA numeric corpus covering composition order, singular inputs, corner ordering, or non-identity plane transforms. Porting tests should compare the original program’s observable results: projected positions, collision

classifications, curve samples, or captured reference values.

7.12 Summary

CNA's math API is broadly XNA-shaped, but its useful contract is the combination of value semantics, row-vector composition, right-handed directions, and operation-specific edge cases. Keep clip-space W , do not reinterpret polymorphic values as packed GPU bytes, validate degenerate inputs, and avoid the current plane-matrix and frustum-ray paths where their missing behavior matters.

Chapter 8

Geometry, Color, Curves, and C++ Value Layout

Chapter 7 gives the method-level reference for CNA’s math types. This chapter takes the complementary view: which geometric conventions cross subsystem boundaries, which C++ object layouts differ from XNA value types, and which apparently harmless casts or degeneracies become observable defects.

8.1 The value-type resemblance has a hard limit

XNA presents vectors, colours, planes, bounds, and curve keys as compact managed value types. CNA preserves their names and most operations, but it cannot preserve that representation merely by spelling the same fields in C++. Several public types inherit polymorphic interfaces:

Type family	Why it is polymorphic	Practical consequence
Color	Implements <code>IPackedVectorT<UInt32></code> , whose base has virtual operations and a virtual destructor.	The object has a <code>vptr</code> ; it is not four raw RGBA bytes.
Built-in vertex types BoundingBox, BoundingSphere, BoundingFrustum, CurveKey	Implement <code>IVertexType</code> . Implement <code>IEquatable<T></code> .	A public vertex object cannot be uploaded by copying <code>sizeof(T)</code> bytes. Their field list is not their complete object representation.
Plain math types	<code>Vector2</code> , <code>Vector3</code> , <code>Vector4</code> , <code>Matrix</code> , <code>Quaternion</code> , <code>Plane</code> , <code>Ray</code> , <code>Point</code> , and <code>Rectangle</code> have no base class.	Their layout is simpler, but padding and C++ object rules still belong to the ABI, not to the XNA contract.

At the pinned revision, `sizeof(Color) == 24`. The single packed value is still `AABBGRRR` on a little-endian machine, yet the object begins with a `vptr`. Reinterpreting a `Color*` as pixel bytes would overwrite or read the virtual-table pointer. CNA’s own `GetBackBufferData<Color>` path therefore reads native bytes into a temporary byte vector and constructs each result as `Color(r,g,b,a)`. `Texture3D` and `TextureCube` use the same conversion rule.

This is not theoretical portability advice. A regression test explicitly rejects the old raw cast, and the graphics module documents the exact 24-byte size where it converts texture data. Application code should use component access, `PackedValue`, or the typed texture APIs; it should never serialize a public math object with `memcpy(sizeof(T))` unless the file format is deliberately CNA-ABI-specific.

8.2 GPU streams are a second, plain-data representation

The graphics core solves the polymorphic-vertex problem with a parallel internal stream layer in `modules/graphics/include/CNA/Internal/Graphics/BuiltInVertexStreams.hpp`. These structs have no virtual base, and both their total sizes and member offsets are compile-time assertions:

Stream representation	Bytes
<code>PositionColorStream</code>	16
<code>PositionTextureStream</code>	20
<code>PositionColorTextureStream</code>	24
<code>PositionNormalTextureStream</code>	32
<code>PositionNormalTangentTextureStream</code>	48
<code>PositionNormalTextureSkinnedStream</code>	52
<code>PositionNormalTangentTextureSkinnedStream</code>	68

Every built-in `VertexDeclaration` takes its stride from the matching stream struct, not from the public vertex class. Conversion is therefore an explicit semantic operation: positions and texture coordinates are copied as floats, while `Color` contributes only its packed four-byte payload. Chapter 18 follows these bytes through binding, multi-stream normalization, and renderer capability gates.

8.3 Planes and half-spaces

CNA uses the plane equation

$$\mathbf{n} \cdot \mathbf{x} + D = 0.$$

The three-point constructor is the only constructor that normalizes automatically and computes $D = -\mathbf{n} \cdot \mathbf{a}$. `DotCoordinate` returns the signed expression above. Operationally, `PlaneIntersectionType::Front` is the positive-normal side and `Back` is the negative side; the enum's own prose comments reverse those meanings, while `Plane`, `BoundingBox`, and `BoundingSphere` consistently implement the positive-side interpretation.

8.3.1 The inverse-transpose path is presently wrong

Transforming a plane should multiply it by the inverse transpose of the point transform. CNA's `Plane::Transform` intends to do that, but calls the out-parameter transpose as `Matrix::Transpose(m, m)`. That overload writes fields while it is still reading the same object, so it is not alias-safe. For a non-symmetric inverse it progressively mixes old and new fields and produces a symmetrization rather than a transpose.

A minimal counterexample is the plane $y = 1$ transformed by a pure translation along X. The correct plane is unchanged. The aliased implementation instead produces a normal proportional to $(1, 1, 0)$, silently rotating it. Both existing transform tests use the identity matrix, whose inverse is symmetric, so they cannot expose the error. This is recorded as CNA-BUG-001; portable application code should avoid `Plane::Transform` for nontrivial matrices until the source uses a separate transpose destination.

8.4 Bounds encode several non-obvious conventions

`BoundingFrustum` independently confirms CNA's Direct3D clip volume. Its extracted planes describe $-w \leq x, y \leq w$ and $0 \leq z \leq w$, and their normals point outward. Consequently a point on the positive side of any plane is outside. This agrees with the projection builders in Chapter 7; it is a second derivation, not a repeated assertion.

The three bounding types do not share one quality level:

- `BoundingBox::GetCorners` preserves XNA's fixed eight-corner order. Its containment tests use half-space and min/max logic, but `Contains(BoundingFrustum)` retains legacy reversed-question behavior.
- `BoundingSphere` builds point sets with `CreateFromPoints`, using a Ritter-style expansion. Under a non-uniform transform, the radius is multiplied by the largest row-vector scale. Its frustum `Contains` path never returns `Disjoint` because the required minimum-distance accumulator is never populated.
- `BoundingFrustum` caches six planes and eight corners. Its `Intersects(Ray)` main case throws `NotImplementedException`; source and test absence must not be mistaken for a completed geometric query.

Those gaps contain no `TODO` marker. They are a useful reminder that marker searches find intent labels, not semantic incompleteness.

8.5 Rectangle, Point, and edge ownership

`Rectangle::Contains` is half-open: the left and top edges belong to the rectangle, while the right and bottom edges do not. `Intersects` requires positive overlap, so rectangles that merely touch at an edge are not intersecting. `Center` uses integer arithmetic, and `IsEmpty` is true only when all four fields are zero; a zero-width rectangle at a nonzero position is not the singleton `Empty` value.

`Point` is simply two 32-bit integers. It deliberately has no implicit `Vector2` bridge. Make rounding policy visible when converting a continuous coordinate to a pixel or cell: truncation, floor, and nearest are different behaviors for negative values.

8.6 Color is packed, named, and polymorphic

The public surface contains 141 named colours: 140 opaque values plus `Transparent`. There are 139 distinct packed values because `Aqua/Cyan` and `Fuchsia/Magenta` are aliases. Three counts can therefore all appear plausible; only a count that names declarations, opacity, or distinct packed values is meaningful.

Construction from float vectors clamps components to $[0, 1]$, scales to $[0, 255]$, and packs the four bytes. Integer constructors clamp to $[0, 255]$. Multiplication scales alpha as well as RGB. These behavioral facts are independent of the 24-byte C++ object layout: packing defines the value, not permission to treat the entire object as packed storage.

8.7 Curves: ordered keys and explicit extrapolation

`Curve` evaluates cubic Hermite segments between sorted `CurveKey` entries. Duplicate positions are allowed; insertion and `setItemProperty` preserve sorted order and can reposition a key.

`CurveContinuity::Step` selects the left value until the segment endpoint, while smooth keys use their incoming and outgoing tangents.

Five loop modes govern positions outside the key range. `Constant` and `Linear` hold or extrapolate an endpoint; `Cycle`, `CycleOffset`, and `Oscillate` repeat a span. The repeating modes depend on a stable cycle count and on a nonzero range. CNA guards some degenerate spans, but its tangent computation uses different near-zero tests for incoming and outgoing sides. Treat a duplicate-position curve as an authored edge case and test it with the same evaluator that will ship.

Curves also cross the content boundary. CNJ can carry a self-contained Curve payload, and the XNB reader has a concrete `CurveReader`; a successful parse proves field recovery, while the runtime evaluator and loop behavior remain separate claims.

8.8 Exceptions are part of the port

Sibling operations do not consistently choose the same exception family. With empty input, `CreateFromPoints` throws different exceptions. `BoundingBox` uses `ArgumentException`; `BoundingSphere` uses the standard invalid-argument exception.

An undersized corner destination throws `ArgumentOutOfRangeException` for the box and `std::out_of_range` for the frustum. Existing tests encode these differences, so a catch block written for one type is not a generic geometry policy.

Degenerate numeric inputs are usually quieter: normalizing a zero vector or quaternion and inverting a singular matrix can yield NaN or infinity. The module's strong argument checks are concentrated in projection factories. Validate user-authored axes, scales, planes, and key spans at the application boundary rather than expecting every math primitive to reject them.

8.9 What the math tests prove

The pinned math module contains 818 statically counted GoogleTest definitions. They use `EXPECT_FLOAT_EQ` or explicit tolerances of 10^{-4} , 10^{-5} , and 10^{-6} ; none uses captured XNA or MonoGame numeric oracle values. The suite is strong on API wiring and many analytic identities, but it does not independently pin matrix/quaternion composition order, the $[0, 1]$ depth mapping, corner order, singular inputs, or the non-identity plane transform.

For a port, pair these types with tests derived from the original game's observable behavior: screen-space points, collision classifications, curve samples, or captured reference values. A familiar type name establishes lineage. It does not erase C++ layout, exception, and evidence boundaries.

Chapter 9

The Module Monorepo

CNA is one repository, but it is not one library with global `src/` and `include/` trees. Its physical ownership unit is the module. This matters to an application author because include spelling, link closure, optional features, and even which tests compile are all derived from those module boundaries.

9.1 Location is ownership

The normal framework layout is:

```
modules/<owner>/
  CMakeLists.txt
  include/
  src/
  tests/
  examples/
```

The repository root deliberately has no legacy production `src/` or `include/` directory. A configure-time guard fails if either returns. A second source-partition validator recursively inventories `*.cpp` and Objective-C++ `*.mm` files and accepts a translation unit only below a declared framework module or renderer family's `src`, `tests`, or `examples` area.

The rule is stronger than tidiness. Moving an implementation file changes the component that owns its symbols, dependency declarations, test registration, and include reachability. The validator turns a physical move into an architectural change that must be reconciled explicitly.

9.2 Fifteen framework modules and one optional ABI module

The declared framework set contains fifteen physical modules. Their responsibilities can be read in four clusters:

Cluster	Modules and ownership
Foundation and host	<code>core</code> owns CNA project vocabulary and leaf infrastructure; <code>math</code> owns the <code>Microsoft::Xna::Framework</code> value and geometry types; and <code>platform</code> owns <code>CNA::Platform::IPlatform</code> and the selected implementation.
Runtime and graphics	<code>runtime</code> , <code>graphics</code> , <code>input</code> , <code>audio</code> , and <code>media</code> own the game loop, renderer-independent graphics surface, device state, mixer, and media playback.
Assets and persistence	<code>content</code> owns native, CNJ, XNB, and glTF loading; <code>storage</code> owns <code>Microsoft::Xna::Framework::Storage</code> .
Optional/service surface	<code>devices</code> , <code>devices-ext</code> , <code>graphics-ext</code> , <code>gamer-services</code> , and <code>net</code> separate faithful APIs, project extensions, local services, and ENet-backed networking.

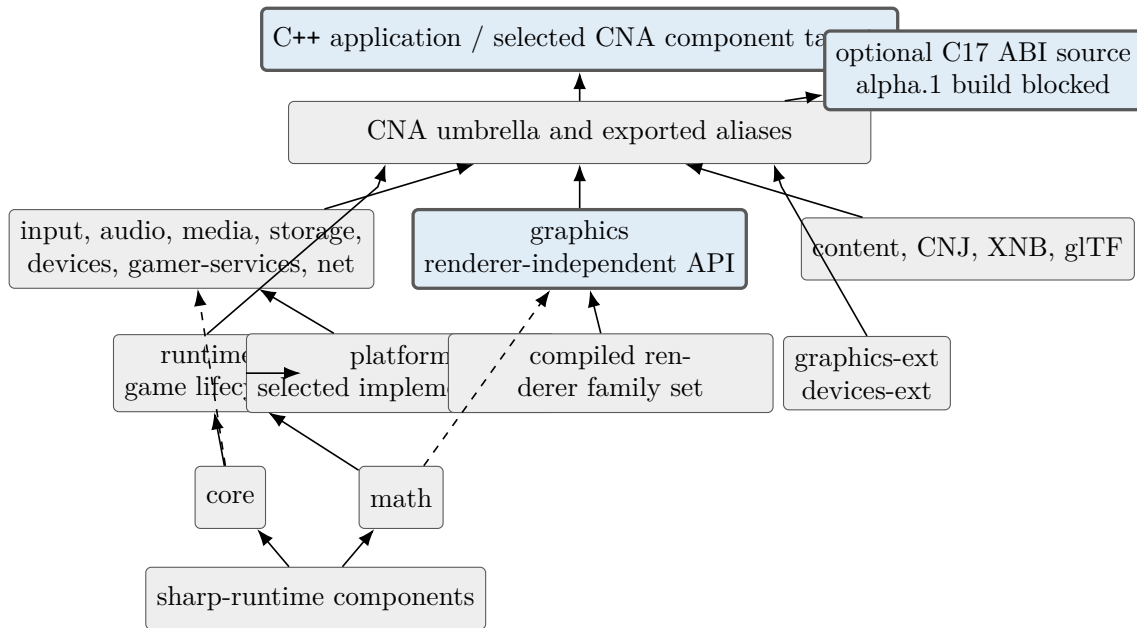


Figure 9.1: Physical ownership layers in CNA’s module tree. The arrows show the architectural direction of dependency, not every transitive CMake edge; each module’s declared link interface remains authoritative. The C API node records source ownership, not a successful alpha.1 artifact.

The concrete aliases are correspondingly narrow. Foundation and runtime code use `CNA::Core`, `CNA::Math`, and `CNA::Runtime`. Framework services use `CNA::GraphicsCore`, `CNA::Input`, `CNA::Audio`, `CNA::Media`, `CNA::Content`, and `CNA::Storage`. Device and extension code uses `CNA::Devices`, `CNA::DevicesExt`, and `CNA::GraphicsExt`. Finally, `CNA::GamerServices` and `CNA::Net` are added only when networking is enabled. Appendix D maps every module to its pinned role without turning that inventory into an inferred dependency graph.

With `CNA_BUILD_C_API=ON`, `modules/c-api` defines a separate consumer surface as the shared `cna_c_api` / `CNA::CApi` target. It wraps the canonical C++ modules and exports C17 headers; it is not another framework implementation. The alpha.1 target is present in the module graph but compile-blocked by its stale renderer-identity map, as Appendix I demonstrates.

9.3 The module factory and one public include root

Most framework modules are created by a small root-level CMake function:

```

1 function(cna_add_module target alias)
2     add_library(${target} STATIC ${ARGN})
3     add_library(CNA::${alias} ALIAS ${target})
4     target_link_libraries(${target} PUBLIC cna_build_flags)
5     target_include_directories(${target} PUBLIC
6         ${CMAKE_CURRENT_SOURCE_DIR}/include)
7 endfunction()

```

Every module exports exactly its own `include/` directory. No module exports `src/`. A header required across modules must therefore live under a public include root, usually beneath `CNA/Internal/` when it is an engine seam rather than application API. Renderer-local shader headers are the narrow exception: they remain in their own source directory and are included only relative to files in that same renderer.

Consumer spelling remains stable because the module root is removed by the include path. A game still writes, for example:

```
1 #include <Microsoft/Xna/Framework/Game.hpp>
2 #include <Microsoft/Xna/Framework/Graphics/Texture2D.hpp>
3 #include <CNA/GraphicsRendererType.hpp>
```

It should not embed `modules/runtime/include/` or any other repository-relative prefix. That physical path identifies the owner for auditors; it is not the public include spelling.

9.4 Two deliberate construction exceptions

`gamer-services` and `net` hand-build targets named `CNA_GamerServices` and `CNA_Net` instead of going through `cna_add_module()`. Their aliases remain regular, but the concrete target naming differs from the twelve `cna_<name>` targets. Build tooling that guesses concrete names from directory names will therefore be wrong; use exported aliases or read the module's own CMake file.

The second exception is headers-only access to the core include tree. `cna_core_headers` / `CNA::CoreHeaders` exposes leaf declarations without linking `cna_core`. Math and Storage use it for items such as the CNAEXT marker, `PlayerIndex`, and path-containment helpers. This keeps their link closures small while preserving the same include spelling.

9.5 Renderer families are physical modules too

`modules/renderers/` declares 46 implementation families plus shared infrastructure. Only the selected family is normally entered. In an opt-in multi-renderer build, each distinct family required by `CNA_GRAPHICS_RENDERERS` is entered and receives a concrete target named `cna_renderer_<family>`. One family may carry more than one public identity: EasyGL owns five GL-profile identities, which is why the public registry contains 50 identities but the source partition contains 46 families.

The common setup function gives a renderer its own include root, build flags, sharp-runtime closure, and reverse links to `graphics/core/math`. The renderer's source glob is deliberately non-recursive. A nested translation unit does not silently join the target merely because it was created on disk.

There are explicit composition exceptions:

- Metal and Glide expose header-interface targets even when another renderer is selected, because their policy suites compile as part of the shared tests.
- `renderers/common/d3d` is entered only for Direct3D 11 and 12; Direct3D 9 and 10 have independent implementations.
- GDI reuses eight software-renderer 2D translation units and a software header target. Physical ownership remains with `software`; GDI lists the borrowed source set explicitly.

These are declared source relationships, not permission for arbitrary cross-family includes. Chapter 20 explains how the generated registry maps the compiled identity set to these physical targets and latches one active family.

9.6 Examples moved to their owners

Framework examples live beside the subsystem they demonstrate; renderer-specific examples live beside the family they exercise; CNAEXT examples live under `graphics-ext`. The graphics module owns shared renderer-agnostic fixtures such as the 2D demo and common pixel-test helpers. The selected renderer reaches them through `CNA_GRAPHICS_EXAMPLES_DIR`.

The repository-level `examples/` directory now retains only `examples/golden/`, the cross-renderer image-oracle corpus. That exception is intentional: a golden image compared by many families has no single renderer owner. Executables are still emitted at the build root for compatibility, so output location is not evidence of source ownership.

9.7 The migration was checked as a no-loss transformation

The physical-module campaign moved 1,776 files in seven pure-rename commits before rewriting the build. Its reconciliation classified 1,357 production files before and after: 1,287 were byte-identical moves, 70 changed only ownership/build directives, and none disappeared. It also compared CTest names for representative HEADLESS and OPENGLES configurations and recorded no removals or renames.

Two later changes are easy to conflate with that move. Module-local example registration removed the old central example manifests, and renderer terminology later replaced backend-named build files. A path from before those follow-up changes may therefore be historically real yet wrong at the pinned revision.

9.8 How to read a source path in this book

A path such as `modules/content/src/Xna/ContentManager.cpp` carries three claims: Content owns the translation unit, its implementation is private to the module, and consumers reach declarations through Content's public include root. It does not by itself prove which umbrella links the module, which features compiled that source, or which runtime path called it.

For any architecture claim, read four artifacts together:

1. the file's physical module;
2. that module's `CMakeLists.txt`;
3. the root module registry and source-partition rules;
4. a link or execution artifact for the selected configuration.

The next chapter applies that method to cycles, umbrellas, and the gates intended to keep module boundaries honest.

Chapter 10

Cycles, Umbrellas, Boundaries, and Enforcement

A dependency cycle can be an architecture defect, a static-linking fact, or a faithful expression of the framework’s semantics. CNA contains all three kinds of pressure, so a flat graph with every edge removed is not the goal. The useful goal is narrower: declare each required edge, keep public closures no wider than necessary, and make accidental edges fail mechanically.

10.1 Three deliberate archive cycles

CNA declares three cycles on both sides so CMake can repeat static archives on the final link line. Each has a concrete runtime reason:

Cycle	Forward responsibility	Reverse responsibility
Graphics–Input	GraphicsDevice updates TouchPanel display metrics and binds Mouse/TextInput to a window.	MouseCursor is built from a Texture2D .
Audio–Media	FrameworkDispatcher pumps MediaPlayer .	Media playback submits songs through the audio mixer.
Graphics–selected renderer	Graphics constructs and invokes the one selected renderer factory.	Every renderer implements the graphics-owned contract and its header-inlined defaults call back into core logging/validation.

The first two arise from XNA-shaped ownership. Moving **MouseCursor** or the dispatcher could trade the cycle for a new abstraction, but deleting one edge without relocating the semantic owner merely creates unresolved symbols or a missing update path. The renderer cycle is a compile-time composition seam: CNA does not load a renderer plug-in after startup.

When diagnosing a static link failure, inspect the generated final link line before adding a new PUBLIC dependency. Repeating the two intended archives is different from making every module link the full framework umbrella.

10.2 The three umbrella targets

The root module file exposes three different compositions:

CNA The historical full-framework INTERFACE target. It carries runtime, framework modules, optional extension modules present in the build, build flags, and the selected renderer. Existing games can continue to link CNA.

`cna_cnaext / CNA::CnaExt`

The extension composition links `CNA::GraphicsExt` with `CNA::DevicesExt`. It is not the declaration marker and it is not a renderer.

cna_build_flags / CNA::BuildFlags A definitions-only INTERFACE target. It carries the selected `CNA_RENDERER_*` define plus feature markers such as `CNA_CNAEXT`, `CNA_DEVICES`, `Draco`, and `FFmpeg` availability. It exports no include directory.

The last point closes a subtle loophole. Older layouts could obtain declarations accidentally through a global include root attached to common flags. At the pinned revision, an include works only when the declaring module is actually reachable.

10.3 Narrow aliases are the architectural surface

A subsystem test or reusable library should link the narrowest CNA alias whose public surface it uses. That makes an undeclared reach visible. A game executable may reasonably use the CNA umbrella; a math-only helper library should prefer `CNA::Math`, and a content tool should state whether it needs `CNA::Content`, graphics, or the whole runtime.

PUBLIC and PRIVATE still mean CMake propagation, not conceptual importance. Content's public headers expose graphics/audio/media types, so those CNA edges are PUBLIC. Its `cgltf`, `stb`, `SDL`, and optional `Draco` implementation needs are private. A consumer that happens to compile because a private third-party include leaked from its environment has no supported contract.

10.4 Fourteen module probes

On supported native test configurations, CNA registers one small consumer for each framework composition boundary. The probe fleet covers `Math`, `Core`, `Graphics`, `Content`, `Runtime`, `Input`, `Audio`, `Media`, `Storage`, `Devices`, `DevicesExt`, `GraphicsExt`, `CnaExt`, and `Net`. Each probe produces two CTest concepts:

1. `ModuleProbe_<name>` builds/runs a minimal consumer to show that the intended alias is sufficient;
2. `ModuleLinkClosure_<name>` inspects the emitted link line for required and forbidden libraries.

The probes are excluded on Emscripten and Android. That is a host boundary, not evidence that the module graph changes there. A platform claim must say whether it is source-validated, cross-compiled, or link-inspected on the target.

10.4.1 Two high-value negative contracts

`Storage` links only its own implementation plus header-level dependencies. Its closure expression forbids any `libcna_` archive that is not `libcna_storage`; even `libcna_core` must remain absent. `PlayerIndex`, the `CNAEXT` marker, and path containment are obtained through `CNA::CoreHeaders`, not by widening the binary closure.

`DevicesExt` encodes the opposite separation: project-owned `CNA::Devices` services must never depend on the XNA-shaped `Microsoft::Devices` sensor module. Its regex escapes the dot after `libcna_devices` so that it rejects the base archive without accidentally matching the extension archive itself. These negative assertions are more informative than a full umbrella consumer that would succeed regardless of accidental edges.

Additional closure gates require ENet for Net, require both extension libraries for CnaExt, isolate the Vulkan SDK from other renderer SDKs, and prove that HEADLESS configurations remain free of native graphics SDKs.

10.5 The Ninja caveat: registered does not mean executed

The closure checker reads `CMakeFiles/<target>.dir/link.txt`. Makefiles generators emit that file, but Ninja normally does not. When the file is missing the script exits with code 77, which CTest records as skipped. CNA's test preset and hosted workflows use Ninja; therefore the `ModuleLinkClosure_*` entries are registered there but do not inspect a link command.

The minimal consumer can still compile, and other validators can still pass. Neither result closes the negative-link claim. A report should distinguish:

- probe source compiled and ran;
- closure test was registered;
- closure test inspected a real link line under a compatible generator;
- forbidden-library mutation was demonstrated to fail.

This is a concrete example of the evidence vocabulary developed in Appendix G: test presence, execution, and oracle engagement are separate coordinates.

10.6 Include reachability and header self-containment

The modularization tools add two source-oriented views. Include reachability resolves public CNA and Microsoft headers through the declared module graph and attributes renderer-gated includes to the selected family. Header self-containment asks whether a public header can be compiled with only its documented prerequisites rather than relying on include order.

These checks answer different questions from the linker probes. A declaration can be reachable while its implementation library is absent; a link can succeed through an overly broad umbrella while a public header still requires an unrelated include to come first. A healthy boundary needs both compile and link evidence.

10.7 Source partition and legacy-root guards

Chapter 9 introduced the physical partition validator. Its complement is a simple root guard that fails configuration if production `src/` or `include/` directories reappear. Together they prevent two regression shapes:

- a new translation unit lands outside every owner and is picked up by an ad hoc glob;
- a global include root quietly restores reachability that module dependencies were meant to control.

The renderer identity validator adds a different invariant: CMake's accepted strings and the C++ enum must remain the same set. It cannot prove behavior, but it prevents an identity from existing on only one side of the configuration/runtime boundary.

10.8 How to change a boundary safely

A module move or new public dependency is complete only when all affected evidence changes together:

1. move the source/header under its real owner;
2. update the module's PUBLIC/PRIVATE dependency declarations;
3. update include reachability and header-isolation expectations;
4. add or narrow a consumer probe;
5. inspect the real link artifact with a generator the checker understands;
6. run a negative mutation that demonstrates the gate can fail;
7. update examples, tests, and source citations that encoded the old owner.

An umbrella build going green is the beginning of that argument, not its conclusion. Chapters [68](#) and [73](#) show the same distinction at test-suite and hosted-workflow scale.

Chapter 11

Dependencies, Tests, Examples, and Developer Tools

A successful CNA configure can involve checked-out submodules, sibling repositories, system packages, downloaded source, prebuilt binary releases, and renderer-specific SDKs. Treating all of them as “vendored dependencies” hides the most common reproducibility failures. This chapter maps the acquisition mechanisms and the developer surfaces that consume them; Chapter 4 owns the command-line configuration details.

11.1 Five acquisition mechanisms

Mechanism	Representative inputs	Reproducibility boundary
Git submodules	GoogleTest, SDL3, SDL3_image, SDL3_mixer	A source archive may omit them; initialize the recorded submodule commits before configuring.
In-tree sources	cgltf, stb, ENet, and a wgpu-native vendor area	Present in the CNA tree, but build/features still depend on the selected module and host.
Sibling checkouts	sharp-runtime, easy-gl/meta-gl, free-direct/free-api	Separate repositories with their own pins and sometimes fixed sibling-relative paths.
Fetch/download or external artifact	bgfx, Blend2D, Diligent, FNA3D, LLGL, Magnum, OpenVG, PortableGL, sokol, WebGPU, Wicked, Skia	Each renderer has a distinct pin, offline override, patch, or prebuilt-root contract.
System packages	FFmpeg, optional Draco, Vulkan/OpenGL/GLES libraries, Threads	Host package discovery can change which files compile or whether configure is admissible.

The categories can compose. FNA3D is fetched from a pin and carries MojoShader through its own submodules. EasyGL is a sibling checkout that itself expects meta-gl as a sibling. Free-direct is a sibling that PUBLIC-links the second-level free-api sibling. A lockfile for only CNA’s own git tree is therefore not a complete build identity.

11.2 Submodules: initialize, do not guess

CNA’s considered instruction is `git submodule update -init`, not a recursive update. The optional codec trees beneath SDL are disabled by CNA’s vendored-SDL configuration, so recursive initialization adds time without contributing compiled inputs. The configure guards

explain when the four required submodule directories are missing and explicitly note that a plain source archive cannot manufacture their content.

For a distributable source snapshot, either include those exact submodule trees or select a supported system-SDL path and provide GoogleTest another documented way. Silently building against whatever SDL happens to be installed changes the dependency claim.

11.3 The unusual SDL configure-time build

The controlling option is `CNA_USE_SYSTEM_SDL`. With the default vendored path, CMake configures, builds, and installs `SDL3`, `SDL3_image`, and `SDL3_mixer` during CNA's *configure* step. The install prefix is persistent and outside the individual build directory, then consumed through the packages' exported CMake configs.

This has three operational consequences:

- deleting or cleaning a CNA build directory does not necessarily rebuild SDL;
- the prefix is keyed by target system and processor so a cross-build does not overwrite a native installation;
- a configure failure may actually be a nested dependency compile/install failure, not a CNA CMake-language error.

The default parallelism ceiling is intentionally small. Image and mixer codec options are pruned: for example, the vendored `SDL_mixer` build does not enable the optional MP3, Opus, FluidSynth, WavPack, or FLAC dependency families. CNA's higher-level Audio and Media behavior must be read against that concrete mixer build, not against every feature `SDL_mixer` can support in principle.

11.4 System-package gates can change source composition

FFmpeg illustrates why “dependency found” is not merely a link flag. CNA computes the `CNA_FFMPEG_AVAILABLE` flag. On Windows/MinGW, Emscripten, and Android it is forced off so a cross-configure cannot accidentally import the host's native pkg-config headers and libraries. When off, the Media module drops its video decoder/player translation units, Content drops the XNB video reader, and the unit-test source set excludes matching cases.

Draco is optional in a different way. Its absence does not remove the glTF importer; a Draco-compressed primitive reaches a named unsupported-format failure. Vulkan and several GL-based renderers, by contrast, require their native SDK/package at configure time. These are three different states: feature source omitted, format rejected at runtime, and configuration refused.

11.5 Sibling repositories are part of the workspace contract

sharp-runtime is always required. `CNA_SHARP_RUNTIME_ROOT` defaults to `../sharp-runtime`, and CNA emits a fatal diagnostic explaining that it is not a submodule. The consumption adapter detects the current component-target shape and links each CNA module to a declared sharp-runtime component closure; it can also recognize the older monolithic target shape. For this edition, only the revisions in Appendix D are authoritative.

CNA's alpha.1 release verification additionally records sharp-runtime `625476d5b5fff5fa89f392c3c9af8638ff237692`. That is release evidence, not a CMake-enforced dependency lock: `CNA_SHARP_RUNTIME_ROOT`

still accepts the caller’s checkout and CNA does not compare its Git SHA. This Bible retains its deeper sharp-runtime technical pin `f827a6c5` for chapters auditing that sibling; Appendix D keeps the two roles explicit rather than arbitrarily repinning the sibling book scope.

EasyGL’s five public identities expect `../easy-gl`; `easy-gl` then expects `../meta-gl`. CNA exposes no override for those two relative paths. FREEDIRECT similarly expects the free-direct sibling checkout, which resolves `free-api` itself and reuses CNA’s already-created SDL targets. A CI checkout that clones only CNA can therefore configure some renderer identities but not others.

Do not paper over this by copying sibling sources into CNA’s build tree. Their independent commit, tests, public targets, and consumer evidence are part of the claim.

11.6 Renderer dependencies are intentionally asymmetric

Some renderer integrations fetch a pinned source tree automatically; some prefer a supplied root; some require a prebuilt external product. Examples at the pin include:

- WEBGPU auto-downloads a pinned `wgpu-native` binary release unless an explicit root is given;
- WICKED has a pinned commit and patch set but auto-fetch is off by default;
- SKIA is never fetched and requires caller-supplied prebuilt directories;
- MAGNUM first tries system Corrade/Magnum packages, then falls back to pinned source;
- BGFX applies an in-tree patch during its fetched-source preparation;
- OPENVG and several Sokol modes still require a real OpenGL system package.

Accordingly, “all renderer identities accepted by CMake” is not “all renderer dependencies are available on this host.” Chapter 20 separates identity, family, internal native sub-selector, and platform admission.

11.7 Build-tree C++ consumption and the native C package

The C++ framework remains primarily a build-tree/subdirectory product: a parent project adds the checkout or follows a sibling template, and the module targets compose into its final consumer. Alpha.1’s optional native C API source defines a deliberately narrower install surface. Its CMake rules would install the shared `CNA::CApi` target, C headers, `CNAConfig.cmake`, and a package-version file; suitable Unix builds also define `CNA::CApiStatic`. The external `hello_cna.c` gate is designed to configure through `find_package(CNA CONFIG)`, link the installed target, and run it. These are source-proven packaging routes, not a successful alpha.1 artifact: the exact tag cannot compile `cna_c_api` because its C renderer table omits NanoVG. Appendix I records the reproduced failure.

The intended package’s version is the native C ABI version, not the CNA product SemVer. It does not turn all C++ module archives into a general installed C++ SDK, nor does it erase the sibling-resolution contract of source-tree builds.

Likewise, CNA is a CMake target, while `CNA::Math` and other namespaced aliases are module targets. Neither spelling implies a file named `libCNA.so`; most framework pieces are static archives composed into the final consumer.

11.8 The CnaTests corpus

The main GoogleTest executable is assembled from module-owned test sources plus the selected renderer configuration. Feature gates, platform exclusions, renderer selection, and optional FFmpeg/Net inputs change its compiled definition population. There is no configuration-free “number of CNA tests.” Chapter 68 explains composition, and Chapter 69 derives the pinned static populations without exchanging files, macros, CTest registrations, and executed cases.

The native `tests` preset deliberately says to run `CnaTests` directly. CTest’s GoogleTest discovery can launch each case as a short-lived process, while this repository includes display-dependent binaries and fixtures sharing fixed temporary paths. CTest remains useful for named standalone gates and renderer executables; it is not automatically the authoritative runner for the monolithic corpus.

11.9 Examples are executable documentation, not one evidence tier

Module-local examples range from tutorial programs to GPU readback tests, golden-image comparisons, host/browser harnesses, and policy-only probes. The filename or registration directory does not tell which one it is. For each example record:

- whether it is compiled by the selected configuration;
- whether it runs automatically or only by explicit dispatch;
- which renderer/native driver/host actually engaged;
- whether its oracle is exit status, structured values, pixels, or human inspection;
- which unavailable-host outcome is skip, refusal, or failure.

Shared golden images remain at repository level because many renderer modules consume the same oracle. A renderer-owned test executable plus a shared image is still two artifacts whose provenance and update rules must travel together.

11.10 Presets are named environments, not universal promises

The pinned presets cover a Web build, three Devices sanitizer builds, and a native Ninja test build. The sanitizer presets pass raw compiler/linker flags even though CNA also exposes the `CNA_SANITIZE` option; this is redundant but observable. They explicitly enable `CNA_DEVICES`, preventing a sanitizer job named for Devices from silently compiling the feature out.

A preset proves only its declared target and commands. The Web preset builds the house demo with tests off; it is not a browser conformance run. The native test preset builds `CnaTests`; it does not make every standalone CTest executable. Hosted workflow reality is audited separately in Chapter 73.

11.11 Developer tools and what they decide

CNA’s mechanical tools fall into distinct authority classes:

Structural validators Compare finite source-owned sets, paths, and identity spellings. They are strong for structural equality and silent about behavior.

Module probes and closure checkers Compile a minimal consumer or inspect a link artifact. Their verdict depends on generator and host engagement, as Chapter 10 shows.

Content and catalog checkers Validate CNJ/glTF fixtures, examples, or documented catalogs. Their parser and denominator define the result.

Renderer/golden harnesses Exercise a chosen native path and compare pixels or structured output. They require identity and driver evidence in addition to a green comparison.

Cross/toolchain launchers Stage DLLs, select Wine prefixes, start browsers, or prepare target files. Successful process creation is not yet proof that the intended runtime did the work.

The durable rule is to keep acquisition, build, registration, execution, and oracle engagement as separate facts. A dependency pin enables reproducibility; it does not establish that a test used the dependency. A built example establishes reachability; it does not establish correct pixels. Appendix G provides the compact claim vocabulary used throughout the rest of the book.

Part III

The Graphics Machine

Chapter 12

GraphicsDevice and the Rendering Pipeline

GraphicsDevice is CNA’s public rendering hub. It owns the selected renderer and window when applicable, stores public pipeline state, tracks resources for ordered teardown, normalizes render-target and vertex bindings, and forwards draws. That sequence matters: public state can be valid even when a family ignores the corresponding native operation. Figure 12.1 separates those layers.

This chapter owns the shared device contract. Resource classes are detailed in Chapters 14–18; renderer defaults and divergences are in Chapter 19.

12.1 Construction and native lifetime

The XNA-shaped constructor accepts a **GraphicsAdapter**, **GraphicsProfile**, and **PresentationParameters**. The CNAEXT default constructor delegates with the default adapter, **Reach**, and default presentation parameters. It is windowless only when the compiled family is inherently windowless (including **HEADLESS**, **SOFTWARE**, **STUB**, and **PORTABLEGL**). D3D12’s runtime **HeadlessEXT** path instead requires explicit presentation parameters. A default constructor on a normal windowed family creates an SDL window, usually 1024×768 when no size was requested.

Game owns a device value member before a derived game’s **GraphicsDeviceManager** exists. During initialization the manager reconfigures that same device through **Reset**; ordinary game code does not construct a second device.

12.1.1 Window, renderer, and presentation setup

The private construction path is observable through its ordering:

1. Initialize SDL video unless the compiled or requested path is windowless.
2. Create a window, attach a caller-owned **DeviceWindowHandle**, or retain null. Both CNA-owned and attached windows are published to mouse and text-input services.

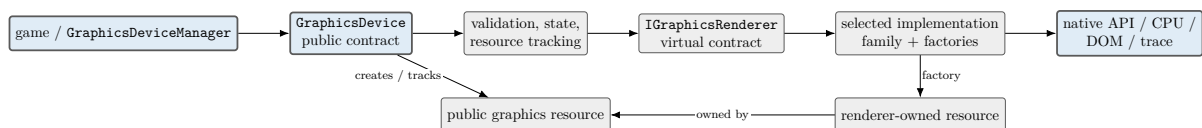


Figure 12.1: The renderer-independent call and resource path. A public call reaching **IGraphicsRenderer** is still weaker evidence than family engagement or a correct native result.

3. Apply SDL window size/fullscreen state.
4. Build `GraphicsRendererCreateArgs` and call the selected family factory.
5. Normalize reported backbuffer/depth formats, query the renderer's logical and physical viewport, and apply initial blend, depth/stencil, and rasterizer state.

An attached window is borrowed: CNA clears matching input handles at teardown but destroys the window only when `ownsWindow_` is true. Chapter 19, §19.1.3, covers the remaining native-handle rules.

Limitation. Construction is not fully transactional. After SDL video initialization, a later window, renderer, or initial-state exception can bypass `GraphicsDevice`'s destructor and leak the raw owned window, published input handle, or video-subsystem lease. Teardown also calls `SDL_QuitSubSystem(SDL_INIT_VIDEO)` even for a device that skipped initialization, which can release another client's reference. The located tests establish windowless operation, not mixed-client reference accounting or injected construction failure.

12.2 Public state

The property surface is easier to use when grouped by ownership and forwarding:

Group	Properties	Contract
Identity/lifecycle	adapter, profile, device status, disposed state, presentation parameters	Adapter/profile are selected state; only D3D9 currently drives the lost/not-reset status callback.
Display	display mode, viewport, scissor rectangle	Viewport/scissor are stored and forwarded; a getter is not a native-state query.
Shader slots	pixel/vertex textures and sampler-state collections	Collections validate bookkeeping; draws bind textures through effects, while sampler state is forwarded before draws.
Pipeline state	blend, depth/stencil, rasterizer, blend factor, multisample mask, reference stencil	State objects are copied into the device, not frozen references. Reassign after mutation.
Bindings	index buffer, vertex-buffer list, render-target list	Singular and plural vertex routes now share one coherent binding state.

Assigning `Textures[0]` alone does not make a draw sample that texture. A stock effect populates `GpuDrawParams`; `ShaderEffect` uses its own texture setters. The device's texture collections enforce slot, disposal, and active-target invariants, but they are not a general shader binding mechanism.

12.2.1 Viewport and scissor

Both setters first ask the renderer to apply the value and then retain public state according to the individual setter's success path. Scissor enable is a separate `RasterizerState::ScissorTestEnable` value. A useful test therefore draws across the boundary; property round trips alone are insufficient.

Three lifecycle rules define when CNA replaces custom rectangles:

- Construction initializes the full logical viewport. Current resize handling also sets the scissor to full bounds after an actual logical or physical-viewport change.

- **Present()** preserves custom state while the renderer’s logical size and physical default rectangle remain unchanged.
- Every 2D, cube, MRT, or backbuffer target switch resets both to the first destination’s full bounds, even for a redundant binding.

Renderers may use a physical default viewport distinct from the logical public dimensions for letterbox or overscan. Chapter 19, §19.4.3, records draw-time coverage and exceptions.

12.3 Clear, present, and reset

The four clear overloads have intentionally different scopes:

Call	Shared request
<code>Clear(Color)</code>	target, depth, and stencil; depth uses current <code>Viewport.MaxDepth</code> , stencil uses zero
<code>Clear(options,color,depth,stencil)</code>	requested aspects after attachment masking
<code>Clear(r,g,b,a)</code>	CNAEXT direct color-hook call; it does not pass through full aspect routing
<code>Clear(Color,depth)</code>	CNAEXT target plus depth convenience form

FNA’s `Vector4`-color overload is absent. CNA also exposes only parameterless `Present()`; FNA’s source/destination-rectangle and override-window form has no CNA counterpart. `Present` refuses while a user target is bound, calls the renderer, then refreshes viewport/scissor only if the reported destination geometry changed.

12.3.1 ClearOptions and attachment masking

`ClearOptions` is the bitwise combination of `Target`, `DepthBuffer`, and `Stencil`. Shared code validates a requested depth value, independently asks whether the active 2D/cube target or backbuffer has depth and stencil, masks only absent aspects, and selects one of seven non-empty renderer methods. A depth/stencil-only clear on an attachment with neither becomes a no-op, matching current FNA.

This independent aspect query is important for `Depth16/Depth24` versus `Depth24Stencil8` and for families with a standalone stencil plane. Family-specific ordering and remaining defects—notably EasyGL’s color-hook/depth coupling and D3D9’s additional backbuffer-format gate—are in §19.4.2.

12.3.2 Reset order and rollback boundary

All four reset overloads end in `Reset(const PresentationParameters&, GraphicsAdapter*)`. The order is public behavior:

1. Raise `DeviceResetting`; handlers still see the old adapter and parameters.
2. Clone the request, normalize the renderer’s applied color/depth formats, store the adapter and logical size, and update touch dimensions.
3. Apply window state and virtual resolution inside a rollback guard. A failure restores public presentation state, adapter, logical size, touch dimensions, and attempts to restore the former window state before rethrowing the original diagnostic.
4. Apply/clamp MSAA and write the applied count back; forward swap interval and the presentation-format update hook.

5. Refresh viewport/scissor and raise `DeviceReset`.

The rollback guard ends before MSAA, interval, and format hooks. A later exception can therefore leave earlier work applied and suppress `DeviceReset`. Successful completion establishes the shared route, not that a compositor accepted fullscreen or an interval request. Applied presentation values are stronger than the original request but still depend on each family's truthful reporting; see §19.2.5 and §19.2.4.

FNA stores the new state before its resetting event and performs one FNA3D backbuffer reset, so event observations and native transaction shape differ.

12.3.3 GraphicsDeviceManager event and ownership model

A game-attached `GraphicsDeviceManager` registers the manager and device-service interfaces but does not own the game device. Initial `CreateDevice()` raises `PreparingDeviceSettings`, resets the existing device, subscribes to later device reset events, and then raises `DeviceCreated`. The initial settle-in reset is intentionally not forwarded as a manager reset pair. Subsequent `ApplyChanges()` reaches device reset; the manager forwards the device's events once.

Edits made in `PreparingDeviceSettings` apply to that candidate only and are not copied back into preferred fields. The callback must retain a non-null adapter and must not re-enter `ApplyChanges`. Listener exceptions propagate and can interrupt the staged operation.

The manager subscribes to `GameWindow::ClientSizeChanged` with a lambda capturing raw `this`, discards the removal token, and removes only service registrations on disposal. Destroying or replacing a manager while its game window remains live can therefore leave a dangling resize callback. Normal member lifetime avoids a later resize during base destruction, but that ordering is not a general ownership guarantee.

Manager disposal raises `DeviceDisposing` even though it does not delete a game-owned device; `Game` uses that notification for `UnloadContent`. The disposed flag is set after the `Disposed` event, so a re-entrant or throwing listener remains another source-proven lifecycle hole.

12.4 Render targets

`SetRenderTarget(RenderTarget2D*)`, the cube-face overload, and `SetRenderTargets(vector)` all normalize through the plural path. An empty list restores the backbuffer. `GetRenderTargets()` returns a new vector; CNA still lacks FNA's no-allocation extension.

12.4.1 Binding contract

Each public binding becomes a descriptor naming a 2D target or cube face, dimensions, and the renderer-applied sample count. The shared route enforces:

- at most four bindings, with D3D9's `Reach` profile narrowed to one;
- non-null, non-disposed targets with a usable private renderer;
- valid cube faces and zero for unsupported 2D array slices;
- equal dimensions and applied sample counts;
- no duplicate subresource, while distinct faces of one cube remain legal.

Native binding must succeed before public bindings and viewport/scissor are committed. CNA has no redundant-binding early return: rebinding the same target reaches the renderer, resets rectangles, and clears a `DiscardContents` destination again. Preserve and Platform usage

retain content; discard requests a black clear of color and each reported depth/stencil aspect. The API does not guarantee that discarded content later reads as black.

12.4.2 A render-to-texture example

The complete example is retained at `tools/cna-screenshot-infra/rendertarget_roundtrip_demo.cpp`. This excerpt shows the public sequence; it is adapted from that executable, with setup and triangle data omitted:

```

1  RenderTarget2D offscreen(dev, 128, 128);
2  dev.SetRenderTarget(&offscreen);
3  dev.Clear(Color(60, 20, 80, 255), 1.0f);
4  fx.Apply();
5  DrawTriangle(dev, offscreenTriangle);
6
7  const Rectangle centerRegion(64, 64, 1, 1);
8  Color centerPixel;
9  dev.GetBackBufferData(&centerRegion, &centerPixel, 0, 1); //
    SOFTWARE-specific
10
11 dev.SetRenderTarget(static_cast<RenderTarget2D*>(nullptr));
12 assert(dev.GetRenderTargets().empty());
13
14 SpriteBatch batch(dev);
15 batch.Begin();
16 batch.Draw(offscreen, Rectangle(146, 20, 90, 90), Color::White);
17 batch.End();

```

The readback probe is intentionally marked renderer-specific: some families read the active target, some always read the backbuffer, Headless synthesizes state, and others throw. Portable target verification uses `RenderTarget2D::GetData` where that family supports it.

Historical note. The captured image in Figure 12.2 predates the pin. It exposed a Software defect: the sampler cast only to `SoftwareTextureRenderer`, so the sibling render-target renderer became a null texture and produced a white inset. At 1bb2145d, `SoftwareColorSurface` is the shared storage capability, and the registered `Software_RenderTargetReadback` test verifies exact target readback plus `SpriteBatch` and textured-primitive sampling. The historical image is retained because it demonstrates why a numeric producer check and a consumer pixel check answer different questions; it is not a current expected result.

12.4.3 Multiple render targets

The public call accepts up to four `RenderTargetBinding` entries. That shape does not establish that one draw writes all attachments. WebGPU rejects MRT; Software's current route does not provide a portable multi-output stock-effect contract; diagnostic renderers may record only logical state. D3D12 can bind several RTVs while stock draws still target slot zero. Use `MultipleRenderTargets` as an initial query, then require a successful bind and a shader and pixel oracle that distinguish every output. Chapter 19, §19.4.1, owns the current qualifications.

12.5 Vertex and index bindings

The singular and plural vertex APIs now describe one coherent state. A singular bind validates the nonnegative element offset, replaces the vector with slot zero, and null clears both. The



Figure 12.2: Historical pre-fix Software output: the target was populated, but its white inset showed that `SpriteBatch` could not sample it. The defect is fixed and covered at the edition pin.

plural form accepts up to sixteen entries, retains legal null slots, and sets the singular pointer to slot zero (or null for an empty list). `GetVertexBuffer()` and `GetVertexBuffers()` therefore agree. This corrects the stale split-state account in an earlier edition.

The `Indices` property, `SetIndexBuffer/GetIndexBuffer`, and method-call `Indices()` spellings all use the same index pointer. This is surface redundancy, not independent state.

12.6 Draw calls

Buffer-backed calls are `DrawPrimitives`, `DrawIndexedPrimitives`, and `DrawInstancedPrimitives`. Shared validation checks bound resources, applied effect, topology-derived element ranges, and vertex/index bounds before native submission. Sampler state is applied before each route.

FNA's generic user-primitive methods become CNA overload sets for raw packed bytes and the four built-in vertex types: `VertexPositionColor`, `VertexPositionColorTexture`, `VertexPositionTexture`, and `VertexPositionNormalTexture`. Indexed variants accept 16- and 32-bit indices. Raw nonstandard layouts require an explicit `VertexDeclaration`. Typed public objects are packed into the asserted stream ABI before upload; callers must not reinterpret their polymorphic C++ object representation as GPU bytes.

Reusable scratch vectors stage user draws. Their capacity persists, so the first large call may allocate while subsequent calls of the same or smaller size do not. That is an implementation warm-up cost, not a per-draw allocation promise.

12.6.1 Instanced binding transport

Current CNA no longer forwards only one instance buffer. The same `FillVertexStreamBindings()` helper serves ordinary and instanced draws and copies up to sixteen slot-aligned descriptions into `GpuDrawParams`: renderer handle, declaration, stride, element offset, instance frequency, and count. Instanced validation applies the caller's base vertex only to per-vertex streams and computes each instance stream's required records as $1 + \lfloor (n - 1) / f \rfloor$.

Families still differ in accepted layouts and native divisor support. The complete current boundary is §19.3.2; a compile-successful two-stream call is not a portable arbitrary-effect claim.

12.6.2 Effect matrices

Every shared draw starts `World/View/Projection` at identity, then uses `dynamic_cast<const IEffectMatrices*>` on the applied effect. Stock effects, `ShaderEffect`, and CNA's current PBR effect implement that interface. A custom effect that deliberately omits it receives identity transforms instead of an exception; screen-space clip coordinates are a legitimate use of that fallback.

The `CNAEXT PrimitiveVerts()` helper converts a primitive count into the required vertex count and rejects unknown topology. It mirrors an internal FNA helper but is public in CNA.

12.7 Backbuffer readback

Three `GetBackBufferData` overloads write `Color`: whole backbuffer, caller offset, or an optional rectangle. Unlike FNA's generic method, CNA exposes no other destination element type. Current shared code derives whole-image dimensions from `PresentationParameters`, validates a positive in-bounds rectangle, checks a minimum `elementCount`, reads RGBA bytes, and constructs each `Color` so its `vpPtr`-bearing C++ layout is never treated as a byte array.

Two caller-safety gaps remain. `startIndex` is not checked for negativity, and the method checks `elementCount >= pixelCount` rather than capacity after the start index. The signed width-times-height calculation is also not explicitly overflow-checked. Validate the complete destination span before calling.

The method name does not define one image across families. Active-target, physical-backbuffer, synthetic, and refusal classes are listed in §19.2.6. Pixel evidence must state the family and coordinate space.

12.8 Resources and disposal

`GraphicsDevice` keeps a non-owning vector of resource addresses. A base-resource constructor registers `this` and raises `ResourceCreated` before the derived native handle exists. Derived disposal releases its renderer handle; the base then raises `ResourceDestroyed`, removes the address, and marks itself disposed. Destructor-driven `Dispose(false)` suppresses the resource's own `Disposing` event but still raises the device's destroyed event.

Device disposal moves and clears the registry, disposes each resource, raises device `Disposing`, destroys renderer/window state, quits SDL video, and finally marks the device disposed. Two limits follow from that order:

- a still-bound `RenderTarget2D` can throw during resource disposal after the registry has already been cleared, preventing renderer teardown and making retry incomplete;
- a device `Disposing` listener can re-enter because `IsDisposed` remains false until after the event and native teardown.

Unbind targets before explicit device disposal and do not dispose the device recursively from an event. Renderer-specific in-flight/deferred destruction rules remain in the family chapters.

12.8.1 GraphicsResource and relocation

GraphicsResource supplies device, disposed state, name, tag, event, and **ToString**. Its copy-/move operations do not repair the device's address registry. **Texture2D** is copyable and shares its renderer handle; several other resources are movable. A destination can therefore remain live but untracked after relocation. Keep live resources address-stable and within device lifetime; Chapter 19, §19.1.1, gives the ownership consequences.

12.9 Adapters and format queries

GraphicsAdapter exposes current/supported display modes, description, display name, monitor handle, default/widescreen flags, and identity fields. On Linux, vendor/device IDs are best-effort values from **sysfs**; elsewhere they can be zero. Revision and subsystem ID are always zero at the pin. **DisplayMode** holds width, height, format, and computed aspect ratio. The three device exception types are thin **runtime_error** subclasses.

12.9.1 Adapter enumeration

getAdaptersProperty() lazily builds one adapter per SDL display and caches the owning **unique_ptr** vector. **AdaptersChanged()** destroys and rebuilds it; do not retain adapter references across that call. **DefaultAdapter** is the current first entry, not a separate GPU-selection service.

12.9.2 Profile and format-query scope

D3D9 is the only family whose **IsProfileSupported**, render-target format query, and backbuffer format query consult its native device caps. It combines profile whitelists with native format support and clamps samples per format. Other builds return true for either profile; render-target queries use a shared format list and report zero samples, while backbuffer queries select **Color** and zero samples. These methods therefore are not portable hardware probes.

12.10 CNA extensions and diagnostics

The extension surface falls into four groups:

Purpose	Representative methods	Boundary
Resource instrumentation	create/destroy hooks, add/remove reference, tracked count	Public for framework plumbing and tests; events occur at base construction/disposal boundaries.
Renderer identity/capability	renderer type/name, SupportsCapability , maximum texture dimension, unsupported-3D policy	Identity is compile-time; capability is optimistic unless the family overrides; the texture limit is enforced by content/texture paths.
State and effect plumbing	depth/blend/write toggles, current effect, graphics-profile switch, stored presentation parameters	Primarily internal routes; not substitutes for XNA state objects or a general runtime profile change.
Debug/recovery	context-recovery policy, string marker, renderer reference, legacy MSAA recreation	Renderer-specific or test-oriented; direct renderer access is not portable.

SetContextRecoveryEnabled(false) can discard future texture CPU shadows even when the family hook is empty; re-enabling cannot rebuild them. **SetStringMarkerEXT** is a no-op on

most families and can throw on D3D9. `RecreateRendererForMultiSampleCount` destroys the renderer without preserving resources, state, events, or rollback; its header rationale is stale now that normal reset applies MSAA. Retain it only for its legacy pre-resource tests.

Some convenience aliases and overloads still lack the expected `CNAEXT` annotation. A strict-surface build is valuable but is not a proof that every untagged declaration belongs to XNA.

12.11 Compiled example: direct two-triangle rendering

The complete executable is `tools/cna-screenshot-infra/software_screenshot_demo.cpp`. It constructs a game, selects a 256×256 backbuffer, and draws two `VertexPositionColor` triangles through `BasicEffect`. The essential draw body is:

```
1  auto& dev = getGraphicsDeviceProperty();
2  dev.setRasterizerStateProperty(RasterizerState::CullNone);
3  dev.Clear(Color(30, 30, 60, 255), 1.0f);
4
5  BasicEffect fx(dev);
6  fx.VertexColorEnabled = true;
7  fx.Apply();
8
9  VertexBuffer background(dev, 3);
10 background.SetData(backgroundVertices, 3);
11 dev.SetVertexBuffer(&background);
12 dev.DrawPrimitives(PrimitiveType::TriangleList, 0, 1);
13
14 VertexBuffer foreground(dev, 3);
15 foreground.SetData(foregroundVertices, 3);
16 dev.SetVertexBuffer(&foreground);
17 dev.DrawPrimitives(PrimitiveType::TriangleList, 0, 1);
18 dev.SetVertexBuffer(nullptr);
```

Evidence. This is adapted from the retained complete source, not standalone copy-and-paste code: headers, vertex arrays, game lifecycle, manager setup, screenshot helper, and `main` are omitted. The named source is the compile target used for verification. Figure 12.3 is its current Software-renderer artifact; it establishes CPU rasterization, depth occlusion, and interpolated vertex color on that configuration.

12.12 Summary

`GraphicsDevice` supplies a coherent public state machine, not one uniform native implementation. The strongest portable assumptions are the shared validation and binding rules. Presentation fidelity, advanced draw paths, target behavior, and readback still require the selected family's evidence. Applied values and discriminating output are more informative than stored requests or successful property round trips.

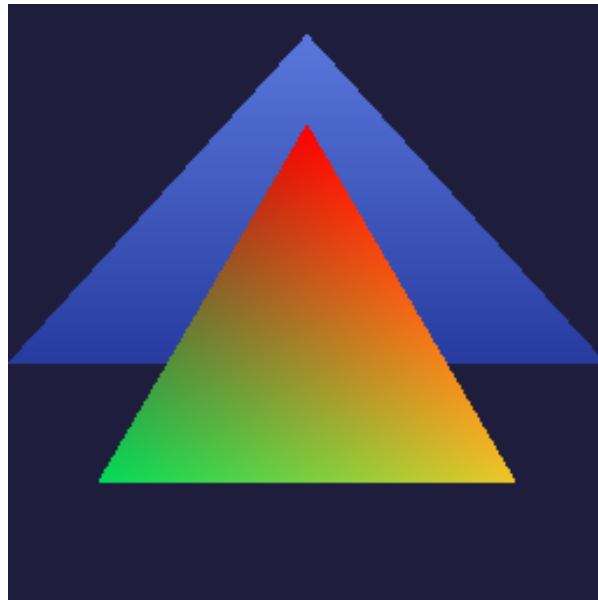


Figure 12.3: Software-renderer output from the direct `GraphicsDevice` example. The nearer triangle occludes the farther one and interpolates its three vertex colors.

Chapter 13

SpriteBatch and 2D Rendering

`SpriteBatch` is among CNA’s most frequently used APIs. Every public renderer identity exposes the contract, but behavior and evidence still vary by implementation family. This chapter describes the current API, its supporting types (`SpriteSortMode`, `SpriteEffects`, `SpriteFont`), and selected defects that clarify the contract or the limits of its verification.

13.1 Begin, Draw, End

`SpriteBatch` inherits `GraphicsResource` and offers five `Begin()` overloads of increasing specificity. The public contract defaults whatever the caller omits: `Begin()` alone defaults to `SpriteSortMode::Deferred`, `BlendState::AlphaBlend`, a `LinearClamp` sampler, no depth-stencil state, `RasterizerState::CullCounterClockwise`, and an identity transform; the fuller overloads let a caller override the sort mode and blend state, the three optional state objects (nullptr falls back to the same defaults), a custom `Effect` (nullptr falls back to an internal default `SpriteEffect`), and finally an explicit transform `Matrix` applied to every sprite in the batch before projection.

The live CNA implementation does not currently honor that whole state contract. `SpriteBatch::Begin()` applies the requested/default blend and depth-stencil values and forwards the resolved sampler, but its `RasterizerState*` parameter is commented out and never read. FNA stores the caller’s value (or `CullCounterClockwise`, the matching `RasterizerState` preset) and applies it in `PrepRenderState()`. CNA instead leaves whatever rasterizer state was already on `GraphicsDevice`. The practical scissor consequence is direct: passing a rasterizer with `ScissorTestEnable=true` to the normal full `Begin()` overload cannot enable clipping on any renderer; a previously-enabled state can also leak through a later default `Begin()`. A game can work around the first half only by assigning `GraphicsDevice.RasterizerState` separately before drawing. Section 19.4.3 then determines whether the selected renderer can actually honor it.

`Draw` itself has nine overloads: six are the canonical XNA forms — position-or-destination-rectangle, an optional source rectangle (represented as `std::optional<Rectangle>` rather than C#’s nullable `Rectangle?`), color, and progressively more control over rotation, origin,

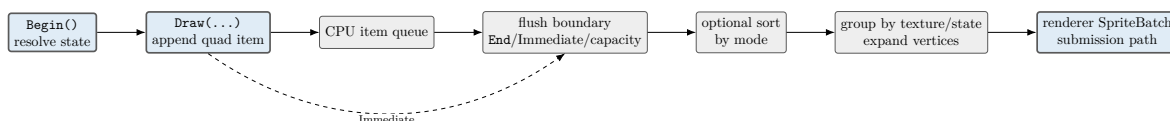


Figure 13.1: `SpriteBatch`’s CPU submission flow. “Batch” describes the public accumulation and flush contract; renderer implementations may issue more than one native draw.

scale (as either a uniform `float` or a non-uniform `Vector2`), `SpriteEffects`, and layer depth — and three are CNAEXT raw texture-and-rectangle convenience forms layered on top. `DrawString` adds six more overloads (`string` or `StringBuilder`, each in a simple 4-argument form, a uniform-scale 9-argument form, and a non-uniform-scale (`Vector2` scale) 9-argument form).

13.2 Draw overloads and rotation origin

The nine `Draw` overloads split cleanly into two families by their second parameter — a `Vector2 position` (four overloads, increasingly parameterized) or a `Rectangle destinationRectangle` that scales the texture to fit (three overloads) — plus the two CNAEXT raw `float x`, `float y` convenience forms already noted. Every XNA-compatible overload takes an explicit `Color` tint — there is no overload that omits it, unlike the CNAEXT convenience forms. The table below abbreviates each overload’s parameter names for width (`tex` for `texture`, `pos` for `position`, and so on) — the full, exact parameter names for every overload are shown in the parameterized example below the table.

Family member	Adds, relative to the simplest form in its family
<code>Draw(tex, pos, color)</code>	The simplest XNA form: whole texture, no rotation, unit scale.
<code>Draw(tex, pos, srcRect, color)</code>	An optional source sub-rectangle (<code>std::optional<Rectangle></code> , not C#’s nullable <code>Rectangle?</code>) — the sprite-sheet-cell pattern.
<code>Draw(tex, pos, srcRect, color, rot, origin, scale, fx, depth)</code>	Uniform float scale.
<code>Draw(tex, pos, srcRect, color, rot, origin, scale, fx, depth)</code>	Same, but <code>scale</code> is a non-uniform <code>Vector2</code> — these two fully-parameterized overloads differ <i>only</i> in that one field’s type.
<code>Draw(tex, destRect, color)</code>	Whole texture, scaled to fill <code>destRect</code> .
<code>Draw(tex, destRect, srcRect, color)</code>	Adds the source sub-rectangle.
<code>Draw(tex, destRect, srcRect, color, rot, origin, fx, depth)</code>	Full control, destination-rectangle family — note there is no separate <code>scale</code> parameter here at all, since the destination rectangle already implies it.

The parameterized overloads share `origin`, a point *in texture space* (not screen space) that both the rotation pivot and the drawn position are measured relative to. Rotating a 64×64 sprite around its own center, rather than its top-left corner, means passing that texture’s own center as `origin`:

```

1  Vector2 center(texture.getWidthProperty() / 2.0f, texture.
    getHeightProperty() / 2.0f);
2  float angle = totalSeconds; // radians; grows every frame for a
    continuous spin
3
4  spriteBatch_>Draw(
5      texture, spritePosition, std::nullopt, Color::White,
6      angle, center, 1.0f, SpriteEffects::None, 0.0f);

```

Passing `Vector2::Zero` for `origin` instead — the easy mistake, since it is also the type’s default-constructed value — rotates the sprite around its top-left corner, which reads as the sprite “orbiting” `spritePosition` in a circle rather than spinning in place; the visual symptom is easy to misdiagnose as a rotation-math bug when the actual cause is simply the wrong `origin`.

13.3 Vector positions are quantized before they reach a renderer

The `Vector2`-position overloads do not preserve sub-pixel destinations. Their shared queue record stores an integer `Rectangle`; CNA validates the position and scaled size, then converts each floating component with truncation toward zero. Thus `(10.9f, 4.9f)` and `(10.1f, 4.1f)` both enter every renderer as `(10,4)`. This is a shared `SpriteBatch` fidelity difference from XNA, not a renderer-specific rasterization rule.

Text follows a subtly different policy. `DrawString()` computes each transformed glyph destination in floating point and rounds it to the nearest integer before storing the same rectangle shape. A sprite and a glyph submitted at the same fractional coordinate can therefore land on adjacent pixels. Both conversion helpers reject non-finite values and results outside the signed 32-bit range; the safety check prevents undefined conversion, but does not restore the lost fractional position. Code that depends on smooth sub-pixel camera motion must account for this quantization above the renderer layer.

13.4 The D3D9 half-pixel offset and a mutation-testing trap

XNA 4.0 ran exclusively on Direct3D 9, whose rasterizer places texel centers at integer pixel coordinates rather than pixel centers — the well-known “D3D9 half-pixel offset” problem, notorious enough that porting guides for other 2D APIs still warn about it decades later. `SpriteBatch`’s own shared, renderer-agnostic `SpriteBatch.cpp` carries no trace of this compensation at all, which is exactly correct behavior for every renderer built on a modern API (EasyGL, Vulkan, BGFX, D3D11, D3D12, WebGPU): none of them share Direct3D 9’s own texel-center convention, so there is nothing to compensate for, and adding an offset unconditionally would introduce an error on every renderer that does not need one. On D3D9, the compensation lives one level down, inside `D3D9SpriteBatchRenderer::BuildMatrixTransformEXT` specifically — baked directly into the projection matrix `SpriteBatch` constructs, the same technique the classic XNA 4.0 D3D9 fix itself used:

```

1 // D3D9-only half-pixel compensation, folded
2 // into the orthographic projection SpriteBatch's own
   MatrixTransform
3 // applies -- not a separate step, not visible anywhere in shared
   code.
4 Matrix projection = Matrix::CreateOrthographicOffCenter(0, width,
   height, 0, 0, 1);
5 projection.M41 += -0.5f * projection.M11;
6 projection.M42 += -0.5f * projection.M22;
7 // MatrixTransform = SpriteBatch's own transform, then this
   projection
8 // (row-vector convention: transform applied first).
```

Verification against the XNA 4.0 oracle (Chapter 28) exposed a mutation-testing trap: two independently designed tests could not detect whether the offset was present. A 1×1 test texture cannot detect it, because the half-pixel offset shifts *which texel a given screen pixel*

samples, not where a sprite’s geometric edges land on screen — a texture with only one texel has nothing else to shift onto, so removing the offset entirely changes nothing a one-texel-texture test can observe. Separately, sampling only at a drawn rectangle’s own quadrant-center points misses it too, for the same underlying reason: those points are exactly where a coarse boundary check happens to line up regardless of the sub-pixel shift. Both traps were only caught by deliberately mutation-testing the passing test itself — commenting out the two M41 / M42 lines above and re-running the existing check suite, which still passed with the fix removed. The replacement samples a specific internal texel boundary a rotated/flipped sprite’s own blended edge produces — a pixel confirmed to shift from one exact, named RGB value to another under the same mutation, the first check in the suite that changed under the mutation. A regression test must fail when its protected behavior is removed; mutation testing establishes that property directly.

13.5 SpriteSortMode and SpriteEffects

`SpriteSortMode` has five values: `Deferred`, `Immediate`, `Texture`, `BackToFront`, and `FrontToBack`. Only the latter two honor `layerDepth`; the other modes ignore it by design.

`Texture` mode applies a stable sort to raw texture pointers. The `TextureGroupsDrawsByTextureAndPreservesGroupOrder` test establishes two properties: sprites sharing a texture become adjacent, and their relative submission order is preserved. The order between different texture groups is unspecified because it depends on pointer order. Overlapping sprites must therefore not use `Texture` mode when the A-versus-B group order matters. A separate SDL Renderer pixel test checks that the reordered draw calls bind the intended textures; a mock renderer can establish grouping but cannot observe that renderer-side binding.

Limitation. `SpriteEffects` is a plain `enum` class with values 0–2 and no bitwise operators. Microsoft XNA defines it as a flags enum, where horizontal and vertical flips can be combined as value 3. CNA’s `DrawString` direction tables also contain only three entries. Combined-axis text flipping is therefore not expressible through the current API. The pinned docs/`spritefont-support.md` records this as an open API-completeness gap.

13.6 Historical SpriteEffects layout defect

Historical note. An earlier `DrawString` path flipped each glyph’s texture coordinates without mirroring the glyph sequence or positions. The correction added FNA-shaped axis-direction and axis-mirroring terms in the shared `SpriteBatch.cpp`. Because the defect was above the renderer interface, one shared fix corrected every renderer path.

13.7 Begin() leaves GraphicsDevice.BlendState active

The `BlendState` passed to `Begin()`—or `BlendState::AlphaBlend` when omitted—remains assigned to `GraphicsDevice.BlendState` after `End()`. This matches FNA. A following 3D draw inherits that mode unless the game assigns another blend state:

```
1 spriteBatch_ ->Begin(SpriteSortMode::Deferred, BlendState::Additive);
2 spriteBatch_ ->Draw(cornerTexture, Rectangle(0, 0, 4, 4), Color::
   White);
3 spriteBatch_ ->End();
4
```



```

5 // No BlendState reassignment here -- this quad draws with Additive,
   not AlphaBlend or Opaque,
6 // because SpriteBatch::Begin() changed GraphicsDevice.BlendState
   and End() never restores it.
7 device.DrawUserPrimitives(PrimitiveType::TriangleList, verts, 0, 2);

```

Historical note. EasyGL once ignored the requested state and installed hard-coded source-alpha factors. A pixel regression now clears to gray, draws a small additive sprite, then draws an opaque 3D quad without another state assignment. Its off-sprite green-channel sample distinguishes the requested-state behavior from the former hard-coded blend.

Limitation. On EasyGL, a full-backbuffer `SpriteBatch` draw before the frame's first 3D draw can still break that frame's 3D rendering. Window-title readback, viewport-reset order, and render-target binding were excluded as causes, but the defect was not root-caused at the pin. The blend-state regression uses a 4×4 corner sprite and therefore does not cover this trigger.

13.8 SpriteFont

`SpriteFont` is not a `GraphicsResource`. It owns a texture atlas, glyph bounds, cropping data, a character set, line spacing, character spacing, kerning data, and an optional default character. Its constructor is `CNAEXT` and public. Microsoft XNA's `SpriteFont` constructor is `internal`, reachable only from the compiled `.xnb` content pipeline's `SpriteFontReader`; CNA lacks that reader and exposes construction instead.

`MeasureString` exists for both `string` and `StringBuilder`. The latter forwards to the `string` implementation, a simplification available in C++ because there is no garbage-collection-pressure argument against sharing code here. A trailing `"\n"` adds a second empty line to the measured height: $Y = 2 \times \text{LineSpacing}$. Both the newline handler and the loop's final height addition run, matching Microsoft XNA's result.

For text encoding, CNA's `String` is UTF-8 (`std::string`), and `SpriteFont` / `SpriteBatch` decode each glyph via an internal `DecodeUtf8CodePoint` helper. Because `charcs` (the glyph-key type, per Chapter 3's type-alias table) is 16 bits wide, only Basic Multilingual Plane code points can serve as glyph keys. An invalid or truncated UTF-8 sequence decodes to `'?'`; the byte index still advances by at least one.

This example uses `MeasureString` to center HUD text before calling the simplest `DrawString` overload:

```

1 const std::string text = "Game Over";
2 Vector2 size = font.MeasureString(text); // Vector2(width,
   height) in pixels
3 Vector2 centered(screenCenterX - size.X / 2.0f, screenCenterY - size
   .Y / 2.0f);
4
5 spriteBatch->DrawString(font, text, centered, Color::White);

```

13.9 SpriteFont evidence by renderer

docs/spritefont-support.md verifies the property surface, `MeasureString`, single- and multi-glyph placement, newline handling, default-character fallback, single-axis flip, and rotation/scale as pixel-correct on both **SDL_Renderer** and **EasyGL**. Vulkan and BGFX reuse the shared layout logic, but that source relationship is not a renderer-engaged or pixel oracle. Their `SpriteFont` paths have not been independently re-verified, as also recorded in docs/graphics-renderer-feature-matrix.md. Combined-axis flip is not representable on any renderer at all, for the `SpriteEffects` reason above.

13.10 Two corrected SDL_RENDERER defects

Historical note. The recorded SDL Renderer campaign found two shared-API defects. First, `SDL_RenderTextureRotated`'s pivot convention differed from XNA's rotation-origin convention; the correction offsets the destination rectangle. Second, the renderer ignored `Begin()`'s `transformMatrix`. The correction added a path using `SDL_RenderTextureAffine`, used only when the transform actually differs from identity, so the common (and cheaper) untransformed path is unaffected.

13.11 Flush timing: sorting without draw-call coalescing

The class is named `SpriteBatch`, and `SpriteSortMode::Deferred` — the default — sounds like it defers submission to build one large GPU-side batch. Reading `SpriteBatch.cpp`'s own `pushSprite` / `flushSingle` / `flushBatch` shows a narrower design with performance implications for code accustomed to GPU-side draw-call coalescing:

```

1  void SpriteBatch::pushSprite(...)
2  {
3      // ...
4      if (sortMode_ == SpriteSortMode::Immediate)
5          flushSingle(info);           // dispatched to the
                                         renderer right here, right now
6      else
7          spriteQueue_.push_back(info); // deferred -- but only the
                                         SORT, not the draw call
8  }
9
10 void SpriteBatch::flushBatch()
11 {
12     // ... optional std::stable_sort by layerDepth or texture
        pointer, per sort mode ...
13     for (const SpriteInfo& s : spriteQueue_)
14         flushSingle(s);               // still one renderer->Draw
                                         () call per sprite
15     spriteQueue_.clear();
16 }
```

`Immediate` sends every `Draw()` directly to `flushSingle`; it is the only mode in which renderer state changes between sprite submissions retain their exact position. The other modes queue and optionally sort, but `flushBatch()` still calls the renderer once per sprite. CNA's batch therefore coalesces sort decisions, not GPU draw calls: roughly one thousand sprites produce roughly one thousand renderer submissions.

`pushSprite` also rejects a disposed texture with `ObjectDisposedException`. This is CNA-specific hardening. Without the guard, a disposed `Texture2D` would leave a null renderer pointer for `GetRenderer()` to dereference.

13.12 DepthStencilState at Begin

Current contract. `Begin()` assigns the resolved `DepthStencilState`; a null argument becomes `DepthStencilState::None`. It does not inherit the state left by a preceding 3D draw. By contrast, `End()` does not restore the earlier `BlendState`, as described in §13.7.

Historical note. An earlier implementation accepted `depthStencilState` without assigning it to the device, so a sprite pass inherited prior 3D state. Applying the resolved state in `Begin()` closed that defect and aligned the null case with FNA.

13.13 Per-glyph advance math in DrawString

§13.8 above already named the raw ingredients — `kerning_`, a `Vector3` per glyph packing (`leftBearing`, `width`, `rightBearing`) — but `MeasureString` and `DrawString` are two independent implementations of the identical walk, in two different files (`SpriteFont.cpp` and `SpriteBatch.cpp` respectively), and it is worth confirming directly, rather than assuming, that they agree. Reading `SpriteBatch::DrawString` shows the same three-line shape, glyph by glyph:

```

1  const Vector3& cKern = spriteFont.kerning_[index];
2  if (firstInLine)
3  {
4      curOffset.X += std::abs(cKern.X);    // first glyph: ABS of left
        bearing
5      firstInLine = false;
6  }
7  else
8  {
9      curOffset.X += spacing_ + cKern.X;  // later glyphs: inter-char
        spacing + left bearing, no abs
10 }
11 curOffset.X += cKern.Y + cKern.Z;      // every glyph: width +
        right bearing

```

Only the first glyph applies `std::abs()` to its left bearing. This prevents a negative initial bearing from moving the whole string before `position`; later negative bearings can still tuck a glyph toward its predecessor. `MeasureString` (§13.8) applies the same rule, so measured and drawn placement agree.

The focused EasyGL fixture is `easygl_spritefont_multiglyph_spacing_test.cpp`. It exercises the non-first-glyph branch. Its two 8×8 glyphs have zero-valued kerning (0, 8, 0) and `spacing = 4.0f`, making placement checkable in whole pixels:

```

1  // 'A' (first in line): curOffset.X = abs(0) = 0          -> glyph
    drawn at dest x = 2 + 0 = 2
2  // after 'A':          curOffset.X += 8 + 0 = 8          (width +
    rightBearing)
3  // 'B' (not first):     curOffset.X += 4 + 0 = 12         -> glyph
    drawn at dest x = 2 + 12 = 14

```

Drawing "AB" at `position = (2, 2)` therefore places 'A' at screen $x \in [2, 10)$ and 'B' at $x \in [14, 22)$ — an exact 4-pixel gap at $x \in [10, 14)$ that must stay background-colored for the test to pass. A sample inside the gap establishes that `spacing` was applied, while each glyph's distinct fill color (white for 'A', green for 'B') at its expected position rules out an index mix-up between the two glyphs — a bug shape that a single-glyph test, no matter how many pixels it samples, structurally cannot detect at all, since there would be no second glyph to place at the wrong index.

13.14 Exception-type mismatch for nested `Begin()`

Calling `Begin()` a second time without an intervening `End()` — or calling `End()` before any `Begin()` at all — is a real, commonly-hit misuse case (a forgotten `End()` inside an early-return branch, most often), and CNA does throw for it. Reading the real implementation shows exactly what it throws, though, and it is not the type a reader would reasonably expect from real XNA:

```

1  if (begun)
2      throw std::runtime_error("Begin has been called before calling
                               End.");
3  // ...
4  if (!begun)
5      throw std::runtime_error("End was called, but Begin has not yet
                               been called.");

```

Real XNA's own `SpriteBatch.Begin()` throws `InvalidOperationException` for the identical misuse, and `System`'s own `InvalidOperationException` type exists in this codebase, real and used elsewhere (Chapter 16 and others cite it directly) — but `SpriteBatch` itself throws a plain `std::runtime_error` here instead, confirmed by reading both guard sites directly rather than assumed from the message text alone. The two exception hierarchies do not meet until `std::exception` itself: `InvalidOperationException` derives from `SystemException`, which derives from `System::Exception` — a hierarchy entirely separate from `std::runtime_error`'s own `std::exception` lineage, confirmed by reading `InvalidOperationException.hpp` / `SystemException.hpp` directly.

```

1  // Worked example: a porting engineer's reasonable, XNA-faithful
   // guard --
2  // this catch block never fires against SpriteBatch's own Begin()/
   // End()
3  // misuse, even though it would fire against the identical misuse on
   // many
4  // other CNA classes that do throw the real
   // InvalidOperationException.
5  try
6  {
7      spriteBatch_>Begin();
8      spriteBatch_>Begin(); // forgot the End() above -- a real,
   // easy mistake
9  }
10 catch (const System::InvalidOperationException& ex)
11 {
12     // Never reached: SpriteBatch::Begin() throws std::runtime_error
   // here,
13     // which this catch clause's type does not match at all.
14     LogAndRecoverGracefully(ex);
15 }

```

```
16 // The exception propagates past this handler entirely, caught only
    by a
17 // broader catch (const std::exception&) further up the call stack,
    if one
18 // exists -- or terminates the program if it does not.
```

This is a narrower, more precisely scoped gap than “wrong error message” or “doesn’t throw at all”: the throw itself is correct and reliable (a real, hittable misuse genuinely does terminate the batch and report a real diagnostic message), and any code written against a generic `catch (const std::exception&)` handler — the common, safe default in ordinary C++ — catches it exactly as expected, message text and all. The gap only bites a porting engineer who specifically wrote a typed `InvalidOperationException` catch clause around `SpriteBatch` usage, reasonably expecting the same exception hierarchy this book’s other chapters confirm many other CNA classes do use faithfully for the identical real-XNA exception type.

13.15 Where this leaves a porting engineer

`SpriteBatch` has broad cross-renderer coverage because each renderer must expose it. The evidence is not uniform: shared queue logic, renderer engagement, targeted pixel tests, and XNA-oracle comparisons establish different claims. It is a practical portability baseline for 2D ports, but the implementation-family matrix and open limitations above still apply.

Chapter 14

Textures and Render Targets

Texture and render-target APIs are shared, but their native formats, readback paths, mipmaps, multisampling, and attachment behavior differ substantially by renderer family. `Texture2D` has the broadest coverage. `Texture3D` and `TextureCube` now share the expected base type but retain narrower shader and renderer evidence. Render targets still have named gaps. This chapter keeps those scopes separate.

14.1 Texture: a shared Color gate with one renderer-specific branch

`Texture` (inheriting `GraphicsResource`) exposes `Format` (`SurfaceFormat`) and `LevelCount`, plus a set of static helpers ported from FNA's own internal size-calculation methods — made `public static` in CNA rather than FNA's `internal`, because three of the four real call sites in CNA are not even `Texture` subclasses. The base `ValidateFormat(format)` method accepts only `SurfaceFormat::Color`; every non-Skia build routes ordinary `Texture2D` and `RenderTarget2D` construction through that gate, and `Texture3D` plus `TextureCube` use it on every renderer.

The compile-time Skia product is the deliberate exception. Its `Texture2D` validator admits 26 of the 27 enum values and retains each promoted native layout, including compressed DXT/BC7 blocks; `Dxt5SrgbEXT` is the one refusal. Its render-target validator admits the 14 formats that FNA reports renderable. The exception is therefore specific to two 2D resource classes, not permission to infer broad format support from the enum alone. Separately, the portable `Texture2D::FromStream DDS` route recognizes DXT1/3/5 and CPU-decompresses them to RGBA8 before ordinary upload.

14.2 Texture3D and TextureCube: current structure and history

The pinned headers declare both `Texture3D` and `TextureCube` as `Texture` subclasses, as XNA does. Regression coverage assigns a `Texture3D` to `GraphicsDevice.Textures[slot]` and checks that disposal releases the renderer handle and unbinds the resource from the device's pixel and vertex texture collections.

Historical note. Earlier revisions did not derive these classes from `Texture`; assignment to a `TextureCollection` could not compile, and `Texture3D` skipped the base unbind-on-dispose path. An earlier manuscript pass described that state. The current contract above replaces it, while the regression preserves the reason for the change.

The practical consequence this earlier pass described — “a custom `ShaderEffect` has no way to

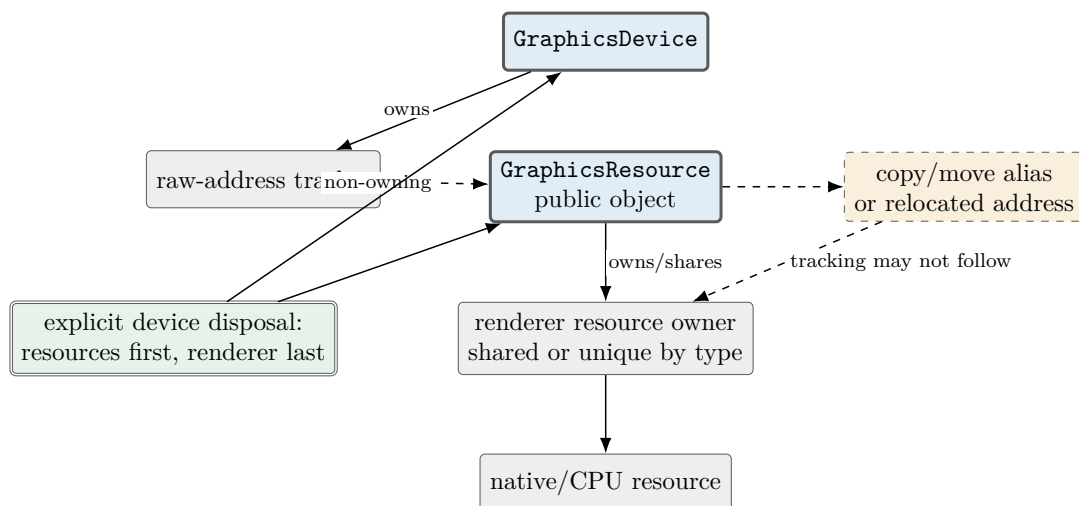


Figure 14.1: Resource ownership and address tracking at the pin. Value-like syntax does not supply CLR lifetime semantics; live copies and relocation require explicit care.

bind a `Texture3D` or `TextureCube`” — is also closed, and by a more direct route than the generic texture-slot mechanism the class-hierarchy fix alone would suggest: `ShaderEffect` gained dedicated `SetTexture(int, TextureCube&)` and `SetTexture(int, Texture3D&)` overloads, each binding straight to the matching GL texture target (`glBindTexture(GL_TEXTURE_CUBE_MAP, ...)` / `GL_TEXTURE_3D`) rather than routing through the ordinary 2D texture-slot path. This is EasyGL-only as of this writing — the other nine renderers were explicitly deferred, per the same task’s own scope note — so “a custom shader can sample a volume or cube texture” is currently a real, tested EasyGL capability, not yet a cross-renderer one. `EnvironmentMapEffect`’s own `EnvironmentMap` field, mentioned in an earlier pass as the *only* way any effect could reach a cube texture, is now one of two paths rather than the sole exception.

14.3 Texture2D

`Texture2D` is copyable and movable (its renderer is a `shared_ptr`-held `ITextureRenderer`), constructible from raw dimensions, from a file path (two `CNAEXT` overloads — real XNA has no such constructor, since real XNA loads textures exclusively through the compiled content pipeline), or via `FromStream` (device plus stream, or a five-argument form that resizes/crops to an explicit width, height, and zoom fit-vs-cover flag). `SetData` / `GetData` cover full-array and partial (level, rectangle, start index, element count) forms, all `Color`-only. `SaveAsPng` / `SaveAsJpeg` round out the surface, alongside several `CNAEXT` escape hatches (`GetRenderer()`, `GetCpuPixelsWeak()`, `CreateFromPixels`) that exist for interop with CNA’s own content-pipeline and testing code rather than for game code.

14.3.1 Building a texture from raw pixel data

The following procedural checkerboard uses the constructor and `SetData` / `GetData` signatures from `Texture2D.hpp`:

```

1  const int size = 64;
2  Texture2D checker(getGraphicsDeviceProperty(), size, size); //
    defaults to SurfaceFormat::Color
3
4  std::vector<Color> pixels(size * size);
5  for (int y = 0; y < size; ++y) {

```



```

6     for (int x = 0; x < size; ++x) {
7         bool light = ((x / 8) + (y / 8)) % 2 == 0;
8         pixels[y * size + x] = light ? Color::White : Color::Black;
9     }
10 }
11 checker.SetData(pixels.data(), static_cast<int>(pixels.size()));
12
13 // Read the CPU shadow back (this does not prove what the GPU
14   samples):
15 Color topLeft;
16 checker.GetData(&topLeft, 0, 1);

```

Two constraints from this chapter’s opening section apply directly to this example. The two-argument `Texture2D(device, width, height)` constructor defaults to `SurfaceFormat::Color`, so `Color` is the portable path across renderer families. On a non-Skia build, passing any of the other 26 `SurfaceFormat` values to the explicit-format constructor throws before upload. The public class does now provide typed `SetData/GetData` overloads for packed vectors, floats, vectors, half formats, bytes, and unsigned shorts; those routes become materially useful with Skia’s promoted native layouts. They are concrete overloads rather than XNA’s arbitrary `T[]` template.

14.3.2 Two SetData paths, and why GetData can hide a failed upload

The simple whole-texture call in the example and the detailed level/rectangle overload do not update the resource the same way. `SetData(Color*, count)` converts the complete level 0 image, replaces this object’s shared renderer reference, and calls `CreateTexture()` for a replacement resource. The detailed overload instead patches CNA’s CPU shadow first. At level 0 it sends the *whole reconstructed level* through `ITextureRenderer::UpdatePixels()`, even when the caller supplied a small rectangle; above level 0 it sends that level’s complete shadow through `UpdatePixelsLevel()`. This differs from FNA, whose detailed overload forwards the requested rectangle directly to `FNA3D_SetTextureData2D()`.

That CPU shadow changes what a test proves. An ordinary `Texture2D::GetData()` reads `cpuPixels_` or `extraMipLevels_`; it does not read the sampled GPU resource. The source file for `easygl_texture2d_mip_test.cpp` says this explicitly: its three-level “round trip” is pure CPU-shadow verification with no framebuffer readback. It would still return every color supplied by `SetData()` if the corresponding renderer update were an empty body. GPU sampling, a native renderer readback, or a rendered pixel is required to establish the other half.

That distinction exposes a live split. EasyGL, Vulkan, BGFX, WebGPU, and D3D9/11/12 allocate and update real `Texture2D` mip subresources. `SDL_RENDERER`, `ASCII`, `Canvas`, and `FREEDIRECT` reject a level-above-zero upload with `std::runtime_error`. Software explicitly discards it, Headless records the call without storing a renderer mip, and SDL GPU inherits the interface’s empty `UpdatePixelsLevel()` body while allocating only one physical level even when public `LevelCount` reports a full chain. In all three discard cases, the public CPU shadow still remembers the supplied level, so a following `GetData(level, ...)` can look successful while rendering can never consume it.

WebGPU has the opposite extra behavior: writing level 0 automatically regenerates every higher level with a real linear-filtered render-pass cascade. That prevents undefined mip content, but differs from FNA and means a later level 0 update overwrites any authored higher levels written earlier. Chapter 19, §19.3.3, gives the complete renderer and evidence matrix.

14.3.3 FromStream format support

Format detection in `FromStream` starts with a minimal internal DDS-header sniffer (recognizing only the DXT1/3/5 FourCC codes, decoding via an internal `DxtUtil`), and falls through to `SDL3_image`'s `IMG_Load_IO` — which auto-detects the container format from the byte stream itself — for everything else. Verified round-trip formats: PNG, JPEG (lossy, within tolerance), 24-bit uncompressed BMP, and DDS (DXT1/3/5). AVIF, TIFF, and WebP are present in the linked `SDL3_image` build's own dependencies but are not independently tested by CNA's own suite. The `zoom` parameter on the five-argument overload matters more than its name suggests: `zoom=false` uses a simplified largest-dimension-fit heuristic (not a general `min(w/w0, h/h0)` bounding-box fit — it assumes a roughly square target box), while `zoom=true` scales up and center-crops to exactly fill the requested box.

14.4 Texture3D and TextureCube

Beyond the structural deviation described above, `Texture3D` and `TextureCube` are both non-copyable (each owns a unique GPU handle) but movable — `Texture3D` specifically gained move construction/assignment to satisfy a glTF content-pipeline reader's needs. Their requested `SurfaceFormat` is not silently ignored: the shared `Texture::ValidateFormat()` gate rejects every value except `SurfaceFormat::Color` before a renderer is created, so every surviving resource is RGBA8 by construction.

Neither type keeps a `Texture2D`-style CPU pixel shadow. Every valid `GetData()` on a constructed renderer therefore reaches that renderer's own readback path. The old support document's claim that only EasyGL really implements this is now stale: EasyGL, Vulkan, BGFX, WebGPU, SDL GPU, and all three Direct3D renderers have real GPU readback for both resource types. Their mechanisms differ — framebuffer reads, staging copies, native lock/map calls, or a temporary BGFX transfer texture — but all eight return actual sub-region bytes. Software adds a real level-0 CPU implementation for `TextureCube`, while its `Texture3D` factory is absent. Headless deliberately returns transparent-black placeholders. Chapter 19's §19.3.3 gives the full factory/readback matrix, including the separate and much narrower `RenderTargetCube` result.

Authored mip-level upload and readback are likewise real on both ordinary resource types across those eight GPU renderers. Exact public mip round trips are registered for EasyGL, Vulkan, BGFX, WebGPU, and SDL GPU; the Direct3D implementations allocate and address mip subresources too, although their cited smoke-test readbacks concentrate on discriminating level-0 subregions. This statement is about explicitly supplied mip data, not a promise that every renderer automatically generates a mip chain after a level-0 upload. `CubeMapFace` validation, by contrast, is a place CNA is *stricter* than FNA: an out-of-range face throws `std::out_of_range` at the public API layer, a safety check real XNA never performs at all.

14.4.1 Six faces, six SetData calls

`TextureCube`'s constructor takes a single face size (every face is square and identical in size, per the real XNA contract) plus `mipMap` and `format` — and, per this chapter's own opening finding, `format` must be `SurfaceFormat::Color` or construction throws immediately. Populating a solid-color test cube (a stand-in for a real skybox's six photographic faces) is a direct, mechanical loop over `CubeMapFace`'s six enumerators, since `SetData` takes the face as its first argument:

```
1  const int faceSize = 128;
2  TextureCube skybox(getGraphicsDeviceProperty(), faceSize, /*mipMap=
    */false,
```

```

3             SurfaceFormat::Color);
4
5  const CubeMapFace faces[6] = {
6      CubeMapFace::PositiveX, CubeMapFace::NegativeX,
7      CubeMapFace::PositiveY, CubeMapFace::NegativeY,
8      CubeMapFace::PositiveZ, CubeMapFace::NegativeZ,
9  };
10 const Color faceColors[6] = {
11     Color::Red,    Color(128, 0, 0, 255),    // +X, -X
12     Color::Green,  Color(0, 128, 0, 255),    // +Y, -Y
13     Color::Blue,   Color(0, 0, 128, 255),    // +Z, -Z
14 };
15
16 std::vector<Color> pixels(faceSize * faceSize);
17 for (int i = 0; i < 6; ++i) {
18     std::fill(pixels.begin(), pixels.end(), faceColors[i]);
19     skybox.SetData(faces[i], pixels.data(), static_cast<int>(pixels.
20         size()));
21 }

```

This loop can now be verified with a real `GetData()` round trip on any of the eight GPU renderers above, rather than only on EasyGL as the stale support document still says. The strongest reusable checks deliberately distinguish slices, faces, off-origin regions, and mip levels; they do not merely assert that the call returned. Software can make the same check at level 0 for a cube. `skybox` can also be bound to a custom `ShaderEffect` on EasyGL, per §14.2 above — either through `SetTexture(int, TextureCube&)` directly, or, since `TextureCube` now inherits `Texture`, through the ordinary `GraphicsDevice.Textures[slot]` assignment every 2D texture in this chapter already uses.

14.4.2 Sampling a volume texture from a custom shader

`modules/renderers/easygl/examples/easygl_shadereffect_texture3d_test.cpp` is the real, EasyGL-only test that proves `ShaderEffect::SetTexture(int, Texture3D&)` genuinely reaches a `sampler3D` uniform, not just that the call compiles. The volume is deliberately tiny — a $1 \times 1 \times 2$ texture, one texel per Z-slice — so the whole capability proof reduces to two texel reads instead of a real ray-marching setup:

```

1  Texture3D volumeTex(device, 1, 1, 2, /*mipMap=*/false, SurfaceFormat
2      ::Color);
3  const Color slices[2] = {
4      Color(255, 0, 0, 255), // Z=0
5      Color(0, 0, 255, 255), // Z=1
6  };
7  volumeTex.SetData(slices, 2);
8
9  auto* fx = dynamic_cast<ShaderEffect*>(fxBase.get());
10 fx->Apply();
11 fx->SetTexture(0, volumeTex);
12 fx->SetUniformInt("VolumeSampler", 0);
13 fx->SetUniformVec3("coord", 0.5f, 0.5f, 0.25f); // samples Z-slice
14     0's texel centre

```

Sampling at $Z = 0.25$ and $Z = 0.75$ — each slice's own texel centre, per $(\text{sliceIndex}+0.5)/\text{depth}$ — reads back $(255,0,0,255)$ and $(0,0,255,255)$ respectively, even with `GL_LINEAR` filtering on both axes: linear interpolation sampled exactly at a texel centre puts its full weight on

that one texel, so there is no blending artifact to account for. The real test’s own false-positive guard is worth naming as a technique in its own right, since it recurs throughout this book’s worked examples: a second, all-black “decoy” `Texture3D` is created and uploaded *after* the real one, so an unmutated `SetTexture()` call would leave GL’s texture-unit-0/GL_TEXTURE_3D binding pointing at the decoy rather than the intended volume — the two-slice colour check only passes for the right reason (a genuine, per-call rebind) if disabling the `SetTexture()` call would make it fail.

14.4.3 Rendering into one `RenderTargetCube` face

`GraphicsDevice` has a dedicated overload for exactly this purpose — `SetRenderTarget(RenderTargetCube*, CubeMapFace)` — distinct from the plain `SetRenderTarget(RenderTarget2D*)` form used for ordinary 2D targets, and from the `SetRenderTargets(const std::vector<RenderTargetBinding>&)` form used for simultaneous multi-target binding (`RenderTargetBinding`’s own `(Texture*, CubeMapFace)` constructor exists specifically to build entries for that vector form, not for single-target binding). Rendering a small scene into a single cube face and then sampling that cube in a later draw is the pattern a real-time reflection probe or a baked-environment-map tool would use:

```

1  RenderTargetCube probe(getGraphicsDeviceProperty(), 256, /*mipMap=*/
    false,
2                                SurfaceFormat::Color, DepthFormat::
    Depth24Stencil8);
3
4  device.SetRenderTarget(&probe, CubeMapFace::PositiveZ);
5  device.Clear(Color::CornflowerBlue);
6  // ... draw the scene as seen looking down +Z from the probe's
    location ...
7  device.SetRenderTarget(nullptr); // back to the backbuffer

```

Sampling and CPU readback are separate capabilities here. The code above can feed the cube to an effect without ever calling `probe.GetData()`; nevertheless public rendered-face readback is now exact on EasyGL, Vulkan, BGFX, WebGPU, SDL GPU, and D3D9/11/12. Skia provides a separate exact six-surface CPU emulation. Unsupported identities now throw instead of converting a renderer no-op into fabricated transparent black. Mip/MSAA scope still differs — notably D3D9 has only level 0 and WebGPU refuses a mipmapped cube target — so the central matrix in §19.3.3 remains the authoritative per-level comparison.

The inherited `SetData` family has narrower renderer support still. It is legal at compile time because `RenderTargetCube` inherits the whole `TextureCube` upload family, exactly as it does in FNA. EasyGL implements the GPU upload with its documented row convention, and Skia implements exact CPU-surface upload. Every other constructed target returns failure from the renderer contract; the public call validates first and then throws `NotSupportedException` instead of returning normally after changing nothing. Thus the feature remains narrow, but the former silent compatibility gap is closed.

The unbind line now routes correctly on Vulkan, WebGPU, SDL GPU, BGFX, D3D9, D3D11 and D3D12. The Direct3D 11/12 renderers explicitly track the active cube and finalize it before every cube/2D/MRT/backbuffer transition; shared public readback, usage and per-face-MSAA fixtures make that edge observable. EasyGL retains a narrower exception: switching directly between faces of the same cube skips outgoing-face finalization. Per-face MSAA storage now preserves each face’s samples, but only a finalized face is resolved/generated for later sampling; insert a backbuffer unbind between freshly rendered faces when all must immediately

be sampled. The full transition matrix and BGFX’s renderer-local (not normalized public) cube-null no-op are recorded in §19.3.3.

Whether this actually produces a correct image is, per §14.5 below, renderer-dependent in ways the call site cannot infer. EasyGL renders it correctly. Vulkan’s two formerly-tracked black-render defects (Tasks 875/876) are now both closed: clear-only targets enter the used-target list, and the original cube-sampling regression test passes repeatedly end to end, although the exact incidental change that retired the second defect was not bisected. BGFX’s former unconditional `static_cast` type-confusion bug is likewise fixed by shared cube-sampling interfaces and virtual handle dispatch. The remaining transition, same-frame, and upload qualifications above still apply; “Vulkan cube sampling is broken” no longer does.

14.5 Render targets: a once-sharp per-renderer divergence, mostly closed since

`RenderTarget2D` inherits both `Texture2D` and `IRenderTarget`, which is what lets it be both drawn into and sampled as an ordinary texture afterward; its `Dispose(bool)` correctly throws `InvalidOperationException` if it is still bound to the device, matching FNA. An earlier explanation for `RenderTargetCube`’s missing equivalent guard is now stale: `RenderTargetCube` does inherit `TextureCube` and `IRenderTarget`, while `TextureCube` itself inherits `Texture`, so `RenderTargetBinding` can store it through a `Texture*`. The guard is still absent for a different reason. The singular cube-target setter clears `currentRenderTargets_` and retains only a boolean “some target is bound,” not the cube pointer; `RenderTargetCube` supplies no `Dispose(bool)` override of its own to compensate. The compatibility gap remains, but the former class-hierarchy explanation does not.

14.5.1 `RenderTargetUsage`: three values, more than three behaviors

`RenderTargetUsage` has the expected three ordinals: `DiscardContents`, `PreserveContents`, and `PlatformContents`. `DiscardContents` says old attachment data need not survive a bind; it does *not* promise a readable replacement color. `PreserveContents` requests survival even at a performance or memory cost. `PlatformContents` permits survival only when cheap. In current FNA, both non-discard values are passed to the native layer as a preserve request, while the exact enum still controls whether the managed layer issues its discard clear.

CNA now centralizes those decisions in `RenderTargetUsagePreservesContentsEXT()`: only `Discard` maps to false, while `Preserve` and `Platform` both map to true, matching FNA. `RenderTarget2D` and `RenderTargetCube` pass that Boolean through their renderer factories, and the shared normalized bind route emits the deterministic black/max-depth/zero-stencil clear only for `Discard`. An explicit `Clear()` remains a distinct ordered operation and wins over the usage policy aspect by aspect.

Cube targets now use the same binding path rather than storing only an enum beside an untracked face. A `RenderTargetBindingDescriptor` preserves the selected face, dimensions, applied sample count, and renderer pointer; usage reaches the cube factory, and the shared discard clear can inspect the cube’s real depth/stencil attachments. The remaining differences are native mechanisms and renderer capability — for example WebGPU rejects MRT — not a missing cube usage parameter.

The backbuffer’s corresponding `PresentationParameters.RenderTargetUsage` is currently stored, cloned, and exposed only. Returning to the backbuffer never consults it. FNA, by contrast, supplies it to native target binding and uses it to choose the unbind-to-backbuffer discard clear. The first 2D target’s usage controls CNA’s MRT implicit clear, which matches

FNA’s first-target policy in shape, but deferred Vulkan MRT passes discard regardless and the public plural cube form cannot reach a cube renderer at all.

Current evidence is substantially broader than the early non-MSAA 2D tests. Shared cube-usage, per-face MSAA preservation, depth/stencil usage, pass-boundary, and plural-cube fixtures are registered across capable renderers; Platform is asserted as preservation rather than left implicit. Backbuffer usage and redundant-binding elision remain separate gaps: returning to the backbuffer still ignores its presentation usage, and CNA still performs native/state work for an identical repeated binding instead of FNA’s early return. The complete source/evidence matrix is §19.4.1; the binding-identity consequences are §12.4.1.

The render-target support ledger once listed five further per-renderer divergences here (Tasks 877–881) as open gaps, and an earlier pass of this chapter reproduced that list faithfully. Reading every renderer’s own current `.cpp` directly instead of trusting that document’s status column finds four of the five already closed — each superseded by a later task (903, 907, or 911) whose own fix comments the tracking document was never updated to reflect, the identical “a status that looked settled turned out to be stale” pattern Chapters 22 and 23 both ran into independently while auditing this exact same document for their own renderers. What is actually still open, and what is now fixed:

Cube sampling. Sampling a `RenderTargetCube` face after rendering into it, then unbinding, works correctly on **EasyGL**. **Vulkan** now works in both discriminating regressions: the clear-only case explicitly records the target, while the original render-all-faces/then-sample test no longer reproduces the black result (the exact fixing commit was not retroactively isolated). **BGFX**: fixed — the unconditional `static_cast` that once read a framebuffer-pool handle where a texture-pool handle was expected (Chapter 24 names the exact mechanism) is gone, replaced by a real `dynamic_cast` to a shared cube-samplable interface for the `EnvironmentMapEffect` path and virtual width/height dispatch for the ordinary `SpriteBatch` path.

Depth/stencil format fidelity. Now real on all three renderers: each maps the actual requested `DepthFormat` ordinal to the correct native format (GL internal format plus attachment point on EasyGL; a per-instance Vulkan format; bgfx’s own format enum), rather than the single, always-the-same format every renderer once allocated regardless of what was asked for.

Mipmap generation. **EasyGL** already worked — real per-level GPU storage, auto-regenerated via `glGenerateMipmap` on unbind, matching FNA3D’s own real mechanism. **Vulkan** and **BGFX** now do too: a real, per-level mip-chain regeneration cascade (`vkCmdBlitImage` on Vulkan; bgfx’s own equivalent) runs against a render target’s just-rendered content once its render pass ends, rather than silently accepting and discarding the request the way both once did.

Multisample antialiasing. **EasyGL** already worked: a real multisampled renderbuffer with a `glBlitFramebuffer` resolve on unbind. **Vulkan** and **BGFX** now genuinely implement it too, rather than honestly reporting `MultiSampleCount = 0` the way both once correctly did while the feature was still unimplemented.

Two further, project-wide findings from the same original list are also now closed, not open. Switching the active render target correctly resets the shared public `Viewport` and `Scissor Rectangle` values to the new target’s (or the backbuffer’s) own dimensions on every renderer; whether those setters then change native rasterization is the separate current renderer contract in §19.4.3. `GraphicsDevice.Viewport` itself now has real GPU wiring on all three hardware renderers investigated — EasyGL applies a custom sub-region `Viewport` on its ordinary 3D

paths whether or not a render target is currently bound (`SpriteBatch` is the documented exception), while Vulkan’s own version of the identical fix is honestly narrower, backbuffer-pass-only, for an architectural reason Chapters 23 and 22 both cover in full. And `SetRenderTargets()` now genuinely enforces FNA’s real four-target cap at the shared, cross-renderer level — throwing if a caller passes more than four bindings, regardless of what any individual renderer’s own internal buffer happens to be sized for — closing what this section once described as no renderer matching that real XNA limit at all.

14.6 SurfaceFormat, without flattening the Skia exception

The 27-value `SurfaceFormat` enum describes a wider contract than most configured products implement. For 41 renderer families, ordinary texture construction still accepts only `SurfaceFormat::Color`; compressed content loading commonly becomes RGBA8 before the GPU. Skia is the bounded exception: 26 formats have native `Texture2D` storage and typed transfer routes (all except `Dxt5SrgbEXT`), while only 14 are admitted for `RenderTarget2D`. Volume and cube textures remain Color-only even there. A format enumerator, a transferable CPU representation, and render-target capability are therefore three separate claims.

One real, historical bug is worth naming because of what it reveals about how subtle a graphics-correctness bug can be: Vulkan’s texture path once created images as `VK_FORMAT_R8G8B8A8_SRGB` (wrong — `SurfaceFormat::Color` is linear, not sRGB) while the swapchain *separately* preferred a matching sRGB format. The two wrong encode/decode steps approximately canceled out for ordinary textured content, which is exactly why the bug went unnoticed for a long time — it only became visible on non-textured content (flat vertex colors, raw lighting output), where a mid-gray value of 128 was read back as 188. Both were corrected to their `_UNORM` equivalents.

14.7 Buffers: VertexBuffer and IndexBuffer

`VertexBuffer` and `IndexBuffer` both inherit `GraphicsResource` and both maintain a private `cpuShadow_` byte buffer purely to support `GetData()` without requiring a real per-renderer GPU readback path — since nothing in CNA’s own pipeline ever writes GPU-side data back into one of these buffers, a CPU-side shadow copy is a faithful, sufficient implementation of read-your-own-writes semantics. `VertexBuffer::SetData / GetData` are typed per concrete vertex struct — `VertexPositionColor`, `VertexPositionColorTexture`, `VertexPositionNormalTexture`, and `VertexPositionTexture` — plus a CNAEXT skinned variant and a raw `SetDataRaw(data, count, stride)` escape hatch for layouts with no dedicated XNA struct. `DynamicVertexBuffer` and `DynamicIndexBuffer` add `IsContentLost` (always `false`) and `SetData(..., SetDataOptions)` overloads. One half of the chapter’s old simplification is still true: the protected `VertexBuffer` and `IndexBuffer` constructors receive a `dynamic` boolean from the two derived classes but deliberately leave the parameter unnamed and never pass it to a renderer factory. Choosing `Dynamic*` therefore does not itself select different storage. D3D9 and D3D11 allocate their ordinary and dynamic buffers through the same always-dynamic native path; BGFX likewise uses dynamic handles for both. The *per-upload* hint is no longer universally ignored, however:

- EasyGL maps `Discard` to buffer orphaning plus a new sub-data upload and `NoOverwrite` to an in-place sub-data update. Its vertex path makes this distinction whenever storage already exists. Its index path has one surprising coupling: when context recovery is disabled, no renderer CPU shadow is retained, so its `NoOverwrite`-eligibility check fails and falls back to a fresh data allocation.

- SDL GPU’s native `cycle` flag is true for `None` and `Discard`, but false for `NoOverwrite`. D3D9 uses the corresponding `DISCARD/NOOVERWRITE` lock flags once storage exists (a fresh allocation forces discard). D3D11 uses the corresponding `WRITE_DISCARD/WRITE_NO_OVERWRITE` map modes.
- Vulkan and BGFX inherit the interface default, which discards the option and invokes plain `SetData`. WebGPU and D3D12 explicitly receive but ignore it before their queue-write or synchronous staging-copy path. Software and Headless likewise replace their CPU/trace data without interpreting the hint. The four 2D-only renderers reject buffer construction before any of these methods can be reached.

The real `easygl_dynamic_buffer_stress_test.cpp` exercises the distinction through the public classes rather than calling a renderer directly. Across twelve frames it cycles `None`, `Discard`, and `NoOverwrite`, uploads a differently colored six-vertex quad and six indices, draws, then reads the center pixel and checks the two public capacities. That is a genuine pixel proof that all three routes preserve current data through repeated use. It is not, by itself, proof of which native allocation primitive ran; the orphan/sub-data distinction is established by the implementation read above.

14.7.1 A shared readback hole in the options-taking path

That audit exposes a separate bug above every renderer. The ordinary typed `VertexBuffer::SetData` and `IndexBuffer::SetData` overloads update the shared `cpuShadow_` after uploading. None of the protected `SetDataWithOptions` overloads does. A `DynamicVertexBuffer` or `DynamicIndexBuffer` created with `BufferUsage::None` can therefore inherit `GetData`, but after its normal options-taking `SetData` call the shared shadow is still empty (or stale after an earlier ordinary upload): a fresh buffer throws `ArgumentOutOfRangeException` instead of returning the bytes just written. `BufferUsage::WriteOnly` still throws `NotSupportedException` by design; that documented restriction does not excuse the broken non-write-only case. This is shared public code, so changing graphics renderer cannot avoid it.

14.8 Vertex layout: fixed by stride, except for EasyGL raw declarations

The ordinary typed paths still choose fixed GPU layouts from the uploaded byte stride. The current project-wide family has grown from five to eight shapes: 16, 20, 24, and 32 bytes for the four stock XNA vertex types; 48 for CNA’s PBR tangent layout; 52 for GPU skinning; 56 for the same skinned data plus vertex color; and 68 for tangent-space PBR plus skinning. Exact shader/effect combinations remain renderer-specific, so recognizing a stride is not a promise that every effect can consume it.

There is now one deliberate exception to the stride-only rule. `VertexBuffer::SetDataRaw()` pushes the buffer’s own `VertexDeclaration` element list through `IVertexBufferRenderer::SetVertexDeclaration()` immediately before upload. That interface method defaults to a no-op, and EasyGL is currently its only override. EasyGL maps all twelve XNA element-format values to the correct GL attribute shape and uses the declaration’s element order as GLSL locations 0, 1, ...; the semantic `VertexElementUsage` does not choose the location.

The pixel proof is stronger today than its original “non-standard stride” description. `easygl_shadereffect_custom_vertex_layout_test.cpp` supplies five elements in a 48-byte record: three `Vector3` fields, one `Vector2`, and a trailing normalized `Color`. Forty-eight bytes is now also the size of CNA’s fixed PBR layout, but that layout uses a four-float tangent and has no trailing color. The test encodes normal, tangent, UV, and color inputs into the output

pixel; its two distinct, tolerance-bounded readbacks therefore prove that EasyGL followed the declaration's offsets rather than merely selecting its different fixed 48-byte case.

The practical rule is consequently renderer-specific. An arbitrary **VertexDeclaration** is a real EasyGL+**ShaderEffect** capability when uploaded through the **CNAEXT SetDataRaw** path. Every other renderer still discards that declaration at the interface default and relies on its known stride/effect combinations; **SDL_Renderer** and the other 2D-only renderers reject vertex-buffer construction outright.

Chapter 15

Stock Effects and Draw Parameters

XNA shipped five built-in shader effects, and CNA ports all five. This chapter covers the shared `Effect` base and its supporting types, then each stock effect, and finally the recurring cross-renderer defect patterns exposed by their verification.

Many histories below come from the project’s original EasyGL/Vulkan/BGFX comparison campaign. “All three” means those three measured implementations, not all 46 current renderer families; closed defects are retained as mutation-tested evidence, not presented as present limitations.

15.1 Effect: stock and compiled modes

`Effect` inherits `GraphicsResource` and is not copyable. The `Effect(device)` constructor creates the small stock-effect base graph. The byte-buffer constructor now accepts a bounded, structurally validated XNA/FNA Direct3D 9 Effect Framework binary — the `.fxb`-shaped payload normally stored by XNA’s general `EffectReader`. It rejects an empty or over-64-MiB buffer, malformed container graphs, and the distinct MonoGame `MGFX` container. It then requires `GraphicsCapability::CompiledEffects`; a renderer that did not opt in throws `NotSupportedException` before any draw.

FNA3D supports this route in its normal build. `SDL_GPU`, the EasyGL profiles, and Vulkan support it only when their respective `CNA*_COMPILED_EFFECTS` option is enabled; those options are off by default because they add MojoShader. Other renderers report the capability false. This is compiled Effect Framework support, not a compiler for `.fx` or HLSL source, not DXBC or MGFX ingestion, and not the renderer-specific `ShaderEffect` API. Chapter 17 separates the formats and backends.

`Clone()` is virtual and, as a CNAEXT deviation from FNA’s garbage-collected reference semantics, returns an owning raw pointer rather than a managed reference — there is no GC in C++ to hand a shared reference back to. `Apply()` itself is CNAEXT — real FNA has no public `Effect.Apply()`, only `EffectPass.Apply()` — and the protected virtual `FillGpuDrawParams(GpuDrawParams&)` is the single mechanism every stock effect below uses to hand its own parameters down to whichever renderer is active; the base implementation is a no-op, and every stock effect overrides it.

`EffectParameter` carries a name, semantic, row/column counts, class/type enums, and a broad family of typed `GetValue*` / `SetValue*` accessors. In a compiled effect, those records are reflected from the binary and mutable values synchronize to the renderer runtime before a pass is applied. In a stock effect, the collection remains the narrower manual shadow described in §15.2. The parameter, pass, and technique collections all store elements behind `unique_ptr` rather than by value, so a reference obtained earlier — including a captured `CurrentTechnique` — survives a later `Add()` reallocation. `EffectPass::Apply()` throws `InvalidOperationException`

if an owned pass does not belong to the effect’s currently selected technique, including when `CurrentTechnique` is null — matching FNA’s own “Applied a pass not in the current technique!” guard, mapped onto a defined C++ exception rather than the null-reference crash the equivalent null case produces in C#.

15.2 Stock parameters are not compiled-effect reflection

The XNA-looking `Parameters` property needs a mode-qualified reading. A compiled effect reflects parameters, techniques, passes, annotations, arrays and structures from its binary; current technique and mutable values are copied by `Clone()`, and pass application pushes the renderer’s blend, depth/stencil, rasterizer, sampler, texture and shader state changes through the device. A bare stock-mode `Effect`, by contrast, starts with one manually created `Default` technique containing one P0 pass and with *zero* parameters.

The five stock effects do not all even present the same structural approximation. `BasicEffect` adds no parameter records at all, even though real FNA’s compiled `BasicEffect` exposes its parsed parameter table:

```
1 auto& parameters = basic.getParametersProperty();
2 EffectParameter* diffuse = parameters["DiffuseColor"]; // nullptr
```

`AlphaTestEffect`, `DualTextureEffect`, `EnvironmentMapEffect`, and `SkinnedEffect` each call a private `CacheEffectParameters()` helper instead. It manually creates a small set of internal shadows: respectively 6, 5, 12, and 12 records for the derived material, fog, matrix, and shader-index calculations. Those lists are neither a bytecode reflection result nor a full public description of the effect; for example, the manually cached `AlphaTest` and `DualTexture` lists contain no texture parameter despite each effect having a texture property.

The normal render flow is consequently one-way:

```
1 stockEffect.setDiffuseColorProperty(tint); // authoritative effect
   field / dirty bit
2 for (EffectPass& pass : stockEffect.getCurrentTechniqueProperty()->
   getPassesProperty()) {
3     pass.Apply(); // OnApply may refresh
   internal shadow records
4     graphicsDevice.DrawIndexedPrimitives(/* ... */);
5     // GraphicsDevice calls stockEffect.FillGpuDrawParams(), which
   reads the effect fields.
6 }
```

Writing a manually found `EffectParameter` therefore does not form a portable alternative to a stock effect’s named property and does not create an arbitrary renderer uniform. The device dispatches `FillGpuDrawParams()` directly, not the collection. A few properties deliberately use a shadow as their own storage — fog color is the important example in the four effects that cache it — but that is an implementation detail that a later property assignment or `OnApply()` calculation can overwrite. Use the public `setDiffuseColorProperty`, `setTextureProperty`, matrix, lighting, fog, and skinning APIs for stock material state; use the explicit `ShaderEffect::SetUniform*` surface, with Chapter 17’s renderer limits, for a custom uniform.

15.2.1 What an individual parameter retains

An `EffectParameter` stores independent C++ float, integer, string, and texture-pointer caches. Its metadata does not check that a requested setter matches the declared class, type, shape, or

count; it is descriptive data, not runtime validation. Array getters return at most the currently stored entries rather than necessarily the requested count. A negative request is caller-invalid: vector/matrix/quaternion and Boolean getters happen to yield an empty result, while the integer and float array implementations form an invalid iterator range. No focused test covers this mismatch or negative-count path.

The texture overloads deserve the same caution. CNA keeps a separate raw slot for each texture overload. Calling `SetValue(Texture*)` therefore does not populate the matching typed texture getter, even when the dynamic object has that derived type; there is no generic texture getter. FNA has one texture reference which its typed getters cast. CNA also accepts and returns strings where FNA's `SetValue(string)` explicitly throws `NotImplementedException`. None of these pointers retains a texture, and none is a GPU binding instruction. The standalone parameter tests cover same-overload scalar/vector/matrix/quaternion/texture round trips and defaults, but not metadata mismatch, truncation, negative counts, generic-texture behavior, lifetime, stock-effect linkage, or rendering.

15.2.2 Pass selection still matters

`Effect::Apply()` is a useful CNA-only convenience: it calls `OnApply()` and makes the effect current on the device even if `CurrentTechnique` is null or names an unrelated technique. It does not itself select or validate a pass. The familiar loop above is safe because `EffectPass::Apply()` compares its owning technique's stable identity with the current one before reaching that convenience method. The public C++ constructors also allow an ownerless pass; applying such a pass simply returns without setting any effect, where FNA constructs passes internally. Keep application code on the current technique's own passes and do not use null/unrelated techniques or ownerless manually assembled passes as a configuration API.

15.3 GpuDrawParams: the shared stock-effect draw packet

Every stock effect's `FillGpuDrawParams` override populates the same struct, defined once in the renderer contract layer (Chapter 19) and forwarded from `Effect::FillGpuDrawParams()` to whichever renderer is currently active. It carries two ordinary texture slots plus a dedicated environment-map cube slot, diffuse/ambient/emissive/specular colors, three directional lights (each with direction, diffuse, and specular), the world matrix, alpha-test parameters, Fresnel/environment-map parameters, a 72-bone skinning palette with a weights-per-vertex count, fog parameters, a set of mode flags (`textureEnabled`, `vertexColorEnabled`, `lightingEnabled`, `dualTexture`, `envMapping`, `skinned`, `pbr`), instancing fields, a pointer for a custom `ShaderEffect`-compiled program, and six PBR-specific metallic-roughness fields. Two flags are worth flagging directly: `preferPerPixelLighting` and `specularEnabled`. Earlier project documentation called both flags "ignored by every renderer except D3D9." That statement is historical; the current scoped result appears after `SkinnedEffect` below.

15.4 BasicEffect

`BasicEffect` is the largest of the five: public `World / View / Projection` fields, `VertexColorEnabled`, the full material color set (`DiffuseColor`, `EmissiveColor`, `SpecularColor`, `SpecularPower`, `Alpha`, `AmbientLightColor`), `LightingEnabled / PreferPerPixelLighting`, `TextureEnabled / Texture`, full `DirectionalLight0/1/2`, and `EnableDefaultLighting()`. Its 22-property audit found and fixed several early default-value bugs (`VertexColorEnabled`, `DirectionalLight0.Enabled`, and `DirectionalLight.Direction`'s default value were all initially wrong), then a sequence of real, per-renderer rendering bugs: the no-texture

`VertexColorEnabled` toggle was ignored by all three of EasyGL, Vulkan, and BGFX (three separate bugs, one per renderer); the textured-plus-vertex-color-plus-diffuse case dropped `DiffuseColor` entirely on EasyGL and BGFX (Vulkan was already correct); and a shared bug across all three renderers never checked `DirectionalLight0.Enabled` at all, so a disabled light kept lighting the surface regardless. BGFX carried one additional, much wider bug: its vertex-layout builder never declared `Normal` / `TexCoord0` attributes for any stride except the skinned 52-byte layout — silently breaking lit-normal *and* UV interpolation for every other stride, invisible until then because every prior test happened to use a UV-insensitive 1×1 texture. `EmissiveColor` was dropped entirely in the no-lighting path on all three renderers; ambient/emissive/specular forwarding and specular highlights were corrected afterward. The focused composition fixture then produced byte-identical output on EasyGL, Vulkan, and BGFX at the pinned revisions; this result does not generalize to untested effect combinations or renderer families.

15.4.1 Lit, textured cube

`BasicEffect` is deliberately the effect a first 3D CNA program reaches for, and its own real headers confirm why it reads like plain XNA rather than a wrapped shader system: `World` and `VertexColorEnabled` are ordinary public fields (matching real XNA's own field-not-property choice for this specific effect), while `Texture` and `TextureEnabled` go through the usual `getXProperty` / `setXProperty` pair:

```

1  BasicEffect effect(getGraphicsDeviceProperty());
2  effect.World = Matrix::CreateRotationY(rotationAngle);
3  effect.setViewProperty(camera.View());
4  effect.setProjectionProperty(camera.Projection());
5
6  effect.setTextureEnabledProperty(true);
7  effect.setTextureProperty(&cubeTexture);
8
9  effect.EnableDefaultLighting();           // sets DirectionalLight0
      //1/2 to XNA's own defaults
10 effect.DirectionalLight0.setDiffuseColorProperty(Vector3(1.0f, 0.95f
      , 0.9f)); // warm key light
11
12 for (EffectPass& pass : effect.getCurrentTechniqueProperty()->
      getPassesProperty()) {
13     pass.Apply();
14     getGraphicsDeviceProperty().DrawIndexedPrimitives(/* ... */);
15 }
```

`EnableDefaultLighting()` is worth calling before touching any individual `DirectionalLightN` property, not after — reading its own implementation directly confirms it unconditionally overwrites `DirectionalLight0`'s diffuse color, direction, specular color, and enabled flag (and likewise for lights 1 and 2) every time it runs; setting `DirectionalLight0`'s diffuse color first and calling `EnableDefaultLighting()` second would silently overwrite the custom color right back to the default. This is exactly the kind of ordering trap the per-renderer bug list below does *not* cover, because it is a real API-usage hazard rather than a renderer bug — `EnableDefaultLighting()`'s own real behavior, faithfully ported, not a CNA-specific quirk.

15.5 AlphaTestEffect

`AlphaTestEffect` implements `IEffectMatrices` and `IEffectFog` only — no lighting. `AlphaFunction` (a `CompareFunction`, defaulting to `Greater`) and `ReferenceAlpha` (an

unclamped 0–255 int) implement the actual alpha test; `VertexColorEnabled`, `DiffuseColor` / `Alpha`, and `Texture` round out the surface. Its audit found zero property-default bugs (the first test coverage for this effect written from scratch), and a full 8-value `CompareFunction` sweep passed 24 of 24 cases on the three campaign renderers. The campaign initially found that `VertexColorEnabled` had no effect on Vulkan or BGFX because their alpha-test pipeline declared no color attribute. The correction added stride-24 sibling vertex shaders and discriminating pixel tests on both products; EasyGL was already correct. Fog forwarding was a real, fixed bug on EasyGL specifically for this effect — and, in the course of fixing it, revealed a much larger finding: fog was a *total* no-op on both Vulkan and BGFX for every 3D effect, because no shader file on either renderer mentioned fog at all. That project-wide gap was closed in one pass, adding real fog uniforms, varyings, and blend formula to every 3D shader on both renderers at once — the same “one shared fix closes it everywhere” pattern already seen for `SpriteBatch`’s flip bug in Chapter 13. A separate, unrelated BGFX-only bug — seven texture-binding call sites with no fallback-to-white-texture else-branch — was found and fixed at the same time, affecting every effect that binds a possibly-null texture, not just this one.

15.5.1 Cutout-alpha chain-link fence

The classic real use of `AlphaTestEffect` is a cutout texture — foliage, a chain-link fence, a wire mesh — where a pixel is either fully opaque or fully discarded, with no blended edge at all (unlike ordinary alpha *blending*, which this effect deliberately does not do). `AlphaFunction` and `ReferenceAlpha` together express the test as a comparison against the texture’s own alpha channel:

```

1  AlphaTestEffect effect(getGraphicsDeviceProperty());
2  effect.setWorldProperty(fenceWorld);
3  effect.setViewProperty(camera.View());
4  effect.setProjectionProperty(camera.Projection());
5
6  effect.setTextureProperty(&fenceTexture);           // alpha channel:
   255 = wire, 0 = gap
7  effect.setAlphaFunctionProperty(CompareFunction::Greater);
8  effect.setReferenceAlphaProperty(128);              // keep only
   pixels with alpha > 128
9
10 for (EffectPass& pass : effect.getCurrentTechniqueProperty()->
    getPassesProperty()) {
11     pass.Apply();
12     getGraphicsDeviceProperty().DrawIndexedPrimitives(/* ... */);
13 }
```

The corresponding regression protects this shape. A stride-24 `VertexPositionColorTexture` fence must multiply its per-vertex color only when `VertexColorEnabled` is true, and the combined alpha must participate in the discard. Reverting the Vulkan/BGFX sibling shaders reproduced the old diffuse-only value and even changed the pass/discard decision; restoring them returned both cases to the expected pixels.

15.6 DualTextureEffect

Two independent texture slots (`Texture`, `Texture2`), plus `VertexColorEnabled`, `DiffuseColor` / `Alpha`, and fog — no lighting. The audit’s headline finding was a shared formula bug across all three campaign renderers: FNA’s real shader doubles the first texture’s RGB value (`color.rgb *= 2`) before multiplying it against the overlay texture, and none of the three

shaders initially implemented that factor — invisible to every prior test, because a test built from 0-or-1-saturated color values cannot distinguish “multiplied by one” from “multiplied by two and then saturated.” The factor is present in all three pinned shader paths now. `Texture2`’s null-texture fallback had its own, BGFX-specific bug (again, no else-branch), fixed alongside the project-wide fallback fix above. Fog needed its own separate EasyGL fix here, because this effect uses its own dedicated shader with no fog infrastructure at all, unlike `AlphaTestEffect`’s shared shader path; Vulkan and BGFX were already covered by the project-wide fog fix. `VertexColorEnabled` was another closed defect: the correction added a stride-24 colored variant on EasyGL, Vulkan, and BGFX and verified enabled and disabled cases independently.

15.6.1 Lightmapped floor

`DualTextureEffect`’s two independent texture slots are the classic pre-baked-lighting pattern from the original Xbox 360/Windows XNA era: a base color texture plus a second, low-resolution lightmap multiplied over it, with no runtime lighting computation needed at all:

```

1  DualTextureEffect effect(getGraphicsDeviceProperty());
2  effect.setWorldProperty(floorWorld);
3  effect.setViewProperty(camera.View());
4  effect.setProjectionProperty(camera.Projection());
5
6  effect.setTextureProperty(&floorDiffuse);    // base color, tiled
        across the floor mesh's own UVs
7  effect.setTexture2Property(&floorLightmap); // low-res baked
        lighting, its own second UV channel
8
9  for (EffectPass& pass : effect.getCurrentTechniqueProperty()->
    getPassesProperty()) {
10     pass.Apply();
11     getGraphicsDeviceProperty().DrawIndexedPrimitives(/* ... */);
12 }
```

The audit’s headline finding directly changes what result this produces: real FNA’s shader doubles `Texture`’s RGB value before multiplying it against `Texture2` — `color.rgb *= 2` — specifically so a lightmap value of 0.5 (mid-gray, the “neutral, unlit” value most lightmap bakers emit) reproduces the base texture’s own color exactly, rather than darkening it to half brightness. A lightmap authored against real XNA/FNA’s doubling behavior will render visibly too dark on any CNA build predating the fix that added this factor to all three campaign shaders at once. A colored lightmapped floor uses the stride-24 variant; the pinned EasyGL, Vulkan, and BGFX routes now distinguish the enabled and disabled `VertexColorEnabled` cases.

15.7 EnvironmentMapEffect

`EnvironmentMapEffect` combines `IEffectMatrices`, `IEffectLights`, and `IEffectFog`. It accepts a diffuse `Texture`, a cube `EnvironmentMap`, `EnvironmentMapAmount`, `EnvironmentMapSpecular`, `FresnelFactor`, `EmissiveColor`, and lighting state. As in FNA, `LightingEnabled` is always true and its setter throws when passed false. The 14-property API audit found no property-contract defects.

Historical note. The stock-effect audit corrected four rendering/state defects. `Clone()` omitted `FogColor` here and in `AlphaTestEffect`, `DualTextureEffect`, and

SkinnedEffect. The cube-map blend was additive instead of FNA's `lerp`; its base and specular terms omitted the cube alpha scale; and all renderers lacked Fresnel edge-weighting. CNA evaluates the corrected Fresnel term per pixel, whereas FNA evaluates it per vertex. EasyGL and BGFX also used the world matrix for normals under non-uniform scale; they now use the required transpose-inverse, as Vulkan already did.

A capstone fixture composes these corrections and produces byte-identical output on EasyGL, Vulkan, and BGFX. That result is scoped to the fixture and those renderer routes.

15.7.1 Chrome car body

A reflective, Fresnel-edge-brightened surface — the archetypal use for this effect — needs both the diffuse texture and the `TextureCube` environment map bound, plus lighting enabled so the Fresnel and specular terms below have a light direction to work from:

```

1  EnvironmentMapEffect effect(getGraphicsDeviceProperty());
2  effect.setWorldProperty(carBodyWorld);
3  effect.setViewProperty(camera.View());
4  effect.setProjectionProperty(camera.Projection());
5
6  effect.setTextureProperty(&carBodyPaint);
7  effect.setEnvironmentMapProperty(&skybox);           // the previously
      created TextureCube
8  effect.setEnvironmentMapAmountProperty(0.85f);       // mostly
      reflective, not a full mirror
9  effect.setFresnelFactorProperty(1.0f);
10 effect.setLightingEnabledProperty(true);             // required --
      setter throws if set false
11
12 for (EffectPass& pass : effect.getCurrentTechniqueProperty()->
   getPassesProperty()) {
13     pass.Apply();
14     getGraphicsDeviceProperty().DrawIndexedPrimitives(/* ... */);
15 }
```

At the pin, `FresnelFactor` brightens reflections toward grazing angles and `EnvironmentMapAmount` interpolates between the base color and reflection. Earlier implementations omitted the Fresnel term and used an additive cube-map blend, which could over-brighten the result. EasyGL and BGFX also once transformed normals by the world matrix; their current transpose-inverse transform preserves reflection shape under non-uniform scale.

15.7.2 Three-light forwarding and a stale plan entry

Historical note. At the pin, `plan_graphics.md` still marks Task 890—forwarding `DirectionalLight1` and `DirectionalLight2` through `EnvironmentMapEffect`—as open. The implementation has moved ahead of that checkbox. `FillGpuDrawParams()` copies both lights' direction and diffuse fields when each light is enabled, and the renderer implementations consume those fields at their uniform-update sites. The EasyGL source comment retains the task number, which makes the stale plan entry traceable. FNA's one-light shader variant is also selected when lights 1 and 2 are disabled.

A three-light scene distinguishes the current forwarding path from the former one-light result; a single-light scene cannot:

```

1  EnvironmentMapEffect effect(getGraphicsDeviceProperty());
2  effect.setWorldProperty(carBodyWorld);
```

```

3  effect.setViewProperty(camera.View());
4  effect.setProjectionProperty(camera.Projection());
5  effect.setTextureProperty(&carBodyPaint);
6  effect.setEnvironmentMapProperty(&skybox);
7  effect.setLightingEnabledProperty(true);
8
9  effect.DirectionalLight0.setEnabledProperty(true);           // key
   light, warm
10 effect.DirectionalLight0.setDiffuseColorProperty(Vector3(1.0f, 0.85f
   , 0.7f));
11 effect.DirectionalLight1.setEnabledProperty(true);           //
   fill light, cool
12 effect.DirectionalLight1.setDiffuseColorProperty(Vector3(0.2f, 0.3f,
   0.5f));
13 effect.DirectionalLight2.setEnabledProperty(true);           // rim
   light, from behind
14 effect.DirectionalLight2.setDiffuseColorProperty(Vector3(0.6f, 0.6f,
   0.6f));
15
16 for (EffectPass& pass : effect.getCurrentTechniqueProperty()->
   getPassesProperty()) {
17     pass.Apply();
18     getGraphicsDeviceProperty().DrawIndexedPrimitives(/* ... */);
19 }

```

Before the fix, this scene ignored `DirectionalLight1` and `DirectionalLight2`. At the pinned revision all three enabled lights reach the renderer.

15.8 SkinnedEffect

`SkinnedEffect` accepts at most `MaxBones = 72`. Its `WeightsPerVertex` property accepts only 1, 2, or 4 and throws for other values. `SetBoneTransforms / GetBoneTransforms(count)`, specular lighting, fog, and three directional lights follow the stock-effect route. CNA adds `VertexColorEnabled` for imported glTF meshes with a `COLOR_0` attribute; XNA's `SkinnedEffect` has no corresponding property.

Identity, single-bone, and two-bone weighted skinning are pixel-verified on the three campaign renderers. Historical defects affected surrounding state: `Clone()` dropped `SpecularColor` and `SpecularPower`; lights 1 and 2 were not forwarded; the specular fields had no GPU implementation; and `WeightsPerVertex` did not gate the four-weight sum. The pinned implementation preserves the cloned fields, forwards all enabled lights, evaluates a Blinn–Phong half-vector specular term, and gates the sum for the selected weight count.

15.8.1 Driving SkinnedEffect from AnimationPlayer

Chapter 38's own worked `AnimationPlayer` example ended with exactly the call this effect exists to receive — `SetBoneTransforms` takes `GetSkinTransforms()`'s output directly, already pre-multiplied by each bone's inverse bind pose, which is specifically the form the GPU skinning shader expects:

```

1  SkinnedEffect effect(getGraphicsDeviceProperty());
2  effect.setViewProperty(camera.View());
3  effect.setProjectionProperty(camera.Projection());
4  effect.setWeightsPerVertexProperty(2);           // must be 1, 2, or 4 --
   throws otherwise

```

```

5  effect.setSpecularColorProperty(Vector3(0.3f, 0.3f, 0.3f));
6  effect.setSpecularPowerProperty(16.0f);
7  effect.EnableDefaultLighting();
8
9  // Every frame, after player.Update(...):
10 effect.SetBoneTransforms(player.GetSkinTransforms());

```

Rigs exceeding 72 palette joints require mesh partitioning into draws with disjoint palettes. Set `WeightsPerVertex` to match the asset. Projects that previously requested two weights but depended on the former four-weight behavior will render differently at the pin.

15.9 Closing the `preferPerPixelLighting` divergence

Current contract. `PreferPerPixelLighting` defaults to `false`, matching XNA's per-vertex (Gouraud-interpolated) default. The pinned D3D9, EasyGL, Vulkan, BGFX, D3D11, and D3D12 paths honor the choice. WebGPU honors it for `BasicEffect`; `SkinnedEffect` and `EnvironmentMapEffect` are not implemented there. It is inapplicable to the 2D-only `SdlRenderer` and non-rasterizing `Headless` renderer. The `Software` rasterizer has no lighting engine, so this remains an open limitation there. Do not extrapolate this campaign result to unlisted implementation families.

Earlier shared dispatch omitted both `PreferPerPixelLighting` and `EnvironmentMapEffect`'s `specularEnabled`. The affected GPU paths therefore used fragment lighting regardless of the public setting. D3D9 already contained Microsoft's per-vertex and per-pixel shader families; the other covered renderers added per-vertex variants using FNA's lighting formula in the vertex stage. `AlphaTestEffect` and `DualTextureEffect` have no lighting path and were unchanged.

The new discriminating fixtures exposed four secondary issues:

Renderer	Finding and disposition
D3D9	Pixel-lit constants were uploaded only to the vertex stage, leaving the pixel shader's lighting registers zero. The upload now targets the stage that evaluates the light.
Vulkan	New pipeline-cache maps were absent from teardown. A validation-layer report at <code>vkDestroyDevice()</code> exposed the leaked <code>VkPipeline</code> objects.
WebGPU	Center values 187/203 differed from linear 127/152 because the preferred swapchain format applies sRGB encoding; the computed encodings match the observations.
D3D12	A specular-exactness fixture depended on per-fragment evaluation. It now sets <code>PreferPerPixelLighting</code> to <code>true</code> , preserving that test's intended geometry.

Lit-scene baselines changed when the XNA-compatible default became effective: low-poly surfaces can appear more faceted and highlights can soften under vertex interpolation. Those changes are expected for the default path; the D3D12 fixture above explicitly selects the alternate path.

15.10 The pattern behind these bugs

The audits found three recurring failure modes:

- a missing shader term, such as doubled dual-texture RGB, Fresnel weighting, or specular;
- a public flag that never reached GPU state, such as the former `VertexColorEnabled` and `WeightsPerVertex` gaps; and
- a shared dispatch defect affecting several renderers, including the former light-enable, fog-forwarding, and clone-state omissions.

The tested core interpolation and lighting formulas were not broadly defective; the observed failures were specific omissions. This distinction keeps the evidence scoped to the paths and fixtures that exposed them.

Chapter 16

State Objects and Value Semantics

CNA’s four state-object classes all inherit `GraphicsResource`. Their static, ready-made presets exist before a `GraphicsDevice`; that is why the base accepts a null device (Chapter 12). An architectural choice worth stating up front is that CNA stores the three device-wide state objects *by value*. A device assignment copies the supplied state into its own CPU field, then calls the corresponding renderer application method. Thus mutating the original local value later cannot change the stored device value. CNA also declares the static presets `const`, whereas FNA’s `static readonly` fields protect only the C# reference — their public state properties remain mutable. The presets themselves are unbound, harmless `GraphicsResource` values in both implementations, not GPU allocations that must be disposed.

There is a second, more important rule than that value/reference difference: reading CNA’s mutable device-state getter gives access only to the CPU copy. Changing that reference does *not* call `ApplyBlendState`, `ApplyDepthStencilState`, or `ApplyRasterizerState`; those calls occur only in their three device setters. Reassign a changed copy to submit it. A former CNA test comment claimed that FNA’s reference alias would make a later `BlendState` mutation take effect on the next draw. Current FNA source does not support that conclusion: its blend/depth flush resubmits only when the selected state *object identity* changes, so a mutation of the already-current object is not dirty-tracked. FNA does reapply its rasterizer state every draw, but that implementation detail is not a portable update route. The safe rule in either API is to configure a state before assignment and assign it again after a change.

16.1 By-value state semantics

The value copy and submission boundary are easy to state abstractly and easy to get wrong in practice. `GraphicsDevice::setBlendStateProperty(const BlendState&)` takes its argument by `const` reference, stores an internal copy, and submits the supported fields. The mutable overload of `getBlendStateProperty()` returns that CPU copy; it is not a proxy that submits every field mutation:

```
1  BlendState custom;
2  custom.setColorSourceBlendProperty(Blend::SourceAlpha);
3  custom.setColorDestinationBlendProperty(Blend::One);    // additive-
                    style blend
4  device.setBlendStateProperty(custom);
5
6  custom.setColorSourceBlendProperty(Blend::Zero);        // mutate
                    the ORIGINAL afterward
7  // device's stored copy and submitted blend factors are still
                    SourceAlpha here.
8
9  // Do not use this as a GPU update: it changes only the device's
                    stored CPU copy.
```

```

10 device.getBlendStateProperty().setColorSourceBlendProperty(Blend::
    One);
11
12 BlendState submitted = device.getBlendStateProperty();
13 submitted.setColorSourceBlendProperty(Blend::One);
14 device.setBlendStateProperty(submitted); // the required submission
    step

```

The practical rule is therefore more conservative than the old “mutate through the getter” advice: construct or copy, change, and assign a fresh value every time the effective blend, depth-stencil, or rasterizer state changes. A C++ getter mutation can make later introspection misleading by showing a value the renderer was never asked to use.

16.2 The value surface is broader than the binding bridge

Every public property round-trips in the small C++ value objects, but not every property has the same submission route or renderer coverage. The three whole-state setters submit first and copy into the public device cache only after the renderer calls succeed; a native rejection therefore cannot leave the cache claiming the rejected state. Samplers are instead resubmitted for all sixteen pixel slots immediately before each draw. The current bridges are:

- **BlendState** forwards its six source/destination/function values plus one **BlendWriteState** containing all four per-MRT color-write masks and the state’s **MultiSampleMask**. Its **BlendFactor** is then sent through a separate renderer hook. The similarly named *device* **MultiSampleMask** property remains only a C++ integer shadow and makes no renderer call; use the state-object property.
- **DepthStencilState** forwards all of its depth/stencil fields on whole-state assignment. Its **ReferenceStencil** is also sent through the distinct device setter, whose renderer default is a no-op; Chapter 16’s EasyGL caveat therefore still applies to a standalone reference change.
- **RasterizerState** forwards cull mode, fill mode, scissor enable, and both bias values, but not **MultiSampleAntiAlias**. The property is a retained CPU value rather than a generic rasterization command.
- On every CNA draw, the device visits all sixteen *pixel* sampler slots. One hook carries filter, AddressU, AddressV, and maximum anisotropy; a second carries MaxMipLevel and MipMapLevelOfDetailBias. AddressW is still absent. The public vertex texture and vertex sampler collections are retained (and cleared when a texture is disposed), but no draw path forwards either collection to the renderer at all.

The presence of a common hook is not a promise that every implementation consumes it. All **ApplyBlendState** overrides had to make an explicit decision about **BlendWriteState**, but historical and 2D families may reject or ignore unsupported masks. The shared differential 3D oracle exercises color masks on SDL GPU, WebGPU, and LLGL; its optional 4x-MSAA leg is enabled only where a functional native sample mask exists. The newer sampler-mip hook has only four overrides: FNA3D stores both values, Glide accepts LOD bias but rejects a non-default maximum level, Metal rejects either non-default, and PortableGL deliberately ignores both because its textures have one level. All other families inherit the no-op default. Thus a pixel sampler entry is submitted at the next draw, but support for its mip controls remains a renderer claim to verify. No located draw test distinguishes AddressW or either vertex collection from its default behavior.

16.3 BlendState

Four presets — `Additive`, `AlphaBlend`, `NonPremultiplied`, `Opaque` — and a property surface covering the color and alpha blend function/source/destination triples independently, four independent `ColorWriteChannels` masks (one per potential MRT slot), a `BlendFactor` color, and a `MultiSampleMask`. Blend has thirteen values (`One`, `Zero`, the four source/destination color and alpha pairs, `BlendFactor` / `InverseBlendFactor`, and `SourceAlphaSaturation`); `BlendFunction` has five (`Add`, `Subtract`, `ReverseSubtract`, `Max`, `Min`). `ColorWriteChannels` is a flags enum with the same hand-implemented bitwise operator set already seen on `DisplayOrientation` (Chapter 6) and `ClearOptions` (Chapter 12).

The common bridge now carries the entire value surface: the six blend ordinals, the four write masks, the state-object sample mask, and the separately submitted blend factor. Native support is still narrower on some families; eleven renderer overrides explicitly ignore the appended write state, and some single-target APIs can honor only slot zero. The generic differential test is therefore stronger evidence than a round-trip property test: with blending disabled it proves that masked channels preserve the destination, enabled channels take the source, and an A-B-A sequence does not reuse stale pipeline state. Its optional MSAA case separately compares all-samples with mask zero.

Vulkan also has a historically serious bug in the six blend ordinals — for a long time its blend implementation was, in the project’s own words, “almost entirely fake”: one hardcoded blend equation applied regardless of what was actually requested, confirmed failing across five separate pixel tests before the fix replaced it with a genuine per-Blend/BlendFunction mapping across all nine of the renderer’s pipeline-creation sites. The later `BlendWriteState` work is a distinct contract and must not be inferred from that earlier factor/function campaign.

16.4 DepthStencilState

Three presets (`Default` — depth test and write on —, `DepthRead` — test on, write off —, and `None` — both off) sit atop a sixteen-property surface: `DepthBufferEnable`, `DepthBufferWriteEnable`, and `DepthBufferFunction`; plus a full stencil-test configuration (`StencilEnable`, `StencilFunction`, `StencilMask/StencilWriteMask`, `ReferenceStencil`, the three stencil-operation slots for pass/fail/depth-fail, `TwoSidedStencilMode`, and four separate counter-clockwise-face variants of the same operations). `CompareFunction` has eight values; `StencilOperation` has eight (`Keep`, `Zero`, `Replace`, `Increment / Decrement` and their saturating variants, `Invert`).

Depth testing itself is solid on the measured paths: `DepthBufferWriteEnable` and a five-case `DepthBufferFunction` selection test (`Always`, `Never`, `Less`, `LessEqual`, `Greater`) verify correctly on EasyGL and, as of a fix, Vulkan. That is not a sweep of all eight enum values. Stencil testing is where this state object’s real history lives. At one point, Vulkan’s apply-state entry point accepted fifteen parameters but genuinely stored only two — every stencil-related parameter passed through was simply discarded, and `stencilTestEnable` was never set on any pipeline at all. Five separate, independent checks (enable flag, masks, front-face operations, two-sided mode, reference-stencil propagation) each separately confirmed failing on Vulkan while passing on EasyGL, before the fix added real per-pipeline compare operations, full front/back `VkStencilOpState` configuration, and dynamic-state reference/mask values. A second Vulkan-specific bug was found in the same pass: depth-format selection checked a stencil-*less* format before it checked any stencil-capable one. EasyGL, meanwhile, had a subtler bug of its own: no window was ever created requesting an actual stencil buffer (`SDL_GL_STENCIL_SIZE`) at all — meaning every stencil test that appeared to pass was passing against a buffer with

zero stencil bits actually allocated, a “correct code, no resource to act on” bug rather than a logic error. One gap named in an earlier pass of this chapter as open on both EasyGL and BGFX is now only open on one of them: `IGraphicsRenderer::SetReferenceStencil` does exist in the shared renderer interface, with a no-op default any renderer may override. BGFX now overrides it, mirroring Vulkan’s mechanism: every other stencil parameter is cached so a later standalone reference-value change can rebuild the renderer’s own front/back stencil state without a full state-object re-application. EasyGL has no override and inherits the interface no-op. Changing `ReferenceStencil` without reassigning the whole `DepthStencilState` is therefore broken on EasyGL, but not BGFX. A second gap named in the same earlier pass of this chapter as still open on all three hardware renderers is also closed: a stencil-only `Clear` call once ignored its flag on every renderer, but the current route dispatches it on EasyGL, Vulkan, and BGFX. The repair also exposed an independent Vulkan-specific bug along the way (every render-pass-creation site had its own stencil load-operation hardcoded to discard, regardless of what `Clear()` itself requested) — see Chapter 12 for the fix in full.

16.4.1 Masking a reflection with the stencil buffer

The classic real use of the stencil-test properties above, taken together, is a planar mirror: render the mirror surface into the stencil buffer only, then render the reflected scene a second time, but only where that stencil mark was left behind — so the reflection never spills past the mirror’s own on-screen footprint even though the reflected geometry itself may extend well beyond it. This needs two distinct `DepthStencilState` configurations, not one:

```

1  DepthStencilState writeMirrorMask;
2  writeMirrorMask.setDepthBufferEnableProperty(true);
3  writeMirrorMask.setStencilEnableProperty(true);
4  writeMirrorMask.setStencilFunctionProperty(CompareFunction::Always);
5  writeMirrorMask.setStencilPassProperty(StencilOperation::Replace);
6  writeMirrorMask.setReferenceStencilProperty(1);
7
8  DepthStencilState maskedByMirror;
9  maskedByMirror.setDepthBufferEnableProperty(true);
10 maskedByMirror.setStencilEnableProperty(true);
11 maskedByMirror.setStencilFunctionProperty(CompareFunction::Equal);
12 maskedByMirror.setReferenceStencilProperty(1); // only draw where
    the mirror already wrote 1
13
14 // Pass 1: draw the mirror quad itself, writing 1 into the stencil
    buffer everywhere it covers.
15 device.setDepthStencilStateProperty(writeMirrorMask);
16 mirrorQuad.Draw();
17
18 // Pass 2: draw the reflected scene, masked to the mirror’s own
    footprint by the stencil test.
19 device.setDepthStencilStateProperty(maskedByMirror);
20 reflectedScene.Draw(); // a full scene draw, using a reflected view
    matrix

```

One of this chapter’s own already-stated facts directly constrains whether this exact technique is safe to ship across all three hardware renderers. `ReferenceStencil` above is set once, as part of constructing each `DepthStencilState` — this worked example does not rely on changing it independently afterward, which sidesteps the still-open gap flagged above (no real `SetReferenceStencil` override exists on EasyGL specifically, BGFX’s own fix now notwithstanding; a technique needing a per-draw-call reference value without rebuilding the whole state object would hit that remaining gap directly on EasyGL). The requested stencil *value*

now reaches all three hardware renderers; later ordered-clear work also made stencil-only clears selective on Vulkan and BGFX. Vulkan records the exact attachment command at the issue position, and BGFX assigns an ordered view carrying only `BGFX_CLEAR_STENCIL` (§19.4.2). Portable code can still rely on what this technique’s own first pass already does — its unconditional `CompareFunction::Always / StencilOperation::Replace` configuration overwrites the stencil value outright every frame, never depending on it having been zeroed first, which is exactly why this specific technique remains correct without assuming selective clear semantics.

16.5 RasterizerState

Three presets (`CullClockwise`, `CullCounterClockwise` — the XNA default — `CullNone`) and six properties: `CullMode`, `DepthBias`, `FillMode`, `MultiSampleAntiAlias`, `ScissorTestEnable`, `SlopeScaleDepthBias`. `CullMode` has three values (`None`, `CullClockwiseFace`, `CullCounterClockwiseFace`); `FillMode` has two (`Solid`, `WireFrame`).

`ScissorTestEnable` is deliberately only an enable switch. Its rectangle lives separately in `GraphicsDevice.ScissorRectangle`; changing either property immediately routes only that half of the state to the renderer. The distinction is observable: EasyGL, Vulkan, BGFX, WebGPU, SDL GPU, D3D9, and D3D11 retain an independent Boolean and rectangle. The SDL renderer and ASCII instead apply the rectangle as an always-active clip and ignore the Boolean. D3D12, Software, Canvas, and FREEDIRECT never turn the pair into a custom raster clip at all; Headless only validates and traces it. The full scope and evidence matrix is §19.4.3. `SpriteBatch` no longer creates a second routing gap here: `Begin()` resolves a null argument to `RasterizerState::CullCounterClockwise` and otherwise submits the caller’s copied state through this same `GraphicsDevice` setter.

All three cull modes verify correctly on both EasyGL and Vulkan via a genuine opposite-winding-order pixel test, and a small architectural detail worth noting: Vulkan flips clip-space Y and *compensates* by setting `VK_FRONT_FACE_CLOCKWISE` instead of the API’s default counter-clockwise convention — confirmed, by the same test, to cull the same input winding identically to EasyGL despite the two renderers taking different routes to get there. `FillMode::WireFrame` is verified on both EasyGL (via a `GL_LINES` re-expansion, since OpenGL ES has no `glPolygonMode` to fall back on directly) and Vulkan (`VK_POLYGON_MODE_LINE`). `DepthBias` and `SlopeScaleDepthBias` have one known, pre-existing failure at an extreme magnitude (`DepthBias = -1e6`) on Vulkan specifically — the other three sub-cases pass, and this was not re-investigated as part of this state object’s own audit. BGFX’s `CullMode / FillMode / DepthBias / ScissorTestEnable` are, by design, not independently pixel-verified at all — BGFX has no GPU-readback API in this project, so its state coverage is smoke-test/no-regression only, a scope difference worth remembering whenever this book or the project’s own docs describe a BGFX row as “correct.”

16.5.1 Shadow-map depth bias and a wireframe debug toggle

`DepthBias` and `SlopeScaleDepthBias` exist for exactly one recurring real problem: shadow acne, the self-shadowing Moiré artifact a shadow-mapped surface shows when its own depth-buffer sample is compared against itself with no tolerance at all. A small, fixed bias plus a slope-scaled term (larger at grazing angles, where acne is worst) is the standard fix:

```
1 RasterizerState shadowMapBias;
2 shadowMapBias.setCullModeProperty(CullMode::CullClockwiseFace);
3 shadowMapBias.setDepthBiasProperty(0.0015f);
4 shadowMapBias.setSlopeScaleDepthBiasProperty(1.5f);
5
```

```

6  device.setRasterizerStateProperty(shadowMapBias);
7  // ... render the scene from the light's own view/projection into
    the shadow map ...

```

This chapter's own audit notes one specific, narrow failure at an extreme magnitude (`DepthBias = -1e6`) on Vulkan — ordinary shadow-mapping bias values like 0.0015 above are nowhere near that magnitude and are unaffected; the caveat only matters if a game computes its bias value dynamically and that computation could, under some real input, blow up toward an extreme.

A second, unrelated everyday use for the same class — toggling a wireframe debug overlay — is a one-property change with no bias involved at all:

```

1  RasterizerState wireframe;
2  wireframe.setFillModeProperty(FillMode::WireFrame);
3  wireframe.setCullModeProperty(CullMode::None); // see both front and
    back edges while debugging
4
5  if (showWireframeOverlay)
6      device.setRasterizerStateProperty(wireframe);
7  else
8      device.setRasterizerStateProperty(RasterizerState::
    CullCounterClockwise);

```

Remember this chapter's own scope caveat before trusting a wireframe-toggle test on BGFX specifically: `FillMode` there has no GPU-readback pixel verification at all (BGFX has no readback API in this project), so `FillMode::WireFrame` on that one renderer is smoke-tested, not pixel-proven, unlike the same toggle on EasyGL or Vulkan.

16.6 SamplerState

Six presets — `AnisotropicClamp / Wrap`, `LinearClamp / Wrap`, `PointClamp / Wrap` — cover exactly the combinations real FNA ships, no more. Seven properties: `AddressU / V / W` (`TextureAddressMode: Wrap, Clamp, Mirror`), `Filter` (`TextureFilter`, nine values spanning point/linear/anisotropic and their per-mip-stage combinations), `MaxAnisotropy`, `MaxMipLevel`, `MipMapLevelOfDetailBias`.

The central, now-fixed finding here was not about the address-mode or filter mapping tables themselves — both were correct in isolation for as long as they were tested — but about whether the supported *pixel* `GraphicsDevice.SamplerStates` route was actually applied before a 3D draw. It was not, for three separate, renderer-specific reasons. EasyGL's eighteen user-primitive overloads omitted the internal sampler-application step; only the ordinary buffer-bound `DrawPrimitives` and `DrawIndexedPrimitives` calls made it. On Vulkan, the dual-texture descriptor-set path hardcoded a default sampler into its slots regardless of what sampler state was actually current; on BGFX, a dual-texture draw bound its second texture slot using the *first* slot's sampler flags. All three were fixed, and — once actually applied — the underlying address-mode and filter mappings themselves were confirmed correct on EasyGL and Vulkan for a real 3D stock-effect draw. That conclusion never extended to `VertexSamplerStates`: the public collection is still retained only, as the binding-boundary section above shows.

Two further, more severe gaps were found and fixed together. First, mip-aware filtering: EasyGL correctly honors any explicit Mip-qualified filter, but `Point` and `Linear` on their

own always sample mip level 0 regardless of minification — a *deliberate*, documented deviation from FNA (which is mip-aware for these two filters too), because CNA does not set `GL_TEXTURE_MAX_LEVEL` per texture, and applying a mip-aware filter unconditionally to the common non-mipmapped case would render as GL-incomplete (solid black) instead. Second, and more severe: `Texture2D::SetData` at a mip level above 0 was a *total, silent no-op* on both Vulkan and BGFX — the same historical failure shape their `Texture3D` / `TextureCube` read-back paths once had (Chapter 14 now records the corrected live matrix). Mip-level `SetData` is now a real GPU upload on all three hardware renderers, but that upload fix must not be mistaken for universal `SamplerState` mip control. `AddressW` still has no common argument. `MaxMipLevel` and LOD bias now reach the separate `ApplySamplerMipState` hook, but only FNA3D consumes both without rejecting a non-default; Glide consumes only bias, Metal explicitly refuses non-default values, PortableGL has a documented one-level no-op, and every other renderer inherits the interface no-op. The API surface is wired, but the implementation matrix is deliberately sparse. Anisotropic filtering itself: correct on Vulkan (queries the real device capability and clamps the requested level); EasyGL was silently falling back to plain trilinear filtering regardless of the requested anisotropy level until a fix wired real OpenGL anisotropic-filtering-extension support, clamped to the driver’s own capability; BGFX remains partially fixed — it enables the anisotropic sampling flag but never communicates the actual requested level, an on/off switch rather than a graduated one, still open. The newer renderers widen that matrix: WebGPU and D3D11/D3D12 create real anisotropic samplers (the Direct3D pair accept only feature level 11.0 or newer, whose maximum anisotropy is 16); D3D9 maps the filter and maximum but neither capability-checks nor clamps against `D3DCAPS9::MaxAnisotropy`; SDL GPU stores the requested maximum but never puts it in a queued command or sampler-cache key; and Software ignores every sampler value after the slot-range check. ASCII does not even forward `ApplySamplerState` to the SDL renderer it wraps. The last three therefore also make `SupportsCapability(AnisotropicFiltering)` definite false positives, since they inherit the interface’s default `true`; D3D9’s answer is conditionally unreliable on the current device. See Chapter 19’s complete capability table.

16.6.1 Crisp pixel art and a minified, mipmapped floor

Two everyday sampler needs pull `Filter` in opposite directions, and the chapter’s own mip-awareness deviation above means the obvious choice is not always the correct one for both. A 2D pixel-art sprite wants hard, un-blurred texel edges at every zoom level — the `PointClamp` preset, unmodified, is exactly right, and mip level 0 is the *only* level a sprite like this has anyway:

```
1  SamplerState pointClamp = SamplerState::PointClamp;
2  spriteBatch_ ->Begin(SpriteSortMode::Deferred, BlendState::AlphaBlend
3      ,
      &pointClamp, nullptr, nullptr); // crisp texel
      edges, no mip chain to worry about
```

A distant, minified 3D floor texture with a real mipmap chain exposes an **EasyGL-specific** choice. On that renderer, plain `TextureFilter::Point` and `Linear` sample level 0 regardless of minification; selecting a mip-qualified value is therefore required for a blocky but distance-stable result:

```
1  SamplerState blockyButMipAware;
2  blockyButMipAware.setFilterProperty(TextureFilter::PointMipLinear);
   // point min/mag, real mip chain
3  blockyButMipAware.setAddressUProperty(TextureAddressMode::Wrap);
4  blockyButMipAware.setAddressVProperty(TextureAddressMode::Wrap);
5
```

```
6 device.getSamplerStatesProperty()[0] = blockyButMipAware;
```

This is EasyGL’s deliberate deviation from FNA (which is mip-aware for plain `Point/Linear` too), not an edition-wide promise about every CNA renderer. EasyGL keeps the plain filters mip-unaware because unconditionally enabling mip selection on the common one-level texture would make it GL-incomplete and black. Other renderers map the same enum through their own native sampler contracts; carry this choice across only after a renderer-specific pixel test.

16.7 The recurring device-default copy defect

Every one of these four state classes independently had the identical small bug at some point in its history: its static presets did not set their own `Name` property. This sounds trivial, but it had a real, non-cosmetic consequence for `DepthStencilState` specifically — `GraphicsDevice`’s own constructor did not actually *copy* `DepthStencilState::Default` and `RasterizerState::CullCounterClockwise` into its own member fields at construction, and this had gone unnoticed because the values happened to coincide by luck, until `Name` existed as a distinguishing field that made the missing-copy bug observable at all. Fixing the missing-`Name` bug on all four classes, in other words, is what *surfaced* a real, separate device-construction bug — a small illustration of how a seemingly cosmetic gap in one place can be hiding a functional one somewhere else entirely.

Chapter 17

Shaders: Four Answers to XNA's .fx

There is no single CNA answer to XNA effects. The current tree contains four different shader-delivery strategies: CNA-owned implementations of the stock semantics, recompilation of Microsoft's stock HLSL sources on D3D9, execution of compiled XNA/FNA Effect Framework bytecode through qualified renderer runtimes, and the renderer-specific `ShaderEffect` route. This chapter separates those strategies and, especially, formats that share a “compiled shader” description but are not interchangeable.

17.1 XNA's compiled-effect format

A normal XNA game does not ship `.fx` shader source text. The XNA content pipeline compiles `.fx` effect files to a proprietary compiled bytecode format at build time, and the compiled bytecode — not the HLSL source — is what ships inside the game's `.xnb` content files and gets loaded at runtime via `new Effect(GraphicsDevice, byte[])`. On the real Xbox 360/Windows XNA runtime, that bytecode is interpreted directly; FNA instead ships `MojoShader`, a from-scratch bytecode *parser and cross-compiler* that turns the same compiled Direct3D 9 shader bytecode into GLSL (or other target shading languages) at load time, which is exactly what lets FNA run historical XNA effect content without its original source.

17.2 The public compiled-effect boundary

Two public entry points now provide one bounded route. The `Effect` byte-buffer constructor validates an XNA/FNA Direct3D 9 Effect Framework container, asks the active renderer for `GraphicsCapability::CompiledEffects`, creates its compiled runtime, and builds the reflected object graph. The general XNB `EffectReader` reads a length-prefixed payload and calls that same constructor. It wraps constructor/backend failures in `ContentLoadException`; it is not a permanent refusal reader at this tag.

The route reflects parameters, arrays, structures, annotations, techniques, passes, shader objects, sampler/texture bindings, and pass state. `EffectPass::Apply()` synchronizes mutable values, selects the technique/pass, applies native effect state, and makes the effect current for `SpriteBatch` or ordinary 3D draws on an implementing renderer. `Clone()` duplicates the native runtime, values and current-technique choice while keeping resources tied to the same graphics device.

Admission is deliberately narrow. The buffer must be an Effect Framework container (including the XNA wrapper form), at most 64 MiB, with bounded reflection/object graphs. A file containing `.fx` source, HLSL text, raw DXBC, or MonoGame's `MGFX` container is a different input and is refused. The feature also needs both a parser and a renderer runtime; metadata parsing alone never promotes the capability.

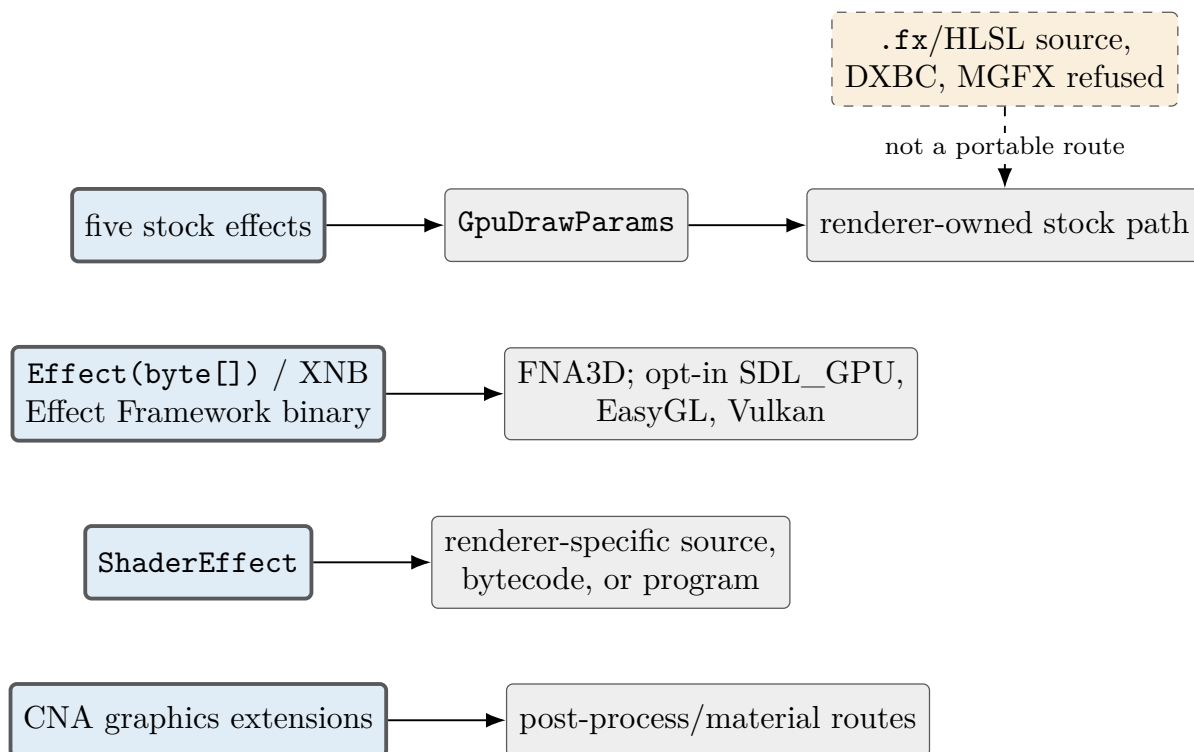


Figure 17.1: CNA's shader and effect strategies are separate APIs, formats and evidence scopes. Compiled Effect Framework support is renderer-qualified, not a universal shader compiler.

Do not attach a current sample count to that gap without retesting the samples. The pinned `cna-samples` catalog contains 153 source-archive entries split among completed ports, tracked placeholders, and deliberate exclusions (Chapter 77); its dated `missing.md` files are case records, not fresh proof that every old blocker remains. For a concrete port, inventory each general `EffectReader` asset and direct bytecode constructor call instead.

17.3 Answer one: reimplement the stock semantics

Most CNA renderers do not consume XNA bytecode. The five public stock effects translate their properties into `GpuDrawParams`; each renderer then chooses CNA-authored shader variants for combinations such as fog, vertex color, alpha test, and lighting. This is broad practical coverage, but it is a semantic reimplement rather than execution of Microsoft's compiled programs. It also explains why adding a new arbitrary parameter to an effect is not merely a map insertion: the aggregate, every relevant renderer, and the shader variants must agree on it.

17.4 Answer two: recompile Microsoft's sources on D3D9

The Direct3D 9 renderer takes a different evidentiary route. It vendors the six Microsoft/FNA stock `.fx` sources — the five public effects plus the internal sprite effect — and their four `.fxh` includes, discovers entry points from the source's own `compile vs_2_0/ps_2_0` statements, and compiles them with `d3dcompiler_47`. Its comparison tool strips only bytecode comment blocks such as CTAB and the compiler creator string: 61 of 66 resulting shader instruction streams match the Microsoft-shipped programs exactly. The five misses are all pixel-lighting

vertex variants; the recorded investigation attributes them to the compiler-47 versus XNA-era compiler-43 difference after testing the available optimization flags. This is unusually strong stock effect evidence, but it still does not compile a game’s arbitrary `.fx` content.

17.5 Answer three: execute compiled Effect Framework bytecode

FNA3D is the always-enabled implementation in this release. It is also the provenance route for the repository’s six stock fixtures. The repository commits six FNA-derived XNA 4.0 `.fxb` files: `Sprite`, `Basic`, `AlphaTest`, `DualTexture`, `EnvironmentMap`, and `Skin` effects, 106 KB in total. The build embeds those exact blobs; renderer initialization calls `FNA3D_CreateEffect`, which runs them through the pinned `MojoShader` dependency and exposes their parameter metadata. Draw dispatch selects the original program’s shader variant and applies its named parameters.

FNA3D exposes no source-string compiler, CNA’s `FNA3D_CreateEffectRenderer()` returns null, and the renderer reports `GraphicsCapability::CustomEffects == false`. The committed blobs are runtime artifacts rather than in-repository build products because producing them requires the old effect toolchain. Nevertheless, the public constructor is no longer closed to those six files: other structurally valid game-authored Effect Framework binaries enter the same FNA3D runtime and reflection path.

Three other families have opt-in implementations, all disabled by default because they add the `MojoShader` dependency:

- `CNA_SDL_GPU_COMPILED_EFFECTS` uses `MojoShader`’s `SDL_GPU` adapter and SPIR-V;
- `CNA_EASYGL_COMPILED_EFFECTS` uses its `OpenGL` adapter for the selected `EasyGL` profile; and
- `CNA_VULKAN_COMPILED_EFFECTS` uses CNA’s `Vulkan` runtime over `MojoShader`’s portable SPIR-V profile.

With an option off, the same renderer reports `CompiledEffects` false and construction fails explicitly. The tag’s shader-route document lists FNA3D, `SDL_GPU` and `EasyGL` in its table but describes `Vulkan` only later in prose; the `Vulkan` CMake option, runtime, tests and capability are the current source authority.

17.6 Answer four: bypass `.fx` with `ShaderEffect`

CNA does not leave custom shading entirely unaddressed, but “cross-renderer” needs a precise definition here. `ShaderEffect` (CNAEXT) presents one constructor shape and one `IEffectRenderer` contract to every renderer. A renderer factory override proves only that an effect object is returned. It does *not* prove that arbitrary source is compiled, that every setter has meaning, or that a 3D draw consumes the resulting program. Reading the live implementations and their registered tests gives this narrower matrix:

Renderer	Input and path that actually consumes it
EasyGL	GLSL source is compiled and linked by the active GL driver. <code>SpriteBatch</code> and the general 3D draw paths consume it. Uniform names and array setters are real; 2D, cube, and volume-texture binds each have a dedicated pixel test.

Renderer	Input and path that actually consumes it
SDL GPU	GLSL source is compiled at runtime to SPIR-V through <code>libshaderc</code> , then consumed by <code>SpriteBatch</code> 's fixed 32-byte vertex contract and 128-byte uniform block. This includes a tested two-output MRT shader.
Vulkan	The two <code>std::string</code> arguments carry raw, precompiled SPIR-V bytes — not GLSL source. <code>SpriteBatch</code> consumes them through a fixed 32-byte vertex contract and 128-byte push-constant layout, verified by <code>Vulkan_ShaderEffect_SpirV</code> .
D3D9	HLSL is compiled at runtime to the active SM2/SM3 profile and consumed by <code>SpriteBatch</code> , with a real color-inversion pixel test.
D3D11/12	HLSL is compiled at runtime as <code>vs_5_0/ps_5_0</code> and consumed by <code>SpriteBatch</code> 's fixed vertex and 128-byte constant-buffer contracts. Scalar/vector setter names are ignored; the sprite texture and sampler still arrive through <code>t0/s0</code> .
BGFX	No input is accepted through this API: <code>CompileProgram()</code> always returns false. The object reports that BGFX requires precompiled binary shaders, but <code>ShaderEffect</code> exposes no route that loads those binaries.
SDL renderer	There is no effect renderer, so construction leaves <code>IsEffectValid()</code> false. Passing the non-null object to <code>SpriteBatch::Begin()</code> throws <code>std::runtime_error</code> ; merely constructing it does not throw.

WebGPU and FREEDIRECT/free-direct likewise inherit the null `CreateEffectRenderer()` default rather than compiling a public custom program; their `SpriteBatch` implementations reject a non-null custom effect. Canvas rejects it for the same no-programmable-stage reason. The Software and Headless overrides serve different testing purposes, not real shader execution: Software accepts any non-empty pair and marks it valid while continuing to render through its fixed CPU shading path, whereas Headless records compilation, bind, uniform, and 2D-texture activity without producing shader pixels. These are useful contracts, but calling either one custom-shader support would be misleading.

None of these `ShaderEffect` paths is the compiled Effect Framework route. A game using that API must provide renderer-specific source for EasyGL, SDL GPU, and Direct3D, or precompile the exact SPIR-V contract Vulkan expects. Even among the working implementations, the public setter names and supported draw shapes are not portable promises. The API is thus a real, pixel-verified alternative on a bounded set of paths, not a write-once shader format for all CNA renderers.

`GpuDrawParams` (Chapter 15) does carry a `customEffectRenderer` pointer. `ShaderEffect` populates it through its `FillGpuDrawParams()` override. At present, however, only EasyGL's general 3D draw dispatch reads that field. Vulkan and SDL GPU retain similarly named custom-effect pointers inside their *SpriteBatch-specific* snapshots; the Direct3D custom paths are also `SpriteBatch` facilities. The default-no-op setters on `IEffectRenderer` (Chapter 19) are another deliberate capability boundary: a public `SetUniformXxx()` call may be structurally accepted without changing GPU state unless that concrete renderer overrides it.

17.6.1 Object lifetime, submission, and cloning are deliberately narrow

The constructor takes its two source arguments by reference but immediately copies both into the effect. It is therefore safe to construct from temporary or subsequently modified strings; the program is compiled from the construction-time copies, not from caller-owned character storage. If the device has a renderer, construction asks it once for an `IEffectRenderer`. A

returned but invalid renderer does not make the common constructor throw: it writes that renderer's `GetCompileError()` text to standard error and leaves `IsEffectValid()` false. A null renderer and a failed compile are intentionally collapsed to the same public boolean, and there is no public `ShaderEffect` accessor for the diagnostic. Renderers may of course fail before returning from their own factory, so this is not a promise that every renderer-specific setup failure becomes a boolean.

Treat that validity result as a precondition, not a delayed exception. `Effect::Apply()` does reject an already disposed effect, but its common code calls `OnApply()` and then makes the effect current. `ShaderEffect::OnApply()` merely writes a debug message when its renderer is absent or invalid; it does not bind a fallback program. Calling `Apply()` on an invalid custom effect can therefore still replace the device's current effect pointer. There is no automatic retry or source recompilation in this object; construct a new effect after correcting source or changing the renderer circumstances.

The uniform and texture calls are submission calls, not an XNA-style parameter collection. `ShaderEffect` keeps no uniform dictionary, no sampler table, and no texture ownership: each `SetUniformXxx()` forwards its name and scalar or caller buffer immediately to the renderer if one exists, while each `SetTexture()` immediately forwards the address of the texture's renderer. It performs no common-layer check of a uniform name, sampler-unit range, array count, or matrix/array pointer. EasyGL uploads/binds at that call; some other renderers copy into a fixed staging block; inherited interface defaults can do nothing; and Headless records a raw texture-renderer pointer for tracing. Thus a successful call is not a portable guarantee of a binding, and it never gives the effect ownership of the texture. Keep every texture alive by its normal owner and set it again whenever the target renderer/path requires rebinding. The standalone setters also do not inspect `IsDisposed`; unlike `Apply()`, they continue to forward while the renderer object remains alive, so disposal must be treated as terminal by the caller.

`Clone()` is equally source-only. It returns an owning raw `Effect` pointer whose concrete object is constructed afresh as `new ShaderEffect(*device_, vertSrc_, fragSrc_)`: it recompiles an independent renderer program and restores the three `IEffectMatrices` values to identity. It does not copy submitted uniforms, texture binds, name/tag metadata, or a compiled-program handle; its base effect begins with its own empty parameter collection and one Default/P0 technique. Use an owning smart pointer at the call site and reapply all desired state to the clone. The existing `ShaderEffectTests` test establishes distinct object identity, copied source strings, and equal validity on the default test device. It does not compile against a renderer or cover matrix/state transfer, texture lifetime, invalid-apply behavior, or a renderer reset.

17.6.2 A minimal GLSL ShaderEffect adapted from a test

Adapted from `easygl_shadereffect_texturecube_test.cpp`, a minimal cube-map-sampling fragment shader and its construction:

```

1  const char* vertSrc = R"(#version 300 es
2  precision highp float;
3  layout(location = 0) in vec3 aPosition;
4  void main() { gl_Position = vec4(aPosition, 1.0); }
5  )";
6
7  const char* fragSrc = R"(#version 300 es
8  precision highp float;
9  out vec4 FragColor;
10 uniform samplerCube CubeSampler;
11 uniform vec3 direction;
12 void main() { FragColor = texture(CubeSampler, direction); }

```

```

13  );
14
15  ShaderEffect effect(getGraphicsDeviceProperty(), vertSrc, fragSrc);
16  if (!effect.IsEffectValid()) {
17      // compilation failed -- check the renderer's own log output
18      // before drawing
19  }
20  effect.SetUniformInt("CubeSampler", 0);
21  effect.SetUniformVec3("direction", dx, dy, dz);
22  effect.SetTexture(0, environmentCube); // TextureCube overload

```

Check `IsEffectValid()` before drawing. Unlike a stock effect selected from CNA's shipped programs or FNA3D's retained stock bytecode, `ShaderEffect` compiles game-supplied source at runtime. A syntax error or unsupported GLSL feature therefore produces an invalid effect that must be reported before submission.

17.6.3 The D3D9 HLSL counterpart

`ShaderEffect`'s constructor takes one pair of source strings for every renderer; there is no HLSL-specific overload. On a Direct3D renderer the same call shape compiles HLSL instead of GLSL. The following color-inversion example is adapted from the pinned D3D9 test and uses its SM2/SM3 `Macros.fxh` texture macros, which the vendored stock `SpriteEffect.fx` also uses:

```

1  const char* vertexSrc = R"(
2  float2 vpSize;
3  struct VSInput { float3 Position : POSITION0; float4 Color : COLOR0;
4      float2 TexCoord : TEXCOORD0; };
5  struct VSOutput { float4 Position : POSITION0; float4 Color : COLOR0
6      ; float2 TexCoord : TEXCOORD0; };
7  VSOutput main(VSInput input) {
8      VSOutput output;
9      float2 ndc = (input.Position.xy / vpSize) * 2.0 - 1.0;
10     output.Position = float4(ndc.x, -ndc.y, 0.0, 1.0);
11     output.Color = input.Color;
12     output.TexCoord = input.TexCoord;
13     return output;
14 }
15 )";
16
17 const char* pixelSrc = R"(
18 texture2D Texture;
19 sampler TextureSampler : register(s0) = sampler_state { Texture = (
20     Texture); };
21 float4 main(float4 color : COLOR0, float2 texCoord : TEXCOORD0) :
22     COLOR0 {
23     float4 texColor = tex2D(TextureSampler, texCoord);
24     return float4(1.0 - texColor.rgb, 1.0); // the same inversion
25     the GLSL example performs
26 }
27 )";
28
29 ShaderEffect invertEffect(getGraphicsDeviceProperty(), vertexSrc,
30     pixelSrc);
31 // ... same IsEffectValid() check as the GLSL example ...

```

```

27 SamplerState pointClamp = SamplerState::PointClamp;
28 spriteBatch_>Begin(SpriteSortMode::Deferred, BlendState::Opaque,
29                    &pointClamp, nullptr, nullptr, &invertEffect);
30 spriteBatch_>Draw(texture, destRect, Color::White);
31 spriteBatch_>End();

```

`vpSize` is worth pausing on, because it is doing real work rather than being decorative: a custom vertex shader supplied this way has no other way to map `SpriteBatch`'s pixel-space `Position` input into normalized device coordinates, so this uniform (set automatically by the renderer, not by game code) is what makes the sprite land at the correct screen location at all — the same real test this example is adapted from separately verifies that sampling *outside* the sprite's destination rectangle stays the clear color, proving this NDC math is genuinely correct rather than happening to paint something plausible-looking over the whole screen. One small, harmless documentation inaccuracy worth knowing if you go reading `ShaderEffect`'s own header next: its class-level doc comment describes it as a “GLSL-source-based effect.” That is accurate for EasyGL and SDL GPU, but this exact worked example proves the same class compiles HLSL on Direct3D, while Vulkan interprets the strings as raw SPIR-V bytes. The comment is stale cross-renderer documentation, not a constraint enforced by the public constructor.

17.6.4 Beyond SpriteBatch: a 3D draw call

Both worked examples so far apply a `ShaderEffect` through `SpriteBatch`. The wider path described in this subsection is specifically **EasyGL support**; no other renderer's general 3D draw dispatch currently reads `GpuDrawParams::customEffectRenderer`. For a long stretch of this project's history that was the only place a `ShaderEffect` could actually take effect: applying one before a raw `GraphicsDevice::DrawIndexedPrimitives()` call — the call shape `ModelMesh::Draw()` and a hand-rolled `RawMesh` both use — was silently ignored. Because `ShaderEffect` did not yet implement `IEffectMatrices` (Chapter 15) and its `FillGpuDrawParams()` override still ran the `Effect` base class's no-op, `GpuDrawParams` stayed at its default, zero value on every 3D draw, and **EasyGL's** `DrawIndexedPrimitivesEx()` fell back to selecting one of its own built-in, stride-keyed shaders regardless of whatever custom effect the caller had just applied — exactly the same silent-fallback failure shape Chapter 12's `ExtractMatrices()` section already documents for stock effects, one layer further out.

This was closed by three coordinated changes, verified directly against `ShaderEffect.hpp` / `.cpp` and `EasyGLRenderer.cpp`: (1) `GpuDrawParams` gained a `customEffectRenderer` field; (2) `ShaderEffect` now implements `IEffectMatrices` — real `World` / `View` / `Projection` properties, extracted by the same `GraphicsDevice::ExtractMatrices()` every stock effect already goes through — and overrides `FillGpuDrawParams()` to populate `customEffectRenderer`; (3) `EasyGL's` `DrawPrimitivesEx()` / `DrawIndexedPrimitivesEx()` check that field first and, when it is set, bind the custom compiled program and its `World` / `View` / `Projection` uniforms directly, bypassing the built-in stride-selected shaders entirely.

A worked example, adapted from `easygl_shadereffect_3d_test.cpp`, makes the wiring concrete. The shader is a minimal N-dot-L diffuse lighting pass over a single textured quad, using the same GLSL attribute-location convention (`location = 0/1/2` for `Position` / `Normal` / `TexCoord`) as `VertexPositionNormalTexture`'s existing 32-byte stride:

```

1 auto* fx = dynamic_cast<ShaderEffect*>(fxBase_.get());
2 fx->setWorldProperty(world);
3 fx->setViewProperty(Matrix::CreateLookAt(Vector3(0, 0, 3), Vector3::Zero, Vector3::Up));
4 fx->setProjectionProperty(Matrix::CreatePerspectiveFieldOfView(

```

```

5     MathHelper::PiOver4, aspectRatio, 0.1f, 100.0f));
6     fx->Apply();                                     // binds the compiled program
    first
7     fx->SetTexture(0, whiteTex_);
8     fx->SetUniformVec3("uLightDir", 0.0f, 0.0f, 1.0f);
9     fx->SetUniformVec3("uDiffuseColor", 200.0f / 255.0f, 100.0f / 255.0f
    , 50.0f / 255.0f);
10
11     device.SetVertexBuffer(vb_.get());
12     device.setIndicesProperty(ib_.get());
13     device.DrawIndexedPrimitives(PrimitiveType::TriangleList, 0, 0, 4,
    0, 2);

```

The real test behind this example is deliberately built to rule out a false positive: it draws the same quad twice, once with `World = Identity` (facing the light, world normal (0,0,1), expected color close to the full (200,100,50) diffuse value) and once with `World = CreateRotationY(180°)`. A 90° rotation was deliberately *not* chosen for the second case — it would turn the quad edge-on to the camera, so “renders nothing” and “renders black” would look identical in a single-pixel readback. 180° keeps the exact same on-screen footprint (the quad is X-symmetric, so mirroring X does not change its silhouette) while flipping the world-space normal to (0,0,-1), which the shader’s own `max(dot(...), 0.0)` clamps to a genuinely lit black. Only a `World` matrix that is actually reaching the vertex shader and actually affecting world-space lighting can produce that second, specific result rather than merely happening to draw something.

17.6.5 A second, narrower gap this one uncovered, then closed: arbitrary layouts

The first version of the 3D draw path above recognized only the five byte-strides EasyGL’s then-existing `ApplyLayout()` switch knew about (16/20/24/32/52). Later PBR and vertex-color work expanded the fixed family to 48/56/68 too, but a stride is still not a description of its internal fields. Arbitrary layouts were deliberately scoped out of the first task, since several real ported sample shaders (`NormalMapping.fx`’s `Position+Normal+Binormal+Tangent+TexCoord` layout, among others) need element arrangements that match none of those fixed interpretations. A truly custom `VertexDeclaration` — arbitrary element count, arbitrary per-element offset, any `VertexElementFormat` — reads back correctly through the same 3D `ShaderEffect` draw path via a second, separately-tracked fix: `VertexBuffer::SetDataRaw()` now pushes the buffer’s own `VertexDeclaration` down to the renderer through a new `IVertexBufferRenderer::SetVertexDeclaration()` call (default no-op, so Vulkan/BGFX/SDL_Renderer needed zero code changes for this to stay correct), and `ApplyLayout()` binds generically from that declaration when one is present — GLSL attribute location equals the element’s own index within the declaration — falling straight through to the original stride-keyed switch, unchanged, whenever no declaration was supplied. This is a purely additive capability, not a replacement of the fixed-stride path the first worked example already exercises.

The real test behind this (`easygl_shadereffect_custom_vertex_layout_test.cpp`) is worth citing for its own verification technique, not just its result: it declares a 5-element, 48-byte layout (`Position / Normal / Tangent / TextureCoordinate / Color`). CNA now also has a fixed 48-byte PBR layout, but that one contains a four-float tangent and no trailing color; sharing a total stride does not make the records equivalent. The test diagnostically encodes three of the four non-position attributes directly into the output pixel (`FragColor = vec4(Normal.x, Tangent.y, TexCoord.x, Color.r)`) and draws the same quad twice with two independently-chosen sets of attribute values. Distinct, correctly-offset readback values across both draws is

what proves each attribute is being read from its own correct byte offset — not aliased to a neighbor, and not a value that happens to be correct by coincidence.

17.7 The honest summary

If your porting target relies only on `SpriteBatch` and the five public stock effects from Chapter 15, it can use CNA’s renderer-owned implementations. If it loads compiled XNA/FNA Effect Framework bytecode, the public constructor and XNB reader work on FNA3D and on opt-in `SDL_GPU`, `EasyGL` and `Vulkan` configurations. They do not compile `.fx`/HLSL source and do not accept `DXBC` or `MGFX`. A separate `ShaderEffect` migration still depends on its renderer-specific contract: `EasyGL` takes GLSL and alone extends the effect to general 3D draws; `SDL GPU` takes GLSL through `shaderc`; `Vulkan` takes precompiled SPIR-V; `D3D9/11/12` take HLSL for `SpriteBatch`; `BGFX` has no valid public path; and the non-programmable or not-yet-wired renderers reject, simulate, or only record the object. The accurate statement is therefore neither “compiled effects are unsupported” nor “all effect formats work everywhere.” Stock semantics, `D3D9` stock-source recompilation, renderer-qualified Effect Framework execution with reflection/state/cloning, and renderer-specific custom programs are separate, implemented contracts.

Chapter 18

Vertex Streams and Capabilities

This chapter is the handoff from renderer-independent graphics objects to the per-renderer Part. It follows a draw from public vertex values through declarations and bindings, then explains what a capability answer does—and does not—guarantee. The distinction is essential in CNA because the shared renderer interface contains useful defaults, explicit refusals, silent no-ops, and one effect-discarding draw fallback.

18.1 Public vertex objects are not upload layouts

The built-in public types preserve XNA-shaped fields and `IVertexType` behavior, but their C++ object representation is not necessarily the GPU byte representation. In particular, `Color` and the vertex interfaces are polymorphic. Chapter 8 documents the plain internal stream structs and their asserted 16/20/24/32/48/52/68-byte layouts.

Typed `VertexBuffer::SetData` and `GraphicsDevice::DrawUserPrimitives` routes therefore pack supported public values into those stream forms. Raw-pointer routes are a different contract: the caller supplies already-packed bytes and, where required, an explicit `VertexDeclaration`. Passing an array of public objects through `void*` does not turn their `vptr`-bearing C++ layout into the XNA stride.

18.2 VertexDeclaration is the byte contract

A `VertexDeclaration`'s layout is immutable after construction. Each element stores an offset, usage, and usage index. Its `VertexElementFormat` describes the byte shape; the declaration's stride is either explicit or computed as the largest offset + format size. The public default constructor is a deliberate exception: it creates an empty, zero-stride declaration used by the legacy `VertexBuffer(device, count)` convenience path. The constructors that explicitly receive an element list reject an empty list with `ArgumentNullException`; explicit non-positive strides throw the out-of-range exception.

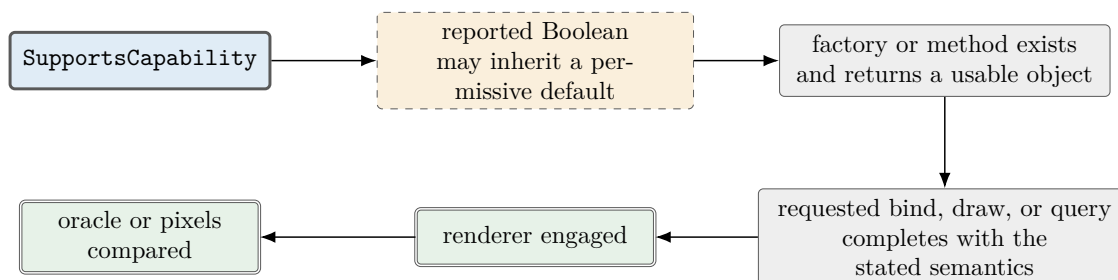


Figure 18.1: Capability reporting and capability evidence. A true query result is an API response, not a substitute for a usable factory, successful operation, or discriminating oracle.

The declaration reaches every renderer through a mandatory call. The vertex-buffer renderer interface method is `SetVertexDeclaration`. An old description that says CNA selects layouts from stride alone is obsolete. Stride is still visible because several renderer implementations optimize known built-in layouts, but a faithful path must also validate and bind the declared elements.

`VertexElement` itself is mutable and its hash currently returns zero, following the corresponding FNA limitation. Do not use the hash as a declaration identity; compare the complete element sequence and stride when building a cache key.

18.3 Buffers, index widths, and dynamic updates

`VertexBuffer` stores a declaration, vertex count, usage, and renderer-owned resource. Its typed transfer families cover CNA's seven built-in stream shapes; the CNAEXT raw route accepts a pointer, element count, and stride. Copy is deleted and move is explicit/noexcept, which avoids duplicating a native buffer handle.

`IndexBuffer` exposes 16-bit and 32-bit transfer forms selected by `IndexElementSize`. The semantic choice is not the enum's ordinal. Content and oracle tools must compare the 16/32 meaning rather than assuming an underlying 0/1 value, a mismatch that has produced real cross-implementation disagreements.

`DynamicVertexBuffer` and `DynamicIndexBuffer` add a content-lost surface and `SetData(..., SetDataOptions)`. At the pin, the dynamic update destination always begins at offset zero; the options influence native update strategy, not an arbitrary destination offset. In normal CNA operation, their content-lost property remains false.

18.4 Bindings carry offset and input rate

`VertexBufferBinding` combines a buffer, `VertexOffset`, and `InstanceFrequency`. Frequency zero is per-vertex input; a positive frequency is per-instance input. `GraphicsDevice` accepts up to sixteen binding slots through `SetVertexBuffers`. It permits null buffers as unused slots. The public ceiling is not proof that the selected renderer can consume sixteen active streams.

The classic shapes need no multi-stream capability:

- one per-vertex binding;
- one per-vertex binding plus one per-instance binding.

More than one per-vertex stream, more than one per-instance stream, or a binding count beyond the renderer-reported native ceiling invokes the `MultiStreamVertexInput` gate. A false answer causes `NotSupportedException` before native submission instead of letting the renderer silently read only stream zero.

Streams compose by (`VertexElementUsage`, `UsageIndex`), not merely by slot order. The shared layer claims each semantic pair while walking bindings. A later stream whose declaration is entirely duplicated contributes nothing, matching the FNA3D driver rule that such a stream is not in use. A declaration that repeats only some earlier semantics is rejected: CNA's combined-stride description cannot express a record whose fields are partly suppressed. Split declarations should therefore use distinct semantic/index pairs deliberately.

18.4.1 The minimum-offset fold

For multiple per-vertex bindings, CNA finds the smallest `VertexOffset` and folds it into the draw's `vertexStart` or `baseVertex`. Each binding carries only its non-negative remainder. Renderers already multiply the shared element-unit base by each stream's own stride, so this scheme applies the common portion exactly once without requiring equal strides.

With one stream, the transformation is byte-identical to the older behavior. With several streams, folding each full offset independently into one global base would be wrong; dropping the remainder would align only the earliest stream. This normalization belongs to the shared layer so every renderer sees the same binding geometry.

18.5 Draw validation has a deliberate order

Every buffer-backed 3D draw first asks the renderer to enforce its 3D policy, then requires the necessary vertex/index buffers and a currently applied `Effect`. Missing effect state is a plain `std::runtime_error`, not an XNA-shaped exception.

Range validation occurs before renderer capability validation. An index span leaving the index buffer or a declared vertex range leaving the vertex buffer is invalid on every renderer and must produce the same public argument failure even on a 2D-only product. Only after those universal invariants are established does CNA reject an unsupported stream shape or instancing path.

The user-draw overloads follow the same conceptual stages: establish a declaration, pack typed objects when necessary, validate counts and ranges, construct transient upload data, fill `GpuDrawParams` from the applied effect, and enter the renderer contract.

18.6 Thirteen capability questions

`GraphicsCapability` contains thirteen entries at the pinned revision:

Capability	Exact question
<code>ThreeD</code>	Is the vertex/index/effect 3D pipeline supported?
<code>DepthStencilBuffer</code>	Is a complete real depth/stencil attachment available?
<code>MultiSampleAntiAliasing</code>	Can a sample count above one be applied?
<code>MultipleRenderTargets</code>	Can more than one target be bound simultaneously?
<code>AnisotropicFiltering</code>	Does the current device/driver honor anisotropic filtering?
<code>WireFrame</code>	Is <code>RasterizerState.FillMode=WireFrame</code> real?
<code>OcclusionQuery</code>	Is there a real query with <code>Begin/End/completion/pixel</code> result?
<code>CustomEffects</code>	Can a non-stock effect enter the documented custom-effect path?
<code>Texture3D</code>	Does real volume texture storage and transfer exist (not necessarily shader sampling)?
<code>MultiStreamVertexInput</code>	Can repeated input rates be represented across several bindings?
<code>Instancing</code>	Can <code>DrawInstancedPrimitives</code> submit distinct instance records?
<code>StencilBuffer</code>	Is a real stencil plane available independently of a depth/3D claim?
<code>AdditiveBlending</code>	Does additive compositing avoid silent source-over degradation?

These are CNA contract questions, not a dump of native API feature bits. For example, Skia can report bounded `Texture3D` CPU storage while its general 3D pipeline remains false. GDI can offer a 2D stencil mask without claiming a depth attachment. HTML DOM additive support depends on whether the browser honors CSS `plus-lighter`.

18.7 The default polarity is permissive

`IGraphicsRenderer::SupportsCapability` returns true by default for every entry except `MultiStreamVertexInput`; `StencilBuffer` delegates to the older stencil query. This opt-out design protects mature renderers from boilerplate, but a new or narrow renderer that forgets an override overclaims.

The implementation is the authority. At the pin, `HEADLESS` and `SOFTWARE` report MRT through the default even though their bind paths do not provide simultaneous target rendering. `WEBGPU` reports MRT and occlusion queries while rejecting a second target and returning no real query object; `SDL_GPU` similarly inherits a positive query answer without a query implementation. Custom effects add two direct false positives: `WEBGPU` inherits true while retaining the null effect factory, and `BGFX` reports true on a real renderer even though its returned effect object's `CompileProgram()` always returns false. `FNA3D` is the useful opposite example: it executes six closed stock bytecode programs but truthfully reports custom effects false (Chapter 17). Appendix B records such contradictions rather than copying the Boolean as support.

A capability check is therefore necessary but, for known contradictory entries, not sufficient. Pair it with a resource/draw path and an observable oracle until the upstream report becomes truthful.

18.8 Unsupported 3D policy is separate from capability

Each `GraphicsDevice` carries `Unsupported3DGraphicsCallBehavior`, with two values:

Throw The default. A deliberately unsupported 3D call follows the renderer's established refusal path.

WarnAndStub The renderer logs once per method name and returns a safe no-op or null-object resource where the contract defines one.

Changing the policy clears the warn-once history. It does not change capability answers and does not suppress invalid arguments, disposed resources, driver failures, or unfinished work on a renderer that otherwise claims 3D. There is no environment variable for it; applications opt in through the device API.

Six deliberately 2D-focused families override the early `Ensure3DSupported` hook: `Blend2D`, `Direct2D`, `HTML DOM`, `OpenVG`, `Skia`, and `SVG DOM`. Other 2D/refusal behavior can live in individual factory or draw methods, which is why an absent override is not itself proof of 3D.

18.9 Four interface-default failure shapes

The shared renderer contract is over two thousand lines and not every default has the same semantics:

1. Effect-discarding fallback.

The ordinary effect-aware draw defaults discard the `GpuDrawParams` block, then call coloured primitive submission. Texture, lighting, fog, skinning, and custom effect state can disappear without an exception.

2. Hard refusal. Instancing and backbuffer readback defaults throw unless a narrowly defined unsupported-3D policy path intercepts the operation.

3. **Silent no-op.** Several state/debug hooks have empty defaults; a public cache may update even when no native state changes unless the shared setter guards that path.
4. **Null factory.** Optional resource/effect factories may return null, leaving the wrapper to refuse, degrade, or expose an inert object.

The renderer chapters in Part IV classify the actual destination of each family instead of inferring it from override presence. Thirty-one families own both ordinary extended draw methods; eleven inherit both. Of the inherited group, DIRECTX10 alone reaches a real coloured draw after effect state is discarded, while refusal/no-op/trace families terminate differently. Four additional override families retain a conditional coloured tail.

18.10 A portable draw claim

“The triangle rendered” is incomplete. A durable vertex-path claim names:

- the public identity and implementation family;
- the vertex declaration, packed stride, index width, and binding rates;
- the capability answers and any known report/path contradiction;
- the applied effect and whether the renderer consumed its `GpuDrawParams`;
- the native driver/host that engaged;
- the pixel, query, or structured oracle that distinguished the expected route from a coloured fallback or no-op.

With those nouns present, a failure can be localized to packing, binding normalization, capability reporting, effect dispatch, or native rendering instead of being summarized as “this renderer does not work.”

Part IV

The Renderers

Chapter 19

The Renderer Contract

Every renderer family implements the same internal C++ contract, but the contract is deliberately permissive. Some methods are pure virtual; others return null, throw, forward to a reduced operation, or do nothing. Consequently, reaching an interface method is not evidence that the selected family performed the requested work. This chapter defines that shared boundary. Chapters 20–32 own family-specific mechanisms and results; Appendix B owns the complete family inventory.

Evidence. Interface declarations, factory routes, and shared public forwarding are source-proven at 1bb2145d. Several cross-family observations below come from a retained fourteen-product audit. Those rows are historically recorded where they were not rerun for this edition. The removed ASCII identity appears only when its historical result explains the current `AsciiPostProcessEffect`; omission of a newer family means “not measured by that cohort,” not “unsupported.”

19.1 Contract shape and default behavior

The contract is one header: `modules/graphics/include/CNA/Internal/Renderers/Common/IGraphicsRenderer.hpp`. Eleven interfaces divide resource operations from device-wide dispatch:

Interface	Responsibility
<code>IVertexBufferRenderer</code>	vertex upload, update hints, and declaration
<code>IIndexBufferRenderer</code>	16- and 32-bit index upload
<code>ITextureRenderer</code>	2D texture data, interop, and optional CPU-shadow behavior
<code>ITexture3DRenderer</code> /	volume and cube storage
<code>ITextureCubeRenderer</code>	
<code>IRenderTargetRenderer</code> /	target binding, applied MSAA, and attachment reporting
<code>IRenderTargetCubeRenderer</code>	
<code>IEffectRenderer</code>	renderer-specific program compilation, binding, uniforms, and textures
<code>IOcclusionQueryRenderer</code>	query begin/end/completion/sample count
<code>ISpriteBatchRenderer</code>	batch state and the three renderer draw forms
<code>IGraphicsRenderer</code>	device lifecycle, factories, state, targets, draws, presentation, readback, and capability queries

The default bodies form five distinct contracts:

Default	Representative routes	Caller-visible consequence
Pure virtual	required 2D texture, SpriteBatch, vertex-buffer, and index-buffer factories	A family must implement the route to compile.
Null factory	volume/cube textures, render targets, effects, and occlusion queries	Public construction may still produce an object with no private implementation.
No-op	state setters, SpriteBatch options, several uniform setters	Stored public state or a successful call may have no native effect.
Reduced fallback	effect-aware draws may call a colored draw	Work can complete after effect parameters were discarded.
Throw/refusal	backbuffer readback, instancing, selected upload paths	Failure is explicit, although exception type and timing vary.

Current contract. Treat every optional method as an independent capability. A non-null renderer object, a true capability answer, and one successful draw establish different facts. Appendix G defines the evidence labels used to keep those facts separate.

19.1.1 Resource ownership and device tracking

A public `GraphicsResource` owns its renderer handle, normally through `unique_ptr`; `Texture2D` uses `shared_ptr`. The device owns neither. It records raw `GraphicsResource` addresses so it can dispose registered resources before destroying the renderer. `GraphicsDevice::Dispose()` moves and clears that list, calls each resource's virtual `Dispose()`, raises `Disposing`, and then releases the renderer. Re-entrant deregistration therefore does not invalidate the iteration.

The registry follows object addresses, not renderer-handle ownership. The `GraphicsResource` copy and default move constructors carry the device pointer without registering the destination or replacing the source address. A `Texture2D` copy has its own disposed flag and shares the private handle; moving buffers, textures, or render targets can leave the device tracking the moved-from value. If a surviving untracked object outlives device disposal, its renderer resource may be released after the native context.

Limitation. Keep live graphics resources address-stable and device-scoped. Prefer non-relocating ownership, do not move them across devices, and dispose every `Texture2D` alias before the device. The pin has no located test for copy/move followed by device-first tear-down. The sibling `docs/graphics-resource-lifetime.md` is stale where it says destructor cleanup suppresses `ResourceDestroyed`; the current implementation still calls that device hook.

Chapter 12, §12.8, covers event ordering and the remaining constructor/destructor gaps. Figure 14.1 summarizes the ownership relation.

19.1.2 Null factories and delayed failure

Null factory results do not share one public failure policy:

Private result	Public result	First use at the pin
Null occlusion query	Constructed <code>OcclusionQuery</code>	Begin/end do nothing; completion is false and count is zero.
Null volume/cube texture	Policy depends on the public wrapper	An honest <code>Texture3D</code> capability answer rejects construction; an optimistic answer can still produce a null object. Cube and volume transfers now throw instead of fabricating or discarding content.
Null 2D/cube target	Constructed target with no cached target renderer	Sampling and transfer operations reject missing storage; target binding throws before public binding state changes.
Null effect	Renderer-dependent public failure	Chapter 17 distinguishes null, invalid, throwing, and working routes.

Software supplies 2D targets and cube-texture storage but still inherits the null factories for volume textures, cube targets, and occlusion queries; several 2D families inherit a wider subset. D3D9 can also return null according to device capabilities. A cube or volume texture passed to `ShaderEffect` remains a sharp edge because its CNAEXT renderer accessor assumes that the private owner exists. Construction alone therefore does not establish usable storage. No individual capability value covers every texture and target factory.

The framework needs one consistent source-level policy—early `NotSupportedException`, or explicit nullable availability plus guards throughout. This book records the current behavior and does not choose or implement that policy.

19.1.3 Borrowed native handles

`GetWindowInternal()` and `GetRendererInternal()` are pure virtual because the shared game path needs a window, even though not every family has one. The returned pointers are borrowed:

- ordinary windowed families return their current `SDL_Window*`;
- `HEADLESS`, `SOFTWARE`, and `D3D12 HeadlessEXT` return null;
- only `SDL_RENDERER` returns its own `SDL_Renderer*`; other families normally return null for that method;
- `GameWindow::GetNativeSdlWindowEXT()` and the integer-shaped XNA `Handle` property expose the same window without transferring ownership.

Do not call `SDL_DestroyWindow` on either result or cache it across device disposal. Explicitly disposing the game-owned device does not invalidate the separate `GameWindow` wrapper, so later use of that wrapper can reach a destroyed SDL object.

`PresentationParameters::DeviceWindowHandle` is the inverse borrowed route. A nonzero value is reinterpreted as `SDL_Window*`; CNA does not validate or destroy it. The pinned implementation publishes an attached window to the global mouse and text-input services and, at teardown, clears either handle only if it still names that window. An embedding host must keep the window alive through renderer teardown. The global slots still have last-writer semantics, so two live devices can replace one another's input target; the located tests do not cover that multi-device lifecycle.

19.2 Presentation and input are family contracts

The public manager stores one presentation request. The selected family determines whether that request changes a compositor, a swapchain, a CPU buffer, only reported state, or nothing.

19.2.1 Input-coordinate routing

Window-to-logical input uses three tiers: an associated SDL renderer; otherwise a static `SDL_Window*` to `IGraphicsRenderer*` registry; otherwise raw coordinates. The inverse mouse-warp route uses the same ordering. The compact matrix below names the current behavioral classes; family chapters retain the implementation detail.

Family group	Route	Boundary
SDL_RENDERER	SDL logical-presentation conversion	Implements all five modes, including offsets; public read-side and direct write-side tests exist.
EasyGL / Canvas	Static renderer registry	Fixed-height scaling is implemented; other stored modes lack matching offset/output behavior.
SDL_GPU / WEBGPU	Static renderer registry	Five-mode math uses physical pixels while events and warps use window coordinates; high pixel density can mis-scale input.
FREEDIRECT	Associated private SDL renderer wins	Mapping appears after first present and is hard-coded to letterbox; direct CNA transform overrides are shadowed.
Vulkan, BGFX, Direct3D families	Raw fallback	Either presentation remains physical or the stored virtual request is not implemented as an output transform.
SOFTWARE / HEADLESS	No window route	Logical coordinates belong only to internal buffers or trace state.

An immediate `Mouse::SetPosition/GetState` test is weak because the setter first writes the requested logical value into `InputManager`; that round trip can pass even if the physical warp is wrong. The strongest located public evidence injects physical SDL motion into a scaled presentation and observes the converted logical point.

19.2.2 Window-registry lifetime

The second tier is one process-wide, unsynchronized `unordered_map<SDL_Window*, IGraphicsRenderer*>`. Registration overwrites by key and unregistration erases by window alone; neither operation checks ownership. EasyGL registers before several fallible constructor steps, so an exception can leave a dangling value. Two devices attached to the same borrowed window create another failure: the second replaces the first, then destruction of the first removes the second's entry. Address reuse and concurrent lookup add equivalent hazards.

This requires an ownership and thread-affinity decision in CNA. Suitable designs include an RAII registration after fallible setup, owner-checked removal, and either rejecting duplicate devices or defining multi-device routing. No located test covers constructor failure after registration, duplicate attachment, window-address reuse, or concurrent access.

19.2.3 Presentation modes

The CNAEXT `PresentationMode` property belongs to `GraphicsDeviceManager`. Direct device construction leaves the `renderer-create` argument at its fixed-height default. The manager route applies the mode before reset so implementations that derive a virtual width see the intended value.

Implementation group	Current behavior	Evidence scope
SDL_RENDERER	Maps all five requests to SDL logical presentation; fixed height derives width.	Source and focused routing tests; no five-mode pixel corpus.
SDL_GPU / WEBGPU	Computes native, fixed-height, stretch, letterbox, and overscan rectangles.	Complete local geometry; visual coverage is partial.
EasyGL / Canvas	Derives fixed-height width; other values retain dimensions without implementing bars or crop.	Source plus fixed-height coverage; browser output remains unverified.
Vulkan / BGFX / Direct3D	Stores, ignores, or partially reports the request while rendering to physical output.	Source and renderer-specific smoke evidence; not a portable five-mode contract.
Software / Headless	Resizes or simulates an internal grid.	No display compositor exists, so bar/crop semantics do not apply.
FREEDIRECT	Stores the enum; the presenter uses letterbox and has a later-resolution defect.	Exact pixels for selected paths, not five policies.

Portable code should configure the manager before normal initialization and use `NativeBackBuffer` when physical readback coordinates matter. An enum round trip is not evidence that letterbox, crop, or stretch reached output.

19.2.4 Swap interval

Public conversion preserves `Immediate=0`, `One=1`, and `Two=2`, but families interpret them differently. The manager's Boolean `retrace` property reaches only 0 or 1; 2 requires explicit `PresentationParameters`. Because `Game` constructs its first renderer before derived-game configuration, a family that consumes only the constructor argument can remain at the default interval.

Group	0 / 1 / 2 mapping	Qualification
EasyGL	Passes the exact integer at construction and reset.	Off/on is runtime-observed through GL state; interval 2 is not timing-verified.
Vulkan	Immediate-or-mailbox / FIFO / FIFO-relaxed-or-FIFO at construction.	Runtime hook is a no-op; FIFO-relaxed is adaptive tearing, not half-refresh pacing.
BGFX, WebGPU, SDL GPU, D3D11/12 D3D9	Collapse positive values to enabled VSync or FIFO. Maps all three to D3D9 intervals and resets.	2 is equivalent to 1 despite some downstream APIs supporting wider intervals. Windowed D3D9 may reject 2; CNA does not preflight that native restriction.
SDL Renderer	Initial positive values collapse to 1; runtime attempts the integer, then falls back to 1.	Tests prove safe reset and pixels, not driver timing.
Canvas, free-direct, Software, Headless	No display-interval implementation at this layer.	Stored public state may still change.

No located timing test distinguishes half refresh from ordinary VSync on any family.

19.2.5 Backbuffer format, depth format, and fullscreen

Reset first asks the renderer to normalize the requested color and depth formats, then stages the result as public state. Window resizing and virtual-resolution application form a rollback block: if either throws, CNA restores the prior presentation, adapter, touch dimensions, and window size before rethrowing. MSAA, interval, and the renderer's presentation-format update occur afterward; they are not covered by that rollback. Only D3D9 overrides the final format-update hook. A family whose applied-format query inherits the identity default can therefore echo a request even when its native attachments use a fixed format.

Group	Native attachments	Reset/fullscreen boundary
D3D9	Maps requested color and exact None/D16/D24/D24S8 depth forms.	Performs a device reset; unsupported combinations can fail after public state changed.
D3D11/12	Fixed RGBA8 and D24S8 when a windowed backbuffer exists.	SDL changes the window; DXGI exclusive fullscreen is not requested.
Vulkan / WebGPU / SDL GPU / BGFX	Selects surface/device formats independently of the XNA enums; depth is fixed or device-selected.	Resize/surface recreation follows the window, not the stored enum fidelity.
EasyGL	SDL/GL chooses the default framebuffer; its MSAA FBO is fixed RGBA8/depth24 without stencil.	Tests establish stored values and selected depth behavior, not format reconfiguration.
SDL Renderer / Canvas / free-direct	2D color target with no default depth/stencil attachment.	Fullscreen is an SDL/window or browser concern.
Software / Headless	Fixed CPU color/depth arrays, or trace state without attachments.	Fullscreen is stored state only.

An SDL fullscreen failure is cleared and ignored, so `IsFullScreen` is a request, not an actual-state query. D3D9 has the strongest located native format evidence; most other tests prove stored-field round trips or continued rendering.

19.2.6 Backbuffer readback

The interface promises top-left RGBA8 data and throws by default. The public wrapper checks a null destination, positive in-bounds rectangles against the stored backbuffer dimensions, and `elementCount < width*height`. It does not reject a negative start index, account for remaining capacity after that offset, or guard the signed area multiplication against overflow.

Renderer behavior falls into four observable classes:

Class	Families	Image returned
Active target	SDL Renderer, EasyGL, Software, free-direct, Canvas; historical ASCII	Bound target when present, otherwise the family backbuffer/logical surface. Canvas browser execution was not observed for this edition.
Physical/default backbuffer	Vulkan, BGFX, D3D9, D3D11, WebGPU	Ignores an active user target. Logical and physical dimensions can diverge on scaled/high-density output.
Synthetic	Headless	Fills with the last global clear color; draw output and source coordinates are not represented.
Explicit refusal	SDL GPU, D3D12	Inherited throw, even though both have separate render-target readback mechanisms.

There is no capability query for this distinction. A portable test must name the selected family, source image, coordinate space, and oracle.

19.3 Resource and draw defaults

19.3.1 Target, effect, and query interfaces

`IRenderTargetRenderer::GetMultiSampleCount()` defaults to zero, while `HasRealDepthBuffer(requested)` defaults to echoing the request. The first is conservative; the second can claim an attachment that the family never created. SDL Renderer overrides the depth answer to false. Applied sample count and actual attachment presence are therefore stronger facts than constructor arguments.

`IOcclusionQueryRenderer` itself is pure virtual, but its factory is optional. `IEffectRenderer` requires program compile/bind/status methods while many scalar, array, and texture setters have no-op defaults. A valid program can consequently discard selected parameters. Chapter 17 owns the renderer-specific matrix.

19.3.2 Buffer updates and instancing

Dynamic buffer constructors currently use the same factory as static buffers; the constructor's `dynamic` flag is ignored. Update options still have meaning on EasyGL, SDL GPU, D3D9, and D3D11, which map discard/no-overwrite to distinct native operations. Vulkan and BGFX inherit the option-discarding default; WebGPU, D3D12, Software, and Headless override it without preserving option identity. The options-taking public wrappers also fail to refresh the shared CPU shadow, so a later public `GetData()` can throw or return stale bytes.

The declaration route is stricter: `SetVertexDeclaration()` is pure virtual, and raw uploads send the complete declaration. A family may translate it, remember it for draw-time validation, or reject it. Chapter 18 defines the packed-stream contract.

Effect-aware `DrawPrimitivesEx` defaults can discard `GpuDrawParams` and enter a colored draw. The exhaustive family audit found DIRECTX10 as the consequential live inherited fallback: an effect request can become a plausible untextured colored result. Several families override the method but still use a colored fallback for unmatched layouts. Appendix B classifies inherited, hybrid, and renderer-owned routes.

Instancing has a throwing default. Shared code now transports up to sixteen bindings, preserving slot, renderer buffer, stride, vertex offset, instance frequency, and count. It rejects partially overlapping semantic declarations and validates per-vertex and per-instance ranges before submission. The broad portable subset is one vertex stream plus one instance stream; multiple streams of either rate additionally require `MultiStreamVertexInput`. EasyGL profiles with native support, Magnum, Vulkan, BGFX, WebGPU, and D3D11/12 have focused pixel evidence for specific layouts. D3D9 has a native step-frequency route but weaker shared-oracle coverage. Headless records counts, and SDL GPU inherits a default-true capability despite the throwing callback; neither establishes working instancing.

19.3.3 Texture updates and readback

Every current 2D texture implementation overrides level-zero updates. The higher-mip default is still reachable on SDL GPU and silently discards the upload; Software explicitly discards higher levels, while SDL Renderer, Canvas, and free-direct throw. EasyGL, Vulkan, BGFX, WebGPU, and D3D9/11/12 implement higher-level storage. GPU-sampled or native-subresource

tests, not public CPU-shadow round trips, establish those paths. WebGPU additionally regenerates the mip chain after level-zero writes, which can overwrite previously authored higher levels.

Volume and cube readback now returns a completion Boolean. Public wrappers convert results only after a true response; null/false raises `NotSupportedException` without modifying the destination. EasyGL, Vulkan, BGFX (when transfer capabilities exist), WebGPU, SDL GPU, D3D9/11/12, and LLGL have native transfer routes. Skia and selected Software cube paths use exact CPU storage. Headless refuses instead of manufacturing transparent pixels. The shared fixtures distinguish slices, faces, subrectangles, mips, and unchanged sentinel buffers.

`RenderTargetCube` inherits public texture upload methods, but most renderer-target objects explicitly refuse them; EasyGL and Skia are the notable storage implementations. Target transitions are a separate concern. Families with deferred work must finalize the outgoing face before another cube face, 2D target, MRT set, or backbuffer becomes active. D3D9/11/12, BGFX, WebGPU, SDL GPU, LLGL, and Vulkan have renderer-owned transition mechanisms. EasyGL retains per-face MSAA storage, but switching faces on the same cube can skip outgoing finalization; callers that need freshly resolved/sampleable faces should unbind between them.

19.4 Targets, clears, viewport, and scissor

19.4.1 Render-target identity and usage

Singular 2D and cube calls normalize into one descriptor list. Each descriptor names a 2D target or cube face, dimensions, and applied sample count. The first descriptor controls viewport, scissor, discard clearing, and usage policy. The shared layer rejects null/disposed targets, nonzero 2D slices, dimension or sample-count mismatches, duplicate subresources, and unsupported mixed/cube MRT shapes before native binding.

Three rules are cross-family:

1. CNA has no FNA-style redundant-binding early return. Rebinding the same discard target clears it again and can repeat resolve/finalization work.
2. `PreserveContents` and `PlatformContents` both pass the preserve Boolean; exact `DiscardContents` triggers the shared implicit clear.
3. The discard mask includes color and each depth/stencil aspect the first descriptor reports as present. Backbuffer usage is outside this policy.

Focused fixtures distinguish discard from preserve/platform for major GPU families, including cube and depth/stencil cases. Software has CPU color, depth, and stencil storage; Headless reports trace changes, not preserved pixels. WebGPU and Software explicitly reject more than one target. D3D12 binds multiple RTVs, but current stock draws write target zero. A true MRT capability answer must therefore be paired with a successful multi-target bind and an oracle for the intended outputs.

19.4.2 Clear selection and ordering

`GraphicsDevice` converts color once, masks unavailable target aspects, and dispatches one of seven non-empty target/depth/stencil combinations. The significant remaining differences are concise:

- Vulkan, BGFX, WebGPU, and SDL GPU preserve ordered clears across deferred pass segments; focused pixel suites cover masks, chronology, and several target kinds.

- D3D11/12 issue native immediate clears. D3D9 further gates depth/stencil bits using its backbuffer format, even when a differently formatted target is active.
- EasyGL's color-only hook also includes the GL depth bit without forcing depth writes, so prior depth-mask state can change a nominal target-only clear.
- SDL Renderer, Canvas, and free-direct mask depth/stencil away on their public 2D paths; their direct 3D hooks may throw but are not reached by the masked public call.
- Software handles color, depth, and stencil in CPU arrays; Headless records trace identity without attachment semantics.

There is no capability value for aspect selectivity or same-pass ordering. Tests must observe the requested aspect after intervening work; an unchanged color pixel alone does not prove that a depth-only clear occurred.

19.4.3 Viewport and scissor

The device calls each renderer hook before committing the corresponding public property, and scissor enable arrives separately through `RasterizerState`. A successful getter round trip therefore establishes accepted bookkeeping, not rasterization.

Vulkan, BGFX, WebGPU, and SDL GPU capture viewport/scissor per deferred command and have focused pixel evidence. EasyGL and D3D9/11 apply both through native state; EasyGL `SpriteBatch` resets the viewport to the full destination and does not restore it. D3D12 applies custom viewport and depth range but installs a full-target scissor on every draw. SDL Renderer applies a nonempty clip rectangle regardless of `ScissorTestEnable` and has no viewport hook. Software applies the rectangle and enable state to its 2D and 3D clip paths; its custom viewport is covered for `SpriteBatch`, while the 3D NDC mapping still spans the full framebuffer. Canvas, free-direct, and historical ASCII do not implement both semantics. Headless validates/traces but does not rasterize.

Target switches reset viewport and scissor to destination bounds. Construction establishes both full rectangles. Present and a same-size reset preserve custom values; a logical-size or physical default-viewport change resets both. No capability value reports either feature.

19.5 Capability answers and operational evidence

`GraphicsCapability` contains `ThreeD`, depth/stencil, MSAA, MRT, anisotropy, wireframe, occlusion query, custom effects, 3D textures, multi-stream input, instancing, stencil, and additive blending. The shared default returns true except for multi-stream input; stencil uses a separate hook. This optimistic polarity makes an answer useful only when the family has audited the corresponding operation.

Figure 18.1 shows the required progression from reported Boolean to an observed result.

Confirmed contradictions and scope mismatches at the pin include:

Capability	Affected families	Reachable behavior
MRT	Headless, Software, WebGPU	Trace-only behavior or explicit rejection, not a usable multi-attachment draw contract.
Occlusion query	WebGPU, SDL GPU, Software; Headless is synthetic	Null factory or non-GPU bookkeeping despite true reporting.
Custom effects	BGFX, WebGPU	Always-invalid object, null factory, or a later throwing route.
Instancing	Headless, SDL GPU	Trace-only result or inherited throwing callback.
MSAA	Software; several device-dependent families are overbroad	Software implements 4x color MSAA for render targets but not its backbuffer; Vulkan/WebGPU/SDL GPU/D3D9 can negotiate a request down while still reporting true.
Anisotropy	SDL GPU, Software; D3D9 is overbroad	Stored/ignored value, discarded sampler state, or device limit not consulted by the query.

For MSAA, the applied value written back to presentation parameters or the target's own `MultiSampleCount` is stronger than the Boolean. For all capabilities, the defensible sequence is query, factory, operation, engagement, then a suitable behavioral or pixel oracle.

19.5.1 Recovery, device-loss, and markers

`SetContextRecoveryEnabled()` changes shared texture-shadow policy before invoking the renderer hook. Disabling it permits a successful full upload to discard CPU pixels, so later readback or a partial update can fail even when the family hook is empty. Re-enabling recovery does not reconstruct discarded data. Call it after device construction but before loading resources; the header's "before device initialization" wording cannot be literal because the method belongs to an existing device.

F9 and F10 key-down events pass through input first, then call renderer loss/restore hooks. EasyGL uses them for desktop or browser context-loss routes. D3D9 exposes a staged lost/reset lifecycle with event and pixel coverage. Other audited families generally inherit no-ops. `SetStringMarkerEXT()` is also a no-op by default; Vulkan can queue a debug-utils marker when the extension function exists, while D3D9 throws. Successful submission is not proof that a native marker was emitted.

19.6 Selection and factory boundary

Chapter 20 is the canonical identity/family registry. At this edition, one configure-time public identity maps to one of 46 implementation families; the five EasyGL identities share one family. Platform and dependency gates run before the factory is compiled.

The selected module provides exactly one `CreateGraphicsRenderer(const GraphicsRendererCreateArgs&)` returning `unique_ptr<IGraphicsRenderer>`. Its create arguments include window, virtual resolution, presentation mode, recovery policy, requested samples and interval, plus CNAEXT backbuffer/depth formats, fullscreen, and profile. Field presence is not a requirement that every family consume the field; the presentation tables above show the current differences.

Historical note. The original audit narrated individual task numbers, intermediate tests, and repaired defaults throughout this chapter. The current contract now incorporates those fixes. History remains here only where it explains a live hazard: permissive defaults allowed

plausible reduced draws; CPU-shadow round trips masked missing GPU uploads; and direct helper tests bypassed the public coordinate route. The detailed dated campaign remains in the repository audit artifacts.

19.7 Implications for users and implementers

For application code:

- choose a renderer by the operations and evidence your program needs, not by selector name;
- treat capability answers as preflight hints and verify applied values or results;
- keep resources address-stable and within device lifetime;
- distinguish logical presentation coordinates from physical readback pixels;
- expect custom effects, MRT, queries, instancing, and readback to require family-specific qualification.

For a renderer implementation, overriding a virtual method is only the beginning. A complete claim for one operation needs a reachable factory, shared public routing, applied state, correct target and lifetime behavior, renderer-engagement evidence, and a discriminating oracle. That is the standard used by the family chapters that follow.

Chapter 20

Renderer Selection and Identity

CNA exposes 50 public renderer identities mapped to 46 implementation families. The normal build still contains one family, but `v0.1.0-alpha.1` also has an opt-in multi-renderer mode: configuration chooses the compiled set and its default, and the process chooses one available identity before constructing its first `GraphicsDevice`. Keeping compiled set, default, selected and active renderer separate prevents the most consequential portability and lifetime mistakes around CNA’s registry.

Authority order. The current identities are jointly pinned by `modules/core/include/CNA/GraphicsRendererType.hpp` and `cmake/RendererSelection.cmake`. The family mapping is stated by `docs/renderer-registry.md` and the physical module list. Older counts in plans, generated scripts, and migration documents are historical snapshots. This edition always prints **50 identities / 46 implementation families**, with both nouns present.

20.1 Identity, family, and native API are different axes

A *public identity* is the spelling supplied to `CNA_GRAPHICS_RENDERER`, such as `VULKAN`, `WEBGPU`, or `OPENGLES3`. It becomes a `GraphicsRendererType` enumerator and a `CNA_RENDERER_*` compile definition.

An *implementation family* is the source module that provides a generated-registry factory. Five public identities share the `easygl` family: `OPENGLES2`, `OPENGLES3`, `OPENGL33`, `WEBGL1`, and `WEBGL2`. That one five-to-one mapping accounts for the arithmetic: $50 - 5 + 1 = 46$ families.

A *native API* is what the implementation ultimately calls. Usually it follows the family name, but not always. BGFX is itself an abstraction layer that can submit through OpenGL, GLES, Vulkan, Metal, or Direct3D. FNA3D similarly selects a driver below CNA. HEADLESS and STUB call no GPU at all. SOFTWARE and PORTABLEGL rasterize on the CPU. Asking “which API is selected?” is therefore ambiguous until the axis is named.

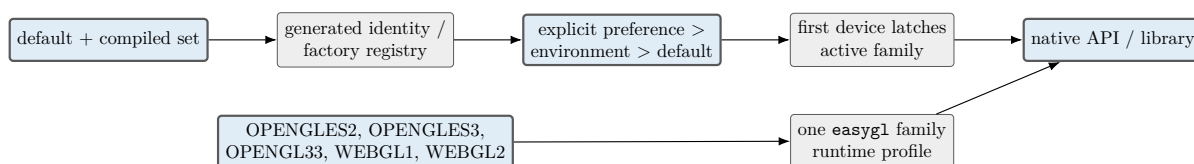


Figure 20.1: Compiled set, default, runtime selection, active family, and native API are distinct. The five-to-one EasyGL mapping explains the 50-identity / 46-family count.

20.2 The complete registry, grouped by implementation

Family group	Public identities
EasyGL, five profiles	OPENGLES2, OPENGLES3, OPENGL33, WEBGL1, WEBGL2
Independent OpenGL-shaped	OPENGL1, OPENGL2, OPENGL4, OPENGLES1, PORTABLEGL
Native modern	VULKAN, SDL_GPU, WEBGPU, METAL, IGL
Cross-API abstractions	BGFX, MAGNUM, LLGL, DILIGENT, SOKOL, WICKED, FNA3D
Direct3D and Direct2D	DIRECTX9, DIRECTX10, DIRECTX11, DIRECTX12, DIRECT2D
DirectX / retro ladder	DIRECTX1, DIRECTX2, DIRECTX3, DIRECTX5, DIRECTX6, DIRECTX7, DIRECTX8, FREEDIRECT, GLIDE
2D raster and vector	SDL_RENDERER, GDI, SKIA, BLEND2D, OPENVG, NANOVG
Browser DOM	CANVAS, HTML_DOM, SVG_DOM, PIXIJS
Diagnostic / CPU	HEADLESS, SOFTWARE, STUB, TINYGL

These groups are explanatory, not another registry. Appendix B remains the quick reference; the chapters that follow group families by the API they speak and how their shaders arrive.

20.3 One renderer remains the default build mode

The normal interface is one cache variable:

```
1 cmake -S . -B build -DCNA_GRAPHICS_RENDERER=VULKAN
2 cmake --build build
```

CMake validates the spelling, applies platform and dependency gates, sets the matching `CNA_RENDERER_*` default definition, and builds the selected family. In this mode `GraphicsRendererSelection::GetAvailable()` has one entry; requesting a different one fails rather than inventing a renderer that is not linked.

The default is platform-sensitive:

- Emscripten selects `WEBGL2`;
- Linux selects `OPENGLES3`;
- other hosts select `SDL_RENDERER`.

Treat those defaults as convenience, not as a portability promise. A game depending on MRT, hardware instancing, or a particular shader source language should select and test an explicit identity.

20.4 Opt-in multi-renderer builds

`CNA_GRAPHICS_RENDERERS` is a semicolon-separated list of identities to compile into one binary. `CNA_GRAPHICS_RENDERER` remains singular: it names the default and must be a member of the list. For example:

```
1 cmake -S . -B build-multi \
2   -DCNA_GRAPHICS_RENDERER=HEADLESS \
3   '-DCNA_GRAPHICS_RENDERERS=HEADLESS;SOFTWARE;STUB'
4 cmake --build build-multi
```

CMake removes duplicate identities, puts the default first, emits `CNA_MULTI_RENDERER`, builds the required family targets, and generates the descriptor and factory registry. A list is not automatically valid. `PORTABLEGL` cannot coexist with a family that imports the real global `gl*` symbols; `GDI` and `SOFTWARE` would compile the same translation units with different bodies; `GLIDE` pins the binary to its 32-bit ABI; and a single toolchain cannot combine Windows-only, Emscripten-only, and macOS-only partitions. The validation fails during configuration with the conflicting pair and reason.

All five EasyGL identities may now coexist when their platform partition permits it: the GL profile became runtime family state. They remain five public identities backed by one source family, so multi-renderer support does not change the 50/46 taxonomy.

20.5 Runtime choice, precedence, and latch

`CNA::GraphicsRendererSelection` is a process-wide CNAEXT policy surface. Before the first graphics device, a program may call `SetPreferred(type)` or `SetPreferred(name)`. The exact precedence is:

1. an explicit `SetPreferred()` call;
2. the `CNA_GRAPHICS_RENDERER` environment variable (or the equivalent Emscripten module property when no environment value exists);
3. the configured build default.

`GetSelected()` reports the identity CNA will attempt without freezing it; `GetAvailable()` and `IsAvailable()` report the compiled set. Construction of the first `GraphicsDevice` latches selection. `GetActive()` becomes meaningful only after a renderer was created and may differ from `GetSelected()` only when explicit fallback succeeded. Changing preference after the latch throws. Reset and multisample recreation may rebuild the *same* renderer; no supported operation migrates live textures, buffers, effects, render targets, query objects, or presentation state to a different family.

Unavailable or failed initialization is an error by default. A caller may opt in to an ordered fallback chain or automatic fallback; CNA records skipped/not-compiled and failed attempts in `GetFallbackHistory()`. This makes substitution observable. Automatic ordering derives from the registry's maturity and category metadata, with CPU renderers after mature GPU routes and STUB last; it is a recovery policy, not a claim of equivalent output.

20.5.1 Legacy Boolean options are a compatibility surface

The `CNA_RENDERER_*` Booleans still exist for scripts that predate the single selector. If any are enabled, exactly one must be true. Mixing them does not create a multi-renderer build; the configure step terminates instead. New build instructions should prefer `CNA_GRAPHICS_RENDERER`, because one string is easier to cache, display, and validate than fifty mutually exclusive switches. Use `CNA_GRAPHICS_RENDERERS` for an actual set.

20.6 Names that no longer select anything

Several spellings remain common in old documentation but are not aliases:

- `EASYGL` is a family name, not a public selector. Choose one of its five profiles.
- `ASCII` was removed as a renderer. Its reusable behavior is now the `AsciiPostProcessEffect` class in the `CNA::Graphics` namespace and `modules/graphics-ext` module.

- DX3 does not select anything. `DIRECTX3` is the real DirectX 3 family; `FREEDIRECT` is the separate renderer built over the sibling free-direct library.
- `D3D9`, `D3D11`, and `D3D12` are not public selector aliases; use `DIRECTX9`, `DIRECTX11`, and `DIRECTX12`.
- The former GLES identity was made explicit as `OPENGLES3`; GLES 2 is the separate `OPENGLES2` profile.

Unknown names fail at configure time. That strictness is useful: silently redirecting `EASYGL` to whichever profile happens to be convenient would erase the capability differences Chapter 22 must preserve.

20.7 Platform gates are part of identity

An entry in the registry means CNA has an implementation family, not that the current host can build it. The most important gates are:

- `WEBGL1`, `WEBGL2`, `CANVAS`, `HTML_DOM`, `SVG_DOM`, and `PIXIJS` are Emscripten identities; native builds reject them.
- The native EasyGL profiles are rejected under Emscripten.
- DirectX, Direct2D, GDI, and Glide are Windows-shaped families. Some can be exercised under Wine or Proton; that is a verification route, not a change in their API ownership.
- `METAL` requires Apple platforms.
- `FREEDIRECT` requires the sibling `../free-direct` checkout; EasyGL profiles require `../easy-gl`. Other families have their own system, vendored, pinned-artifact, or `FetchContent` dependencies.

The configure error is usually the most faithful evidence available on an unsupported host. Forcing past it would not prove the renderer portable; it would only bypass the source tree's declared boundary.

20.8 How the selected identity reaches C++

`GraphicsRendererType.hpp` owns the public enum and `GetCurrentGraphicsRendererName()`. For EasyGL it decodes the profile compile definition back into the correct public identity, rather than returning a generic family name. `GraphicsDevice::GetGraphicsRendererType()` exposes the result to game code.

The per-family factory boundary is deliberately smaller:

```
1 std::unique_ptr<IGraphicsRenderer>
2 CreateGraphicsRenderer(const GraphicsRendererCreateArgs& args);
```

The generated registry associates each compiled descriptor with the appropriate function. `GraphicsRendererCreateArgs` carries platform window/native handles, virtual dimensions, presentation mode, swap interval, MSAA request, recovery policy, graphics profile, and CNAEXT presentation fields. A field's presence does not mean every family honors it: Chapter 19 shows why the contract includes truthful no-op, refusal, and renderer-specific interpretations.

20.9 Family selection is not BGFX driver selection

Selecting BGFX chooses CNA’s BGFX family. The environment variable `CNA_BGFX_RENDERER` can then ask bgfx to use OpenGL, GLES, Vulkan, Metal, D3D11, D3D12, or its own no-op driver. That second choice happens below CNA’s identity layer: `GetGraphicsRendererType()` still reports BGFX, and all CNA capability and lifetime behavior still belongs to `BgfxRenderer`.

The same reasoning applies when FNA3D selects a native driver. A lower layer changing from OpenGL to D3D11 does not turn the public identity into `OPENGL4` or `DIRECTX11`; it changes how one implementation family fulfills its contract.

20.10 Mechanical checks and the remaining CI trap

Registry validators compare the enum, CMake selector list, descriptors, factories, categories, maturity values and generated metadata. At this point those sources settle fifty identities and forty-six physical families. The release changelog’s claim of 49 identities and the runtime selection document’s phrase “45 factories” are both one low; neither overrides the registries.

One live build-system defect demonstrates why registry checks and workflow checks are different. Two CI workflow areas still pass `EASYGL`, even though the selector rejects it. The identity checker can remain green while those jobs fail before compilation, because it proves the two registries agree; it does not parse every workflow command line. The defect is recorded for upstream repair rather than patched from this book.

20.11 A selection checklist

Before choosing a renderer for a port, answer in this order:

1. Is the public identity buildable on every target platform?
2. Does its *implementation behavior* provide the required 2D/3D, target, effect, and query paths? Do not stop at a default-true capability flag.
3. What verification tier exists on each platform: real-XNA oracle, pixel tests, smoke test, compile-only gate, or no execution?
4. Does a lower abstraction layer add a second runtime driver choice?
5. Are the required sibling checkouts or fetched dependencies acceptable to the product?

The registry answers *what can be selected*. The rest of Part IV answers the harder question: *what that selection is actually proven to do*.

Chapter 21

OpenGL I: The Independent Compatibility Ladder

CNA does not have one OpenGL renderer with a version switch. It has several independent families that happen to speak related APIs. This chapter follows the compatibility ladder: desktop OpenGL 1.x through `OPENGL1`, the GLSL 1.10 transition through `OPENGL2`, mobile fixed function through `OPENGLES1`, and the CPU `TINYGL` family. Chapter 22 then covers the five-profile EasyGL family, `OPENGL4`, and `PORTABLEGL`.

That distinction matters operationally. Selecting a newer identity does not mutate an older implementation, and a fix in one family does not automatically reach another. The renderer contract is shared; context creation, resource ownership, draw submission, and evidence are not.

21.1 Four identities, four implementation families

Identity	Pipeline	Shader delivery	Deliberate boundary
OPENGL1	desktop fixed function	none	OpenGL 1.x concepts; optional FBO and query entry points are detected at run-time
OPENGL2	programmable compatibility profile	GLSL 1.10 source compiled by the driver	compatibility-profile state plus shaders; MRT and queries depend on loaded functions
OPENGLES1	mobile fixed function	none	OpenGL ES 1.1; no MRT and no occlusion-query contract
TINYGL	CPU fixed function	none	one software framebuffer; no render targets, cube/volume textures, or queries

All four implement CNA directly. None routes through EasyGL, and none is an alias for an EasyGL profile. The apparently similar names therefore describe API eras, not modes of one code base.

21.2 OPENGL1: a modern contract on a fixed pipeline

`OPENGL1` is available only on Linux or Windows and links the system OpenGL library. There is no shader stage to compile, cache, or bind. CNA state is translated into texture environment,

matrix, lighting, fog, alpha, blend, depth, stencil, and client-array state that the fixed pipeline understands.

This is not equivalent to a 2D fallback. Both extended primitive entry points are native overrides and submit fixed-function geometry. A `GpuDrawParams` packet must be reduced to the subset expressible by the pipeline rather than silently falling through the base colored path. The result can draw 3D geometry, but custom programmable effects are an architectural impossibility, not a missing weekend task.

21.2.1 Optional extensions are part of the public truth

OpenGL 1.x predates framebuffer objects and standardized occlusion queries. The renderer therefore treats both as runtime discoveries:

- `RenderTarget2D` is available only when the required FBO entry points were resolved. Binding the default backbuffer remains valid without them.
- multiple render targets are not implemented;
- occlusion queries are available only when the ARB or later core query functions can be loaded.

This conditional shape is more honest than returning an unconditional capability bit based only on the build identity. An executable can be built successfully and still meet a driver whose OpenGL 1.x extension surface is narrower.

The platform gate and system-library dependency are build facts; the FBO and query clauses are runtime facts. A build matrix can prove the former, but only execution against the selected driver can prove the latter.

21.3 OPENGL2: the GLSL 1.10 bridge

OPENGL2 is the transition renderer. Its context remains a compatibility-profile world, while vertex and fragment work is expressed as GLSL 1.10. That combination makes it materially different from both neighbours: it is programmable unlike OPENGL1, but cannot be served by EasyGL's ES-oriented shader rewriting and profile macros.

The renderer owns real extended primitive paths, render targets, and effect-aware submission. Its optional ceiling again follows loaded functions rather than the name alone:

- 2D render targets are supported;
- MRT is conditional on `glDrawBuffers`;
- occlusion queries are conditional on query creation being available.

GLSL 1.10 also explains why shader source is an integration boundary. A shader written only for modern core GLSL cannot simply be handed to this renderer, and an ES shader is not made desktop-compatible merely by changing its version line. Attribute syntax, fragment outputs, precision qualifiers, and built-ins belong to the selected language profile.

21.3.1 Why keep this renderer beside EasyGL?

Compatibility hardware and virtualized drivers can expose a usable OpenGL 2.x path without providing the ES or modern-core environment expected elsewhere. Keeping this implementation separate preserves that deployment option and makes its compromises reviewable. Folding

it into EasyGL would hide a different shader language and a different context contract behind a single selector.

21.4 OPENGLES1: fixed function on the mobile branch

OPENGLES1 targets `libGLESv1_CM` and `GLES/gl.h`. The dependency is not vendored, and the module refuses configuration when the system library is absent. Stock Mesa builds commonly disable GLES1, so the repository supplies a dedicated ES1-enabled Mesa environment script for this renderer’s evidence path.

Like `OPENGL1`, it has no shaders. Unlike `OPENGL1`, its API ceiling is the smaller ES 1.1 fixed-function surface. It implements 2D targets but neither MRT nor occlusion queries.

21.4.1 A hybrid draw route, not unconditional effect parity

The extended draw methods are overridden, so classifying this renderer as a base fallback would be wrong. Their internal dispatch is nevertheless hybrid. Combinations the fixed pipeline can represent are handled natively; skinning, PBR, custom effects, instancing, and other unavailable combinations are sent to the colored tail. The call may therefore draw geometry while losing the semantic content of `GpuDrawParams`.

This is narrower than the unconditional downgrade described for `DIRECTX10` in Chapter 19: supported fixed-function cases remain native here. It is still a crucial portability warning: successful submission does not prove that the requested effect model survived dispatch.

21.5 TINYGL: fixed function as a CPU rasterizer

TINYGL fetches the pinned C-Chads TinyGL revision and renders a fixed-function OpenGL 1.x-style pipeline into its own CPU framebuffer. It owns ordinary and indexed extended draws, including colored, textured, colored-textured, normal-textured and skinned layouts. It reports only `ThreeD` and `WireFrame` among the graphics capabilities.

The single-context framebuffer is also the hard boundary: there is no FBO model, so `RenderTarget2D`, cube targets and MRT are unavailable; texture cubes, volume textures, occlusion queries, programmable effects, instancing, multi-stream input, MSAA and anisotropic filtering are refused. This is a real CPU 3D renderer, not the `SOFTWARE` family and not a native OpenGL context. The tag contains fourteen TinyGL suites with 113 named checks and a dedicated workflow; this edition records their presence rather than claiming a fresh workflow run.

21.6 The same contract does not imply the same proof

The three families require different verification environments.

Identity	Strongest practical path		Remaining evidence boundary
OPENGL1	native Linux or Windows system OpenGL, including extension- dependent cases		driver variation in FBO and query availability
OPENGL2	compatibility-profile	desktop OpenGL with real shader compilation	loaded-function and GLSL- driver variation

Identity	Strongest practical path	Remaining evidence boundary
OPENGLES1	custom Mesa build with GLES1 enabled	specialized environment is not the stock developer path
TINYGL	native CPU execution with framebuffer readback	fixed-function subset and fetched dependency

Registration totals are deliberately omitted here. They drift as cases are split or merged and do not measure assertion quality. The useful question is whether a test reaches a real context, submits pixels through the family-specific path, and observes the result.

21.7 Choosing among the compatibility identities

Choose `OPENGL1` only when fixed-function desktop compatibility is itself a goal. Choose `OPENGL2` when GLSL 1.10 and compatibility contexts match the deployment target. Choose `OPENGLES1` when the ES 1.1 environment is intentional and its fixed-function ceiling is acceptable. For a broader programmable profile set, continue with Chapter 22. Choose `TINYGL` when a CPU-only fixed pipeline is the intended deployment; a similar API surname does not erase the implementation boundary.

Before shipping on any of the four, verify four things on the target implementation:

1. the selected identity reported at runtime is the one configured at build time;
2. the render-target and query features actually used by the game are available;
3. representative effect-aware draws retain their intended semantics rather than taking a reduced path;
4. pixel evidence is collected from the real context, not only from a construction or no-throw smoke test.

That checklist is the recurring lesson of the compatibility ladder: an old API can implement a surprising amount of CNA, but its limits must be stated as limits of a particular path, not blurred into the capabilities of the OpenGL family as a whole.

Chapter 22

OpenGL II: Programmable, Web, and Portable Profiles

Five public profiles — `OPENGLES2`, `OPENGLES3`, `OPENGL33`, `WEBGL1`, and `WEBGL2` — share CNA’s internal EasyGL implementation and the sibling `easy-gl` project introduced in Chapter 2. Linux defaults to `OPENGLES3`; Emscripten defaults to `WEBGL2`. `EASYGL` itself is not an accepted public selector. This shared family is only one half of the chapter: the independent `OPENGL4` and CPU-only `PORTABLEGL` implementations return in the closing sections after the common EasyGL path has established the programmable-profile baseline.

22.1 Documentation status at the pin

Before any specific claim in this chapter: `docs/easygl_bugs.md`, the primary per-bug record for this renderer, carries its own explicit staleness banner. It was last fully written weeks before this book’s own source material was gathered, and states plainly that it “has not been re-audited against thousands of lines of EasyGL changes since.” Only two of its rows have been independently re-confirmed as fixed; everything else is “spot-checked, not exhaustively re-verified.” This chapter treats that banner as Chapter 1 asked this book to treat every dated claim: reproduced because it is evidence of real methodology, not because it is guaranteed current.

22.2 Confirmed fixed since the original bug list

The current source closes substantially more than the old list records. Real anisotropic filtering forwards the requested `maxAnisotropy` to the GPU, gated on the `GL_EXT_texture_filter_anisotropic` extension, and cube render targets have a real, parameterized `depthFormat`, replacing what had been a hardcoded “always has depth” boolean. Further corrections below cover MRT lifetime, wireframe, colour-write masks, mip storage, fog, custom viewport handling, and vertex-binding transport.

22.3 Confirmed pre-existing gaps

The old MRT leak diagnosis is no longer current. EasyGL now owns one reusable MRT frame-buffer, calls `FinalizeCurrentMRT()` before every destination transition, resolves/regenerates every surviving member, clears every old attachment, and installs the new ordered set atomically. Its public MRT fixture now writes distinct outputs to one through four attachments and covers set replacement, per-target colour masks, MSAA resolve, usage, readback, and immediate sampling. The historical two-attachment failure expected an old blue value to survive a `DiscardContents` bind through a single-output `BasicEffect`; that expectation could

not diagnose whether attachment 1 was active. Separately, an upstream **easy-gl** GL-state-desynchronization bug (not a CNA defect) remains visible in that sibling project's own resource smoke test.

Two profile ceilings remain. On the ES 3.0 path, a nonzero base vertex still reaches **glDrawElementsBaseVertex** without checking the ES 3.2/extension entry point. **meta-gl** terminates if that function pointer is absent, so this is a deterministic process-failure risk, not a safe silent fallback. The ES 2 path is separately corrected by rebasing each enabled attribute pointer and restoring it after the draw. Also, **OcclusionQuery::PixelCount()** returns only 0 or 1 on the ES profile, because the route uses **GL_ANY_SAMPLES_PASSED**: a boolean visibility result rather than a true sample count.

The old wireframe and fog diagnoses are both stale. **EasyGL** re-expands triangle indices to **GL_LINES**, so it needs no **glPolygonMode**; the shared asymmetric-triangle oracle proves empty interiors and all three edges, and **SupportsCapability(WireFrame)** now reports that implementation (conditionally on 32-bit element-index support for ES 2). Stock-effect fog is computed from FNA's CPU-prepared view-space fog vector as **1-saturate(dot(position,fogVector))**, including the post-skin position. The bare **DrawColoredPrimitives** callbacks receive matrices but no **GpuDrawParams** by contract; **EasyGL**'s effect-aware stock path is its separate **Draw*PrimitivesEx** route, so the colored callback alone is not evidence that a supplied effect payload was discarded.

The old colour-mask row is stale too. **ApplyBlendState()** stores all four **ColorWriteChannels** values and **ApplyCurrentColorWriteMasks()** submits slot 0 through **glColorMask**; profiles with indexed-mask support submit every active MRT slot independently, while a profile without it rejects unequal active masks instead of silently collapsing them. Because GL clear operations respect the current mask but XNA clears do not, each **EasyGL** clear temporarily enables all active channels and then restores the requested masks.

Missing features confirmed still open include: the renderer-level **preserveContents** factory flag (ignored), although the chapter's old conclusion that all targets therefore behave as **DiscardContents** is stale — shared **GraphicsDevice** code black-clears exact **Discard** on every CNA bind, while this renderer's persistent FBO preserves both **Preserve** and **Platform** when not explicitly cleared (pixel-proven for **Discard** versus **Preserve**); ordinary **Texture2D::SetData** uploads allocating and accepting explicit mip levels but never automatically downsampling level 0 (render-target unbind does generate its chain, a separate contract); **Texture3D** and **TextureCube** honoring **mipMap** by allocating every level but ignoring the requested **SurfaceFormat** and always using **RGBA8**; neither type being registered with the GL-context-loss recovery mechanism the way ordinary 2D textures are; **EasyGLEffectRenderer::Unbind()** being a pure no-op; and **SpriteBatch** carrying a 65,535-index limit from its **uint16_t** index type, silently wrapping or corrupting any batch exceeding roughly 16,383 quads. **VertexBuffer::GetData** / **IndexBuffer::GetData** have no real per-renderer GPU readback path on this renderer, or on any other — Section 22.6 below corrects an earlier pass's own overstatement of this gap and explains precisely what does and does not work.

One gap is formally **BLOCKED** rather than merely open: forwarding a real, non-**ColorSurfaceFormat** all the way to the GPU conflicts directly with an already-shipped, already-tested contract on **Texture::ValidateFormat** (Chapter 14) — closing this gap would require revisiting a decision the project has already committed tests against, not simply writing new code.

The default backbuffer has a related but distinct selection gap. The shared factory arguments now carry **BackBufferFormat**, **DepthStencilFormat**, and **IsFullScreen**, but the **EasyGL**

factory does not pass those three fields into its constructor. SDL/GL chooses framebuffer 0 after CNA asks for the selected GL profile and eight stencil bits. When backbuffer MSAA is active, CNA replaces that path with its own fixed RGBA8 plus Depth24 renderbuffers — still not the requested enums, and with no stencil attachment in that MSAA FBO. `easygl_depth_format_test` is intentionally honest about this: it cycles all four public depth values and proves only storage/no throw. The window can separately change through the shared SDL fullscreen call, whose failure is non-fatal. The full requested/native distinction is in §19.2.5.

EasyGL is the only product whose renderer hook adds real work to `GraphicsDevice`’s recovery setter. Calling it with `false` after device construction but before content loading both enables the shared `Texture2D` CPU-shadow discard and prevents subsequently-created GL resources from joining EasyGL’s recovery registry. It does not unregister existing resources or reconstruct pixels after re-enabling, so toggling is not a reversible migration. The global F9/F10 debug keys then expose two platform shapes: on desktop either key performs a complete synchronous loss-and-restore cycle, while on Web F9 requests loss and F10 separately requests restore through asynchronous browser events. The cross-product contract and evidence boundary are in §19.5.1.

22.3.1 Finding the SpriteBatch quad ceiling

The `uint16_t` index-type finding above is worth turning into an actual number, since “roughly 16,383 quads” is a real, checkable threshold a game can hit without realizing it if it draws enough distinct sprites inside one `Begin()` / `End()` pair (a particle system, a large tile map batched in one call, a text-heavy UI drawn glyph by glyph):

```
1 constexpr int kMaxUint16Value = 65535;           // uint16_t's own
   maximum representable index
2 constexpr int kVerticesPerQuad = 4;              // one quad = 4
   unique vertices, 2 triangles
3 constexpr int kMaxQuadsBeforeWrap = (kMaxUint16Value + 1) /
   kVerticesPerQuad; // = 16384
```

The one-off discrepancy between this computed 16,384 and the chapter’s own “roughly 16,383” figure is itself worth internalizing rather than smoothing over: exactly where the wraparound first becomes *visibly* corrupt versus merely at its theoretical limit depends on `SpriteSortMode` and batch-flush timing details this specific finding does not fully pin down — treat 16,384 as the hard ceiling derived from the index type itself, and anything close to it as already in the danger zone worth testing directly rather than assuming safe. A game that might legitimately approach this — a bullet-hell shooter, a particle-heavy effect, a very large uniformly-tiled background drawn one `Draw()` call per tile instead of one call per texture atlas — has two real mitigations: split the single `SpriteBatch Begin()` / `End()` pair into multiple smaller batches well under the ceiling, or batch fewer, larger draws (one atlas texture covering many tiles) so the quad count per batch stays low regardless of how much is actually drawn on screen.

22.4 D3D9 oracle cross-comparison

A separate, one-off investigation ran EasyGL against the same 31-scene oracle corpus built for the D3D9 renderer’s own pixel-for-pixel verification against real XNA 4.0 (Chapter 28). The result: 10 of 31 scenes matched pixel-for-pixel; 21 diverged. This was deliberately *not* promoted to a permanent, ongoing test, because an EasyGL-versus-real-XNA comparison runs through an entirely different GPU driver stack than the DXVK-versus-DXVK comparison D3D9’s own

oracle performs — meaning any diff conflates genuine rendering differences with ordinary driver differences, and cannot cleanly attribute a mismatch to one or the other. Of the 21 divergent scenes, the investigation separated three distinct patterns: seventeen were silhouette-edge-only rasterization-convention differences (an inherent property of any two different GPU rasterizers, not a bug); two were whole-primitive but imperceptible, 1–2-out-of-255 rounding differences consistent with ordinary GPU/driver floating-point noise; and two were real, large, whole-primitive bugs worth naming specifically: a negative-FogEnd configuration rendered as solid black on EasyGL where D3D9 correctly renders a white-to-black gradient, and an environment-map Fresnel term failed to interpolate correctly (Gouraud-style) across a primitive on EasyGL — a concrete, independent confirmation of a divergence Chapter 15 already predicted from the effect-level audit alone.

22.4.1 Current shader-source corrections

Historical note. docs/d3d9-divergence-report.md recorded the fog-gradient and Fresnel findings as open. Both are closed in the pinned EasyGLRenderer.cpp; the document is now historical.

For FogStart=0 and FogEnd=-1, the current `clamp((z + FogEnd) / (FogEnd - FogStart), 0, 1)` becomes $1 - z$. The named oracle scene therefore runs from unfogged at $z = 0$ to fully fogged at $z = 1$, avoiding the former solid-black collapse. The environment-map vertex shader computes its Fresnel scalar from each vertex’s normal and eye vector before interpolation, matching the stock XNA shader. These corrections apply to the renderer’s stock-effect shader variants, not only to the scenes that exposed them.

22.5 Viewport and scissor

Current contract. `SetViewport()` immediately calls `device.set_viewport()` and `device.set_depth_range()`. The route works for the backbuffer, 2D targets, and cube faces, using the active target height for the Y-origin conversion. `SpriteBatch` preserves a custom sub-viewport and builds its orthographic projection from that viewport’s dimensions. Focused pixel tests cover one custom viewport and two viewport changes in one frame.

This scope is broader than Vulkan’s pinned route, which is limited to the backbuffer because its deferred recording does not retain the viewport associated with each target draw; see Chapter 23. EasyGL applies the state at the call site.

`ApplyRasterizerState()` controls scissor enablement, while `SetScissorRect()` sets the rectangle and flips Y against `currentRtHeight_` for an active target. `easygl_scissor_test` compares disabled, enabled, and disabled-again pixels. The contrary rows in `docs/rendertarget-support.md` and `docs/easygl_bugs.md` predate these implementations.

22.5.1 A claim this chapter itself got wrong: two-sided stencil’s shared mask is correct, not a bug

An earlier pass of this chapter listed “two-sided stencil incorrectly reusing the front-face read mask for the back face too” among this renderer’s confirmed gaps. Reading `ApplyDepthStencilState()` in `EasyGLRenderer.cpp` directly shows exactly that: the function’s Back-face call to `set_stencil_func_separate()` passes the same `stencilMask` local variable the Front-face call already used, with no separate back-face mask value anywhere in the function:


```

1 device.set_stencil_func_separate(::easygl::CullFace::Front,
2     ToEasyGLCompareFunc(stencilFunc),
3     referenceStencil, static_cast<unsigned int>(stencilMask));
4 // ...
5 // Back-face call below passes the same stencilMask -- not a
   separate ccw one.
6 device.set_stencil_func_separate(::easygl::CullFace::Back,
7     ToEasyGLCompareFunc(ccwStencilFunc),
8     referenceStencil, static_cast<unsigned int>(stencilMask));

```

Microsoft XNA 4.0 defines separate clockwise and counter-clockwise stencil *operations*, but only one `StencilMask` and one `StencilWriteMask`. CNA therefore passes each mask unchanged to both faces. The four counter-clockwise operation fields remain independent: `ccwStencilFunc`, `ccwStencilFail`, `ccwStencilDepthFail`, and `ccwStencilPass` reach a separate `set_stencil_op_separate()` call for the back face. A two-sided mirror technique may use different front/back operations, but the public XNA contract exposes no per-face mask to vary.

22.6 GetData() and CPU shadows

An earlier pass described `VertexBuffer` and `IndexBuffer` `GetData` as absent. The public method instead returns bytes from the resource-level `cpuShadow_`; it does not issue a GPU readback on any renderer. Each `SetData` call caches its packed GPU-layout bytes:

```

1 // Packed bytes from the most recent SetData call.
2 std::vector<std::uint8_t> cpuShadow_;

```

Microsoft XNA 4.0 exposes no GPU-write route for these buffers: no compute shader or transform feedback can alter them. A CPU shadow therefore preserves the observable `SetData` to `GetData` contract. `IndexBuffer` uses the same mechanism. Both types throw `NotSupportedException` when `GetData` is called on a `BufferUsage::WriteOnly` buffer, so that mode deliberately bypasses the shadow.

There is one shared typed-path exception this chapter's earlier wording missed: the options-taking `SetData` wrappers on both dynamic buffer classes upload correctly but never refresh their public `cpuShadow_` vectors. With `BufferUsage::None`, an inherited `GetData` after that normal dynamic upload sees an empty or stale shadow; this fails identically on every renderer and is not an EasyGL readback limitation. Chapter 14 traces the exact path.

22.6.1 The separate raw-layout gap: `SetDataRaw`

EasyGL's CNAEXT custom-layout route lets a caller supply an arbitrary `VertexDeclaration` instead of one of the seven typed `VertexPositionXxx` shapes. It reaches the renderer through `SetDataRaw()`, which does *not* participate in the shadow mechanism:

```

1 void VertexBuffer::SetDataRaw(const void* data, int count, int
   stride)
2 {
3     const std::size_t uploadStride =
4         stride > 0 ? static_cast<std::size_t>(stride) : 0;
5     if (!ValidateSetDataRange(
6         data, 0, count, uploadStride, uploadStride, true))
7         return;
8     UploadValidatedData(
9         data, count, uploadStride, SetDataOptions::None, false);

```

10 }

`GetData` itself is declared for those seven typed shapes, but there is no `GetDataRaw` counterpart to call after a custom-layout upload in the first place — a caller who populates a buffer through `SetDataRaw()` and then genuinely needs the data back has no API surface to ask with at all, typed or raw. This is a narrower, more precisely scoped gap than the earlier “not implemented at all” framing suggested: ordinary XNA-shaped vertex data uploaded through the ordinary non-options overloads round-trips correctly through `GetData` on this renderer exactly like every other one. The CNAEXT custom-layout extension has no raw readback surface at all, while the separate dynamic-options shadow bug above affects typed data project-wide.

22.7 Clear is immediate GL state, but Target is not colour-only

The seven renderer methods are real immediate `glClear` paths, and the depth/stencil variants correctly force full depth/stencil write masks before clearing. GL ignores the viewport for this operation but honors an enabled scissor, so a clear can cover less than the active target even though the custom viewport cannot narrow it.

The plain `Clear(r,g,b,a)` hook is the exception: it requests both `Color|Depth` and does not force the depth mask. The shared full overload dispatches *that same hook* for a target-only request; CNA’s raw-float overload calls it directly as well. Either spelling can therefore clear depth that the caller did not request, or leave it untouched solely because an earlier state disabled depth writes. Existing overload tests verify the resulting colour and that a depth-only clear preserves colour, but never prime/test depth around a target-only call. This is why the cross-renderer matrix in §19.4.2 does not promote those passing tests to complete option selectivity.

22.8 Test baseline

Do not carry the old “exactly three failures” count forward as a current baseline. One of its three names was the historical MRT fixture whose expectation has now been replaced by the one-through-four-target contract above, while later stabilization runs also widened the registered test set. Current persistent EasyGL-specific classes include the independent `GraphicsDevice.ReferenceStencil` override gap; the sibling `easy-gl-resource-smoke-tests` failure is upstream, and networking/audio failures observed in larger corpora are environment/control classes rather than renderer evidence. A baseline is meaningful only with its exact commit, configured registrations, display/driver, and test filter.

22.9 Xvfb screenshot evidence

EasyGL creates an SDL3 window and issues OpenGL ES calls. Unlike the `SOFTWARE` renderer’s display-free screenshots, this path needs a display server. The retained campaign used `Xvfb` with Mesa’s `llvmpipe`; earlier feasibility notes had identified that route but left it untested.

Building this renderer with `CNA_BUILD_EXAMPLES=ON` and running a small headless demo under `xvfb-run` exercises the route using the same scene and capture helper as Chapter 30’s equivalent section: the `modules/renderers/easygl/examples/easygl_spritebatch_rotation_golden_test.cpp` rotate-around-origin scene (also used in Chapter 13), captured via `SaveBackBufferScreenshotEXT` after `xvfb-run -a -server-args="-screen 0 1024x768x24" env -u WAYLAND_DISPLAY ./build-easygl/cna_xvfb_screenshot_demo_easygl`.

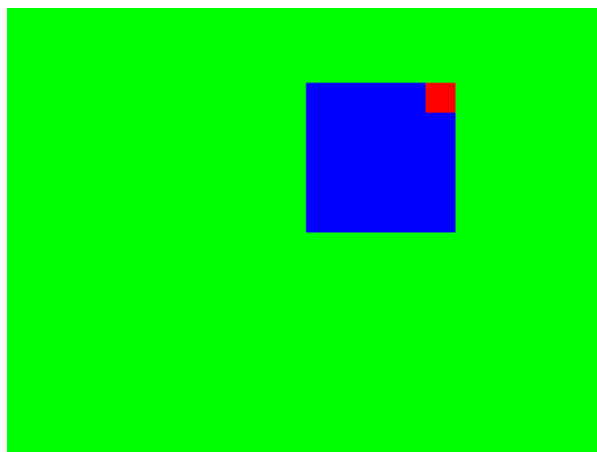


Figure 22.1: Output of the rotate-around-origin `SpriteBatch` scene from Figure 30.1, rendered by `EasyGL` through OpenGL ES under `Xvfb`. The retained `EasyGL` and `SDL Renderer` images are pixel-identical for this fixture; the comparison does not extend beyond the recorded scene.

The two retained 400×300 PNG files are byte-for-byte identical. This establishes cross-renderer agreement for the named fixture only; the D3D9 oracle corpus elsewhere in this chapter covers a broader set of scenes.

22.10 OPENGL4: modern core without the EasyGL family

OPENGL4 is an independent implementation built against system OpenGL and a hand-rolled function loader. It targets GLSL 410 core. Neither its source tree nor its shader path is shared with `EasyGL`, so `OPENGL33` and `OPENGL4` are not two settings on one renderer despite their adjacent version numbers.

The distinction is useful in both directions. `EasyGL` carries profile rewriting and Web/ES compatibility workarounds; `OPENGL4` can assume a modern desktop core surface. A bug in one shader generator or context path is therefore not evidence about the other.

The renderer owns real 2D targets, MRT, and occlusion-query paths. Its extended primitive dispatch is nevertheless hybrid: `BindProgramForStride()` selects a native program for supported vertex/effect combinations, while an unmatched combination reaches the colored draw tail. That tail is a bounded internal fallback, not the base-class inheritance defect, but it can still discard effect intent. Tests of custom layouts must therefore verify pixels or bindings, not merely that a draw call returned.

For selection and bug triage, treat `OPENGL33` as an `EasyGL` profile and `OPENGL4` as a separate renderer family. The public names encode neither shared code nor an automatic upgrade path.

22.11 PORTABLEGL: OpenGL-shaped software, not a GPU driver

PORTABLEGL occupies a different point on the portability axis. CNA fetches a pinned PortableGL revision and executes the pipeline on the CPU. Its shaders are C function pointers, not GLSL, and it creates no GPU resource of any kind.

That makes it valuable for deterministic experiments and systems without a graphics device, but its boundaries are explicit:

- its extended primitive paths are native to the CPU pipeline;
- `RenderTarget2D`, MRT, and occlusion queries are not supported;
- unsupported GPU features should be treated as deliberate refusals rather than as promises that happen to run slowly in software.

PortableGL is consequently not a substitute for EasyGL conformance evidence. Agreement between their pixels can be useful, but it compares two independent implementations with different rasterization and shader machinery.

22.12 A ten-identity decision map

The complete OpenGL-named surface now separates cleanly:

Need	Candidate	Principal caution
desktop fixed function	OPENGL1	optional FBO/query surface
GLSL 1.10 compatibility	OPENGL2	legacy language and context
ES 1.1 fixed function	OPENGLES1	specialized system dependency
ES 2/WebGL 1	OPENGLES2, WEBGL1	narrow profile ceilings
ES 3/WebGL 2	OPENGLES3, WEBGL2	the two names share the same internal profile path
desktop EasyGL	OPENGL33	EasyGL family, not OPENGL4
desktop core GL	OPENGL4	hybrid tail for unmatched draw combinations
CPU OpenGL-shaped execution	PORTABLEGL	no GPU targets, MRT, or queries

Across all ten identities, the selector name is only the beginning of the contract. Record the implementation family, shader language, context source, optional-function boundary, and verification environment alongside it. Those five facts explain more real portability failures than the nominal GL version alone.

Chapter 23

Native Modern GPU APIs

Five public identities belong in this cluster, but they do not share one implementation. They share an explicit-API design vocabulary: pipelines and resource transitions are visible, shader delivery is a first-class build concern, and a successful device creation says little about which CNA features the path actually proves.

Identity	Extended route	draw	RT2D / MRT	Query		Shader boundary
VULKAN	native		yes / yes	yes		CNA GLSL compiled to SPIR-V
SDL_GPU	native bounded tail	with colored	yes / yes	no query	real	API-specific pre-compiled bytecode
WEBGPU	native bounded tail	with colored	yes / one attachment only	no query	real	WGSL and native WebGPU binding model
METAL	native		yes / no	no		embedded MSL compiled at run-time
IGL	native		yes / yes	explicit no		IGL shader stages on OpenGL or Vulkan

The table states observed behavior as well as capability bits. `SDL_GPU` inherits the base null query while the default capability policy reports support. `WEBGPU` likewise reports MRT and query support, but rejects more than one target and has no real query object. Those are contract defects tracked by the audit, not reasons to promote a no-throw smoke test to feature evidence.

The Vulkan history illustrates a recurring defect shape: an interface accepts a complete state packet while the implementation consumes only part of it.

23.1 Historical Vulkan campaign baseline

The recorded Vulkan validation campaign reported 4,371 passes in a 4,373-case configured population, with two hardware-dependent skips; its renderer-specific `ctest` population passed 126 of 127. These are campaign populations, not a universal current test count; the counting model is in Chapter 69. The single remaining failure, `Vulkan_DepthBias`, is a narrow one: only the most extreme tested magnitude of depth bias produces no observable effect (§23.5), a symptom shared with an equivalent finding on `D3D9`. Historically this renderer also carried five

failing `BlendState` tests and three segfaulting skinned-model content tests. Both were closed before the recorded campaign baseline, without test exclusions.

23.2 `BlendState`: native mapping and historical defect

The single most serious historical bug on this renderer, in the project’s own words, is that its blend-state implementation was “almost entirely fake”: one hardcoded blend equation was applied regardless of what `BlendState` actually requested, confirmed failing across five independent pixel tests before the fix. The fix replaced this with a per-`Blend` / `BlendFunction` mapping, applied consistently across all nine of this renderer’s own pipeline-creation call sites — meaning the fix had to be threaded through every place a graphics pipeline object gets built, not just one central function.

23.3 Stencil testing: another discarded-parameter defect

`DepthStencilState` (Chapter 16) had an almost identical history. The renderer’s state-application entry point accepted fifteen parameters but stored only two of them (the depth-enable and depth-write flags) — every stencil-related parameter passed through the call was simply discarded, and the pipeline’s own stencil-test-enable flag was never set at all, on any pipeline. Five separate, independently-run checks (the enable flag itself, the stencil masks, front-face operations, two-sided stencil mode, and reference-stencil propagation) each separately confirmed this failure mode. The fix added real per-pipeline depth-compare operations, full front- and back-face `VkStencilOpState` configuration, and dynamic-state reference/-mask values — and, as a direct side effect of the same fix, connected `ReferenceStencil`’s independent-override path (Chapter 16) on Vulkan specifically. BGFX has since closed the same gap with its own dynamic rebuild and differential test; EasyGL remains the audited exception. A second, smaller bug found in the same investigation: depth format selection tried a stencil-*less* format before it tried any stencil-capable alternative.

23.4 OcclusionQuery: current contract and render-pass limit

Before its fix, occlusion queries on this renderer were disconnected from draw submission. The renderer defers 3D and 2D draws into a pending-batch snapshot, recorded into real Vulkan commands only once per frame, well after `Begin()` / `End()` had already returned synchronously; queries therefore always reported zero regardless of visibility. The fix resolved three design questions: how to tag an individual pending draw with the query that should observe it; whether a single query may legitimately span multiple draw calls (resolved: yes, as long as they share one render pass, tracked via contiguous-run detection, with an explicit, documented limitation that a query spanning a render-pass *boundary* is not correctly summed — a real capability gap against what a real device could do, stated plainly rather than silently assumed); and how per-frame query-pool resets need to be sequenced before any render pass begins. Verified in both directions — a visible quad producing a positive pixel count, an occluded quad behind a nearer opaque object producing none — plus a genuine multi-draw-span case (two non-overlapping half-quads inside one `Begin` / `End` span, summing correctly), discriminating for real in this project’s own software Vulkan implementation.

23.4.1 Using OcclusionQuery on Vulkan

The real `OcclusionQuery` surface is small — `Begin()`, `End()`, `getIsCompleteProperty()`, `getPixelCountProperty()` — and the standard visibility-culling pattern is exactly what the

verification above tested directly:

```

1 OcclusionQuery query(getGraphicsDeviceProperty());
2
3 query.Begin();
4 DrawBoundingProxyGeometry(occluderCandidate); // a cheap stand-in
   for the detailed mesh
5 query.End();
6
7 // One or more frames later, after the GPU result is available:
8 if (query.getIsCompleteProperty() && query.getPixelCountProperty() >
    0) {
9     DrawFullDetailMesh(occluderCandidate);
10 }

```

A query may cover several draws within one render pass. The pinned Vulkan route does not sum a query correctly across a render-pass boundary. Keep render-target changes outside an OcclusionQuery's Begin() / End() span.

23.5 Remaining limitation

One unresolved observation remains: the extreme-magnitude `RasterizerState.DepthBias` case produced no visible bias in the recorded environment. The same result occurred on D3D9 (Chapter 28), so the evidence does not attribute it to Vulkan rather than the shared driver environment.

23.6 Two corrected render-target black-frame defects

Historical note. `docs/rendertarget-support.md` still lists two black-frame defects as open. The pinned source and regressions close both; the document is historical.

The first defect was not cube-specific. `VulkanRenderer::Clear()` once changed only global clear values, while command recording discovered passes solely from draw calls. A clear-only target therefore never entered command recording:

```

1 device.SetRenderTarget(&probe, CubeMapFace::PositiveZ);
2 device.Clear(Color::CornflowerBlue);
3 device.SetRenderTarget(nullptr); // no intervening draw: the former
   failure trigger

```

The first repair added a deduplicated target list so a clear could create a pass without a draw. The current architecture has superseded that intermediate mechanism: each public `Clear()` is now a `PendingClear` carrying its target, bind-cycle segment, monotonic command order, selected aspects, values, and an owning target reference. Command recording discovers clear-only segments from that stream itself. The dedicated clear-only `RenderTarget2D` regression still fills two targets without a draw and samples both, while the newer ordered-clear suite proves this is no longer merely a pass-existence fix.

The second regression renders `SpriteBatch` content into all six faces of a `RenderTargetCube`, unbound, then sampled through `EnvironmentMapEffect`. The original regression now produces the expected blue center across five repeated runs and its registered invocation. The project did not bisect the fixing commit. A later cube-view/layout rewrite is a plausible cause, but the relevant transition is mip-count-gated while this fixture uses one level. The defect is verified closed; its fixing commit is unknown.

23.7 Clear is aspect-selective and issue-ordered

Focused attachment tests establish that colour, depth, and stencil values reach Vulkan. **REMED-GFX-129** closes the ordering contract: every public clear is now an ordered command at its exact position among **SpriteBatch** and 3D draws, with independent target/depth/stencil flags. A leading clear on a discard cycle may fold into the render-pass load action; a clear after any draw is emitted as **vkCmdClearAttachments** after replaying the preceding command slice. Preserve targets use the latter path even for a leading explicit clear, so their load operation does not swallow it. **ClearDepth()** also records a clear-only segment rather than merely updating a global fallback value.

Two renderer-neutral suites make those details observable. The eight-mask **Vulkan_GraphicsDevice_ClearOptions** fixture establishes old colour/depth/stencil contents and encodes each surviving aspect into distinct colour probes. The ordered-clear fixture queues draw-clear, clear-draw, multiple-clear, backbuffer, 2D-target, cube-face, and MRT sequences without an intervening readback or present. A renderer that collapsed them into one pass-opening value would therefore fail even if its eventual clear colour were correct.

Render-target usage now follows one shared predicate: only **DiscardContents** maps to discard; **PreserveContents** and **PlatformContents** both map to preservation. Vulkan receives that Boolean for 2D and cube targets. Its single-sample and MSAA render-pass caches provide matching clear/load variants, and preserving depth/stencil attachments receive the layouts and store operations needed to survive a full unbind/rebind cycle. Cube colour is per face while its depth/stencil image is deliberately shared by all six faces, matching FNA. The cube-usage, per-face-MSAA, and depth/stencil-usage suites distinguish those paths, including Platform. Discard still produces the deterministic shared black/max-depth/zero-stencil bind clear, and an explicit ordered clear supersedes either usage policy. See §19.4.1.

23.8 A confirmed non-gap: device-feature gating

Not every investigation into this renderer found a bug. A dedicated check of whether optional device features (**fillModeNonSolid**, needed for wireframe rendering, and **samplerAnisotropy**) are properly negotiated before being requested found the gating already correctly implemented — both are queried via **vkGetPhysicalDeviceFeatures** before being enabled, with a graceful fallback if the underlying device lacks either one. This is included here deliberately: a book that only ever reports bugs found would give a skewed picture of a renderer whose overall health is, in fact, good.

23.9 Notable feature-matrix specifics

A cube-map render target's MSAA route uses a multisampled 2D texture array and a separately cube-flagged resolve texture. Multiple render targets are capped at four by shared code mirroring FNA's **MAX_RENDERTARGET_BINDINGS**; the cap does not come from Vulkan's device limit. **Texture2D** uploads above mip level zero allocate and update the requested subresource, and a minification test samples its authored color. Chapter 19, §19.3.3, gives the cross-renderer proof boundary. Non-Color formats remain blocked by the shared **Texture::ValidateFormat** contract described in Chapter 22, not by a Vulkan-specific refusal.

23.10 Render-target mipmaps, MSAA, and viewport

Historical note. docs/render-target-support.md lists render-target mip generation, MSAA, and custom viewport effects as open for Vulkan. All three are implemented at the pin; those status rows predate the current source and tests.

`VulkanRenderTargetRenderer::MaybeGenerateMips()` runs a per-level `vkCmdBlitImage` cascade against resolved level-0 content after the render pass. Render-target MSAA covers both `RenderTarget2D` and `RenderTargetCube`. Each `SpriteBatch` batch and 3D draw captures the active viewport and applies it while replaying its render-target segment. Custom sub-regions therefore work on the backbuffer and `RenderTarget2D` without collapsing to the final value at `Present()` time. Dedicated target and deferred-capture pixel tests cover both scopes and multiple viewport changes inside one unflushed bind cycle.

Scissor follows the same per-command rule. Each queued draw retains the rectangle and `ScissorTestEnable`; replay clamps an enabled rectangle to the current target and expands disabled or degenerate state to its full extent before `vkCmdSetScissor`. The target and deferred-scissor oracles establish that later state cannot retroactively unclip earlier queued draws. Pass-opening full-target state is a default; each draw replaces it with its snapshot.

23.11 How a swapchain gets built: format, color space, and present mode

`VulkanRenderer::CreateSwapchain()` is where two real, easy-to-get-wrong decisions get made once, at swapchain-creation time, rather than left to chance: which pixel format the swapchain presents through, and which `VkPresentModeKHR` actually governs frame pacing. Both are worth reading directly rather than assumed, since Vulkan’s own API surface makes it easy to pick a format that looks equivalent but silently changes what a game sees on screen.

23.11.1 UNORM, not SRGB: a deliberate color-space decision

A physical device typically reports several supported (`VkFormat`, `VkColorSpaceKHR`) swapchain pairs, and it is common in Vulkan tutorials to prefer an SRGB-variant format (e.g. `VK_FORMAT_B8G8R8A8_SRGB`) for its automatic linear-to-sRGB gamma encode on every presented pixel. This renderer deliberately does the opposite, and says why directly in its own source comment: real XNA’s default backbuffer format, `SurfaceFormat::Color` (Appendix A), is linear — a separate, CNAEXT `ColorSrgbEXT` value exists specifically for the gamma-encoded variant (Appendix E) — so an SRGB swapchain format would apply an automatic gamma encode XNA itself never performed, silently darkening every frame relative to what a real XNA game produced. The fix is a preference search rather than a hardcoded pick, since not every device lists the exact pair in the same order:

```
1  VkSurfaceFormatKHR fmt = fmts[0]; // fallback: whatever the device
   lists first
2  for (auto& f : fmts)
3      if (f.format == VK_FORMAT_B8G8R8A8_UNORM &&
4          f.colorSpace == VK_COLOR_SPACE_SRGB_NONLINEAR_KHR)
5          { fmt = f; break; }
```

The two fields answer two different questions, which is exactly the subtlety worth stating plainly: `colorSpace` here is `VK_COLOR_SPACE_SRGB_NONLINEAR_KHR` regardless — that value describes the *display*’s own expected color space (the near-universal default virtually every consumer monitor and every Vulkan implementation supports), not whether the swapchain itself gamma-encodes on write. `format` is the field that controls the encode: `_UNORM` writes the

raw bytes a game submits with no transformation, while a sibling `_SRGB` format of the identical byte layout would gamma-encode automatically. Picking `UNORM` with the standard nonlinear color space is therefore not a contradiction — it is precisely “present to a normal monitor, but do not add an encode step XNA’s own `Color` format never had.” A porting engineer who free-associates “SRGB” with “correct color space, always prefer it” and swaps this one line risks introducing a real, visible darkening regression across every scene in the game, not just an edge case.

That deliberate `UNORM` choice is fixed renderer policy, not a response to the caller’s current `BackBufferFormat`. The Vulkan factory discards both presentation format ordinals: it prefers this same `B8G8R8A8 UNORM` pair even when public `PresentationParameters` reports something else, and always creates a device-selected depth image (`D24S8`, then `D32S8`, then `D32`) even for `DepthFormat::None`. Render-target depth formats use a newer, genuinely per-target selection path and must not be confused with this default-backbuffer behavior. SDL can still change the window to fullscreen; a later out-of-date/resize rebuild follows the surface size without consuming `isFullScreen` itself.

23.11.2 SwapInterval to VkPresentModeKHR: the same three-way switch Chapter 6 introduced

`PresentationMode`’s `PresentInterval` (`Immediate` / `Default`-a.k.a.-`One` / `Two`, Chapter 6) is not merely stored on this renderer — it selects among the device’s actually-supported present modes with an explicit fallback chain for each case, verified directly against `CreateSwapchain()`:

```

1  VkPresentModeKHR mode = VK_PRESENT_MODE_FIFO_KHR; // safe default:
    guaranteed available
2  if (swapInterval_ == 0) { // Immediate: tearing allowed,
    lowest latency
3      // prefer true IMMEDIATE; MAILBOX (triple-buffered, no tearing)
    is the fallback
4      // if IMMEDIATE isn't supported; FIFO if neither is
5  } else if (swapInterval_ == 2) { // Two: allow a slower-than-vsync
    present without stalling
6      // prefer FIFO_RELAXED; falls through to FIFO if unsupported
7  }
8  // swapInterval_ == 1 (Default/One): FIFO unconditionally -- always
    available, real VSync

```

`FIFO` is the only present mode the Vulkan specification *guarantees* every conformant implementation supports — choosing it as the unconditional starting value, and as every fallback’s own final fallback, is what makes swapchain creation itself never fail for lack of an exotic present mode. `PresentInterval::Two`’s mapping to `FIFO_RELAXED` is a deliberately different request from `Immediate`’s: `FIFO Relaxed` still waits for vsync when the application is keeping up, and only presents immediately (accepting a single tear) on the specific frame where the application would otherwise have missed vsync and stalled — the Vulkan-native shape of “prefer vsync, but don’t let a single slow frame cascade into a visible stutter,” not a request for tearing as a matter of course the way `Immediate` is. A game that sets `PresentInterval::Two` expecting literally half the refresh rate (the historical, D3D9-era “present interval two” reading some porting engineers carry over) will not get that on this renderer — it gets `FIFO Relaxed`’s own, different throughput/latency tradeoff instead, worth checking explicitly against a real target frame-rate requirement before assuming interval semantics carried over unchanged from a `Direct3D` background.

That switch runs only in `CreateSwapchain()`, from the interval captured by the renderer constructor. `VulkanGraphicsRenderer` does not override `IGraphicsRenderer::SetSwapInterval()`, so a later `GraphicsDevice::Reset()` stores the new public value but cannot update `swapInterval_` or recreate the swapchain for it. This is especially visible through `Game`: its device is eagerly built with default `PresentInterval::Default` before the derived constructor can set `GraphicsDeviceManager.SynchronizeWithVerticalRetrace=false`. The subsequent `ApplyChanges()` therefore still leaves Vulkan in FIFO mode. Only constructing a `GraphicsDevice` directly with the desired parameters (or using CNA's separate test-only full-renderer recreation helper) can currently change that initial choice. No dedicated test queries or times the real Vulkan present mode.

23.12 Two frames in flight, and the deferred-present trick that makes readback pixel-safe

Everything in §23.11 covers how the swapchain itself gets built; this section covers what actually happens every frame once it exists. This renderer double-buffers its own CPU-side work against the GPU — `MaxFramesInFlight` is a real, fixed `constexpr int` value of 2, read directly from `VulkanRenderer.hpp` — with one `VkFence`, one pair of `VkSemaphores` (image-available, render-finished), and one command buffer per in-flight frame, indexed by a wrapping `currentFrame_` counter.

23.12.1 The ordinary present path

`Present()` calls the renderer's real `SubmitFrame(bool deferSwap)` with `deferSwap=false`. Reading it directly shows five sequential steps, each with a real purpose rather than boilerplate:

```

1  vkWaitForFences(device_, 1, &inFlightFences_[currentFrame_], VK_TRUE
    , UINT64_MAX);
2  // ... vkAcquireNextImageKHR, vkResetFences, RecordCommandBuffer ...
3  vkQueueSubmit(graphicsQueue_, 1, &si, inFlightFences_[currentFrame_
    ]);
4  // deferSwap == false: present immediately
5  vkQueuePresentKHR(presentQueue_, &pi);
6  currentFrame_ = (currentFrame_ + 1) % MaxFramesInFlight;
```

The opening `vkWaitForFences` is the throttle that makes double-buffering safe at all: before this frame's slot can be reused, the GPU must have finished the *previous* frame that used the same slot two frames ago, or the CPU would start overwriting a command buffer the GPU might still be reading. `vkAcquireNextImageKHR` signals `imageAvailableSemaphores_[currentFrame_]` when the swapchain image is actually ready to be written to (not necessarily immediately — the presentation engine may still be displaying it); `vkQueueSubmit` makes the GPU work wait on that same semaphore before touching the image, and signals `renderFinishedSemaphores_[currentFrame_]` when done; and `vkQueuePresentKHR` waits on that render-finished semaphore before handing the image to the presentation engine. Three separate synchronization primitives, three separate questions each one alone answers: is the CPU allowed to reuse this frame slot yet (fence), is the GPU allowed to start rendering into this image yet (image-available semaphore), and is the presentation engine allowed to display it yet (render-finished semaphore).

23.12.2 The deferred path: why `GetBackBufferData` cannot simply call `Present`

`GraphicsDevice::GetBackBufferData` (Chapter 12) needs a real answer to a question the ordinary present path never has to ask: are the exact pixels the GPU just rendered still readable, or has the presentation engine already taken ownership of the image and possibly begun compositing it to the screen? Reading `ReadBackbuffer()`'s own code comment states the race directly: *“SubmitFrame(true) renders and waits for the GPU but HOLDS the present, so we read the staging buffer below BEFORE the image is handed to the presentation engine — no race possible.”* The mechanism is the same `SubmitFrame()` used for ordinary presentation, called with `deferSwap=true` instead:

```

1  if (deferSwap) {
2      // Wait for render + readback copy to complete, but hold the
        image. The caller
3      // (ReadBackbuffer) reads the staging buffer before the image is
        presented, so
4      // presentation-engine timing can never corrupt the captured
        pixels.
5      vkWaitForFences(device_, 1, &inFlightFences_[currentFrame_],
        VK_TRUE, UINT64_MAX);
6      deferredPresentImageIndex_ = imageIndex;
7      hasDeferredPresent_        = true;
8      return true;
9  }
```

`RecordCommandBuffer` has already copied the whole swapchain image into a real staging buffer (`readbackStagingBuf_`) as part of the same command buffer the ordinary render work goes into — so this second `vkWaitForFences` call (on top of the one at the very start of `SubmitFrame`) blocks until that copy is guaranteed complete on the GPU side, before `ReadBackbuffer` maps `readbackStagingMem_` and reads it directly. *Only after* the CPU has finished reading does `ReadBackbuffer` call `FinishDeferredPresent()`, which performs the `vkQueuePresentKHR` call that `SubmitFrame(true)` deliberately skipped — late, but still exactly once, so no frame is silently dropped from the screen because it was read back.

23.12.3 Two additional readback findings

Reading `ReadBackbuffer()`'s own surrounding logic in full surfaces two further details worth knowing, neither one hypothetical:

A real read cache, not a re-render-every-call assumption. A naive readback implementation might call `SubmitFrame(true)` unconditionally on every `GetBackBufferData` call — but the real code guards it behind `hasNewWork || !readbackStagingValid_`, where `hasNewWork` checks whether any pending 3D or batched 2D draws actually exist since the last present. The code's own comment states why this matters concretely: re-presenting an already-rendered, empty-queue frame would re-render whatever `Clear` color is currently set and silently destroy the content every read after the first one would have returned, for any caller reading the same frame's backbuffer more than once (a golden-image test that reads several sub-regions of one frame, for instance).

`vkAcquireNextImageKHR` can genuinely fail on the very first frame. A real, environment-specific comment in the same function notes that swapchain-out-of-date is *“common on first frame under Wayland/RADV”* — handled by zeroing the output buffer and returning, rather than throwing, so a caller can detect a blank frame and retry instead of crashing outright on a platform combination this specific to trigger.

A genuinely small but real detail rounds out the picture: swapchain images are frequently `VK_FORMAT_B8G8R8A8_UNORM` (per the color-space decision in §23.11), which stores channels in the opposite byte order from the RGBA layout every other renderer and `Color` itself use — `ReadBackbuffer` checks `swapchainFormat_` directly and performs a real per-pixel B / R channel swap only when the format is actually BGRA, rather than assuming one fixed byte order across every device this renderer might run on.

23.13 The 512-live-descriptor-set ceiling

Every `Texture2D` and every `RenderTarget2D` constructed on this renderer allocates one `VkDescriptorSet` from a single shared pool, sized at construction time and never grown:

```
1 static constexpr uint32_t MaxDescriptorSets = 512;
2 // ...
3 VkDescriptorPoolSize ps{ VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER,
4   MaxDescriptorSets };
5 VkDescriptorPoolCreateInfo ci{};
6 ci.flags = VK_DESCRIPTOR_POOL_CREATE_FREE_DESCRIPTOR_SET_BIT;
7 ci.maxSets = MaxDescriptorSets;
8 ci.poolSizeCount = 1;
9 ci.pPoolSizes = &ps;
10 vkCreateDescriptorPool(device_, &ci, nullptr, &descriptorPool_);
```

`VulkanTextureRenderer`'s own constructor and `VulkanRenderTargetRenderer`'s own constructor both allocate from this identical `descriptorPool_` — confirmed by reading both call sites directly, not just the pool's own creation code — so the real budget a game actually has is 512 *simultaneously live* `Texture2D` plus `RenderTarget2D` instances combined on this renderer, not 512 of each independently. The `VK_DESCRIPTOR_POOL_CREATE_FREE_DESCRIPTOR_SET_BIT` flag means this is a real budget of *concurrently undisposed* resources, not a lifetime total: each renderer's own `ReleaseVulkanResources()` calls `vkFreeDescriptorSets` on `Dispose()`, returning its slot to the pool for reuse. A game that creates and disposes textures across many frames — streaming a level's assets in and out, for instance — never accumulates toward the ceiling as long as disposal genuinely keeps pace with creation.

The failure mode itself is a clean, loud exception rather than a silent cap or a corrupted allocation, confirmed by reading the allocation call site directly:

```
1 if (vkAllocateDescriptorSets(dev, &dsInfo, &descriptorSet_) !=
2   VK_SUCCESS)
3   throw std::runtime_error("vkAllocateDescriptorSets (texture)
4     failed");
```

A 513th simultaneously-live `Texture2D` or `RenderTarget2D` throws at construction time, on this renderer specifically — not a per-frame budget, and not shared with any other renderer's own resource limits (EasyGL and BGFX have no equivalent fixed-pool ceiling at all; each stock effect type on this same Vulkan renderer additionally has its own, separately-sized dedicated pool — `descriptorPoolEnvMap_`, `descriptorPoolSkinned_`, `descriptorPoolPbr_`, and others — so this specific 512-set ceiling applies only to plain `Texture2D` / `RenderTarget2D` allocation, not to how many objects can simultaneously use a given stock effect). A game targeting Vulkan specifically with a very large, non-atlased texture library, or one that creates many short-lived render targets without disposing the previous batch first, is the realistic way to reach this ceiling in practice — worth knowing as a real, renderer-specific constraint before assuming texture/render-target counts that work fine on EasyGL port unchanged to Vulkan.

23.14 Coverage gap

One honest gap in *testing*, not rendering: at the pinned revision there is no dedicated pixel test for `SpriteBatch`'s sort-mode/rotation/scale/crop/flip behavior, for `SpriteFont` glyph placement, or for a multi-mesh `Model` hierarchy on this renderer specifically — confirmed by directly searching the test suite for matching test names and finding none, despite basic `SpriteBatch`/2D-demo smoke tests passing. This does not mean the underlying shared C++ code is wrong (it is the same code every other renderer runs), only that this specific renderer has not had these specific features independently re-verified against its own rendering path.

23.15 METAL: a narrow implementation with explicit refusals

METAL is macOS-only, enables Objective-C++, and links Apple's system frameworks. Its MSL lives as an embedded source string and is passed to `newLibraryWithSource:` at runtime. That gives it a simpler packaging story than the SPIR-V pipelines, but also makes compiler and device creation part of runtime startup.

The implementation is intentionally transparent about its present boundary. It has native extended primitive submission and `RenderTarget2D`, but deliberately rejects MRT, MSAA, custom effects, occlusion queries, instancing, multi-stream input, and backbuffer readback. These refusals cite missing macOS pixel evidence rather than pretending that interface presence proves rendering correctness.

Its automated macOS path proves construction, clear/present, and capability reporting; it does not yet establish representative 3D draw pixels. That is a sharper statement than calling the renderer either complete or untested. The device path executes, while much of the feature surface remains deliberately outside the claimed evidence boundary.

23.15.1 Why explicit refusal is useful

The renderer contract's default-true capability polarity makes accidental overclaiming easy. Metal's policy demonstrates the safer integration pattern: enumerate the unsupported surface, throw at the boundary, and widen it only when a macOS-specific observable proves the new path. Callers get an early diagnostic instead of a later blank frame.

23.16 IGL: one CNA identity over two IGL backends

IGL integrates Meta's IGL 1.1.1 and can create an OpenGL or Vulkan device below the one CNA identity. Its renderer owns effect-aware 3D draws, sprite submission, real 2D/cube targets, two-to-four-target MRT, and volume-texture upload/sampling. It has no IGL occlusion-query API; CNA returns an immediately complete zero-sample object and reports the capability false, avoiding a spin or false GPU promise.

The backend distinction remains evidence-relevant. OpenGL and Vulkan have separate context, coordinate and resource behavior even though `GetGraphicsRendererType()` says IGL. The release examples bring up both backends, but their existence is not a claim that this edition reran every IGL check. Volume upload and sampling are supported; volume readback is an explicit IGL 1.1.1 gap and raises rather than fabricating voxels.

23.17 Comparing the five evidence tiers

VULKAN has the broadest renderer-specific pixel history in this group. `SDL_GPU` is exercised through its currently selected native driver, which is not proof of every driver SDL can choose. `WEBGPU` has real application and pixel evidence but retains the MRT/query capability overclaims above. `METAL` has a real macOS lifecycle gate while its 3D pixel surface remains explicitly unproven. `IGL` has backend-specific example evidence and explicit capability boundaries that must be named as OpenGL or Vulkan.

For portability work, record both axes: which CNA identity ran and which native driver or device path served it. An abstraction selecting Vulkan on one machine and Metal on another does not turn one successful run into proof of both.

23.18 Legacy WebGPU Chapter Material

`WEBGPU` is CNA's fifth graphics renderer, and its youngest: the project owner explicitly lifted a former prohibition on this renderer and authorized its implementation. It targets native `wgpu-native` (pinned to version 29.0.1.1), selected the same way as every other renderer via `-DCNA_GRAPHICS_RENDERERER=WEBGPU`. Its own tracking plan spans fourteen phases and over 130 individually numbered tasks — large enough that this chapter covers its trajectory rather than every task.

23.19 What “experimental” concretely means here

Unlike the four renderers covered in the previous three chapters, `WEBGPU` is deliberately excluded from the cross-renderer feature matrix used throughout Chapters 30–23, on the matrix's own stated grounds that its feature surface was not yet broad enough for a meaningful side-by-side comparison at the time that matrix was written. This chapter instead draws on the renderer's own dedicated documentation and milestone history.

23.20 The native 2D baseline

Early milestones established device/surface setup, clear/present, a working `Texture2D` path, vertex/index buffer uploads, and a WGSL-based `SpriteBatch` implementation. A 120-frame clear-and-present lifecycle test, and a manually-reviewed `SpriteBatch` validation scene (covering cropping, tint/alpha, rotation, flips, filtering, and all three address modes), both passed before the native 2D baseline was declared closed. The most instructive milestone here, though, is an independent one: a real, independently-developed XNA-style game (a *Windows Phone Speedy Blupi* port, from the `mobile-eggbert` sibling repository) was run against this renderer on a real desktop, reached its main menu, rendered its `SpriteBatch` content pixel-correctly, and played through a real animated cutscene — with progress bar and cross-fade — triggered by a simulated button click, with no WebGPU validation errors of any kind. This is a genuinely different, and stronger, kind of proof than any of this renderer's own purpose-built test scenes: an unmodified, independently-authored game simply working.

23.21 A bug a manual screenshot review missed, and a pixel assertion caught

One specific finding is worth naming as a methodological lesson in its own right: the `SpriteBatch` pipeline's blend factors did not actually match the non-premultiplied output the shader itself

produced, so a translucent sprite — one with an alpha value strictly between fully transparent and fully opaque — rendered fully opaque instead. This was completely invisible to the earlier manual screenshot review covering rotation, flips, and filtering; it was only caught once a real pixel-value assertion replaced “does this look right in a screenshot.” The fix matched Vulkan’s own blend-factor pairing. This is the same lesson Chapter 13 drew from `SDL_Renderer`’s ignored-transform-matrix bug: a rendering pipeline can look entirely correct in every way a human screenshot review checks, and still be measurably wrong.

23.22 3D depth testing and the gap it exposed

The first 3D draw path (`DrawColoredPrimitives` / `DrawIndexedColoredPrimitives`) was verified with a genuine near/far depth-ordering test rather than trusting draw order alone. Building it surfaced a real, previously-invisible gap: applying a `DepthStencilState` had no effect on this renderer at all before this point — the apply-state entry point was entirely unimplemented, meaning `GraphicsDevice.DepthStencilState` had silently done nothing on WebGPU up to that point. From there, `PbrEffect` (a real glTF 2.0 metallic-roughness BRDF, ported line for line from EasyGL’s own GLSL implementation) and skinned variants of both `SkinnedEffect` and `PbrEffect` (bone-palette skinning up to 72 bones, again ported directly from EasyGL) were added and both pass their dedicated test suites in full.

23.22.1 PbrEffect: a CNAEXT metallic-roughness material

`PbrEffect` is worth a concrete look before moving on, since — unlike every stock effect in Chapter 15 — it has no real-XNA counterpart at all: real XNA 4.0 predates the glTF 2.0 metallic-roughness PBR model by years, so this is a CNAEXT addition ported line for line from EasyGL’s own GLSL implementation, not a port of anything Microsoft ever shipped. Its property surface mixes ordinary `IEffectMatrices/IEffectLights / IEffectFog` plumbing (identical in shape to every stock effect) with a genuinely new four-texture-map PBR material:

```

1  PbrEffect effect(getGraphicsDeviceProperty());
2  effect.setWorldProperty(worldMatrix);
3  effect.setViewProperty(camera.View());
4  effect.setProjectionProperty(camera.Projection());
5
6  effect.setTextureProperty(&baseColorMap);           // glTF "
   baseColorTexture"
7  effect.setNormalMapProperty(&normalMap);
8  effect.setMetallicRoughnessMapProperty(&metalRoughMap); // one
   texture: G=rough, B=metal
9  effect.setOcclusionMapProperty(&aomap);
10 effect.setEmissiveMapProperty(&emissiveMap);
11
12 // Both factors multiply the map's channel value (glTF's own "value
   * factor" convention),
13 // or stand alone with no map bound:
14 effect.setMetallicFactorProperty(1.0f);
15 effect.setRoughnessFactorProperty(1.0f);
16 effect.setEmissiveFactorProperty(Vector3(0.0f, 0.0f, 0.0f));
17
18 effect.setLightingEnabledProperty(true);
19 effect.EnableDefaultLighting();

```

The `MetallicRoughnessMap` single-texture, two-channel packing above is not an implementation shortcut CNA introduced — it is glTF 2.0’s own real specified layout for this material

model (green channel roughness, blue channel metallic, red and alpha unused), and `PbrEffect` follows it exactly so a metallic-roughness texture exported by any standards-conformant glTF pipeline can be bound directly with no channel-repacking step. This is also, concretely, the effect Chapter 38’s two independent skinning systems both connect to on this renderer: the WebGPU port adds a skinned variant of `PbrEffect` alongside skinned `SkinnedEffect`, so a glTF character rig authored with a metallic-roughness material and driven by `AnimationPlayer` reaches GPU skinning through this effect rather than through `SkinnedEffect` at all.

23.23 Render targets, environment mapping, instancing, and mip generation

`RenderTarget2D` support combines depth and stencil into a single format regardless of what was requested (unlike Vulkan’s newer per-render-target depth-format selection, Chapter 23), and needed a real type-safety fix when sampling a render target back as an ordinary texture. MSAA support, once implemented, initially appeared to fail its own dedicated test — but investigation found the infrastructure was already correct, and the reported failure was a defect in the *test itself* (a missing `RasterizerState::CullNone`), not in the MSAA implementation. `EnvironmentMapEffect` and genuine hardware instancing were added together, with instancing implemented as a real per-instance vertex stream needing no bind-group changes, unlike the effect families that do require one per draw. `RenderTargetCube` support surfaced a cross-cutting sprite-geometry bug that has since been fixed: the original queue path computed clip-space positions from the backbuffer’s logical dimensions even while a differently-sized off-screen target was bound. `REMED-GFX-019` now derives dimensions from the current `RenderTarget2D` or cube face instead. Mip generation for `Texture2D` and `TextureCube` is implemented as a genuine filtered downsample (a series of full-screen-triangle render passes, since `wgpu-native` v29 has no direct blit-image-equivalent call). Notably, this renderer auto-regenerates the chain after every level-zero write to a plain `Texture2D` or `TextureCube`. Real XNA/FNA and the other audited plain-texture paths keep upper levels explicitly authored; `SDL_GPU` is the narrow exception for plain cubes, where a complete level-zero face upload also invokes its native generator, while its plain 2D and 3D textures remain authored-level resources. WebGPU’s behavior prevents undefined upper levels after a level-zero upload, but it is a deliberate timing divergence rather than an unqualified superiority: a later level-zero write can overwrite upper levels that a game authored earlier. Render-target mip generation remains the separate unbind/pass-finalization contract.

The default surface is equally independent of public presentation-format fields, but its choice is unusually visible. `ConfigureSurface()` prefers BGRA8-sRGB, then RGBA8-sRGB, before either UNORM variant. Thus a public `Color` request can accompany a native sRGB swapchain, the opposite default-color policy from Vulkan. Its fixed depth texture uses `Depth24PlusStencil8`. The renderer recreates it for every non-minimized surface, including when the public depth value is `None`. SDL performs any fullscreen window transition; the next frame detects its physical size and reconfigures the surface, but neither requested format enum participates.

23.23.1 The corrected render-target-size trap

The former sprite-geometry bug is worth walking through because it produced no error of any kind — only a subtly wrong result:

```
1 // e.g. a 256x256 render target, while the game window itself is
   1920x1080:
```

```

2 device.SetRenderTarget(&smallOffscreenTarget);
3 spriteBatch.Begin();
4 spriteBatch.Draw(sourceTexture, Vector2::Zero, Color::White); // "
    fill the target"
5 spriteBatch.End();

```

Before REMED-GFX-019, the same `Draw()` call intended for the 256×256 target was converted as though the target were the 1920×1080 window, silently changing its scale and placement. The current `QueueSprite()` instead selects the bound 2D target's width/height, the bound cube face's size, or the backbuffer logical viewport as three explicit cases before converting pixels to NDC.

The dedicated `WebGPU_SpriteBatch_RenderTarget` regression makes the distinction discriminating rather than visual-only: a 96×72 backbuffer is paired with asymmetric 48×32 and 64×40 targets; the same destination rectangle must occupy the same target-relative pixels in both. It also covers target-backbuffer-target isolation, a non-identity `SpriteBatch` transform, texture orientation, and a 48×48 cube face. Probe pixels inside the old half-scale rectangle but outside the correct one must remain black. The old practical prohibition on differently-sized targets is therefore no longer needed.

23.23.2 RenderTargetCube and its former silent cast failure

`RenderTargetCube` support (Task WEBGPU-114) landed after the plain `RenderTarget2D` path above, and reused most of its design directly: one shared 6-array-layer `WGPUTexture` (the same layout `WebGPUTextureCubeRenderer` uses for an ordinary sampled, readback-capable `TextureCube`), one shared `size×size` depth/stencil texture reused across all six faces (only one face is ever bound at a time, the same choice Vulkan's own `RenderTargetCube` renderer makes), and a `WGPUTextureViewDimension_2D` view per face for the render-pass color attachment alongside one `WGPUTextureViewDimension_Cube` view spanning all six layers for sampling the whole thing back later.

That whole-cube sampling view is where a real, previously-invisible bug lived. Before this task, `EnvironmentMapEffect`'s `envMap` field on this renderer was hardcoded to the concrete type `const WebGPUTextureCubeRenderer*` — so binding an ordinary `TextureCube` as `EnvironmentMapEffect.EnvironmentMap` worked, but binding a `RenderTargetCube` (a different concrete class implementing the same `ITextureCubeRenderer` interface) did not: the hardcoded-type cast failed silently, and the effect fell back to its own 1×1 white cube default instead of throwing or producing an obviously wrong image. The fix is a small shared interface, read directly from the real header:

```

1 // WEBGPU-114: the cube-map sibling of IWebGPUSamplable -- lets both
2 // WebGPUTextureCubeRenderer and WebGPURenderTargetCubeRenderer
3 // (render-into) resolve safely to their own whole-cube view
    wherever a
4 // GpuDrawParams::envMap needs to be sampled, via a dynamic_cast
    that
5 // degrades to nullptr for a null or incompatible-type input.
6 class IWebGPUCubeSamplable
7 {
8 public:
9     virtual ~IWebGPUCubeSamplable() = default;
10    [[nodiscard]] virtual WGPUTextureView CubeView() const = 0;
11 };

```

Both concrete classes now implement `IWebGPUCubeSamplable` alongside their own primary interface — `WebGPUTextureCubeRenderer` and `WebGPURenderTargetCubeRenderer` alike — and the draw-dispatch code resolves through it with a single `dynamic_cast` rather than the old hardcoded concrete-type cast:

```

1  [[nodiscard]] const IWebGPUCubeSamplable* ResolveCubeSamplable(const
    ITextureCubeRenderer* tex)
2  {
3      return tex != nullptr ? dynamic_cast<const IWebGPUCubeSamplable
        *>(tex) : nullptr;
4  }
5  // ...
6  command.envMap = ResolveCubeSamplable(params.envMap);

```

`modules/renderers/webgpu/examples/webgpu_rendertargetcube_test.cpp`'s own Check C is built specifically to prove this fix rather than merely exercise the feature: it clears all six faces of a real `RenderTargetCube` to blue, binds it as `EnvironmentMapEffect.EnvironmentMap`, draws a full-screen reflective quad, and asserts the center pixel is genuinely blue — a check that could not have passed under the old hardcoded-type cast no matter how correctly the rest of the render-target machinery worked, since the cast itself was the failure point.

The same test file's Check D is worth walking through on its own, because it targets a different, easy-to-get-wrong question: does switching *into* a cube face and back correctly restore the backbuffer's own render pass, with nothing leaking either direction?

```

1  device.SetRenderTarget(nullptr);           // targeting the
    backbuffer
2  device.Clear(Color::Blue);
3  device.SetRenderTarget(&cubeTarget, CubeMapFace::PositiveZ);
4  device.Clear(Color::Red);                 // targets the cube
    face, not the backbuffer
5  device.SetRenderTarget(nullptr);           // back to the
    backbuffer
6
7  // The backbuffer's own centre pixel must STILL be Blue -- the
    intervening
8  // cube-face Clear(Red) must not have leaked into it -- and the cube
    face
9  // itself must genuinely have received Red, not been silently
    skipped.

```

This mirrors the identical architecture check Task WEBGPU-53/54 already established for plain `RenderTarget2D` in the section above, extended to confirm the same "eager flush on target switch" design generalizes correctly to a third target kind rather than being coincidentally correct for the two it was originally built against. Two scope cuts are documented honestly rather than silently: mip-chain regeneration throws for `RenderTargetCube` (`mipMap=true` is rejected, matching plain `RenderTarget2D`'s own precedent on this renderer), and MSAA is not implemented — `MultiSampleCount` is accepted and ignored, always reporting 0 honestly rather than silently pretending to honor a request it cannot fulfil.

23.23.3 Why block-compressed texture upload remains open

The "no renderer anywhere in the project does real block-compressed texture upload" gap named below is worth a fuller account on this renderer specifically, because it was genuinely investigated here rather than assumed unfixable. A direct diagnostic against this project's own

pinned `wgpu-native v29.0.1.1`, run on the real development GPU, confirmed the hardware side is not the obstacle at all: `wgpuAdapterHasFeature(adapter, TextureCompressionBC)` returned true, and requesting a device with that feature in `requiredFeatures` succeeded too — this machine’s real adapter genuinely supports DXT1/3/5 upload today. The actual blocker sits one layer up, and applies project-wide, not just to `WEBGPU: Texture2D::FromStream()` always fully CPU-decompresses DXT-compressed source bytes to plain RGBA8 *before* any renderer ever sees them, and the shared `CNA::Internal::Graphics::ImageData` struct every renderer’s texture-creation path consumes has no field at all for a surface format or a compressed-bytes flag — it is hardcoded to a plain `std::vector<uint8_t>` of RGBA8 pixels. A WebGPU-only workaround was considered and deliberately rejected: hand-crafting a direct `IGraphicsRenderer::CreateTexture()` call with raw BC1 bytes stuffed into that RGBA8-typed field would compile and might even render correctly on this one renderer, but would be reachable by no real game (nothing in the actual `Texture2D` loading path could ever produce it) and would misrepresent `ImageData`’s own documented contract — exactly the shape of shortcut this project’s own culture treats as worse than leaving the gap honestly open. Closing this for real needs a cross-renderer `ImageData` design change (or a parallel compressed-texture entry point) plus a matching `Texture2D.cpp` change to stop always-decompressing DXT source data — deliberately scoped out of any single renderer’s own task list, tracked as a dedicated follow-up instead.

23.24 Three former limitations that are no longer true

The WebGPU plan moved quickly after this chapter’s first draft, so three earlier shorthand limitations need a precise correction rather than being allowed to turn into folklore. First, an arbitrary plain `Texture2D` now has a real renderer-level `GetData()` path. That method on `WebGPUTextureRenderer` copies the requested mip level into a temporary `MAP_READ` buffer with WebGPU’s required row alignment, waits through the asynchronous map callback, and then extracts the requested x, y, w, h rectangle. The dedicated `WebGPU_Texture2D_GetData` test proves three independently useful cases: an exact full-gradient round trip, a non-origin 2×2 sub-rectangle, and a distinct level-one mip round trip. That is deliberately tested through `IGraphicsRenderer`, not merely through the public `Texture2D`: the latter normally returns its shared CPU pixel shadow for a plain texture, so an XNA-layer round trip alone could pass while this GPU copy path remained a silent base-class no-op.

Second, culling, scissor, and viewport state are no longer merely stored values. Culling is baked into the WebGPU 3D pipeline key and mapped to `WGPUCullMode`; a differential pixel test proves that a deliberately chosen winding disappears under one XNA cull mode and remains visible under the other. Scissor and viewport are genuine dynamic render-pass state via `wgpuRenderPassEncoderSetScissorRect` and `wgpuRenderPassEncoderSetViewport`. Wiring the latter exposed an otherwise-hidden logical-versus-physical-size hazard: CNA’s stored logical viewport can exceed the real swapchain dimensions. The renderer therefore clamps it at application time, which is both a WebGPU API requirement and what restored the pre-existing 2D and 3D regression tests. Every queued `SpriteBatch` or 3D command captures its own viewport and scissor state at the public draw call. Replay applies those snapshots dynamically through the two native setters on the backbuffer, 2D-target, and cube-face paths; consecutive commands with identical state skip the redundant native call. Thus several changes inside one deferred pass no longer collapse to the last live value. The shared deferred pixel fixtures prove the per-command result, while the WebGPU cardinality tests prove it did not require a pipeline variant, render-pass split, extra submit, or one setter call for every same-state draw. A disabled or zero scissor expands to the full current target instead of issuing an invalid zero-area rectangle.

Third, this is not a claim that every state object is complete. `FillMode`'s `WireFrame` value is stored but polygon draws now refuse it because the pinned WebGPU API offers no polygon-mode switch. The renderer stores the complete stencil configuration but has deliberately not yet baked stencil operations into its many pipeline families. The last deferral is evidence-based rather than cosmetic: the cull-mode mapping itself required a pixel test to correct an initially plausible but wrong winding interpretation, so copying a stencil front/back mapping without an equivalent differential stencil test would risk making an invisible wrongness look “implemented.”

23.25 Clear ordering and usage are consistent across target types

Each public clear enters the same ordered stream as all eleven deferred draw families, carrying independent colour, depth, and stencil flags and values. Replay partitions one bind cycle into native pass segments: leading/consecutive clears can fold into the next segment's load actions, while a clear after a draw closes that segment so the following pass observes it at the exact public position. A discard-content target may clear its first segment as its usage permits; later explicit clears remain ordered rather than becoming one global final value. The shared clear-options and ordered-clear fixtures cover backbuffer, 2D-target, cube-face, and MRT-rejection boundaries without an intervening flush.

The shared FNA predicate maps only `DiscardContents` to discard; `Preserve` and `Platform` both preserve. WebGPU receives that Boolean for 2D and cube targets, and the first native segment selects clear versus load for colour, depth, and stencil together. Later segments load prior contents unless an explicit ordered clear selects an aspect. A cube face owns its colour view while all faces share the target's depth/stencil attachment, matching the reference contract. Cube-usage and depth/stencil-usage regressions distinguish all three enums and the six faces rather than inferring behavior from a combined clear.

The backbuffer's `PresentationParameters.RenderTargetUsage` remains outside this off-screen policy, and plural binding still rejects MRT rather than creating more than one attachment. The full comparison is §19.4.2; the separate usage matrix is §19.4.1.

23.26 What remains open

Several gaps are shared with every other CNA renderer rather than unique to this one: no renderer anywhere in the project does real block-compressed texture upload (the paragraph above covers exactly why, and why it is a shared architectural gap rather than a per-renderer one), and the cross-renderer design change that would make it reachable from a real game remains larger than any one renderer task. Specific to this renderer: multiple simultaneous render targets, actual stencil-operation mapping (not merely the already-stored state and dynamic reference value), `WireFrame` rendering, custom `SpriteBatch` effects, and custom WGSL effects authored outside the stock-effect set. `BlendState` is no longer on that list for the one attachment WebGPU can bind: all source/destination factors and operations, dynamic `BlendFactor`, slot-zero `ColorWriteChannels`, and `MultiSampleMask` reach keyed `SpriteBatch` and 3D pipelines with dedicated differential tests. Slots 1–3 remain inapplicable until MRT exists. The two `RenderTargetCube` scope cuts also remain honest: mipmapped cube targets throw at construction, and their per-target MSAA request is accepted but reports zero rather than claiming an unimplemented multisampled array-texture resolve. Browser/Emscripten WebGPU is entirely out of scope so far — this renderer is native `wgpu-native` only.

23.27 Verification methodology, and how it differs from D3D9’s

This renderer’s own verification runs on a native automated CTest suite (gracefully skipping where no Wayland/X11 display is available) plus a battery of dedicated pixel-assertion tests per feature area, and the one independent real-game integration test described above. It is explicitly *not* covered by the oracle-versus-real-XNA methodology Chapter 28 describes for D3D9 / D3D11 / D3D12 — that comparison against a genuine, running XNA 4.0 reference is reserved for the three Direct3D renderers specifically, not applied here.

23.28 Legacy SDL GPU Chapter Material

SDL_GPU is a CNA graphics renderer, and one this book itself was slow to give a proper account of — Chapter 79 names that delay honestly as a gap in this book, not in CNA’s own code. This chapter closes it: a real, Linux/Vulkan-driver-verified renderer implementing the full `IGraphicsRenderer` contract (Chapter 19), built on top of SDL3’s own `SDL_gpu` API rather than a native vendor API of its own.

23.29 Why this renderer costs nothing new

SDL_gpu dispatches internally to Vulkan, D3D12, or Metal depending on platform and driver availability, and CNA’s own `plan_sdlgpu.md` states plainly why this renderer was worth adding on top of the hardware-backed renderers the project already had: SDL3 is already a vendored dependency of this project (`third_party/SDL`), and `SDL_gpu.c` is already compiled into every prebuilt SDL3 package this repository produces — unlike Vulkan (the Vulkan SDK/loader) or WebGPU (`wgpu-native`, a separate library), standing this renderer up required zero new third-party dependency, only new CNA-side code against an API already sitting in the build. The main class is `CNA::Internal::Renderers::SdlGpu::SdlGpuRenderer`. Its source lives under `modules/renderers/sdl-gpu/src/`. Build the target `cna_renderer_sdl_gpu`. Select this renderer with `-DCNA_GRAPHICS_RENDERER=SDL_GPU`. The project’s own naming table is explicit that this class name is deliberately distinct from the pre-existing `SdlRenderer` in `SdlRenderer/` — the older `SDL_Renderer` 2D API and the newer `SDL_gpu` API are unrelated, and this project was careful not to let the similar SDL naming collide.

23.30 Precompiled bytecode only, in a fixed resource-binding order

SDL_gpu does not accept GLSL or HLSL source text at runtime the way `ShaderEffect`’s own public contract promises a game — `SDL_CreateGPUShader` takes only `SDL_GPUShaderFormat`-tagged precompiled bytecode: SPIR-V for the Vulkan driver, DXBC/DXIL for the D3D12 driver, MSL/`metallib` for the Metal driver. A further constraint worth knowing before assuming any existing shader is reusable as-is: every stage’s samplers, storage textures, storage buffers, and uniform buffers must be declared in a fixed, HLSL-style binding order (`t[n]` / `s[n]` in one address space, `u[n]` in another, `b[n]` in a third) — a different convention from the raw `layout(set=, binding=)` numbering the existing `VulkanRenderer`’s own GLSL shaders use. This means Vulkan’s compiled SPIR-V is a useful *algorithmic* reference for this renderer’s own shader authoring (the same lighting math, alpha-test logic, and dual-texture blend) but not directly reusable output — every shader’s binding layout had to be re-authored specifically for `SDL_gpu`’s own convention, confirmed directly against the real `.glsl` sources under this renderer’s own `modules/renderers/sdl-gpu/src/shaders/` directory (23 files today, one pair

per stock effect — `colored3d`, `alpha_test3d`, `dual_texture3d`, `env_map3d`, `lit_textured3d`, `pbr3d` / `pbr_skinned3d`, `skinned3d` / `skinned_colored3d`, and more).

23.30.1 The screenshot that exposed a Y-flip

This renderer’s own first real milestone — a textured, tinted, rotated, multi-flip sprite scene surviving a resize — is a concrete instance of a theme this book returns to repeatedly (Chapter 70): a “did it throw” test alone cannot catch a wrong-but-plausible image. `SDL_gpu`’s Vulkan driver flips clip-space Y internally, for cross-renderer NDC consistency reasons of its own — the vertex shader’s first, naively-ported NDC computation (`gl_Position = vec4(ndc, 0, 1)`), the same shape every other renderer’s own sprite vertex shader already uses) compiled cleanly, ran without error, and rendered every single sprite upside down, a texture’s top row appearing at the bottom of the screen. Only a real captured screenshot caught it; the fix negates Y explicitly (`vec4(ndc.x, -ndc.y, 0, 1)`), confirmed against an isolated single-sprite diagnostic tool kept permanently in the tree (`modules/renderers/sdl-gpu/examples/sdlgpu_diag_single_sprite.cpp`, the same “keep the diagnostic, don’t throw it away once the bug is fixed” precedent Chapter 28 already covers for `cna_diag_d3d12_swapchain`).

23.31 SkinnedEffect’s storage-buffer workaround and push-uniform limit

Wiring `SkinnedEffect` through this renderer surfaced a genuine hardware/driver constraint rather than a CNA design choice: `SDL_PushGPUVertexUniformData`, the mechanism every other stock effect above uses to deliver its per-draw uniforms, has a real, undocumented cap of roughly 4096 bytes per slot on this renderer’s own Vulkan driver — found by a real empirical binary-search spike, not read out of any SDL3 header or release note, since the limit is not documented anywhere in `SDL_gpu.h` itself. A 72-bone skin palette (72×64 bytes = 4608 bytes) exceeds that cap outright, so `SkinnedEffect`’s bone matrices are uploaded through a real GPU storage buffer (`SDL_BindGPUVertexStorageBuffers`) instead of a uniform push — every other stock effect’s smaller per-draw payloads (world/view/projection, `DiffuseColor`, alpha-test parameters, light parameters) stay on the ordinary uniform-push path, since none of them come close to the cap. This is exactly the kind of real, hardware-shaped constraint this book asks every renderer chapter to name precisely rather than paper over with “uniforms are uploaded per draw” as if the mechanism were uniform across every payload size.

23.32 Custom ShaderEffect runtime compilation without a development package

Custom `ShaderEffect` support (Chapter 17) is real on this renderer too, and its own implementation is worth knowing about for a reason beyond the feature itself. Since `SDL_gpu` accepts only precompiled bytecode, `SdlGpuEffectRenderer` bridges `ShaderEffect`’s public GLSL-source contract to that requirement with a genuine runtime `libshaderc` GLSL-to-SPIR-V compile, linked directly into the renderer binary rather than merely invoked at build time. Reading `SdlGpuRenderer.cpp` directly shows a small, telling detail: this dev environment has no `libshaderc-dev` package installed, so there is no real `shaderc.h` to include — the renderer’s own source hand-declares the exact handful of `shaderc_*` C entry points it needs. These include `shaderc_compiler_initialize`, `shaderc_compile_into_spv`, and `shaderc_result_get_bytes`. Their `extern "C"` function signatures, shader-kind values, and optimization-level values are hand-authored as named constants rather than pulled from a missing header. This is a real, working engineering accommodation to a genuinely constrained dev environment, not

a shortcut around the feature itself — the compiled shader still goes through the real linked `libshaderc.so`, only the header describing its C ABI is hand-authored instead of vendored. Building this custom-shader path also surfaced a genuine finding one layer over: the existing `VulkanRenderer`’s own `ShaderEffect` support does not actually runtime-compile GLSL text at all, despite `ShaderEffect`’s own documented contract implying every renderer does — a real, previously-unflagged discrepancy between one renderer’s implementation and the class’s own stated cross-renderer promise, left for Chapter 23 to account for on its own terms rather than resolved here.

23.32.1 The `.cnj` fixture exposes the remaining shader-compatibility boundary

The runtime compiler does not make every GLSL text accepted elsewhere in CNA portable to this renderer. The real `CnjEffectTest.LoadsRealCnjFixture` content test writes an ordinary `.cnj` effect manifest plus a GLSL ES 3.00 vertex/fragment pair: `#version 300 es`, unqualified stage outputs/inputs, and loose `uniform mat4 projection / uniform sampler2D texture1` declarations. Under `SDL_GPU`, that fixture fails before a pixel is drawn. Its source is not in the SPIR-V-compatible dialect required by the runtime compiler and by the resource counts CNA supplies to `SDL_CreateGPUShader`.

The passing `SdlGpu_ShaderEffect` regression makes the required contrast concrete. Its two sources use desktop `#version 450`, explicit `layout(location=...)` qualifiers for every vertex attribute and stage I/O, and the exact fixed binding spaces this renderer declares: the fragment sampler is `layout(set=2, binding=0)`, while the per-draw values live in a named uniform block at `set=1` for the vertex stage and `set=3` for the fragment stage. It compiles both stages at runtime, draws a white texture into a `RenderTarget2D`, and reads back the effect’s own blue-tinted value; repeating the same draw without the effect must read white. Thus the supported dialect and binding contract are proven by pixels, not merely by `IsEffectValid()`.

This is deliberately an open compatibility gap, not evidence that `ShaderEffect` is a no-op on `SDL_GPU`. Closing it needs a separately designed decision: either migrate the shared `.cnj` fixture/source convention to the stricter portable dialect, or teach this renderer to translate its existing loose GLSL into the blocks, locations, and bindings `SDL_gpu` requires. Neither is a safe one-line relaxation; the first changes content expectations across renderers, and the second is a real shader-translation layer. The chapter therefore records the narrower, honest current contract: custom effects work when authored for `SDL_GPU`’s binding dialect, while the generic `.cnj` fixture is not yet cross-renderer portable to it.

23.33 Pipeline state is snapshotted at draw time, except where it is deliberately not

`SDL_gpu` graphics pipelines are immutable enough that CNA cannot simply bind an XNA state object at the end of a deferred frame and hope it represents every earlier draw. Each queued sprite or 3D draw therefore captures a `RenderStateSnapshot`: blend factors/functions, cull and fill mode, stencil operations/masks/reference, and the ordinary depth-test fields travel with the command that was issued under them. Pipeline lookup folds the pipeline-relevant portion into a `size_t` hash together with primitive topology, target color format, and sample count. This replaced an earlier hand-packed integer key, whose fixed bit budget would have become silently collision-prone as state dimensions were added.

The detail that keeps the cache both correct and finite is conditional hashing. Disabled blend does not hash its six blend factors/functions; disabled stencil does not hash operations or masks;

and disabled two-sided stencil does not hash the counter-clockwise fields. Two commands that produce the same pipeline consequently share it even if their irrelevant XNA state-object fields differ, while a meaningful state change cannot reuse a pipeline with stale `SDL_gpu` descriptors.

The target format included in that key is not the public `backbuffer-format` field. Each frame queries SDL for the swapchain texture's actual format and builds the matching pipeline. Independently, construction chooses D24S8 or D32S8 by a real device-support query and later allocates that combined attachment whenever a backbuffer frame needs it. A public `DepthFormat::None` does not suppress it; absence means the device supported neither combined format. Fullscreen remains an SDL-window operation outside this renderer hook, and swapchain acquisition follows the resulting window without selecting formats from the public enums.

Stencil reference itself is intentionally *not* pipeline state: the captured value is sent per draw through `SDL_SetGPUStencilReference()`, the missing call that an adversarial pixel test once exposed. A left-half stencil write followed by an Equal-reference reveal now proves that this dynamic value reaches the GPU rather than merely being stored in CNA.

Viewport, scissor, and blend constants follow the same per-command rule. Every queued draw reference snapshots the active viewport rectangle/depth range, scissor rectangle and enable bit, and blend factor; replay applies them through `SDL_SetGPUViewport`, `SDL_SetGPUScissor`, and `SDL_SetGPUBlendConstants` immediately before that draw. Enabled scissors are clamped to the target; disabled or degenerate ones expand to its full extent. The shared deferred-viewport/scissor suites queue several changes without a flush and cover backbuffer, 2D-target, and cube-face routes, so a final live state cannot satisfy them by accident.

Depth bias is pipeline-static in `SDL_gpu` 3.5 rather than dynamic. CNA normalizes both `RasterizerState.DepthBias` and `SlopeScaleDepthBias` by primitive topology, includes their bits in every affected pipeline key, and fills SDL's constant and slope factors when creating the pipeline. The dedicated test combines structural key checks, differential depth pixels, and validation-fatal execution. Samplers likewise carry the full filter ordinal, U/V address modes, and clamped `MaxAnisotropy` through both cache key and native descriptor; anisotropy is no longer merely a stored capability bit.

23.34 Deferred rendering still preserves the game's draw order

Deferring all `SDL_gpu` work until `Present()` once introduced a correctness bug hidden by the renderer's family-specific command vectors: it rendered every 3D family first and every sprite last, regardless of the order the game issued them. A background sprite, 3D model, and HUD sprite therefore became background/model/HUD only by accident; even the opposite sprite-then-3D order was replayed as 3D-sprite. This is visible whenever later opaque or blended content should cover earlier content, not merely a performance detail.

Every `Queue*Draw()` and `QueueSprite()` call now appends a compact kind/index reference beside its family-specific command. At render time, `RenderQueuedDraws()` makes one chronological pass over those references and dispatches the existing per-family issue routine only when its command belongs to the current target. No sort is needed: append order is already the public draw-call order. Pipeline rebind avoidance is then tracked across all kinds rather than only sprites, so fixing correctness does not require forgetting the ordinary bind cache.

`SdlGpu_DrawOrder` proves both directions with no alpha arithmetic ambiguity. It renders an opaque red full-target sprite and an opaque green full-target 3D quad into separate targets

with depth disabled. Sprite then 3D must read back green; 3D then sprite must read back red. The former fixed-family implementation produced red in both cases because sprites always ran last, so the pair is a discriminating regression rather than a no-crash smoke check.

23.35 Present timing belongs to the device manager, not a test-only renderer poke

The `SDL_GPU` tests establish a small but important rule about CNA’s initialization lifecycle. `GraphicsDeviceManager` defaults `SynchronizeWithVerticalRetrace` to true, which is the right XNA-compatible default for an ordinary game. On this project’s virtual test display, however, there is no useful vertical-retrace signal; leaving it enabled made each frame wait roughly one second. A 60- or 120-frame GPU regression test therefore consumed its CTest budget without doing more useful rendering. The correct setup, used by all 22 current `SDL_GPU` examples and diagnostics, is public configuration made before `Game::DoInitialize()` creates or resets the device:

```
1  gdm_ = std::make_unique<GraphicsDeviceManager>(this);
2  gdm_ -> setPreferredBackBufferWidthProperty(64);
3  gdm_ -> setPreferredBackBufferHeightProperty(64);
4  gdm_ -> setSynchronizeWithVerticalRetraceProperty(false);
```

This order matters. Earlier tests called `SdlGpuRenderer::SetSwapInterval(0)` directly as a local workaround. During initialization, though, the device manager converts its still-true public property to `PresentInterval::One`; `GraphicsDevice::Reset` then forwards that presentation interval to every renderer. The correct global forwarding was itself a real repair: before it landed, the reset path did not call `IGraphicsRenderer::SetSwapInterval()` at all. Once repaired, it rightly overwrote the private pre-initialization poke, revealing the test timeout instead of preserving an accidental ordering dependency.

For this renderer, interval zero asks `SDL_gpu` for `SDL_GPU_PRESENTMODE_IMMEDIATE`, falling back to `MAILBOX` if Immediate is unavailable; any positive interval selects `VSync`. `SDL_gpu` has no half-rate present mode, so XNA’s `PresentInterval::Two` has the same `VSync` outcome as `One`. That makes the example’s explicit false a test-environment choice, not a claim that games should globally disable synchronization: it uses the same public property a game would use, at the phase of the lifecycle where CNA will not subsequently replace it.

23.36 Two violations exposed by validation mode

This renderer now passes `SDL_gpu`’s `debug_mode` flag from CNA’s ordinary build mode: debug builds request validation and release builds do not. That deliberately modest toggle paid for itself immediately. With validation previously hardcoded off, two API-contract violations had appeared to work; with it on, each could hang the Vulkan driver rather than merely print a warning. The lesson is more useful than a generic “enable validation” recommendation: a renderer’s release-mode pixels can look plausible while resource descriptions are still invalid.

First, the MSAA implementation for `RenderTargetCube` used a six-layer 2D-array texture. `SDL_gpu` forbids multisampling on an array texture. CNA now uses one single-layer, multisampled 2D attachment for whichever cube face is currently rendering and resolves it into the actual cube texture; cycling is permitted only for that temporary attachment, never for the cube texture that must retain all six faces. Second, the initial automatic-mipmap implementation for plain `Texture3D` and `TextureCube` requested only `SAMPLER` usage, although `SDL_gpu`’s generator requires `COLOR_TARGET` too. Widening the usage exposed a third, deeper

failure: SDL’s Vulkan path created invalid 2D views and blits for depth planes that no longer exist in smaller 3D mip levels.

That third finding is no longer open. REMED-GFX-099 removed the invented automatic generation behavior from plain Texture3D: XNA/FNA’s contract allocates an authored chain and lets each `SetData(level,...)` call populate the named level. The resource is therefore SAMPLER-only again, with no target views or generator blits to violate the Vulkan contract. A validation-fatal regression now covers exact level-zero and nonzero-mip round trips, partial volumes, distinguishable depth planes, forward/reverse write order, NPOT dimensions, and two alternating resources. Plain cube textures retain their separately tested per-face behavior. This sequence is why validation must be treated as evidence with scope: it first exposed tolerated violations, then forced the design back to the actual public resource contract rather than merely suppressing their diagnostics.

23.37 A pre-release render-target use-after-free

`RenderTarget2D` and `RenderTargetCube` support (with real multisampling and mip regeneration) surfaced a genuine lifetime hazard while its own test was still being written, not after: this renderer defers its actual render pass to `Present()` time, so a render target destroyed *before* that deferred pass executes was a real use-after-free the moment anything still referenced its underlying GPU state. The fix, `SdlGpuRenderTarget2DState` / `SdlGpuRenderTargetCubeState`, is a pair of `shared_ptr`-owned structs holding the actual GPU state independently of the `SdlGpuRenderTargetRenderer` / `SdlGpuRenderTargetCubeRenderer` wrapper’s own C++ object lifetime — a short-lived, local render target destroyed mid-`Draw()` now keeps its pending content alive long enough to render correctly and remain safely sampleable elsewhere the same frame. The bug was confirmed via a real segfault in `sdlgpu_mrt_test.cpp` before the fix landed, not merely reasoned about in the abstract.

23.37.1 A regression that makes destruction precede submission

The permanent regression test is deliberately stricter than “a short-lived target does not crash.” In `modules/renderers/sdl-gpu/examples/sdlgpu_rendertarget_lifetime_test.cpp`, one 8×8 `RenderTarget2D` is created as a local variable inside a single `Draw()` call, cleared red, then sampled by a queued `SpriteBatch` draw into a still-live 16×16 destination target that was first cleared blue. The local variable reaches the end of its scope before `Present()` invokes `EnsureFrameRendered()`, so the sampled draw is still only a command waiting to be submitted:

```

1  void DrawThroughShortLivedRenderTarget(GraphicsDevice& dev) {
2      RenderTarget2D localRt(dev, 8, 8, false, SurfaceFormat::Color,
3                              DepthFormat::None, 0, RenderTargetUsage::
4                                  DiscardContents);
5      dev.SetRenderTarget(&localRt);
6      dev.Clear(Color::Red);
7      dev.SetRenderTarget(nullptr);
8      dev.SetRenderTarget(rtDest_.get());
9      dev.Clear(Color::Blue);
10     sb_>Begin();
11     sb_>Draw(localRt, Rectangle(0, 0, 16, 16),
12              Rectangle(0, 0, 8, 8), Color::White);
13     sb_>End();
14     dev.SetRenderTarget(nullptr);
15 } // localRt is destroyed here, before this frame is submitted

```

On the first frame the test reads the centre pixel of `rtDest_back` and requires exact red (within its small channel tolerance), proving that the queued sampling operation really ran; a no-crash-only assertion could otherwise pass after silently omitting the draw. It then performs the same create-clear-sample-destroy pattern for 120 frames, which checks that the deferred-release queue is drained after each submission rather than merely retaining every texture until shutdown. The implementation has two necessary lifetime stages: queued targets and draw targets retain a `shared_ptr` to their internal state until frame rendering has finished, then that state's destructor places its raw color, depth, and MSAA handles in `pendingTextureReleases_`. After a successful command-buffer submission, the renderer releases each queued handle through `SDL_ReleaseGPUTexture()`. SDL may still fence the physical memory internally; CNA's narrower rule is that a recording cannot release a handle which an unsubmitted command still names.

23.38 MRT as a shader capability

`GraphicsDevice::SetRenderTargets()` makes the first `RenderTarget2D` the primary target and records every later target as one simultaneous `SDL_gpu` color attachment. At `Present()` the primary's render pass contains $1 + |\text{mrtSiblings}|$ `SDL_GPUColorTargetInfo` entries; the custom effect pipeline cache includes that attachment count, so a pipeline created for one color output cannot accidentally be rebound for two. All attachments receive the same CNA blend state, just as one `GraphicsDevice::BlendState` governs the whole draw rather than one state per target.

That does *not* mean every draw writes every attachment. CNA's stock sprite and 3D fragment shaders declare one output, so their real, deliberately narrow MRT contract is: target zero receives the draw while targets one onward can still be bound and cleared independently. The second half of `sdlgpu_mrt_test.cpp` proves the broader custom-effect path by binding two fresh targets and issuing exactly one `SpriteBatch` draw with a `#version 450` fragment shader containing `layout(location=0) out vec4 outColorA` and `layout(location=1) out vec4 outColorB`. Sampling a white texture with tint (0.2, 0.4, 0.8, 1) must read (51, 102, 204, 255) from A and its shader-computed channel swap (102, 204, 51, 255) from B. Those distinct read-backs rule out both an extra clear and a copied single-output result: one fragment invocation really wrote two simultaneous attachments.

MRT lifetime follows the same ownership repair as ordinary deferred targets. Each immutable `PassSegment` retains its primary and every secondary attachment as `shared_ptr<SdlGpuRenderTarget2DState>`; explicit clears operate on the segment rather than a parallel wrapper-pointer list. Destroying a public secondary wrapper therefore cannot invalidate an already-recorded bind cycle, and the render-target-lifetime regression covers the create-bind-draw/clear-destroy-present sequence.

23.39 Usage selects per-resource first-use load/store behavior

Each 2D target and each cube face begins with independent first-use colour/depth/stencil flags. The shared Discard bind records the FNA-style black/max-depth/zero-stencil clear into its new bind-cycle segment; Preserve records none. Once a pass has written the resource, later cycles load its stored aspects unless an explicit ordered clear says otherwise. MRT applies the primary cycle's explicit colour clear to all simultaneous colour attachments while retaining each attachment's owning state. The depth/stencil usage fixture renders distinct occlusion and stencil-gate outcomes across unbind/rebind cycles, so the result is not inferred from colour.

Cube usage now reaches construction too. A single-sample face naturally loads its cube layer;

a multisampled face owns a persistent per-face attachment and uses `RESOLVE_AND_STORE` for Preserve so the next cycle can load its samples. Discard may use plain `RESOLVE` when no later segment in the frame needs them. The six-face MSAA/mip and depth/stencil usage suites cover these paths. `PlatformContents` follows the shared FNA predicate `usage != DiscardContents` and therefore preserves like Preserve; cube and depth/stencil usage fixtures assert that policy explicitly. See §19.4.1.

23.39.1 Texture3D’s cycle=true orphaned-write defect

`Texture3D` support on this renderer — sub-volume upload/readback plus mipmap generation — surfaced a real data-corruption bug of exactly the kind this book keeps returning to: code that compiles cleanly, throws nothing, and produces a wrong answer silently. The relevant upload operation is `SDL_UploadToGPUTexture`. Its `cycle` parameter offers a real performance trade-off SDL3’s own documentation states plainly: `cycle=true` swaps the texture to a fresh, separate underlying GPU resource on each call, avoiding a stall if the GPU is still reading the previous resource’s content. That is correct and desirable for `Texture2D`’s single-full-texture-replace `SetData()` path. CNA’s `SDL_GPU` texture renderer calls that path `UpdatePixels`; it deliberately retains `cycle=true`. The first `Texture3D` implementation copied that choice without accounting for a 3D texture’s multiple writes. The assumption breaks for `Texture3D` specifically, because its content is built up through *multiple independent* sub-volume and per-level `SetData()` calls that all need to land on the *same* underlying resource — with `cycle=true`, an earlier partial write (say, a sub-volume at $z = 0$) silently orphaned itself onto an abandoned GPU resource the moment a second, later write (a sub-volume at $z = 1$) followed it, reading back as zero/uninitialized rather than the real content actually uploaded.

The real regression test that caught this (`modules/renderers/sdl-gpu/examples/sdlgpu_texture3d_test.cpp`) is deliberately shaped to make the bug unmissable rather than merely possible to catch:

```

1 Texture3D tex(dev, 4, 4, 2, false, SurfaceFormat::Color);
2 // A 2x2x2 sub-volume at offset (1,1,0) -- red at z=0, green at z=1
   -- deliberately
3 // off-center within the full 4x4x2 volume, and two SEPARATE SetData
   () calls, one per Z slice.
4 tex.SetData(0, /*x=*/1, /*y=*/1, /*z=*/0, 2, 2, 1, redPixels.data(),
   redPixels.size());
5 tex.SetData(0, /*x=*/1, /*y=*/1, /*z=*/1, 2, 2, 1, greenPixels.data
   (), greenPixels.size());
6 // With cycle=true, the z=0 write above would already be silently
   orphaned by the time this
7 // GetData() runs, reading back zero instead of red.
8 std::vector<Color> gotRed(4), gotGreen(4);
9 tex.GetData(0, 1, 1, 0, 2, 2, 1, gotRed.data(), gotRed.size() *
   sizeof(Color));
10 tex.GetData(0, 1, 1, 1, 2, 2, 1, gotGreen.data(), gotGreen.size() *
   sizeof(Color));

```

Two design choices make this test genuinely discriminating rather than accidentally passing either way: the sub-volume is off-center (a bug that only corrupted, say, the texture’s origin corner would still show up), and each Z slice gets its own distinct solid color rather than reusing one color for the whole volume — a uniform-color test cannot tell correctly-downsampled or correctly-retained content apart from an out-of-range read that happens to land on more of that same color, the identical discriminating-test principle Section 23.41’s own generated-mipmap gap needs applied to close it. The fix, confirmed against the real `SdlGpuTexture3DRenderer`:

`SetData` implementation directly, is a one-word change: pass `cycle=false` for every `Texture3D` upload, leaving `Texture2D`'s own `cycle=true` single-replace path untouched and correct as-is. The identical fix (and the identical reasoning, cited by name in that renderer class's own source comment) was applied proactively to `TextureCube`'s per-face uploads too, since a cube map is built up through six separate per-face `SetData()` calls that must all land on the same underlying resource — caught and fixed before shipping rather than found by reproducing the same bug a second time.

23.40 Forced swapchain failure preserves the queued frame

`SDL_gpu` distinguishes two acquisition outcomes that a renderer must not conflate. A successful `SDL_WaitAndAcquireGPUSwapchainTexture()` call with a null texture is a documented, non-error case such as a minimized window: CNA submits the command buffer and simply skips that frame. A false return is a hard acquisition failure. SDL forbids cancelling the command buffer after this call, so CNA submits it first and then throws with the captured SDL error. Crucially, it leaves `framePending_` true: a caller which recovers the window must be able to submit the same queued clear and draws rather than silently lose them.

`sdlgpu_swapchain_recovery_test.cpp` proves that latter path without pretending a real device loss is easy to reproduce. On frame 10 it calls SDL's own `SDL_ReleaseWindowFromGPUDevice()`, makes `Present()` fail, then uses SDL's matching claim call to reclaim the identical window before calling `Present()` again. The test requires all five stages: nine ordinary frames, the induced throw, successful reclaim, successful presentation of the preserved frame, and 30 further frames without exception. This is a bounded, controlled proof of recovery after a hard acquisition failure; it does not claim to solve the separate lazy-proxy backbuffer readback path described below.

23.41 What remains open

Three limits are worth stating precisely rather than glossing over. First, occlusion queries are a *permanent* limitation of the underlying API, not a task gap: the vendored `SDL_gpu.h` has no query-pool or occlusion-query type anywhere in it (only `SDL_GPUFence` for CPU/GPU synchronization), so `CreateOcclusionQuery()` correctly returns `nullptr` on this renderer, the identical posture `HEADLESS` and `SOFTWARE` already take for their own, unrelated reasons — worth re-opening only if a future SDL3 release adds real query support, not before. Second, the generic `.cnj` custom-effect fixture remains a cross-renderer portability gap: its loose GLSL ES 3.00 declarations are not accepted by this renderer's explicit-location, uniform-block binding contract, even though the dedicated `#version 450` effect regression is pixel-verified. That choice is deliberately left open pending either a shared content-dialect migration or a real shader-translation layer. Third, hardware instancing is absent at a different layer: this otherwise 3D-capable renderer does not override `DrawInstancedPrimitivesEx()`, so every valid public `DrawInstancedPrimitives()` call reaches the common `std::runtime_error` default. `SDL_gpu` itself exposes per-instance vertex input; the gap is CNA-side pipeline/binding work, not a permanent underlying-API limitation. The public `GraphicsCapability::Instancing` entry now exists, but this renderer does not override the permissive base policy and therefore incorrectly reports it as supported. This is a capability overclaim: callers cannot safely use the bit as a guard until `SDL_GPU` either implements the draw route or returns false explicitly (Section 19.3.2).

Five former entries no longer belong in this list. Depth bias and slope-scale bias are normalized per polygon topology, hashed into every relevant pipeline key, and applied through SDL's

pipeline-static rasterizer fields; a dedicated pixel/validation test covers them. `MaxAnisotropy` is part of the complete sampler key and native descriptor. Secondary MRT attachments are retained as `shared_ptrs` in the immutable pass segment, closing the raw pointer lifetime hazard. Finally, ordinary `Texture2D` allocates its declared native mip count and overrides `UpdatePixelsLevel`; the native-allocation/upload regression proves that a nonzero level reaches the GPU resource. None of these closures adds automatic downsampling to plain `Texture2D`: its upper levels remain explicitly authored, as in the XNA/FNA contract.

Backbuffer readback is also closed without charging games that never use it. SDL's swapchain texture remains permanently write-only, but the first `GetBackBufferData()` lazily enables a self-owned, swapchain-sized proxy. The pending frame renders into that proxy, one 1:1 blit presents it, and `ReadBackbuffer()` downloads the requested logical rectangle through a transfer buffer and fence, converting BGRA to public RGBA when needed. Before the first read the proxy and blit do not exist. First-read, whole-logical-dimension, and fully-written-range tests cover the observable contract.

Beyond these three remaining limits, Windows (D3D12 driver) and macOS/iOS (Metal driver) remain code paths only, not validation claims, until run on real (or Wine/DXVK-class) hardware — the same phased-claim discipline this book already applies to D3D11 / D3D12 (Chapter 28) and WebGPU (Chapter 23.18).

23.42 Clear state is per segment and issue-ordered

This renderer carries independent colour, depth, and stencil requests on each backbuffer, 2D-target/MRT, or cube-face segment. Consecutive clears before any draw coalesce safely into the segment's load action, aspect by aspect. Once a draw has made ordering observable, the next clear opens another segment over the same destination; the following native pass loads unselected aspects, clears selected ones, and then replays later draws. MSAA segments use `RESOLVE_AND_STORE` whenever a later segment must load their samples.

The renderer-neutral clear-options and ordered-clear fixtures distinguish isolated aspects, draw-clear versus clear-draw, multiple clears, and backbuffer/2D/cube/MRT destinations without an intermediate flush. Pass-boundary and backbuffer-order suites separately prove that target rebinding and target-backbuffer interleaving remain in public order. Deferred submission is therefore an implementation strategy, not a remaining clear-order exception; the comparative matrix is §19.4.2.

23.43 Verification methodology

The executable claims above have dedicated tests rather than resting on code reads alone. Counting the actual module-local registrations directly (`modules/renderers/sdl-gpu/examples/CMakeLists.txt`) rather than trusting an old plan snapshot shows **83** `SdlGpu_*` CTest registrations today, spanning the 2D and 3D smoke tests, every stock effect (including `EnvironmentMapEffect`, `SkinnedEffect`, and the CNAEXT `PbrEffect` this book's WebGPU chapter already introduced), custom `ShaderEffect`, every render-target variant (2D, cube, multiple render targets, MSAA), both extra texture types, sampler/render state, draw order, and swapchain recovery, plus the render-target-lifetime regression above — run, like every other renderer's own CTest suite, against this project's own `cna_register_renderer_test()` convention, gracefully skipping with a clear reason rather than hard-failing when no real `DISPLAY` / `WAYLAND_DISPLAY` is present, the identical convention D3D11 / D3D12 / WebGPU already follow. All verification to date is against this project's own real dev-machine GPU via `SDL_gpu`'s Vulkan driver, the only driver this project's own prebuilt SDL3 packages compile in

on Linux — the same single-driver-verified, other-drivers-unverified posture this chapter’s own “What remains open” section already names honestly rather than implying broader coverage than actually exists.

Chapter 24

BGFX, Magnum, LLGL, and Diligent

BGFX integrates the third-party `bkaradzic/bgfx` rendering library (fetched directly via CMake’s `FetchContent`, Chapter 19) behind the same contract every other renderer implements, using `bgfx`’s own native API — window and platform initialization, texture creation, sprite draws, frame submission — rather than routing through `SDL_Renderer`.

The chapter compares four wrappers whose underlying API can be chosen below CNA. That second selection axis is part of every bug report: BGFX on Vulkan and BGFX on OpenGL are one CNA family but not the same native execution path.

Identity	Backend resolution	Shader strategy	RT / MRT / query
BGFX	runtime library selection, CNA override available	bgfx shader assets	yes / yes / device bit
MAGNUM	compile-time desktop GL path	raw GLSL compiled at runtime	yes / runtime maximum / yes
LLGL	runtime module selection, Linux currently OpenGL	generated SPIR-V and verbatim GLSL	yes / yes / yes
DILIGENT	runtime ordered device fallback	one HLSL source cross-compiled at runtime	yes / yes / device features

The abstraction boundary also owns this renderer’s default attachments. CNA passes `bgfx` a window, dimensions, renderer selection, and reset flags, but neither the requested `SurfaceFormat` nor `DepthFormat`. The exact platform backbuffer and depth/stencil format are therefore `bgfx` choices which CNA neither selects nor queries, even though it submits depth/stencil clear flags. A shared `SDL` fullscreen call can still resize or restyle the window, after which `EnsureViewState()` notices the dimensions and calls `bgfx::reset`; the empty presentation-format hook does not prevent that window route.

24.1 Clear owns an ordered, aspect-selective view

BGFX exposes clearing through view state rather than an issue-ordered clear command, so CNA now makes each public clear its own view. `RecordClear()` allocates the next per-frame view, binds the current backbuffer/2D/cube/MRT destination, installs exactly the requested

BGFX_CLEAR_COLOR, DEPTH, and/or STENCIL mask and values, then touches the view so a clear-only cycle is observable. A later draw or clear receives another view; `bgfx::setViewOrder()` remaps those views into public call order. Clear therefore ignores a custom viewport and covers the full target, while draw-clear, clear-draw, repeated clear, target-switch, cube-face, and MRT sequences retain their chronology. The shared clear-options and ordered-clear suites discriminate masks and order rather than merely proving that a final colour appeared; see §19.4.2.

BGFX’s explicit-view model implements target usage without a native load-policy object. The shared layer maps only `DiscardContents` to a deterministic black/max-depth/zero-stencil clear on every bind; that request becomes its own ordered clear view. Preserve and Platform issue no bind clear, so the attachment survives by construction under `CLEAR_NONE`. The same rule now reaches 2D targets and cube faces, including per-face multisample colour; depth/stencil follows it too. The cube-usage, per-face-MSAA, and depth/stencil-usage pixel suites distinguish Discard, Preserve, and Platform rather than relying on the older no-crash usage smoke test. The full comparison is §19.4.1.

24.2 Recorded BGFX campaign

The recorded BGFX validation campaign reported 4,375 passes in a 4,377-case configured population, with two hardware skips; its renderer-specific `ctest` population passed 103 of 105. These are campaign populations, not a universal current test count; see Chapter 69. The two remaining outcomes were classified as environment ceilings: one render-target MSAA-resolve test fails because this project’s own headless `Xvfb` display has no DRI3 support to resolve against, and one render-target-cube depth-format test (a `Depth24Stencil8`-attached cube face producing no color output) has been investigated three times without finding a root cause and remains deferred. The row-by-row triage recorded 38 gaps, closed 37 in its final batch, and left the depth-bias item described below; that item was subsequently fixed. These are historical campaign results, not present-day population counts.

24.3 Choosing bgfx’s underlying native renderer

Unlike every other renderer in this book, BGFX is itself an abstraction over several further, distinct native graphics APIs — OpenGL, OpenGL ES, Vulkan, Direct3D 11/12, and Metal — and CNA exposes a real, previously-undocumented mechanism to choose among them: the `CNA_BGFX_RENDERER` environment variable, resolved through three small, dedicated functions in `BgfxRendererSelection.cpp`. `GetDefaultRendererType()` picks `bgfx::RendererType::OpenGL` on Linux specifically and `Count` (bgfx’s own “let the library auto-detect” sentinel) everywhere else; `ParseRendererTypeOverride()` case-insensitively accepts `AUTO`, `OPENGL`, `OPENGLES`, `VULKAN`, `METAL`, `DIRECT3D11` / `D3D11`, `DIRECT3D12` / `D3D12`, and `NOOP` (bgfx’s own headless no-op renderer, useful for pure logic testing with no GPU work issued at all), throwing a `std::runtime_error` naming every valid value for anything else; `ResolveRendererType()` ties the two together, falling back to the platform default whenever the environment variable is unset or empty.

24.3.1 Routing a test around a BGFX/OpenGL limitation

This mechanism exists for a concrete, real reason, not as a speculative escape hatch: `modules/renderers/bgfx/examples/bgfx_rendertarget2d_msaa_test.cpp`’s own header comment documents a genuine investigation. In this project’s own Linux development sandbox, bgfx’s default OpenGL renderer negotiates only a legacy OpenGL 2.1 context — confirmed

via `glxinfo` that the underlying Mesa/RadeonSI driver actually supports OpenGL 4.6 with full hardware acceleration, so this is bgfx itself requesting an old context by default, not a driver ceiling — and MSAA-flagged framebuffer-attached textures do not actually resolve with real sub-pixel blending under that legacy GL 2.1 path, despite `bgfx::getCaps()` itself reporting `BGFX_CAPS_FORMAT_TEXTURE_FRAMEBUFFER_MSAA` support for the exact format requested. The investigation ruled out a CNA-side defect specifically: forcing bgfx onto its Vulkan renderer instead (`CNA_BGFX_RENDERERER=VULKAN`, routing through the identical GPU) made both of the test’s own checks pass cleanly, including a genuine intermediate (blended) pixel value along the test triangle’s diagonal edge, with *zero* code changes on either side — proof the C++ wiring itself is correct and renderer-agnostic, and the gap is specifically in bgfx’s own default legacy-GL-context resolve path. The real `cctest` registration (`modules/renderers/bgfx/examples/CMakeLists.txt`) reflects this finding directly rather than leaving it as a comment nobody acts on:

```
1  cna_register_renderer_test(NAME Bgfx_RenderTarget2D_MsaaResolve
2    COMMAND cna_test_bgfx_rendertarget2d_msaa
3    TIMEOUT 30
4    ENVIRONMENT "SDL_VIDEODRIVER=x11;DISPLAY=${CNA_TEST_DISPLAY};
    CNA_BGFX_RENDERERER=VULKAN")
```

This is worth reading alongside, not instead of, this chapter’s own “Test baseline” section above and `docs/graphics-renderer-feature-matrix.md`’s repeatedly-reconfirmed count (spanning multiple independently-dated task closures) that `Bgfx_RenderTarget2D_MsaaResolve` remains one of exactly two still-failing tests in *that* project’s own tracked baseline, attributed there to a related but distinctly-framed cause: “this sandbox’s `Xvfb` has no DRI3 support.” The two explanations are not actually in tension — DRI3 is specifically what a windowed OpenGL context needs to negotiate real hardware acceleration under X11 indirect rendering, so its absence is a plausible reason bgfx’s OpenGL path falls back to a legacy context in the first place, while Vulkan’s own presentation integration does not depend on the same DRI3/GLX negotiation at all. What this book cannot independently confirm is whether the specific sandbox behind that repeatedly-cited count genuinely has real Vulkan/GPU access available to the `CNA_BGFX_RENDERERER=VULKAN` override the way the test’s own comment describes, or whether that particular environment lacks real GPU access down either path — this book’s own screenshot-feasibility investigation (Chapter 12) independently confirmed no GPU passthrough at all in *this* book-writing session’s own container, a genuinely different environment from CNA’s own separate development sandbox. Both real, sourced explanations are given here rather than silently picking one, consistent with this book’s own methodology of not resolving an ambiguity the underlying sources themselves leave open.

24.4 DepthBias: an upstream BGFX limit

`RasterizerState.DepthBias` is the one feature in this chapter whose fix required working around a genuine absence in bgfx itself, confirmed by reading bgfx’s own vendored source directly: its high-level state API has no depth-bias mechanism of any kind — no corresponding flag in its own headers, and no `glPolygonOffset` call anywhere in its vendored OpenGL renderer. The fix emulates the feature entirely at the CNA level, via a per-draw vertex-shader Z-offset (a new `u_depthBias` uniform threaded through every 3D vertex shader). `SlopeScaleDepthBias`, by contrast, remains *deliberately* unimplemented — a project-owner decision, not an oversight — because a true per-fragment, screen-space-slope computation would force every 3D shader off the GPU’s early-Z optimization path, a cost judged not worth paying for this specific, rarely-used parameter.

24.5 Render-target readback crashes: one root cause, five fixes

Five of six render-target-related `glReadPixels` / `Xvfb` crashes were traced to a single, shared root cause: bgfx processes its internal views in ascending ID order every frame, which means any render-target view is always the *last* one processed — and therefore still GL-bound at the exact moment `glReadPixels()` fires for a screenshot. The fix adds a dedicated “flush” view, touched immediately before every screenshot specifically to force the correct view to be current first. Only the render-target-cube depth-format crash mentioned above has a different, still-unidentified root cause and was not resolved by this fix.

24.6 View IDs: a free-list-backed pool, not a hardcoded slot

The screenshot flush route follows from bgfx’s numbered *views*, an ordering mechanism distinct from render-target binding. Every `bgfx::submit()` call targets a specific numeric view id, and bgfx processes views in ascending id order each frame. The original renderer assigned view 1 to every render target. `Detail::AllocateRtViewId()` now allocates from a free-list-backed pool:

```

1  static bgfx::ViewId AllocateRtViewId()
2  {
3      static constexpr bgfx::ViewId kMaxBgfxViews =
4          kBackbufferFlushViewId;
5      auto& pool = RtViewIdFreeList();
6      if (!pool.empty()) {
7          const bgfx::ViewId id = pool.back();
8          pool.pop_back();
9          return id;
10     }
11     static bgfx::ViewId nextId = 1;    // 0 is permanently reserved
12     // for the backbuffer
13     if (nextId >= kMaxBgfxViews)
14         throw std::runtime_error("Bgfx: exhausted view ids (more
15                                 concurrently-live "
16                                 "render targets than bgfx supports
17                                 views)");
18     return nextId++;
19 }
```

View 0 is reserved for the backbuffer. `kBackbufferFlushViewId` is the highest reserved view, so bgfx processes it last; the allocator’s exclusive upper bound keeps it out of the render-target pool. Freed IDs return to `RtViewIdFreeList()`, preventing normal creation/destruction churn from exhausting bgfx’s `BGFX_CONFIG_MAX_VIEWS` ceiling (256 by default). Simultaneously live targets can still exhaust the bounded pool, in which case creation throws `std::runtime_error`.

24.7 Sampling a render target: a wrong-handle-type cast, since fixed

Both `BgfxSpriteBatchRenderer::Draw` and the environment-mapping branch once cast any `ITextureRenderer` to the ordinary-texture concrete class. Render targets use sibling classes whose first member is a framebuffer handle. Because bgfx represents framebuffer and texture handles with the same `uint16_t` idx layout, the cast compiled and usually did not crash; it used a framebuffer-pool index as a texture-pool index and could sample an unrelated texture.

The current `SpriteBatch` route obtains dimensions through virtual `GetWidth()` and `GetHeight()`. Environment mapping resolves cube sampling through `IBgfxCubeSamplable` with a checked cast. Focused 2D and cube render-target tests cover the non-crashing type-confusion that ordinary smoke tests missed.

24.8 Corrected silent-black-mesh defect

One bug is worth naming for how invisible it was until specifically tested: this renderer’s graphics-renderer class never overrode the effect-aware indexed-draw entry point at all. Any indexed draw bound to an `Effect` whose vertex format lacked a `Color` attribute — which describes essentially any `Model` loaded through `ContentManager` (Chapter 38) — silently fell back to the base class’s default implementation, discarding the entire `GpuDrawParams` struct and reading an unbound color attribute (which GL defaults to black). The practical, visible consequence: any loaded 3D model rendered as a solid black silhouette on this renderer, regardless of its real diffuse color, texture, or lighting — fixed by adding a real override.

24.8.1 Recognizing the trigger condition

The black-mesh bug’s real trigger condition — a vertex format with no `Color` attribute, which is essentially every `Model` loaded from content rather than hand-authored with vertex colors baked in — is checkable directly against a real `VertexDeclaration`, grounded in `VertexElement`’s real accessor:

```

1  bool HasColorAttribute(const VertexDeclaration& decl)
2  {
3      for (const VertexElement& element : decl.GetVertexElements()) {
4          if (element.getVertexElementUsageProperty() ==
5              VertexElementUsage::Color) {
6              return true;
7          }
8      }
9      return false;
10 }
```

Before this bug’s fix, any effect-aware indexed draw where `HasColorAttribute` would have returned `false` was exactly the condition that silently rendered solid black on this renderer — which, per the finding above, covers essentially every ordinary `Model` draw, since a typical imported mesh carries position/normal/texture-coordinate data but no per-vertex color. This is precisely why the bug was invisible for so long despite being this common: nothing about it depended on an unusual asset or an edge-case vertex layout, only on whether anyone had specifically pixel-tested a textured, unlit 3D model on this one renderer.

24.9 OcclusionQuery: a real implementation whose correctness cannot be fully proven here

`OcclusionQuery` was previously a pure no-op on this renderer; the fix wires the query handle into `bgfx`’s own dedicated submit-with-occlusion-query overload, across all twelve 3D-draw submission call sites — no separate correlation/tagging machinery was needed the way Vulkan’s fix required, since `bgfx` submits synchronously. Two honestly-documented caveats remain: first, pixel/query correctness genuinely *cannot* be established in this project’s own sandbox, because sabotaging the fix back to a pure no-op produces identical `IsComplete()` / `PixelCount()` output to the fixed version — the sandbox’s own software Mesa driver returns a non-null

result even for a query that was never submitted anywhere at all, a real ceiling of the software renderer itself, not a CNA defect. Second, bgfx’s own official usage example attaches an occlusion measurement to a separate, dedicated view specifically so the query is not polluted by other geometry sharing the same depth buffer; CNA’s current fix instead shares whichever view the game’s own 3D draw already targets — a real, acknowledged architecture gap against true scene-depth query correctness, not yet attempted.

24.10 Instancing: exact transport, with a native-profile shader fault

`GraphicsDevice::DrawInstancedPrimitives()` reaches a real renderer implementation — `BgfxRenderer::DrawInstancedPrimitivesEx()` is a substantial implementation, not a stub. There is no standalone `modules/renderers/bgfx/examples/*instanced*.cpp` binary, but concluding “untested” from that filename search would now be wrong. The shared graphics suite contains BGFX-specific pixel, cache-cardinality, submission-count, exact native-range, transient-allocation, wireframe, and `InstanceFrequency` cases. These prove considerably more than a no-throw smoke test while also exposing a profile-dependent defect described below.

The shared transport is no longer the old pointer-only bridge. A game marks a bound `VertexBuffer` as per-instance data by giving its `VertexBufferBinding` an `InstanceFrequency` greater than zero when calling `SetVertexBuffers()`. `GraphicsDevice` composes every semantically contributing binding into the same immutable `GpuDrawParams::vertexStreams` array used by ordinary draws, preserving slot, renderer pointer, declaration stride, vertex offset, frequency, and element count by value. The classic BGFX-supported shape is exactly one stream of each input rate:

```
1  FillVertexStreamBindings(p, /*foldedOffset=*/0,
2                          /*allowLegacyEmptyDeclarationFallback=*/
3                          false);
4  ValidateVertexStreamRanges(p, baseVertex + minVertexIndex,
5                             numVertices,
6                             "numVertices", std::to_string(numVertices));
7  ValidateInstanceStreamRanges(p, instanceCount);
8  ValidateVertexStreamCapability(p);
```

BGFX reads the lowest-slot positive-frequency entry as the instance stream and explicitly rejects more than one per-vertex or more than one per-instance stream. This is why its `MultiStreamVertexInput` capability remains false: a declaration cannot be split across two geometry buffers and two instance buffers cannot contribute independently. The ordinary one-plus-one instancing shape does not require that capability and remains valid.

Both binding offsets are now real. The geometry stream’s `VertexOffset` is added to `baseVertex` through `bgfx::setVertexBuffer`’s start-vertex term, while the instance stream begins copying at its own `VertexOffset`. BGFX exposes no native instance divisor in `setInstanceDataBuffer`; CNA therefore expands frequency grouping when filling the transient instance buffer: destination instance i copies source record $VertexOffset + \lfloor i / InstanceFrequency \rfloor$. Frequency one stays a single bulk copy. Shared validation first proves that every geometry stream covers the declared vertex window and that each instance stream contains $1 + \lfloor (n - 1) / f \rfloor$ source records, so an undersized binding throws before native submission instead of reading uninitialized transient memory. Chapter 19, §19.3.2, compares this transport and its proof boundary across every renderer.

Two silent exits remain, but their current mechanisms are narrower than the former pointer-only account. If no semantically contributing binding has a positive `InstanceFrequency`, `FirstInstanceStream(params)` returns null and the renderer draws nothing. A transient-capacity shortage reported by `bgfx::getAvailInstanceDataBuffer()` also skips the draw. Invalid vertex or instance ranges no longer share that outcome: they throw during the validation described above. A game whose large instanced batch disappears on BGFX should therefore check that one binding really is per-instance and then consider transient instance-buffer pressure; malformed ranges should already have produced a diagnostic.

The remaining correctness fault is in the checked-in instancing shader, not this transport. `vs_instanced3d.sc` constructs the world matrix as `mat4(i_data0, i_data1, i_data2, i_data3)`. BGFX's GLSL profile treats those arguments as columns, but its HLSL/SPIR-V/Metal/WGSL profiles interpret the generated matrix constructor differently; BGFX supplies `mtxFromCols()` precisely to normalize this cross-profile difference. The dedicated test proves per-instance transforms on OpenGL/OpenGL ES and pins their failure on Vulkan. Thus "working" is justified only for the verified GLSL native profile today. Exact ranges, frequency expansion, and one-submit behavior remain proven independently of that shader-orientation bug; changing the source to `mtxFromCols()` and regenerating `bgfx_shaders.hpp` is the distinct remaining repair.

One further real detail is worth stating precisely because it is easy to assume otherwise: the draw call's own `world` matrix parameter is explicitly unused on this path (a commented-out parameter name in the real signature, `const Matrix& /*world*/`), because per-instance world transforms are expected to arrive already baked into each instance's own per-instance vertex data, not supplied once as a shared uniform the way a single, non-instanced draw's `World` property works — only `view` and `projection` are combined into the one `vp` uniform every instance in the batch shares.

24.11 Texture3D and TextureCube readback: copy first, then read

The shared renderer interfaces once left `Texture3D` and `TextureCube` `GetData()` as silent no-ops. The current copy-before-read route avoids placing a readback-only usage flag on ordinary sampled textures. BGFX documents `BGFX_TEXTURE_READ_BACK` as incompatible with a texture that is also a normal GPU resource, so marking the game texture itself readback-capable would undermine the very texture that an effect needs to sample.

Instead, each `GetData()` call creates a short-lived transfer texture exactly matching the requested region. For a cube source it is a 2D RGBA8 destination; `bgfx::blit()` selects the requested cube face through the source Z coordinate. For a volume source it is a matching 3D destination, so the requested *x, y, z, w, h, depth* sub-volume remains explicit on both sides of the copy. A temporary destination is marked for a blit destination and for readback; the ordinary source texture keeps its normal sampled form.

This is not synchronous memory access. `bgfx::readTexture()` returns the frame at which its bytes become available. The shared `AdvanceFramesUntil()` helper advances up to four BGFX frames before the transfer texture is destroyed, so the caller's buffer is not handed back before the asynchronous copy completes. If a selected BGFX renderer cannot create the temporary transfer texture, CNA now logs the missing `BGFX_CAPS_TEXTURE_BLIT/READ_BACK` capability instead of reverting to the old indistinguishable no-op. The project's Xvfb/llvmpipe OpenGL environment was checked to support both capabilities before this path was adopted.

The same change also finally made mip allocation testable: `mipMap` is now passed through to

BGFX's `createTexture3D` and `createTextureCube` calls instead of being silently hardcoded false. Four already-renderer-agnostic programs are registered for BGFX rather than being reimplemented as cosmetic duplicates: a distinct-slice volume round trip, a non-origin cube partial rectangle, a three-level volume mip round trip, and a per-face cube mip round trip. Together they distinguish a correct transfer from reading level zero, the wrong cube face, or a CPU buffer left untouched by the former interface default.

24.12 Viewport uses ordered views; scissor remains per draw

Both public setters first become stored BGFX-side values, but that similarity ends at submission. An enabled, nonzero scissor is reissued through `bgfx::setScissor()` before every 3D submit and every `SpriteBatch` submit, because `bgfx` scissor is per draw and otherwise resets to “none.” The Boolean from `RasterizerState` is retained independently from the rectangle. `bgfx_scissor_test` proves both halves, then disables the test again to exclude a one-way/stuck-state implementation.

Viewport is view-level state rather than a draw bit, so CNA preserves changes by selecting an ordered view segment before each 3D draw or `SpriteBatch` batch. Consecutive work with the same target, viewport, and — for sprites — Begin transform reuses the segment; a changed viewport or sprite transform allocates the next view, binds the same target with `CLEAR_NONE`, and keeps execution order through the frame's explicit view-order table. This works for the backbuffer and render targets. `SpriteBatch` keeps the view rectangle full-sized so a public clear still covers the target, then folds the captured viewport origin into its orthographic matrix; 3D sets the sub-rectangle directly. The public min/max depth values remain discarded as an explicit `bgfx` limitation.

The original single-subregion proof now sits beside `Bgfx_Deferred_Visport`, which queues several viewport changes without a flush, and the transform-specific regression that separates two Begin matrices even when the viewport is unchanged. The analogous deferred scissor oracle proves that each draw still receives its own captured rectangle and enable bit. Section 19.4.3 records that broader current scope.

24.13 Current qualification

The recorded closure predates the readback and mip work above. The still-deliberate `SlopeScaleDepthBias` omission, the non-GLSL instancing matrix-constructor defect, and the two environment-bound render-target checks remain important limitations, while later feature work should be evaluated from the current renderer plan and its registered tests rather than from the historical campaign summary.

24.14 MAGNUM: a wrapper with no offline shader generator

MAGNUM first searches for an installed Magnum and otherwise fetches pinned Corrade and Magnum revisions. Emscripten is rejected. Its default desktop path is GLX, with EGL an explicit CMake option rather than a runtime driver choice.

Its shader boundary is unique in this cluster: GLSL lives as C++ string literals and is compiled by `Magnum::GL::Shader` at runtime. There is no checked-in generated shader header and no staleness checker to run. That removes an offline tool dependency while moving syntax and driver compatibility failures into device startup.

Extended draws and 2D targets are native. MRT capability follows the discovered maximum

target count rather than an unconditional constant, anisotropy follows the runtime sampler limit, and occlusion queries are real. This is the desirable capability shape: report the device actually created, not the feature ceiling of the wrapper library in the abstract.

24.15 LLGL: a broad library behind a deliberately narrower CNA path

LLGL is fetched at a pinned release unless `CNA_LLGL_ROOT` supplies it. The renderer has its own module-selection helper, but the available set is platform-bounded: on Linux CNA presently offers OpenGL, and an explicit request for Vulkan throws instead of quietly selecting something else.

Two shader forms are checked in. Vulkan-style sources are compiled to SPIR-V, while OpenGL-specific sources are preserved as GLSL; both are assembled into a generated header. The generator has a check mode, so CI can distinguish a source change from a forgotten regeneration.

The current renderer owns native extended draws, 2D targets, MRT, and occlusion queries. Its documentation nevertheless describes a narrower boundary than upstream LLGL. Library support is not automatically CNA support: each resource and state path still requires an adapter, a capability answer, and observable evidence through the selected module.

24.16 DILIGENT: one HLSL program, several device types

DILIGENT fetches DiligentCore and resolves a device at runtime. The preference order is D3D12, Vulkan, D3D11, then OpenGL, filtered by what the build contains; on Linux the real candidate list is Vulkan followed by OpenGL. Tests that exercise only the first successful candidate therefore do not prove the fallback path.

Rather than shipping one shader package per device type, the renderer stores a single HLSL source and asks DiligentCore to cross-compile it at runtime. The row-major pragma is part of that contract because the HLSL-to-GLSL converter recognizes the pragma form. A shader edit can thus affect several native backends without generating separate checked-in blobs, but each translation path still needs its own execution evidence.

Native extended draws, 2D targets, and MRT are implemented. Occlusion-query support follows device features. Of the wrappers in this chapter, Diligent is closest to broad CNA parity, yet its runtime fallback order creates the sharpest test-design warning: one test binary registered twice is useful only if the device override genuinely forces two different paths.

24.17 Wrapper-selection checklist

For every reproduction, capture the CNA identity, underlying native backend, shader delivery path, and whether fallback was automatic or forced. Then verify the specific feature through pixels or query values. Construction proves that two abstraction layers agreed on startup; it does not prove that state and resources survived both translations.

Chapter 25

Abstraction Layers II: Sokol, Wicked, and FNA3D

These three renderers place another graphics system between CNA and the native API, but that is nearly all they share. Sokol chooses its native API at compile time; Wicked is hardwired to its Vulkan device in CNA; FNA3D chooses a driver at runtime and consumes precompiled XNA effect programs rather than CNA-authored shaders.

Identity	Native-backend axis	Shader delivery	RT / MRT / query
SOKOL	compile-time CNA_SOKOL_API	GLSL through offline sokol-shdc	yes / yes / GL build only
WICKED	hardcoded Vulkan de- vice	HLSL compiled by Wicked's DXC wrapper at startup	yes / yes / yes
FNA3D	runtime FNA3D driver selection	pinned D3D9 Effect Framework bytecode via MojoShader	yes / yes / yes

All three own native extended primitive paths; none inherits the base colored downgrade. That common endpoint hides radically different build and deployment risks.

25.1 SOKOL: two selectors, only one mature cross-product

Selecting SOKOL chooses the CNA renderer. A second CMake value, CNA_SOKOL_API, chooses GLCORE, GLES3, DIRECTX11, METAL, or WGPU before compilation. The build warns for anything other than GLCORE because the implemented surface is functionally verified as a single-backend path, not as five interchangeable products.

The shader source is processed offline by `sokol-shdc`. Its generated C++ header contains conditional blocks for the native APIs and has a check mode for stale output. The generator binary is a sizeable prebuilt tool and is not vendored, so shader maintenance has a distinct developer dependency even though consuming an already-generated header does not.

25.1.1 Capabilities leak the compile-time API

2D targets and MRT are implemented. Occlusion queries and custom effects are reported only when the build exposes the renderer's GL readback/extension path. That conditional answer is important: the public identity alone is insufficient to predict these features.

The current Texture3D path illustrates a different danger. Capability reporting says true because the object has exact per-mip CPU voxel storage and real box-region `SetData/GetData`

transfers, while it deliberately owns no `sg_image` and no shader path samples it. The answer therefore proves CNA’s bounded resource-transfer contract, not a GPU-samplable volume texture. Code that only constructs and round-trips bytes must not be promoted into evidence that a shader sampled the volume; for that stronger claim, resource evidence would have to cross a draw boundary that this renderer does not presently expose.

25.2 WICKED: an experimental Vulkan adapter

CNA fetches Wicked Engine and applies local patches for SDL3 platform integration, orderly device teardown, and staging-footprint behavior. This is not a transparent dependency: upgrading upstream means revalidating both the adapter and the patch set.

Although Wicked can support several graphics devices, CNA’s renderer constructs `GraphicsDevice_Vulkan` unconditionally. D3D12 is not a fallback. HLSL entry points are compiled at device creation by Wicked’s DXC wrapper with Shader Model 6 as the minimum.

That runtime compilation creates a packaging trap: `libdxcompiler.so` or `dxcompiler.dll` must be discoverable when the executable starts. A successful link does not prove that the shader compiler will be present on a clean target machine.

Wicked implements native extended draws, 2D targets, MRT, and occlusion queries. Its experimental label describes evidence maturity, not a base-class stub. The useful next proof is representative pixel behavior under the packaged runtime, including an environment where DXC is found exactly as an end-user installation would find it.

25.3 FNA3D: the XNA shader-program path

FNA3D is architecturally special. CNA creates an FNA3D device and the library chooses among its available SDL GPU, D3D11, and OpenGL drivers; the `FNA3D_FORCE_DRIVER` hint can constrain that choice. The repository’s declared verification scope currently pins the OpenGL driver, so other driver paths remain separate claims.

CNA authors no shaders for this renderer. Six checked-in `.fxb` files are compiled D3D9 Effect Framework programs taken from a pinned FNA revision. FNA3D accepts effects only through that compiled representation and executes them through MojoShader. This makes the renderer CNA’s closest path to the shader programs XNA content itself expects, rather than a fresh reimplementations of those effects in GLSL, HLSL, WGSL, or MSL.

The effect blobs are versioned assets, not outputs fetched transitively with FNA3D. Their provenance and pinned source revision belong in any regeneration review. Rebuilding the C++ dependency alone cannot refresh them.

25.3.1 A strong oracle with a bounded public surface

Native extended draws, 2D targets, MRT, and occlusion queries are implemented. Custom CNA effects are not: the only shader entry point accepts compiled effect bytecode. Instancing is also reported false because the stock shader/input contract does not expose the required instance stream, even though multi-stream vertex input itself is available.

Those refusals are consistent rather than contradictory. A library may accept multiple vertex streams without the selected stock effect understanding per-instance data; likewise, it may

execute real XNA effects without accepting arbitrary source strings through CNA's custom-effect abstraction.

25.3.2 Why this path is a useful comparison oracle

When a CNA-authored stock shader and FNA3D's compiled XNA program render the same scene, pixel agreement compares independent shader implementations. Disagreement does not automatically make FNA3D correct, but it narrows investigation to semantics that one translation likely lost: matrix convention, fog, alpha test, lighting, texture coordinates, or blend state.

The repository's corpus comparison script deliberately forces the OpenGL driver and tallies results as measurements rather than treating them as a universal gate. This is the right scope: a fixed driver makes comparisons reproducible, while the label still records that FNA3D's other runtime drivers were not exercised by that run.

25.4 Six shader strategies across the wrapper chapters

Together with Chapter 24, the abstraction renderers demonstrate that a wrapper does not standardize shader delivery:

1. BGFX consumes its own platform shader assets;
2. Magnum compiles raw GLSL strings at runtime;
3. LLGL embeds generated SPIR-V and GLSL variants;
4. Diligent cross-compiles one HLSL source at runtime;
5. Sokol uses an offline multi-backend generator;
6. Wicked invokes DXC at device creation;
7. FNA3D executes pinned compiled XNA effect programs through MojoShader.

The list has seven entries because the two-chapter cluster contains seven renderers, and no two delivery contracts are quite identical. Build reproducibility, package contents, runtime compiler availability, and shader-source ownership must therefore be audited separately.

25.5 Selection and verification

Choose Sokol when its compile-time native axis and generated-shader workflow fit the product. Choose Wicked for deliberate experimentation with its patched Vulkan path. Choose FNA3D when running the XNA effect representation and its supported runtime driver is the central value.

For every one, record both layers of selection and test a representative draw, target switch, MRT write, and query result where claimed. A wrapper can make device creation portable; it cannot make unobserved renderer behavior portable by declaration.

Chapter 26

DirectX 1–3 and free-direct

Four identities meet here, but only three are Microsoft-era rungs. **DIRECTX1**, **DIRECTX2**, and **DIRECTX3** use historical **DirectDraw**/**Direct3D** interfaces on Windows. **FREEDIRECT** is a portable 2D renderer over the separate **free-direct** project and is not an implementation of the **DIRECTX3** selector.

Identity	Native surface	3D route	RT / MRT / query
DIRECTX1	DirectDraw v1	unavailable; base fall-back rejects	off-screen surface / no / no
DIRECTX2	DirectDraw and Direct3D v2	fixed-function DrawPrimitive	yes / no / no
DIRECTX3	DirectDraw v2 and Direct3D v2	fixed-function DrawPrimitive	yes / no / no
FREEDIRECT	free-direct surface API	unavailable; base fall-back rejects	yes / no / no

26.1 **DIRECTX1: before Direct3D existed**

DirectX 1 shipped **DirectDraw** but no **Direct3D**. CNA’s renderer is consequently a 2D implementation, not an unfinished 3D renderer. Its off-screen surfaces provide **RenderTarget2D**; **MRT** and occlusion queries do not exist at this API level.

The extended primitive methods are inherited. Their base route reaches colored rendering, and this 2D family rejects that request. This is the correct visible failure for a permanent API ceiling. Reporting successful effect-aware 3D would be more misleading than throwing.

Build and execution are Windows-shaped. The repository can cross-compile with mingw-w64 and exercise the binary under Wine’s **DirectDraw** implementation, but that proves a compatibility layer as well as CNA. Native historical drivers remain a different environment claim.

26.2 **DIRECTX2: fixed function without execute buffers**

Direct3D 2 is often associated with execute buffers, yet CNA submits immediate **DrawPrimitive** calls. A spike established that Wine’s implementation accepts the route and provides the transform, lighting, texture, depth, and raster behavior needed by the bounded renderer.

Both extended draw entry points are native fixed-function adapters. CNA’s modern draw packet is reduced to texture, matrix, lighting, fog, alpha, and state concepts available in the old pipeline. Custom programmable effects are outside the API’s design. 2D targets work; multiple attachments and queries do not.

26.3 DIRECTX3: a small port, not a renamed free-direct

This renderer uses the DirectDraw 2 and Direct3D 2 interface generation available with the DirectX 3 SDK surface. It landed under a temporary `DX30` name because the old `DIRECTX3` selector was still occupied by what is now `FREEDIRECT`. The identity cleanup separated historical Microsoft DirectX from the portable sibling project.

The measured source delta from the DirectX 2 port was small, roughly 3.7 percent, but that number describes lineage rather than semantic equivalence. Interface acquisition, presentation, and platform behavior still belong to a separate family and build target. Its draw and target feature shape matches the preceding rung: native fixed-function draws, one 2D target, no MRT, and no occlusion queries.

26.4 Why the first three rungs matter

The sequence isolates what the API itself added. DirectX 1 can compose surfaces but cannot express 3D. DirectX 2 introduces an immediate fixed-function route sufficient for CNA’s bounded 3D mapping. DirectX 3 changes the surrounding SDK/interface generation without turning that fixed pipeline into a programmable one. The same CNA call surface therefore reveals a historical API boundary instead of flattening every old renderer into one generic legacy label.

26.5 FREEDIRECT: a separate portable 2D branch

`FREEDIRECT` is CNA’s second 2D-only renderer, and its odd one out architecturally: it does not use SDL3’s 2D API at all, instead fronting `IDirectDraw` / `IDirectDrawSurface`-shaped calls against `free-direct`, the narrow DirectDraw reimplement introduced in Chapter 2 and covered as its own project later in this book. This chapter covers only what is specific to CNA’s own renderer layer on top of it; this book’s `free-direct` chapter covers the library itself.

26.6 An unanticipated architectural constraint

`free-direct`’s `IDirectDrawSurface::Lock()` never exposes a writable pointer for the *primary* surface—only for offscreen ones. Instead of treating the primary surface as a render target directly, this renderer owns an internal, always-lockable “shadow backbuffer”: every `Clear()` and every `SpriteBatch` draw targets that shadow surface, and `Present()` is a single identity `Blit()` copying it onto the primary surface. This design makes the renderer possible.

26.7 A capability beyond `SDL_RENDERER`

`TextureAddressMode::Wrap` and `Mirror` — the two features Chapter 30 named as formally `BLOCKED` on `SDL_RENDERER` — have implemented sampling paths here. This renderer’s CPU compositor already samples per-source-pixel for any non-identity draw, so wrap/mirror addressing costs nothing extra to add. `FREEDIRECT` also implements separate `Opaque`, `AlphaBlend`, `NonPremultiplied`, and `Additive` formulas, in contrast to the `SOFTWARE` renderer’s single collapsed baseline formula.

26.7.1 Address modes beyond SDL_RENDERER

The fixture from Chapter 30 draws a two-texel red/green texture through a four-texel source rectangle onto four destination pixels. With point filtering, each destination sample is unambiguous. FREEDIRECT reads back all four red-channel values for each address mode:

```

1 Texture2D tex(dev, 2, 1);
2 std::vector<Color> px = {Color(255, 0, 0, 255), Color(0, 255, 0,
   255)}; // texel 0 = red, 1 = green
3
4 SamplerState point;
5 point.setFilterProperty(TextureFilter::Point);
6 point.setAddressUPROPERTY(TextureAddressMode::Wrap); // or Mirror,
   or Clamp
7 point.setAddressVProperty(TextureAddressMode::Wrap);
8
9 spriteBatch_>Begin(SpriteSortMode::Deferred, BlendState::AlphaBlend
   , &point, nullptr, nullptr);
10 spriteBatch_>Draw(tex, Rectangle(0, 0, 4, 1), Rectangle(0, 0, 4, 1)
   , Color::White);
11 spriteBatch_>End();
12 // Read back R at destination x = 0,1,2,3:

```

Wrap yields (255, 0, 255, 0), or red/green twice. Mirror yields (255, 0, 0, 255), and Clamp yields (255, 0, 0, 0). SDL Renderer provides only the clamp-shaped result through this draw path. FREEDIRECT can implement the three index rules inside its existing per-source-pixel CPU loop; SDL's texture-blit route exposes no equivalent sampler control.

26.8 Corrected FREEDIRECT defects

The renderer audit corrected several independent faults:

- `Clear()` wrote alpha 255 through its color-fill route; locked-pixel writes now preserve all four requested channels.
- the general CPU blend path omitted source-alpha multiplication;
- `CreateOcclusionQuery()` threw instead of using the project's null-safe query;
- blend-preset detection compared factors but ignored `BlendFunction`, so a custom subtract state could be mistaken for a preset; and
- display-free tests were unnecessarily registered as requiring X11.

The current regressions cover the behavior, while historical test-count bookkeeping is not used as a capability claim.

26.9 RenderTarget2D: the same shadow-backbuffer trick

`RenderTarget2D` reuses the shadow-backbuffer design once per target. The private render-target renderer owns an always-lockable offscreen surface, defined (like `FreeDirectTextureRenderer`) entirely inside `FreeDirectRenderer.cpp` so `<ddraw.h>` never leaks into a header. Binding one is a pure redirect: `Clear()` and `ReadBackbuffer()` both route through an internal `Impl::ActiveSurface()` indirection that resolves to whichever surface is currently bound, while `Present()` always targets the shadow backbuffer from §26.6 regardless of what is bound

at the time — an intentional asymmetry, since only the shadow backbuffer is the only surface copied to the primary.

`freedirect_texture_rendertarget_test.cpp` verifies this bind-redirect concretely rather than by code inspection alone, using two distinct, deliberately different clear colors so a readback that returned either the wrong surface’s color or a stale value would fail visibly:

```

1  dev.Clear(Color(20, 40, 60, 255));           // shadow backbuffer's
    own color, established first
2
3  auto rt = std::make_unique<RenderTarget2D>(dev, 8, 8);
4  dev.SetRenderTarget(rt.get());
5  dev.Clear(Color(210, 30, 90, 255));         // the render target's
    own, different color
6  // GetBackBufferData() now reads back (210,30,90) -- the render
    target's surface, not the shadow's.
7
8  dev.SetRenderTarget(nullptr);
9  // GetBackBufferData() over the same region now reads back
    (20,40,60) again --
10 // unbinding really redirects Clear()/ReadBackbuffer() back to Impl
    ::backBuffer.

```

Two further checks round out the same test. `RenderTargetUsage::DiscardContents` auto-clearing to black on rebind needed *zero* FREEDIRECT-specific code at all — it falls straight out of shared `GraphicsDevice.cpp` logic once bind/Clear/read were wired correctly, the same “comes for free” shape Chapter 14 documents for other renderers. This is also a CNA/FNA identity difference: CNA repeats that clear on an identical currently-bound target, while FNA treats the redundant call as a no-op. The persistent `DirectDraw` surface makes `Preserve` and `Platform` retain color in source, but the test proves only the Discard-to-black case; presentation usage is ignored. `SetRenderTargets()` with two or more bindings throws, honestly: `IDirectDrawSurface` has exactly one active surface at a time, the identical single-active-surface conclusion Chapter 30 reached for `SDL_RENDERER` for an unrelated reason (that renderer’s `SDL2` render target API, not `DirectDraw`’s own surface model) — two structurally different renderers landing on the same MRT answer for two different underlying reasons.

The same test confirms the size boundary: a 5000×5000 `Texture2D` construction throws. `free-direct::CreateSurface` enforces a fixed 4096×4096 cap, and the renderer propagates that exception.

26.10 Public depth/stencil clear

The ordinary colour hook locks and fills the complete active surface with all four requested channels, and target-redirection tests prove that “active” changes between the shadow backbuffer and a `RenderTarget2D`. Depth and stencil follow two different layers. `GraphicsDevice` sees `SupportsDepthStencil()==false`, strips both flags, and returns silently when no target bit remains — the same depthless-target rule current FNA uses. `freedirect_no3d_test.cpp` separately invokes all six renderer combination hooks and proves their `ThrowNo3D` errors precisely because those hooks are unreachable through the public mask. Treating either result as evidence for the other would conflate shared and renderer contracts; §19.4.2 keeps them separate.

26.11 Known limitations at the pin

No 3D pipeline exists, matching **free-direct**'s own explicit "Direct3D not implemented" stance; 8-bit palette surfaces and **GetDC** / **SetPalette**-style GDI-adjacent calls are unsupported, since XNA itself has no palette-texture concept to begin with; mip levels above zero throw, since **IDirectDrawSurface** has no native mip chain; and **SetPresentationMode()** is honestly downgraded from an earlier full-pass mark to a documented partial: it stores the requested presentation mode but never changes the actual physical output scale and always presents in letterbox. Closing this limitation requires a change in **free-direct**, not only CNA.

The same stored-versus-applied distinction affects two ordinary XNA properties: **Dx3Graphics.Renderer** overrides neither **SetViewport()** nor **SetScissorRect()**, and it has no rasterizer-state hook. Both values therefore round-trip through **GraphicsDevice** without changing the 2D **DirectDraw**/**SDL** output. The **SDL** renderer that **free-direct** owns privately does not rescue this path: CNA never routes these graphics-state setters into it (unlike the separate public input-discovery route below). There is no dedicated negative pixel test, so this is a source-proven permanent 2D boundary rather than an executed claim.

There is a related resize defect that is easier to miss because the new logical surfaces *are* created correctly. Once the first **Present()** has configured **free-direct**'s internal renderer, a later **SetVirtualResolution()** replaces CNA's primary and shadow surfaces but cannot reset the library's private **logicalPresentationSet_** flag. The next frame therefore uploads the new-sized primary texture into the *old* logical-presentation rectangle. CNA's input-transform methods do recompute their scale from the new logical dimensions, but the public input path does not call them: it discovers **free-direct**'s associated **SDL** renderer first, as the section below explains. That renderer retains the same old logical-presentation rectangle as physical output, so public input and presentation remain tied to the old coordinate canvas while the game's newly allocated surfaces use the new size. Closing that three-way mismatch needs **free-direct** to reapply logical presentation when the primary changes, not an unilateral CNA-side approximation.

DirectSound / **DirectPlay** networking are explicitly out of scope for this renderer by the project owner's own instruction — CNA's own audio and networking go through the ordinary renderer-agnostic paths from Chapter 45 and Chapter 51 of this book, not through **free-direct** at all.

26.12 The transform math is sound, but public input takes another route

The renderer implements both **TransformWindowToLogical()** and its inverse from the physical **SDL_Window** size. For a logical size (L_w, L_h) and physical window size (P_w, P_h) , they recompute the same centered-letterbox transform that **free-direct** uses:

$$s = \min(P_w/L_w, P_h/L_h), \quad o_x = (P_w - L_w s)/2, \quad o_y = (P_h - L_h s)/2.$$

A logical point maps to $(xs + o_x, ys + o_y)$; a mouse/window point maps back through $((x - o_x)/s, (y - o_y)/s)$. Thus the black bars introduced by the otherwise unavoidable letterbox mode are accounted for explicitly, rather than being accidentally treated as game pixels. The calculation intentionally queries the live physical window every time it is used: the primary/shadow surfaces retain the requested logical resolution, while window managers may choose a quite different actual window size.

Those two correct methods are nevertheless shadowed during normal public input. Both public callers first ask `SDL_GetRenderer(window)` for an associated renderer and use SDL’s own logical-coordinate conversion when one exists. Only a renderer-less window proceeds to the renderer registry. `free-direct` creates an SDL renderer on this same window during `SetCooperativeLevel()`, so SDL’s first tier wins even though `Dx3GraphicsRenderer::GetRendererInternal()` returns `nullptr`. CNA can still reach the renderer through the registry; this accessor does not return it.

This route has two time-dependent consequences. Before the first `Present()`, `free-direct` has created the renderer but has not yet called `SDL_SetRenderLogicalPresentation`; public input therefore passes through at window scale. The first presentation installs the hardcoded letterbox mapping, after which SDL’s own conversion correctly follows the displayed output. A later logical resize hits the stale `logicalPresentationSet_` problem above: both output and public input retain the old SDL mapping, while the new game surfaces no longer share that coordinate size.

The dedicated `Dx3_LogicalTransform` test avoids a fragile fixed-size expectation for that reason. It asks the running window for its physical dimensions, requests a 64×64 logical canvas, then verifies four properties: logical-to-window-to-logical round trips preserve five points (corners and centre); logical (32,32) becomes the physical geometric centre; equal horizontal and vertical logical steps have equal physical length; and that measured length is exactly $\min(P_w/64, P_h/64)$. These checks prove both the centering and the absence of silent anisotropic stretch on the active display, including the environment where an attempted `SDL_SetWindowSize()` was not honoured by the window manager. They do *not* prove the public route above: the test obtains `FreeDirectRenderer&` and invokes both methods directly, bypassing `Mouse::SetPosition()`, `SdlInputBridge`, and the SDL-renderer-first branch. Chapter 19, §19.2.1, places that proof boundary in the full renderer matrix.

26.13 The fixed 32-bit surface decision

`FreeDirectRenderer.cpp` creates the primary, shadow backbuffer, render targets, and `Texture2D` surfaces at 32 bits per pixel. It has no palette/8-bit route. This makes two limitations in `free-direct`’s `docs/directdraw-limitations.md` unreachable from CNA:

The discarded bit-depth parameter. `IDirectDraw::SetDisplayMode`’s `dwBPP` parameter is silently discarded — `free-direct`’s own `CreateSurface` always allocates a 32bpp primary regardless of the requested depth. CNA’s `CreateSurfaces()` always requests 32, so its request and the library result agree.

The mixed-depth blit silent-no-op. `DirectDrawSurfaceImpl`’s `BlitFrom` method implements same-depth 8-on-8 and 32-on-32 copies; a mixed pair copies nothing. CNA creates only 32bpp surfaces, so every renderer-issued blit follows the 32-on-32 path.

XNA’s `SurfaceFormat` has no 8-bit indexed-color member, so this fixed direct-color choice fits CNA’s public texture model. Direct users of `free-direct` that request 8-bit surfaces or mixed-depth blits remain exposed to the underlying limitations.

`PresentationParameters` cannot change that decision. The `FREEDIRECT` factory reads the window, virtual size, and presentation mode, but discards `BackBufferFormat`, `DepthStencilFormat`, and `IsFullScreen`. Shared device code may still ask SDL to change the window, while `free-direct` remains at `DDSCD_NORMAL`; there is no exclusive `DirectDraw` transition. The primary and shadow buffers stay fixed 32-bpp, and no depth attachment

exists. Thus the public fields can round-trip while the native answer remains unchanged, exactly as the cross-renderer matrix in §19.2.5 records.

26.13.1 Why raw factors do not identify a preset

The blend-preset defect becomes visible with concrete factor values. `BlendState.cpp` constructs `BlendState::AlphaBlend` with two color factors: `ColorSourceBlend` is `Blend::One`, and `ColorDestinationBlend` is `Blend::InverseSourceAlpha`. A hypothetical custom state sharing exactly those two factors but pairing them with `BlendFunction::Subtract` instead of the preset's own (default) `Add` computes a different result whenever the destination pixel is not fully black: $src + dst \times (1 - srcA)$ versus $src - dst \times (1 - srcA)$. They coincide only when the destination term is zero, as it is for `BlendState::Opaque`; `AlphaBlend`'s nonzero destination factor exposes the mismatch over existing color. A fixture that draws only onto cleared black can therefore miss it. Preset matching now compares both `BlendFunction` fields as well as the raw factors.

At the pin, the renderer plan records no open `BLOCKED` decisions. That plan status does not broaden the scoped execution evidence above.

Chapter 27

The Retro Ladder: DirectX 5–8 and Glide

The second retro chapter follows four successive Direct3D interface generations and one contemporary rival. Every family maps CNA’s modern contract onto fixed-function hardware concepts; none has a programmable shader stage in its declared scope.

Identity	API shape	Extended draws	RT / MRT / query
DIRECTX5	DirectDraw 4 + Direct3D 3	native fixed function	yes / no / no
DIRECTX6	same COM generation, real stencil	native fixed function	yes / no / no
DIRECTX7	flattened Direct3D 7 device model	native fixed function	yes / no / no
DIRECTX8	Direct3D 8 device API	native fixed function	yes / no / no
GLIDE	Glide 3 runtime ABI	native fixed function	no / no / no

There is no **DIRECTX4** selector because DirectX 4 was not a public release. The jump in names is historical, not a missing CNA family.

27.1 DIRECTX5: the stable immediate-mode baseline

DIRECTX5 moves to DirectDraw 4 and Direct3D 3, where flexible vertex formats and `DrawPrimitive` form the baseline. Execute buffers are fully absent from CNA’s path. The earlier spike result therefore becomes a repeatable port strategy rather than a one-off experiment.

The renderer translates effect-aware draws into fixed-function matrices, texture stages, lighting, fog, alpha test, blend, depth, and raster state. A native override does not mean arbitrary effects are possible; it means supported stock semantics are reduced intentionally inside the family instead of being dropped by the interface default. 2D targets work, while MRT and occlusion queries remain outside the era’s contract.

27.2 DIRECTX6: a small delta with stencil support

The DirectX 6 port retained the same broad COM interface shape as DirectX 5. The measured source delta was about six percent, an example of why close lineage is useful: shared behavior can be brought up first, then the changed surface can be audited directly.

One change is user-visible. This rung implements and tests real stencil behavior. Its `SupportsCapability(StencilBuffer)` answer nevertheless returns false. That is a capability-reporting defect, not evidence that the implementation lacks stencil. Callers that trust the bit will unnecessarily disable a working feature; callers that ignore it may use behavior the renderer demonstrably provides.

27.3 DIRECTX7: the flattened device model

Direct3D 7 removes the separate viewport-object model and folds more state into the device. The CNA contract above it remains stable, which makes this renderer a useful adapter study: the same viewport and state calls require a different ownership mapping even when the visible scene is unchanged.

Its feature outline remains native fixed-function draws, 2D targets, no MRT, and no occlusion queries. Real stencil exists, while the same false capability answer found in DirectX 6 persists here.

27.4 DIRECTX8: a new device generation, deliberately no Shader Model 1

Direct3D 8 is a much more recognizable device API, but CNA intentionally stops at its fixed-function surface. Shader Model 1 assembly is too narrow for realistic XNA effect content and would create a misleading partial programmable promise. The renderer therefore keeps the same honest semantic ceiling as the preceding rungs.

The build has a distinctive dependency. A normal 64-bit mingw-w64 installation does not provide the required D3D8 import library; CNA expects the D8VK/DXVK import path and refuses configuration when it is absent. Runtime evidence uses a dedicated Wine prefix and can force llvmpipe through the DXVK device filter for reproducibility.

2D targets are implemented; MRT and queries are not. Stencil is implemented but its capability answer is false, completing the same defect across DirectX 6, 7, and 8.

For all four DirectX identities, a passing Wine run proves CNA plus Wine or DXVK. It does not retroactively prove every native driver shipped with the historical SDK. The compatibility layer is part of the recorded environment.

27.5 GLIDE: a contemporary fixed-function branch

GLIDE targets the Glide 3 ABI and is intentionally constrained to Windows and 32-bit builds. CNA does not vendor the runtime DLL; a user-supplied `glide3x.dll` is required. That packaging boundary exists partly because common compatibility implementations have licenses unsuitable for vendoring here.

The renderer owns a native fixed-function extended draw path, but it cannot create `RenderTarget2D`; the factory throws. MRT and occlusion queries are likewise absent. This makes Glide the only family in the retro table without even the single off-screen target that the DirectX rungs provide.

Its evidence boundary is sharper still. The repository contains a real example target, but deliberately does not register it with `cctest`; no automated job executes Glide rendering. Com-

pilation can validate headers and ABI shape, while pixels remain a manual claim requiring a suitable 32-bit runtime.

27.6 What the ladder teaches

The sequence is less about nostalgia than contract design:

- a single public draw packet can be reduced across several fixed pipelines without pretending that arbitrary shaders exist;
- measured port similarity accelerates implementation but cannot replace per-family capability and presentation evidence;
- capability bits can be less accurate than executable behavior, as the three stencil false negatives demonstrate;
- build, compatibility-layer, and runtime-DLL success are distinct proof tiers.

27.7 A practical retro verification pass

For each DirectX rung, exercise clear/present, one textured 2D draw, one depth-ordered 3D scene, target bind/unbind, and every state feature the family claims. On DirectX 6–8 add a stencil pixel test and record the known capability mismatch. Capture the Wine version, translation layer, prefix, and device name beside the result.

For Glide, keep compile evidence and execution evidence separate. A successful 32-bit link does not show that a compatible DLL loaded, and a loaded DLL does not show that a triangle reached the screen. Until the example is executed and its pixels observed in a documented environment, the honest status remains implemented ABI with unautomated rendering evidence.

Chapter 28

Modern Direct3D

Four CNA identities occupy the modern Direct3D line. Three broad implementations share one development loop; `DIRECTX10` is a deliberately narrower bridge between them: cross-compiled from this project’s own Debian machine using the same MinGW-w64 toolchain Chapter 4’s Windows cross-build section already introduced, then run and verified under Wine with GPU execution — `D3D9` and `D3D11` through `DXVK`, `D3D12` through a different translation layer, `vk3d-proton`, in its own separate Wine prefix. Native-Windows CI now covers `DIRECTX11` and `DIRECTX12`; local Wine evidence remains a distinct tier and `DIRECTX9`/`DIRECTX10` have no equivalent CI job.

Identity	Shader model	Extended route	RT / MRT / query
<code>DIRECTX9</code>	<code>D3D9</code> / XNA-native effects	native	yes / yes / yes
<code>DIRECTX10</code>	two embedded SM4 shaders	inherited downgrade	colored yes / yes / no
<code>DIRECTX11</code>	SM5 shared generated package	native	yes / yes / yes
<code>DIRECTX12</code>	SM5 plus PSO/root-signature caches	native	yes / yes / yes

28.1 D3D9: zero-tolerance XNA oracle comparisons

`D3D9` targets pixel-for-pixel agreement with XNA 4.0 on its measured corpus. It uses Microsoft’s vendored XNA stock-effect HLSL and compiles it through `d3dcompiler_47.dll`; it does not substitute a reimplemented shader package.

28.1.1 Oracle: XNA 4.0 under Wine

A dedicated tool, `tools/xna-oracle/`, runs XNA 4.0 under Wine using the GAC assemblies and the in-prefix `csc.exe`. It renders each declarative scene through XNA and CNA’s `D3D9` renderer, then compares the images at zero tolerance. Both sides use the same `DXVK` layer on the same GPU, isolating CNA behavior from differences between graphics stacks. The pinned corpus contains 39 scene files. Every retained comparison is an exact match under this Wine/`DXVK` configuration; the result does not cover paths absent from that corpus.

28.1.2 Current contract and evidence

Full device lifecycle via plain `Direct3DCreate9` (not `D3D9Ex`, a deliberate choice), with `DeviceLost` / `DeviceResetting` / `DeviceReset` handling; all five XNA stock effects, with

61 of 66 compiled shader variants byte-identical to Microsoft’s own shipped bytecode (the remaining five differ only by HLSL compiler version, and are separately compared through the oracle); an oracle- and mutation-verified `SpriteEffect.fx`, including the half-pixel offset Direct3D 9 is notorious for — caught before it could become a false-positive “closed” claim, because the offset’s absence is invisible on a 1×1 test texture and only shows up at a larger size; and `GraphicsProfile.Reach / .HiDef` enforcement through a D3DCAPS9 query — the only audited CNA renderer where the distinction reaches a native capability query, as Chapter 12 already noted. Render-target sampling and the `PreferPerPixelLighting`/specular gaps described in Chapter 15 were corrected. The specular repair produced zero-tolerance frame matches for four of five `PerPixelLighting` bytecode variants via the oracle. The fifth, an untextured vertex-lit variant, is unreachable at the pin because the required position-only vertex layout is missing on the measured stock-effect paths.

D3D9’s lost-device lifecycle is reachable through two framework-reserved keys: after ordinary input processing, non-repeated F9 enters `Lost` and raises `DeviceLost`; F10 performs the reset, bracketing it with `DeviceResetting` and `DeviceReset`. The smoke suite checks exact event counts, that rendering fails while lost, exact pixels after recovery, and release/recreation of default-pool resources. This strong deterministic debug path should not be confused with two other public extensions: D3D9’s `SetStringMarkerEXT()` throws, and `SetContextRecoveryEnabled()` changes the shared `Texture2D` shadow policy before its D3D9 hook throws. Section 19.5.1 compares all fourteen products in the inherited audit cohort and calls out that partial-state outcome.

28.1.3 Coverage limits

Every `SurfaceFormat` besides `Color` remains unsupported (a shared, cross-renderer limitation from Chapter 14, not a D3D9-specific one). `Texture3D` has no oracle-scene coverage, but it has a capability-gated sub-volume `SetData()/GetData()` byte round trip: D3D9_Smoke creates a $4 \times 4 \times 4$ volume only when `D3DCAPS9::MaxVolumeExtent` is nonzero, writes 32 distinct RGBA bytes into the off-origin $2 \times 2 \times 2$ box at (1, 1, 1), and requires an exact readback of that same box. The test proves D3D9’s `LockBox()` row- and slice-pitch copy in both directions; it does not prove an oracle rendering path or automatic mip generation. `OcclusionQuery` is not built for this renderer at all, the same gap D3D12 carries; and `SpriteSortMode.Immediate / .Texture` were confirmed not viable as deterministic oracle scenes at all — `Immediate` is not something a raster diff can observe, and `Texture` sorts by an implementation-defined `GetHashCode()`, which no two implementations are obligated to agree on. The caveat that applies to every result in this section: D3DCAPS9 itself is synthesized by DXVK on this dev loop, not reported by XNA-era Windows driver hardware — native-Windows hardware verification remains open, as stated above.

28.1.4 D3D9 render-target limits and evidence

The D3D9 smoke test establishes render-target storage and recovery. An 8×8 target with `Depth24Stencil8` is bound, cleared through the public device, and copied with D3D9’s required `GetRenderTargetData()`–system-memory-surface–`LockRect()` route; its own surface must contain the exact requested color. A cube-face target has the same proof for one selected face, followed by an independent back-buffer clear/readback after unbind.

MSAA is capability-gated by `IDirect3D9::CheckDeviceMultiSampleType()`, not assumed from the requested count. When 4x is available, a multisampled target is cleared and unbound; D3D9’s `StretchRect()` resolve must make its separate sampleable texture read exactly (200, 210, 220, 255). The assertion reads the resolved result, not the multisample surface. One

naming trap matters here: this is an internal device-capability check in the creation path, not `GraphicsDevice::SupportsCapability`. D3D9 never overrides that public method, so `MultiSampleAntiAliasing` still reports true even when `CheckDeviceMultiSampleType()` rejects every count above one. The same mismatch affects `AnisotropicFiltering`: sampler creation uses `D3DTEXF_ANISOTROPIC` and `D3DSAMP_MAXANISOTROPY`, but neither the public query nor the setter consults and clamps to `D3DCAPS9::MaxAnisotropy`.

Multiple render targets are bounded by `D3DCAPS9::NumSimultaneousRTs`: two targets receive one exact-color clear, unbinding restores the back buffer, and requesting one more target than the device reports throws rather than silently truncating. This proves D3D9's multi-attachment binding and clear semantics. It does not separately prove a shader with several fragment outputs, so the result should not be generalized into a G-buffer draw claim.

28.1.5 One oracle scene

The oracle's own scene format is a small, declarative text file, not a compiled asset or a piece of C++. The simplest scene in the corpus, `colored3d.scene`, fits in eleven lines:

```

1 width=256
2 height=256
3 profile=HiDef
4 clearcolor=100,149,237,255
5 vertexcolor=true
6 lighting=false
7 primitive=TriangleList
8 vertex=-0.6,-0.6,0,255,0,0,255
9 vertex=0.0,0.7,0,0,255,0,255
10 vertex=0.6,-0.6,0,0,0,255,255
```

With `vertexcolor=true` and `lighting=false`, `BasicEffect` draws one red/green/blue-cornered triangle under identity transforms over `Color.CornflowerBlue`, RGBA (100, 149, 237, 255). XNA and CNA render the scene independently. Matching corners establish vertex-color plumbing; matching interior pixels also exercise Gouraud interpolation. The more elaborate `fog_gradient_quad` (Chapter 22's own negative-FogEnd finding), uses the identical file format with a handful of additional `fog*` keys — the same declarative scheme scales from this eleven-line triangle through the corpus's 39 scenes.

28.1.6 Two defects exposed by the oracle

The first was a sort-mode Z-clipping defect: `zFarPlane=1` mapped a nonzero `SpriteBatch` layer depth outside Direct3D 9's own valid clip-space Z range, silently clipping away any sprite with a nonzero layer depth entirely — fixed by using `zFarPlane=-1` instead. The second was shared across renderers: `GraphicsProfile` never reached the device because `Game's GraphicsDevice` was eagerly default-constructed (hardcoded to `Reach`) before `GraphicsDeviceManager` even existed to apply a game's requested profile — meaning a game setting `GraphicsProfile = HiDef; ApplyChanges();` had no path to the live device at all. This is a shared, cross-renderer bug, only *discovered* while building D3D9's own tests, because D3D9 is the one renderer where the profile actually matters enough to notice.

28.1.7 Why the `zFarPlane` sign hid nonzero-depth sprites

The sign controls clip-space placement. `SpriteBatch's` orthographic projection maps a `layerDepth` value in `[0, 1]` (XNA's own documented range: 0 = front, 1 = back) onto Direct3D 9's own native clip-space Z range, which is `[0, 1]` — *not* the `[-1, 1]` range OpenGL-family

APIs use. Constructing that projection with `zFarPlane=1` produces a near/far mapping where any `layerDepth` strictly greater than what the incorrect matrix treats as its own valid far bound gets clipped by the GPU’s fixed-function Z-clip test without an error. The default `layerDepth=0` was unaffected, so the defect appeared only with nonzero depth ordering. The `zFarPlane=-1` matrix maps the full documented `[0,1]` `layerDepth` range maps inside Direct3D 9’s own valid clip volume instead of partly outside it.

28.2 Three D3D11 resource-lifetime groups

Window resize and device-removed recovery affect different resource sets. D3D11 separates them into three lifetime groups:

Device lifetime. `ID3D11Device`, `ID3D11DeviceContext`, the `IDXGIFactory2` chain, and a cached `allowTearingSupported_` capability flag — created once at startup, torn down only by full device-removed recovery.

Swap-chain lifetime. The `IDXGISwapChain1` object itself — also created once at startup (and recreated only alongside the device, on recovery), but a plain resize deliberately *reuses the same object* via `IDXGISwapChain1::ResizeBuffers(...)` rather than destroying and recalling `CreateSwapChainForHwnd`.

Window-size lifetime. The back-buffer `ID3D11RenderTargetView`, the depth-stencil texture and its `ID3D11DepthStencilView`, and the viewport — unbound and released *before* `ResizeBuffers`, then recreated *after* it, on every resize and on device-removed recovery alike.

Resize releases and recreates only the window-size group, with one `ResizeBuffers()` call on the existing swapchain. Device removal rebuilds all three groups. This division avoids both recreating the device on resize and omitting the swapchain from recovery.

All D3D11, DXGI, and D3D12 interfaces use `Microsoft::WRL::ComPtr<T>`. A MinGW-w64/Wine spike exercised device creation, `.As()`, `GetParent()`, and the `ReleaseAndGetAddressOf()` repopulation needed by resize before that ownership model was adopted.

28.3 DIRECTX10: a bounded MVP with one unsafe default

DIRECTX10 uses Wine’s D3D10/D3D10.1 forwarding path into DXVK’s `d3d10core` and DXGI. Its two embedded shaders provide colored 2D and 3D submission, render targets, and MRT, but it deliberately leaves effect creation, occlusion queries, `Texture3D`, `TextureCube`, and cube targets at their base null implementations.

The sharpest boundary is not a throw. The renderer does not override either effect-aware extended primitive method, so the base class calls its colored draw implementation after discarding `GpuDrawParams`. A `BasicEffect`-shaped call can therefore produce pixels while silently losing texture, lighting, fog, skinning, PBR, instancing, or custom-effect intent. This is the sole unconditional inherited colored downgrade among all 46 renderer families.

That behavior makes capability checks necessary but insufficient. `ThreeD` can be true because colored geometry is implemented while effect semantics remain absent. A porter should treat DIRECTX10 as an explicit MVP target and verify the exact draw vocabulary used by the game rather than infer parity from a successful triangle.

28.4 D3D11: native Windows implementation

D3D11 is CNA's first renderer built directly on Direct3D 11, verified in the same MinGW-w64-plus-Wine-plus-DXVK loop as D3D9. Its covered path includes an `ID3D11Device / ID3D11DeviceContext` pipeline: buffers, textures, and render targets (including MSAA and multiple render targets), state objects, all ten stock HLSL shader variants across the five stock effects, `SpriteBatch`, and — unique to this renderer and D3D12 — a runtime-`D3DCompile()`-backed `ShaderEffect` path for custom shaders, already introduced in Chapter 17. Pixel assertions use GPU readback, so they establish more than successful API return codes.

Device-removed recovery has *detection-only evidence*: the check is wired, but the Wine/DXVK campaign did not trigger device removal, so full recovery remains unobserved.

An early build exposed a static-link cycle when the linker could not resolve `Effect::Apply()`: the D3D11 `SpriteBatch` renderer calls back into `Effect::Apply()`, which lives in the main CNA library, which itself depends on the renderer). MinGW's single-pass archive resolution never revisited the already-searched `libCNA.a` to satisfy it. The fix was an explicit repeated `target_link_libraries(... CNA)` for the D3D11/D3D12 targets, using CMake's documented support for a repeated static-library dependency. The failure reproduced on the base revision and was not introduced by the D3D11 work.

28.4.1 The public presentation-mode call reaches D3D11 and D3D12, but neither scales yet

Chapter 6's `CNAEXT PresentationMode` is not lost before it reaches these renderers: the public manager/device forwarding path passes the selected enum to each renderer. But the final handlers are currently intentional no-ops. D3D11 explicitly discards `mode`; D3D12 has the same empty handler. They create and present their swap chain at the SDL window's pixel size, not a separately rendered logical surface which could be letterboxed, cropped, or stretched.

Their `SetVirtualResolution(width,height)` methods only retain the supplied integers. On D3D11, `GetViewportSize()` still queries the window's physical pixel size; on D3D12, the stored virtual dimensions are returned to the caller but do not create a scaling pass. Consequently setting `Letterbox`, `Overscan`, `Stretch`, `NativeBackBuffer`, or `FixedHeightDynamicWidth` produces no different presentation geometry on either native Direct3D renderer today. This is a renderer limitation, not an XNA-compatibility promise: the enum itself is a CNA extension, and there is no test that could honestly pixel-prove a mode-specific result until a logical render target plus final composite/scaling path exists.

28.4.2 One `PresentInterval::Two`, two opposite Direct3D mistakes

The three renderers do not share one presentation-interval policy. D3D9 preserves CNA's 0/1/2 conversion in `D3DPRESENT_PARAMETERS`: Immediate becomes `D3DPRESENT_INTERVAL_IMMEDIATE`, Two becomes `D3DPRESENT_INTERVAL_TWO`, and everything else becomes `D3DPRESENT_INTERVAL_ONE`. A runtime change marks presentation state dirty and immediately calls the same device-reset path used for a resize. This is the only CNA renderer in the Direct3D family whose source requests a literal native two-retrace interval.

It is not yet a safe implementation. Microsoft's D3D9 contract says interval Two must be present in `D3DCAPS9::PresentationIntervals`, and windowed mode supports only Default, Immediate, and One.¹ CNA checks neither condition. Its ordinary game window is windowed, so requesting Two can make `IDirect3DDevice9::Reset()` return failure. The renderer logs

¹<https://learn.microsoft.com/en-us/windows/win32/direct3d9/d3dpresent>

that result, restores its previous dimensions, leaves `presentationDirty_` true, and retries at each later `Present()`; the public `PresentationParameters` still reports Two. No dedicated test exercises this route, and DXVK's synthesized capability structure would not replace the still-needed real-Windows check anyway.

D3D11 and D3D12 make the opposite mistake. Both reduce the construction argument and every later setter call to `vsyncEnabled_ = interval > 0`; their `Present()` then passes only sync interval 1 or 0 to DXGI. DXGI itself accepts 1 through 4 and defines them as waiting for at least the corresponding number of vertical blanks,² so nothing in the native API forces CNA to collapse Two. The real windowed smoke paths prove that default presentation succeeds, but the D3D plans explicitly say the no-VSync/tearing branch is untested; no located test distinguishes sync interval 2 either.

The useful porting conclusion is deliberately narrow. D3D9 has the correct representation but incomplete validation and failure reporting; D3D11/12 have robust per-present selection but an unnecessarily Boolean representation. A successful reset or a stored `PresentInterval::Two` is not yet evidence of half-refresh pacing on any of the three. The cross-renderer table in §19.2.4 places these results beside the other eleven products in that audit cohort.

28.4.3 Only D3D9 consumes the presentation format/fullscreen hook

D3D9 likewise stands apart for the other three presentation fields. It maps the requested back-buffer enum to `D3DFORMAT` (with a required Color-specific A8B8G8R8-to-A8R8G8B8 display substitution), maps None/Depth16/Depth24/Depth24Stencil8 to no attachment, D16, D24X8, and D24S8 respectively, and writes `D3DPRESENT_PARAMETERS.Windowed = !IsFullScreen`. A later manager reset reaches `UpdatePresentationFormatEXT()` and immediately resets the native device. The smoke suite observes a real D24S8 surface after manager-driven setup, so this is more than a stored field.

D3D11 and D3D12 ignore all three renderer arguments. Their windowed swapchains are fixed R8G8B8A8 UNORM and their default depth attachments fixed D24S8, even when public state says Depth24 or None. Shared SDL code may still enlarge the containing window for fullscreen. Both `CreateSwapChainForHwnd` calls omit a fullscreen descriptor, and neither renderer calls DXGI `SetFullscreenState`; their `exclusiveFullscreen_` fields remain false. D3D12's `HeadlessEXT` route is a third case: no window means no swapchain and no default depth attachment at all. It can render only into explicitly-created targets.

These distinctions prevent two tempting overclaims. D3D9's enum mapping is not a guarantee that every mapped format is a legal display mode; failed native reset can leave the previous attachments active while public state has already changed. And an SDL fullscreen window on D3D11/12 is not DXGI exclusive fullscreen. The inherited fourteen-product comparison and test boundary are in §19.2.5.

28.4.4 Evidence ladder for D3D11 3D draws

The D3D11 3D checks form a progressively discriminating GPU-readback ladder. The first uses the stride-16 `VertexPositionColor` path: a known blue clear was read back, an NDC-space opaque-red triangle was then drawn through both indexed and non-indexed calls, and the same region had to become red. That proves the input layout, shaders, constant buffer, primitive topology, rasterization, and readback path together; a stale clear or an unbound shader cannot pass it accidentally.

²<https://learn.microsoft.com/en-us/windows/win32/api/dxgi/nf-dxgi-idxgiswapchain-present>

The next rung separated texture sampling from vertex-color modulation. A stride-20 textured draw sampled a known texel exactly, through both indexed forms, while the stride-24 variant multiplied a known vertex color through a white texture and read back that exact color. The stride-32 lighting path then exercises the per-light constant buffer: its unlit control is byte-exact, while its lit control must differ from both the unlit result and the clear color. That deliberately avoids pretending a hand-derived GPU floating-point result is more stable than it is, while still distinguishing execution of the lighting branch.

Later tests applied the same discipline to features whose happy paths look deceptively similar. Fog draws the same geometry with the flag off (exact vertex red) and at the fog end (exact fog green); an environment-map fixture constrains the reflection to the cube's -Z face; and a skinned fixture supplies an identity bone instead of an all-zero default matrix. These fixtures falsify different failure modes. At the pin, the D3D11 smoke executable contains 147 checks exercised through Wine and DXVK GPU execution. The result supports the specific paths named here; it is not a license to extrapolate that any arbitrary, untested D3D11 state combination must therefore be correct.

28.4.5 Viewport/scissor: three viewport paths, but only two scissor paths

D3D9 and D3D11 take the simplest possible immediate route. Their public setters call the corresponding native viewport and scissor setters; their rasterizer mapping separately controls scissor enable. Neither SpriteBatch implementation replaces those native rectangles, and both derive their 2D projection size from the currently-bound native viewport. D3D11 Smoke Check O reads the exact native viewport, depth range, and scissor back through the corresponding native getters. That is strong native-state evidence but still one layer short of a public behavioral pixel test because the check invokes renderer methods directly.

D3D9 implements both states, but its evidence label needed correction. The renderer plan now calls viewport and scissor “oracle-proven.” A direct recount/search of the current 39-scene `tools/xna-oracle/scenes` corpus finds cull-mode scenes but no viewport or scissor scene, and the D3D9 smoke program's resize check proves a full-size post-reset viewport, not a custom sub-region. The safest current claim is therefore native-source implementation plus broad D3D9 draw coverage, not a zero-tolerance XNA comparison for these two states.

D3D12 now splits the two states. Its `SetViewport()` override stores rectangle and depth range, and every ordinary, extended, instanced, and SpriteBatch command list obtains that value through `GetEffectiveViewportEXT()`. The getter clamps depth to $[0, 1]$ and falls back to the current full target only when no positive custom rectangle is active. SpriteBatch also uses the same effective width and height as its projection basis, so coordinates remain local to the sub-viewport rather than being transformed twice. A native off-screen GPU check distinguishes inside/outside pixels, two custom rectangles in successive draws, depth-range behavior, and the full-target reset on a target switch. Scissor remains the negative half: there is still no rectangle hook, `scissorTestEnable` is not part of the PSO, and every draw installs a full-target `D3D12_RECT`. The complete product comparison is §19.4.3.

28.4.6 Clear: exact native bits do not imply one shared public contract

D3D11 immediately clears every bound RTV and selects the native depth and stencil clear bits independently when a DSV exists. D3D12 mirrors that selection with synchronous command lists. Both have strong direct-renderer proof: stencil-plane bytes are read back from the GPU, changed to a second value, and depth is distinguished by whether one identical triangle passes. Those tests bypass `GraphicsDevice`'s public route, but the route now derives depth and stencil availability independently from the bound 2D/cube attachment instead of inferring stencil

from every non-None depth enum. D3D12 still requires a bound colour resource before its combination helper can record any command list; once one is bound, however, a requested depth/stencil aspect with no DSV is deliberately a legal no-op, matching D3D11.

D3D9’s native `IDirect3DDevice9::Clear` bits are likewise immediate and individually selected, but their final gate consults `depthStencilFormatOrdinal_`, the presentation format captured for the backbuffer. Binding a differently-formatted 2D or cube target never updates that field. A target can therefore lose a depth clear when the backbuffer has no depth, or receive a stencil bit selected because the backbuffer has one. The smoke test’s seven public combinations all use a D24S8 backbuffer and mostly check colour preservation, so they cannot reveal this mismatch. See §19.4.2 for the corresponding thirteen-product contrast within that audit cohort.

28.5 MRT finalization across all bound targets

An ordinary single `RenderTarget2D` owns a simple end-of-use rule. When it is unbound, its `ResolveAndGenerateMipsEXT()` helper asks D3D11 to resolve the multisampled draw attachment into the separate sampleable texture. If the target has a mip chain, it then calls `GenerateMips()` on the shader-resource view. Thus the texture a later sprite samples is the resolved, current image rather than the still-multisampled attachment or an old mip level.

That rule was once silently absent for an $N > 1$ MRT bind. The single-target pointer could not represent several targets, so the old `SetRenderTargets()` path bound all their RTVs with `OMSetRenderTargets()` but did not call any target’s finalization when the set was replaced. A game could clear or draw to two MSAA targets successfully, then sample stale single-sample resolve textures without an exception. The repair keeps a non-owning array plus a count for the active MRT set. Both `SetRenderTargets()` and `SetRenderTarget2D()` first run `FlushPendingMRTResolveEXT()`, which visits every stored target, invokes its `ResolveAndGenerateMipsEXT()`, clears the array entry, and only then binds the next target or restores the back buffer. This matters for all three transitions: MRT to back buffer, MRT directly to one target, and MRT to a different MRT set.

D3D11_Smoke makes the error observable. It binds two distinct 8×8 , 4x-MSAA targets, clears the shared MRT pass to (200, 30, 40, 255), unbinds it, and reads both targets’ *sampleable* textures directly. Both must contain that exact color. A repair that resolved only target zero, or that merely left the MSAA draw attachment valid, fails the second readback. A second case binds another two-target set, clears it to (5, 6, 7, 255), and switches straight to an unrelated single target rather than unbinding to null; its first prior target must still read the new value. This distinguishes all target transitions from a narrower null-unbind repair.

The shared helper also calls `GenerateMips()`, but the scope of its proof should remain honest. The suite separately reads an 8×8 single-target mipmapped render target’s 4×4 and 2×2 levels after a solid (200, 90, 10, 255) clear, proving that the normal unbind path creates non-garbage downstream levels. The *combination* of $N > 1$ MRT and `mipMap=true` uses the same helper and is therefore wired, but is not independently pixel-tested yet. That is a verification boundary, not a claim that the two mechanisms are magically proven together.

28.6 D3D12: shared foundations and split verification

D3D12 reuses D3D11’s HLSL/DXBC bytecode, format/state mappings, and constant-buffer layouts through `D3DCommon`. The implemented route includes a device, queues, descriptor heaps, command lists, fences, per-resource barriers, pipeline-state objects, root signatures,

per-slot samplers, buffers, 2D/cube/3D textures, render targets with MRT, mip generation and device-queried MSAA, occlusion queries, all ten stock shader variants, `SpriteBatch`, runtime-compiled sprite shaders, and device-removed recovery.

Evidence. The routine D3D12 CTest suite is entirely off-screen. A separate manual diagnostic, launched through Proton, establishes swapchain creation and `Present()` through a window. That manual result is not part of the routine automated suite because the Proton bootstrap is too heavy for this development loop.

28.6.1 Backbuffer readback is not part of the shared D3D11 foundation

The repeated phrase “GPU readback” in this chapter names several different mechanisms. D3D11 has a `ReadBackbuffer()` override: it always copies `backBufferTexture_` into a staging texture, honors `RowPitch`, and backs the public `GraphicsDevice::GetBackBufferData()` route with exact windowed clear/draw tests. It does not redirect to a currently bound render target.

D3D12 has no corresponding override. A public backbuffer call reaches `IGraphicsRenderer`’s default `runtime_error` on a windowed device, while `HeadlessEXT` has no swapchain to read in the first place. The D3D12 smoke suite’s many exact GPU pixel assertions call its working render-target-resource readback helpers directly; the shared public `SpriteBatch` example below likewise reads the off-screen render target, not `GetBackBufferData()`. Those results prove the named draw/resource path but cannot be quoted as evidence for a public backbuffer bridge that does not exist. The complete renderer matrix and FNA contrast are in §19.2.6.

28.6.2 Two frames in flight and blocking call sites

Unlike the driver-managed D3D11 model, D3D12 makes the command allocator lifetime explicit. CNA therefore creates exactly two `ID3D12CommandAllocator` objects (`kFramesInFlight = 2`), one reusable graphics command list, one shared `ID3D12Fence`, and one remembered fence value per allocator slot. The essential operation, `SignalAndWaitForFrameEXT(frameIndex)`, deliberately has a slightly counter-intuitive ordering: it signals a *new*, monotonically increasing value for the frame just submitted, then waits only for the *previous* value recorded for that same slot before allowing its allocator to be reused. Thus a call for slot 0 may submit value `v2` while waiting for older `v0`; it must not wait for `v2` itself, because doing so would erase the overlap the two slots exist to permit.

The smoke test invokes the primitive for slots 0, 1, and 0 again, verifies that `v0 < v1 < v2`, and, on the second use of slot 0, requires `GetCompletedValue()` to have reached `v0`. It intentionally does *not* assert that `v2` has completed on return: `ID3D12CommandQueue::Signal()` is asynchronous, so that would be a race, not a correctness condition. A separate explicit event wait establishes eventual completion. The test also places 48 64-MiB GPU copies ahead of the old fence and comparing that wait with an already-complete control wait; the loaded wait is measurably slower, establishing that `WaitForSingleObject()` supplies back-pressure.

The reusable two-slot primitive is directly tested, but the current ordinary clear/draw helper uses `ExecuteCommandListAndWaitEXT()`, its intentionally simpler synchronous sibling: it submits a closed list, signals a fresh fence value, and waits for *that* value before returning. `Present()` likewise uses that synchronous helper for its explicit back-buffer transition before calling `IDXGISwapChain3::Present()`. Consequently a “Two frames in flight” therefore describes tested infrastructure and the allocator-reuse rule; it does not claim that current ordinary CNA drawing overlaps two CPU frames. The off-screen test paths retain a deterministic readback boundary.

Render-target, depth-stencil, shader-resource, and sampler descriptors come from four separate heaps; their current allocators are fixed-capacity, monotonically advancing bump allocators, not a general free-list. A destroyed resource does not return its descriptor slot during the device’s lifetime, and exhaustion throws a named `std::runtime_error` instead of silently aliasing another resource’s view. The capacities in the live renderer — 64 RTV, 8 DSV, 64 CBV/SRV/UAV, and 16 sampler descriptors — are therefore engineering limits of this implementation, not Direct3D 12 limits. Destroying many transient resources can therefore exhaust a descriptor heap even when GPU memory has been released; callers cannot assume unlimited descriptor churn during one device lifetime.

28.6.3 External Wine/vkd3d architecture mismatch

Swap-chain creation under *plain* Wine (as opposed to a Proton-managed launch) crashes outright, reproduced twice with a full symbolized backtrace pointing to a null-pointer read inside Wine’s own `dxgi.dll`. The cause is external to CNA: a mismatch between Debian’s own system `dxgi.dll` and vkd3d-proton’s separately overridden `d3d12.dll` — two DLLs that were never meant to be paired this way. Once launched through Proton, which supplies the matched DLL pair expected by vkd3d-proton, swapchain creation and a ten-frame clear-and-present loop through a window succeed in two runs.

28.6.4 Explicitly corrected claims

Earlier D3D12 documentation listed runtime state objects, sixteen-slot sampler state, public 2D and cube render targets including MRT, `Texture3D`, and `TextureCube::GetData()` as open. They are implemented at the pin. The live tracking file remains the authority for revisions after this edition.

28.7 One public ShaderEffect API, two Direct3D contracts

Chapter 17 draws an important boundary: D3D11 and D3D12 compile custom HLSL at runtime, but their implementations are `SpriteBatch` custom shader facilities, not general replacements for XNA’s compiled `Effect`. The two renderers accept the same constructor strings, compile the same `main` entry points as `vs_5_0` and `ps_5_0`, and expose the same public setters. Under that shared surface, both require one exact 32-byte vertex:

Semantic	Byte offset	Required HLSL type
POSITION0	0	float2: pixel-space X and Y
TEXCOORD0	8	float2: texture U and V
COLOR0	16	float4: sprite tint

This is `SpriteBatch`’s own vertex, not an inferred convention. D3D11 creates a three-element `ID3D11InputLayout` from the compiled vertex blob; D3D12 embeds the same three elements in the custom effect’s pipeline-state object. A shader declaring `float3 POSITION`, a tangent, or a game-specific stride therefore cannot be made compatible by supplying a matching `VertexDeclaration`: unlike EasyGL’s separately tested general-3D path, neither Direct3D renderer reads `GpuDrawParams::customEffectRenderer` from its 3D draw dispatch.

28.7.1 The names are documentation; the byte offsets are the API

Both implementations store public uniform writes in the same 128-byte block. The caller’s `name` argument is deliberately ignored. What selects a value is which setter was called:

Bytes	Writer	Meaning
0–15	renderer only	<code>vpSize</code> ; <code>SpriteBatch</code> writes width and height before binding the effect.
16–79	<code>SetUniformMat4</code>	one 64-byte matrix, regardless of the supplied name.
80–95	<code>SetUniformVec2/3/4</code>	one overlapping vector slot; a later vector setter overwrites the components it supplies.
96–99	<code>SetUniformFloat/Int</code>	one scalar slot. The integer overload stores a floating-point conversion, not integer bits.
100–127	none	reserved padding in the current implementation.

There are two further consequences that the common public surface does not make obvious. `SetUniformFloatArray()` and `SetUniformVec2Array()` inherit `IEffectRenderer`'s no-op defaults on both renderers. So do all three `SetTexture()` renderer hooks. The sprite's primary resource still works because `SpriteBatch` itself binds its texture and sampler to `t0/s0`; a second 2D texture, cube map, or volume texture set through `ShaderEffect` does not. Effect authors should therefore treat the layout above as a tiny fixed ABI, not as name-reflected HLSL constants.

28.7.2 Where D3D11 and D3D12 stop being the same

D3D11 can bind the compiled vertex shader, pixel shader, input layout, and dynamic constant buffer as separate context state. Every `Bind()` performs a write-discard map, copies the 128 bytes, and attaches the buffer to `b0` for both shader stages. The native map mode is `D3D11_MAP_WRITE_DISCARD`.

D3D12 has no equivalent independent shader bind. Its effect constructor must build a complete `ID3D12PipelineState` up front, using the same cached root-signature shape as the stock sprite path: one CBV at `b0`, one SRV table at `t0`, and one sampler table at `s0`. The 128-byte constant buffer lives in a persistently mapped upload resource; `Bind()` only copies the current values, while `D3D12SpriteBatchRenderer::FlushBatch()` binds the resource and the prebuilt PSO inside its command list.

That PSO also makes D3D12's boundary stricter and directly visible in source: it is compiled for one `DXGI_FORMAT_R8G8B8A8_UNORM`, single-sample color target, with blending, depth, and stencil disabled and culling set to none. Those choices match the custom-effect pixel test. They do not form a dynamic promise that the same object can be reused for an MRT set, an MSAA target, another color format, or caller-selected pipeline state; those dimensions would require separate PSOs and a cache key that the current effect renderer does not build.

28.7.3 Shared D3D11/D3D12 pixel-test example

The two smoke tests use byte-for-byte equivalent HLSL to turn a solid red texture into exact cyan. The vertex shader's `vpSize` field is at byte zero because the renderer fills it; the four padding vectors advance the next public vector slot to byte 80:

```

1  const char* vertexHlsl = R"(
2  struct VSIn  { float2 pos : POSITION0;
3                  float2 uv  : TEXCOORD0;
4                  float4 col : COLOR0; };
5  struct VSOut { float4 pos : SV_Position;
6                  float2 uv  : TEXCOORD0;

```

```

7         float4 col : TEXCOORD1; };
8     cbuffer CB : register(b0) {
9         float4 vpSize;
10        float4 pad1[4];
11        float4 uColor;
12        float4 uFloat0;
13    };
14    VSOut main(VSIn input) {
15        VSOut output;
16        float2 ndc = (input.pos / vpSize.xy) * 2.0 - 1.0;
17        output.pos = float4(ndc.x, -ndc.y, 0.0, 1.0);
18        output.uv = input.uv;
19        output.col = input.col;
20        return output;
21    });
22
23    const char* pixelHlsl = R"(
24    Texture2D texSampler : register(t0);
25    SamplerState texSamplerSampler : register(s0);
26    struct PSIn { float4 pos : SV_Position;
27                float2 uv : TEXCOORD0;
28                float4 col : TEXCOORD1; };
29    float4 main(PSIn input) : SV_Target {
30        float4 source = texSampler.Sample(texSamplerSampler, input.uv);
31        return float4(1.0 - source.rgb, 1.0);
32    });
33
34    ShaderEffect invert(device, vertexHlsl, pixelHlsl);
35    if (!invert.IsEffectValid()) {
36        throw std::runtime_error("custom HLSL did not compile");
37    }
38
39    SamplerState pointClamp = SamplerState::PointClamp;
40    SpriteBatch batch(device);
41    batch.Begin(SpriteSortMode::Deferred, BlendState::Opaque,
42                &pointClamp, nullptr, nullptr, &invert);
43    batch.Draw(redTexture, destination, Color::White);
44    batch.End();

```

D3D11 runs this through a windowed Wine/DXVK device and reads the sprite's top-left region. D3D12 runs the same public `GraphicsDevice/SpriteBatch` sequence against a `HeadlessEXT` off-screen device, then reads the render target. In both cases solid (255, 0, 0, 255) must become exactly (0, 255, 255, 255). That result proves more than successful compilation: the fixed input layout, automatic viewport slot, b0 constant-buffer/root binding, t0/s0 resource path, custom pixel shader, and GPU readback all participated in one observable public-API draw.

28.8 Persistent Direct3D resources implement the shared usage contract

All three Direct3D factories receive the `preserve` Boolean for 2D and cube targets but need not translate it into a native load action: their resources persist across binding. Shared code supplies the visible policy. `DiscardContents` clears black plus each available depth and stencil aspect; `PreserveContents` and `PlatformContents` do not. The first descriptor controls the same policy for MRT. A redundant bind is not a no-op: native resolve/finalization can run before the resource is rebound, and a discard target receives its three-aspect clear again.

Public pixel fixtures establish this policy. The shared cube-usage fixture renders an asymmetric face, rebinds it and adds only a small marker, so a discard masquerading as preservation cannot pass. Parallel depth and stencil fixtures preserve an occluder or stamp across a bind cycle, while pass-boundary and ordered-clear tests separate usage policy from explicit clear chronology. D3D9 and D3D11 register these tests in their routine renderer suites; D3D12 cross-builds the same public fixtures, while its headless/direct smoke coverage supplies the local GPU mechanisms. See §19.4.1.

28.9 Cube-target MSAA, mip generation, and public finalization

Shared `D3DCommon` tables do not imply identical resource algorithms. D3D11 and D3D12 use parallel designs for `RenderTargetCube` MSAA but different mip-generation routes. An interface-default audit also exposed a public routing hole after those mechanisms landed: early smoke checks invoked `BindAsRenderTargetFace()` and `UnbindAsRenderTarget()` directly, while the public route never finalized the outgoing cube. That defect is now closed. The following subsections describe both the renderer algorithms and the tracking edge that connects them to the public device route.

28.9.1 `RenderTargetCube` MSAA

Neither `D3D11_RESOURCE_MISC_TEXTURECUBE` nor a `D3D11_SRV_DIMENSION_TEXTURECUBE` view can ever be multisampled, and D3D12's equivalent `D3D12_SRV_DIMENSION_TEXTURECUBE` has no multisampled variant either — so on both renderers, the MSAA color resource becomes a plain, non-cube six-slice `Texture2DMSArray` used only as a render-target view, while a second, single-sample, cube-flagged resource receives `ResolveSubresource()` on unbind. The shader-resource view targets the resolved resource. D3D12 additionally requires two explicit resource-barrier transitions (`RESOLVE_SOURCE` on the color resource, `RESOLVE_DEST` on the resolve target) that D3D11 does not need at all — its context-level `ResolveSubresource()` call handles the equivalent state transition implicitly. Both renderers resolve only the currently active face on unbind, the same one-face-at-a-time convention their mip-chain handling follows. On both renderers, an 8×8 cube face requested at 4x MSAA is cleared, resolved, and read back exactly; `GetMultiSampleCount()` must also report four.

28.9.2 Mip-chain generation: one native GPU call versus an explicit CPU round trip

After drawing one cube face, D3D11 calls `ID3D11DeviceContext::GenerateMips()` on a shader-resource view spanning all six faces. The GPU regenerates every face's chain; unchanged level-zero faces reproduce their prior mips. D3D12 has no corresponding convenience operation. Its `GenerateMipsEXT()` reads the active face's preceding level through `ReadbackSubresourceRGBA8()`, box-filters it on the CPU, and uploads the next level with `UploadSubresourceRGBA8()`. It repeats this round trip for each level of that face. Both routes produce the required chain, but their synchronization and cost differ substantially. Shared bytecode and state mappings do not imply a shared resource algorithm.

28.9.3 How renderer tracking closes the public finalization edge

D3D11 and D3D12 now override the cube-face route and retain the active cube separately from the single-2D and MRT state. Before any cube-to-cube, cube-to-2D, cube-to-MRT or cube-to-backbuffer transition, `FlushPendingCubeResolveEXT()` clears that tracking pointer and

calls the outgoing cube's `UnbindAsRenderTarget()`. That ordering is important: the callback performs the active-face MSAA resolve and mip generation before the new destination replaces native state. The plural normalized-descriptor path recognizes a single cube face and delegates to this same route; cube faces in a multi-target set remain an explicit renderer rejection rather than being silently treated as 2D targets.

The original direct smoke checks still prove the two low-level algorithms described above, but they are no longer the only evidence. Shared `GraphicsDevice` fixtures now drive an asymmetric rendered cube face through public binding and readback, rebind it for the usage test, and isolate preserved multisample content across two faces. Those fixtures make a skipped finalization, wrong face/subresource, stale resolve or false preservation visibly fail. Thus the former caution remains a useful account of how the hole escaped an implementation-local smoke test, not a current feature limitation.

28.10 HeadlessEXT: routine off-screen D3D12 coverage

`SpriteFont`, `Model`, and part of the `Texture2D.FromStream/SaveAsPng` route require a `GraphicsDevice`. That constructor formerly created an SDL window for every renderer except `HEADLESS` and `SOFTWARE`. On D3D12, window creation also required the Proton-managed swapchain route, which was too expensive for the routine plain-Wine test loop.

`PresentationParameters` therefore gained the CNA-only `HeadlessEXT` property:

```
1 CNAEXT bool getHeadlessEXTProperty() const;  
2 CNAEXT void setHeadlessEXTProperty(bool value);
```

The property defaults to `false`; only D3D12 honors it at the pin. D3D11 always creates a swapchain and EasyGL requires a window-bound GL context, so both reject it. A headless D3D12 device renders only to explicit targets and cannot present. This supports routine tests as well as off-screen applications such as server-side rendering and thumbnail generation.

28.10.1 Four D3D12 defects exposed by headless `GraphicsDevice` tests

The first routine public-device tests exposed four corrected D3D12 defects:

- `SpriteBatch` ignored sampler filter and address-mode updates, inheriting state from a prior 3D draw;
- render targets failed to bind their depth-stencil views;
- the six combined color/depth/stencil clear variants threw; and
- `SetDepthTestEnabled`, `SetDepthWriteEnabled`, and `SetBlendEnabled` threw when the `Model` route called them.

All four are fixed at the pin. Their discovery also shows why direct renderer smoke tests and public `GraphicsDevice` fixtures are complementary: the former had not traversed these shared-framework call paths.

Chapter 29

Windows 2D: Direct2D and GDI

DIRECT2D and GDI share a platform and a 2D scope, but not a rendering architecture. Direct2D submits retained native drawing commands. GDI presents a CPU-raster bitmap produced by the same software core used by the platform-independent SOFTWARE renderer.

Identity	Delivery path	3D result	RT / MRT / query
DIRECT2D	ID2D1 over Win- dows/Wine D2D, D3D11, and DXGI components	inherited route rejects	yes / no / no
GDI	privately composed CPU 2D raster, then Win32 DIB presentation	explicit rejection	yes / no / no

29.1 DIRECT2D: a different Windows delivery stack

The renderer is not a simplified mode of DIRECTX11. It calls the ID2D1 API and has no shader stage. Under Wine or Proton it relies on those environments’ own built-in D2D, D3D11, and DXGI components; every registered path skips the project’s ordinary DXVK gate. Replacing those built-ins with DXVK changes the delivery stack being tested.

Sprite and shape work is expressed through native 2D resources and commands. The renderer implements `RenderTarget2D`, but neither MRT nor occlusion queries. Effect-aware 3D methods inherit the common colored route and terminate in the family’s explicit 3D refusal. That is preferable to showing a plausible sprite-colored triangle after discarding a stock effect packet.

29.1.1 Evidence and the release boundary

The repository has a dedicated native-Windows graphics workflow whose matrix includes Direct2D, while local development also uses Wine/Proton. Its Direct2D leg runs the five labelled tests with the D3D11 debug layer, a WARP lifetime pass and a live-object log gate. The renderer’s own release gate remains stricter: it is blocked pending an attributable native adapter/driver run and a physical multi-DPI presentation capture. A workflow definition and even a hosted hidden-window pass do not satisfy those two recorded-evidence criteria.

The practical report for a Direct2D failure must therefore record whether it ran through native Windows or the Wine built-ins, and whether presentation, readback, or only off-screen rendering was observed. A clean device construction proves neither pixel output nor the native release gate.

29.2 GDI: Win32 presentation over the software renderer

The key line in the implementation is now composition, not public inheritance. `GdiRenderer` derives directly from `IGraphicsRenderer` and privately owns a `GdiSoftware2DCore`. The build publishes a reviewed subset of the Software module's CPU-2D translation units into the GDI archive, and GDI forwards only its intended sprite, texture, target and state operations. No complete Software renderer pointer escapes, so a future Software 3D virtual cannot silently appear in GDI's public vtable. GDI separately owns the Windows bitmap, damage tracking and DIB presentation edge.

This architecture provides more than a trivial blit. The CPU path has a real standalone 8-bit stencil plane and an opt-in, exact 4x CPU-MSAA backbuffer mode, but no depth buffer. A 2D render target has color, stencil and optional authored mip generation; it deliberately has neither depth nor an MSAA attachment. GDI implements one active 2D target, but not MRT or occlusion queries. Its extended 3D methods reject unsupported input directly rather than relying on the base colored fallback.

29.2.1 Why shared code does not make the identities equivalent

`SOFTWARE` can run with no native window and returns pixels from its own buffers. GDI must also create, update, and present a Win32 bitmap. A CPU raster test can prove the shared math while missing a stride, channel-order, invalidation, or lifetime defect in the GDI boundary. Conversely, a present smoke test can show a bitmap without distinguishing the depth or stencil semantics beneath it.

The renderer-specific CTests are excluded while cross-compiling, so a Linux-host build cannot execute them merely because it produced a Windows binary. A separate native MSVC workflow is the meaningful automated platform tier.

29.3 Choosing and testing the Windows 2D paths

Choose Direct2D when native ID2D1 resource and composition behavior is the product goal. Choose GDI when a deterministic CPU-rendered image delivered through the classic Win32 surface is valuable. Neither identity is a route to programmable 3D.

A useful comparison scene should include alpha and additive sprites, a clipped draw, a target round trip, and a format-sensitive readback. Add depth, stencil, and MSAA probes for GDI's shared software core. Run the same scene off-screen and through real presentation so the raster and delivery boundaries are tested separately.

Chapter 30

2D and Vector Rasterizers

`SDL_RENDERER` is CNA's simplest renderer, its most exhaustively audited, and the one whose scope is narrowest by explicit design: a 2D-only rendering path built directly on SDL3's own 2D texture-blit renderer, with no 3D pipeline, no programmable shader stage, no depth/stencil buffer, and no multisample antialiasing at all.

It is one of five 2D/vector families in this chapter. Their common output is a raster image; the route to that image ranges from SDL texture blits through two CPU rasterizers to an OpenVG-on-fixed-OpenGL implementation.

Identity	Raster route	3D result	RT / MRT / query
<code>SDL_RENDERER</code>	SDL3 texture-blit API	inherited route re-jects	yes / no / no
<code>SKIA</code>	Skia raster or Ganesh mode	explicit rejection	yes / no / no
<code>BLEND2D</code>	CPU Blend2D, streamed through SDL	inherited route re-jects	yes / no / no
<code>OPENVG</code>	ShivaVG on fixed-function OpenGL	inherited route re-jects	no / no / no
<code>NANOVG</code>	NanoVG on a real OpenGL context	explicit rejection	no / no / no

30.1 Scope, by design

Every 3D-facing entry point on this renderer — `CreateVertexBuffer`, `DrawColoredPrimitives`, and the rest of the 3D draw dispatch from Chapter 19 — throws immediately, with a consistent "SDL_Renderer does not support 3D" message. This is not treated as a bug: a systematic full-suite run confirms exactly thirteen known, expected pre-existing test failures, every one of them this exact throw, matching the renderer's documented scope precisely. A custom `Effect` passed to `SpriteBatch::Begin(effect)` also throws, for the same underlying reason — there is no shader stage to run it on.

30.1.1 The 2D boundary is at execution, not at every 3D-looking object

"2D-only" does not mean that every type normally associated with 3D becomes impossible to construct. CNA deliberately distinguishes a renderer-independent description of work from the first operation that asks this particular renderer to execute that work. A `VertexDeclaration`, for example, is only a stride plus a list of `VertexElement` records; it has no `GraphicsDevice` ownership and no renderer allocation. All four of its public constructor shapes therefore work on `SDL_RENDERER`. The same declaration becomes unsupported only when it is handed to a real draw call:

```

1 VertexDeclaration decl(16, {
2     VertexElement(0, VertexElementFormat::Vector3,
3         VertexElementUsage::Position, 0),
4     VertexElement(12, VertexElementFormat::Color,
5         VertexElementUsage::Color, 0)
6 });
7 static const VertexPositionColor vertices[3] = {
8     {Vector3::Zero, Color::White}, {Vector3::Zero, Color::White},
9     {Vector3::Zero, Color::White}
10 };
11
12 // Construction above is pure data. This execution request throws
here:
13 device.DrawUserPrimitives(PrimitiveType::TriangleList, vertices, 0,
14     1, decl);

```

This is not merely an API-design claim. The renderer’s dedicated regression test constructs the declaration, confirms that it retains the supplied stride and two elements, then makes the `DrawUserPrimitives` call and verifies that the failure happens there rather than while declaring the layout. It finally clears and reads back the 2D device successfully, proving the exception did not leave renderer state unusable. This boundary lets portable tools and asset code prepare vertex layouts without a false renderer failure, while still refusing the first request that would require an SDL renderer 3D pipeline it does not have.

The same split applies to the five stock 3D effects. `BasicEffect`, `AlphaTestEffect`, `DualTextureEffect`, `EnvironmentMapEffect`, and `SkinnedEffect` can all be constructed, have ordinary properties assigned, and run `Apply()` on this renderer. They are data/-configuration objects until a primitive draw is attempted; that draw still throws. In particular, the effect test assigns a real `Texture2D` where applicable but deliberately leaves `EnvironmentMapEffect`’s `TextureCube` pointer null, separating this verified lifecycle rule from the still-blocked question of cube-texture construction.

There is a third, easy-to-miss case: `RasterizerState` and `DepthStencilState` assignments also round-trip as data, but their renderer application is an intentional no-op. `SDL_Renderer` inherits the interface’s no-op `ApplyRasterizerState` and `ApplyDepthStencilState` defaults because neither state can affect a 2D texture-blit draw. Code can therefore store and inspect a desired `CullMode` or depth-enable bit without an exception; it must not infer that either state will change pixels. By contrast, `VertexBuffer` and `IndexBuffer` constructors immediately call their renderer factories and fail before their own argument validation runs. On this renderer even a zero or negative requested count therefore produces the same explicit “does not support 3D” runtime error, rather than an `ArgumentOutOfRangeException`. The ordering is observable and tested, so callers that need portable validation should validate their input before asking `SDL_Renderer` to allocate a 3D buffer.

30.1.2 A narrow positive capability among explicit refusals

Chapter 19’s `GraphicsCapability` mechanism defaults to answering `true` for everything unless a renderer overrides it. Reading `SdlRenderer.hpp` directly shows this renderer instead makes one narrow positive claim and refuses every broader 3D/resource capability:

```

1 bool SupportsCapability(GraphicsCapability capability) const
2     override
3 {
4     // SDL_ComposeCustomBlendMode represents the standard Additive
preset.

```

```

4     return capability == GraphicsCapability::AdditiveBlending;
5 }

```

Of the current thirteen values, only `AdditiveBlending` is true. SDL's custom blend mode can express that preset's independent colour and alpha equations; this does not imply arbitrary `BlendState` support. `ThreeD`, complete depth/stencil, standalone stencil, MSAA, MRT, anisotropy, wireframe, queries, custom effects, volume storage, multistream input and instancing all return false. The important property is not a blanket answer but an explicit, auditable boundary: adding the legitimate 2D blend claim did not reopen any 3D promise.

```

1  if (device.SupportsCapability(GraphicsCapability::ThreeD)) {
2      DrawModel(model, world, view, projection);
3  } else {
4      // Reached on SDL_RENDERER (and CANVAS) -- checking first here
5      // is strictly better
6      // than letting CreateVertexBuffer() or DrawColoredPrimitives()
7      // throw
8      // "SDL_Renderer does not support 3D" and catching it after the
9      // fact.
10     DrawFlatPlaceholderSprite();
11 }

```

30.2 Recorded SDL Renderer audit and later repairs

The recorded 65-item technical audit found and fixed fifteen defects across most of the `SpriteBatch` surface. Its closing document is historical; later repairs described below supersede parts of that snapshot.

- **SpriteBatch (2 bugs).** `SDL_RenderTextureRotated`'s rotation-pivot convention did not match XNA's own — fixed by offsetting the destination rectangle. The `transformMatrix` argument to `Begin()` was silently ignored in its entirety — fixed via a new path using `SDL_RenderTextureAffine`, used only for a genuinely non-identity transform (Chapter 13 covers both in full).
- **SpriteFont (1 bug, shared across every renderer).** The `SpriteEffects` flip bug described in Chapter 13 — fixed once, in the shared `SpriteBatch.cpp`, benefiting every renderer at once rather than being an `SDL_Renderer`-specific fix.
- **BlendState (2 bugs).** Only `Opaque` and `Additive` were specially handled; every other preset mapped incorrectly. Separately, `SpriteBatch::Begin()` was unconditionally clobbering whatever blend mode had already been set, every single call.
- **SamplerState (1 bug).** Four of nine `TextureFilter` values were silently downgraded to nearest-neighbor sampling rather than mapping to their real `SDL_ScaleMode` equivalent.
- **RenderTarget2D (4 bugs).** A shared depth-clear code path crashed outright when asked to clear the depth buffer of a render target with `DepthFormat::None` (i.e., no depth buffer at all, which is every render target on this 2D-only renderer). A second bug performed an unchecked, unsafe downcast between two sibling renderer classes when sampling a render target as an ordinary texture — a genuine memory-safety hazard, not merely a logic error. The requested `DepthFormat` itself is honestly acknowledged as emulated: it is silently accepted and echoed back, but no functional depth test exists behind it at all on this renderer. A new `HasRealDepthBuffer()` check was added specifically so calling code can ask, rather than assume. The persistent SDL target texture also makes

shared `RenderTargetUsage` behavior unusually direct: Discard is black-cleared on bind, while Preserve and Platform retain color. The strong usage test proves the first two with pixels, but its “every bind” wording omits CNA’s divergence from FNA: a redundant bind clears again here, while FNA returns early.

- **Present timing (1 fixed runtime bug, 1 remaining construction asymmetry).** Runtime `PresentInterval::Two` used to collapse silently to `One`. `SetSwapInterval()` now sends the exact value 2 to `SDL_SetRenderVSync()` and falls back to 1 if the driver rejects it. The constructor still uses `swapInterval > 0 ? 1 : 0`, despite SDL3 supporting integer intervals there too. This usually stays hidden because `Game` constructs with the default 1 and later requests arrive through the repaired reset path; direct `GraphicsDevice` construction with `Two` still loses the distinction. ASCII wraps this same renderer and inherits both behaviors.
- **Presentation formats (stored, not selected).** Neither the constructor nor a later reset receives the requested backbuffer/depth format: the factory passes only window, logical-presentation, and swap-interval fields. SDL chooses the renderer’s window output, while every CNA-created SDL texture is RGBA32 and no depth/stencil attachment exists. Fullscreen is different: shared device code still makes SDL’s window transition request before the renderer’s empty format hook. The dedicated fullscreen test therefore proves stored state, no throw, and continued pixels — not display acceptance under Xvfb. ASCII inherits this complete split too.
- **Disposed-resource guards (3 bugs).** A missing disposed-texture check inside the internal sprite-push path; a missing equivalent check in `SetRenderTargets`; and a dangling renderer pointer left behind after `RenderTarget2D::Dispose()`.
- **OcclusionQuery (1 bug).** Construction previously succeeded silently even with a null renderer — now throws, consistent with the 2D-only-renderer pattern of failing loudly at construction rather than producing a query object that can never do anything.

30.3 One blocked sampler decision, and one resource gap that later closed

One renderer-local architecture decision remains open; the former resource-construction item no longer does:

TextureAddressMode::Wrap / Mirror via SpriteBatch. `SDL_RenderTexture`’s own `srcrect` handling has exactly one fixed, clamp-like behavior at the edges. SDL3 does expose `SDL_SetRenderTextureAddressMode`, but it only affects `SDL_RenderGeometry` — an entirely different draw call than the one this renderer’s `Draw()` actually issues. Three concrete options were identified and none chosen: throw unconditionally whenever `Wrap / Mirror` is requested, rewrite `Draw()` onto `SDL_RenderGeometry` project-wide, or a hybrid that only throws when the source rectangle actually exceeds the texture’s bounds (the common case where it does not would keep working, coincidentally, under the existing clamp-like behavior).

Texture3D / TextureCube: former silent-resource gap, now closed. `Texture3D` now consults the false `Texture3D` capability and throws `NotSupportedException` during construction. A plain `TextureCube` may still exist as a renderer-independent description with no native resource, but its public `SetData/GetData` paths detect that absence and throw instead of accepting a write or fabricating a transparent-black face. Shared read/write contract fixtures pin both outcomes on this renderer. The old 94-test blast-radius

note explains why the first proposed blanket-construction change was deferred; it is not the current observable contract.

30.3.1 What “clamp-like by coincidence” means in pixels

The blocked `TextureAddressMode::Wrap / Mirror` decision above is easy to read as an abstract API gap; a concrete pixel case makes it precise instead. Take a 2×1 texture, texel 0 red and texel 1 blue, and draw it with a source rectangle *twice* the texture’s own width — source texel range `[0,4]` against a 2-texel-wide texture, the classic XNA scrolling-background tiling technique, which FNA’s real `SpriteBatch` never clamps `sourceRectangle` to the texture’s own bounds to prevent:

```
1 Texture2D redBlue(getGraphicsDeviceProperty(), 2, 1);
2 const Color pixels[2] = {Color::Red, Color::Blue};
3 redBlue.SetData(pixels, 2);
4
5 Rectangle oversizedSource(0, 0, 4, 1); // spans texel [0,4] on a 2-
    texel-wide texture
6 spriteBatch->Begin(SpriteSortMode::Deferred, BlendState::Opaque, &
    pointClamp, nullptr, nullptr);
7 spriteBatch->Draw(redBlue, destRect, oversizedSource, Color::White)
    ;
8 spriteBatch->End();
```

Reading back the destination pixel corresponding to source position 1.25 (three-quarters of the way into the oversized range, past the real 2-texel-wide texture entirely) is where the “clamp-like, by coincidence” finding becomes a concrete, checkable number rather than a qualitative claim: with `PointClamp` requested, the correct XNA answer is **blue** — clamped to the texture’s last real texel — and `SDL_RenderTexture`’s one fixed edge behavior genuinely produces exactly that, with zero production code written to make it happen. Requesting `PointWrap` at the identical source position is where the still-open gap becomes equally concrete: the XNA-correct answer for a wrapped read at position 1.25 is **red** (wrapping back around to texel 0), but this renderer’s `Draw()` produces **blue** again — the same fixed clamp-like behavior applies regardless of which `TextureAddressMode` was actually requested, since (per the finding above) `SDL_RenderTexture` has no address-mode parameter reachable from this specific draw path at all. A game relying on horizontally-scrolling tiled backgrounds — a common enough technique that this is not a hypothetical edge case — gets the wrong pixel on this renderer today, silently, with no exception and no visual artifact obviously identifiable as “wrong” unless you already know to look for texel 0 failing to reappear at the wrap boundary.

30.4 Native and emulated behavior

Beyond the `DepthFormat` case above, `TextureAddressMode::Clamp` is flagged as working “by coincidence” — it matches `SDL_RenderTexture`’s own fixed out-of-bounds behavior, not because any real sampler-state value is being honored. `MultiSampleCount` is accepted and ignored, always reporting zero. Fullscreen toggling round-trips correctly at the API level, but the audit is explicit that genuine OS-level fullscreen state cannot actually be verified inside this project’s own headless `Xvfb` test environment. `ClearOptions::DepthBuffer / Stencil` expose a shared-routing distinction from the rest of this chapter’s 3D methods. The renderer’s six direct combination hooks do call `ThrowNo3D`, but public `GraphicsDevice.Clear` first sees `SupportsDepthStencil()==false` and masks both bits away. A depth/stencil-only request is therefore an FNA-faithful silent no-op; adding `Target` still clears colour. The two SDL-specific tests that expect public throws predate that mask and are stale; they do not establish that the

direct hooks are reachable. Chapter 12 and §19.4.2 separate the shared and renderer layers in full. Five real, multi-frame compatibility samples — a 2D demo, a bouncing-sprite physics scene, a keyboard-driven sprite, a two-glyph `SpriteFont` test, and an animated spritesheet — are registered as genuine `cctest` entries rather than left as manual verification, giving this renderer end-to-end coverage beyond its unit-level pixel tests.

Viewport and scissor expose a more subtle 2D-state boundary. There is no `SetViewport()` override, so a custom public viewport changes stored state and `Viewport`'s own projection mathematics but never remaps an SDL sprite. Scissor *does* reach SDL: a positive `ScissorRectangle` installs a renderer clip, and a non-positive size disables it. However, the rasterizer-state hook is also absent. The clip is consequently active solely because the rectangle is non-empty, even when `RasterizerState.ScissorTestEnable == false`. The existing rasterizer construction test cannot detect this: it verifies no-throw assignment and getter round trip, then clears one pixel, but never sets a scissor rectangle or asks whether drawing outside it survives. This is the precise distinction the inherited fourteen-product audit matrix in §19.4.3 preserves.

30.5 Reading the feature matrix for this renderer

The repository's feature-matrix document is a dated planning snapshot, not a live test census. Its historical custom-`Effect` refusal remains accurate, but its two blocked rows do not: only `Wrap/Mirror` still needs the renderer-architecture decision described above; cube/volume silent success was closed by the shared resource-contract repairs. Current claims in this chapter therefore come from the registered renderer tests and pinned source, not from the snapshot's old pass total or status count.

30.6 Xvfb screenshot evidence

Screenshots credited to CNA's `SOFTWARE` renderer (Chapter 12) need no display server. `SDL_RENDERER`, by contrast, creates an SDL3 window and submits `SDL_RenderTexture` calls through an OpenGL context. The retained campaign therefore used `Xvfb` with Mesa's `llvmpipe`. Earlier feasibility notes had identified this route but left it untested; CNA's `CNA_TEST_DISPLAY` cache variable already permits a virtual display such as `:99` in place of the default `:0`.

Building this renderer (`cmake -B build-sdlrenderer -DCNA_GRAPHICS_RENDERER=SDL_RENDERER`) and running the screenshot demo under `xvfb-run` exercised the display and capture path. The demo reuses the same scene as the golden-image regression test `modules/renderers/easygl/examples/easygl_spritebatch_rotation_golden_test.cpp`, also used in Chapter 13: a 100×100 `Texture2D` whose top-left 20×20 block is red and the remainder blue, drawn through `SpriteBatch` rotated 90 degrees (`MathHelper::PiOver2`) around origin (100,100) — the source's own bottom-right corner — then captured with the same `SaveBackBufferScreenshotEXT` helper the `SOFTWARE` renderer screenshots use:

```
1 xvfb-run -a --server-args="-screen 0 1024x768x24" \
2   env -u WAYLAND_DISPLAY ./build-sdlrenderer/
   cna_xvfb_screenshot_demo
```

The marker position matches the geometric derivation used by the golden test (Chapter 13). The retained artifact closes the screenshot-infrastructure feasibility item for a renderer that requires a virtual display.

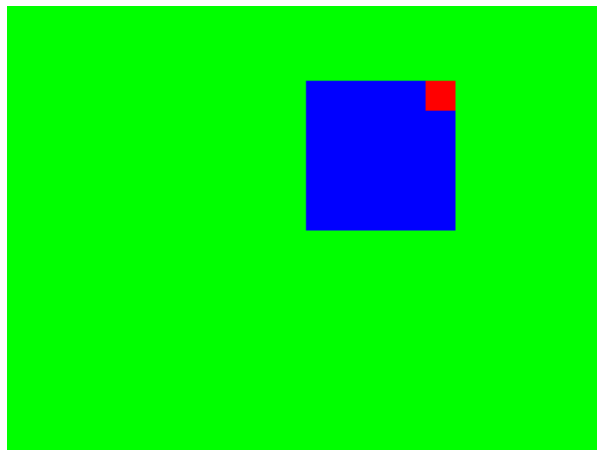


Figure 30.1: Output of the rotate-around-origin `SpriteBatch` scene rendered by `SDL_RENDERER` under `Xvfb`, without a physical GPU or desktop. The red marker lands in the destination rectangle’s top-right corner, the expected result of a 90-degree rotation about the source’s bottom-right corner.

30.7 Pixel readback and presentation mode

The pixel fixtures share one prerequisite, documented in their source comments:

Requires `PresentationMode::NativeBackBuffer`: `SDL_RenderReadPixels` operates in physical output coordinates, while this renderer’s default presentation mode (`FixedHeightDynamicWidth`) does not map logical pixels 1:1 to physical ones.

`GetBackBufferData()` delegates to `SDL_RenderReadPixels`, which reads physical output coordinates. `FixedHeightDynamicWidth` may letterbox logical output, so a logical coordinate does not necessarily name the same physical pixel. `NativeBackBuffer` makes the two sizes equal. The fixtures select it explicitly; without that precondition, a plausible coordinate check can sample the wrong pixel.

30.7.1 Animated sprite-sheet frame selection

`modules/renderers/sdl-renderer/examples/sdlrenderer_sample_animated_spritesheet_test.cpp`, the fifth entry in a five-sample compatibility suite, exercises a different route from the single-frame checks above. These samples use multiple `Update()` calls. The fixture covers the `SpriteBatch::Draw` `sourceRectangle` parameter, re-selected every `Update()` call to animate through a spritesheet. The deterministic sheet is an 8×4 texture, left 4×4 solid red (frame 0), right 4×4 solid green (frame 1), with a fixed per-`Update()` frame-index increment instead of elapsed-time-based timing, so the outcome never depends on wall-clock frame speed:

```
1 void Update(GameTime&) override
2 {
3     if (done_) return;
4     ++frameCounter_;
5     ++updatesRun_;
6 }
7
8 void Draw(const GameTime&) override
9 {
10    // ...
11    const int frameIndex = frameCounter_ % 2;
```

```

12     const Rectangle srcRect(frameIndex * kFrameSize, 0, kFrameSize,
        kFrameSize);
13     const Rectangle destRect(2, 2, kFrameSize, kFrameSize);
14
15     sb_>Begin();
16     sb_>Draw(*sheet_, destRect, srcRect, Color::White);
17     sb_>End();
18     // ...
19 }

```

After exactly three `Update()` calls, `frameCounter_ % 2` evaluates to 1, so the animation should have settled on the green frame. The test then reads the rendered pixel instead of checking the frame-index arithmetic alone:

```

1  Color px(0, 0, 0, 0);
2  Rectangle region(3, 3, 1, 1);
3  dev.GetBackBufferData(&region, &px, 0, 1);
4  check(px.getRProperty() <= 15 && px.getGProperty() >= 240 && px.
    getBProperty() <= 15,
5      "frame 1 (green) reached the rendered output");

```

Without `PresentationMode::NativeBackBuffer`, the frame index can be correct while the pixel check samples a scaled or letterboxed coordinate. That failure would concern fixture setup, not animation.

30.8 SKIA: a bounded contract written down before expansion

SKIA is the only renderer whose dependency is neither a system library nor fetched by the project. Configuration requires a separately built Skia artifact through `CNA_SKIA_ROOT` and `CNA_SKIA_BUILD_DIR`; absence is fatal.

`CNA_SKIA_MODE` selects raster or Ganesh inside the family. It is not a second public renderer identity. Both modes retain the same CNA contract and must be named separately only when their execution evidence differs.

The implementation has a deliberately bounded 2D surface. It owns CPU-raster targets and explicitly rejects 3D, MRT, MSAA, and occlusion queries. Optional SkSL extensions widen specific shader-like operations without turning the renderer into a general `ShaderEffect` route. The companion capability matrix explains each refusal rather than relying on a default-true switch; this is one of the strongest capability-documentation patterns in the repository.

30.9 BLEND2D: CPU vectors presented as an SDL texture

BLEND2D fetches pinned Blend2D and AsmJit revisions unless root overrides are supplied. It rasterizes on the CPU, uploads the resulting pixels to a streaming SDL texture, and uses SDL only for the final presentation edge.

The family implements 2D targets. It has no MRT, query, or 3D path; inherited extended draws reach the explicit refusal. Per-channel color-write masks are not natively expressible in its current compositor, a useful example of a state property that can exist in the public object without a portable 2D mapping.

The evidence should separate CPU raster output from streaming-texture presentation. Direct buffer inspection can prove shape and blend math, while only a real SDL present/readback path can prove pitch, upload format, and final channel order.

30.10 OPENVG: a vector API on a historical GL substrate

OPENVG uses ShivaVG, pinned to its only upstream commit, plus a local leak patch, a synthesized configuration header, a GL type shim, and GLU. The resulting OpenVG 1.1 path runs over fixed-function OpenGL and needs a real compatible context.

Its capability policy returns false for all 13 flags. That is conservative but internally coherent: the renderer supplies its 2D drawing surface without promising the broader GPU features those flags describe. It has no render-target factory, MRT, queries, shaders, or 3D; effect-aware draws inherit the common route and reject.

30.11 NANOVG: immediate vectors without off-screen storage

NANOVG fetches a pinned upstream revision and uses NanoVG's OpenGL backend for paths, images and 2D composition. It is deliberately 2D-only: there is no vertex/index-buffer 3D pipeline, programmable custom effect, depth/stencil contract, MRT or query. Although NanoVG ships an optional framebuffer helper, CNA does not include it in this family's scope; `RenderTarget2D` construction therefore fails transactionally rather than returning a resource that cannot be bound.

The family reports the relevant capabilities false and refuses anisotropic sampling, wireframe and non-default color-write masks it cannot preserve through NanoVG's flush. Its tests cover real context creation, images, scissor, transformations, blending and refusal behavior; a dedicated workflow declares NanoVG automation. These are source-visible gates here, not a newly executed pixel campaign by this edition.

30.12 Five ways to prove a 2D renderer

For SDL Renderer, observe the actual SDL target and presentation mode. For Skia, name raster or Ganesh and test the bounded SkSL path separately. For Blend2D, compare the CPU pixels before and after SDL upload. For OpenVG, capture the GL implementation and context profile alongside the vector result. For NanoVG, name the OpenGL context and distinguish immediate vector output from absent off-screen-target storage. A single screenshot can look identical across all five while exercising materially different ownership and delivery paths.

Chapter 31

Web and DOM Renderers

Four CNA identities render XNA-style sprites in a browser without exposing CNA's WebGL renderer contract. **CANVAS** paints pixels into a Canvas2D context. **HTML_DOM** represents sprites as pooled, CSS-transformed elements. **SVG_DOM** emits SVG image nodes and filters. **PIXIJS** uses the pinned PIXIJS 7 UMD runtime over the browser's own rendering choice. All are Emscripten-only and 2D-only, but their browser object models and evidence differ sharply.

Identity	Browser representation	Additive blend	RT / MRT / query
CANVAS	CanvasRenderingContext2D pixels	reported true	yes / no / no
HTML_DOM	pooled div elements with CSS transforms	browser-dependent	yes / no / no
SVG_DOM	SVG image nodes and feColorMatrix	browser-dependent	yes / no / no
PIXIJS	PIXIJS sprites and render textures	Pixi blend mode	yes / no / no

31.1 CANVAS: immediate raster operations

The renderer obtains `canvas.getContext('2d')` and maps texture uploads and sprite operations onto `drawImage`, `putImageData`, and `fillRect`. A 2D render target is another Canvas2D surface. There is no shader stage, depth buffer, MRT, occlusion query, or 3D path; inherited effect-aware draws terminate in the explicit 3D refusal.

Only additive blending is reported among the 13 capability flags. That narrow switch is more useful than the renderer contract's default-true policy because callers do not have to discover the 2D ceiling by constructing unsupported objects.

31.1.1 Built tests are not registered tests

Canvas test executables exist, but the CMake files deliberately do not register them with CTest. A green host test run therefore says nothing about Canvas pixels. The meaningful path is an Emscripten build executed in a browser with observable Canvas output.

Node alone is not a substitute: it supplies neither a browser canvas nor the rendering event loop expected by the module. Headless Chromium can provide those APIs, but only if the test actually opens the page, drives frames, and reads pixels or a screenshot.

31.2 HTML_DOM: sprites as pooled CSS objects

HTML_DOM does not rasterize every sprite into one surface. It maintains a pool of DOM elements, assigns image content, and expresses destination position, scale, rotation, origin, opacity, crop, and visibility through CSS and element properties. Reusing nodes avoids unbounded allocation as the visible sprite set changes frame to frame.

This representation makes browser layout and compositing part of the renderer contract. Ordering becomes DOM stacking order, transforms use CSS conventions, and additive blending depends on browser support for `plus-lighter`. The renderer therefore cannot promise one identical native blend implementation across browser versions.

Its off-screen target is DOM-backed, not a GPU framebuffer. MRT and queries are absent. The inherited 3D route reaches a local unconditional throw and bypasses the common `WarnAndStub` policy, so diagnostic configuration cannot turn that refusal into a warning-only placeholder.

31.2.1 The strongest browser evidence in this cluster

HTML DOM has a host contract target and a real Playwright/Chromium CI path under Xvfb. This is stronger than merely compiling Emscripten output because it executes the browser object model. It still matters what the script observes: node count and style strings prove a different layer from final composited pixels.

31.3 SVG_DOM: retained vector structure

SVG_DOM creates SVG image nodes and uses SVG transforms for sprite placement. Tinting is expressed through a real `feColorMatrix` filter rather than by rewriting texture pixels. Additive sprites set `mix-blend-mode: plus-lighter`; the capability query returns the running browser's `CSS.supports` answer, just as HTML_DOM does. The retained tree is inspectable and naturally matches vector composition, but it inherits SVG's filter, sampling, blending, and browser-layout semantics.

Like HTML DOM, it owns a DOM off-screen target, has no MRT or queries, and its 3D refusal bypasses `WarnAndStub`. Its browser smoke page checks raw CSS support, the public capability answer and the emitted sprite style together, and separate screenshot fixtures cover pixels and scissor order. Its verification tier nevertheless remains weaker: the native host target is behind an off-by-default `CNA_BUILD_SVG_DOM_HOST_TESTS` option, and no CI workflow drives either it or the browser pages. A runnable browser script is a real test route, not evidence that the route currently gates changes.

31.4 PIXIJS: browser library objects and render textures

PIXIJS is Emscripten-only and resolves a pinned PixiJS 7.4.2 UMD artifact with a recorded SHA-256; automatic download is on unless `CNA_PIXIJS_ROOT` supplies the file. It maps textures, `SpriteBatch`/`SpriteFont` draws, transforms, scissor, blend state and one active `RenderTarget2D` to PixiJS objects. The browser library may use WebGL internally, but the CNA identity remains PIXIJS and its contract is the 2D adapter, not CNA's WEBGL2 family.

MRT, mip levels above zero, 3D, custom/compiled effects, cube and volume textures, cube targets and occlusion queries are explicit refusals. Render textures make its off-screen route stronger than NanoVG's, while more than one simultaneous target still throws. Browser execution must prove the resolved library, object lifetime and final pixels; a successful Emscripten link alone does not.

31.5 The removed ASCII identity

Older editions paired Canvas with an ASCII renderer. That public identity and its selector are gone. The reusable quantization logic now lives in `CNA::Graphics::AsciiPostProcessEffect`; Appendix E records the current CNAEXT surface. Historical architecture is not a fourth row in the live renderer registry.

31.6 Comparison by object model

The four families can display an identical sprite scene while failing in different places:

- Canvas failures cluster around pixel upload, context state, clipping, and immediate compositing;
- HTML DOM failures cluster around pooling, CSS transforms, stacking, and browser blend support;
- SVG DOM failures cluster around retained-node lifetime, SVG transforms, filters, and sampling.
- PixiJS failures cluster around UMD artifact resolution, Pixi object/render-texture lifetime, batching, and the browser driver selected below the library.

A useful cross-renderer scene therefore includes crop, origin-centred rotation, alpha and additive overlap, clipping, target bind/unbind, and node-count stability over several frames. Inspect both structure and final pixels: the DOM can look mechanically correct while the browser composites a different color, and a correct screenshot can hide unbounded node growth.

31.7 Deployment checklist

Record the CNA identity, Emscripten version, browser and browser version, device-pixel ratio, canvas/SVG dimensions, and whether execution was visible or headless. Confirm that the page reaches multiple animation frames and that the observable is meaningful for the feature under test. Browser compilation is only the first gate; these renderers exist in the DOM that runs after it.

Chapter 32

Non-GPU and Diagnostic Renderers

HEADLESS, SOFTWARE, and STUB all run without a GPU, but they answer three different questions. Headless asks whether calls and lifetimes form a valid trace. Software asks what a bounded CPU rasterizer would draw. Stub asks whether framework code can survive a renderer that intentionally does almost nothing.

Identity	Observable	Extended route	RT / MRT / query
HEADLESS	validation, trace, counters	validate and trace	trace target / no / constant book-keeping
SOFTWARE	real CPU pixels	native bounded raster	yes / no / no
STUB	no pixels or bookkeeping	inherited no-op	no / no / no

32.1 HEADLESS: behavior without an image

Headless builds resource objects, validates arguments and sequencing, records state and draw events, and exposes counters and a trace log. Its extended primitive methods do not rasterize; they validate and describe the call. A render target is likewise a bookkeeping target whose bind/unbind transitions can be inspected.

This makes the renderer valuable for deterministic unit tests, servers, and API misuse checks. It can prove that a draw used the expected primitive count, target, texture, or state sequence. It cannot prove coverage, interpolation, sampling, blending, depth, or final color.

32.1.1 A query result that is not an occlusion measurement

The Headless query object returns a constant pixel count of one. That value keeps control-flow tests moving; it is not derived from geometry. Treat it as validation scaffolding, never as an oracle for visibility or sample count.

The capability switch currently falls through to true for MRT and occlusion queries even though setting more than one target throws and the query is trace-only. These are default-true contract contradictions recorded in the audit. Callers requiring real feature behavior must not infer it from the bit.

32.2 SOFTWARE: bounded CPU rasterization

Software is a renderer, not a recorder. It rasterizes sprites and a bounded 3D subset into CPU buffers: triangle lists, depth/stencil, complete per-stream vertex fetch, texture/sampler

state, the five stock-effect packet shapes, cube storage and environment-map sampling, and the public blend-factor/equation surface all have real CPU paths. The fixed shader logic remains deliberately narrower than a programmable renderer: stock lighting and fog flags are still outside the v1 surface, and accepting source text into `SoftwareEffectRenderer` does not execute that source as a custom shader.

Both extended draw methods are native CPU paths, and `RenderTarget2D` produces real pixels. Plain `TextureCube` has every declared mip/face stored and can feed the bounded environment-map path, but `RenderTargetCube`, `Texture3D`, instancing, MRT and occlusion queries remain outside v1. The capability switch explicitly returns false for `Texture3D` and instancing and true for multistream input, but its default branch still overclaims MRT and queries — the same default-policy shape as Headless, with a real rasterizer beneath other operations.

That distinction matters for test design. A Software screenshot is genuine pixel evidence for the bounded CPU algorithm. It is not evidence for GPU precision, driver shader compilation, MSAA resolve, or a programmable effect path.

32.2.1 A useful reference with known semantic limits

Software’s deterministic memory buffers make it a strong local oracle for geometry ordering, stream offsets, texture/cube sampling, target usage, depth/stencil and blending. When another renderer differs, first check that the scene stays inside Software’s declared subset. A lit, fogged, instanced, custom-shader or render-target-cube scene cannot use the CPU result as an XNA-exact ground truth.

32.3 STUB: the smallest terminal implementation

Stub intentionally keeps no rendering state worth observing. It creates no targets, records no trace, and its inherited extended draw path ends in a no-op. Its capability method returns false for every flag, avoiding the default-true trap entirely.

The family is useful as an architectural teaching device. It shows which renderer hooks must exist for the framework to construct and run even when their bodies have no presentation work. It can also expose accidental assumptions in code that should be renderer-independent, such as unconditional dereferences of optional resource factories.

Stub must not be called a headless rasterizer. It supplies neither Headless validation nor Software pixels. A passing Stub lifecycle test proves only that the surrounding framework did not demand more than the deliberately minimal contract.

32.4 Choosing the right non-GPU test double

Use Headless when the assertion concerns call shape, order, counters, lifetime, or validation. Use Software when the assertion concerns pixels within its bounded raster vocabulary. Use Stub when the assertion concerns framework survival in the absence of rendering behavior.

Question	Best identity	Invalid inference
Was the target bound before the draw?	HEADLESS	that the target contains correct pixels
Did a simple textured triangle produce this color?	SOFTWARE	that a GPU shader or MSAA path agrees

Question	Best identity	Invalid inference
Can the game loop run with no rendering facilities?	STUB	that resource or draw semantics were validated
Did an occlusion query count visible samples?	none of these	Headless's constant or Software's null factory is a measurement

32.5 Capability discipline

The cluster illustrates all three outcomes of the renderer contract's default policy. Stub overrides everything false. Software implements substantial behavior but accidentally overclaims features it does not own. Headless intentionally supplies validation objects yet lets their interface presence look like pixel capability.

Documentation and tests should therefore name the observable, not only the method. A target can be a trace object or a pixel buffer; a query can be a control-flow token or a sample count; a draw can be a log record, a CPU rasterization, or a no-op. The same C++ signature does not make those results equivalent.

Part V

The Content Pipeline

Chapter 33

ContentManager and Asset Resolution

ContentManager maps a logical name and requested C++ type to an XNB object, a CNJ object, or a native loose asset. Its most important rule is selection, not decoding: the first existing candidate is terminal even if the selected reader later rejects it. This chapter owns path choice, caching, and manager lifetime. Chapters 34 and 35 own XNB; Chapter 36 owns CNJ.

33.1 The resolution algorithm

On a cache miss, let `base = BuildAssetPath(assetName)`. The generic `Load<T>` path and the `Texture2D`, `SoundEffect`, and `TextureCube` specializations follow this order:

```
1  if (exists(base + ".xnb"))
2      return LoadXnbAsset<T>(base + ".xnb");
3
4  reader = registered LooseFileContentTypeReader<T>;
5  if (exists(base))
6      return reader.Read(base, content);
7  if (exists(base + ".cnj"))
8      return reader.Read(base + ".cnj", content);
9  for (const auto& extension : reader.GetExtensions())
10     if (exists(base + extension))
11         return reader.Read(base + extension, content);
12 return reader.Read(base, content); // preserve the reader's
    diagnostic
```

Current contract. Existence selects a candidate; successful decoding does not. A malformed XNB, invalid CNJ, corrupt literal file, directory, or rejected first native extension stops the load. CNA does not retry a lower-priority sibling.

The XNB probe appends `.xnb` to the complete name. Therefore `Load<Texture2D>("art/logo.png")` first tests `art/logo.png.xnb`, then the literal PNG. To parse `art/logo.xnb` as XNB, pass `"art/logo"`. Passing the suffix itself tests `logo.xnb.xnb`; if absent, the loose reader receives the original XNB file. Dots in logical names have no special meaning.

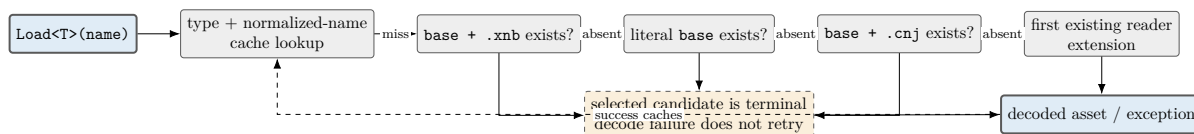


Figure 33.1: ContentManager candidate order at 1bb2145d. Existence selects a path; it does not prove that the selected reader can decode or use the asset.

A literal path wins over its sidecar. With `hero.png` and `hero.cnj` present:

```
1 auto native = content.Load<Texture2D>("sprites/hero.png");
2 auto sidecar = content.Load<Texture2D>("sprites/hero");
```

The first call selects the literal PNG after the extra XNB probe. The second selects `hero.cnj` before the native-extension list. A CNJ `sourceFile` then starts a checked nested load; it is not a direct decoder bypass.

33.1.1 Native extension order

Requested type	Ordered native candidates after literal and CNJ
Texture2D	.png, .jpg, .jpeg, .bmp, .gif, .tga, .tif, .tiff, .qoi
TextureCube	.dds
SoundEffect	.wav
Song	.mp3, .ogg, .wav, .flac, .opus, .aac, .wma
Video	.mp4, .ogv, .webm, .mkv, .avi, .mov; loose reader excluded on Emscripten, Android, and MinGW
Model	.cnj, .gltf, .glb; the first entry is redundant with the manager-level CNJ probe
SpriteFont, Effect, AnimationClipEXT, Curve	self-contained .cnj

The global CNJ probe runs even when a reader does not advertise or understand CNJ. In particular, the current loose `Song` and `Video` readers treat the selected path as media. An extensionless name with a same-named `.cnj` therefore feeds JSON to the media object instead of falling through to the native audio/video file. Pass the explicit media filename for those types; a generic media-sidecar contract does not exist at the pin.

33.2 RootDirectory and containment

`RootDirectory` is a base path, not a sandbox. `BuildAssetPath` joins it with the logical name but does not reject absolute paths, canonicalize dot segments, or check symlinks:

```
1 root = "Content"
2 "sprites/logo" -> "Content/sprites/logo"
3 "../shared/logo" -> "Content/../shared/logo"
4 "/srv/logo" -> "/srv/logo"
```

Top-level names must therefore be trusted or validated by the application. Internal references have narrower rules:

- CNJ `sourceFile` rejects absolute paths, traversal and symlink escape, another CNJ, and implicit sidecar cycles.
- `ContentReader::ReadExternalReference<T>` rejects absolute, drive, UNC, and normalized above-root paths.
- XNB `Song` and `Video` readers contain companion files within the content root. For an explicitly loaded external XNB, the XNB's own directory becomes the authorized bundle root.

These checks compare canonicalized paths before the eventual open; they are not a race-proof open-by-handle policy. Chapter 37 records the TOCTOU boundary.

33.2.1 Cache identity is textual

`NormalizeKey` converts backslashes to slashes and lowercases bytes. It does not include the root, resolve dot segments, or canonicalize a file. Consequences include:

- `Sprites\Logo` and `sprites/logo` share a cache key;
- `textures/../logo` and `logo` may name one file but occupy two entries;
- two case-distinct files on a case-sensitive filesystem collapse to one logical key;
- changing `RootDirectory` does not invalidate an already cached logical name.

Call `Unload` before intentionally reusing names under a new root.

33.3 Direct and managed loading

A direct texture constructor with a concrete path remains useful for a small program:

```
1 Texture2D logo("assets/logo.png", getGraphicsDeviceProperty());
```

`Game` instead exposes its configured manager in `LoadContent`:

```
1 ContentManager& content = getContentProperty();
2 content.setRootDirectoryProperty("Content");
3 auto logo = content.Load<Texture2D>("sprites/logo");
4 auto font = content.Load<SpriteFont>("fonts/hud");
```

`RegisterTypeReader<T>` installs one loose reader for a C++ type. `RegisterCnjLoader<T>` installs named CNJ factories that produce a type without a pre-existing reader. Both are CNAEXT surfaces; complete registration before loading begins.

33.4 Caching and ownership

Successful generic loads enter a strong cache keyed by requested type and normalized logical name. The write occurs only after a reader returns, so a failed top-level load can be retried. Specializations differ:

Type	Manager state	Ownership consequence
Ordinary copyable T	Strong <code>std::any</code> value	Repeated loads return copies of the cached value; caller-held shared handles can outlive eviction.
<code>Texture2D</code>	Weak renderer and optional pixel shadow	A live returned texture keeps its renderer alive; after eviction, a new load can construct a new renderer.
<code>SoundEffect</code> , <code>TextureCube</code>	No manager cache	Move-only values are decoded independently on each call.

`Unload` clears only the generic and texture caches. It does not dispose caller-held values, clear readers, reset the root/device/provider, or refresh the manifest. `Dispose` calls `Unload` once and marks the manager; later loads throw `std::runtime_error`. The default C++ destructor does not call that public method.

Nested reader activity is not transactional. A reader can cache dependencies and then fail, leaving those successful nested entries. There is no provisional key or cycle detector, so a recursive load of the same uncached type/name can recurse until stack exhaustion.

33.4.1 Game content assignment copies state

`Game::setContentProperty(const ContentManager&)` performs memberwise assignment into the existing manager. It copies caches, readers, manifest, disposed state, and raw service/device pointers; it does not replace a managed-object reference as FNA does. Assigning a standalone manager can overwrite Game's valid graphics-device pointer with null. Configure the existing manager in place when only the root should change:

```
1 auto& content = getContentProperty();
2 content.setRootDirectoryProperty("OtherContent");
```

33.5 Failure and concurrency boundaries

`ContentLoadException` covers many validated content errors, including malformed XNB/CNJ data, absent reader registration, bad reader indices, and unsupported reader versions. It is not a universal wrapper. Depending on the selected path, callers can also see `std::runtime_error`, `FileNotFoundException`, `EndOfStreamException`, `std::bad_any_cast`, or an exception from custom reader code. Empty names are not rejected up front.

```
1 try {
2     auto level = content.Load<GameLevelData>("levels/intro");
3     StartLevel(level);
4 } catch (const ContentLoadException& error) {
5     ShowContentDiagnostic(error.what());
6 } catch (const std::exception& error) {
7     ReportAssetFailure(error.what());
8 }
```

The manager's maps, manifest vector, strings, pointers, and disposed flag are unsynchronized. Concurrent load/unload/registration/root/manifest operations can form C++ data races. Single-threaded reentrancy also has hazards: self-loading cycles have no sentinel, and a reader that replaces its own registration can destroy the active object. Register and configure first, serialize manager mutation through one owner, and publish completed assets to workers.

33.6 Provider, ResourceContentManager, and TitleContainer

The `IServiceProvider*` constructors retain a raw pointer, but the pinned `ContentManager` never calls `GetService`. A GPU-backed standalone manager needs an explicit device:

```
1 ContentManager assets(&services, "Content");
2 assets.setGraphicsDevice(device);
3 auto logo = assets.Load<Texture2D>("sprites/logo");
```

`Game` performs that association for its own manager. Both pointers remain borrowed.

`ResourceContentManager` is a surface stub. Its protected `OpenStream` ignores the asset name and throws `std::runtime_error`; inherited `Load<T>` does not call that virtual method. The class therefore loads ordinary files only through its base behavior and does not provide embedded resources.

33.6.1 TitleContainer

`TitleContainer::OpenStream` is an independent direct-file helper. It converts backslashes, resolves relative names against the process-global `TitleLocation`, and returns a read stream.

`ContentManager` does not call it. The CNAEXT title-location setter is mutable global configuration and is unsynchronized.

Resolution is lexical rather than contained: parent segments, absolute paths, and symlinks can leave the title directory. Android adds an SDL asset fallback and copies the loaded bytes into a `MemoryStream`. The helper is suitable for application-controlled paths, not untrusted filenames or content-root redirection.

33.7 Content manifest

`GetContentManifest` lazily scans the root and returns a reference to manager-owned rows. `RefreshContentManifest` rebuilds them. A row records a logical path, XNB/CNJ presence, native extensions, and—for readable uncompressed XNBs—reader names. `GetXnbReaderUsageSummary` aggregates those names and reports whether the global registry currently contains each key.

The manifest is diagnostic only:

- Load performs fresh filesystem probes and never consults it.
- Presence does not establish a valid file, compatible reader, device, or loadable graph.
- Compressed or malformed XNBs can appear with an empty reader-name list.
- Directory and hash-map iteration make row and summary order unstable.
- Missing or inaccessible trees can yield an empty or partial snapshot without a public scan-status result.
- Refresh can invalidate held element references and iterators; copy before sorting or retaining the data.

Unload, disposal, and root changes do not automatically refresh this separate snapshot.

33.8 Audio and video readers

Loose readers and XNB readers solve different problems. Loose `Song` and `Video` simply wrap the selected media path (`Video` also requires the manager device). XNB readers decode the XNA-specific body and resolve an external companion.

33.8.1 SoundEffect

The XNB `SoundEffectReader` handles native PCM16 directly. PCM8, IEEE float, IMA-ADPCM, and MS-ADPCM are wrapped as an in-memory WAV and sent through SDL's decoder; XMA2 is rejected. MS-ADPCM content without its coefficient extension receives the standard coefficient table and a block-derived sample count. The full wire contract and loop-boundary defect are in §35.4.10.

33.8.2 Song

An XNB `Song` body contains a reference string and duration in milliseconds. The reader resolves the reference within the authorized root, strips the final four characters when long enough, and probes the extensionless stem, `.ogg`, `.oga`, then `.qoa`. If none exists, it restores the serialized path. This preserves the desktop substitution convention for XNA `.wma` placeholders; it is not general extension parsing.

Construction checks that the selected path exists, so a well-formed XNB with no companion fails during `Load<Song>` with `FileNotFoundException`. Decoding remains deferred until playback. The retained `MonoGame` fixture establishes XNB dispatch, sibling lookup, duration, and the `.ogg` route; it does not establish playback or the other substitutions.

33.8.3 Video

An XNB Video body contains a reference, duration, width, height, frame rate, and soundtrack enum. The reader contains the reference, probes the extensionless stem, `.ogv`, then `.ogg`, and stores the serialized metadata in a `Video`. It does not open or validate the media during content loading.

`VideoPlayer::Play` performs the FFmpeg open on supported native builds and compares stream dimensions and frame rate with the serialized metadata. Missing or unusable media can leave CNA stopped without the FNA-style file exception. The strongest test uses a test-built XNB and CNA's FFmpeg fixture; no external-producer Video XNB is retained. On platforms where the FFmpeg-backed implementation is excluded, declarations alone do not establish a usable route.

33.9 Porting implications

Use an extensionless logical name when XNB/CNJ precedence is desired; use a concrete literal filename when the native file itself must win. Treat the root and `TitleContainer` as trusted-path conveniences, not sandboxes. Register readers and attach a graphics device before loading, keep manager mutation serialized, and call `Unload` when deliberately changing the cache namespace.

Existing XNA/FNA games can reuse compiled assets only for reader families supported at this pin. Arbitrary Effect bytecode remains a separate unsupported contract (Chapter 17); CNJ or a native conversion is appropriate where the XNB reader or runtime capability is absent.

33.10 Summary

`ContentManager`'s contract is a deterministic existence-priority resolver with type-specific caching. XNB, literal, CNJ, and native-extension paths are not fallback attempts after decode failure. The distinction explains most surprising content behavior: explicit filename choice, root changes hidden by cache, sidecars shadowing native media, and a manifest that inventories candidates without certifying them.

Chapter 34

The XNB Container

An XNB begins with a small framing header, optionally decompresses into one object stream, and then hands that stream to the reader architecture in Chapter 35. Keeping the layers separate is important: a valid header does not make the object graph safe, and a correct type reader cannot repair a truncated or allocation-hostile container.

34.1 The ten-byte header

Offset	Field	Rule
0–2	magic	three separately read bytes: <code>XNB</code>
3	platform	one of the 16 FNA platform identifiers
4	version	exactly 4 or 5
5	flags	compression bits plus profile flags
6–9	total length	little-endian signed 32-bit file length

Reading the magic one byte at a time is a deliberate deviation from FNA’s implementation: a truncated stream throws `EndOfStreamException` at the missing byte rather than comparing partially initialized data against one packed magic constant.

The accepted platform bytes are FNA-exact: `w x m i a d X W n u p M r P g l`. MonoGame’s later post-fork identifiers are rejected. Compatibility here means accepting the historical FNA/XNA set, not every container another descendant also calls XNB.

34.2 Compression is four-valued

Two independent flag bits produce four outcomes:

Bits	Result	Runtime behavior
neither	<code>None</code>	object stream starts at byte 10

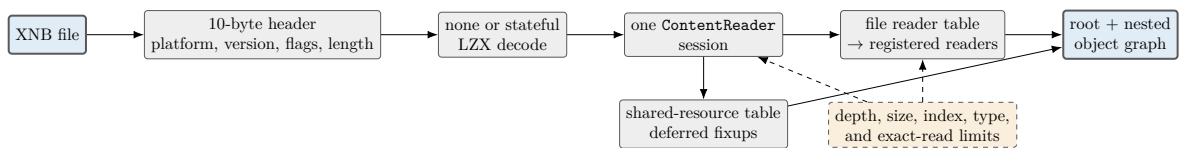


Figure 34.1: XNB container, reader-table, and object-graph layers. Passing an earlier boundary does not establish the later ones.

Bits	Result	Runtime behavior
0x80	Lzx	read decompressed-size hint and decode framed LZX blocks
0x40	Lz4	named <code>ContentLoadException</code> ; deliberately deferred
both	Unknown	reject instead of guessing which codec wins

The LZ4 bit comes from MonoGame’s implementation, while LZX is the historical XNA route. The reader keeps this provenance explicit rather than treating compression as one Boolean.

34.3 LZX preserves state across blocks

CNA’s decoder is a line-by-line C++ port of FNA’s `LzxDecoder.cs`. One decoder instance serves one file so the sliding window, repeated-offset LRU, and Huffman tables persist across the framed blocks. Constructing a fresh decoder for each block would pass trivial independent payloads and corrupt real multi-block streams.

The window exponent is bounded to 15–21; XNB uses 16. Block framing uses a two-byte big-endian compressed size, with the `0xFF` sentinel introducing the extended form and explicit frame size. After all blocks, the produced byte count must exactly equal the declared decompressed size.

The strongest decoder oracle is cross-implementation: CNA output is compared byte-for-byte with FNA’s unmodified decoder executed under Mono. That oracle found a real heap-buffer-overflow in the long-code table-growth path, a class of defect that self-consistency fixtures could miss.

34.4 The reader table starts inside the decoded stream

The object stream begins with a 7-bit-encoded reader count, followed by reader-name/version pairs. The count is bounded, each name is retained in raw and normalized form, and parse errors are translated into `ContentLoadException` so callers do not need to catch a parser’s `std::invalid_argument` separately.

34.4.1 Canonical names require a parser

A reader name may contain nested generic arguments, assembly qualifiers, version, culture, and public-key tokens. Splitting at the first comma is therefore wrong. CNA’s recursive parser tracks nested brackets, strips assembly metadata, and rebuilds a canonical `Base[[arg1],[arg2]]` key. Its recursion is bounded by the same object-depth limit used later by the graph reader; before that guard, a sub-megabyte crafted name could exhaust the C++ stack.

34.5 Limits and enforcement points

Limit	Value	Enforcement
<code>maxFileSize</code>	64 MiB	compressed payload before allocation; not the initial whole-file read
<code>maxDecompressedSize</code>	256 MiB	size hint and decoded output

Limit	Value	Enforcement
<code>maxStringBytes</code>	1 MiB	post-read type-reader-name bound
<code>maxTypeReaderCount</code>	4,096	reader table count
<code>maxObjectNestingDepth</code>	256	type-name and object-graph recursion
<code>maxSharedResourceCount</code>	1,000,000	shared-resource table

The first row is intentionally uncomfortable. `LoadXnbAsset<T>()` and the reader-name scanner read the entire file into memory before the compressed-payload guard runs. The declared 64-MiB limit therefore does not cap an uncompressed file or the initial allocation. It is a partially enforced control, not a complete file-admission firewall.

34.6 Length checks close the container boundary

The loader cross-checks the header's `totalLength` against the bytes actually read before using offsets into the body. Uncompressed content starts at byte 10; LZX adds the four-byte decompressed-size field and begins its compressed payload at byte 14. Truncation, impossible lengths, unsupported compression, and decoded-size disagreement all fail before object readers receive the stream.

This cross-check was added after a confirmed heap-buffer-overflow. The durable rule is broader: never trust the internal length merely because the filesystem supplied that many bytes, and never trust the filesystem length merely because an internal field points inside it. Validate the relationship in both directions.

34.7 What the container does not prove

A clean container says the envelope, compression, and reader table are structurally acceptable. It does not prove that reader versions are supported, nested object indices are in range, shared-resource fixups resolve, texture mip sizes match, or referenced paths remain inside their allowed root. Those checks belong to the object session and concrete readers in the next chapter.

Chapter 35

XNB Type Readers and Object Graphs

An XNB body is a serialized object graph. Its table names reader factories; one `ContentReader` session constructs the root and nested values, then resolves shared resources. Chapter 34 covers the envelope and compression. This chapter covers reader registration, graph semantics, built-in layouts, and their evidence.

35.1 Two registration systems

XNB readers use the process-wide `ContentTypeReaderManager`. Loose and CNJ readers use a per-`ContentManager` map. The similarly named extension points are not interchangeable:

Operation	Scope	Selection
<code>AddTypeCreator(name, factory)</code>	Process-wide XNB registry	Canonical reader name from the file's type-reader table.
<code>RegisterTypeReader<T></code>	One manager	Requested C++ type after XNB is absent.
<code>RegisterCnjLoader<T></code>	One manager	CNJ envelope type string through the generated generic CNJ reader.

CNA has no reflection fallback for XNB names. The global registry starts empty and is not populated by `Game` or `ContentManager`. Applications using built-in XNB content must register it before the first load:

```
1  #include "CNA/Internal/Xnb/XnbBuiltInReaders.hpp"
2
3  CNA::Internal::Xnb::RegisterAllBuiltInXnbReaders();
4  ContentManager content(nullptr, "Content");
5  auto model = content.Load<Model>("ship");
```

The umbrella call is idempotent. Closed generic collection readers beyond the known `SpriteFont` dependencies still require exact registration because C++ cannot construct an arbitrary `ListReader<T>` from reflection.

`AddTypeCreator` retains the first factory for an exact key; later registrations are ignored. `ClearTypeCreators` clears all families for all managers and is intended for test isolation. The map is unsynchronized, and empty names or empty factories are not rejected. Complete registration before concurrent loads.

35.2 One object-graph session

35.2.1 ContentReader contract

The manager constructs `ContentReader` at the body immediately after the container header. `ReadAsset<T>` then:

1. parses the type-reader table at the current stream position;
2. reads the shared-resource count;
3. deserializes one indexed root object;
4. reads every shared object;
5. runs queued typed fixups.

The constructor comment saying the table was already consumed is stale. A second `ReadAsset` does not rewind or reuse the first table. Treat one reader as one body session.

The reader borrows raw pointers to its stream and manager. Its base `BinaryReader` closes the stream by default but does not delete it. The manager's stack ordering is safe; a standalone caller must keep both referents alive and account for stream closure.

Initialization creates one fresh reader for every table entry, enforces its serialized-version policy, then calls `Initialize` in a second pass. The `ContentTypeReaderManager` passed to initialization is temporary; a custom reader must not retain that reference. The reader instances themselves die with the session.

35.2.2 Indices, shared resources, and external references

Object indices are one-based: zero means null/default, and positive n selects table reader $n - 1$. A zero for a non-default-constructible target with no existing value cannot represent C#'s `default(T)` and raises `ContentLoadException`. A reader/result type mismatch can instead surface `std::bad_any_cast`.

A positive shared-resource index queues a fixup. CNA reads all shared objects before running any fixup, enabling forward and cyclic references. Index zero queues nothing; a negative index also queues nothing because the implementation tests only `index > 0`. Out-of-range positive indices are rejected. Wrong typed values and callback exceptions are not normalized into one transaction, so reader-side effects can survive a later failure.

Disposal recording is narrower than its documentation. It recognizes only a statically typed `shared_ptr<U>` where `U` derives from `IDisposable`, and only when the caller supplied a callback. Normal manager XNB loading supplies none; type-erased shared reads skip tracking. Do not treat the reader as a general lifetime coordinator.

`ReadExternalReference<T>` returns empty for an empty string, otherwise requires a manager, confines the logical path, and re-enters ordinary `Load<T>`. Despite its template declaration, the implementation is linked only for `Texture2D` and `TextureCube` at the pin.

35.2.3 Limits

`XnbReadLimits` is applied at named sites, not as one aggregate budget. Reader-table and shared-resource counts are checked; collection and decoded-image limits apply when a reader calls their helpers. Object dispatch and nested generic names have a bounded depth, with RAII restoring the counter after failure. The type-reader-name bound is checked only after the underlying string allocation, and ordinary serialized strings do not use that same limit.

The manager reads an uncompressed file before the compressed-payload size control could help. Consequently these limits are useful defenses, not proof that every allocation or total object graph is bounded.

35.3 `ContentTypeReader` and type erasure

`ContentTypeReader<T>` lets an author implement typed `Read(ContentReader&, optional<T>)`. CNA's separate `ContentTypeReaderBase` erases that call to `std::any`. Canonical target names are strings, not checked runtime types; the eventual `any_cast` is the effective type check.

Copyable targets are stored directly in `any`. Move-only targets are boxed as `shared_ptr<T>`. Indexed graph dispatch unwraps both forms, which is how bare `SoundEffect` and `TextureCube` roots load. The public direct `ReadObject<T>(reader)` and `ReadRawObject<T>(reader)` overloads always cast to a bare `T`; they therefore fail for those move-only targets. Use indexed dispatch for them.

An existing instance is transported into the reader and a value is returned; the caller's original C++ object is not updated in place. Collection readers additionally differ: List appends to supplied elements, Dictionary clears first, and Array resizes. Duplicate dictionary keys are silently retained at their first value because CNA uses `unordered_map::emplace`; FNA's `Dictionary.Add` throws.

Closed collections create their named element readers from the global registry rather than using the initialized peer instance in the current file table. This works for stateless built-ins. Custom stateful or closed-generic readers must register and test their exact combinations.

35.4 Built-in reader families

Unless a reader widens `SupportsVersion`, CNA accepts serialized reader version zero only. FNA generally reads and ignores these reader-version integers. Version admission is therefore a CNA compatibility boundary separate from the XNB container version.

35.4.1 Scalar and math layouts

Primitive readers consume explicit little-endian binary operations, not host-memory layouts. Boolean is one byte; integers and IEEE floats use their fixed widths. XNB `Char` decodes one UTF-16 code unit from one to three UTF-8 bytes and rejects a four-byte code point. String is a 7-bit byte length followed by bytes in `std::string`; this layer does not validate UTF-8.

Math readers are literal field sequences:

- vectors, quaternion, and matrix read Singles in public component/field order;
- Color reads four RGBA bytes, not its packed integer;
- Plane, Point, Rectangle, Box, Sphere, Frustum, and Ray read their obvious constituent values and construct the public type.

They do not validate finiteness, normalization, nonnegative dimensions/radii, min/max ordering, or matrix invertibility. CNA also has no generic `EnumReader<T>` fallback; an asset naming one needs an exact application-provided factory.

Evidence is mostly local writer/reader symmetry. External `SpriteFont` fixtures exercise selected nested `Char`, `Rectangle`, and `Vector3` values, but no independent corpus covers every scalar/math layout or Unicode edge case.

35.4.2 Decimal, TimeSpan, and DateTime

Decimal reads the .NET lo/mid/hi/flags wire into a 96-bit mantissa, sign, and scale. It is not registered on MSVC-family builds because sharp-runtime uses `unsigned __int128`. Reserved flags are not validated, and negative zero is canonicalized, unlike the managed reference behavior.

TimeSpan preserves a signed 64-bit tick count. DateTime preserves the lower 62 tick bits but drops the two-bit `DateTimeKind`; kind value three is also admitted after masking where the managed constructor rejects it. The focused tests establish ordinary positive values, not bit-for-bit Decimal or DateTime metadata fidelity across compilers.

35.4.3 CurveReader

CurveReader consumes `PreLoop`, `PostLoop`, signed key count, then each key's position, value, precomputed incoming/outgoing tangents, and continuity. It does not call `ComputeTangents`.

Enum integers are cast without range checks. A negative count becomes an empty loop, while a large positive count bypasses the common collection ceiling. NaN, infinity, duplicate positions, and invalid loop/continuity values are admitted. Direct existing-value use appends keys without clearing. The retained test uses a locally written ordinary body; no external XNB Curve fixture or manager-level integration artifact is present.

35.4.4 Texture2DReader

The payload is surface format, width, height, level count, then a byte count and payload for every mip. CNA accepts Color and DXT1/3/5; DXT data are software-decompressed to RGBA before upload. Dimensions, topology, checked decoded size, device limit, and exact per-level byte counts are validated.

Version-4 legacy mapping is narrow, and a recognized `ColorBgraEXT` mapping still reaches the reader's accepted-format refusal. CNA does not perform FNA's Xbox 360 texture byte swaps. External-producer evidence includes an exact one-pixel MonoGame Color XNB, an LZX Color texture checked for nonuniform output, and the DXT3 atlas nested in SpriteFont. No retained DXT1/DXT5, version-4, or Xbox Texture2D artifact establishes those paths.

35.4.5 Texture3DReader

Texture3D uses a copyable `shared_ptr<Texture3D>` target. Its wire adds depth and stores one blob per mip. Color and DXT1/3/5 are supported; compressed volume data are divided into depth slices and decompressed per slice. Dimensions and decoded multiplication are checked.

Compressed byte counts need not be exactly divisible by depth or equal the required block total; extra complete data and a remainder can be ignored after required slices succeed. No aggregate mip budget or per-axis device limit is imposed here. Texture creation cleanly refuses a renderer that reports no volume-texture capability. Evidence is a locally written small Color body with renderer-dependent success/refusal; there is no external Texture3D XNB or compressed-volume fixture.

35.4.6 TextureCubeReader

TextureCube reads format, base size, level count, then six faces in XNA face order, each with all mips. It supports Color and DXT1/3/5 and validates raw Color sizes. Compressed levels require enough blocks but can contain surplus bytes. The base decoded-face cap is not an aggregate six-face/mip-chain budget, and there is no pre-allocation cube-capability query.

A retained MonoGame DXT1 cube travels through ContentManager across all faces and seven levels. The test samples only selected nonuniform and terminal-level outcomes; it does not establish exact pixels, orientation, DXT3/5, or every face.

35.4.7 SpriteFontReader

SpriteFont is a nested graph: Texture2D atlas; Rectangle glyph and crop lists; Char list; line spacing; float spacing; Vector3 kerning list; then optional default character. The umbrella registers its three closed list dependencies and Texture2D.

Count limits apply per list, but the reader does not require parallel lists to have equal lengths, validate rectangles against the atlas, require finite spacing, or reject duplicate characters. The runtime lookup keeps the last duplicate index. Direct existing-font input is unsupported.

Two externally produced MonoGame fixtures establish the ordinary graph: an uncompressed DXT3 font and an LZX-compressed font. They are strong layout and compression evidence, not malformed font validation.

35.4.8 ModelReader

ModelReader reconstructs declarations, vertex/index buffers, bones, meshes, parts, and their shared buffer/effect references. Counts and major byte multiplications are bounded. Vertex declaration offsets/enums versus stride, index-byte divisibility, draw ranges, and much mesh metadata remain trusted.

The serialized parent field on each bone is consumed but ignored; parent links are formed from child lists. Contradictory lists can therefore overwrite a child's parent or disagree with the serialized scalar. Tags are consumed to keep alignment but any non-null tag is rejected.

The retained uncompressed MonoGame cube establishes two bones, one mesh/part, buffers, a BasicEffect, and shared-resource fixups. Deterministic whole-container mutation exercises clean failure around that artifact. It does not validate arbitrary hierarchy/draw metadata or prove that every accepted graph is safe to draw.

35.4.9 Stock-effect readers

Five readers construct **BasicEffect**, **AlphaTestEffect**, **DualTextureEffect**, **EnvironmentMapEffect**, and **SkinnedEffect** from their ordinary FNA-shaped material fields. They require the manager's GraphicsDevice and return `shared_ptr<Effect>`; textures are external Texture2D/TextureCube references with CNA's logical containment.

The general **EffectReader** is separately registered for a length-prefixed XNA/FNA Direct3D 9 Effect Framework binary. It requires a GraphicsDevice, reads at most 64 MiB exactly, and calls the public **Effect** byte-buffer constructor. FNA3D accepts it in the normal build; SDL_GPU, EasyGL and Vulkan require their compiled-effects option. A renderer whose **CompiledEffects** capability is false yields a wrapped **ContentLoadException**. This route is distinct from the five field-wise stock readers and does not accept .fx source, HLSL source, DXBC, or MGFX.

The tag's `XnbBuiltInReaders.hpp` umbrella comment still calls the general reader a "known-unsupported" registration, but `EffectContentTypeReader.cpp` constructs the effect as described above. The implementation and focused tests supersede that stale comment.

The external Model fixture establishes its nested BasicEffect. The other four have locally built field-order tests but no external-producer root XNB or texture graph. Reader success also does not establish identical field consumption by every renderer; Chapter 15 owns draw-time behavior.

35.4.10 SoundEffectReader

SoundEffectReader consumes a WAVEFORMAT block, audio bytes, loop start, loop length, and duration. PCM16 enters the native raw-buffer route. PCM8, IEEE float, IMA-ADPCM, and MS-ADPCM are wrapped as WAV and decoded by SDL; missing MS-ADPCM coefficient data is synthesized from the standard table. XMA2 is rejected. Xbox WAVEFORMAT fields receive the same selective byte swaps as FNA.

Loop metadata is not bounded against decoded audio. In the wrapped-WAV route, a positive loop computes signed `loopStart + loopLength` before writing the WAV sample-loop endpoint; an extreme positive pair can overflow signed Int32. Ordinary loop metadata is tested, but this adversarial edge is not.

External PCM16 and ADPCM fixtures establish several decode paths; move-only root dispatch is also covered. Those artifacts do not establish XMA2, arbitrary WAVEFORMAT extensions, malformed loop safety, or every platform decoder.

35.5 Registration and platform inventory

`RegisterAllBuiltInXnbReaders` registers primitives, math, Decimal/DateTime/TimeSpan, Curve, 2D/3D/cube textures, SpriteFont, SoundEffect, Song, stock effects, Model support, and the renderer-qualified general Effect reader. Video is included only when FFmpeg is compiled. Decimal availability also varies by compiler. A single unconditional reader count would misrepresent those builds.

Song and Video wire behavior is in §33.8.2 and §33.8.3. The registration inventory establishes factory presence; it does not establish that a companion media file decodes or that platform playback is linked.

35.6 Rules for custom readers

A custom binary reader should preserve these session rules:

1. register the exact canonical XNB name before loading;
2. treat zero as the null/default object index and positive indices as one-based;
3. declare a serialized-version policy deliberately;
4. use indexed object dispatch for polymorphic and move-only values;
5. bound counts and decoded sizes before allocation;
6. queue shared-resource fixups and run them only after all shared objects are read;
7. confine external references and re-enter ContentManager for their cache/resolution policy;
8. keep reader, temporary manager, stream, and owning manager lifetimes within the session.

Field-by-field decoding can appear correct while violating identity, null semantics, lifetime, or resource bounds. A complete test should therefore include a real container table, manager load, nested/shared references where applicable, a failure case before allocation, and an observable result beyond successful construction.

35.7 Summary

CNA implements a substantial XNB object graph without .NET reflection. That strength creates explicit boundaries: applications must register readers; versions are stricter than FNA; closed generics are finite; move-only values use a separate erased representation; and reader-specific validation varies. The per-family evidence above should be read as a set of scoped contracts, not as one claim that all XNB content is interchangeable.

Chapter 36

CNJ: CNA's Own Content Format

CNJ is ordinary JSON. It has no magic bytes, binary primitive layer, 7-bit integer encoding, or shared-resource object graph. The earlier `.cnb` name was retired precisely because it suggested a binary container. File extension, envelope fields, and type-specific readers provide the contract instead.

36.1 A small envelope, validated separately

Three top-level fields are recognized before type-specific parsing:

Field	JSON type	Meaning
<code>cnjVersion</code>	number	integer format version, bounded by a per-type ceiling
<code>type</code>	string	cross-check against the caller's requested C++ type
<code>sourceFile</code>	string	optional delegation to a native file for selected readers

`ParseCnjEnvelope()` records failure in `parseErrorDetail`; it does not throw. Validation is a separate step. The parser retains both the truncated integer and raw numeric value so 1.5 cannot be silently accepted as version 1.

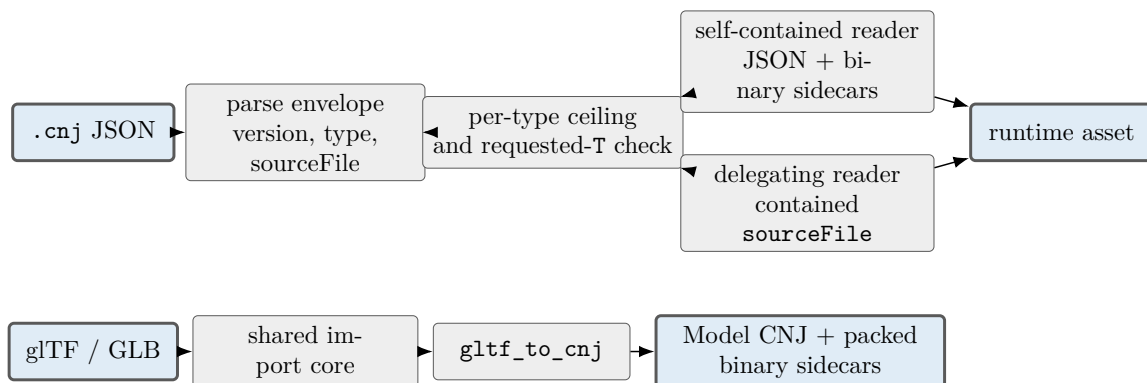


Figure 36.1: CNJ runtime and offline-tool routes. The envelope is a type/version cross-check, not a reflection-based object factory.

36.2 Version ceilings belong to types

The old documentation's global `cnjVersion == 1` rule is no longer true. `ValidateCnjEnvelopeBaseline()` accepts a maximum supplied by the reader. Most formats retain a ceiling of 1; Model admits version 2 for its bone hierarchy and per-mesh parent-bone fields. Raising one type's ceiling does not widen any other type.

Dispatch still begins from the caller's compile-time `T`. The envelope's `type` field is a cross-check, not a reflection-based factory selector. A request for `Load<Texture2D>` against a Model envelope fails naming both sides instead of constructing whatever the file asks for.

36.3 Built-in type vocabulary

Current readers recognize `Texture2D`, `TextureCube`, `Texture3D`, `SoundEffect`, `SpriteFont`, `AnimationClip`, `Curve`, and `Model` envelopes. The effect reader additionally distinguishes `Effect`, `BasicEffect`, `AlphaTestEffect`, `DualTextureEffect`, `EnvironmentMapEffect`, and `SkinnedEffect`.

That vocabulary is not the same as the native-extension matrix. A Model reader can also accept `.gltf` and `.glb`; a texture reader can accept image formats. CNJ is one candidate in `ContentManager`'s resolution order, not a universal wrapper required around every loose asset.

36.4 `sourceFile` is a capability, not a convention

Readers fall into two enforced groups:

Policy	Types	Result
required delegation	<code>Texture2D</code> , <code>TextureCube</code> , <code>SoundEffect</code>	absence is an error; reader loads the referenced native file
self-contained	<code>Texture3D</code> , <code>SpriteFont</code> , <code>AnimationClip</code> , <code>Curve</code> , <code>Model</code> , <code>SkinnedModel</code> , <code>Texture2D</code> color keys, and the custom-GLSL effect branch	presence is an error; JSON and declared sidecars own the data

`ResolveCnjSourceFileSafely()` rejects an empty path, an absolute path, a component-wise escape after weak canonicalization, a target whose extension is itself `.cnj`, and a native-looking target that would resolve back to a sibling CNJ through normal selection. The last rule closes an implicit self-cycle, not just the obvious `sourceFile: "x.cnj"` case.

This containment applies to a reference inside the content document. It does not contradict the top-level `ContentManager` contract that permits a caller to supply an explicit absolute asset name.

36.5 Two JSON implementations coexist

Newer readers use the recursive-descent `CNA::Internal::ParseJson` implementation, which has strict number grammar, escape handling, and surrogate-pair support. Older readers still use field-specific substring scanners and manual brace-depth loops. `SpriteFont`, `Model`, `SkinnedModel`, `Texture2D` color keys, and the custom-GLSL effect branch remain on that legacy path.

The split is visible behavior. A malformed document can be rejected by the real parser while a legacy scanner accepts a partial structure. In particular, the `SpriteFont` glyph loop stops when

it runs out of opening braces, so truncation can yield a partial font rather than a uniform parse error.

Neither path currently has a document-size cap, and the recursive parser has no nesting-depth limit. CNJ therefore lacks the object-depth, string, decompressed-size, and fuzzing boundaries now present on XNB. Chapter 37 treats that asymmetry as an open boundary, not as proof of an exploit CNA's audit did not execute.

36.6 Custom loaders receive raw JSON

`RegisterCnjLoader<T>(typeName, factory)` installs a deterministic per-type extension. The factory receives the raw document text and the manager, not a parsed JSON value. Empty names or factories fail, a type that already has a loose reader fails, and duplicate `(T, typeName)` registration fails rather than silently replacing the earlier factory.

The first custom registration for a type lazily installs a private generic CNJ reader. That reader validates the baseline envelope, then dispatches on the envelope type name. This is the exception where the type string selects among user factories; the outer `ContentManager::Load<T>` still fixes `T`.

36.7 Binary sidecars remain format-specific

Plain JSON does not mean every payload is textual. Model and animation content can reference raw vertex/index blobs and `.skeleton.bin/.clip.bin` sidecars. A bespoke binary reader decodes their fixed layouts. Bone counts are sanity-bounded, while an optional appended skeleton-root block is detected by remaining bytes rather than a tagged field.

That compatibility pattern is useful but fragile: a trailing block can only be distinguished from truncation or a future extension through expected remaining size. New sidecar evolution should prefer an explicit tag/version rather than another implicit length test.

36.8 Production and provenance

The graphics-free `gltf_to_cnj` developer tool can produce Model CNJ plus sidecars and is built independently of a window or graphics device. Runtime Model loading can also consume `.gltf` and `.glb` directly through the same import core, making the offline conversion optional for the common path.

No CNJ file is checked into the pinned CNA source tree; tests generate them in temporary directories. That means parser and envelope behavior has synthetic coverage, while long-term compatibility with a versioned real-world CNJ corpus is not yet established. When shipping CNJ, record the producing tool revision and keep representative assets under version control outside the transient test-only corpus.

36.9 The format in one rule

Treat CNJ as a typed JSON contract whose version, source-file policy, parser, and sidecars depend on the requested asset type. Do not infer safety or evolution rules from the extension alone, and do not describe it as binary merely because some readers own binary sidecars.

Chapter 37

Content and Asset Boundaries

Content loading combines attacker-controlled lengths, recursion, compression, paths, decoders, and GPU allocations. CNA’s XNB side has undergone a substantial hardening campaign; CNJ and glTF retain explicitly narrower controls. This chapter states the enforcement points and the remaining gaps without turning either into a blanket claim of safety or insecurity.

37.1 Limits are useful only where they are enforced

Control	Bound	Enforcement point	Status
XNB compressed payload	64 MiB	before decompression allocation	partial: initial file read uncapped
XNB decoded payload	256 MiB	hint and final decoded size	enforced
XNB type-reader name	1 MiB	after <code>ReadString</code>	enforced post-read
reader count	4,096	type-reader table	enforced
object/type nesting	256	RAII graph guard and type-name parser	enforced
shared resources	1,000,000	table count before fixups	enforced
CNJ document size	none	whole text read	open
CNJ JSON depth	none	recursive parser	open

The 64-MiB row is the canonical warning against reading a constant name as a guarantee. `maxFileSize` protects a compressed payload after the entire XNB file has already been read into a string. An oversized uncompressed file can therefore allocate before the guard.

37.2 Top-level names and internal references have different trust rules

`ContentManager::Load<T>` deliberately accepts an explicit absolute asset name; `RootDirectory` is a base, not a sandbox. Containment applies instead to paths supplied *inside* content. Logical XNB external references reject absolute spellings and normalization above the content root before re-entering the manager. Filesystem-shaped CNJ `sourceFile`, model sidecars, and embedded media paths add weak-canonical symlink checks against their authorized root.

Across those routes, the containment tests cover traversal, absolute and drive paths, Unicode-shaped input, and existing-symlink escapes on the filesystem-aware paths. They also assert

that rejection occurs before file access and does not poison the manager cache. Legitimate . segments, repeated separators, and . . that normalize back inside the root remain allowed.

Symlink checking is not claimed to be race-proof. Weak canonicalization followed by later open has a time-of-check/time-of-use window. The correct statement is that resolved escapes are rejected in the tested filesystem state, not that an adversarial concurrent filesystem cannot change between check and use.

37.3 Object graphs are bounded on both recursive axes

XNB contains two distinct recursion surfaces. Canonical reader names can nest generic arguments, and object data can recursively dispatch nested readers. Both are capped at 256. The object path uses an RAII guard so depth is decremented on success and on every exception; otherwise one failed read could poison the session's remaining depth budget.

Shared resources use a separate two-pass discipline. Every shared object is read before any queued fixup runs, preserving identity even when objects refer forward. Counts and indices are validated before lambdas capture them.

37.4 Fuzzing reaches the public loader

The strongest container fuzz target mutates complete real XNB fixtures by flipping, truncating, overwriting, and inserting bytes, then loads them through the real `ContentManager::Load<T>` path. Acceptable outcomes are success or an enumerated clean exception; `std::bad_alloc` is explicitly treated as an allocation-guard failure, not a valid rejection.

A second target mutates LZX payloads and the decoded-size hint. Any successful decode must still produce exactly the declared length. These targets found memory-safety defects that review did not, including table growth and malformed texture payload paths.

Fuzzing scope is not universal. CNJ has no equivalent document fuzzer, no size limit, and no depth limit. glTF relies on cglTF parsing plus CNA's extraction tests, but production code does not call `cglTF_validate`. Those are remaining coverage boundaries.

37.5 Differential oracles reduce self-consistency risk

The LZX decoder is compared byte-for-byte with FNA's unmodified C# decoder under Mono. This matters because a compressor and decoder sharing the same mistaken assumption can agree with one another. A separate implementation gives the bytes an external reference.

Texture and audio readers use other independent checks: decoded byte counts against block-rounded format expectations, duration metadata as a broad audio sanity oracle, and real fixtures from multiple encodings. Each oracle has a stated scope; duration agreement cannot prove waveform identity, and storage round-trip cannot prove shader sampling.

37.6 Error normalization and its limits

Container and reader errors are generally translated to `ContentLoadException` with asset context. Parser-specific `std::invalid_argument` is caught at the reader-table boundary. Some glTF extraction failures still escape as plain `std::runtime_error`, and custom CNJ factories can throw arbitrary exceptions. A caller should not assume every failed `Load<T>` has one dynamic exception type.

37.7 Current asymmetries to keep visible

- the initial whole-XNB read is uncapped despite `maxFileSize`;
- CNJ has neither document-size nor recursion-depth controls and mixes strict and legacy parsers;
- SpriteFont does not cross-check the lengths of its parallel glyph, crop, character, and kerning tables;
- Texture3D/TextureCube mip topology and compressed-byte exactness remain weaker than the Texture2D reader's checks;
- symlink containment is not TOCTOU-proof;
- a low-severity misaligned-load issue remains open in the vendored cgltf path.

The remediation record is strong precisely because these gaps are not erased from the account. One critical and several high-severity content findings were closed; future work should extend the same observable limits and fuzz discipline to the still-asymmetric formats.

Part VI

3D Content: Models, glTF, and Conformance

Chapter 38

Models, Meshes, and Drawing

38.1 The runtime hierarchy

`Model` owns a `ModelBoneCollection` and a `ModelMeshCollection`; each `ModelMesh` owns a `ModelMeshPartCollection` and a `ModelEffectCollection`; each `ModelMeshPart` references a shared `VertexBuffer / IndexBuffer` pair and an `Effect`. This is the same shape as real XNA. The reader-side contract is deliberately separate (and is covered by §38.4 below and Chapter 33); at runtime, the useful distinction is between a normally loaded, internally consistent model and a hand-built one. The latter can use several CNAEXT construction/mutation hooks, so it must preserve the invariants that a content reader normally supplies. The exact boundary, including the remaining intentional C++ safety deviations from FNA, matters more than a blanket claim of zero gaps. `Model` exposes `CopyAbsoluteBoneTransformsTo`, `CopyBoneTransformsFrom / To`, and a convenience `Draw(world, view, projection)`; one small, intentionally-kept deviation from FNA is that CNA's `Copy*BoneTransforms*` methods loop by `Bones.Count` rather than trusting the caller's array length the way FNA does — strictly safer against an oversized destination array, never a compatibility problem for correctly-sized callers.

`ModelBone` carries `Name`, `Index`, a get/set `Transform`, its `Parent`, and a `Children` collection built via a CNAEXT `AddChild`. CNA also exposes setters for a `ModelMesh`'s `BoundingSphere` and `ParentBone`; FNA reserves both setters for its internal reader. They are useful for CNA hand construction and the XNB reader, but turn reader-populated metadata into caller-maintained state, which is exactly where the two model loaders and the draw contract below diverge sharply.

38.2 Two ways to draw a loaded model

The simplest path is the convenience method already named above:

```
1 model.Draw(worldMatrix, camera.View(), camera.Projection());
```

This is correct for a loaded, renderable model with at least a root bone, even if it has no interesting hierarchy to speak of. A real, multi-bone model (a character, a vehicle with independently-animated wheels) needs each mesh drawn with its *own* absolute bone transform composed into world space first — the classic pattern, grounded directly in the real `CopyAbsoluteBoneTransformsTo` and `IEffectMatrices` signatures:

```
1 std::vector<Matrix> boneTransforms(model.getBonesProperty().  
   getCountProperty());  
2 model.CopyAbsoluteBoneTransformsTo(boneTransforms);  
3  
4 for (ModelMesh* mesh : model.getMeshesProperty()) {
```

```

5      Matrix meshWorld = boneTransforms[mesh->getParentBoneProperty()
        ->getIndexProperty()]
6          * worldMatrix;
7
8      for (Effect* effect : mesh->getEffectsProperty()) {
9          if (auto* matrices = dynamic_cast<IEffectMatrices*>(effect))
10             {
11                 matrices->setWorldProperty(meshWorld);
12                 matrices->setViewProperty(camera.View());
13                 matrices->setProjectionProperty(camera.Projection());
14             }
15      mesh->Draw(); // issues actual indexed GPU draws for each valid
        ModelMeshPart.
16  }

```

`boneTransforms[mesh.getParentBoneProperty()->getIndexProperty()]` selects the mesh's parent-bone absolute transform, not bone 0 or the model root. The former loose-JSON route assigned one default bone to every mesh and welded parts together. That defect is fixed; the example applies to current XNB, CNJ, and direct glTF/GLB outputs.

38.3 The draw contract: a model is more than a bag of pointers

`Model::Draw` is deliberately a thin convenience path, not a validator or a general scene graph. It retains one `thread_local` static vector of absolute bone matrices (the same scratch-buffer shape as FNA's static array, but one instance per thread), grows it only when a later model on that thread has more bones, copies the absolute transforms into it, then visits every mesh. For every distinct effect listed by that mesh it requires the matrix interface, writes `boneMatrix * world`, `view`, and `projection`, and only then asks the mesh to issue its part draws. An effect in the mesh's effect collection that does not implement that interface is therefore a hard `runtime_error`; it is not silently ignored or drawn with some other matrix convention.

That compact loop establishes three input invariants which a normal content load satisfies, but which the public/extended hand-built API does not enforce:

- A renderable mesh needs an applicable bone matrix. CNA deliberately treats a null `ParentBone` as bone index 0, whereas FNA immediately dereferences the mesh parent to obtain its index and fails for null. This is a sensible C++ safety improvement for a one-root-bone hand-built model, and the real EasyGL/Bgfx hierarchy pixel tests exercise it for their root mesh. It does *not* make a zero-bone model renderable: with a positive-effect mesh, index 0 is outside an initially empty scratch vector; after another model has grown that thread's vector, it can instead reuse that earlier model's stale slot. FNA's static array is likewise retained, but its usual null `ParentBone` path fails first rather than recovering. Give every model that can draw at least one bone, and give each mesh a parent explicitly when it is not that root.
- `CopyAbsoluteBoneTransformsTo` computes slots in collection order. A child therefore has to name a parent whose result was already written; it neither topologically sorts nor detects cycles, duplicate parents, or an index that disagrees with the vector position. The XNB reader's malformed-graph boundary is documented in Chapter 33; the same ordering rule applies to a hand-built graph.
- The manual loop in the previous section intentionally dereferences the mesh parent pointer so that an invalid attachment is visible at the call site. Unlike `Model::Draw`,

it has no null-to-root fallback. Well-formed outputs of the current loaders assign a mesh parent, but an XNB shared-resource reference of zero can still leave it null; a manually assembled mesh must either set one or choose and check its root fallback before indexing `boneTransforms`.

`ModelMesh::Draw` has a different, narrower safety rule: it skips a part with a null `Effect` or a non-positive `PrimitiveCount`. FNA skips only the latter, then dereferences a null effect, so the CNA skip is another deliberate no-undefined-behaviour deviation. It does *not* validate the remaining draw fields: a positive-count part still passes its raw vertex buffer, index buffer, offsets, vertex count, start index, and primitive count to `GraphicsDevice`. A missing buffer or invalid range consequently fails at the graphics-device/renderer boundary rather than becoming a harmless skipped part. This is why a model's successful construction is not proof that it is drawable.

38.3.1 Keep parts and effects connected through the part setter

The mesh's `ModelEffectCollection` is a derived, distinct-effect view of the effects used by its parts; it is what `Model::Draw` configures before `ModelMesh::Draw` applies each part's current technique. The supported way to maintain that relationship is:

```
1 part.setEffectProperty(&basicEffect); // adds basicEffect to mesh.
   Effects if absent
2 // ...
3 part.setEffectProperty(nullptr);      // removes it only if no other
   part still uses it
```

The implementation first retains an old effect while any sibling part still points to it, then removes it only for the final user; it adds a new non-null effect only if the collection does not already contain that pointer. Thus two parts sharing one `BasicEffect` produce one matrix setup in `Model::Draw`, while each part still produces its own indexed draw. The focused part/effect unit tests cover exactly this retain-on-shared/release-on-last-user transition and the no-duplicate case.

`ModelEffectCollection::Add/Remove` and `ModelMesh::getEffectsPropertyMutable()` exist only as CNAEXT escape hatches. Unlike FNA's internal-only collection mutation, CNA exposes them to make its content reader possible. They neither deduplicate `Add` nor repair a collection that a caller has manually made inconsistent with its parts: a stale extra effect is configured but never applied by a part, and a duplicate is configured twice. Do not populate `Effects` directly in ordinary game code; set each part's effect and retain the buffers/effects for at least as long as the model uses their raw pointers.

38.3.2 Thread-local scratch and evidence boundary

At alpha.1 the bone-matrix scratch vector is `thread_local`. Draws on different threads therefore no longer resize or overwrite the same scratch storage. That change does not make a graphics device, its effects, buffers, or a shared mutable model graph generally thread-safe: concurrent mutation of bones, effects, or parts remains ordinary shared-state contention, and renderer resources retain their platform-specific thread-affinity rules. Keep a model's update and draw sequence on its owning graphics thread unless the application supplies a stronger synchronization contract.

The focused CPU tests cover transform multiplication, destination-size checks, constructor parent wiring, and part-to-effect collection bookkeeping. Renderer tests go further: the EasyGL root/hierarchy model tests read back pixels, Bgfx has equivalent two-mesh/hierarchy tests,

and the `SDL_Renderer` test proves that a fully populated positive-primitive model reaches the shared 3D draw failure boundary. No located test deliberately covers zero-bone rendering, invalid parent order/cycles, raw out-of-range part fields, direct mutable-effect desynchronisation, or concurrent `Draw`; treat those as construction-time invariants, not tested recovery paths.

38.4 Four model asset routes, not two

The old two-loader and `.model.json` framing no longer describes the resolver. CNA now has four distinct model-producing paths: the wire-compatible XNB `ModelReader`, self-contained `.cnj`, direct `.gltf/.glb`, and the older Avatar-specific `.skinnedmodel.json` reader. The first three return the ordinary `Model`; the fourth returns `SkinnedModelEXT`, a separate runtime type that must not be mistaken for another serialization of the same graph. For an extensionless cache miss, `ContentManager` looks for `name.xnb`, then a literal `name`, then `name.cnj`, `name.gltf`, and `name.glb`. Thus a same-named CNJ hides a glTF/GLB asset, and an XNB sibling hides both; passing an existing literal `name.gltf` explicitly reaches that literal before the extensionless sidecar probe. The full resolver contract is in Chapter 33; the resulting runtime graphs are the important distinction here.

The real `.xnb ModelReader` (`CNA::Internal::Xnb::ModelReader`) is wire-compatible with real XNA/MonoGame/FNA compiled model assets. It reconstructs the full bone hierarchy, assigns each mesh's serialized `ParentBone`, computes its serialized `BoundingBoxSphere`, and resolves shared `VertexBuffer/IndexBuffer/Effect` resources. Its exact malformed-graph, draw-field, tag, and shared-resource boundary is covered in Chapter 33.

The self-contained `.cnj ModelTypeReader` route validates a `Model` CNJ envelope and rejects `sourceFile`; the former `.model.json` name survives only in historical comments and test-target names, not in this reader's extension list. A version-2 descriptor reconstructs its complete parent-before-child `bones` array and attaches every mesh to the indexed `parentBone`; this is how the converter preserves the selected glTF scene hierarchy and rigid placement. A version-1 descriptor (or one without a general hierarchy) retains the older compatibility shape: one root plus a synthetic named child bone per mesh. An optional `skeleton/animations` sidecar pair builds `SkinningData` on `Model.Tag`, and morph data lives on the corresponding part's `Tag`.

This route still leaves `BoundingBoxSphere` degenerate and `ModelMesh.Tag` null. It allocates a fresh vertex/index buffer and built-in stock effect for each described mesh part; a named custom effect instead goes through the manager's cached `shared_ptr<Effect>` route. This is not XNB's serialized shared-resource graph, even where repeated custom-effect names happen to share a cached object.

The direct `.gltf/.glb ModelTypeReader` route parses glTF 2.0 and its buffers without producing a CNJ or binary sidecars first. It reuses the offline converter's import core, including sparse-accessor-safe extraction, topological skin ordering, material/image extraction, and morph/animation data. Its runtime shape is one `Load<Model>` containing the selected scene's mesh placements, including independent skin groups, with a fixed unit scale of 1.0. `Model::SkinsEXT` preserves each skin-to-mesh mapping while the legacy `Model.Tag` convention aliases the first skin. Convert offline when a scale factor or one-output-per-group asset layout matters.

Direct glTF creates an identity synthetic root followed by one `ModelBone` per reachable node of the selected scene, in parent-before-child order. A rigid mesh is attached to its instancing node; a skinned mesh is attached to the synthetic root so the skin palette does not double-apply the mesh-node transform. The separate reordered skin hierarchies are

retained through `Model::SkinsEXT`; morph targets and their tracks are retained on each `ModelMeshPart.Tag`. Imported cameras live in `Model::CamerasEXT`; material variants have a source-order selection API; and `GltfImportReportEXT` records counts plus named dropped/approximated features. Extracted images are decoded in memory and cached by glTF image pointer during that one import, but mesh buffers and effects remain per primitive.

The legacy `.skinnedmodel.json` route dispatches to `SkinnedModelTypeReader` and produces `SkinnedModelEXT`, the Avatar-oriented runtime described later in Chapter 41. It is neither a CNJ alias nor the historical name of the current Model descriptor. Code loading it opts into a different public type, animation representation, and draw path; its continued presence is therefore a compatibility route, not evidence that the generic `Model` reader accepts a fourth file spelling.

The evidence is much stronger than the old chapter wording suggested. The focused CNJ tests cover descriptors, envelope rejection, report persistence, variants, cameras, and shared clips. The generated glTF corpus and its L1–L7 ladder cover scene graphs, multiple skins, rigid and skeletal animation, morphs, material state, and pixels; Chapter 43 states the exact denominators. Those tests do not make the four routes interchangeable: use XNB for compiled XNA asset fidelity, CNJ for a controlled self-contained/imported description, direct glTF/GLB for an in-process glTF 2.0 import, and the legacy route only when the application consumes `SkinnedModelEXT`.

38.5 Lifetime and copies: value syntax, one mutable graph

`Model` is a copy-constructible C++ value, but copying it is not a deep model clone. Its compiler-generated copy operation copies the two collection vectors, `Root`, and `Tag` as raw pointers, while copying its private type-erased `shared_ptr<void>` ownership handle. The vectors are separate little wrappers after the copy, but every `ModelBone`, `ModelMesh`, `ModelMeshPart`, buffer, effect, and the graph below them is the same object. A bone-transform or part/effect mutation observed through one copy is therefore observed through every copy; this is shared mutable model state, not a way to pose or material-override one instance independently.

Each normal reader intentionally fills that last ownership handle before returning. The XNB reader's private bundle retains unique bones, meshes, and parts plus shared vertex buffers, index buffers, and effects. The CNJ and direct-glTF routes share a corresponding bundle which also retains their buffers, graph objects, effects, decoded textures, skinning data, and morph data. That is why a cached or caller-held loaded `Model` can safely keep the raw pointers used by its collections: the bundle itself is shared among every copy. It also explains why CNJ/glTF's `Model.Tag` and part Tags remain valid while such a model lives — their `SkinningData` and morph objects are in that same bundle. XNB rejects non-null serialized Tags, so it has no analogous reader-provided Tag object to retain.

The visible cache behavior follows directly from that representation:

```

1  Model first  = content.Load<Model>("ship");
2  Model second = content.Load<Model>("ship"); // another C++ wrapper,
      same bones/meshes/buffers
3
4  first.CopyBoneTransformsFrom(newPose);      // second now observes
      the same bone transforms
5  content.Unload();                          // removes manager
      cache entries only
6  // first and second still retain the reader-owned graph through
      their shared ownership handles

```

The real XNB test checks the decisive GPU identity part of this rule: two loads expose the same underlying `VertexBuffer`, not a fresh decode/upload. The generic content-cache and `Unload()` tests separately prove cache eviction, but no located test holds a *Model* through `Unload()`, copies one and checks a shared mutation, or exercises the final bundle destruction. The code makes the first two outcomes clear; the missing tests remain an evidence boundary rather than permission to assume a complete shutdown protocol.

There are two qualifications to “shared” worth keeping precise. Reassigning the top-level `Tag` changes only that particular C++ wrapper’s raw pointer; it does not rewrite another wrapper’s `Tag` field, although both initially point to the same reader-owned object. In contrast, a mesh `Tag` or a bone/part/effect change is made on the shared pointed-to graph and is visible through all copies. FNA returns its cached `Model` *class* reference, so ordinary managed assignment aliases even the top-level wrapper; CNA’s value syntax is consequently not a deep-copy compatibility promise.

38.5.1 Hand-built graphs have no automatic owner

The public three/five-argument `Model` constructors merely store the caller’s raw bone and mesh addresses. Likewise, meshes store raw part and graphics-device addresses; parts store raw buffer, index-buffer, effect, and `Tag` addresses; and bones store raw parent/child addresses. They allocate none of those objects and do not install an ownership bundle. The constructors are CNAEXT extensions precisely because FNA creates these graph objects through internal readers rather than asking game code to manage them.

Consequently a hand-built model is drawable only while its complete graph outlives every model copy that uses it; its `GraphicsDevice` must survive for the same draw lifetime. Stack-allocated construction is fine for a local, synchronous test, but returning the model after its backing bones, meshes, parts, buffers, effects, or `Tag` objects have died leaves dangling pointers. An advanced CNA owner can deliberately attach one aggregate lifetime object with the CNAEXT `setOwnedResources(shared_ptr<void>)` hook, mirroring the three readers; otherwise the application must retain its own aggregate explicitly. The hook retains only what its supplied shared object owns—it does not make a raw graphics-device pointer, an externally assigned `Tag`, or another unowned raw address safe by itself.

Finally, `ContentManager::Unload()` clears its generic asset and texture lookup maps; it does not walk a loaded model or invalidate a caller-held C++ value. A retained reader-created model therefore keeps its bundle after unload, but still relies on the separately unowned graphics device being alive when it draws. Drop every model copy before tearing down that device, and do not mistake cache eviction for a safe concurrent reload or a universal resource destructor. Chapter 33 gives the manager-wide cache, thread, and disposal rules; this section supplies the model-specific raw-pointer consequence.

38.6 Skinning: two independent systems, not one

CNA ships two separate, unrelated systems for animated meshes, and conflating them is an easy mistake to make. `AnimationPlayer` (CNAEXT, mirroring Microsoft’s own unshipped XNA “Skinned Model Sample” reference code rather than anything in the framework assembly itself) drives a bone-track timeline: `StartClip(clip)`, `Update(TimeSpan, relativeToCurrentTime, loop=true)`, and `GetBoneTransforms()` / `GetWorldTransforms()` / `GetSkinTransforms()` — the last of which is exactly what feeds `SkinnedEffect::SetBoneTransforms` (Chapter 15). Its skinning data (bone count, hierarchy, bind pose, inverse bind pose, animation

clips) is stashed on a loaded `Model`'s own `Tag` — the same convention the original Microsoft sample used, since real XNA's `Model` has no dedicated skinning-data property of its own to hold it in.

`MorphTargetEXT` (also `CNAEXT`) is a completely independent, glTF-style morph-target blending system: CPU-side re-blend plus a full vertex-buffer re-upload on every call to `SetMorphWeightsEXT`, a deliberate simplicity-over-throughput tradeoff. Because glTF's "weights" animation channel targets a mesh instance node rather than a bone index, this system has no relationship to `AnimationPlayer`'s bone timeline at all; it supports linear, step, and real cubic-spline (Hermite) interpolation, and attaches itself via a loaded `ModelMeshPart`'s own `Tag` — the same attachment convention, applied to a different object in the hierarchy, for a different purpose.

A third, still more separate system exists for the Avatar subsystem specifically: `SkinnedModelEXT` (its own `.skeleton.bin` / `.clip.bin` binary formats) is deliberately not built on `Model` / `ModelBone` / `ModelMesh` at all, has its own full test coverage, and is covered alongside the rest of the Avatar system in this book.

38.6.1 Driving AnimationPlayer with a test-adapted example

`AnimationPlayerTests.cpp` builds its fixtures around the smallest rig that can actually exercise bone-hierarchy composition: two bones, where bone 1 is a child of bone 0 offset by (0, 1, 0) in bind pose, and bone 0's own track moves it from the origin to (2, 0, 0) over one second. Adapted directly from that fixture:

```

1  SkinningData data;
2  data.BoneCount = 2;
3  data.SkeletonHierarchy = {-1, 0}; //
   bone 0 = root, bone 1's parent = 0
4  data.BindPose = {Matrix::getIdentityProperty(),
5                  Matrix::CreateTranslation(Vector3(0, 1, 0))};
6  data.InverseBindPose = {Matrix::getIdentityProperty(), Matrix::
   getIdentityProperty()};
7
8  BoneTrackEXT track;
9  track.BoneIndex = 0;
10 track.Keys.push_back(Keyframe{System::TimeSpan::FromSeconds(0.0),
   Vector3(0, 0, 0)});
11 track.Keys.push_back(Keyframe{System::TimeSpan::FromSeconds(1.0),
   Vector3(2, 0, 0)});
12
13 AnimationClip clip;
14 clip.Duration = System::TimeSpan::FromSeconds(1.0);
15 clip.Tracks.push_back(track);
16 data.AnimationClips["Move"] = clip;
17
18 AnimationPlayer player(data);
19 player.StartClip(data.AnimationClips["Move"]);
20
21 // Every frame:
22 player.Update(gameTime.getElapsedGameTimeProperty(), /*
   relativeToCurrentTime=*/true);
23 skinnedEffect.SetBoneTransforms(player.GetSkinTransforms());

```

Two details this fixture makes concrete rather than abstract: `SkeletonHierarchy` is a flat parent-index array (-1 means "no parent, this is a root"), the same representation glTF and

most DCC export pipelines use, not a pointer-based tree; and `GetWorldTransforms()` versus `GetSkinTransforms()` answer two different questions — the former is each bone’s absolute transform (what §38.4’s manual mesh-draw loop above would want for a non-skinned mesh attached to a bone), the latter is pre-multiplied by each bone’s own `InverseBindPose`, which is specifically the form `SkinnedEffect::SetBoneTransforms` expects, since GPU skinning needs the bone’s *delta* from bind pose, not its absolute pose.

38.6.2 A two-target morph from the test suite

`MorphTargetEXTTests.cpp` builds every one of its cases around the same small, hand-crafted fixture, which doubles as a clear, minimal illustration of how the whole system fits together: a single stride-32 triangle (three vertices, each `Position/Normal/TextureCoordinate`) as the base pose, and two morph targets — target 0 pushes every vertex by +1 in *Z* with no normal change; target 1 moves only vertex 0 by +2 in *X* and tilts its normal toward +*X*:

```

1 MorphTargetDataEXT morph;
2 morph.BaseVertexBytes = BuildBaseTriangleBytes(); // 3 verts, stride
   -32, +Z normal, origin plane
3 morph.Stride = 32;
4
5 morph.PositionDeltas.push_back({Vector3(0, 0, 1), Vector3(0, 0, 1),
   Vector3(0, 0, 1)}); // target 0
6 morph.NormalDeltas.emplace_back();
   // no normal delta, target 0
7
8 morph.PositionDeltas.push_back({Vector3(2, 0, 0), Vector3(0, 0, 0),
   Vector3(0, 0, 0)}); // target 1
9 morph.NormalDeltas.push_back({Vector3(1, 0, 0), Vector3(0, 0, 0),
   Vector3(0, 0, 0)});
10
11 morph.Weights = {0.0f, 0.0f}; // both targets off -- base pose
12
13 part.setTagProperty(&morph); // ModelMeshPart::Tag, the same
   attachment convention SkinningData
14 // uses on Model::Tag, but one level
   down the hierarchy

```

Driving both targets to full weight at once is the case worth reasoning through by hand, because it is where an easy mental model (“morph targets are independent”) breaks in a way the real math does not: *additive* combination means vertex 0 (the only vertex both targets touch) receives **both** deltas simultaneously, while vertices 1 and 2 (untouched by target 1) receive only target 0’s:

```

1 SetMorphWeightsEXT(part, {1.0f, 1.0f}); // re-blends CPU-side, re-
   uploads the vertex buffer
2
3 // Vertex 0: base (0,0,0) + target 0's (0,0,1) + target 1's (2,0,0)
   = (2, 0, 1)
4 // Vertex 1: base (1,0,0) + target 0's (0,0,1) + target 1's zero-for-
   -this-vertex = (1, 0, 1)

```

The blended normal at vertex 0 is the second detail worth internalizing, because it is a place `BlendMorphTargetsEXT` does strictly more work than a naive weighted sum would: the base normal (0,0,1) plus target 1’s full-weight delta (1,0,0) sums to (1,0,1), which has length $\sqrt{2}$, not 1 — `BlendMorphTargetsEXT` renormalizes every blended normal back to unit length before

returning, exactly because a weighted sum of unit vectors is not itself a unit vector in general. `TextureCoordinate` (and, by the same rule, `BlendWeight`/`BlendIndices` on a rigged mesh part) is copied from the base pose completely unchanged by any of this — only `Position` and `Normal` ever participate in morph blending. Passing a weight vector whose length does not match `PositionDeltas.size()` throws rather than silently truncating or ignoring the extra entries, verified directly by the test suite’s own `WrongWeightCountThrows` case.

Per-frame animated weights layer directly on top of this via `EvaluateMorphWeightsEXT` and a `MorphWeightTrackEXT` attached to `MorphTargetDataEXT::WeightTrack` — the same three interpolation modes glTF’s own “weights” animation channel supports (`LINEAR`, `STEP` hold-last-value, and real Hermite `CUBICSPLINE` when a keyframe’s tangents are populated), evaluated independently of `AnimationPlayer`’s own bone-track timeline, consistent with this system’s complete independence from bone-based skinning noted above:

```
1  const auto weights = EvaluateMorphWeightsEXT(morph.WeightTrack,
        animationTimeSeconds);
2  SetMorphWeightsEXT(part, weights);
```

38.7 A worked example: naming an arbitrary root bone, and how the fix was proven to work

`Model`’s hand-built constructor had a real, previously-undocumented gap versus FNA, confirmed by reading FNA’s own `Model.cs` directly: FNA’s constructor never sets `Root` at all — it is real XNA’s own `ModelReader` that assigns `model.Root = bones[rootBoneIndex]` *afterward*, where `rootBoneIndex` can name any bone in the model, not necessarily the first one. CNA’s original 4-argument constructor (the one that also accepts `meshParentBones`) had no equivalent parameter at all, silently defaulting `Root` to `bones[0]` unconditionally — correct for the common case, but unable to represent a hand-built model whose true root bone is not the first entry in its own `bones` vector.

The fix is a fifth, defaulted constructor parameter, additive-only so no existing call site’s behavior changes:

```
1  Model(GraphicsDevice* graphicsDevice,
2        std::vector<ModelBone*> bones,
3        std::vector<ModelMesh*> meshes,
4        std::vector<ModelBone*> meshParentBones,
5        std::size_t rootBoneIndex = 0);    // CNAEXT -- FNA's
        ModelReader assigns this
6
6                                          // externally; this is the
6                                          public equivalent
7                                          // for a hand-built Model.
```

One edge case is worth reasoning through explicitly, because a naive implementation gets it wrong: an empty `bones` vector must leave `Root` as `nullptr` regardless of the requested index, rather than throwing on the harmless default value 0 — the real constructor body checks emptiness first, matching the existing 3-argument constructor’s own leniency, and only bounds-checks `rootBoneIndex` against a genuinely non-empty `bones` vector:

```
1  if (!bones_.bones_.empty())
2  {
3      if (rootBoneIndex >= bones_.bones_.size())
4          throw std::out_of_range("rootBoneIndex");
5      root_ = bones_.bones_[rootBoneIndex];
```

```

6 }
7 // bones_ empty: root_ stays nullptr regardless of rootBoneIndex,
  // matching the
8 // 3-argument constructor's own existing behavior for this case.

```

The real regression tests earned by this fix (`ModelTests.cpp`) are worth naming for the verification technique they demonstrate, not just their existence:

```

1 FiveArgConstructorDefaultRootBoneIndexMatchesFourArgBehavior
2 FiveArgConstructorHonorsNonZeroRootBoneIndex
3 FiveArgConstructorThrowsWhenRootBoneIndexOutOfRange
4 FiveArgConstructorEmptyBonesLeavesRootNullEvenWithDefaultIndex

```

The nonzero-index case uses a three-bone hierarchy, selects index 2, and checks both equality with the expected bone and inequality with bone 0. A test that only checked equality could pass even if the parameter were silently ignored and the implementation happened to default to the requested bone by coincidence. The fix's own commit history records a real sabotage-and-revert check applied to confirm these tests actually exercise what they claim to: with the constructor temporarily edited to ignore `rootBoneIndex` entirely (falling back to the old, hardcoded `bones[0]`), exactly the two tests that depend on honoring a non-default index failed as predicted, while the other 24 `ModelTest.*` cases were unaffected — the same discriminating-power methodology this book applies throughout (a test suite's own *ability* to catch a real regression is itself worth verifying, not assumed from the fact that it currently passes).

This fix closes the hand-build runtime-API gap only. Neither loose route calls the five-argument constructor, and both reserve index 0 as their root. Direct glTF/GLB appends one `ModelBone` per reachable selected-scene node beneath a synthetic identity root; a generated version-2 CNJ serializes and reconstructs that same parent-before-child array. The version-1 CNJ fallback instead appends one synthetic child per mesh. A glTF skin's independently reordered palette hierarchy still lives in `SkinningData`, not in the model-bone index space. Consequently neither route has a serialized arbitrary `rootBoneIndex` for this constructor parameter to represent. The gap is closed for a hand-built model, but remains deliberately inapplicable to the two loose content routes; the reader-specific distinctions are tracked in §38.4, not hidden by the runtime fix.

38.8 Collections are read views, but not uniform ones

The four model collections deliberately expose C++ iteration rather than FNA's managed enumerators, but they do *not* have one identical API.

`ModelBoneCollection` has checked integer lookup, name lookup, `TryGetValue`, `Contains`, and iterators. Its storage is populated by `Model` or a bone's `CNAEXT AddChild`; name lookup returns the first matching name, so names are useful handles, not enforced unique identifiers.

The mesh collection has the same named-lookup family. Its integer lookup is checked, as is the bone collection's, so an invalid index throws rather than invoking C++ undefined behaviour. One small FNA divergence remains: FNA rejects a null *or empty* mesh-name argument before searching, while CNA's `std::string` API has no null spelling and accepts the empty string. It can consequently return a deliberately empty-named mesh or simply return false. Do not use an empty name as a portable not-supplied sentinel.

`ModelMeshPartCollection` has checked integer access, `Count`, and iterators — there is no part name, `Contains`, or `TryGetValue`. Its operator explicitly rejects a negative index and one

greater than or equal to `Count` with `ArgumentOutOfRangeException`, matching the bone/mesh collections rather than exposing raw `vector[]` behaviour.

`ModelEffectCollection` likewise has integer access, `Count`, `Contains`, and iterators, but no name lookup or `TryGetValue`. Its integer operator has the same explicit negative/upper-bound checks. Its mutable operations are the `CNAEXT` escape hatches described in §38.3, not a fourth normal collection-builder API.

All name searches are linear and return the first matching pointer; neither loader nor collection constructor rejects duplicate names. The focused bone/mesh suites cover normal name/index/iterator behaviour and missing names, while `ModelCollectionIndexTests.cpp` checks negative, `Count`, and above-count indices for all four collection families plus first/last identity. No located collection test covers the empty-name FNA difference. The safe cross-collection rule is therefore simple: keep the model's ownership and topology coherent at construction time and use name lookup only where that collection actually exposes it; invalid integer indices fail loudly but are still caller errors.

Chapter 39

From glTF File to Model: the Import Core

CNA does not have a small “glTF reader” hidden inside `ContentManager`. It has a shared import library, `CNA::Internal::GltfImport`, used by two independent front ends: the runtime `.gltf/.glb` path and the offline `cna_tool_gltf_to_cnj` converter. This distinction matters. Parsing and semantic extraction are shared; turning the extracted records into live graphics objects or persistent CNJ sidecars is not.

The shared core is about 2,500 lines across `GltfImportCore.hpp` and `GltfImportCore.cpp`. It delegates container parsing and accessor decoding to the vendored `cgltf` 1.15 library. Exactly one translation unit defines `CGLTF_IMPLEMENTATION`; the stb image implementations in the same translation unit are declared static so they cannot collide with another copy linked through SDL image support. That is a build invariant, not incidental preprocessor decoration.

39.1 The parser boundary

Both JSON `.gltf` and binary `.glb` enter through the same `cgltf_parse_file` path. If `asset.version` is present, both front ends require it to equal "2.0"; their explicit check is conditional, however, so a document with the field absent can pass this boundary if `cgltf` accepts the rest. `asset.minVersion` is not inspected. Buffers are then loaded relative to the source path. Embedded buffer views, base64 data URIs, and external percent-decoded file URIs are supported. Images use the same three storage forms and are exposed as memory blocks; the runtime front end decodes those blocks without staging temporary files. External paths pass through `ResolveExternalUriEXT`: URI schemes, drive-absolute and root-absolute names, lexical escapes, and symlink-resolved escapes outside the asset directory are refused.

Alpha.1 performs a structural gate before semantic extraction. Metadata-only alignment and

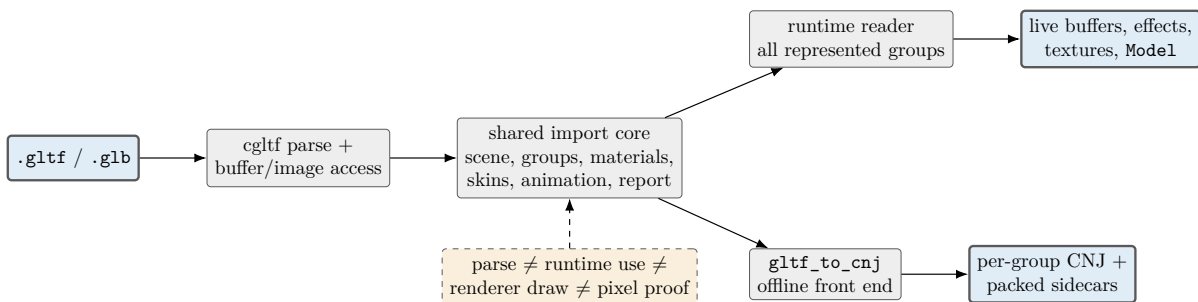


Figure 39.1: The shared glTF import core and its two front ends at 1bb2145d. Shared parsing does not make the runtime and offline output routes equivalent.

overflow-safe span checks run before `cgltf_validate`, because the vendored validator can otherwise dereference misaligned index data while attempting to validate it. The importer then cross-checks authored accessor bounds, refuses undecoded `EXT_meshopt_compression`, rejects every unsupported `extensionsRequired` entry, and records warnings for unsupported optional `extensionsUsed`. This is materially stronger than syntax parsing, although it does not make CNA's importer a replacement for Khronos's independently pinned validator.

Practical rule. Run the Khronos validator before shipping third-party content. A successful CNA load proves that the code paths it used were acceptable; it does not prove that the complete document is spec-conformant or that every required extension was honoured.

39.2 Scene graph first, meshes second

`BuildSceneGraph` selects the document's default scene, falling back to the first scene, then walks it iteratively in parent-before-child order. Index 0 is a synthetic identity node named `Root`; each reachable glTF node follows exactly once. The core asks `cgltf` for the local matrix, converts it to CNA's row-vector convention, and composes

$$W_i = L_i W_{parent(i)}.$$

The conversion copies the affine basis into CNA's matrix convention. It does not apply an axis or handedness conversion, and that is correct: both glTF and the XNA conventions used here are right-handed, use $+Y$ as up, $-Z$ as forward, and place the UV origin at the top left. The only handedness value imported separately is a tangent's fourth component.

Each flattened scene node becomes a `ModelBone`, index-for-index after the synthetic root. Vertex positions remain in mesh-local space. Consequently one glTF mesh instanced by two nodes yields two `ModelMesh` placements with distinct parent bones instead of one destructively pre-transformed vertex buffer. For rigid geometry the mesh is parented to its instancing node. For skinned geometry the node still exists, but the mesh is parented to the synthetic root because skin matrices already account for the mesh-node coordinate space; doing both would apply the transform twice.

Nodes unreachable from the selected scene do not enter this graph. If no scene node references any mesh, the importer has a compatibility fallback that exposes every mesh at the identity root. That fallback is useful for incomplete authoring exports, but it is not the same semantic result as a fully authored scene.

39.3 Groups are an ownership boundary

`CollectMeshGroups` partitions scene instances into one group for each distinct `skin` pointer plus one group for unskinned content. A group record contains the source node, mesh, flattened scene-node index, composed world transform, and whether the instance is skinned. This division is the unit that later becomes one model output.

The offline converter writes every group as a separate CNJ model, with stable suffixes for the static group and named skins. The runtime path instead assembles all groups into one `Model`. `Model::SkinsEXT` maps each independent skin to the meshes using its palette, while `Model.Tag` remains a compatibility alias for the first skin. This avoids both the old `groups.front()` data loss and the equally incorrect alternative of posing all meshes with one palette.

The two front ends also differ in scale control. The core’s `unitScale` multiplies translations—including vertices, bind translations, animation translations and tangents, morph position deltas, inverse binds, and ancestor terms—but never rotations or dimensionless scale factors. The runtime path fixes it at 1.0; only the command-line converter exposes a caller-supplied value.

39.4 What extraction preserves

For each triangle primitive, the core produces an explicit semantic record before any graphics object is allocated. It decodes positions, normals, tangents, texture coordinates, colours, joint indices and weights; validates every index; synthesises a sequential index stream for a non-indexed primitive; and computes tangents when required by a normal-mapped material. Sparse attributes and sparse indices, including an accessor with no base buffer view, are handled.

The factor-only material failure recorded as D7 is fixed. The extracted material record retains base colour, metallic, roughness, emissive, IOR/specular factors, alpha mode/cutoff, double-sidedness, seven texture slots, independent UV-set selectors, per-map transforms, samplers, and colour-space intent. Rigid and skinned PBR effects receive the same shared parameter convention. This is a broad transport contract, not proof that every renderer implements every optional extension texture.

The source-controlled extension registry is the acceptance authority. Representative outcomes include:

- `KHR_texture_transform`, independent per map rather than baked into one stream;
- `KHR_materials_unlit`, variants, IOR, emissive strength, and core material factors;
- `KHR_materials_specular`, whose factors are implemented but whose optional texture bindings retain renderer-specific gaps;
- `KHR_materials_transmission` and archived specular–glossiness, accepted only as named approximations and therefore not claimed when required;
- `KHR_lights_punctual`, reduced to at most three directional-light approximations with lost range/cone/intensity facts reported; and
- `KHR_draco_mesh_compression`, only when CNA was compiled with libdraco.

PNG/JPEG fallbacks can rescue a texture that also advertises KTX2/BasisU or WebP; without a fallback those formats remain unsupported. Clearcoat, sheen, and volume are parsed but ignored; meshopt compression and GPU instancing are refused or reported at their actual loss boundary. Cameras and material variants are preserved through `CNAEXT Model` properties. `GltfImportReportEXT` carries scene counts and ordered diagnostics for every dropped or approximated feature on both direct and CNJ routes. Keep “parsed”, “represented”, “reported”, and “drawn by this renderer” as separate claims.

39.5 Failure and lifetime boundaries

The core throws `std::runtime_error` for several semantic rejections, including an unsupported primitive topology. The runtime reader does not normalize all of these into `ContentLoadException`; a caller catching only the latter can miss a glTF import failure even though it derives from `std::runtime_error`. This exception asymmetry is part of the current contract.

On success, the runtime front end creates model bones, meshes, parts, buffers, effects, textures, skinning records, and morph records in one private ownership bundle, then attaches that bundle to the otherwise raw-pointer-heavy `Model`. Images are cached by source image pointer for that one import. The result is self-retaining, but repeated primitives still receive their own mesh-part graphics resources unless the front end explicitly shares them.

The next four chapters follow the same pipeline at progressively sharper boundaries: exact GPU bytes in Chapter 40, skeletal coordinate spaces in Chapter 41, persistent CNJ output in Chapter 42, and the conformance evidence in Chapter 43.

Chapter 40

GPU Packing and the Stride ABI

Between glTF's typed accessors and a renderer's vertex fetch lies a compact byte protocol that is not declared as a C++ type. `ExtractMesh` chooses one of ten layouts, writes tightly packed bytes, and records only the stride. Later, `ContentManager` switches on that stride to choose an upload route. Renderer shaders must independently agree with the same offsets. The stride is therefore a de-facto ABI.

40.1 The ten glTF layouts

Stride	Fields and byte offsets	Effect family
20	position 0, UV 12	dual texture
24	position 0, colour u8x4 at 12, UV 16	basic, vertex colour
32	position 0, normal 12, UV 24	basic
48	position 0, normal 12, tangent vec4 at 24, UV 40	PBR
52	position 0, normal 12, UV 24, weights 32, joint u8x4 at 48	skinned
56	stride 52 plus colour u8x4 at 52	skinned, vertex colour
60	rigid PBR stride 48, UV1 at 48, colour u8x4 at 56	PBR, dual UV/colour
68	position 0, normal 12, tangent 24, UV 40, weights 48, joint u8x4 at 64	skinned PBR
76	skinned PBR stride 68 plus UV1 at 68	skinned PBR, dual UV
80	stride 76 plus colour u8x4 at 76	skinned PBR, dual UV/colour

Table 40.1: The glTF vertex-byte ABI. Offsets and strides are bytes.

Selection is semantic rather than user-configurable. A base-colour plus occlusion texture on an unskinned, uncoloured, non-PBR primitive selects stride 20 and `DualTextureEffect`; colour selects 24 unless skinning changes it to 56; normal or metallic-roughness materials select a tangent-bearing PBR layout; and skinning otherwise selects 52. PBR layouts widen to carry two independently selected source UV sets and `COLOR_0`. Missing normals, UVs, and colours receive stable defaults.

The downstream upload switch also recognizes stride 16 for a typed `VertexPositionColor` path used by non-glTF raw content. That makes the upload switch an eleven-case protocol, but stride 16 is not an eleventh layout emitted by the current glTF selection expression. Keeping those two counts separate avoids a recurring documentation contradiction.

40.2 Why ordinary C++ structs were unsafe

Several CNA vertex structs implement polymorphic `IVertexType`. Their in-memory C++ objects consequently contain a vtable pointer, so `sizeof(Vertex...)` is larger than the clean GPU layout and the first declared field need not begin at byte 0. Reinterpreting the tightly

packed glTF byte array as an array of those objects shifts every field. The renderer then sees plausible-sized buffers containing nonsense, a root cause of the long-lived “invisible model” symptom family.

The current bridge is explicit:

- strides 16, 20, 24, and 32 are reconstructed field-by-field as real typed vertex objects before typed `SetData`;
- strides 48, 52, 56, 60, 68, 76, and 80 use `SetDataRaw`, preserving the packed bytes; and
- the glTF extractor emits the ten layouts above. A hand-written Model CNJ naming any other stride is outside this inferred-layout contract and must not be described as another supported layout.

This fixes the object-layout error, but it leaves the protocol stated in more than one place: the packer, the upload switch, renderer declarations, morph-target offset logic, and the L5 golden oracle. CNA now also states all eight layouts once in `VertexDeclarationFidelity.hpp`; stride-infering renderers use that table to refuse a declared layout they cannot reproduce faithfully instead of fetching the wrong bytes. That table is a safety oracle, not a generator: the glTF packer, content upload switch, renderer-native layouts, morph offsets, and independent L5 golden tables remain separate statements. A future refactor could generate the production views from one declarative source. Until then, tests that compare independently stated tables are more valuable than comments claiming they agree.

40.3 Indices: source width is not destination width

CNA decodes glTF indices itself rather than relying on `cgltf`’s index helper, because the helper cannot report why a sparse read failed. The decoder accepts unsigned byte, unsigned short, and unsigned int scalar accessors, honours buffer-view/accessor offsets and sparse overrides, and uses `memcpy` for possibly unaligned components. A non-indexed primitive receives the implicit sequence $0, 1, \dots, n - 1$.

Every decoded index is checked against the position count before packing. Only after that proof does the destination width get selected:

$$\text{destination width} = \begin{cases} 16 \text{ bits,} & \text{vertexCount} \leq 65,535, \\ 32 \text{ bits,} & \text{vertexCount} > 65,535. \end{cases}$$

The source component type is deliberately irrelevant. A glTF accessor stored as 32-bit indices for a 100-vertex mesh becomes a 16-bit GPU index buffer; the preceding range check makes that narrowing safe. This is a useful example of proving a conversion rather than preserving a source representation merely because it existed.

The decoder’s component reads are alignment-safe, but the final front-end handoff is not yet as careful: both direct glTF and Model CNJ cast a `vector<uint8_t>::data()` address to a `uint16_t*` or `uint32_t*` before calling `IndexBuffer::SetData`. The callee immediately forwards the region as bytes, but the converted pointer is not guaranteed to meet that type’s alignment; any typed access through it would have undefined behaviour. Copying into aligned typed storage or adding a byte-oriented upload would remove that avoidable contract hazard without changing the GPU bytes.

40.4 All seven topology modes have an explicit route

D5 is fixed end to end. Points, line lists, and line strips retain their topology; a line loop is closed and becomes a line strip; triangle strips and fans are expanded into winding-correct triangle lists; triangle lists remain lists. `ModelMeshPart`'s `CNAEXT PrimitiveType` property travels through the direct and CNJ routes, and primitive counts use the selected topology rather than universally dividing by three. An incomplete final primitive/run is dropped with a counted diagnostic. Conversion is therefore visible in the source-vs-output topology fields, not a silent reinterpretation.

40.5 Tangents and winding

If a glTF primitive supplies `TANGENT`, CNA preserves it. Otherwise PBR extraction computes tangent and handedness from positions, normals, and UV gradients. Contributions are weighted by corner angle, degenerate UV triangles are skipped, and the result is Gram–Schmidt-orthogonalized. This is intentionally not a bit-exact MikkTSpace implementation: corners are not welded across matching position/normal seams.

No coordinate-system conversion is required, but one winding case remains: a node with a negative-determinant transform changes front-face orientation. CNA carries and reports the per-placement mirror fact, but does not change draw-time cull state; flipping a shared index buffer would break an unmirrored instance of the same mesh. Default culling can therefore hide that placement. This is an open application/state boundary separate from tangent handedness.

40.6 Hard limits carried by the bytes

The ABI makes several limits concrete rather than advisory:

- at most two distinct source UV sets reach PBR effects; a third requested set is reported;
- only `JOINTS_0/WEIGHTS_0` are imported: four influences per vertex;
- joint indices are four unsigned bytes, so at most 256 palette addresses are encodable;
- `SkinnedEffect` and `SkinnedPbrEffect` cap actual palettes at 72 bones; and
- morph targets are CPU-blended into the base byte array and re-uploaded. Position, normal, and tangent offsets are resolved from the canonical layout table; absent or illegal target streams are reported rather than inferred from an old stride subset.

These are different layers of limit. Raising the shader palette above 72 does not widen an 8-bit joint index, and widening the index does not add a fifth influence or a second UV set. Any format change must update the full stride ABI and its oracles together.

Chapter 41

Skinning and Animation across Two Systems

CNA contains two skeletal-animation systems that deliberately do not share a runtime graph. `SkinningData` plus `AnimationPlayer` animate an ordinary `Model`; `SkinnedModelEXT` is an Avatar-oriented model, skeleton, clip, and draw system of its own. Similar interpolation code appears in both. The duplication is documented as intentional because forcing the public types together would couple two distinct asset contracts.

41.1 The ordinary Model path

A skinned `Model` carries `SkinningData` through `Model.Tag`. The record contains bind-pose local matrices, inverse-bind matrices, parent indices, animation clips, and a fourth matrix array named `SkeletonRootPrefix`. An `AnimationPlayer` samples a clip into local transforms, composes worlds in topological order, then builds the skin palette used by `SkinnedEffect` or `SkinnedPbrEffect`.

The model itself does not do that last step. `Model::Draw` sets the ordinary `IEffectMatrices` world matrix for each mesh, but never installs a bone palette. The application or animation helper must call the skinned effect's `SetBoneTransforms` before drawing. A model whose effect does not implement `IEffectMatrices` is rejected by `Model::Draw`.

41.2 The Avatar path

`SkinnedModelEXT` is explicitly not built from `ModelBone`, `ModelMesh`, or `ModelMeshPart`. It owns its Avatar-specific geometry and animation representation and is loaded through the legacy `.skinnedmodel.json` reader. Its superficially similar `SampleTrack` logic is a local copy, not a hidden conversion to `AnimationPlayer`.

Both players nevertheless enforce the same essential inputs. Bone arrays must agree in size,

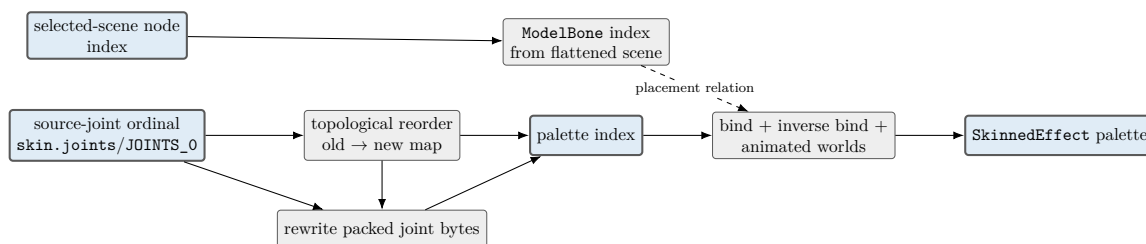


Figure 41.1: Scene-node, source-joint, and palette indices are distinct. Convenient source ordering can hide a missing conversion, so the importer records and tests the maps explicitly.

and every non-root parent index must precede its child; a parent index greater than or equal to the current index throws. Any negative parent value is treated as “root”, not only `-1`, and an animation track with an empty key list or an out-of-range bone index is silently ignored. Looping uses floor-modulo on raw ticks, so negative or over-duration times wrap consistently. Runtime translations and scales interpolate linearly, rotations spherically. Track search is linear in the number of keyframes per bone.

The size agreement is not a complete hand-construction validator. Content readers reject a negative or implausibly large bone count, but `AnimationPlayer`’s public constructor first casts `BoneCount` to `size_t` for its working-vector allocations and only then runs the array-size check in `RecomputeTransforms`. A manually supplied negative count can therefore request an enormous allocation (typically failing with a standard allocation/length exception) rather than producing the later `ArgumentException`. Validate custom `SkinningData` as non-negative before construction.

41.3 Scene indices are not palette indices

The glTF scene is flattened into `ModelBone` indices, while a skin’s `joints` array defines a separate palette. Those spaces need not have the same order and the glTF joint array need not be parent-before-child. `BuildSkeleton` therefore reorders joints breadth-first into topological order and records an old-to-new map. Packed `JOINTS_0` values are rewritten through that map.

This yields three indices worth naming explicitly:

scene-node index selects the model bone representing placement in the selected scene;

source-joint index is the ordinal stored in glTF `JOINTS_0`; and

palette index selects the reordered bind, inverse-bind, and animated world arrays.

Treating any two as interchangeable works only on conveniently authored fixtures. Reversed joint-order tests exist precisely to rule out that accident.

41.4 The coordinate-space failure that produced D8

A joint root may have ancestors that are not themselves listed in `skin.joints`. Older logic walked parents only within the joint set. It therefore dropped the armature node’s world transform from the reconstructed bind hierarchy while retaining the file’s inverse-bind matrix, which already encoded that ancestry. A uniform armature scale became an inverse scale in the final palette, collapsing the character toward the origin.

The corrected root term is

$$P_{root} = W_{ancestor} W_{meshNode}^{-1}.$$

The first factor recovers scene ancestry above the joint set. The second cancels the transform of the node that instances the skinned mesh, because the mesh is parented to the model’s synthetic root rather than to that node. The importer does not stop at glTF’s optional `skin.skeleton` hint, and can obtain ancestry for a joint outside the selected scene through cgltf’s world-transform calculation.

41.4.1 Why the prefix must remain separate

It is tempting to multiply P_{root} into the root bind-pose local matrix. That works until an animation channel replaces that local transform. At the first sampled frame the recovered ancestry disappears again. Instead, CNA stores one prefix per root in `SkeletonRootPrefix` and composes

$$W_{root}(t) = L_{root}(t)P_{root}.$$

An empty prefix array means identity, preserving older CNJ skeleton sidecars. This separation is both the D8 fix and the compatibility mechanism that lets the runtime animate roots without undoing it.

The conformance fixture for the mesh-node term was chosen so the three possible results are numerically distinct: missing cancellation, cancellation exactly once, and applying the inverse twice. The test expects the middle case rather than merely checking that a vertex “moved”.

41.5 Clip extraction

For skeletal clips, CNA forms the union of translation, rotation, and scale key times for each animated bone, deduplicated at a tight tolerance. Missing channels at a time fall back to the decomposed bind pose. Linear and step samplers are evaluated directly; cubic splines use the glTF Hermite basis with the required Δt tangent scaling, and cubic quaternion results are normalized *while the importer samples that union*. The resulting ordinary `KeyframeEXT` records do not retain bone-channel interpolation modes or tangents; `AnimationPlayer` later uses linear/Slerp interpolation between them. A cubic channel is therefore exact at every union time but generally only a piecewise linear/Slerp approximation between those stored times. Morph tracks below are different: they retain cubic tangents for lazy Hermite evaluation.

Only channels whose target resolves through the skin’s joint map enter a skeletal clip. Rigid node TRS animation is retained separately as scene-node-indexed clips. On an unskinned model, `Model.Tag` carries `ModelAnimationsEXT`; its tracks address the same flattened scene-node indices used by `ModelBone`. A mixed skinned model cannot put that second carrier into the already occupied legacy Tag slot, so rigid tracks outside the retained skin palettes are explicitly counted and reported as dropped. D6 is therefore fixed for the ordinary rigid-model route without pretending that one legacy pointer can hold two unrelated carriers.

Morph-weight animation follows a different, skin-independent route. It locates a weights channel on the node instancing the mesh, preserves cubic in/out tangents unbaked, and evaluates Hermite weights lazily during playback. If one mesh is instanced by several nodes, the first matching node wins. Morph position/normal deltas are then CPU-blended into vertex bytes and the buffer is uploaded again; this is not GPU morphing.

41.6 Scaling and capacity

The converter’s `unitScale` applies to every translation-bearing quantity, including animation translation values and their cubic tangents. It never applies to rotation or scale channels. Runtime glTF import fixes it at 1.0, while the CLI can choose another unit basis.

Both stock skinned effects accept at most 72 palette matrices. Each packed vertex has four 8-bit joint indices and four weights. A legal glTF skin larger than those runtime limits must be split or otherwise reduced during content processing; successful container parsing does not promise it can be drawn in one part.

41.7 Practical update order

A frame using the ordinary path should perform these operations in order:

1. choose and sample the active clip into local bone transforms;
2. compose absolute skeleton transforms, including root prefixes;
3. form skin matrices from worlds and inverse binds;
4. install the palette on every skinned effect used by the model;
5. evaluate and upload morph weights if they changed; and
6. call `Model::Draw` on the graphics thread.

The draw scratch vector is `thread_local` at `alpha.1`, removing the earlier cross-thread scratch race. Graphics resources and the shared model graph are not thereby made generally thread-safe; keep sampling, palette installation, mutable model updates, and drawing under the application's graphics-thread ownership rule.

Chapter 42

The CNJ Model Toolchain

Direct runtime glTF loading is convenient and now retains all scene groups in one `Model`, but fixes unit scale at 1. The offline `cna_tool_gltf_to_cnj` path uses the same semantic import core and material packing, then writes each group as explicit CNJ and binary sidecars. It is the reproducible path for controlled scaling, one-output-per-group packaging, and inspectable build artifacts.

42.1 A renderer-free build tool

The tool is configured by `cmake/ToolGltfToCnj.cmake`, built unconditionally rather than behind `CNA_BUILD_TESTS`, and links CNA plus sharp-runtime. It does not initialize a window, graphics device, or renderer. The command line is

```
1 cna_tool_gltf_to_cnj input.gltf outputDir baseName [unitScale]
```

The input may be `.gltf` or `.glb`. For each group, the tool writes a model descriptor and sidecars:

- `<name>.cnj`, a version-2 `Model` envelope;
- `<name>_meshN_verts.bin` and `_idx.bin`;
- optional `_morph.bin` files;
- optional `<name>.skeleton.bin`;
- one version-1 `AnimationClip` CNJ per clip; and
- extracted base, occlusion, and other texture images with stable suffixes.

Multiple groups receive stable `_static` or skin-name suffixes. Runtime and offline routes now preserve the same group set but package it differently: one multi-skin runtime model versus separately named converter outputs.

42.2 The Model envelope is version 2

CNJ's common envelope contains a version and a logical type. Most types accept version 1 only; `Model` has a per-type maximum of 2. Raising the Model ceiling did not globally bless version 2 for textures, effects, or animation clips. Validation receives the requested type and that type's maximum, so a version accepted for one reader can still be rejected by another.

Version 2 adds a top-level `bones` array. Each record carries a name, parent index, and 16-float local transform, and records are parent-before-child with an identity root at index 0. Each mesh also receives a `parentBone` index. These fields preserve the glTF scene hierarchy and rigid instancing placement that a flat list of pre-transformed meshes could not.

The same version-2 envelope can also carry the alpha.1 import report, cameras, rigid clips, material-variant state, topology, multi-UV material metadata, and newer PBR fields. These are additive fields within the per-type version ceiling, not evidence that every CNJ type accepts a new global format version. The reader supplies empty/default carriers for older Model documents.

Those last two shape rules are producer invariants, not fully validated schema rules. The reader always constructs entry 0 as its root and does not apply that entry's serialized transform or parent. For later entries it bounds-checks the parent against the complete array, but does not require `parent < child`; a forward parent therefore builds a pointer tree whose `CopyAbsoluteBoneTransformsTo` order is wrong. A `transform` array with any count other than 16 silently becomes identity. Generated files satisfy all three rules, but a hand-edited version-2 descriptor can load into an internally inconsistent graph rather than fail at the schema boundary.

Version-1 Model files remain loadable. When no general hierarchy exists, the reader constructs the older per-mesh fallback bone. This makes version 2 an additive fidelity improvement rather than a flag day.

42.3 Binary sidecars and bounded reads

Vertex and index bytes use the packed ABI from Chapter 40. The descriptor supplies strides and paths, from which the reader derives counts and primitive counts. Its validation is incomplete: a non-positive stride silently drops the mesh, an unknown positive stride can produce a vertex buffer without uploaded contents, and vertex/index byte lengths not divisible by their derived element size are truncated by integer division rather than rejected. Morph sidecars retain base bytes and target deltas used by `MorphTargetDataEXT`; their mandatory reads are bounds-checked, but the descriptor/sidecar pair is still trusted to agree semantically.

The skeleton sidecar historically contained two matrix blocks: bind poses and inverse binds. Version 2 can append a third block containing `SkeletonRootPrefix`. The reader checks whether at least `boneCount * 64` bytes remain before reading it; if not, it supplies identity prefixes. This compatibility test does not distinguish a genuinely old file from a partially written new third block: any trailing prefix fragment shorter than the complete block is ignored rather than rejected, and extra bytes after a complete block are not required to be exhausted. The mandatory hierarchy/bind/inverse-bind reads remain bounds-checked. Appending an optional fixed-size block rather than changing the meaning of the old two is what keeps earlier sidecars valid, but producers should still publish each generated sidecar atomically.

The descriptor may reference separate clip CNJs. Those pass through the manager's normal cached loading path, so repeated names can share a clip object. Mesh buffers, built-in effects, and most graph objects are still created per described part and retained by the model's private ownership bundle.

42.4 Offline and runtime parity

Sharing `GltfImportCore` reduces drift, but does not prove parity by itself. The two front ends perform different work after extraction: one creates textures, effects, buffers and a live model; the other serializes bytes and descriptors that a later CNJ reader recreates. Tests therefore compare semantic and rendering-visible results across both routes, including reversed skin order, animation, PBR data, morph tracks, texture extraction, and hierarchy.

The important contract is not byte identity between a live object and JSON. It is that a group

imported directly and the same group converted then loaded from CNJ produce the same scene-node placement, palette coordinate spaces, vertex/index bytes, material choice, and animation records. A fix made only in one front end is a regression even if the shared parser still builds.

42.5 Occlusion through DualTextureEffect

glTF occlusion uses 1.0 for fully visible light and 0.0 for full occlusion. XNA's `DualTextureEffect`, however, multiplies its second texture contribution by 2; in that equation 0.5 is neutral. Passing glTF's occlusion image unchanged would brighten rather than preserve an unoccluded surface.

For the non-PBR dual-texture approximation, CNA therefore decodes the occlusion image, halves its RGB channels, and re-encodes it. The resulting mapping is

$$o_{dual} = \frac{o_{gltf}}{2}, \quad 2o_{dual} = o_{gltf}.$$

The remapped image is cached separately from the original because the same source image may also be bound to `PbrEffect`, where glTF's original convention is correct. The runtime and converter both use this shared extraction rule; tests cover successful remapping and decode failure.

42.6 Choosing a route

Use direct glTF for rapid iteration on a file known to have one relevant group and unit scale. Use the converter when the file contains multiple characters or static plus skinned content, when unit conversion is required, or when asset bytes must be reviewed and committed. Use XNB when compatibility with an existing XNA content pipeline is the governing requirement. The legacy `.skinnedmodel.json` path is for `SkinnedModelEXT`, not an alternate CNJ spelling.

Do not hand-edit counts or offsets in a generated Model CNJ without regenerating or validating the sidecars. The JSON is readable, but its numerical metadata and binary files form one asset contract.

Chapter 43

glTF Conformance and Oracles

CNA’s alpha.1 glTF story is an evidence system as much as an importer: a pinned specification, a deterministic committed corpus, numerical observations from container through bound effect state, renderer-owned pixel campaigns, and an append-only defect ledger. The resulting claim is substantial but deliberately qualified. CNA declares its *import/runtime model* correct for the scoped core campaign; it does not declare every optional extension, renderer shader, or hostile input complete.

43.1 Pinned authority and deterministic corpus

`docs/glTF-conformance.md` pins the Khronos glTF 2.0 specification at commit `2b29723d025a995971726f2989697cdc49b1222a`; the 149,830-byte AsciiDoc source has SHA-256 `55986799907693d3f51b0a474497852c0d6318b85084811fdc05ff0db4b27967`. That document is research/test authority, not a build-time dependency.

The standard-library-only generator under `tools/glTF_fixtures` emits each authored fixture, its GLB twin, its independently stated expectations, and any byte goldens. Generated outputs are committed and guarded by a manifest; check-only regeneration must be byte-identical. At the tag, `tests/assets/glTF` contains 148 `.glTF` assets, 148 `.glb` twins, and 148 `.expected.json` records in a 744-file tree. The paired container audit therefore examines 296 asset containers, while CNA’s own validation report separately classifies the valid/expected-invalid cases it feeds to the external validator. These denominators answer different questions and must not be merged into one “number of glTF tests.”

The external Khronos validator is pinned and used by the recorded corpus audit, but is not needed to build or run CNA. Likewise, Khronos Sample Assets and Sample Renderer are fetched for a bounded comparison campaign rather than redistributed as CNA runtime dependencies.

43.2 The ladder, including its L0 gate

The semantic ladder has seven observation layers. A separate L0 gate first proves the corpus and oracle machinery are intact; a higher layer is meaningful only when the lower ones agree.

- L1: container structure** parser counts, selected scene, relationships, and declared extensions;
- L2: decoded accessors** typed arrays after offsets, strides, normalization, and sparse overrides;
- L3: semantic import** mesh streams, materials, topology, grouping, skins, animation, and reportable loss before GPU packing;
- L4: composed geometry** scene/world positions after node and skin coordinate-space rules;

- L5: GPU bytes** vertex and index bytes, compared with field-aware diagnostics;
- L6: bound effect state** matrices, material factors, maps, samplers, alpha state, lights, and palettes reaching `GpuDrawParams`; and
- L7: rendered pixels** per-renderer PNGs and explicit safe-rejection dispositions.

L1–L6 run as separately named CTest entries. The L5 corpus has 140 byte-golden dispositions; eight fixtures have no golden because seven are deliberate import rejections and one positive generated-tangent case refuses to duplicate the production tangent algorithm as its oracle. L6 calls the same virtual effect-parameter route used by a draw, but intentionally stops before renderer rasterization.

43.3 Pixel evidence is renderer-specific

The tag carries four complete L7 campaign reports: OPENGLES3/EasyGL, Vulkan, SOFTWARE, and DIRECTX11 under Wine/DXVK. Each report gives all 148 canonical assets an explicit disposition: 140 deterministic captured PNGs and eight safe rejections. Each renderer owns its goldens and requires two independent viewer processes to agree; no renderer silently borrows another’s image or tolerance.

That is *Historically recorded* evidence in this edition: the reports, image hashes, commands, and guards are present at the immutable tag, but the Bible update did not reproduce all four platform campaigns. Same-renderer repeatability also is not an independent implementation oracle. The tag’s supplementary Sample Renderer retake supplies that second perspective for a curated real-world matrix: its report records 15 captures across 14 gate rows against a pinned Khronos renderer and pinned Sample Assets. Its comparison includes explicit silhouette, coverage, and colour-error thresholds; it is broader than a smoke test and narrower than pixel-identical proof of all 148 fixtures.

43.4 The append-only defect ledger

Normative expectations and measured current behaviour remain separate fields. A known defect must both differ from the normative value and equal its recorded actual value; a fix moves the old observation to history rather than deleting it. At alpha.1 all eight original forensic defects are fixed:

Defect	Original failure	Tag state
D1–D3	node TRS, hierarchy, and matrix transforms	fixed
D4	sparse index accessors decoded as zero	fixed
D5	non-triangle topology silently reinterpreted/rejected	fixed
D6	rigid node animation dropped	fixed for the supported carrier model
D7	factor-only PBR material lost	fixed
D8	ancestry above a skin’s joint set lost	fixed

Table 43.1: The original glTF forensic ledger at CNA alpha.1.

D5 now carries point/line topology or converts loops, strips, and fans explicitly. D6 stores rigid scene-node clips on unskinned models and reports the remaining mixed-carrier loss rather than hiding it. D7 reaches the L6 material contract and the recorded renderer campaigns. D8 keeps skeleton-root ancestry and mesh-node cancellation in a separate prefix so animation cannot overwrite them.

43.5 Current capability and loss reporting

`GltfImportReportEXT` is part of the public CNAEXT result surface. Direct glTF and converter-produced CNJ models expose scene, mesh, skin, camera, light, animation, primitive, and clip counts plus ordered diagnostic codes. `AnythingLost()` and the aggregate dropped/approximated counts make a successful load distinguishable from a lossless one. Applications should branch on diagnostic codes, not human-readable messages.

The runtime model also exposes all imported skin mappings, cameras, material-variant names and selection, whole-model bounds, and per-part topology. These additions close several old “parsed but unreachable” gaps. They do not erase the current limits:

- at most two distinct texture-coordinate sets are packed for a PBR primitive;
- only the first four joint influences reach XNA-shaped vertex streams, and stock skinned effects retain their 72-matrix palette ceiling;
- point and spot lights are approximated through XNA’s three directional-light slots; range and cone information is reported as lost;
- `EXT_mesh_gpu_instancing` does not carry its per-instance transforms into the model, and `EXT_meshopt_compression` is refused without a decoder;
- KTX2/BasisU and WebP sources require a usable PNG/JPEG fallback; and
- mirror winding is reported per placement but remains an application/draw-state decision.

43.6 What the milestone name means

The tag’s current declaration is `GLTF CORE 2.0 IMPORT/RUNTIME MODEL CORRECT`. It covers the scoped importer, CNJ route, model graph, effects/state transport, generated corpus, and recorded renderer evidence. The qualification matters: the earlier unqualified wording was superseded because a correct import model cannot by itself prove every renderer’s final pixels.

The broader `GLTF ROBUST` gate is 9 of 12 rows. Three rows remain open at the tag:

1. complete `KHR_materials_specular` texture sampling in the remaining Metal and Wicked PBR implementations;
2. native point/spot light terms with falloff and cones in the cross-renderer shader ABI; and
3. a fourth renderer passing the bounded Khronos real-world retake matrix (three qualify, including the Direct3D path).

Those open rows are not contradictions to the qualified core declaration; they define why the stronger milestone remains open. The practical reading rule is therefore precise: a green import plus an empty loss report establishes the represented model for the exercised path, L6 establishes what reaches an effect, and only renderer-engaged/pixel evidence establishes the final image.

Part VII

Input, Audio, Media, Devices, Networking, and Services

Chapter 44

The Input System

CNA's `Microsoft::Xna::Framework::Input` namespace covers keyboard, mouse, gamepad, and touch — and, layered underneath it, a set of CNA-only extensions with no XNA counterpart at all. The current source audit counts 522 input test cases and a compile-time strict-API freeze. That is substantial automated evidence, but it does not replace physical keyboard, mouse, gamepad, touchscreen, IME, and high-DPI verification.

44.1 Architecture: event-driven, not poll-driven

FNA queries SDL when `GetState()` is called. CNA caches state from the events drained by `Game::PollEvents()`. `SdlInputBridge::ProcessEvent` updates `InputManager`, `TextInputEXT`, and the touch gesture detector; the public `GetState()` methods read those snapshots. `FrameworkDispatcher::Update()` drives the per-frame touch gesture update.

Repeated reads without another event return the same keyboard, absolute-mouse, gamepad, and touch state. Relative mouse mode is the exception: the first `Mouse::GetState()` drains its motion accumulator, and a second read returns zero until more motion arrives. The event pump and state reads are single-threaded and unsynchronized; both belong on the game-loop/video thread.

Two ordinary keys have an additional framework side effect: **F9** and **F10** simulate graphics-context loss and restoration in `Game::PollEvents()`. The event is sent to `SdlInputBridge::ProcessEvent` first, so both keys still reach `Keyboard::GetState()`; application code may observe them but cannot suppress the debug action. Avoid binding gameplay to them unless that game-loop behavior is changed.

Three tags recur in the headers: **STRICT** (the frozen FNA/XNA-shaped surface), **EXT** (an FNA-compatible extension, using the `...EXT` suffix convention from this book's game-loop chapter), and **CNAEXT** (a CNA-only addition with no FNA counterpart at all). `PublicApiInputSignatureFreezeTests.cpp` compile-checks the strict member set.

44.2 Keyboard

`Keyboard` is static-only, offering `GetState()` and a per-`PlayerIndex` overload. `KeyboardState` stores keys in a `std::unordered_set<Keys>` instead of FNA's bitfield while preserving the observable queries, sorted `GetPressedKeys()` result, and FNA-shaped hash formula. `Keys` has exactly 160 values, independently confirmed byte-for-byte identical to FNA's numeric values, including several distinctly odd hexadecimal outliers (`Pause = 0x13`, `Kana = 0x15`, `ChatPadGreen = 0xCA`) preserved exactly rather than renumbered for tidiness. The SDL bridge has 125 scancode-to-key cases but 126 cases in the reverse direction, and many enum values have no SDL scancode source at all. IME keys, ChatPad keys, and several browser/media keys

therefore cannot round-trip through the EXT scancode helpers. Typed text must go through `TextInputEXT`, not through assumptions that every Unicode character has a `Keys` value.

44.2.1 KeyboardState and keyboard extensions

`KeyboardState`'s public surface is compact:

`KeyboardState(std::initializer_list<Keys> keys)` the ordinary way application and test code constructs one, for example `KeyboardState{Keys::W, Keys::LeftShift}`.

`getItem(Keys key) / operator[] (Keys key)` both return a `KeyState` (Down or Up) for one key.

`IsKeyDown(key) / IsKeyUp(key)` boolean key queries.

`GetPressedKeys()` returns every currently-down key as an ascending-sorted `std::vector<Keys>`; the sort is part of the contract because the backing hash set has no stable iteration order.

`Keyboard`'s EXT methods bridge XNA's US-QWERTY-shaped `Keys` enum and SDL3's distinction between a physical scancode and a layout-dependent keycode. `GetKeyFromScancodeEXT()` maps a US-named physical position into the current layout, which supports position-based movement controls on non-US keyboards. The public API has no general keycode-to-physical-position inverse; the inverse helpers apply to textual names. `GetModStateEXT()` returns SDL's modifier mask. `GetScancodeNameEXT` and `GetKeyNameEXT`, plus their name-to-key forms, provide current-layout labels for rebinding interfaces.

```

1  // Physical-position movement keys for a non-QWERTY layout.
2  // WASD's physical positions, expressed as scancodes rather than as
   the
3  // QWERTY-specific Keys::W/A/S/D values directly.
4  const Keys forwardKey = Keyboard::GetKeyFromScancodeEXT(Keys::W);
5  const Keys leftKey    = Keyboard::GetKeyFromScancodeEXT(Keys::A);
6  const Keys backKey    = Keyboard::GetKeyFromScancodeEXT(Keys::S);
7  const Keys rightKey   = Keyboard::GetKeyFromScancodeEXT(Keys::D);
8
9  // On an AZERTY keyboard, forwardKey resolves to the physical key
   labelled Z.
10 // GetKeyFromScancodeEXT(Keys::W) treats Keys::W as identifying the
11 // physical scancode a US layout would call W, then maps that
   scancode
12 // back through the *current* layout to find what key is physically
   there.
13 const KeyboardState keyboard = Keyboard::GetState();
14 if (keyboard.IsKeyDown(forwardKey))
15 {
16     // move forward
17 }
```

44.3 Mouse

`Mouse` is static-only: `GetState()` and `SetPosition(x, y)`, the latter converting logical coordinates to physical window coordinates before calling SDL's warp function. The shared routing first asks an associated SDL renderer, then a registered graphics renderer, then falls back to pass-through. Those tiers make the result renderer- and presentation-dependent; the audit below and Chapter 19's matrix state the exact boundary. `MouseState` carries the expected X/Y,

five buttons, and a cumulative, 120-units-per-notch `ScrollWheelValue`; a CNAEXT horizontal-scroll addition is deliberately excluded from `Equals` / `GetHashCode` / `ToString` specifically so those stay byte-identical to FNA's own output despite the extra field existing. A wholly CNAEXT class, `MouseCursor`, wraps `SDL_Cursor*` with eleven lazily created, process-lifetime stock system cursors. `MonoGame's PlatformDispose()` frees a shared stock cursor handle unconditionally, which can invalidate other users; CNA makes `Dispose()` a no-op for shared stock instances.

44.3.1 MouseState and mouse extensions

`MouseState` is a plain data carrier — `X` / `Y`, five `ButtonState` properties (`LeftButton` / `RightButton` / `MiddleButton` / `XButton1` / `XButton2`), the cumulative `ScrollWheelValue` already named above, and (CNAEXT) a parallel `HorizontalScrollWheelValueEXT` for horizontal scroll, deliberately excluded from `Equals` / `GetHashCode` / `ToString` as already noted. `Mouse` itself, beyond `GetState()` and `SetPosition`, exposes relative (pointer-lock-style) mouse mode and window/capture-related EXT methods:

`IsRelativeMouseModeEXT` (get/set) toggles SDL's own relative mouse mode, where motion is reported as unbounded deltas rather than an absolute, screen-clamped position — the standard technique for an FPS-style camera look. Both accessors are safe to call with no window at all (no published window handle and no focused window): the getter simply reports `false` rather than querying SDL with a null window (whose behavior SDL leaves undefined), and the setter is a no-op. While relative mode is active, each `SDL_EVENT_MOUSE_MOTION` adds its relative delta to a separate accumulator; motion outside that mode is not accumulated, and toggling the mode flushes stale deltas. `MouseState's` public `X` / `Y` read and immediately drain that accumulator in relative mode; otherwise they report the absolute position.

`SetPosition(x, y)` is a documented no-op in relative mode because an absolute warp has no meaning while the mouse reports deltas.

`SetCaptureEXT(enabled)` engages OS-level mouse capture (input keeps arriving even if the cursor leaves the window bounds), distinct from relative mode and combinable with it.

`GetGlobalPositionEXT` / `WarpGlobalEXT` read or set the mouse position in *desktop* rather than window-relative coordinates.

`ClickedEXT` a CNAEXT `System::MulticastAction<int>` event (Chapter 56 covers this sharp-runtime type) firing once per click with the button index, for code that wants a discrete click event rather than polling button state every frame.

```

1 // FPS-style camera input: relative mode reports motion as deltas.
2 game.setIsMouseVisibleProperty(false);
3 Mouse::setIsRelativeMouseModeEXTProperty(true);
4
5 // Each frame: read accumulated relative motion, not an absolute
  position.
6 const MouseState mouse = Mouse::GetState();
7 const float yaw = mouse.X * mouseSensitivity;
8 const float pitch = mouse.Y * mouseSensitivity;
9 cameraRotation = Quaternion::CreateFromYawPitchRoll(yaw, pitch, 0.0f)
  * cameraRotation;
```

44.3.2 Logical coordinates are a renderer contract

The write direction is only half the story. Incoming `SDL_EVENT_MOUSE_MOTION` and mouse-button positions pass through `SdlInputBridge::to_logical_position()` before updating `InputManager`; touch starts from normalized window position and reaches the same helper. Thus a bad or missing transform affects absolute mouse reads, click coordinates, and touch together, while relative mouse motion deliberately remains an unscaled delta.

The `SDL_RENDERER` route has the strongest existing proof. A test creates a 100×100 logical presentation inside a 200×200 window, injects a physical motion event at (100,100), and observes logical (50,50) through `Mouse::GetState()`. Separate conversion tests cover the inverse and a horizontal letterbox offset. The latter tests cannot inspect the OS cursor under Wayland, however: `SetPosition()` writes the requested logical point into `InputManager` *before* attempting conversion and warp, so immediately reading that point back proves the cache assignment even if the physical cursor landed elsewhere. The former ASCII renderer's integration test uses this same weak immediate set/get shape.

Other renderers in the inherited audit cohort are not interchangeable:

- EasyGL and Canvas register an offset-free height scale. It matches their default fixed-height, dynamic-width mode, but not the other presentation modes they store.
- WebGPU and SDL GPU implement all five viewport modes, but their transform uses physical pixel dimensions for SDL event and warp values expressed in window coordinates. A high-pixel-density display therefore introduces a density-factor error. Reads in a letterbox bar also return `false`, causing raw window coordinates to replace the outside logical point the method had already computed.
- Vulkan has no override or registry entry. Its `SpriteBatch` projection stretches the virtual canvas across the physical swapchain, so absolute input becomes misaligned after a non-1:1 resize. Keeping its window-coordinate size equal to the requested virtual size is the only portable caller-side guard today.
- BGFX and D3D9/11/12 also pass through, but those renderers in the inherited audit cohort ignore logical presentation and draw in physical coordinates; input matches that limitation rather than the requested mode.
- `FREEDIRECT`'s correct direct helper methods are shadowed by the SDL renderer created inside `free-direct`. Public mapping is pass-through until the first `Present()` installs its hardcoded letterbox, then follows SDL's mapping. Its four-property test invokes the helpers directly and does not exercise this public route.

The complete inherited fourteen-product matrix and its test evidence are in Chapter 19, §19.2.1. FNA has no corresponding renderer split: `Mouse.GetState()` multiplies window X/Y by the independent backbuffer/window ratios, and `SetPosition()` applies their inverses. CNA needs more machinery because its presentation modes include offsets and dynamic logical dimensions, but callers should not mistake that richer vocabulary for uniform implementation.

44.4 GamePad

The full XNA surface — `GetCapabilities`, `GetState` with configurable `GamePadDeadZone` handling, `SetVibration` — is wired to real `SDL_Gamepad` hardware, not stubbed. CNA exposes at most four player slots; an out-of-range player-index override is clamped rather than creating a fifth slot. Dead-zone constants match FNA exactly (`LeftDeadZone` = 7849/32768,

`RightDeadZone = 8689/32768`, `TriggerThreshold = 30/255`). Beyond the strict XNA surface, an extensive EXT layer exposes real hardware capability FNA itself already extends beyond stock XNA: GUID, light-bar and trigger-rumble control, gyroscope/accelerometer reads, and several CNA-only additions with no FNA counterpart at all (player-index assignment, power info, button-label queries, device name/path/serial/firmware, Steam handle, connection state, touchpad finger tracking).

44.4.1 Bugs found and fixed

Historical note. The gamepad route originally omitted `SDL_INIT_GAMEPAD`, so it received no events. Another implementation queried rumble support by calling `SDL_RumbleGamepad(0,0,0)`, which stopped active vibration; it now uses a non-mutating property query. Stick normalization used 32768 for negative samples instead of FNA's 32767, making -16384 yield -0.5 instead of approximately -0.50001 . Finally, the initialized gamepad subsystem was not shut down on device recreation. An idempotent `ShutdownGamepadSubsystem()` now runs during game disposal.

44.4.2 GamePadState and dead-zone modes

`GamePadState` composes four sub-structures, each a plain value carrier read directly from its header: `GamePadThumbSticks` (Left / Right, each a `Vector2`), `GamePadTriggers` (Left / Right, each a scalar float), `GamePadButtons` (eleven `ButtonState` properties — every face/shoulder/stick-click/back/start/big button — plus a `ButtonStateFromFlag(Buttons)` convenience for looking one up by the `Buttons` flag enum instead of by name), and `GamePadDPad` (Up / Down / Left / Right, each a `ButtonState`). `GamePadState` itself adds `IsConnected`, `PacketNumber` (incremented when the hardware state changes), and `IsButtonDown` / `IsButtonUp` convenience wrappers over the same `Buttons` flag enum.

The three `GamePadDeadZone` modes differ in shape as well as threshold. `None` filters nothing. The default `IndependentAxes` excludes and clamps each axis separately, which can distort direction near the boundary. `Circular` excludes by vector magnitude and clamps to the unit circle. `None` and `IndependentAxes` retain a square clamp, so changing modes can also change maximum diagonal magnitude.

```

1  // Dead-zone-aware movement with a circular clamp and rumble
   feedback.
2  const GamePadState pad = GamePad::GetState(PlayerIndex::One,
   GamePadDeadZone::Circular);
3  if (pad.isConnectedProperty())
4  {
5      const Vector2 moveInput = pad.getThumbSticksProperty().
   getLeftProperty();
6      playerPosition += moveInput * moveSpeed * deltaSeconds;
7
8      if (pad.IsButtonDown(Buttons::A))
9      {
10         // Independent left/right motor strengths in [0, 1].
11         GamePad::SetVibration(PlayerIndex::One, 0.0f, 0.6f);
12     }
13 }
```

Beyond the strict surface, `GetGyroEXT` / `GetAccelerometerEXT` expose a controller's built-in motion sensors, distinct from host-device sensors. They require hardware that reports them, and `GamePadCapabilities::getHasGyroEXTProperty()` / `getHasAccelerometerEXTProperty()`

exist specifically so calling code can check before relying on either. `SetLightBarEXT` and `SetTriggerVibrationEXT` (independent trigger motors rather than the two whole-controller motors) are modern-controller extensions, gated by their `GamePadCapabilities` flags.

44.5 Touch and gestures

`TouchPanel` is static-only (`MAX_TOUCHES = 8`), providing `GetCapabilities()`, `GetState()`, and `ReadGesture()`. `TouchCollection` is immutable only by contract, not by construction — its `IsReadOnly` flag is advisory, matching an identical inconsistency already present in FNA’s own implementation. `GestureType` is an eleven-value bit-flag enum (Tap, DoubleTap, Hold, Drag, Flick, Pinch, and their variants). One value is `None`; all ten actual gesture types are implemented by `GestureDetector`, including both completion events. There is no declared gesture that silently lacks a detector.

Historical note. The touch route once ignored `SDL_EVENT_FINGER_CANCELED`, leaving canceled touches active. `GetCapabilities()` used only an observed-touch flag, so an untouched enumerated device appeared disconnected. Touch-state advancement also occurred inside `GetState()`, making repeated reads mutate a frame. The current implementation handles cancellation, enumerates SDL devices with the observed flag as a fallback, and advances state once per frame through `AdvanceTouchFrame()`.

44.5.1 TouchCollection, TouchLocation, and GestureSample

`TouchCollection` is a per-frame snapshot of tracked touches. It provides `Count / IsConnected` (the sticky “a touch has been observed” flag described below), the by-index `operator[]`, and the same general ordered-container surface Chapter 7’s own `CurveKeyCollection` coverage already described — `Contains / IndexOf`, a `CNAEXT empty()`, and `CNAEXT` iterator support for range-based `for`. Each `TouchLocation` inside it carries an `Id` stable across frames for one finger, a `State` (`TouchLocationState`: `Pressed`, `Moved`, `Released`, or `Invalid` for a stale/expired entry), a `Position`, and (`CNAEXT`) a `PressureEXT` reading on hardware that reports one. `GestureSample` (returned by `TouchPanel::ReadGesture()`) carries a `GestureType`, a `Timestamp`, and up to two positions/deltas (`Position / Position2`, `Delta / Delta2`); the second pair is used only by two-finger gestures like `Pinch`, left at their default for single-finger gestures.

The two connected flags do not use the same evidence. `TouchPanel::GetCapabilities()` enumerates SDL touch devices on every call, then falls back to the sticky observed-touch flag or a currently live touch. It can therefore report a device before the first interaction. `TouchCollection::IsConnected`, in contrast, reads only that sticky flag; it can remain false for an enumerated but untouched device until the first finger-down event. Use capabilities for device discovery and the collection’s count for whether a finger is currently tracked.

```

1  // Per-finger drag tracking uses the stable Id, not collection
   position.
2  const TouchCollection touches = TouchPanel::GetState();
3  for (const TouchLocation& touch : touches)
4  {
5      switch (touch.getStateProperty())
6      {
7          case TouchLocationState::Pressed:
8              activeDrags[touch.getIdProperty()] = touch.
                getPositionProperty();
9              break;

```

```

10     case TouchLocationState::Moved:
11         if (auto it = activeDrags.find(touch.getIdProperty()); it !=
            activeDrags.end())
12         {
13             const Vector2 delta = touch.getPositionProperty() - it->
                second;
14             PanCamera(delta);
15             it->second = touch.getPositionProperty();
16         }
17         break;
18     case TouchLocationState::Released:
19     case TouchLocationState::Invalid:
20         activeDrags.erase(touch.getIdProperty());
21         break;
22     }
23 }
24
25 // Gestures are read separately from raw touches, and only the types
26 // enabled via EnabledGestures are ever produced by ReadGesture().
27 TouchPanel::setEnabledGesturesProperty(GestureType::Pinch |
    GestureType::Flick);
28 while (TouchPanel::getIsGestureAvailableProperty())
29 {
30     const GestureSample gesture = TouchPanel::ReadGesture();
31     if (gesture.getGestureTypeProperty() == GestureType::Pinch)
32     {
33         // Position/Position2 are the two fingers' current positions
34         ;
35         // Delta/Delta2 (unused by single-finger gestures) hold
36         their
37         // per-frame motion -- combine both to derive a zoom factor.
38         ZoomCamera(gesture.getPositionProperty(), gesture.
            getPosition2Property());
39     }
40 }

```

`EnabledGestures` is an active filter: `ReadGesture()` returns only enabled types. `GestureType` uses powers of two from `Tap = 1` through `PinchComplete = 512`, so callers combine selections with `|`.

`GestureSample::Timestamp` deliberately differs from an FNA defect. FNA computes it as `TimeSpan.FromTicks(Environment.TickCount)` — but `TickCount` is a millisecond value, while `FromTicks` expects 100-nanosecond units. CNA converts milliseconds to ticks. Neither engine defines an absolute epoch for the field; the regression establishes non-negative, strictly increasing values across two gestures under an advanced test clock.

44.6 TextInputEXT and CNA-only CNAEXT extensions

`TextInputEXT` is not itself a CNA invention — FNA ships this same extension beyond the strict XNA 4.0 surface, and CNA ports it directly: `TextInput` / `TextEditing` events, `StartTextInput` / `StopTextInput`, on-screen-keyboard visibility, and input-rectangle placement. The implementation has three event channels: committed `TextInput`, in-progress `TextEditing`, and the CNA-only `TextEditingCandidatesEXT` candidate-list event. CNA also adds an explicit on-screen-keyboard/IME type hint.

The `CNA::Input` namespace also hosts these CNAEXT subsystems: `Clipboard` (system clipboard

text); **Joysticks** (the raw, unmapped SDL joystick API — axes, buttons, hats, trackballs — deliberately independent of **GamePad**’s own mapped view of the same physical device); **Sensors** (host device accelerometer/gyroscope, distinct from a gamepad’s own motion sensors covered above); **Power** (battery state); and **Haptics** (SDL3 force feedback). Haptic actuation requires capable hardware *and* an initialized SDL haptic subsystem; most gamepads support only simple rumble and this layer does not initialize `SDL_INIT_HAPTIC` itself). This entire `CNA::Input` extension layer is compiled unconditionally, even when `CNA_DEVICES` and `CNA_CNAEXT` are off. **Clipboard** and **Power** overlap the optional `CNA::Devices` surface; because `CNA_DEVICES` defaults off, the input versions are the only clipboard and host-power APIs present in a default build.

44.6.1 TextInputEXT contract

`TextInput` is a `System::MulticastAction<charcs>` that fires once per composed UTF-16 unit; a non-BMP code point arrives as a surrogate pair. `TextEditing` carries the uncommitted IME candidate, cursor start, and selection length. `StartTextInput()` and `StopTextInput()` enable or disable this event period, and `IsTextInputActive()` reports it. `SetInputRectangle()` positions a composition popup in logical window coordinates, while `IsScreenKeyboardShown()` reports a platform keyboard. The CNA-only `StartTextInputWithTypeEXT()` adds a plain-text, numeric, email, or similar mobile keyboard hint.

```

1  // An in-game chat box consumes composed text rather than raw key
   states.
2  System::MulticastAction<charcs>::Token textInputToken =
3      System::MulticastAction<charcs>::InvalidToken;
4
5  void ChatBox::Open()
6  {
7      TextInputEXT::SetInputRectangle(chatBoxScreenBounds);
8      TextInputEXT::StartTextInput();
9      if (textInputToken == System::MulticastAction<charcs>::
   InvalidToken)
10     {
11         textInputToken = TextInputEXT::TextInput.Add(
12             [this](charcs c) { composedText += c; });
13     }
14 }
15
16 void ChatBox::Close()
17 {
18     TextInputEXT::StopTextInput();
19     TextInputEXT::TextInput.Remove(textInputToken);
20     textInputToken = System::MulticastAction<charcs>::InvalidToken;
21     composedText.clear();
22 }
```

44.6.2 CNAEXT-only device subsystem examples

These five subsystems are small static CNAEXT classes with no XNA counterpart. **Clipboard** wraps SDL with `GetTextEXT()`, `SetTextEXT(text)`, and `HasTextEXT()`. **Joysticks** exposes the *raw*, unmapped view of connected joystick hardware (`GetJoysticksEXT()` enumerates every connected device; `GetCapabilitiesEXT(id)` / `GetStateEXT(id)` read one device’s shape and current state directly by axis/button/hat index) — deliberately independent of **GamePad**’s

own mapped, Xbox-360-shaped view of the same physical hardware, for the rarer case where an application needs a device's true native layout rather than GamePad's normalized abstraction. `Sensors` (`GetSensorsEXT()`, `GetAccelerometerEXT` / `GetGyroscopeEXT`) reads the *host device's* own motion sensors — a laptop's built-in accelerometer, a Steam Deck's gyro — distinct from a gamepad controller's own motion sensors already covered under `GamePad`'s EXT surface above. The current implementation never initializes `SDL_INIT_SENSOR` itself, however. Unless another subsystem has already initialized SDL sensors, enumeration is empty and both readers return `false`; fake-backend tests do not expose this latent production no-op. `Power` is a single method, `GetInfoEXT(secondsLeft&, percent&)`, returning a `PowerStateEXT` plus two output parameters — the host device's own battery state, not a gamepad's (that is `GamePad::GetPowerInfoEXT` instead).

```

1 // Host and controller batteries use separate query methods.
2 int hostSecondsLeft = 0, hostPercent = 0;
3 if (Power::GetInfoEXT(hostSecondsLeft, hostPercent) == PowerStateEXT
4     ::OnBattery
5     && hostPercent < 15)
6 {
7     ShowLowBatteryWarning("this device");
8 }
9
10 int gamepadPercent = 0;
11 if (GamePad::GetPowerInfoEXT(PlayerIndex::One, gamepadPercent) ==
12     PowerStateEXT::OnBattery
13     && gamepadPercent < 15)
14 {
15     ShowLowBatteryWarning("controller 1");
16 }

```

`Haptics` maintains device and effect lifetimes. `GetHapticsEXT()` enumerates force-feedback-capable devices, and `OpenEXT(id)` / `OpenFromJoystickEXT(joystickId)` / `OpenFromMouseEXT()` each return an owning `HapticDevice` handle. That handle's own surface splits cleanly into two tiers: a simple `InitRumbleEXT()` / `PlayRumbleEXT(strength, lengthMs)` / `StopRumbleEXT()` trio for the common single-motor-strength case (the same conceptual level as `GamePad::SetVibration`, just reachable for non-gamepad haptic hardware too), and a full `HapticEffectEXT` lifecycle — `CreateEffectEXT` / `UpdateEffectEXT` / `RunEffectEXT` / `StopEffectEXT` / `DestroyEffectEXT`, plus `GetEffectStatusEXT` to poll whether an effect is running. Shaped feedback such as a directional kick or sustained curve is available only when `IsEffectSupportedEXT` accepts that effect type.

Like `Sensors`, this wrapper assumes its SDL subsystem already exists. No input-module path calls `SDL_InitSubSystem(SDL_INIT_HAPTIC)`, and the focused haptic tests replace the backend rather than exercising subsystem startup. Unless the host or another module has initialized it, enumeration/open calls can fail before hardware capability is relevant. Treat subsystem ownership as an application integration requirement for both extension families.

```

1 // A haptic-capable mouse is outside GamePad::SetVibration's scope.
2 if (Haptics::IsMouseHapticEXT())
3 {
4     HapticDevice mouseHaptic = Haptics::OpenFromMouseEXT();
5     if (mouseHaptic.IsOpenEXT() && mouseHaptic.InitRumbleEXT())
6     {
7         mouseHaptic.PlayRumbleEXT(0.5f, 150); // half strength,
8         150 ms
9     }
10 }

```

```

9      // mouseHaptic disposes (closes the device) automatically at
      scope exit.
10 }

```

44.7 Focus loss and held input

Current contract. Desktop focus loss and gain update `Game::IsActive` and raise the corresponding events. Keyboard and mouse snapshots are retained across focus loss, matching FNA. Gate gameplay input on `IsActive`; do not expect focus loss to synthesize key-up or button-up events.

Historical note. `Game::PollEvents()` once handled only mobile background/foreground events. It now also handles `SDL_EVENT_WINDOW_FOCUS_LOST` and `FOCUS_GAINED`. FNA additionally restores an X11 fullscreen-desktop flag and toggles the screensaver; CNA does not reproduce those side effects at the pin. `WindowFocusLostDoesNotClearHeldKeysMatchingFna` preserves the held-state policy.

```

1  // Gate held input on the current focus state.
2  void MyGame::Update(GameTime& gameTime)
3  {
4      if (!getIsActiveProperty())
5      {
6          // CNA retains keyboard and mouse snapshots while unfocused,
          but
7          // gameplay should ignore them here.
8          return;
9      }
10
11     const KeyboardState keyboard = Keyboard::GetState();
12     if (keyboard.IsKeyDown(Keys::Space))
13     {
14         // Process held input only while the window is active.
15     }
16 }

```

44.8 Platform-specific boundaries

A few platform constraints are external to CNA. On Wayland, `SDL_GetGlobalMouseState` silently returns (0,0), because the compositor's own security model forbids querying the global pointer position at all — absolute mouse warping is similarly focus-gated, while relative (pointer-lock) mouse mode works normally. On Windows, XInput controllers report no USB vendor/product ID, so the `EXT` GUID query returns "xinput" instead of a hexadecimal identifier. Some platforms enumerate a touch device only after its first interaction. CNA's capability query therefore combines live SDL enumeration with an observed-touch fallback; the separate collection flag retains the first-touch boundary described above. In a browser via Emscripten, gamepads remain invisible to SDL until the user presses a button on one, a browser privacy restriction, and relative mouse mode maps onto the Pointer Lock API, which itself requires a preceding user gesture to engage. Non-US keyboard layouts expose an XNA limitation: several accented and non-ASCII characters common on European layouts have no corresponding `Keys` enum value at all and are simply dropped in keycode mode — `TextInputEXT` is the correct API for capturing that text instead.

44.9 Status

The 522-case corpus stated at the chapter opening remains bounded by fifteen formally blocked, hardware-only campaign tasks: physical keyboard, mouse, gamepad, touchscreen, IME, and high-DPI display checks. These are absent observations, not automated passes. The **Joysticks** / **Sensors** / **Power CNAEXT** extensions have fake-backend unit coverage but no demo-UI or manual-hardware verification pass; **Sensors** also has the concrete subsystem-initialization gap described above.

Chapter 45

The Audio System

CNA parses XACT containers with its own `XactParser`. The default audio implementation plays through `SDL3_mixer`, but audio is now an independent build axis rather than an implied property of the window/input platform. It does not use Microsoft's XACT engine or FNA's FAudio. This choice yields usable sound effects, streaming, capture, and much of the XACT object model, but it also defines the limits: no auxiliary reverb bus, no native 3D graph, and one per-track cooked-callback slot shared by filtering and stereo crossfeed.

45.1 Audio implementation is a separate axis

`CNA_AUDIO_PLATFORM` accepts implemented values `SDL3`, `SDL2`, and `NULL`; the default is `SDL3`. It is independent of `CNA_PLATFORM` and renderer selection. A `HEADLESS` or `TERMINAL` program may retain `SDL3` audio, while an `SDL3` windowed program may select `NULL` for deterministic no-device behavior. The matching `CNA_AUDIO_PLATFORM_*` definition selects the implementation sources and links only the needed mixer edge. `OPENAL`, `WASAPI`, and `ALSA` are recognized reserved identifiers that fail configuration; they are not hidden aliases or implemented routes.

This separation is architectural, not a larger set of XNA audio APIs. The public `SoundEffect`, `DynamicSoundEffectInstance`, `XACT`, `microphone`, and `Media-facing` contracts remain the same. Capability, device enumeration, playback and capture behavior belong to the chosen audio implementation, while window/event/input behavior belongs to `CNA::Platform::IPlatform` and `pixels` belong to the active renderer.

45.2 Mixer architecture and lifetime

Under the `SDL3` audio selection, the process-wide `MIX_Mixer` requests signed 16-bit stereo at 44.1 kHz and logs the format `SDL3_mixer` negotiates. Decoded `MIX_Audio` objects hold assets; each playback uses a `MIX_Track` for gain, rate, looping, and the cooked callback. `SoundEffect` owns decoded audio, while `SoundEffectInstance` controls a track.

Mixer creation is serialized. First use pins an `SDL_INIT_AUDIO` reference so mixer destruction cannot shut down `SDL` audio while a `DynamicSoundEffectInstance` still owns an `SDL_AudioStream`. A generation counter rejects stale tracks after `DestroyMixer()`, but production code does not call that function; mixer lifetime is effectively process lifetime.

45.3 SoundEffect and instances

`SoundEffect` is move-only. It can load a file, load a stream, or accept headerless 16-bit PCM. Supplying an entire RIFF/WAV file to a raw-buffer constructor is an error; `FromStream(std::istream)` is the separate path that expects a WAV container and returns

a caller-owned pointer. `GetSampleDuration` and `GetSampleSizeInBytes` convert between PCM buffer length and time.

The vendored decoder configuration includes WAVE, AIFF, VOC, AU, FLAC, MP3, Ogg Vorbis, and MIDI routes. It does not make AAC, WMA, Opus, GME, tracker modules, or WavPack decodable. Availability still depends on the configured `SDL3_mixer` build.

```
1 std::vector<SharpRuntime::bytes> pcm =
2     SynthesizeBeep(440.0f, 0.2f); // mono, headerless S16
3 SoundEffect beep(pcm, 44100, AudioChannels::Mono);
4 beep.Play(1.0f, 0.0f, 0.0f); // volume, pitch, pan
```

`SoundEffectInstance` exposes volume, pan, pitch, looping, state, transport controls, and `Apply3D`. Process-wide `SoundEffect` values—`MasterVolume`, `DistanceScale`, `DopplerScale`, and `SpeedOfSound`—affect all live instances.

Pitch uses 2^{pitch} . Calling `Apply3D` latches the instance into its spatialized path. Reapply it when listener or emitter state changes. With normalized distance $d_n = d / \text{DistanceScale}$, attenuation is one inside the unit distance and $\text{clamp}(1/d_n, 0, 1)$ outside. Pan projects displacement onto *Forward* $\times$ *Up*, not world X. Doppler uses F3DAudio-style velocity projections, clamps its factor to $[0.5, 4]$, and guards NaN. The final rate is $2^{\text{pitch}} \times \text{doppler} \times \text{DopplerScale}$.

This is a defined stereo approximation. It lacks elevation, HRTF, multi-speaker diffusion, and per-sample spatial filtering. The cooked callback crossfeeds channels for pan instead of simply muting the opposite source channel.

```
1 AudioListener listener;
2 listener.setPositionProperty(cameraPosition);
3 listener.setForwardProperty(cameraForward);
4 listener.setUpProperty(Vector3::Up);
5 listener.setVelocityProperty(cameraVelocity);
6
7 AudioEmitter emitter;
8 emitter.setPositionProperty(explosionPosition);
9 emitter.setVelocityProperty(Vector3::Zero);
10
11 explosion.Apply3D(listener, emitter);
12 explosion.Play();
```

45.4 Dynamic streaming

`DynamicSoundEffectInstance` accepts headerless S16 chunks and, through CNAEXT, normalized float chunks. `BufferNeeded` asks for more data; `PendingBufferCount` includes queued and submitted chunks. The instance owns its `SDL_AudioStream`; the mixer does not. An exhausted track remains available for later submissions. As in FNA, `Stop(false)` throws instead of draining gracefully.

```
1 DynamicSoundEffectInstance tone(44100, AudioChannels::Mono);
2 tone.BufferNeeded.Add(
3     [&tone](System::Object*, const System::EventArgs&) {
4         while (tone.getPendingBufferCountProperty() < 2) {
5             tone.SubmitBuffer(GenerateNextSineChunk(440.0f));
6         }
7     });
8 tone.Play();
```

Queue accounting once removed a whole chunk as soon as any of it had been consumed. The current algorithm computes one consumed-byte budget and removes a chunk only when that budget covers its full size. Same-format tests lock in this behavior. Under resampling, submitted sizes are source bytes while `SDL_GetAudioStreamQueued` reports the converted stream; those units may differ. Pending count is therefore not established as a sample-accurate playback clock across 22.05/44.1/48-kHz conversion. Frame-alignment validation for integer and float submissions is also open.

45.5 XACT objects

`AudioEngine` parses `.xgs` settings and owns associated `SoundBank`, `WaveBank`, and `Cue` relationships. Wave banks can be memory-resident or offset/packet streamed. `SoundBank` disposal stops its cues.

`Cue` implements created, preparing, prepared, playing, stopping, and stopped states, plus an independent pause flag. It advances fades and Runtime Parameter Control curves and supports Ordered, OrderedFromRandom, Random, RandomNoRepeats, and Shuffle variations. `AudioCategory::SetVolume` changes cues already playing, not only future cues. `AudioEngine::Update()` must run every frame to advance fades/RPC state and collect finished fire-and-forget cues.

```

1  AudioEngine engine("Content/Audio/GameAudio.xgs");
2  SoundBank sounds(&engine, "Content/Audio/Sounds.xsb");
3  WaveBank waves(&engine, "Content/Audio/Sounds.xwb");
4
5  engine.SetGlobalVariable("PlayerSpeed", currentPlayerSpeed);
6  sounds.PlayCue("footstep"); // bank owns this fire-and-forget cue
7
8  std::unique_ptr<Cue> loop(sounds.GetCue("engine_loop"));
9  loop->Play();
10 // ... once per frame ...
11 engine.Update();

```

A cue registers with its bank in the constructor and unregisters on disposal. Earlier code registered only on `Play()`, allowing an obtained-but-never-played cue to outlive a destroyed bank. The current lifetime covers the interval from `GetCue()` through cue disposal.

`RendererDetails` contains one hardcoded `SDL3_mixer` entry. It preserves the output renderer enumeration shape but does not enumerate physical audio devices.

45.6 Content formats and historical corrections

The `XNB SoundEffectReader` accepts mono/stereo PCM 16-bit directly and wraps PCM 8-bit, IEEE float 32-bit, MS-ADPCM 4-bit, and IMA-ADPCM 4-bit for SDL decoding. XMA2 is rejected. For XNA MS-ADPCM with `cbSize=0`, the wrapper inserts the standard seven coefficient pairs.

Historical note. User reports changed the test boundary Reports of high-pitched, distorted, or missing sound led to three independent corrections: compressed XACT/XNB formats were not decoded, stereo panning discarded the opposite channel, and pitch used a linear approximation instead of 2^{pitch} . Prior tests happened to use pitch values -1 , 0 , and 1 , where the formulas agree; a midpoint exposed the difference.

The resulting `OfflineAudioRenderer` drives `SDL3_mixer`'s decode, resample, properties, callbacks, and mix into memory without an audio device. It measured output frequency within 0.1% for 22.05, 44.1, 48, and 96 kHz sources declared at their correct rates. That result excludes SDL's tested resampler route as the cause of that pitch report; it is not a perceptual quality oracle.

Memory-safety review also corrected a mixer-generation use-after-free, an XACT name-parser buffer overflow, and a WaveBank cache race between disposal and decoding. Two narrower crashes remain recorded: an intermittent late audio-filter failure and a process-teardown failure that depends on Cue/DynamicSoundEffectInstance cross-suite order. ASan and TSan perturb both reproductions. They are open findings, not evidence that every normal run crashes.

45.7 Deliberate and open limits

- XACT `REPLACE_QUIETEST` instance policy is incomplete; queue and replace-oldest share behavior, and victim search is not filtered by category/same cue.
- RPC curves targeting DSP presets are ignored; no DSP preset system exists.
- Reverb and auxiliary sends are no-ops.
- Parsed per-track filters can select low-pass or high-pass, not band-pass, following the inherited bit-decoding behavior.
- `InstancePlayLimitException` exists but no production path throws it.
- Production mixer teardown is implemented but unreachable.

The release-blocker entries for high pitch, failing loads, and distorted audio remain unchecked in the pinned `plan_audio.md`. The corrections narrow those reports but do not authorize this book to mark the campaign complete.

45.8 Microphone

`Microphone` enumerates SDL capture devices, exposes a process-lifetime default/device list, captures into a stream, and raises `BufferReady`. `GetData` drains available bytes. Sample-size/duration helpers use the device sample rate.

The intended `BufferDuration` range is 100–1000 ms in 10-ms increments. The setter checks the *millisecond component*, not total milliseconds, so 1,100 ms passes and the upper-bound branch is ineffective. `Start()` requests mono S16 at 44.1 kHz but marks the state `Started` even when device opening fails; a later zero-byte read cannot distinguish that failure from a healthy empty stream. Device refresh/hotplug, permission denial, returned-format validation, frame alignment, callback cadence, and concurrent stop/read remain open.

```

1  Microphone* mic = Microphone::getDefaultProperty();
2  if (mic == nullptr) return;
3
4  DynamicSoundEffectInstance monitor(
5      mic->getSampleRateProperty(), AudioChannels::Mono);
6  auto token = mic->BufferReady.Add(
7      [mic, &monitor](System::Object*, const System::EventArgs&) {
8          std::vector<SharpRuntime::bytecs> bytes(4096);
9          int count = mic->GetData(bytes);
10         if (count > 0) {
11             bytes.resize(count);

```

```
12         monitor.SubmitBuffer(bytes);
13     }
14 });
15
16 mic->Start();
17 monitor.Play();
18 // Before monitor leaves scope:
19 mic->Stop();
20 mic->BufferReady.Remove(token);
```

45.9 Evidence status

Source and focused tests establish the listed format routes, scalar 3D formulas, state and lifetime rules, XACT parsing bounds, and offline waveform measurements. They do not establish perceptual equivalence to XACT/FAudio, hardware capture behavior, resampled pending-count accuracy, or the unresolved release-blocker campaign. “Playback implemented” and “audio conformance complete” are therefore different claims at this revision.

Chapter 46

The Media System

Media is no longer the shell-heavy namespace described by CNA’s older README count. At the pinned revision it contains file-backed songs and libraries, playlist parsing, queued music playback, visualization data, picture libraries, and FFmpeg video on the platforms where FFmpeg is enabled. This chapter states those behaviors by subsystem; it does not replace the stale README count with another undated test total.

Audio and Media form an intentional static-library cycle. **FrameworkDispatcher** in Audio advances **MediaPlayer**, while Media sends song playback to the audio mixer. CMake repeats the archives to resolve that link cycle. Removing either edge would require reassigning the shared dispatcher and playback responsibilities.

46.1 Song contract

Song can be constructed from a path or discovered by a **MediaLibrary** scan. A directly constructed song has null, non-owning **Album**, **Artist**, and **Genre** links; a scan supplies those links from the library hierarchy. **IsRated** and the 0–10 **Rating** are derived from ID3v2 POPM or Vorbis RATING tags. A source rating of zero means “unrated.” **IsProtected** is always false because protected files are not indexed.

Duration and **PlayCount** are mutable. Playback replaces the duration with the decoder-reported value, and **MediaPlayer::Stop()** resets play counts across the current queue. **Handle** is a CNAEXT path/handle accessor. **Equals** compares handles; **GetHashCode** hashes the resolved handle. The latter deliberately avoids FNA’s identity-hash inconsistency for two equal songs.

```
1 std::unique_ptr<Song> intro(  
2     Song::FromUri("Intro Theme", "Content/Audio/intro.ogg"));  
3 MediaPlayer::Play(intro.get());  
4 // MediaPlayer clones the Song; the caller still owns intro.
```

The fragment assumes the Media header, <memory>, and the applicable namespaces.

Historical note. FNA is not the complete API oracle FNA’s **Song** omits **Album**, **Artist**, **Genre**, and **ToString()** from the XNA 4.0 surface. Repeated FNA-only reviews missed the difference; comparison with the pinned xna4-spec data found it. Chapter 76 explains why API parity with FNA and with XNA are separate claims.

Limitation. Advertised loose-file extensions The ContentManager song reader advertises .aac and .wma, but CNA’s vendored SDL3_mixer has no decoder for either format. The MediaLibrary scanner omits them. A loose-file resolution match therefore establishes that

a `Song` can name the file, not that the mixer can decode it. MP3, Ogg, WAV, FLAC, and Opus follow the installed decoder routes.

46.2 MediaLibrary and metadata

`MediaLibrary` scans the host music and picture directories returned by `SDL_GetUserFolder`; a CNAEXT override makes tests independent of the CI account's files. It builds `SongCollection`, `Album`, `Artist`, and `Genre` relationships, plus the parallel picture hierarchy. `SavePicture()` creates a "Saved Pictures" node on first use.

The in-tree metadata parser handles:

- Ogg Vorbis comments and Ogg Opus `OpusTags`;
- ID3v2.3/2.4 title, artist, album, genre, track, rating, and picture frames;
- Latin-1, UTF-16 with byte-order mark, UTF-16BE, and UTF-8 ID3 text;
- native FLAC `VORBIS_COMMENT` and picture blocks;
- `.m3u` and `.m3u8` playlists as UTF-8.

Files without readable tags fall back to filename, parent directory, and grandparent directory for title, album, and artist. Missing playlist entries are skipped. Album art comes from `cover.jpg/folder.jpg` or embedded ID3/FLAC pictures; the front-cover picture type wins when present.

The conversion from ID3's 0–255 rating and the various informal Vorbis rating scales to XNA's 0–10 property is CNA policy, not an XNA rule. Likewise, scans are synchronous snapshots; there is no `MediaLibrary::Refresh()`.

Historical note. Testing values across layer boundaries The tag parser once read `TRCK` correctly while `MediaLibrary` discarded the value when constructing `Song`. A fixture with several distinct track numbers exposed the drop; a parser-only test could not. The current integration fixture checks the values after library construction.

46.3 Playback and queue ownership

`MediaPlayer` supplies song and collection playback, pause, resume, stop, shuffle, repeat, queue navigation, events, and visualization. `Playlist` is read-only, as on XNA. `MediaQueue::Add(Song*)` adopts its pointer, whereas `SongCollection` is non-owning. To keep the queue from deleting library-owned songs, `MediaPlayer::Play` clones each input song and enqueues the clone.

In linear mode, next/previous clamp at the ends unless repeat is enabled. With repeat, the end wraps to the beginning. Shuffle chooses uniformly from the full queue, so a track may repeat immediately. Sound-enabled builds use `SDL3_mixer`'s stopped callback to advance. The no-sound fallback compares elapsed time with a known, nonzero duration; a zero-duration song does not auto-advance.

Volume and mute are independent state. Setting volume while muted retains the new value for the next unmute; the mixer alone receives zero gain while muted. `MediaStateChanged` and `ActiveSongChanged` are deferred until `FrameworkDispatcher::Update()`. `Game` calls the dispatcher during its normal update loop, but a caller using the static media API outside `Game` must pump it explicitly.

46.4 Visualization

`MediaPlayer::GetVisualizationData()` consumes samples captured by `SDL3_mixer`'s post-mix callback. The callback writes to a single-producer/single-consumer ring buffer without allocation, locking, or exceptions. Sample slots are `std::atomic<float>` with relaxed ordering so audio-thread writes and game-thread reads are defined C++ behavior. A reader may still observe samples spanning adjacent callback batches; the resulting one-frame visual imprecision is accepted.

A 512-sample Hann-windowed radix-2 FFT produces XNA's 256 bins. CNA scales a full-amplitude sine to approximately 1.0 in its bin; XNA did not document a normalization. Disabled or not-yet-fed visualization returns zero-filled arrays. Tests cover failed installation and removal of the mixer tap so the enabled flag does not claim a callback that is absent or cannot be removed.

46.5 Video contract

Where `CNA_FFMPEG_AVAILABLE` is enabled, `VideoPlayer` uses FFmpeg for decoding, a CNA renderer texture for frames, and `SDL3` audio streaming. `Play()` rejects a declared width, height, or frame rate that disagrees with the decoded stream. `VideoSoundtrackType` is meta-data only in both CNA and FNA; it does not duck or mute tracks.

The public methods check disposed state and throw `ObjectDisposedException` after `Dispose()`. `Dispose()` itself is idempotent. This differs from FNA because the C++ destructor also calls `Dispose()`; throwing during that destructor could terminate the process.

`SetAudioTrackEXT` and `SetVideoTrackEXT` select streams independently. A live audio switch reopens only the audio output; a live video switch recreates the frame texture if dimensions change. The audio buffer is cleared even when no audio device or audio track is active, preventing unbounded accumulation and stale samples on a later play. Tests use a multi-track fixture to observe sample-rate change and verify that the unrelated video texture is retained.

```
1  VideoPlayer player;
2  player.Play(introVideo);
3  player.SetAudioTrackEXT(1); // switches audio without recreating
    video output
```

46.6 Platform limitations

The pinned CMake configuration disables FFmpeg on Windows (MinGW and MSVC), Android, and Emscripten. Video translation units are excluded on those targets, so using `Video/VideoPlayer` is a link-time failure, not a runtime `NotSupportedException`. Linux and macOS are the intended FFmpeg-enabled routes when the required development packages are available.

There is also a native-Windows guard mismatch. CMake's unavailable set includes `WIN32`, but two `ContentManager.cpp` preprocessor guards check Emscripten, Android, and MinGW; one repeats `__MINGW32__` and neither checks `_WIN32`. An MSVC build can therefore retain `Video` dispatch references after the implementations were excluded. This is an open pinned-revision build defect.

46.7 Evidence status

Source and focused tests establish parsing, hierarchy construction, queue ownership, event timing, visualization state, disposal, and multi-track reconfiguration. FFmpeg fixture tests exercise decoding on the configured host. Those results do not establish video availability on the CMake-excluded targets, nor do extension lists establish decoder support. The platform gate and Windows mismatch are source-proven; no Windows video execution is claimed.

Chapter 47

Sensors and Microsoft.Devices

`Microsoft::Devices::Sensors` ports the Windows Phone 7 sensor model to SDL3 and Android NDK inputs. The CNA-original host utility layer in Chapter 48 is separate. Sensor callbacks may run outside the game thread, which makes lifecycle and synchronization part of the public usage contract.

47.1 Sensor model and delivery

`SensorBase<TSensorReading>` supplies `CurrentValue`, `IsDataValid`, `TimeBetweenUpdates` (default 2 ms), `CurrentValueChanged`, and abstract `Start/Stop`. Do not touch `GraphicsDevice` or other main-thread-only objects in a sensor handler; transfer the reading through a synchronized queue.

```
1 Accelerometer accelerometer;
2 accelerometer.CurrentValueChanged.Add(
3     [](System::Object*,
4         const SensorReadingEventArgs<AccelerometerReading>& args) {
5         Vector3 acceleration =
6             args.getSensorReadingProperty().getAccelerationProperty(
7                 );
8         // Enqueue acceleration for the game thread.
9     });
10 accelerometer.Start();
11 // ... later ...
12 accelerometer.Stop();
13 accelerometer.Dispose();
```

Desktop Accelerometer and Gyroscope install an SDL event watch rather than a polling thread. `SDL_EVENT_SENSOR_UPDATE` is handled on the thread that pushes it. Acceleration is converted from SI units to *g* by division by 9.80665; angular velocity is already in radians per second. Readings receive UTC wall-clock timestamps.

Android Compass and Motion use `ASensorManager`, event queues, and `ALooper` on worker threads, without JNI. A Motion instance can own six workers for rotation, game rotation, gravity, linear acceleration, gyroscope, and magnetic field. The native route expects Android API 24 or later.

Type	Implemented route	Boundary
Accelerometer	SDL3 on desktop, Android, and iOS	WP7 legacy <code>ReadingChanged</code> also fires
Gyroscope	SDL3 sensor events	CNAEXT <code>State</code> ; no legacy event
Compass	Android NDK only	Unsupported elsewhere; <code>Start()</code> throws
Motion	Android NDK fusion only	Unsupported elsewhere; six possible workers

47.2 Concurrency contract

Disposal is claimed atomically so only one racing caller performs cleanup. A losing caller waits for terminal state, and an RAI guard publishes that state even if cleanup throws. Accelerometer and Gyroscope track callbacks in flight so a handler can dispose its own sensor without waiting for itself.

Current value and validity are published under one lock by `SetCurrentValueAndMarkDataValid`. Separate locks previously allowed a reader to observe `IsValid=true` with the prior value. Update throttling compares elapsed time at the configured interval's coarser resolution; the earlier reverse conversion overflowed signed 64-bit ticks for `TimeSpan.MaxValue`.

Compass and Motion use a two-phase start/stop protocol: reserve under the owner mutex, invoke the backend without that lock, then commit or roll back. A condition variable prevents a new start while an orphaned start's cleanup call is still in flight. Stop deliberately remains able to supersede a start without waiting on it.

Historical note. Why sanitizer evidence matters here The first lifecycle rewrite appeared race-free under ordinary tests and manual review. ThreadSanitizer found that Stop cleared a transition flag before an orphaned Start had finished its backend cleanup, allowing a third Start to enter concurrently. The stress test reproduced both the race and a use-after-free in callback bookkeeping. The in-flight-call counter and condition variable are the current fix. UndefinedBehaviorSanitizer separately found the update-interval overflow.

Exceptions cannot cross SDL, Android C callbacks, or raw thread entry points. CNA records backend, operation, native error, device, timestamp, and severity through a `noexcept` diagnostic channel. The Android bridge distinguishes `std::exception::what()` from an unknown throw. Other device paths still retain older ad hoc counters, so the shared diagnostic record is not yet universal.

47.3 Compass and motion math

Compass uses upright heading when gravity meets the $\cos 45^\circ$ threshold; otherwise it derives flat heading from the quaternion. Results are normalized to $[0, 360)$, and an invalid quaternion maps to 0° . Accuracy values High/Medium/Low/Unreliable map to 5/15/20/180 degrees. Without declination data, `TrueHeading` equals `MagneticHeading`.

Motion normalizes Android's rotation quaternion and derives pitch, yaw, and roll. A process-wide CNAEXT landscape remap affects acceleration, rotation rate, gravity, and linear acceleration, but not `Attitude`: the requested axis reflection has determinant -1 and cannot be represented by a quaternion. Fusion accepts the latest readings within 500 ms; it does not interpolate them to one timestamp.

47.4 Vibration

`VibrateController` is a process-lifetime singleton. `Start(TimeSpan)` accepts the WP7 range `[0, 5s]`; `Start(TimeSpan, float)` and `StartLeftRight` are CNAEXT intensity controls. SDL's Android haptic route blends motor intensities; independent left/right controller rumble is available through `GamePad`, not this phone-style API.

```
1  VibrateController* vibration = VibrateController::getDefaultProperty  
    ();  
2  if (vibration->getIsSupportedProperty()) {  
3      vibration->StartLeftRight(  
4          1.0f, 0.3f, System::TimeSpan::FromSeconds(0.2));  
5  }  
6  // Start(duration, 0.0f) still installs a zero-strength effect.  
7  vibration->Stop();
```

`DevicesShutdownCoordinator::Shutdown()` must precede the application's `SDL_Quit()`. It prevents the singleton's later static destructor from closing an SDL device after SDL has already released native state. One global mutex serializes sensor and haptic subsystem initialization and preserves the documented per-instance-before-global lock order.

47.5 Known limitation and evidence status

All four concrete sensors expose `Dispose(bool)` publicly even though the base declares it protected. External `Dispose(false)` marks the instance disposed without stopping it, decrementing use counts, or releasing SDL ownership. This is an open API/lifetime defect; call the parameterless public `Dispose()`.

Focused tests cover math, throttling, lifecycle, fake backends, reentrancy, and TSan stress. They do not constitute hardware QA. The Android source notes that its sign and zero conventions have not been checked on a physical device, and the Android demo is an older copy of the desktop demo. Android behavior is source-proven and host-tested where fakes permit, not device-verified.

Chapter 48

Host Integration through CNA.Devices

`CNA::Devices` is an SDL3 host-integration layer with no XNA or Windows Phone precedent. It is also absent from a default CNA build. Both facts belong before its API tour because code that compiles in a devices-enabled demo can otherwise look like an unconditional framework surface.

48.1 The option removes the types

`CNA_DEVICES` defaults to `OFF`. It gates all 25 installed headers, 14 production sources, and ten test files in `modules/devices-ext`; with the default setting the archive contains object files with no defined API symbols and its tests register no cases. Names such as `Camera`, `FileDialog`, and `SystemTray` do not merely throw at runtime—they do not exist as C++ types.

The option does *not* gate Chapter 47’s `Microsoft::Devices::Sensors` module. The option help text’s broad “device/sensor” wording is therefore misleading: `Accelerometer`, `Gyroscope`, `Compass`, `Motion`, and `VibrateController` compile in every configuration, while this chapter’s namespace is entirely conditional.

Enable it explicitly at configure time:

```
1 cmake -S . -B build -DCNA_DEVICES=ON
```

Any public header or application source using these types must be compiled with the same definition propagated by CNA’s build targets; hand-defining the macro without linking the enabled archive only postpones failure to link time.

48.2 The SDL wrapper map

The implemented classes are deliberately thin:

Clipboard gets, sets, and probes SDL clipboard text;

DisplayInfo exposes a window’s display scale and safe interactive area;

Locale returns SDL’s preferred locale list;

MessageBox shows simple or multi-button native dialogs;

FileDialog opens file, save, and directory choosers asynchronously;

SystemTray owns a tray icon and flat callback-driven menu;

PowerInfo exposes host power state, percentage, and remaining seconds;

SystemInfo reports logical CPU cores and physical RAM;

UrlLauncher forwards a URL to the platform; and

Camera polls SDL camera frames into a **Texture2D**.

There are no **TODO**, **NotImplemented**, or throw-placeholder tokens in the module, but thin forwarding still has behavioral edges. **MessageBox** reports support unconditionally, never parents the dialog to a window, sets no default/cancel button, and does not validate an empty label list despite the header contract. **FileDialog** and **SystemTray** platform checks are advisory: show/-construct still forwards. **PowerInfo**'s three properties call **SDL_GetPowerInfo** separately, so they can represent three different snapshots. **SystemTray** exposes no usable icon object, and **UrlLauncher** does no URL validation.

48.3 Camera: poll, do not wait for a callback

Camera chooses the first reported device, requests RGBA32, and closes/reports unsupported if SDL negotiates another pixel format. Permission state is re-read on every **State** query because a platform prompt can resolve long after construction. **TryAcquireFrame** requires a destination texture with exactly the negotiated dimensions; it returns **false** without modifying the texture if permission is not ready, dimensions do not match, or no fresh frame is available.

```

1  #ifdef CNA_DEVICES
2  Camera camera;
3  std::unique_ptr<Texture2D> frame;
4
5  if (camera.getStateProperty() == CameraState::Ready)
6  {
7      if (!frame)
8      {
9          frame = std::make_unique<Texture2D>(
10             graphicsDevice,
11             camera.getFrameWidthProperty(),
12             camera.getFrameHeightProperty());
13      }
14
15      if (camera.TryAcquireFrame(*frame))
16      {
17          DrawCameraPreview(*frame);
18      }
19  }
20 #endif

```

The production tree never calls **SDL_InitSubSystem(SDL_INIT_CAMERA)**. Unless the host initializes that subsystem itself, available-camera enumeration is empty and **Camera** settles in **NotSupported**. This is a latent initialization gap, not a permission denial. The backend's **Lost** and **Closed** states are also unreachable in its current implementation.

48.4 Platform support is per class

SystemTray is genuinely desktop-only in SDL3; Android, iOS, and Emscripten have no tray backend. **FileDialog** is not equivalent: SDL3 supplies an Android implementation. Display scale and safe area depend on a real associated window. Message boxes and URL launchers inherit whatever the host platform permits, even though CNA's support predicates can be optimistic.

This makes a namespace-level “desktop utilities” label useful as orientation but wrong as a capability test. Query the particular class, expect callbacks to be platform-threaded, and retain a non-dialog fallback for sandboxed or web targets.

48.5 Overlap with CNA::Input

`Clipboard` and `PowerInfo` duplicate functionality already present as the unconditional `CNA::Input::Clipboard` and `CNA::Input::Power`. They were created on parallel workstreams and use type-incompatible enums despite wrapping the same SDL calls. With `CNA_DEVICES=OFF`, the Input versions are the only clipboard and host-battery API.

Input’s `Sensors` is not a duplicate of Chapter 47: it is a stateless, poll-oriented open/read/close API returning raw vectors, while `Microsoft.Devices` exposes stateful WP7 objects, events, throttling, and disposal. `InputDevices` has no counterpart here.

48.6 Lifetime and test boundary

File dialogs, tray menus, and camera permission all cross native asynchronous boundaries. Application-owned state captured by their callbacks must outlive the native operation, and UI callbacks should hand work back to the game thread before touching graphics state. Host shutdown must destroy these objects before global SDL teardown; the sensor/haptic side additionally uses the explicit shutdown coordinator described in Chapter 47.

There is a sharper callback-boundary gap in the present implementation. Both the file-dialog result trampoline and the tray-entry trampoline invoke caller-supplied C++ callbacks directly from an SDL C callback, with no `try/catch`. A handler that throws can therefore unwind across a C ABI boundary; callers must currently keep those handlers non-throwing. This is distinct from the sensor module’s native diagnostic policy above, which already catches and records exceptions at the corresponding boundary.

The ten devices-ext test sources exercise fake/native-wrapper contracts only when the gate is on; in a default build they silently contribute zero tests. A CI matrix that never configures `CNA_DEVICES=ON` proves neither the public types nor their platform forwarding. Camera, dialogs, tray integration, and permissions still require real-host validation beyond those unit tests.

Chapter 49

GamerServices and Local Identity

XNA tied **GamerServices** to Xbox Live. FNA retains much of the API surface on PC but leaves the service-dependent paths inert. CNA supplies offline identities, achievements, leaderboards, and two Guide overlays. It does not provide authentication, friends, matchmaking, or a remote account service.

This chapter uses three statuses:

Implemented The API has local in-memory behavior.

Locally persisted State survives through CNA's file store, with CNA-defined policy where Xbox Live supplied the original policy.

Stub The API shape exists, but no local service can perform the operation.

These statuses describe the pinned revision; they are not claims of Xbox Live compatibility.

49.1 Local identities and object lifetime

Gamer is the base of **SignedInGamer**. Initialization creates four fixed local identities: **Stub Gamer** and **Stub Gamer** (1) through (3). Player One is not a guest; the other three are guests. All four report **IsSignedInToLive=true**, although no online sign-in occurs.

GetFromGamertag, **GetPartnerToken**, and their asynchronous pairs are stubs. **IsFriend** returns **false**, and **GetFriends()** returns an empty **FriendCollection**. The friend types themselves work as containers; CNA has no source from which to populate them.

Limitation. Address stability A **Gamer** owns a **LeaderboardWriter** that stores the owning **Gamer***. The default copy and move operations copy that pointer. Moving a gamer after construction can therefore leave its writer pointing at the old address. Keep gamers at stable addresses—heap allocation is the documented choice—once their leaderboard writer may be used.

XNA exposes **Presence** as a get-only reference property whose returned object is mutable. CNA adds a non-const **getPresenceProperty()** overload so that the same usage remains possible in C++:

```
1 SignedInGamer* gamer = /* obtain from SignedInGamer::SignedIn */;  
2 gamer->AwardAchievement("ACH_FIRST_BOSS_DEFEATED");  
3 AchievementCollection earned = gamer->GetAchievements();  
4  
5 gamer->getPresenceProperty().setPresenceModeProperty(  
6     GamerPresenceMode::SinglePlayer);
```

The fragment assumes the corresponding **GamerServices** headers and namespace imports.

49.2 Persistence contract

The store is rooted at `StorageDevice::GetStorageRootEXT()`, which is based on `SDL_GetPrefPath`. Achievement files live below `GamerServices/achievements`; leaderboard files live below `GamerServices/leaderboards`. Calling `StorageDevice::SetAppNameEXT` changes the root for both systems. Chapter 50 defines that shared boundary.

Achievements use one JSON file per sanitized gamertag, with records of the form `{"key":..., "earnedTicks":...}`. Leaderboards use one file per sanitized `(key)_(gameMode)`. Characters outside `[A-Za-z0-9._-]` become underscores; an empty name becomes `_`. Writes use an adjacent temporary file followed by rename, with direct writing as a fallback. Missing or malformed files produce an empty store.

JSON numbers are represented as `double`. Achievement ticks, leaderboard ratings, and 64-bit property values therefore lose low bits above 2^{53} . For a 2026 `DateTime`, the quantization step is about 128 ticks, or 12.8 microseconds. The stored integer remains stable after the first serialization, but the original 64-bit value is not a bit-exact round trip.

49.3 Achievements

`AwardAchievement` and `GetAchievements` use the local store. Returned entries contain a persisted `Key`, `IsEarned=true`, and the earned time, subject to the numeric limitation above. `Name`, `Description`, and `GamerScore` remain empty or zero; `DisplayBeforeEarned` remains `true`. Xbox Live supplied that catalog metadata outside the `AwardAchievement(string)` call, and CNA has no equivalent catalog. Applications that display achievement names or scores must maintain their own mapping.

`Achievement::GetPicture()` throws `NotImplementedException`. Artwork is another service-owned field for which the local store has no source.

49.4 Leaderboards

`LeaderboardReader`, `LeaderboardWriter`, and `LeaderboardEntry` are disk-backed. Their policy is CNA-specific because FNA exposes no functioning PC implementation from which to inherit sorting or paging behavior:

- entries sort by rating, descending;
- paging around an absent gamer starts at the top;
- persisted entries whose gamertags have no matching live `Gamer` are omitted;
- the asynchronous pairs complete synchronously;
- assigning `Rating` on a writer-created entry persists it immediately.

A reader-created entry has no persistence hook. Changing its in-memory rating does not update the file.

```
1  LeaderboardWriter& writer = gamer->getLeaderboardWriterProperty();
2  LeaderboardIdentity board = LeaderboardIdentity::Create(
3      LeaderboardKey::BestScoreLifeTime);
4
5  LeaderboardEntry* entry = writer.GetLeaderboard(board);
6  entry->setRatingProperty(newHighScore); // the assignment is the
    commit
```

49.5 Guide overlays and stubs

Most `Guide` UI calls are no-ops: compose message, friend request, friends, game invite, gamer card, marketplace, messages, party, player review, players, sign-in, and `ShowAchievementsEXT`. `DelayNotifications` is also inert.

Two asynchronous families are CNA-rendered overlays: `BeginShowMessageBox/EndShowMessageBox` and `BeginShowKeyboardInput/EndShowKeyboardInput`. They stay pending until the game's `Draw()` path invokes the matching `RenderPending*EXT` hook with a `SpriteBatch`, `SpriteFont`, and white pixel. The hooks render the prompt and poll keyboard or mouse input. `Guide.IsVisible` reports whether either operation is pending.

Keyboard input preserves UTF-16 surrogate pairs. Password mode masks displayed code units but returns the unmasked text. Escape cancels the operation. Because `std::string` cannot distinguish XNA's null-on-cancel result from a confirmed empty string, callers must query `WasKeyboardInputCanceledEXT` before interpreting the result.

```

1  System::IAsyncResult* pending = Guide::BeginShowKeyboardInput(
2      PlayerIndex::One, "Enter Passphrase", "Protect this save file",
3      "", nullptr, std::any{}, true);
4
5  // Call once per Draw while the request is pending.
6  Guide::RenderPendingKeyboardInputEXT(
7      *graphicsDevice, spriteBatch, uiFont, whitePixel);
8
9  if (!Guide::getIsVisibleProperty()) {
10     if (!Guide::WasKeyboardInputCanceledEXT(pending)) {
11         std::string passphrase = Guide::EndShowKeyboardInput(pending
12             );
13     }
14     delete pending; // the caller owns the IAsyncResult
15 }

```

Only one `Guide` overlay can be pending. The test corpus covers pending state, callback identity, Enter and Escape, prompt fields, password display, and competing requests.

Historical note. Why the overlay contract is stated explicitly An independent review found that an earlier implementation ignored the title and description, reported `IsVisible=false`, displayed password text without masking, and had no cancel path. The current implementation and tests cover all four corrections. The episode is useful because a completed task record had overstated the observable behavior.

49.6 Presence, privileges, and profile

`GamerPresence` supplies all 60 XNA display strings and formats parameterized modes. Its `SetPresenceModeStringEXT` publication hook is empty: no service consumes the result. `GamerPrivileges` is fixed and permissive, and nothing enforces it. `GamerProfile` uses synthetic defaults except for host-derived `Region`. These objects carry useful local state, not authentication or policy authority.

49.7 Dispatcher, component, and the historical hang

`GamerServicesComponent` must be added to `Game.Components` before using the `Gamer` or `Guide` APIs, as on XNA 4.0 for Windows. Its dispatcher creates and frees the local `gamer`

set. The `Update()` method on `GamerServicesDispatcher` is empty; the local asynchronous implementations complete their own actions.

Historical note. Cross-namespace completion bug After initialization, `GamerServicesDispatcher::UpdateAsync()` returns `true` indefinitely. FNA's `NetworkSession` action logic waited for it to become `false`; combining `GamerServices` initialization with network creation could therefore spin forever. `SignedInGamer::BeginGetAchievements` once had the same dependency.

CNA's network create, find, and join work is performed synchronously by the corresponding `End*` path. Its `NetworkSessionAction` now starts completed instead of waiting for an event that cannot occur. A separate-process harness exercises the initialized dispatcher under a ten-second watchdog because its process-lifetime statics have no test reset hook. The harness also checks the four local identities, sign-in events, achievement completion, and reinitialization cleanup. Chapter 51 covers the network side.

49.8 Evidence and user implications

The pinned headers and implementations establish the API shapes, fixed identities, file layout, overlay hooks, and stubs. Unit tests cover persistence behavior and Guide state; the isolated harness covers the dispatcher interaction. These tests do not establish interoperability with Xbox Live, because CNA has no such transport.

Feature	Pinned-revision status
Local gamers	Four fixed identities; implemented in memory
Achievements	Locally persisted; catalog metadata and pictures absent
Leaderboards	Locally persisted with CNA-defined sorting and paging
Presence, privileges, profile	Local or synthetic state; no service enforcement
Friends and remote gamer lookup	Empty or unsupported
Most <code>Guide.Show*</code> calls	No-op stubs
Message-box and keyboard input	CNAEXT-rendered overlays
Avatar base API	Inert; see Chapter 52

Chapter 50

Storage and Persistence Boundaries

`Microsoft::Xna::Framework::Storage` maps XNA's device-selection API onto one host filesystem root. It provides usable save-file operations, but the abstraction is not a security sandbox: two path inputs can escape their intended roots at the pinned revision.

50.1 StorageDevice contract

`StorageDevice` reports filesystem `FreeSpace`, `TotalSpace`, and `IsConnected`. Its `Begin*/End*` pairs have XNA's asynchronous shape but complete synchronously. Selector overloads accept player, free-space, and directory-count constraints; CNA ignores the constraints because it has no removable-device choice. `DeviceChanged` is never raised.

`SetAppNameEXT(appName)` selects the per-application root and invalidates any cached resolution. Call it before storage or `GamerServices` access: achievements and leaderboards also live below this root. `GetStorageRootEXT()` exposes the resulting path.

Storage result ownership is explicit. `EndShowSelector` returns `std::unique_ptr<StorageDevice>`; `EndOpenContainer` returns `std::unique_ptr<StorageContainer>`.

```
1  StorageDevice::SetAppNameEXT("MyGame");
2
3  auto select = StorageDevice::BeginShowSelector(nullptr, nullptr);
4  std::unique_ptr<StorageDevice> device =
5      StorageDevice::EndShowSelector(select.get());
6
7  auto opening = device->BeginOpenContainer(
8      "MyGame Save Data", nullptr, nullptr);
9  std::unique_ptr<StorageContainer> container =
10     device->EndOpenContainer(opening.get());
```

50.2 Storage root resolution

The root is resolved lazily, then cached until `SetAppNameEXT` changes the app name:

1. `SDL_GetPrefPath(nullptr, app);`
2. `XDG_DATA_HOME/<app>` when `SDL` fails;
3. `HOME/.local/share/<app>`;
4. the current working directory when the environment supplies no home.

The final fallback is process-location-dependent and should not be treated as a stable save location. If the root does not yet exist, space queries return `LLONG_MAX`. For `IsConnected`,

CNA walks upward to the first existing ancestor; `true` therefore does not prove that the per-app directory has been created.

50.3 StorageContainer contract

`StorageContainer` provides create, delete, existence, enumeration, and glob-pattern operations for files and directories. Its `OpenFile` overloads return `System::IO::Stream`. Paths are intended to be relative to the container.

```

1  {
2      std::unique_ptr<System::IO::Stream> out =
3          container->CreateFile("savegame.dat");
4      std::vector<SharpRuntime::bytecs> bytes = SerializeGameState();
5      out->Write(bytes.data(), 0, static_cast<int>(bytes.size()));
6  }
7
8  if (container->FileExists("savegame.dat")) {
9      std::unique_ptr<System::IO::Stream> in = container->OpenFile(
10         "savegame.dat", System::IO::FileMode::Open);
11      std::vector<SharpRuntime::bytecs> bytes(in->getLengthProperty())
12         ;
13      int offset = 0;
14      while (offset < static_cast<int>(bytes.size())) {
15          int n = in->Read(bytes.data(), offset,
16              static_cast<int>(bytes.size()) - offset);
17          if (n == 0) break;
18          offset += n;
19      }
20      if (offset != static_cast<int>(bytes.size())) {
21          /* reject a truncated save */
22      }
23  }

```

`Dispose()` is idempotent, sets `IsDisposed`, and raises `Disposing`; it does not guard later operations. The container retains a non-owning `const StorageDevice*`, so the device must outlive it. Treat a disposed container as unusable even though the implementation still permits path operations.

50.4 Open path-containment defects

Limitation. Container name can escape the storage root `BeginOpenContainer(displayName)` retains the name verbatim. Construction joins `root/displayName/playerFolder` and rejects only an empty name. An absolute name discards the root under `std::filesystem::path::operator/;` `..` can climb above it. `DeleteContainer` uses `ResolveContainedPath` and rejects these cases, so opening and deleting do not share one boundary.

Limitation. File and directory names can escape a container The shared `ResolvePath(relative)` helper only evaluates `storagePath/relative`. It rejects an empty string but neither absolute paths nor `...` For example, `CreateFile("../../outside.txt")` can write above the container. Validate every externally influenced filename before passing it to this namespace.

The focused storage tests cover containment for `DeleteContainer`; they do not cover

`StorageContainer`, opening a container, enumeration, or the hand-written `*/*` glob matcher. The untested boundary is therefore the same boundary where the defects remain.

50.5 File sharing is not enforced

The fullest `OpenFile` overload accepts `System::IO::FileShare`, but comments the parameter name out and constructs `FileStream` with path, mode, and access only. The pinned `FileStream` has no share-mode constructor, and its `std::fstream` implementation supplies no cross-process locking policy. Consequently, `None`, `Read`, and `ReadWrite` produce the same CNA behavior. Do not use `FileShare::None` as protection against concurrent save writers.

50.6 Exception boundary and evidence

`StorageDeviceNotConnectedException` is thrown when space queries translate a filesystem exception. Container and file operations do not first check `IsConnected` or uniformly translate host errors into this type.

The public headers and implementations establish ownership, root selection, ignored parameters, and the two traversal paths. Tests establish the protected `DeleteContainer` cases and a normal deletion. Container read/write behavior is source-proven here; the pinned storage test set does not supply an end-to-end save round-trip or hostile container-path oracle. The example uses fixed names and therefore does not demonstrate containment of untrusted input.

Chapter 51

Networking and Session Semantics

CNA's `Microsoft::Xna::Framework::Net` surface combines one functional LAN route with several compatibility lifecycles. `SystemLink` uses the vendored ENet transport; `Local`, `LocalWithLeaderboards`, `PlayerMatch`, and `Ranked` do not. Code that only observes session objects can miss that distinction, so this chapter starts with it.

51.1 Session types and ownership

NetworkSessionType	Pinned-revision behavior
<code>SystemLink</code>	ENet host/join, LAN discovery, handshake, application-data relay, disconnect handling, simulated latency/loss, and host migration
<code>Local</code>	Synthetic state and events; no transport or local data delivery
<code>LocalWithLeaderboards</code>	Same synthetic lifecycle plus Chapter 49's local persistence
<code>PlayerMatch</code> , <code>Ranked</code>	Stubs; no matchmaking service

`Create`, `Find`, `Join`, and `JoinInvited` return a caller-owned `NetworkSession*`. `Dispose()` releases transport and owned gamer resources, but it does not delete the session object. The caller must perform both operations, normally through an owning smart pointer with explicit disposal where required.

The game loop must call `Update()` each frame. Initial `GamerJoined` events are different: subscribing replays the handler synchronously for gamers already in `AllGamers`. Later joins arrive through the update queue.

```
1  std::unique_ptr<NetworkSession> session(NetworkSession::Create(
2      NetworkSessionType::SystemLink, 1, 8));
3
4  session->GamerJoined.Add(
5      [](System::Object*, const GamerJoinedEventArgs& args) {
6          // Existing gamers replay now; later joins arrive from
7          Update().
8      });
9  // Once per game update:
10 session->Update();
```

Limitation. The `maxGamers` compatibility quirk The create path does not forward the caller's `maxGamers`. The constructed session reports the literal 69, preserving the FNA/XNA behavior recorded in the implementation. A game that needs an eight-player limit in the example above must enforce eight itself.

51.2 SystemLink transport

The internal `ENetBackend` uses a star topology. Clients send application data to the host; the host relays it to the target. `NetworkGamer::Id` and `IsHost` are consistent across machines, unlike FNA's fixed zero/true placeholders. `RoundtripTime` reads ENet's peer measurement.

LAN discovery uses UDP port 61190, protocol version 1, and a 150-ms search window. A finder sends broadcast and loopback queries, then deduplicates replies by connect port. The game session uses an OS-assigned ENet port; Emscripten reserves 61191 but cannot perform raw UDP browser discovery. `QualityOfService`'s network-backed construction reports discovery round-trip time; bandwidth remains unmeasured.

The protocol opcodes are `ClientHello=0x01`, `ServerWelcome=0x02`, gamer join/leave `0x03/0x04`, reserved host change `0x05`, state change `0x06`, and application data `0x10`. There are two ENet channels and at most 31 peers. Up to 64 application sends may wait for handshake completion; the oldest is evicted on the 65th. Disconnect, gamer removal, and migration purge affected queued sends. An internal dropped-packet counter supports tests and diagnostics but is not part of `NetworkSession`'s public API.

Discovery packets are version-checked. Property indices must be in $[0, 256)$, one-byte list counts reject values above 255, and completed peers that repeat a hello are disconnected. Host-only broadcasts from a non-host peer are also rejected. These checks assume that an untrusted client can reproduce the documented wire format.

51.3 Host migration

When `AllowHostMigration=false`, loss of the host ends the session. When it is true, all survivors choose the lowest remaining wire ID from their shared roster. No election packet is needed. The peer owning that ID promotes itself; the others rediscover its cached gamertag and perform a normal hello/welcome reconnect. Discovery is retried for three 150-ms windows.

Migration clears remote gamers and wire-ID state first. Consequently, reconnecting peers see leave and join events and receive new `NetworkGamer` objects; sockets and object identity are not preserved. If the new host cannot be discovered, the session ends.

`SimulatedLatency` and `SimulatedPacketLoss` affect received application data. They do not delay session-management packets or the host's relay hop. FNA stores these values without applying them; CNA uses them as transport-test controls.

51.4 Delivery guarantees

ENet can express three behaviors for XNA's five `SendDataOptions` values:

XNA option	ENet flag	Observed guarantee
<code>None</code>	<code>UNSEQUENCED</code>	best effort, unordered
<code>InOrder</code>	<code>none</code>	best effort, sequenced
<code>Reliable</code>	<code>RELIABLE</code>	reliable and ordered
<code>ReliableInOrder</code>	<code>RELIABLE</code>	reliable and ordered
<code>Chat</code>	<code>RELIABLE</code>	reliable and ordered

ENet's reliable channel is ordered, so CNA cannot reproduce "reliable but unordered" without adding another protocol. `Reliable` therefore receives the stronger ordered guarantee. `Chat` uses the same route by CNA policy.

51.5 Compatibility limits and asymmetries

Even in a `SystemLink` session, voice transport and invitations are absent. `EnableSendVoice()` and `SendPartyInvites()` do nothing; voice and private-slot properties retain false defaults. `NetworkMachine::RemoveFromSession()` throws `NotImplementedException`. `Invite` and `TrueSkill`/arbitration events are never raised. `JoinInvited()` creates a synthetic `PlayerMatch` lifecycle rather than joining a service-backed session.

Two inherited data-shape problems are visible to applications:

- `PacketWriter` writes `Color` as four bytes, while `PacketReader::ReadColor()` expects four floats. They are not inverses.
- `ReceiveData(PacketReader&, NetworkGamer*&)` fills and rewinds the reader but returns zero because its local length is never updated. Gate it with `IsDataAvailable`, or use the byte-vector overload for an accurate count.

```

1 PacketWriter writer;
2 writer.Write(playerTintColor); // four bytes
3 localGamer->SendData(writer, SendDataOptions::Reliable);
4
5 std::vector<SharpRuntime::bytecs> rgba(4);
6 NetworkGamer* sender = nullptr;
7 while (localGamer->getIsDataAvailableProperty()) {
8     int count = localGamer->ReceiveData(rgba, sender);
9     if (count == 4) {
10         Color tint(rgba[0], rgba[1], rgba[2], rgba[3]);
11     }
12 }
```

The mutable `NetworkSessionProperties` indexer also grows the collection on an out-of-range access because C++ cannot distinguish read and write through the translated mutable reference. Use a `const` reference for bounds-checked reads.

51.6 Asynchronous lifecycle constraint

All session `Begin*/End*` pairs share one static `NetworkSessionAction*`. Only one create, find, or join operation can be outstanding in the process, even across different sessions. A second `Begin*` throws until the matching `End*` deletes and clears the shared action. The operations are positioned to complete synchronously because their work occurs in the matching `End*` path.

This design once depended on `GamerServicesDispatcher::UpdateAsync()` returning false, which it does not after initialization. The resulting busy loop and its isolated regression harness are described in Section 49.7.

51.7 Lifetime hardening

Historical note. Defects found under integration and sanitizers The network module has corrected the following distinct failures:

- `Dispose()` could run twice, traverse gamer pointers freed by the first call, and trigger a sanitizer-detected use-after-free;
- one disposal left raw pointers visible through gamer collections and `Host`;

- session-created gamer objects leaked because the session did not own their destruction;
- asynchronous callbacks were stored but not invoked, including reentrant replacement;
- **End*** cleared the shared action without deleting it;
- local IDs derived from collection size collided after remove/add;
- pre-handshake application data was dropped, then a first queue fix failed to purge entries when peers left;
- a disposed **GamerCollection** enumerator's **MoveNext()** dereferenced a null collection even though **Current** already guarded the same state.

The current **Dispose()** is idempotent, clears all gamer collections and **Host**, and releases owned gamer objects. Focused tests cover double disposal, post-disposal exposure, action counts, callbacks, ID allocation, and queue eviction. **AddressSanitizer** was the observation point for the original double-dispose failure; ordinary unit success alone had not exposed it.

51.8 Evidence status

Codec, hostile-input, policy, ownership, simulated loss/latency, and compatibility behaviors have focused tests. An out-of-process, two-instance **ENet** harness establishes cross-process **SystemLink** delivery on the recorded host. It does not establish Internet matchmaking, voice, browser UDP discovery, or delivery on an untested platform. Host migration is runtime-observed through the local discovery/reconnect route; it preserves neither sockets nor gamer-object identity.

Chapter 52

Avatar: Faithful Absence and CNAEXT Rendering

CNA has two independent avatar systems. The XNA-compatible classes reproduce the observable behavior of a reference assembly whose Xbox Live content service is gone; that path is inert. The `SkinnedModelEXT` path is a CNA extension that loads content and renders GPU-skinned meshes. Calling the extension does not change the base API's state or semantics.

	Base Avatar API	SkinnedModelEXT (CNAEXT)
Purpose	Preserve observed XNA surface without service content	Load and render CNA assets
Skeleton	Fixed 71-transform contract; zero transforms	Content-defined (bundled assets: 19 bones)
Draw	Validates, then performs no rendering	<code>GraphicsDevice/SkinnedEffect</code> draw
Activation	Normal API surface	Explicit opt-in
Evidence	Source translation and behavioral tests	Content, runtime, renderer, and pixel tests

“Faithful” here is scoped to the translated behavior and tests. It does not mean that a C++ port is binary-identical to the managed assembly or that Xbox avatar services are available.

52.1 Base API contract

`AvatarRenderer` ignores both constructor arguments. Reading `State` sets and returns `AvatarRendererState::Unavailable`; no path reaches `Ready` or `Loading`. The `IAvatarAnimation` `draw` overload rejects null, reads its transforms and expression, and delegates. The transform-array overload requires exactly 71 entries and then does nothing. `BindPose` throws `InvalidOperationException` unless the unreachable `Ready` state is present.

`AvatarDescription` requires 1,021 bytes. CNA treats a correctly sized value as valid when byte zero is nonzero; the other bytes remain opaque. The description getter copies the bytes, height/body type default lazily, and the static `Changed` event is never raised. Both `CreateRandom` overloads return the same all-zero invalid description. The body-type overload validates its enum but does not use it when building the result.

`BeginGetFromGamer/EndGetFromGamer` complete synchronously and return that invalid description. The caller owns and must delete the heap-allocated `IAAsyncResult`. `AvatarAnimation` supplies 71 zero transforms, zero duration, and a default expression for all 31 presets. The preset is retained only as the CNAEXT clip name returned by `GetRealClipNameEXT()`.

These results are expected when the proprietary service-supplied mesh, texture, animation,

and description formats are unavailable. FNA has no corresponding Avatar classes, so CNA's source was translated from the XNA 4.0 reference assembly instead of inferred from FNA.

52.2 SkinnedModelEXT contract

SkinnedModelEXT is move-only because it owns GPU resources. It contains:

- parent indices, local bind poses, and global inverse bind poses;
- named clips with per-bone translation, scale, and quaternion-rotation tracks;
- skinned mesh parts and their vertex, index, material, and texture resources.

Its skeleton is unrelated to **AvatarRenderer**'s 71-entry validation array. The bundled male and female assets use 19 bones; other assets may use different counts up to **SkinnedEffect**'s 72-bone draw limit.

AvatarRenderer::EnableRealRenderingEXT(device, model) creates the extension's **SkinnedEffect**. **DrawRealEXT(clip, position, loop)** samples a clip and draws the model, or throws **InvalidOperationException** if opt-in has not occurred. It does not make **State** ready or give the base **Draw()** a renderer. Conventional roots are **avatar/male/avatar** and **avatar/female/avatar**.

```

1 AvatarDescription desc = AvatarDescription::CreateRandom();
2 AvatarRenderer renderer(&desc); // base state remains Unavailable
3
4 auto& device = getGraphicsDeviceProperty();
5 auto model = getContentProperty().Load<std::shared_ptr<
    SkinnedModelEXT>>("avatar/male/avatar");
6
7 renderer.EnableRealRenderingEXT(device, model);
8 renderer.SetAppearanceEXT(AvatarAppearanceEXT());
9
10 renderer.setWorldProperty(Matrix::getIdentityProperty());
11 renderer.setViewProperty(camera.View);
12 renderer.setProjectionProperty(camera.Projection);
13 renderer.DrawRealEXT("Stand0", elapsedClipTime, true);

```

The fragment is intended for a **Game** subclass with the Avatar, content, graphics, and **<memory>** headers already included.

52.3 Animation and palette computation

ComputeBoneTransformsEXT first initializes every local transform from the bind pose, then samples matching tracks. Translation and scale use linear interpolation; rotation uses **Quaternion::Slerp**. It composes children with already computed parents in stored topological order and finally multiplies each global transform by its global inverse bind pose. The result is the palette consumed by **SkinnedEffect**.

```

1 SkinnedModelEXT model;
2 model.BoneCount = 2;
3 model.ParentBoneIndices = {-1, 0};
4 model.BindPoseLocal = {
5     Matrix::getIdentityProperty(),
6     Matrix::CreateTranslation(Vector3(0, 1, 0))};
7 model.InverseBindPoseGlobal = {

```



```

8      Matrix::getIdentityProperty(), Matrix::getIdentityProperty());
9
10     BoneTrackEXT track;
11     track.BoneIndex = 0;
12     track.Keys.push_back(KeyframeEXT{
13         TimeSpan::FromSeconds(0), Vector3(0, 0, 0)});
14     track.Keys.push_back(KeyframeEXT{
15         TimeSpan::FromSeconds(1), Vector3(2, 0, 0)});
16
17     AnimationClipEXT clip;
18     clip.Duration = TimeSpan::FromSeconds(1);
19     clip.Tracks.push_back(track);
20     model.Clips["Move"] = clip;
21
22     std::vector<Matrix> palette;
23     model.ComputeBoneTransformsEXT(
24         "Move", TimeSpan::FromSeconds(0.5), false, palette);
25     // palette[0].Translation.X == 1.0f

```

Current guards address four earlier failures:

- looped time uses tick modulo instead of repeated subtraction, including negative time;
- each non-root parent must precede its child, which also rejects cycles;
- skeleton arrays must match `BoneCount` before indexing;
- empty or out-of-range animation tracks are skipped without touching memory.

The time regression uses a position 1,000,000,000.5 clip durations from the origin and a 100-ms bound, specifically detecting the former duration-by-duration loop.

52.4 Lighting, appearance, and wardrobe

The extension uses one directional light plus ambient light. It disables the other two stock directional lights and zeroes specular and emissive terms before applying the caller's values. Default ambient and key-light values keep an avatar visible without configuration.

`AvatarAppearanceEXT` is a CNA tint model, not a reconstruction of the proprietary appearance format. Case-sensitive part-name substrings select `Hair`, `Shirt`, `Pants`, or `Shoes`; unmatched names receive skin tint. It changes colors, not geometry or per-part textures.

`AttachPartEXT` requires a matching bone count. Reattaching a part with the same name first calls `RemovePartEXT`, which releases the old owned vertex/index buffers, mesh part, and texture before inserting the replacement. This closes both the former overlapping-part behavior and its GPU-resource leak.

Historical note. Lighting defects found through avatar rendering An early draw path applied ambient light before `EnableDefaultLighting()`, which then overwrote it. A separate EasyGL shader defect multiplied emissive color by diffuse color twice, turning the ambient term into $ambient \times diffuse^2$. That defect affected all related skinned, vertex-lit, and environment-mapped variants. A pixel test's near-black fraction fell from 4.1–6.0% to 0% after the shader formula was aligned with FNA's lighting expression.

52.5 Content pipeline

The `.skinnedmodel.json`, `.skeleton.bin`, and `.clip.bin` pipeline loads the extension assets. Three corrected implementation traps are relevant to other importers:

- manifest-relative paths must resolve from the manifest directory, not the content root;
- chained `reader.Read<float>()` constructor arguments relied on unspecified C++ argument evaluation order, so components are now read into named locals;
- hierarchy reordering must remap inverse bind matrices and per-vertex joint indices as well as parent indices.

Chapter 41 covers the corresponding glTF joint-space rules.

52.6 Open garment interpenetration

The procedural avatar still shows shirt and skin fragments around the raised shoulder and chest in the Wave clip. Measurement found the same 64 of 228 shirt vertices crossing the body in both neutral and Wave poses; the animation changes which crossings face the camera. This locates the remaining defect in the rest-pose geometry, not clip sampling.

The largest visible intersections are the collar and cuffs. The shirt's `Spine1` shell has radius 0.176 and spans $z = 1.40$ to 1.576 ; the head sphere has radius 0.15 and center $z = 1.60$, placing its bottom at $z = 1.45$. Their calculated overlap of about 0.126 m agrees with the measured 0.131 m collar penetration. A waist overlap is mutually hidden by garments.

Four attempted remedies did not remove the cause:

Attempt	Observed result
Narrow garment blend radius	No clear visual improvement
Trim capsule boundary caps	Pants crossings rose from 21 to 39
Flat cylinders plus joint spheres	More speckle, 28 new groin crossings, lower brightness
32 instead of 12 segments	Smoother edges, same fragments, $4.2\times$ vertex data

The open choices are to size collar and cuff shells against the body radii along their full span, accept the limitation of convex per-bone placeholder shells, or explicitly accept the tessellation cost. The extension is therefore renderer-functional but not visually defect-free.

52.7 Renderer and evidence status

EasyGL and Vulkan run registered avatar integration programs with pixel readback. Their synthetic one-bone quads isolate palette/effect/draw wiring; they do not validate the entire asset converter. Demo programs add full humanoid content, clip changes, appearance, attachment, wardrobe replacement, API-boundary comparison, and two-process network synchronization.

BGFX contains relevant pipeline code but has no registered avatar integration result at the pinned revision; existing harnesses stop earlier at unimplemented 3D depth/blend setup. `SDL_RENDERER` rejects the required 3D resources. No conclusion for either renderer should be inferred from EasyGL or Vulkan pixels. Appendix G defines the evidence terms used here.

Part VIII

sharp-runtime: The Foundation Layer

Chapter 53

Why sharp-runtime Exists

This book’s ecosystem chapter introduced **sharp-runtime** as the foundation layer every XNA-facing CNA type quietly depends on. This chapter treats it as a project in its own right — one considerably larger, and considerably more broadly scoped, than its role inside CNA alone might suggest.

53.1 Scale

At the audited revision, **sharp-runtime** contains 1,032 public headers plus five private implementation headers, 217 implementation files, and 78,934 lines of production C/C++. Its tests are larger still: 102,342 lines, roughly 30% more than production. The registered GoogleTest surface contains 16,310 tests across 37 executables; the repository’s continuation record reports 16,303 passing, one skipped, and six failing, rather than concealing those failures behind an obsolete all-green count. This is a considerably larger surface than “just enough .NET to make XNA’s API vocabulary work in C++” might suggest — coverage genuinely extends to `System.Text.Json`, `System.Xml.Linq / XPath`, `System Buffers.Text / Binary`, `System.Threading.Channels`, and `System.Net.WebSockets / Http / Security`, well beyond a minimal XNA-support subset.

53.2 Namespace structure

The public headers live below two include roots. `System/` mirrors real .NET’s namespace tree directly: `Collections` (with `Concurrent`, `Frozen`, `Generic`, `Immutable`, `ObjectModel`, `Specialized` sub-namespaces), `IO` (with `Compression`, `Hashing`, `IsolatedStorage`), `Net` (with `Http`, `Sockets`, `WebSockets`, `Security`), `Text` (with `Json`, `RegularExpressions`, `Encodings`), `Threading` (with `Channels`, `Tasks`), `Xml`, `Security`, `Globalization`, `Runtime`, and roughly 185 flat top-level types (`Object`, `String`, `Exception`, `EventHandler`, `IDisposable`, `TimeSpan`, `DateTimeOffset`, `Delegate`, `Guid`, `Random`, `Uri`, `Version`, `Nullable`, `Span`, `Memory`, and more). A separate `SharpRuntime/` namespace holds project-internal helpers that are not part of the .NET mirror at all — most importantly `SharpRuntimeHelper.hpp`, home to the type-alias table Chapter 3 already introduced (`bytecs`, `intcs`, `Single`, and the rest).

There is no umbrella header. The on-disk include spelling follows the fully-qualified .NET name mechanically: `System.IO.FileStream` becomes `System/IO/FileStream.hpp`. CNA consequently consumes far more than the two headers whose spelling begins with `SharpRuntime/`: its source directly includes 54 distinct headers from this repository, 52 below `System/` and two below `SharpRuntime/`. Those two are `SharpRuntimeHelper.hpp` and `Prop.hpp`; the latter exports property-declaration macros used downstream even though the runtime’s own modules do not use it. Chapter 54 maps these include paths to their owning link components.

53.3 Object and IDisposable: deliberately not two roots

`System::Object` is not a universal root for this C++ type system. It is an abstract, vestigial compatibility type with a pure-virtual `GetTypeName()`, and only `DateTime` and `DateTimeOffset` derive from it in the entire runtime. `Stream` does not; exceptions derive from `std::exception`. Its `GetHashCode()`, `Equals`, `ToString()`, and RTTI-backed `GetType()` are useful to the few types that opt in, while the `GetTypeNameHPP()` / `GetTypeNameCPP()` macros reduce their override boilerplate. Reading those helpers as evidence that every ported class inherits `Object` would import a .NET invariant the implementation intentionally does not have.

`System::IDisposable` is a single pure interface method, `virtual void Dispose() = 0`; its own documentation is explicit that C++ has no equivalent of C#'s `using` statement, so callers either use RAII or call `Dispose()` explicitly, and every implementation is expected to be idempotent. It is a capability interface, not a second hierarchy root.

53.3.1 String and Exception expose the same adaptation strategy

`System::String` is a non-instantiable collection of static source-porting helpers; both its constructor and destructor are deleted. Runtime strings remain `std::string`, also exported as `SharpRuntime::String`. Likewise, `System::Exception` derives from `std::exception`, carries an `exception_ptr` for its inner exception and a default HRESULT, and returns an empty stack trace because no managed stack is available. These are deliberate C++ representations of useful .NET behavior, not attempts to reproduce the CLR's object layout.

53.4 EventHandler and MulticastAction: solving a problem C# doesn't have

`System::EventHandler<TEventArgs>` deliberately diverges from real .NET's own design, and its own header comment states why directly: in C#, `EventHandler<T>` is only a delegate *type* — the actual multicast subscriber list and invocation mechanism are supplied separately, by the compiler-generated code behind the `event` keyword. C++ has no `event` keyword, so `sharp-runtime`'s own `EventHandler<T>` has to be both the type *and* the subscriber-list infrastructure at once: `operator+=`, a token-based `Add / Remove` pair, `Clear()`, and `Raise(sender, e)`. `Raise` specifically invokes over a *snapshot* of the current handler list, not the live list — a direct, deliberate guard against a handler that removes itself (or another handler) from within its own invocation, which previously surfaced as a real `std::bad_function_call` crash before this snapshot discipline was adopted. A further mechanism, `SetReplayHook()`, exists specifically to model a real XNA quirk this book has already touched on earlier — `NetworkSession.GamerJoined` (Chapter 51) is expected to replay already-happened state to a newly-added subscriber, not only notify about future events, and this hook is the general mechanism that makes that pattern implementable once, rather than reinvented per event.

`System::MulticastAction<Args...>` is a second, purpose-built multicast type built for the same underlying reason: since `std::function`/lambda values have no C#-style target-plus-method equality, ordinary value comparison cannot implement C#'s `-=`-style single-subscriber removal, so this type uses the same token-based-removal-plus-snapshot-invocation discipline as `EventHandler<T>` to solve the identical problem for plain multicast callback fields that are not shaped like a C#-style public event. The motivating real-world case, named directly in this type's own header comment, is re-parenting a scene-graph hierarchy: a node that resubscribes to a different set of ancestor callbacks needs to drop its *specific* old subscription

without clearing every other subscriber sharing the same field, which plain value comparison over `std::function` objects cannot distinguish.

```

1 // Worked example: token-based removal, the scenario this type
  // exists for
2 // -- unsubscribing one specific handler without disturbing any
  // other
3 // subscriber on the same field.
4 System::MulticastAction<float> onParentTransformChanged;
5 auto token = onParentTransformChanged.Add(
6     [this](float deltaScale) { ApplyParentScale(deltaScale); });
7
8 // ... later, when this node is re-parented onto a different
  // ancestor ...
9 onParentTransformChanged.Remove(token);           // only this
  // subscription is dropped
10 token = newParent->onTransformChanged.Add(        // resubscribe to
  // the new ancestor
11     [this](float deltaScale) { ApplyParentScale(deltaScale); });

```

53.5 Value types used by later chapters

`System::TimeSpan`, the type underlying `GameTime` and `SensorBase::TimeBetweenUpdates` throughout this book, stores its value internally as 100-nanosecond ticks and carries one small, diagnostic-only surprise: static copy and move counters exposed for test instrumentation. Both counters are now `std::atomic<int>`; loads, resets and increments use relaxed atomics because only each final count matters, not an ordering relationship with the copied value. A downstream sanitizer suppression that still describes their increments as a known `TimeSpan` race therefore documents an older implementation, not the pinned runtime.

`Collections::Generic::List<T>` and `Dictionary<TKey,TValue>` both implement .NET's fail-fast collection-modified-during-enumeration behavior via an internal version counter checked on every enumeration step — with one small, explicitly documented deviation on `Dictionary`: its `Remove()` bumps the version counter, which real .NET's own `Dictionary.Remove()` does not do, because `std::unordered_map`'s own iterator-invalidation guarantees on erase differ from the array-backed entry table real .NET's dictionary uses internally — a deliberate, reasoned adaptation to the underlying C++ standard library type, not an oversight. This is not just a two-type pattern: every mutable generic collection this library ships — `HashSet`, `LinkedList`, `Queue`, `Stack`, `SortedDictionary`, `SortedList`, `SortedSet`, and `OrderedDictionary` alongside the two above — carries the identical version-counter invariant, made project-wide as a deliberate late addition rather than left as a two-type special case. `OrderedDictionary` specifically needed a follow-up fix of its own even after that rollout: its `EnsureCapacity` method resized the backing storage without bumping the version counter, meaning an enumerator already in flight across a capacity-triggered resize would not detect the mutation the way every other structural change on the same type correctly does — closed as its own, narrower fix once found, on the same underlying invariant.

53.6 Concurrency primitives

`Threading::Tasks::Task` implements a real, if scoped, async-task model: `Wait()` rethrows a faulted task's exception (matching .NET's own `Wait()` / `ThrowIfExceptional` contract), and `ContinueWith` uses weak-pointer-based cycle avoidance specifically to avoid leaking a continuation chain rather than spawning an extra OS thread per continuation. A detail worth

stating plainly, since it governs how a caller should reason about ordering: this port has no thread pool or scheduler at all, so a continuation always runs synchronously and inline, either on whichever thread completes the antecedent task or, if the antecedent is already complete by the time `ContinueWith` is called, immediately on the calling thread — effectively as if `TaskContinuationOptions::ExecuteSynchronously` were always set, regardless of whether a caller actually passes it. Most of the accepted `TaskContinuationOptions` filter bits (`NotOnFaulted` / `NotOnCanceled` / `NotOnRanToCompletion` and their `OnlyOnX` compositions) are genuinely honored; the rest (`PreferFairness` / `LongRunning` / `AttachedToParent` / `DenyChildAttach` / `HideScheduler` / `LazyCancellation`) are accepted only for API-surface parity, since there is no scheduler or parent-task tracking for them to affect in the first place.

```

1  // Worked example: ContinueWith's continuation does NOT rethrow the
2  // antecedent's exception automatically the way Wait() does -- it
   must
3  // inspect the antecedent explicitly, and it runs synchronously,
   inline,
4  // with no extra thread spawned for it.
5  System::Threading::Tasks::Task loadTask = System::Threading::Tasks::
   Task::Run(
6      [] { LoadLevelFromDisk(); });
7
8  System::Threading::Tasks::Task continuation = loadTask.ContinueWith(
9      [] (System::Threading::Tasks::Task antecedent)
10     {
11         if (antecedent.getIsFaultedProperty())
12         {
13             LogLevelLoadFailure();
14             return; // the antecedent's exception is NOT rethrown
                       here automatically
15         }
16         FinishLevelSetup();
17     });

```

53.6.1 WhenAll cancellation and the external-future bridge

`Task::WhenAll` and `Task::WhenAny` round out the async surface, and `WhenAny` specifically was rebuilt to match real .NET's own zero-extra-thread design (`TaskFactory.CommonCWaitLogic`'s completion-registration strategy) rather than the naive one-watcher-thread-per-input-task approach it started as. The current implementation registers an inline callback on each input's shared state; the first callback wins an atomic compare-exchange and later callbacks become cheap no-ops. No losing watcher is joined or detached because the current path creates no watcher threads at all. `WhenAll` waits on every input task to completion (never short-circuiting on the first fault, matching real .NET) and, if one or more faulted, rethrows the first one encountered in input order when the returned task is waited on — a deliberate simplification versus real .NET's `AggregateException` wrapping, consistent with the same choice `Task::Wait` already makes for a single faulted task. If no input faults but one is canceled, the action implementing `WhenAll` throws `TaskCanceledException`; because that returned task has no associated cancellation token, its constructor records the escape as *Faulted*, not *Canceled*. Waiting still throws the expected exception type, but the status flags remain a documented deviation from .NET.

`WhenAll` must distinguish that genuine canceled-input state from a task that merely faulted with a directly thrown `TaskCanceledException`; it now checks the input task's status instead of sniffing the exception type. The same category of defect previously existed in the

external-future bridge used by `TaskCompletionSource`: both `TrySetCanceled()` and a caller's `SetException(TaskCanceledException)` settle a promise with that exception type. A separate producer-set atomic flag now identifies only the genuine cancellation path, preserving the caller's exception and `Faulted` state otherwise. `WhenAny` needs neither rule: its outer `TaskT<Task>` always completes successfully with the first terminal input, whose own fault/-cancellation status the caller inspects.

`Threading::Thread` is a thin wrapper over `std::thread` that deliberately does not start at construction — a second call to `Start()` throws `ThreadStateException`, matching real .NET's own one-shot-start contract exactly, rather than silently allowing a thread object to be restarted the way a naive wrapper might.

53.7 Permanent scope boundaries: what will never be ported, and why

`sharp-runtime`'s own `CLAUDE.md` states its scope discipline directly, and it is worth quoting rather than paraphrasing, since the wording distinguishes a real category this book's own methodology depends on: "Known permanent deviations (not bugs, not TODO)." Five areas are named explicitly, each with a stated reason rather than left as an unexplained gap — the delegate-tier deviation is the one already covered in full in Chapter 57 (the three-tier `Action / Delegate / MulticastAction` split); the remaining four are covered here:

Reflection (`System::Type`, `System::Activator`, `Enum.GetNames / GetValues`) — "completely out of scope. Stubs are the correct end state," not a gap awaiting engineering effort.

GC (`System::GC`) — mutating operations are callable no-ops and queries return documented sentinel values (for example zero, false, `NotApplicable`, or the constant three-generation maximum); memory is managed by RAII / `std::shared_ptr` throughout, so there is no garbage collector for these calls to meaningfully forward to.

Serialization (`[Serializable]`, `SerializationInfo`) — ignored outright; not needed for game code.

P/Invoke and interop — out of scope entirely.

Cryptography (symmetric/asymmetric ciphers, X.509 certificates, TLS — `Aes*` / `RSA*` / `EC*` / `ChaCha20Poly1305` / `CryptoStream`, `X509Certificates`, `SslStream`) — out of scope by an explicit, dated decision (2026-07-07): correctly implementing this needs either a large new external dependency (OpenSSL/mbedTLS) or a hand-rolled, security-critical implementation, and neither is worth it for game code. This deviation is deliberately narrow, not a blanket "no crypto" rule: digest, authentication, and derivation APIs (`MD5` / `SHA*` / `HMAC` / `PBKDF2`) remain in scope and are already ported. HMAC keys and PBKDF2 passwords are sensitive material even though these APIs do not provide reversible confidentiality, X.509 validation, or a TLS channel.

53.7.1 Reflection's stub and its deliberate contradiction

`System::Type`'s own header comment is worth reading in full for what it says about *how* a permanent stub should behave, not just that it exists: constructed via `Type::From<T>()`, backed by C++ RTTI's `std::type_info` rather than any real reflection metadata, its boolean predicates (`IsClass`, `IsValueType`, `IsAbstract`, `IsSealed`, `IsInterface`) each return a fixed value regardless of what the actual type argument is — because C++ RTTI cannot determine any of them — and the comment states plainly that these fixed values are *not* mutually

consistent: `Type::From<int>()` reports both `IsClass()==true` and `IsValueType()==false` simultaneously, a combination that would be self-contradictory for a real .NET type (`int` is `IsValueType()==true`, `IsClass()==false`). This is not an oversight left for a future fix — the header says explicitly that these predicates exist only so ported code calling `type.IsClass` still *compiles*, not to answer the question correctly, and that calling code must not branch on them to distinguish value types from class types. `System::Activator` narrows the same permanent gap to exactly the part that survives the loss of reflection: its real, callable `CreateInstance<T>()` template constructs any default-constructible `T` at compile time, while the reflection-based overloads real .NET exposes — `CreateInstance(Type)`, `CreateInstanceFrom` — are absent entirely, not stubbed, because `sharp-runtime` has no `System.Reflection` for them to resolve a `Type` argument against in the first place.

53.8 The build is part of the public architecture

Since the 2026-08-10 modularization merge, `sharp-runtime` is not one static archive. Its 41 module directories register 44 link components: 30 static libraries, 13 interface components, and the `Xml.XPath` alias of `Xml`. Compatibility umbrellas named `Core` and `Collections` coexist with the narrower components. The distinction matters to CNA because an include path names a source-level dependency while a `SharpRuntime::<Component>` target declares its link closure. The complete registry, component selection rules, and CNA's old-checkout adapter are covered in Chapter 54.

One platform-specific edge remains useful context here. On Android, the storage-path implementation calls SDL3 path APIs, so a parent such as CNA must have created an `SDL3::SDL3` target before adding the runtime. This conditional edge does not turn SDL into a universal runtime dependency.

53.9 Local and tracked CI gates have different scope

The local gate configures and builds under strict shell failure handling and rejects compiler warnings and errors. The modular test topology is finer-grained than the old single-binary design: 36 component executables plus one integration executable feed GoogleTest discovery. Each component test links its own component and declared test-only dependencies, so an undeclared cross-module use can fail at link time instead of remaining hidden inside a monolith. Python meta-tests and deliberately failing consumer fixtures add build-boundary checks that ordinary positive unit tests cannot express.

The pinned repository also tracks `.github/workflows/components.yml`. On every push and pull request it runs nine selective Ubuntu configurations, a full compatibility build, and a separate Doxygen-warning job, with the build-job ceiling fixed at two. That proves a hosted trigger and named Linux jobs exist; it does not prove Windows, macOS, Emscripten, or every historical commit ran successfully. Older continuation prose saying no workflow existed was superseded by the workflow now present in the pinned tree.

53.9.1 Recorded failures and platform claims need scope

The current continuation note records six failing and one skipped test; an older baseline that claims every test passes is therefore historical, not authoritative. The same discipline applies to platform reports: a past MinGW or Emscripten library build does not prove that all 37 current executables cross-compile, much less that they execute on their target. Native Linux results establish neither a cross compiler's header and linker path nor runtime behavior on a

different operating system. Keeping those scopes separate is part of the evidence model, not mere cautionary wording.

Chapter 54

Components and CNA Integration

The most consequential sharp-runtime change near CNA’s audited revision did not alter a `System::` method. On 2026-08-10, the runtime merged a modular remediation snapshot that replaced the one-archive build shape with named components. The merge touched 3,828 files and landed one day before the CNA revision examined for this book. A description based on the earlier monolith can therefore be internally coherent and still be wrong for the actual sibling pair.

This chapter follows one question through both repositories: when a CNA source includes a runtime header, what target supplies it, and how does that target reach the final program?

54.1 Directories, components, and targets are different counts

The runtime has 41 module directories, but those directories register 44 components. The registry produces the shape in Table 54.1.

Table 54.1: The audited sharp-runtime component shape.

Kind	Count	Meaning
STATIC	30	Components with compiled implementation objects.
INTERFACE	13	Header-only surfaces or compatibility umbrellas carrying usage requirements.
ALIAS	1	<code>Xml.XPath</code> aliases <code>Xml</code> , whose implementation boundary is shared.
Total	44	Registered components across 41 physical module directories.

Two extra names explain why a directory count cannot predict the component count. `Core` and `Collections` are compatibility umbrellas over narrower components, and `Xml.XPath` is the alias. The remaining names cover families such as `Core.Base`, `Collections.Core`, `IO`, `Text.Json`, `Threading.Tasks`, and `Net.WebSockets`. Consumers link the namespaced form, for example `SharpRuntime::Core.Base`, rather than guessing an archive filename.

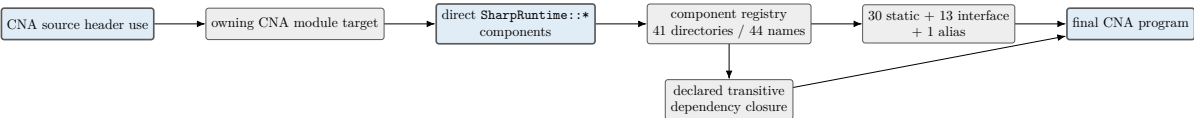


Figure 54.1: How CNA consumes modular sharp-runtime. Component availability and a target’s declared link interface are different questions.

The module CMake files do not each call `add_library` directly. They register their kind, dependencies, include directory, and optional test dependencies with a common registry. The registry later materializes targets in a consistent order. Production sources and module tests are globbed within the registered directory, while a source-partition validator and an empty dependency allowlist make accidental ownership leaks visible. The generated component catalogue measures 92 direct production dependency edges at the pin; this edge count is distinct from both the 41 physical modules and 44 selectable/compatibility names.

54.2 Header names predict location, not the link closure

There is no `SharpRuntime.hpp`. If ported code needs `System::IO::MemoryStream`, it includes `System/IO/MemoryStream.hpp`; the IO component owns that public include directory. The component can in turn expose public dependencies needed by the header. Consequently, a consumer names its direct component and lets CMake propagate the remainder.

CNA directly includes 54 distinct sharp-runtime headers: 52 below `System/` and two below `SharpRuntime/`. The latter count must not be misread as the total dependency. `SharpRuntime/SharpRuntimeHelper.hpp` supplies primitive aliases, while `SharpRuntime/Prop.hpp` supplies downstream property macros. Both belong to `Core.Base`; headers such as `System/IO/BinaryWriter.hpp` and `System/Security/Cryptography/SHA256.hpp` introduce other component needs.

54.3 Standalone selection and embedded selection differ

When sharp-runtime is the top-level CMake project, it exposes `SHARP_RUNTIME_COMPONENTS`. A developer may select a subset, or leave the default `All`. The compatibility `SHARP_RUNTIME` interface target exists only for that complete selection; a deliberately partial build should use namespaced component targets.

That cache option is created only when the runtime is the top-level project. CNA adds the runtime with `add_subdirectory`, so the option is not created and selection falls through to `All`. Every component is therefore available in a normal CNA build. CNA's narrow component lists reduce each target's *link interface*; at this revision they do not prevent unrelated runtime components from being configured and compiled.

This distinction prevents two opposite mistakes:

- a narrow CNA declaration is meaningful because it documents and enforces the target's link boundary;
- it is not yet a build-time pruning mechanism for the embedded sibling.

54.4 CNA's sibling contract

Sharp-runtime is CNA's one unconditional sibling dependency. CNA first forces `SHARP_RUNTIME_BUILD_TESTS` off; otherwise embedding it would add 37 runtime test executables to every CNA build. It then resolves `CNA_SHARP_RUNTIME_ROOT`, whose default is the adjacent `../sharp-runtime` checkout. A missing `CMakeLists.txt` is a configure-time fatal error with a clone remedy. There is no submodule, vendored copy, `FetchContent` fallback, or reduced internal substitute.

After `add_subdirectory`, CNA loads its 59-line `SharpRuntimeConsumption.cmake` adapter. The adapter recognizes two historical build shapes:

Modular `SharpRuntime::Core.Base` exists, so requested component names are translated to namespaced targets. This is the authoritative shape for the audited siblings.

Monolithic That target does not exist, so CNA links the legacy `SHARP_RUNTIME` archive. This is a compatibility path for older checkouts, not the architecture of current sharp-runtime.

The detection intentionally tests a target rather than a version string. That makes the seam describe the capability CNA needs and avoids teaching CNA the runtime's release-numbering history.

54.5 The linking helper makes closures explicit

Every CNA module calls `cna_link_sharp_runtime(target visibility [Components...])`. In a modular checkout, the helper prefixes each supplied name with `SharpRuntime::`; in a legacy checkout it falls back to the single archive. Omitting the optional names uses CNA's default closure:

```
1 Core.Base
2 IO
3 Collections.Core
4 Collections.ObjectModel
5 Runtime
6 Threading
7 Text
8 Globalization
9 Storage
```

That list is not every transitive component. Dependencies such as `Uri`, `TimeZone`, `ComponentModel`, and `Buffers` arrive through the public edges of the named targets. Repeating the transitive closure in CNA would duplicate the runtime registry and make upgrades more fragile.

Eighteen CNA module call sites declare their own closure. Small modules can be genuinely narrow: `math` and `input` request only `Core.Base`. Graphics requests `Core.Base`, `IO`, `Collections.Core`, and `Text`. Networking adds `Runtime` and `Threading`; gamer services additionally needs `Globalization`. The renderer aggregation target uses the default closure because it spans more of the framework.

```
1 # Shape-independent CNA module declaration
2 cna_link_sharp_runtime(CNA_Input PUBLIC Core.Base)
3
4 cna_link_sharp_runtime(CNA_Net PUBLIC
5     Core.Base IO Collections.Core Runtime Threading)
```

The helper is therefore more than syntactic convenience. It lets present-day modules state precise boundaries while retaining a controlled path to old sibling checkouts.

54.6 Tests enforce component ownership

The modular test layout is part of the boundary design. Each component with tests gets a `SharpRuntimeTests_<component>` executable linked only to its component, `gtest_main`, and explicitly declared test-only dependencies. GoogleTest discovery adds the component name as a prefix. A test that accidentally reaches into an undeclared component can consequently fail to link; the old monolithic test executable could have masked the same dependency.

Thirty-six components currently contribute such an executable, and a separate integration executable brings the total to 37. Five module directories have no component tests: `io-compression-zip`, `io-isolated-storage`, `security-cryptography-random`, `storage`, and `text-regular-expressions`. The last is a 12-header interface component with no implementation and no tests, a concrete coverage gap rather than an implied success.

Outside GoogleTest, Python meta-tests validate boundaries and consumer fixtures exercise both positive and intentionally failing configurations. A negative fixture is successful when a forbidden dependency fails to compile or link; treating every non-zero compiler exit as a CI failure would invert the purpose of that test.

54.7 Upgrade rules for the seam

The safe upgrade procedure follows directly from the architecture:

1. pin and identify both CNA and sharp-runtime revisions;
2. configure through `CNA_SHARP_RUNTIME_ROOT` rather than moving sibling directories to satisfy a hidden assumption;
3. confirm the shape-detection message and inspect unexpected fallbacks to the monolith;
4. keep each CNA module's direct component list narrow and let public dependencies close transitively;
5. run runtime boundary fixtures when the registry changes, then run CNA's normal build and tests with runtime tests still disabled in the embedded build;
6. distinguish "all components were built" from "this CNA target links only its declared closure" in any verification report.

The final rule is the most important. Modularity has two observable layers here: what the parent configures and what an individual target consumes. At the audited revision CNA gets all 44 components from its sibling, but its module targets no longer pretend that they need all 44. That narrower declaration is the durable integration contract on which later build-time pruning can be based.

Chapter 55

The Object Model, Honestly

Porting C# names into C++ does not import the CLR with them. Sharp-runtime retains the APIs that help source translation, but its actual object model is standard C++: values, explicit interfaces, RAII, smart pointers where reference identity matters, and RTTI for the small amount of runtime type identity that survives. This chapter makes those seams explicit so that familiar names such as `Object`, `String`, and `Delegate` do not invite managed-runtime assumptions.

55.1 Object is opt-in, not universal

Real .NET gives every value an `object` ancestry. C++ has no equivalent root, and sharp-runtime does not simulate one. `System::Object` is abstract because `GetTypeName()` is pure virtual. At the audited revision, exactly two runtime classes derive from it: `DateTime` and `DateTimeOffset`. Streams, collections, exceptions, strings, and almost every other public type do not.

For the few opt-in classes, the surface is useful and concrete:

- `Equals(const Object*)` defaults to pointer identity;
- static `Equals` handles nulls and then dispatches to the first object;
- `ReferenceEquals` is strictly pointer equality;
- `GetHashCode()` derives a non-negative value from the object's address;
- `ToString()` returns the stable name supplied by `GetTypeName()`; and
- non-virtual `GetType()` uses `typeid(*this)`, so a base pointer still reports its most-derived RTTI identity.

The `GetTypeNameHPP()` and `GetTypeNameCPP()` macros only remove override boilerplate. They are not a hidden registration system, and their existence does not make `Object` universal.

```
1 // DateTime participates in the optional Object surface.
2 System::DateTime now = System::DateTime::Now();
3 const System::Object* object = &now;
4 System::Type dynamicType = object->GetType();
5
6 // Stream does not: this is ordinary C++ inheritance, not a CLR tree
7
8 std::shared_ptr<System::IO::Stream> bytes =
9     std::make_shared<System::IO::MemoryStream>();
10 // System::Object* impossible = bytes.get(); // unrelated types
```

55.2 String is an API vocabulary, not storage

`System::String` has a deleted constructor and destructor. It is a static utility class whose methods accept and return `std::string`; the actual string value type is `std::string`, also exposed as `SharpRuntime::String`. This keeps familiar method names such as `IsNullOrEmpty`, `StartsWith`, `Split`, and `Trim` available without wrapping every character buffer in a new heap-aware object.

The adaptation changes what null means. A `std::string` value is never a null object, so `String::IsNullOrEmpty` can only test emptiness. Code whose behavior distinguishes null from "" must retain that state separately, for example with `std::optional<std::string>`.

```
1  std::string asset = "Content/Textures/player.png";
2  bool isTexture = System::String::StartsWith(asset, "Content/Textures
   /");
3
4  std::optional<std::string> absent;
5  // The optional owns nullability; System::String operates on the
   value.
6  bool missing = !absent || System::String::IsNullOrEmpty(*absent);
```

`System::Text::StringBuilder` is a separate, real mutable object backed by one `std::string`. Its chunk enumerator preserves .NET's API shape but yields one chunk, not the linked character chunks used by the CLR implementation. Chapter 56 follows these type families and their include paths in more detail.

55.3 Exceptions join the C++ hierarchy

`System::Exception` derives directly from `std::exception`, not from `Object`. This makes sharp-runtime exceptions catchable by ordinary C++ infrastructure while preserving a .NET-shaped message, inner exception, data map, source, help link, and HRESULT. The base HRESULT is 0x80131500; `what()` exposes the stored message.

Some managed facilities have no honest representation. `StackTrace` returns an empty string because the runtime captures no managed stack. `GetBaseException()` is absent because returning a base value would slice a derived C++ exception without a virtual clone protocol. Reflection-dependent `TargetSite` and serialization-dependent `GetObjectData` are absent for the same reason.

```
1  try {
2      throw System::ArgumentOutOfRangeException("index");
3  } catch (const System::Exception& ex) {
4      Log(ex.getMessageProperty());
5  } catch (const std::exception& ex) {
6      // Non-sharp-runtime C++ failures remain observable too.
7      Log(ex.what());
8  }
```

There is no central `ThrowHelper`; concrete methods construct and throw the appropriate type directly. API parity therefore includes the exception class and message at each site, not merely the fact that some exception escaped.

55.4 Lifetime is RAII first

`System::GC` cannot collect anything. Its mutating operations are no-ops and its queries return documented constants or sentinel values rather than live heap statistics. Value types use value semantics; ownership-bearing types use destructors and standard smart pointers. `std::shared_ptr` appears where managed reference identity genuinely matters, while `std::weak_ptr` represents exclusive ownership. There is no project-specific smart pointer or tracing heap.

`System::IDisposable` remains a one-method capability interface. Implementations are expected to release deterministically, avoid throwing in normal disposal, and tolerate repeated calls. It does not replace destructors or establish another object root. A caller either uses RAII or invokes `Dispose()` explicitly when the API's semantic close point matters.

That split becomes important when disposal can report an error. A C++ destructor is implicitly `noexcept`; allowing an I/O exception to escape during stack unwinding can terminate the process. Sharp-runtime audit work found and fixed several such destructor paths. An explicit `Dispose()` or `Close()` is therefore the place to observe failures; the destructor remains a best-effort safety net.

55.5 Delegates have three representations

Most delegate-shaped declarations are simple `std::function` aliases. They model one callable target and deliberately have no C# multicast equality or late binding. Two concrete runtime facilities cover cases where that simplification is insufficient.

`System::Delegate` is a real multicast base. It stores either one erased `void()` callable or an invocation list of shared delegates, and implements `Combine`, `Remove`, `RemoveAll`, ordered `Invoke`, and `GetInvocationList`. Equality of entries is pointer identity rather than a CLR method-target pair. A single-target object must already be owned by `shared_ptr` before `GetInvocationList()` can return itself; otherwise `shared_from_this` throws `std::bad_weak_ptr`. `DynamicInvoke` always throws because C++ has no reflection-driven object[] invocation.

`System::MulticastAction<Args...>` is the event-field alternative. It uses subscriber tokens and snapshot invocation, allowing one subscription to be removed safely even when a callback mutates the list during dispatch. `EventHandler<T>` adds sender/event-argument shape and an optional replay hook. These specialized types solve observable event semantics; they do not make every `std::function` multicast.

```

1  auto first = std::make_shared<System::Delegate>([] { Log("first");
    });
2  auto second = std::make_shared<System::Delegate>([] { Log("second");
    });
3
4  auto both = System::Delegate::Combine(first, second);
5  both->Invoke();                               // first, then second
6  auto onlyFirst = System::Delegate::Remove(both, second);

```

55.6 Type is identity without reflection

`System::Type::From<T>()` and RTTI-backed equality provide genuine type identity. That is sufficient for dictionary keys, service registries, and equality tests. It is not enough to

reconstruct CLR metadata. Predicates such as `IsClass`, `IsValueType`, `IsAbstract`, and `IsInterface` return fixed placeholder values and are explicitly not mutually consistent. Ported code may compile against them but must not branch on them.

`System::Activator` follows the same boundary. Its template `CreateInstance<T>()` can default-construct a type known at compile time; overloads that accept a runtime `Type`, assembly name, or constructor argument array are absent. The honest object model is therefore smaller than .NET's, but every surviving mechanism has a clear C++ owner: RTTI for identity, templates for construction, smart pointers for shared lifetime, and explicit interfaces for capabilities.

Chapter 56

Namespaces, Types, and Include Paths

Sharp-runtime makes a source-porting promise visible in the filesystem: if a developer knows the .NET fully-qualified type name, the public include spelling should be predictable. The module graph adds a second question that a header alone cannot answer: which component owns that include and which dependencies follow transitively? This chapter establishes both rules, then applies them to representative text, stream, encoding, and task types.

56.1 Two public roots, no umbrella

Exactly 1,032 public headers are reachable below two roots; five further .hpp files are private implementation headers below module `src/` trees. `System/` contains the literal .NET namespace tree; `SharpRuntime/` contains project-specific aliases and source-porting helpers. There is no catch-all header. Private headers below a module's `src/` directory are not added to consumer include paths.

The transliteration rule is mechanical:

```
1  System.IO.FileStream
2      -> #include "System/IO/FileStream.hpp"
3
4  System.Threading.Tasks.Task
5      -> #include "System/Threading/Tasks/Task.hpp"
6
7  SharpRuntime::intcs
8      -> #include "SharpRuntime/SharpRuntimeHelper.hpp"
```

One public type per like-named header is the norm. The rule makes a port review tractable: an include that cannot be derived from the type name deserves inspection, and an accidental include from another module's private tree cannot be hidden behind a universal umbrella.

56.2 Namespaces do not equal components one-for-one

The C++ namespace hierarchy describes API identity; the 44 CMake components describe build ownership. Several namespace families split across components to keep link surfaces narrow. Table 56.1 gives representative mappings used by CNA.

The direct component is not the full closure. Its public dependencies carry lower-level requirements, just as a C++ header's own includes do. This is why CNA names `SharpRuntime::IO` rather than copying IO's current dependency list into its own CMake files. Chapter 54 covers the adapter and its legacy monolith fallback.

Table 56.1: Representative namespace/include/component mappings.

API family	Include example	Direct component
System core	System/TimeSpan.hpp	Core.Base
Collections	System/Collections/ Generic/List.hpp	Collections.Core
Object model	System/Collections/ ObjectModel/Collection.hpp	Collections. ObjectModel
Streams	System/IO/MemoryStream.hpp	IO
Text	System/Text/UTF8Encoding.hpp	Text
Tasks	System/Threading/Tasks/Task.hpp	Threading.Tasks
Cryptographic hashes	System/Security/ Cryptography/SHA256.hpp	Security. Cryptography

56.3 CNA’s dependency subset

CNA directly names 54 distinct runtime headers. Fifty-two use the `System/` root. The two `SharpRuntime/` spellings are `SharpRuntimeHelper.hpp`, which defines the C#-fidelity primitives and limits, and `Prop.hpp`, which exports property-declaration macros for downstream code. Counting only the latter prefix and concluding that CNA consumes two runtime headers mistakes a namespace prefix for a repository boundary.

`SharpRuntimeHelper.hpp` is unusually load-bearing: hundreds of public runtime headers include it, and CNA uses aliases such as `intcs`, `bytecs`, and `Single` throughout its public surface. It also exports two unnamespaced macros, `CONTAINS` and `INTERNAL`; consumers must treat those as global preprocessor names, not members protected by the `SharpRuntime` namespace.

`Prop.hpp` is the opposite kind of seam. The runtime’s own property methods are hand-written, and its modules do not include this macro header. CNA does. The file is therefore an exported downstream facility even though a search limited to the library’s implementation would misleadingly classify it as unused.

56.4 String and StringBuilder: static utility versus mutable buffer

`System::String` is deliberately *not* a wrapper class at all — its default constructor and destructor are both explicitly deleted, making it a pure static-method utility operating on ordinary `std::string` values (`IsNullOrEmpty`, `Compare`, the `IndexOf` / `LastIndexOf` family, `Split`, `Substring`, `Trim*`, `PadLeft` / `PadRight`). `Text::StringBuilder`, by contrast, is a real, stateful, mutable buffer type — the many-overloaded `Append` (string, char, int, double, float, bool, and the `longcs` alias from Chapter 3), `AppendLine`, `Insert`, `Remove`, `Replace`, and a nested `ChunkEnumerator` class mirroring real .NET’s own chunked-buffer enumeration shape — real .NET’s own `StringBuilder` is internally a linked list of separate character chunks, and `ChunkEnumerator` walks that list without copying; this port’s `StringBuilder` is backed by a single `std::string` instead, so its own `ChunkEnumerator` always yields exactly one chunk holding the entire current buffer — the enumeration *shape* is preserved faithfully even though the simpler single-buffer implementation underneath never actually produces more than one chunk to walk.

```

1 // Worked example: System::String is a pure static utility -- there
   // is no
2 // instance to construct, only free functions operating on std::
   // string.
```

```

3  std::vector<std::string> parts = System::String::Split(
4      "Content/Textures/hero.png", '/');
5  std::string trimmed = System::String::Trim(parts.back());
6
7  // Text::StringBuilder, by contrast, is a real mutable buffer.
8  System::Text::StringBuilder sb;
9  sb.Append("Score: ").Append(playerScore).AppendLine();
10 sb.Append("Lives: ").Append(livesRemaining);
11
12 for (const std::string& chunk : sb.GetChunks())
13 {
14     // Exactly one iteration, always -- see the ChunkEnumerator note
       above.
15     RenderHudText(chunk);
16 }

```

56.5 Stream: a lightweight subset, honest about it

`IO::Stream`'s own class comment states its scope plainly: “lightweight subset of the .NET Stream API.” `Read(buffer, offset, count)`, `Close()`, and `getLengthProperty()` are pure virtual; `Write / WriteByte` default to throwing `NotSupportedException` unless a writable stream subclass overrides them — the same shape real .NET's own read-only stream implementations follow, rather than forcing every stream subclass to implement a full read/write/seek contract regardless of whether it actually supports all three. `getCanWriteProperty()` defaults to false and `getCanReadProperty()` defaults to true, matching a read-only stream's honest self-report without requiring every subclass to override both just to state the obvious.

```

1  // Worked example: the minimal real subclass this base class
   actually
2  // requires -- only three overrides, matching a genuinely read-only
3  // stream's real capabilities (Write/WriteByte are correctly
   inherited as
4  // NotSupportedException-throwing, never called by a well-behaved
   reader).
5  class MemoryBlobStream final : public System::IO::Stream
6  {
7  public:
8      explicit MemoryBlobStream(std::vector<SharpRuntime::bytecs> data
9          )
       : data_(std::move(data)) {}
10
11     SharpRuntime::intcs Read(SharpRuntime::bytecs buffer[],
12                             SharpRuntime::intcs offset,
13                             SharpRuntime::intcs count) override
14     {
15         if (buffer == nullptr && count != 0)
16             throw System::ArgumentNullException("buffer");
17         if (offset < 0)
18             throw System::ArgumentOutOfRangeException("offset");
19         if (count < 0)
20             throw System::ArgumentOutOfRangeException("count");
21
22         SharpRuntime::intcs n = std::min<SharpRuntime::intcs>(
23             count, static_cast<SharpRuntime::intcs>(data_.size()) -
                position_);
24         if (n != 0)

```

```

24         std::memcpy(buffer + offset, data_.data() + position_, n
25             );
26         position_ += n;
27         return n;
28     }
29     void Close() override {}
30
31     [[nodiscard]] SharpRuntime::intcs getLengthProperty() const
32     override
33     {
34         return static_cast<SharpRuntime::intcs>(data_.size());
35     }
36 private:
37     std::vector<SharpRuntime::bytecs> data_;
38     SharpRuntime::intcs position_ = 0;
39 };

```

56.5.1 MemoryStream: writable by default, read-only when requested

`System::IO::MemoryStream` is the concrete `Stream` subclass this ecosystem actually uses in practice — CNA’s own `ContentManager` reaches for it to wrap data loaded from sources like Android APK assets, where a real filesystem path is not available at all. Its default constructor creates an empty, writable buffer, matching real .NET. The buffer-copy constructor now makes the same default explicit while retaining a C++-only way to request a read-only copy:

```

1  MemoryStream(const bytecs* buffer, intcs size, bool writable = true)
    ;

```

The implementation validates before forming the vector range. A negative size is reported as `ArgumentOutOfRangeException`; a null pointer with non-zero size is reported as `ArgumentNullException`. The one deliberate source-language adaptation is `nullptr`, 0, accepted as an empty range because C++ can express an empty pointer range whereas .NET’s array overload simply rejects a null array. Passing `false` creates a readable, seekable, non-writable copy whose `Write` and `WriteByte` calls throw `NotSupportedException`; omitting it matches the writable .NET array constructor.

56.5.2 Closing a memory stream closes operations, not the bytes it already owns

The constructor policy above should not be confused with this class’s close/dispose behavior. `MemoryStream::Close()` changes only its `isOpen_` state; it deliberately retains both the vector of bytes and the current position. After close, `Read`, `Write`, `WriteByte`, `Seek`, `Length`, and `Position` correctly reject use with `ObjectDisposedException`, while `CanRead` and `CanSeek` return false. A closed stream created as writable deliberately retains `CanWrite == true`: .NET’s own property reports the construction-time writable flag even though subsequent writes throw. In further deliberate contrast, `ToArray()` and `GetBuffer()` remain usable:

```

1  System::IO::MemoryStream stream;
2  const SharpRuntime::bytecs payload[] = {1, 2, 3, 4};
3  stream.Write(payload, 0, 4);
4  stream.Close();
5

```



```

6  std::vector<SharpRuntime::bytecs> saved = stream.ToArray(); //
    still {1,2,3,4}
7  // stream.GetLengthProperty(); // throws ObjectDisposedException
8  // stream.Read(buffer, 0, 1); // throws ObjectDisposedException

```

That asymmetry is intentional, not a use-after-dispose loophole. Real .NET explicitly preserves the buffer in its own dispose implementation, `MemoryStream::Dispose(bool)`, so post-disposal `TryGetBuffer`, `GetBuffer`, and `ToArray` can still retrieve data; the runtime's `ensureNotClosed()` guard follows the same rule by guarding operations but not the two extraction methods. CNA callers can therefore finish producing an in-memory content blob, close the stream to catch accidental later I/O, and still hand a stable copy to a consumer. The distinction between the methods matters: `ToArray()` returns an independent vector, safe across later stream writes, whereas `GetBuffer()` is a reference to the live vector and may be invalidated by a reallocation.

The same implementation also has two less obvious safety properties whose tests prevent a “memory only” stream from becoming an unchecked byte container. Null buffers produce `ArgumentNullException`; negative offsets or counts produce `ArgumentOutOfRangeException`. Neither case is mistaken for an EOF return of zero. And a caller may legally set `Position` past the current end, but a later write computes `position + count` in `int64_t` before resizing. If that sum exceeds the runtime's signed `intcs` range it throws `IO::IOException` (“Stream was too long.”) instead of wrapping negative and writing beyond the vector. This was not defensive theorizing: the former narrow-integer calculation had the same signed-overflow shape already reproduced under UBSan in a related slicing path, and `StreamTests.cpp` now exercises `Position = 2147483647` followed by a ten-byte write specifically to require the exception rather than an allocation or memory corruption.

56.6 UTF8Encoding: conformance validation and fallback evidence

`Text::UTF8Encoding`'s own class comment states a real, specific guarantee that is easy to assume any UTF-8 encoder provides and that this port genuinely earns rather than skips: `GetBytes()` / `GetString()` *validate* that input is well-formed UTF-8 — not merely pass bytes through as opaque `char` values — and route any ill-formed byte at each invalid position through the configured fallback before resuming. Reading `UTF8Encoding.cpp`'s own local `wellFormedUtf8Length()` helper directly shows this is real conformance checking, not a length-only byte count: a leading byte is walked against the real UTF-8 grammar (1/2/3/4-byte forms, each continuation byte checked for the `10xxxxxx` pattern), and even a *structurally* well-formed sequence is rejected if the code point it decodes to is an overlong encoding (a multi-byte sequence encoding a code point that a shorter form could already represent — a real, historically security-relevant UTF-8 attack shape, since a naive decoder that accepts overlong encodings lets an attacker smuggle a byte value like `0x2F` (`'/'`) past a filter checking only the raw byte stream) or a UTF-16 surrogate code point (`U+D800–U+DFFF`, which UTF-8 must never directly encode, since surrogates exist only as a UTF-16 encoding artifact).

```

1  // From wellFormedUtf8Length() -- the 3-byte case, real conformance
    logic,
2  // not just "are there two continuation bytes after this lead byte":
3  if ((c0 & 0xF0) == 0xE0 && /* two real continuation bytes follow */)
    {
4      uint32_t cp = /* decode the 16-bit code point from all three
        bytes */;
5      return (cp >= 0x800 && !(cp >= 0xD800 && cp <= 0xDFFF)) ? 3 : 0;

```

```

6      //      ^ rejects overlong (cp < 0x800 has a shorter valid
      encoding)
7      //      ^ rejects UTF-16 surrogates directly
      encoded in UTF-8
8  }

```

The constructor also earns a real, specific correction over the naive default: real .NET's own `UTF8Encoding.SetDefaultFallbacks()` substitutes the Unicode replacement character `U+FFFD` for invalid input, not the generic `Encoding` base class's plain `"?"` default (correct only for single-byte code pages, where every valid character already has an unambiguous byte representation to fall back to). This port's constructor reproduces that distinction precisely:

```

1  UTF8Encoding::UTF8Encoding() {
2      // Real UTF8Encoding.SetDefaultFallbacks() uses a U+FFFD
      replacement
3      // fallback, not the generic Encoding base class's "?" default.
4      setEncoderFallbackProperty(
5          std::make_shared<EncoderReplacementFallback>("\xEF\xBF\xBD")
6      );
7      setDecoderFallbackProperty(
8          std::make_shared<DecoderReplacementFallback>("\xEF\xBF\xBD")
9      );
10 }

```

A worked example makes the whole mechanism concrete — three real byte sequences, one well-formed and two deliberately malformed in different ways, through the same `GetString()` call:

```

1  System::Text::UTF8Encoding utf8;
2
3  // Well-formed: a real 3-byte sequence for U+20AC (EURO SIGN).
4  const SharpRuntime::bytecs euroSign[] = {0xE2, 0x82, 0xAC};
5  std::string ok = utf8.GetString(euroSign, 0, 3);           // EURO
      SIGN, unchanged
6
7  // Overlong encoding of U+002F ('/') using the 2-byte form --
      structurally
8  // valid continuation-byte shape, but wellFormedUtf8Length() rejects
      it
9  // because cp (0x2F) is below the 2-byte form's own 0x80 minimum.
10 const SharpRuntime::bytecs overlongSlash[] = {0xC0, 0xAF};
11 std::string rejected = utf8.GetString(overlongSlash, 0, 2); // two
      U+FFFD
12
13 // A lone continuation byte with no lead byte before it -- fails
      every one
14 // of wellFormedUtf8Length()'s four branches, falls through to the
      final
15 // "return 0" unconditionally.
16 const SharpRuntime::bytecs loneContinuation[] = {0x80};
17 std::string alsoRejected = utf8.GetString(loneContinuation, 0, 1);
      // one U+FFFD

```

The overlong and lone-continuation cases each substitute one `U+FFFD` per invalid *byte*, not per attempted sequence — `overlongSlash`'s two bytes both fail validation independently (the loop advances one byte at a time through the fallback path, per `UTF8Encoding.cpp`'s own `++i` in the

non-well-formed branch), so a 2-byte overlong sequence becomes two replacement characters, not one. This matches real .NET's own observable behavior for the identical input, confirmed by working through the .NET runtime's own documented per-byte fallback-and-resynchronize algorithm for UTF-8 decoding rather than assumed by analogy.

This behavior now has direct regression evidence. `UTF8EncodingTests` covers malformed continuations, overlong input, truncated sequences, directly encoded surrogates and exception fallbacks, including the one-replacement-per-byte resynchronization used above. The separate `Utf8Tests` family validates overlong sequences, surrogates, code points above `U+10FFFF`, truncation and lone continuation bytes through the span-oriented validator. The source read explains the algorithm; these two distinct test surfaces pin both the encoding facade's observable substitutions and the lower-level validity classification.

56.7 Generic continuations exist; generic combinators do not

Both non-generic `Task` and generic-result `TaskT<TResult>` have real `ContinueWith` implementations. Their shared-state shapes each contain a mutex, condition variable, and callback list; registered callbacks execute inline on the thread that completes the antecedent, or synchronously on the caller when it is already complete. `TaskT<TResult>` provides both an action overload returning `Task` and a result-producing overload returning `TaskT<TNewResult>`. The tests cover success, fault, cancellation inspection, filtered options, result chaining, empty callables, and release of captured state. Both implementations capture the antecedent state weakly to avoid a cycle from that state's own continuation list back to itself.

The narrower missing surface is aggregation over generic tasks. Static `WhenAll` and `WhenAny` accept only `std::vector<Task>`; there is no corresponding overload for `std::vector<TaskT<TResult>>` that produces a result array or a winning generic task. Thus callers can attach a continuation to one generic result, but cannot express .NET's generic multi-task combinators through this API. A second minor boundary is that the lightweight antecedent reconstructed for a callback shares terminal status/result/exception state but not the original separately-held cancellation-token object, so its token property reports `None`.

56.8 Reading a dependency from source

A reliable dependency review uses three layers in order. First derive the public include from the type's fully-qualified name. Then locate the module whose public include directory owns the file. Finally inspect that component's declared public dependencies rather than inferring a closure from nested namespaces. Namespace depth, directory depth, and link depth often align, but the component registry — particularly compatibility umbrellas and the `Xml.XPath` alias — is the authority when they do not.

The representative types in this chapter also show why include availability is weaker than semantic parity. `UTF8Encoding`'s malformed-input behavior is implemented and directly tested; `TaskT<TResult>` has generic continuations but not generic multi-task combinators. A mechanically correct path proves that a type can be named. It does not prove that every .NET behavior behind that name exists.

Chapter 57

Parity and Permanent Deviations

Every project this book has covered draws a line between what it will faithfully reproduce and what it will not. Sharp-runtime states its target as *maximum practical parity*: public API, semantics, defaults, error messages, and algorithms should match .NET as closely as C++ allows. That target is stronger than name compatibility and narrower than reimplementing the CLR. Its permanent deviations are part of the contract, not embarrassing footnotes.

57.1 Three outcomes, not one generic stub

An implementable operation should behave like .NET. An operation whose mechanism cannot be represented should throw `NotImplementedException` with an explanation rather than return a plausible but wrong value. A meaningful operation that the current host cannot provide should instead throw `PlatformNotSupportedException`. These outcomes answer different questions:

Parity the operation is implemented and its observable behavior is the verification target;

Permanent deviation the CLR-dependent capability is outside the runtime’s design;

Platform limitation the abstraction is valid, but this platform lacks its mechanism.

A silent success value is not a fourth acceptable category. This taxonomy lets callers and tests distinguish a design boundary from an environment boundary.

57.2 Reflection: correctly a stub, not a gap

`System::Type`, `Activator`, and enum reflection (`Enum.GetNames / GetValues`) are, in the project’s own words, “completely out of scope. Stubs are the correct end state.” This is a stronger claim than “not yet implemented” — it states that a stub *is* the intended final behavior, not a placeholder awaiting more engineering. Reflection depends on runtime type metadata a compiled C++ binary simply does not carry the way a .NET assembly does; reproducing it would mean building a real, if partial, runtime type system from scratch, disproportionate to what CNA’s own XNA-facing code actually needs.

57.3 Garbage collection: inert compatibility answers, by design

`System::GC` has no collector behind it. Mutating methods such as `Collect` and `SuppressFinalize` are no-ops, while queries return fixed documented answers: allocation statistics are zero, no-GC-region requests return false, notification waits report `NotApplicable`, and `MaxGeneration` remains the compatibility constant 2. Object lifetime is managed by RAII and `shared_ptr`; callers may compile defensive GC calls but must not infer live heap telemetry from the surface.

57.4 Delegates: a three-tier model, corrected once in the project's own history

This is worth covering in detail because the project's own documentation records a real self-correction: an earlier description of the delegate model was found inaccurate during an audit and rewritten. The corrected, current model has three distinct tiers, each solving a different shape of problem. Most delegate-shaped C# types — `Action`, `Func`, most `*Callback` and `*EventHandler` aliases — are bare `using` aliases over `std::function`: single-target, no multicast, no `BeginInvoke` / `EndInvoke`, sufficient for the overwhelming majority of call sites that only ever needed one callback anyway. `System::Delegate` is a real multicast base (`Combine`, `Remove`, `GetInvocationList`) for the smaller set of call sites that genuinely need C#'s multicast-delegate combining semantics. `System::MulticastAction<Args...>` (Chapter 53) is a third, purpose-built type specifically for multicast event *fields*, using token-based removal since C++ callables have no target-plus-method equality to compare by the way C# delegates do. The specialized `EventHandler<TEventArgs>` collection applies the same event-field idea to the sender/argument shape and adds an optional replay hook. Of these representations, only `Delegate` exposes `DynamicInvoke`; it always throws `NotImplementedException`. The alias and event-field types do not pretend to expose a late-bound argument-array call at all.

```

1  // Worked example: all three tiers side by side, each solving the
   // problem it exists for.
2
3
4  // Tier 1 -- Action/Func alias: single-target, the overwhelming
   // common case.
5  System::ActionT<int> onScoreChanged = [](int newScore) {
6      UpdateHudScore(newScore); };
7
8  // Tier 2 -- System::Delegate: real multicast combining semantics,
   // when a
9  // caller genuinely needs to build up and later shrink an invocation
   // list.
10 auto d1 = std::make_shared<System::Delegate>(System::Delegate::
11     ErasedInvoke([] { PlayFanfare(); }));
12 auto d2 = std::make_shared<System::Delegate>(System::Delegate::
13     ErasedInvoke([] { UnlockNextLevel(); }));
14 std::shared_ptr<System::Delegate> onLevelComplete = System::Delegate
15     ::Combine(d1, d2);
16 onLevelComplete->Invoke(); //
   // runs both
17 onLevelComplete = System::Delegate::Remove(onLevelComplete, d1); //
   // now only d2 remains
18
19 // Tier 3 -- MulticastAction: the right choice for an event FIELD,
   // since
20 // std::function values have no target-plus-method equality to
   // remove by.
21 System::MulticastAction<> onGameOver;
22 auto token = onGameOver.Add([] { ShowGameOverScreen(); });
23 onGameOver.Remove(token);

```

57.5 Serialization, P/Invoke, and cryptography: three different reasons for the same conclusion

Serialization is “ignored, not needed for game code” — a scope judgment rather than a technical impossibility. P/Invoke and general native interop are out of scope for the same underlying reason Chapter 3’s type-alias table exists in the first place: this project already *is* native code, so a marshaling layer for calling into native code from managed code has no problem left to solve here.

Cryptography and TLS are the one deviation with the most fully spelled-out reasoning of all, and it deserves quoting closely: symmetric and asymmetric cryptography, X.509 certificates, and `SslStream` are out of scope by an explicit, dated project-owner decision, on the grounds that “implementing this correctly needs either a large new external dependency (OpenSSL/mbedTLS) or a hand-rolled, security-critical implementation, neither of which is worth it for game code.” This is a materially different kind of reasoning than the other deviations in this chapter — not “this feature has no meaning here” (reflection, GC) and not “this is out of scope for capacity reasons” (serialization), but an explicit refusal to carry the security liability of a hand-rolled cryptographic implementation, or the dependency weight of a real one, for a class of application that does not need transport security built into its own runtime layer. Digest and derivation APIs — MD5, SHA, HMAC, and PBKDF2 — remain in scope. That does not make them “non-cryptographic,” nor does it imply that HMAC or PBKDF2 has no key or password material. The narrower project distinction is that these APIs do not implement reversible confidentiality, certificate validation, or a TLS channel. Callers still own algorithm choice, parameter strength, and security review.

57.6 IDisposable and the exception hierarchy: parity by pattern, not by mechanism

Two further deviations are less often stated outright as deviations, because the project does not treat them as ones — they are cases where matching .NET’s *observable behavior* required deliberately *not* matching its mechanism. `IDisposable`’s own header comment states the tradeoff directly: “In C#, the `using` statement calls `Dispose()` automatically at the end of the block. In C++, use RAII or call `Dispose()` explicitly.” There is no C++ language construct that runs arbitrary code at scope exit the way a `using` block does short of writing a dedicated RAII wrapper per call site, so this project’s actual parity target is narrower and more precise than “reproduce the `using` statement”: reproduce *Dispose()*’s own contract — idempotent, generally non-throwing, and safe to call more than once — and let the caller choose their own C++-native mechanism (an explicit call, a destructor, or a scope guard) for invoking it at the right time.

`System.Threading.ReaderWriterLockSlim` is a concrete, real implementation of this contract worth reading closely, because its own header comment documents a genuine self-correction of exactly the kind this chapter’s delegate discussion above already showed this project performing on itself. An earlier version of this class had three real, independently-found bugs, each one a case where a C++ implementation that merely *looked* equivalent to the real .NET semantics was not actually equivalent under closer verification: it discarded the `millisecondsTimeout` parameter on every `TryEnter*` overload, always making a single non-blocking attempt regardless of what the caller passed; it ignored the constructor’s `LockRecursionPolicy` entirely, so same-thread recursive acquisition always deadlocked instead of throwing `LockRecursionException` under the real .NET-default `NoRecursion` policy; and it tracked reader-lock ownership by set *membership* rather than by count, so

a legitimately nested `EnterReadLock()` / `EnterReadLock()` / `ExitReadLock()` sequence's second `ExitReadLock()` call threw `SynchronizationLockException` instead of correctly decrementing — which additionally meant the internal reader tally could never reach zero from a genuinely nested acquisition, permanently starving any waiting writer. All three are fixed in the current header, verified directly against real .NET's own `TryEnterReadLockCore` / `TryEnterWriteLockCore` / `TryEnterUpgradeableReadLockCore` logic per the header's own verification comment.

```

1  // Worked example: IDisposable's contract, applied explicitly (no C
    ++ "using"
2  // block exists), with the real ObjectDisposedException guard every
    public
3  // entry point in ReaderWriterLockSlim shares (throwIfDisposed(),
    called
4  // first inside TryEnterReadLock/TryEnterWriteLock/
    TryEnterUpgradeableReadLock).
5  System::Threading::ReaderWriterLockSlim rwLock;
6
7  rwLock.EnterReadLock();
8  // ... read shared state ...
9  rwLock.ExitReadLock();
10
11 rwLock.EnterWriteLock();
12 // ... mutate shared state ...
13 rwLock.ExitWriteLock();
14
15 rwLock.Dispose();           // explicit -- no using-block equivalent
    exists
16 rwLock.Dispose();           // safe to repeat: Dispose() only ever
    sets a flag
17 try {
18     rwLock.EnterReadLock();   // throwIfDisposed() now fires
19 } catch (const System::ObjectDisposedException&) {
20     // "ReaderWriterLockSlim" -- the disposed object's own class
    name,
21     // matching real .NET's ObjectDisposedException(objectName)
    constructor.
22 }
```

The exception hierarchy this last block leans on is itself worth a concrete scale rather than a vague gesture at “lots of exception types”: a project-wide search finds roughly 111 exception classes across the `include/System` tree, the overwhelming majority deriving not directly from `System::Exception` but through the intermediate `System::SystemException` base — matching real .NET's own two-level shape, where framework-thrown exceptions (`ArgumentException`, `InvalidOperationException`, `ObjectDisposedException`, and the rest this book's other chapters have already used by name) sit under `SystemException` rather than hanging directly off the root. The root class's own header comment makes explicit exactly which real .NET `Exception` members this project deliberately does not provide, and why each one is a concrete instance of a deviation this chapter has already stated abstractly: `GetBaseException()` would require cloning the thrown object through its base type, which needs a virtual-clone mechanism this port does not have; `ToString()` would need a `GetType()`-equivalent class name, out of scope under this chapter's opening reflection deviation; and `TargetSite` and `GetObjectData` are out of scope for the same reflection and serialization deviations covered earlier in this chapter. Nothing here is a new decision — it is the same two decisions, now traceable to the one root class every exception in the ecosystem

inherits from.

57.7 Naming is a compatibility surface

Sharp-runtime uses PascalCase .NET names, but C++ cannot expose C# properties directly. The binding convention is `getXxxProperty()` and `setXxxProperty()`; indexers are the systematic exception and use `getItem()` / `setItem()`. Public headers contain thousands of these accessors, so a cosmetic refactor would be an ecosystem break rather than a local cleanup.

The project's own rules name CNA as the reason broad header renaming is frozen. That is strong evidence of co-evolution: the runtime is a separate repository, yet its source conventions are constrained by CNA's compiled public surface. Porting rules similarly require the `intcs/bytecs` family and direct nested namespace syntax, while LINQ-shaped work should use `std::ranges` rather than growing a second query runtime.

When a naming correction is nevertheless intentional, the project does not preserve a stale alias indefinitely. The `getCurrent()` to `getCurrentProperty()` migration changed 61 call sites across 23 files in one commit and required downstream repair. The lesson is not that names may churn casually; it is that a deliberate convention repair should be atomic, auditable, and free of permanent compatibility debris.

57.8 Two further, narrower accepted boundaries

Platform behavior is split more finely than a single "POSIX-only" label. `Sockets` and `IO::RandomAccess` have Windows and POSIX implementations and throw on Emscripten. `AppDomain`'s base directory works on Windows, macOS and Linux/POSIX, with a relative virtual-filesystem fallback on Emscripten; `TimeZoneInfo` likewise has Windows and POSIX paths but only UTC/local fallback there. `Diagnostics::Process` and POSIX-signal registration are POSIX implementations that throw on Windows/Emscripten, while `NetworkInterface` enumeration and `FileSystemWatcher` are narrower Linux-only operations. Unsupported paths compile and fail explicitly rather than disappearing from the headers.

Separately, `Decimal`, `Int128`, and `UInt128` require a compiler-provided 16-byte `__int128`. The CMake probe admits x86-64 GCC/Clang, including x86-64 MinGW GCC, but excludes MSVC and i686 MinGW GCC. This is a compiler-capability boundary, not simply a Windows boundary, and reflects the deliberate, dated decision not to hand-roll 128-bit arithmetic for toolchains that lack the native type.

57.9 A documented deviation still needs verification

Calling a gap permanent does not exempt its boundary from tests. GC's inert methods and sentinel queries should remain callable and stable; `Type`'s identity operations must work even though its classification predicates are placeholders; `Delegate::DynamicInvoke` must fail explicitly; and unsupported platform operations must throw the promised exception rather than silently degrade. Chapter 58 shows how component tests, negative consumers, sanitizer runs, and the repository's large audit ledger test both implemented behavior and the honesty of these boundaries.

Chapter 58

Verification and Audit in sharp-runtime

Sharp-runtime’s verification system is not one test executable and a green badge. It combines component-isolated GoogleTest programs, integration tests, compile boundaries, positive and negative consumers, Python meta-tests, sanitizer probes, and an audit record approaching one file per source file. The layers answer different questions, and the current result is deliberately not summarized as “all passing.”

58.1 Three test locations have three jobs

The singular and plural directory names are semantically distinct:

modules/<module>/tests/ holds the primary unit tests. Its 477 C++ files mirror their owning module trees.

tests/ holds cross-cutting C++ integration tests and the i686 compile-boundary source. These are intentionally outside any one module.

test/ holds Python meta-tests and CMake consumer fixtures. Some fixtures are expected to fail compilation or linking.

GoogleTest is vendored as a required submodule. When tests are enabled, CMake discovers test sources and creates one executable for each component that owns tests, plus an integration executable. The component name prefixes discovered cases, making failures attributable without reconstructing ownership from a monolithic binary.

58.2 The executable boundary tests the dependency boundary

Each component test executable links its component, `gtest_main`, and only declared `TEST_DEPENDENCIES`. A test that reaches across an undeclared module edge can fail to link. This converts a build-architecture rule into an executable check: the same test body linked against a universal archive would compile and conceal the leak.

Source partition validation reinforces that boundary before execution. The module dependency allowlist is empty, so there is no standing collection of exceptions silently legitimizing misowned files. Test-only edges remain separate from production edges, preventing a fixture’s helper dependency from widening the shipped component.

```
1 # Conceptual shape produced by the component registry
2 add_executable(SharpRuntimeTests_Core_Base ...)
3 target_link_libraries(SharpRuntimeTests_Core_Base PRIVATE
4     SharpRuntime::Core.Base gtest_main)
5
```

```

6 gtest_discover_tests(SharpRuntimeTests_Core_Base
7   TEST_PREFIX "Core.Base::")

```

Thirty-six components currently have such a test executable. Five module directories do not: `io-compression-zip`, `io-isolated-storage`, `security-cryptography-random`, `storage`, and `text-regular-expressions`. The last is a 12-header interface component with neither implementation nor tests. Listing it explicitly is more informative than letting the global count imply uniform coverage.

58.3 The measured gate is not green

The repository continuation record reports 16,310 tests across 37 executables: 16,303 passing, one skipped, and six failing from two recorded causes. An older baseline of 15,071, or a still older claim that every test passes, describes a different revision. The current failures are neither converted into skips nor omitted from the denominator.

This arithmetic is independently reconcilable: 36 component executables plus one integration executable equals 37. It also clarifies what the count excludes. Python meta-tests, individual consumer configure/build cases, compile-only platform probes, and audit documents are evidence, but they are not GoogleTest registrations and must not be added to 16,310.

The test corpus contains 102,342 lines, about 30% more than the 78,934 production lines. That ratio signals investment, not proof. Dedicated malformed-UTF-8 tests now close one formerly reported gap, while untested components and recorded failures remain visible despite the large total.

58.4 Negative consumers prove absence

A positive consumer answers “can the intended dependency compile?” A negative consumer asks “does a forbidden dependency fail?” The latter is essential for component isolation. Current fixtures cover, among other boundaries, forbidden Text.Json-to-Collections edges, XML-to-Diagnostics leakage, collection setter/enumerator constraints, generic math requirements, and cryptographic key-material exclusions.

The harness injects and configures these small consumers through CMake. For a negative fixture, a compiler or linker failure at the expected site is success; a clean build means the boundary was accidentally weakened. A generic CI wrapper that treats every non-zero child process as a failure would reverse the verdict, so Python meta-tests inspect the intended outcome and site. The continuation record tracks 13 negative-fixture files and 116 checked sites.

Three top-level Python tests cover complementary build invariants:

- `validate_module_boundaries_test.py` checks partition and dependency rules;
- `check_negative_consumer_fixtures_test.py` verifies must-fail consumers; and
- `check_version_seam_odr_test.py` checks the version seam for one-definition rule failures.

58.5 Audit is a tracked data set

The `audit/` directory contains 1,748 `*.audit.md` files against 1,796 tracked C/C++ source and header files. `AUDIT_FINDINGS_INDEX.md` assigns stable `SR-AUD-###` identifiers, severity, status,

and disposition. At the audited handoff, numbering is frozen at 364: 161 remediated and 203 confirmed, including 53 marked design-complete.

“Confirmed” is not synonymous with “forgotten.” It can denote an observed deviation whose repair requires approval, unavailable evidence, a platform, or an intentional design decision. “Remediated” records a landed correction. Freezing identifiers prevents later batches from making the denominator look better by renumbering or discarding inconvenient findings.

The supporting apparatus is large in its own right: 72 tracked Markdown documents under `docs/`, plus the root `plan.md`. The pinned tree also tracks the exact 7,471,104-byte (about 7.13 MiB) `plan.sqlite3` blob present in this checkout. Its name matches a current `.gitignore` rule, but ignore rules do not untrack an existing file; both Git and a blob-hash comparison prove this snapshot belongs to commit `f827a6c5`. These artifacts are not runtime code and should not inflate implementation size, but they explain how a multi-thousand-file audit remains navigable and resumable.

58.6 A completed task list triggered more skepticism

Earlier stabilization work once reached a fully checked-off ticket table. The useful response was not to declare parity complete, but to commission fresh reviews across API consistency, silent behavior differences, exceptions, stubs, platforms, and high-risk missing tests. Those reviews found defects that ordinary happy-path tests had missed.

Representative findings illustrate distinct failure modes:

- floating-point `Convert.ToInt32` used `static_cast` truncation instead of .NET’s round-half-to-even behavior;
- a Dictionary read through non-const `operator[]` inserted a default value instead of throwing `KeyNotFoundException`;
- `ConcurrentDictionary` factories ran under a non-recursive mutex, so reentrant factories could self-deadlock;
- mutable generic collections lacked fail-fast version checks during enumeration; and
- `MemoryStream::Write` allowed a negative offset to reach an unchecked copy.

The fixes were not one generic “audit cleanup.” Conversion needed correct numeric semantics; Dictionary needed a proxy to distinguish reads from writes; concurrent factories needed an unlocked callback window; each enumerable collection needed its own version discipline; and `MemoryStream` needed argument validation. The finding taxonomy located the problems, while each repair still required a type-specific oracle.

58.7 Sanitizers answer a different class of question

Independent sanitizer runs exposed failures that API comparison alone was unlikely to find. `ThreadSanitizer` identified a production lost-wakeup path in a `Channel` missing `notify_all`, along with test-only races. `AddressSanitizer` found an aligned-reallocation heap overflow and XML node leaks. `UndefinedBehaviorSanitizer` reached a vendored miniz call with an invalid empty-buffer pointer contract.

One particularly instructive non-sanitizer race lived between two valid-looking operations. `TaskCompletionSource<TResult>::TrySetResult` atomically claimed completion before copying the result into its promise. If `TResult`’s copy constructor threw, the claim remained

set but the promise never became ready; no later producer could settle it and all waiters hung. The repair catches the copy failure, stores it into the promise so waiters become ready with an exception, and then rethrows to the immediate producer.

```
1  bool TrySetResult(const TResult& result) {
2      bool expected = false;
3      if (!completed_.compare_exchange_strong(expected, true)) return
        false;
4      try {
5          promise_.set_value(result);
6      } catch (...) {
7          promise_.set_exception(std::current_exception());
8          throw;
9      }
10     return true;
11 }
```

The lesson is not that sanitizers prove .NET parity. They detect memory, undefined-behavior, and concurrency classes that semantic comparison may overlook; numerical or API deviations can remain in a sanitizer-clean run.

58.8 Reproducibility has an external seam

The porting rules refer to a .NET reference-source tree at an absolute `/rv/tmp/runtime/src/libraries/` path. That tree is neither vendored nor pinned in sharp-runtime, and the environment recorded in the current handoff did not contain it. Doxygen and `ccache` were absent too. Existing source comments and audits may accurately cite a past comparison, but a new contributor cannot reproduce every line-by-line reference check from the repository alone.

There is a fail-fast local gate and a tracked Ubuntu GitHub Actions workflow. Its matrix runs nine selective component/consumer configurations, the full compatibility gate, and Doxygen warning validation on pushes and pull requests with a two-job ceiling. Workflow presence is not proof that every historical revision completed it, and it supplies no Windows, macOS, or Emscripten execution. Platform claims still require the same scope discipline: a native test pass, a successful cross-compile of libraries, and actual execution of cross-built tests are three separate results.

Sharp-runtime's strongest verification property is therefore not a perfect green number. It is the ability to state what each layer proved, preserve failures and gates under stable IDs, and resume from a measured boundary without converting missing evidence into success.

Part IX

Sibling Foundation Libraries

Chapter 59

easy-gl and meta-gl: Profile, Loader, and Dispatch

This book introduced **easy-gl** as the C++23 OpenGL/OpenGL ES wrapper behind CNA’s five shared-family profiles: **OPENGLES2**, **OPENGLES3**, **OPENGL33**, **WEBGL1**, and **WEBGL2**. That implementation family uses a two-library stack. **meta-gl** owns loading, typed calls, context facts, and validation; **easy-gl** owns resources, object-oriented state, and RAII.

59.1 The ownership split

easy-gl’s own README frames it as toolkit-independent, meaning independent of a windowing toolkit like SDL or Qt — but it is not independent of a lower C++ GL wrapper underneath it. **easy-gl** itself is built on a sibling project, **meta-gl**, which provides the actual procedural, typed, ownership-free OpenGL/GLES call surface (typed enum-class wrappers, `std::span`-based views, lightweight typed handles, no RAII of its own); **easy-gl** then layers real object-oriented, RAII, move-only ownership semantics on top. The dependency is real and direct in the build system (**meta-gl** is pulled in as a sibling `add_subdirectory` and linked publicly), and in the source: 18 production files include **meta-gl** headers directly, comprising three public headers and fifteen implementation files. The layering is worth remembering precisely because it clarifies what “toolkit-independent” actually means here: independent of the host windowing/event toolkit, not independent of every other C++ abstraction below it.

Neither library creates a window or GL context, pumps events, nor presents a frame. The host owns those operations and supplies a `GetProcAddress` callback after making its context current. CNA fills that host role through SDL, while the same libraries remain usable with a different toolkit.

The size inversion is easy to miss. **Easy-gl** exposes 30 headers and about 3,606 production code lines; **meta-gl** exposes only 12 headers but roughly 8,856 true C/C++ code lines. The lower procedural layer is about 2.5 times larger because its generated-looking surface is still a hand-audited typed mapping of hundreds of GL entry points.

59.2 Device initialization delegates context truth downward

`easygl::Device` is the largest single entry point, covering initialization, the bulk of GL state (clear, viewport/scissor, blend — including indexed, per-attachment blend functions for multiple render targets — depth/stencil, culling, polygon/line mode, sample coverage), pixel read-back (a bounds-checked `read_pixels_robust` alongside the plain form), the full draw-call family (instanced, indirect, ranged, base-vertex), compute dispatch, tessellation patch configuration, and debug facilities (labeled debug groups, a settable debug callback).

Older easy-gl code duplicated version-string parsing. The audited implementation calls `metagl::Initialize()` and then copies `GetContextInfo()` and `GetCapabilities()` into its higher-level model. This matters on the web: meta-gl classifies Emscripten contexts as `ApiKind::WebGL`, so WebGL no longer masquerades as ordinary OpenGL ES merely because its shading language is ES-shaped.

Meta-gl also computes a `DesktopEsTier`: the OpenGL ES tier whose mandatory functions a desktop OpenGL context can supply. That mapping is what lets CNA’s `OPENGL33` identity share one implementation with native ES and WebGL profiles without pretending that their version numbers have identical meanings.

Once detection completes, easy-gl requires vertex arrays, shaders, programs, buffers, and basic rendering. A missing baseline throws during initialization. Optional entry points — queries, samplers, transform feedback, syncs, program pipelines, and texture-level queries — are checked through `metagl::IsFunctionAvailable()` and fail with `easygl::UnsupportedFeatureException` instead of calling a null function pointer. WebGL 1 may obtain vertex arrays through `GL_OES_vertex_array_object`; separable program pipelines have no WebGL equivalent, including WebGL 2.

59.3 Capability gating by feature, not version

`easygl::Capabilities::detect_common_features()` implements a genuine, per-feature gating table rather than a single “desktop GL is capability X, GLES is capability Y” heuristic, and the two API families frequently diverge in specific, non-obvious ways. Vertex array objects need GL 3.0 (or the `GL_ARB_vertex_array_object` extension) on desktop, but GLES 3.0 or the differently-named `GL_OES_vertex_array_object` extension on GLES. Framebuffer objects need GL 3.0 or an extension on desktop, but are simply always present on GLES 2.0 and above, since FBOs are core to ES2 from the start. Tessellation shaders need GL 4.0 on desktop but only GLES 3.2 — a case where the GLES requirement is numerically *lower*. Direct State Access, by contrast, is not modeled for GLES at all, since it is a desktop-only OpenGL 4.5 capability with no GLES equivalent whatsoever. This per-feature table — not a single “is this GLES” branch — is exactly the mechanism Chapter 22 described CNA’s own EasyGL renderer implementation relying on via `device.supports(...)` / `device.require(...)`.

```

1  // Worked example: querying the per-feature table directly, rather
   // than
2  // branching on Device::capabilities().is_opengles() and guessing.
3  if (device.supports(easygl::Feature::TessellationShader))
4  {
5      ConfigurePatchPipeline();
6  }
7  else
8  {
9      UseFallbackSubdivision(); // e.g. desktop GL < 4.0, or GLES <
   // 3.2
10 }
11
12 // With the default Config::throw_on_missing_feature == true,
13 // Device::require() throws immediately if the feature is missing.
14 device.require(easygl::Feature::FramebufferObject);

```

59.4 Meta-gl’s typed call surface and invalid-input contract

Meta-gl exports 358 numbered `metagl::gl*` wrappers. A Python verifier requires the markers to form one contiguous 1–358 sequence with unique names and cross-checks the mandatory OpenGL ES 3.0, 3.1, and 3.2 sets (104, 68, and 44 additions) against the vendored Khronos header. The public type vocabulary contains 105 enum classes in `Enums.hpp` and 16 struct declarations in `Types.hpp`: typed object handles, locations, indices, an image unit, and a bitfield-traits helper. Older continuation counts of 99 enums and 15 structs are stale.

These handles are intentionally ownership-free:

```
1  metagl::TextureId texture{rawName};
2  metagl::glBindTexture(metagl::TextureTarget::Texture2D, texture);
3
4  // TextureId does not delete rawName. easygl::Texture owns that
   lifetime.
```

The two layers also choose different failure semantics. Easy-gl reports unsupported features with C++ exceptions. Meta-gl’s invalid-input contract calls `std::terminate()` even in Release for conditions such as a `size_t` that cannot fit in `GLsizei`, incomplete matrix data, or an unsupported bitfield. Those are treated as caller contract violations, not recoverable driver capabilities.

This difference has a build consequence on Emscripten. Easy-gl enables `-fwasm-exceptions` globally before adding meta-gl so every object in the combined stack agrees on the Wasm exception model. A failure-policy choice at the OOP layer therefore affects how the lower procedural layer must be compiled even though meta-gl itself does not throw for its input contract.

Meta-gl’s small header count also hides unusually strict binary discipline: hidden visibility, a public export macro, a GNU/Clang version script, SONAME and exported-symbol policy tests, and an installed-package consumer. It can be found as `meta-gl 0.3`; an uncommitted `0.4.0-snapshot` file in the local worktree is not a released or pinned version.

59.5 Design corrections and one breaking change

easy-gl’s own planning documents record several real, reasoned design corrections.

`Program::uniform_block_index()` originally returned GL’s own raw `GL_INVALID_INDEX` sentinel on a failed lookup, forcing every caller to know and compare against a raw GL constant; it now returns `std::optional<unsigned int>` instead, so a caller never needs to include a GL header at all just to check for failure. Several resource classes (`Framebuffer`, `ProgramPipeline`, `TransformFeedback`) were changed from accepting a raw `unsigned int` GL handle to accepting a real `Texture` / `Program` reference directly, removing a whole class of “passed the wrong handle type by mistake” bug at the API boundary.

```
1  // Worked example: both fixes in the same worked snippet. No raw GL
2  // constant comparison, and no raw GLuint handle to mix up with an
3  // unrelated resource’s handle by mistake.
4  if (std::optional<unsigned int> blockIdx = shaderProgram.
      uniform_block_index("Lighting"))
5  {
6      shaderProgram.set_uniform_block_binding(*blockIdx, /*
          bindingPoint */ 0);
7  }
```

```

8 // else: no "Lighting" uniform block in this program -- handled
  without
9 // ever needing to know or compare against GL_INVALID_INDEX directly
  .
10
11 Framebuffer gbuffer;
12 gbuffer.attach_texture_2d(FramebufferTarget::Framebuffer,
    FramebufferAttachment::Depth,
13                          TextureTarget::Texture2D, depthTexture,
    /* level */ 0);
14 // depthTexture is a real Texture@, not a bare GLuint -- passing a
15 // Renderbuffer's handle here by mistake is now a compile error, not
  a
16 // runtime GL_INVALID_OPERATION discovered later.

```

One specific, genuine breaking behavioral change is worth knowing if you are porting code against an older version of this library: `Device::clear()` used to call `glDisable(GL_SCISSOR_TEST)` before every clear, silently overriding whatever scissor state the caller had already set. This is no longer true — `clear()` no longer touches scissor state at all, and a caller that was relying on the old implicit disable must now call the scissor-disable method explicitly. The project's own migration notes frame this as low-impact, since it only matters when scissor testing happens to already be active at the moment `clear()` is called — but it is exactly the kind of silent, implicit side-effect this book has flagged elsewhere (Chapter 13's ignored transform matrix, for instance) as the class of bug most likely to go unnoticed until specifically tested for.

59.5.1 Four focused RAII additions

Two small utility classes, added after this chapter's own original pass, are worth a direct mention because each closes a real, easy-to-get-wrong ergonomic gap rather than adding a new GL capability. `ScopedDebugGroup` is a two-call RAII pair — its constructor calls `Device::push_debug_group`, its destructor unconditionally calls `pop_debug_group` — specifically so a labeled debug region in a `RenderDoc` or `NVIDIA Nsight` capture can never be left unbalanced by an early return or an exception unwinding through the scope:

```

1 {
2     ScopedDebugGroup group(device, "Shadow Pass");
3     RenderShadowCasters();
4     // pop_debug_group() always runs here, even if
      RenderShadowCasters()
5     // throws or returns early -- no separate try/finally needed.
6 }

```

`UniformCache` addresses the other classic OpenGL ergonomics problem: repeatedly calling `glGetUniformLocation` by name is a real, measurable per-draw cost real engines avoid by caching the resolved integer location once. `operator[]` looks up a cached location by name, falling through to a real `Program::uniform_location` call and memoizing the result only on a cache miss — and, worth knowing explicitly since it is not stated in the header's own brief comment, a *failed* lookup (a mistyped or since-optimized-out uniform name, returning `-1`) is memoized exactly the same way a successful one is. Calling `operator[]` again with the same wrong name will not re-attempt the GL query and magically start succeeding once the shader changes — only an explicit `invalidate()` call (needed after any program relink, since old locations are not guaranteed valid against a newly linked program) clears the cache and allows a fresh lookup:

```

1 UniformCache uniforms(shaderProgram);
2 shaderProgram.set_uniform(uniforms["uTime"], elapsedSeconds);
3 shaderProgram.set_uniform(uniforms["uLightDir"], lightDir.x,
   lightDir.y, lightDir.z);
4 // ... later, only after shaderProgram is actually relinked ...
5 uniforms.invalidate(); // required -- old cached locations are not
   safe to reuse otherwise

```

Two further small RAII utilities, added in the same wave as the two above, round out this pattern rather than introducing a new one. `ScopedBind` generalizes the same constructor-runs-first/destructor-always-runs shape into a reusable, resource-agnostic wrapper — constructed from any pair of bind/unbind callables, not tied to one specific GL object type at all:

```

1 {
2     ScopedBind bound(
3         [&]{ vao.bind(); },
4         [&]{ vao.unbind(); });
5     ConfigureVertexAttributes(vao);
6     // unbind() always runs here, even on an early return or a
   thrown
7     // exception mid-configuration -- the same guarantee
   ScopedDebugGroup
8     // gives for a debug region, applied generically to any bind/
   unbind
9     // pair -- VertexArray here, but the constructor takes any two
10    // callables, not one fixed resource type.
11 }

```

`ResourceRegistration` closes a different, narrower gap: any GL resource that participates in this library’s context-loss recovery mechanism must register itself with a `ResourceRegistry` to be told when to rebuild itself after a context is lost and recreated, and must remember to deregister on destruction to avoid the registry holding a dangling pointer. Before this addition, both calls were the caller’s own responsibility to remember; `ResourceRegistration`’s constructor calls `registry.add(&resource)` and its destructor unconditionally calls `registry.remove(&resource)`, the identical RAII shape as every other addition in this section, applied specifically to the register/deregister pair a `RecoverableResource` would otherwise have to manage by hand at every one of its own constructor and destructor sites.

59.6 Two tiers of context-loss recovery

`ResourceRegistration` above closes the register/deregister half of the context-loss story, but reading `GenerationTracked.hpp` and the context-lifecycle test suite together reveals a fuller picture worth spelling out on its own: `easy-gl` actually has *two* separate mechanisms for surviving a lost GL context, at two different levels of commitment, and only one of them is currently used by anything real.

The first tier covers every integer-named GL resource class and is effectively free. Those eleven classes inherit a common base class, `easygl::detail::GenerationTracked`. It stamps each resource with the meta-gl-tracked context generation number at creation time (`creation_generation()`) and exposes `is_valid_for_current_generation()` so any caller can cheaply ask “was this handle created against the context that is actually current right now, or a since-lost one” without any registration step at all. The pointer-shaped `Sync` object is the exception: it has its own creation and no-generation reset surface, so it can discard a

stale handle but cannot answer the generation-validity query. `reset_handle_no_gl()` zeroes either representation without issuing a single `gl*` call, since a lost context cannot be trusted to process one correctly anyway.

The second tier is the opt-in, full self-healing mechanism this chapter already covers: `RecoverableResource`'s `release_gl_handle_only()` / `recreate_gl_resource()` pair, driven by `ResourceRegistry` (itself a real `metagl::ContextListener`, receiving `OnContextLost()` / `OnContextRestored()` automatically once `register_with_meta_gl()` is called). This tier is where the real commitment lives: implementing it means a resource class must retain enough CPU-side state (raw pixel data, shader source text, buffer contents) to rebuild the GPU object from scratch, not merely notice that its handle went stale.

A genuinely worth-knowing gap: the infrastructure has production implementations — `ResourceRegistry` dispatches the two lifecycle calls and `ResourceRegistration` supplies RAI add/remove — but *zero concrete resource classes* adopt it. `Texture`, `Buffer`, `VertexArray`, `Framebuffer`, and every other real GL object class inherit `GenerationTracked` (tier one) but none of them implement `RecoverableResource` (tier two). The library's own planning document confirms this is a known, tracked state rather than an oversight this chapter is the first to notice: Task U2 ("Test context-loss / `ResourceRegistry` recovery cycle") is explicitly scoped against "a concrete `RecoverableResource` subclass" precisely because no real one currently exists to test against — and, consistent with that, `ContextLifecycleTests.cpp` tests this directly, with two dedicated `ResourceRegistry` tests:

- `test_resource_registry_context_lost`
- `test_resource_registry_wired_to_meta_gl`

Both construct a minimal, test-only `FakeResource` stub rather than exercising any real resource type, for exactly this reason:

```

1  // From the real test suite -- note this is a hand-rolled stub, not
   Texture,
2  // Buffer, or any other production easy-gl resource class. None of
   those
3  // currently implement RecoverableResource.
4  struct FakeResource : easygl::RecoverableResource
5  {
6      int& released;
7      int& recreated;
8      void release_gl_handle_only() override { ++released; }
9      void recreate_gl_resource()   override { ++recreated; }
10 };

```

The practical upshot for a caller today: a context loss is cheaply *detectable* per resource via `is_valid_for_current_generation()` (tier one, universal), but nothing in `easy-gl` itself will *automatically rebuild* a lost `Texture` or `Buffer` for you (tier two, fully built and independently tested as a mechanism, but with no adopters yet) — a caller that needs real survive-context-loss behavior today has to detect staleness via tier one and rebuild the resource manually, or write its own `RecoverableResource` subclass around its own CPU-side data. This is the same shape of gap this book has already named on the CNA side more than once (a mechanism exists and is correctly wired, but nothing yet plugs into it) — here, one level further down the stack, in the sibling library CNA's own `EasyGL` implementation is built on.

59.7 Testing without a GPU

Easy-gl's suite has four hand-rolled, mock-loader binaries: the initialization, resource, context-lifecycle, and WebGL smoke suites. Their CMake target names all begin `easy-gl-` and end in `-tests`. The mechanism making them useful without any real GPU or display is worth describing directly: fake `GLGetProcAddressFn` loaders return hard-wired callbacks, literal strings such as `"OpenGL ES 3.0"`, and literal enum values in place of driver responses. The smallest loaders provide the context-query handful needed by `Device::initialize()` and use a non-callable sentinel for other loaded core functions; the resource suite supplies a broader set of callable stubs for the operations it actually exercises. This lets context/capability detection and resource lifecycle assertions execute entirely off-GPU without accidentally issuing a real GL call.

The lifecycle suite drives context loss and restoration; the WebGL suite checks explicit classification, ES2/ES3 tiers, OES vertex-array gating, and exception-not-crash behavior for unavailable entry points. The desktop-shaped initialization and resource smoke binaries are not built or registered under Emscripten because their invented version strings cannot occur in a real WebGL host. This is an honest test-model boundary, not proof that the corresponding browser behavior was executed.

Nested build defaults are asymmetric. Easy-gl defaults both tests and examples ON. CNA's web preset explicitly sets both `EASYGL_BUILD_*` options to OFF, but CNA's root CMake and renderer-selection helper do not force either value. A direct GL-profile configuration that does not use that preset or pass its own overrides therefore inherits the four test binaries, and its SDL hello-triangle example can configure because CNA already supplies SDL. Meta-gl defaults all tests, GPU tests, examples, documentation, sanitizers, and debug options OFF when nested. The effective extra work consequently depends on the CNA configure entry point; the read-only source audit did not build the siblings to measure its wall-clock cost.

Meta-gl's standalone tests add compile, mock-loader, release-contract, thread, desktop-tier, SONAME, export-symbol, and installed-package checks. A real EGL/Mesa smoke test is separately opt-in, and requesting GPU tests without ordinary tests is a configure-time error.

59.8 Two known, honestly stated limitations

Two gaps are worth knowing about directly rather than assuming closed.

First, `Config::enable_debug_logging` currently does nothing at all — no `log_callback` field yet exists on `Config` to actually receive the output, despite the flag existing to request it. Second, `Query::result_u64()` is implemented using the 32-bit `glGetQueryObjectiiv` internally, because the underlying `meta-gl` layer does not expose the real 64-bit query variant — meaning it is suitable for frame counts, but not for a high-resolution GPU timer query, despite its 64-bit-suggesting name. Both are stated plainly in the project's own tracking rather than silently left for a reader to discover.

59.9 The public history was rewritten without changing the trees

On 2026-08-09, both libraries' public histories were replayed to remove co-author trailers. The current easy-gl and meta-gl tree hashes are byte-identical to the content CNA had already accepted, but the old commit objects are unreachable from any branch. Archive tags were

repointed while retaining annotations that name old targets, and the rewritten commits are unsigned even though the pre-rewrite objects had valid signatures.

For this book, the durable evidence is branch, tree hash, and audit date. The current public heads are easy-gl `0b46d35c` and meta-gl `571d3a62`; older SHAs from the integration campaign may exist only as dangling local objects and should not be offered as reader-resolvable citations. The rewrite changed provenance objects, not the implementation described in this chapter.

Chapter 60

free-direct and free-api

CNA’s FREEDIRECT renderer sits on two compatibility libraries. **free-direct** reimplements a narrow DirectX 3-era DirectDraw, DirectSound, and DirectPlay surface; **free-api** supplies the Win32-shaped types and services below it using SDL3. Neither is a general emulator, and CNA’s native DIRECTX1 through DIRECTX7 renderers do not use either library.

60.1 A game-driven boundary

Free-direct’s original scope is derived from calls made by two games: *Speedy Blupi* through free-eggbert, and *Planet Blupi*. A third game cannot assume that a familiar DirectX symbol exists merely because the header name does. It needs its own call-site audit and an explicit decision about widening the library.

The audited public headers make that boundary measurable. Across **ddraw.h**, **dplay.h**, and **dsound.h**, their documentation contains 95 explicit **Status:** annotation lines. Count that tagged spelling, not every bare status word: **dsound.h**’s opening overview repeats twelve of the later classifications without the tag. The tagged lines cover functions and interfaces but also some supporting structures and constant groups, so 95 is a documentation-marker census, not a public-method count:

Table 60.1: Free-direct’s public-header status-annotation census.

Header	Implemented	Partial	Stub
ddraw.h	24	12	7
dplay.h	12	7	7
dsound.h	20	4	2
Total	56	23	16

The table is stronger than a narrative “mostly implemented” claim because it can be regenerated from the headers, provided the unit is not mislabeled as methods. A status still needs semantic reading: DirectDraw’s **IsLost()** and **Restore()** are honest stubs returning **DD_OK**, whereas a partial operation may implement the game-used path and reject other flags.

60.2 Three DirectX subsystems, no Direct3D

DirectDraw owns CPU pixel buffers, primary and offscreen surfaces, clipping, source color keys, palettes, **Blt/BltFast**, **Lock/Unlock**, and SDL texture upload on **Flip**. Surface registration allows renderer invalidation. It is the deepest and most directly exercised part of CNA’s 2D compositor.

DirectSound opens a real SDL3 audio device and stream behind a shared, mutex-protected device. Static PCM buffers and volume are implemented; volume uses $10^{cB/2000}$. Pan is a sharper boundary than its helper name suggests: **SetPan()** clamps and stores the value, and a constant-power helper computes left/right gains for mono, but **applyGain()** then discards those gains because an SDL stream channel map cannot express them. Pan is therefore not audibly applied yet. The tests lock in success and no-crash behavior rather than samples. The subsystem also enforces a 64 MiB buffer ceiling and contains a deliberate **PCMWAVEFORMAT** versus **WAVEFORMATEX** layout correction.

DirectPlay implements sessions, players, unicast, broadcast, and host-side relay behind a transport interface. A default build has only in-process loopback. Real UDP peers and LAN discovery require **FREE_DIRECT_ENABLE_ENET**, which is OFF by default and adds ENet privately. Its packets are FreeDirect-to-FreeDirect, not Microsoft DirectPlay wire compatibility.

There is no Direct3D source or public interface at all. This library is “DirectX 3” only in the era-bundle sense; its graphics path is DirectDraw 2D.

60.3 The two libraries form a one-way dependency

Free-direct adds its sibling `../free-api` when no **free-api** target already exists, disables free-api’s tests, and links `free-api::free-api` publicly. Free-api does not depend on free-direct. It supplies roughly 80 `extern "C" WINAPI` declarations across flat Win32-named headers and a real `windows.h` umbrella.

The public types deliberately do not expose SDL. Internally an `HWND` represents an `SDL_Window*` cast through `void*`; message translation, GDI-lite buffers, files and paths, timers, MCI/MIDI playback, joystick input, string resources, and diagnostics all translate to SDL-backed machinery. MIDI uses a shared audio stream, a mixing thread, and vendored TinySound-Font/TinyMidiLoader. Free-api itself links SDL3, `SDL3_image`, and `SDL3_mixer`, but neither ENet nor sharp-runtime.

Only three bridge functions are explicitly designed for free-direct: **FreeApiCreateSurfaceDC**, **FreeApiDestroySurfaceDC**, and **FreeApiSetWindowFullscreen**. Keeping those in `free_api_bridge.h` names the scope exception instead of letting free-direct reach arbitrarily into private implementation.

60.4 CNA is the odd consumer

Free-eggbert and Planet Blupi add free-api first and then free-direct. CNA adds only free-direct when `CNA_GRAPHICS_RENDERER=FREEDIRECT`; the public link edge pulls free-api into the final closure transitively. CNA is therefore a real third consumer of free-api even though its own build file never calls `add_subdirectory` for it.

CNA’s renderer is not a thin adapter. Its 1,374-line implementation creates DirectDraw surfaces, performs locked pixel write/read/clear operations, composites quads on the CPU, and maintains a shadow-backbuffer resize transaction with failure injection. Twenty-one registered CNA test binaries now exercise its renderer-specific and shared graphics contracts under SDL’s dummy video driver, including shadow-buffer pixels, resize transactions, sampling, 3D refusal, resource-transfer and render-pass ordering boundaries.

60.5 The `fopen` macro is a real containment cost

Free-api's `windows.h` defines `fopen` as `free_api_fopen`. Because free-direct's `ddraw.h` includes that umbrella, exposing `ddraw.h` from a CNA public header would rewrite every later `fopen(...)` token in the translation unit. CNA contains the hazard with `pimpl`: `FreeDirectRenderer.hpp` does not include `ddraw.h`; the implementation file owns all `DirectDraw` types.

```

1 // Public header: no ddraw.h, no free-api windows.h, no fopen macro.
2 class FreeDirectRenderer final : public IGraphicsRenderer {
3     struct Impl;
4     std::unique_ptr<Impl> impl_;
5 };
6
7 // Implementation file only:
8 #include <ddraw.h>
9 struct FreeDirectRenderer::Impl { /* IDirectDraw state */ };

```

This is more than “keep headers small.” It is a preprocessor quarantine around a second-level dependency whose public compatibility contract intentionally includes global Win32 macros.

60.6 FREEDIRECT is not DIRECTX3

CNA's `DIRECTX1`, `DIRECTX2`, `DIRECTX3`, `DIRECTX5`, `DIRECTX6`, and `DIRECTX7` renderers use the real Microsoft/MinGW/Wine `DirectDraw` SDK and link `ddraw/dxguid`; they deliberately do not use free-direct. The SDL-backed renderer was itself named `DIRECTX3` until 2026-08-04, when it became `FREEDIRECT` precisely to remove this ambiguity.

Selector spelling therefore determines two different dependency graphs:

```

1 FREEDIRECT -> CNA FreeDirectRenderer -> free-direct -> free-api ->
  SDL3
2 DIRECTX3   -> CNA DirectX3Renderer  -> MinGW/Wine ddraw + dxguid

```

The same public `ddraw.h` spelling can resolve to different providers under those graphs, which is why source-path and target evidence matter more than API-name resemblance.

60.7 Verification boundaries

Free-direct has nine default CTest registrations, ten with ENet, using a hand-rolled `CHECK` mechanism. Compile-only header tests and a shell hygiene gate prevent SDL or ENet identifiers from leaking into public headers. Tests default OFF. The `DirectPlay` suite also records that enabling ENet makes 29 of 61 loopback-shaped checks fail by design because the timing model changes; a transport toggle is not a free strengthening of the same oracle.

Free-api has 29 CTest registrations: 20 C++ programs and nine CMake-level tests, including negative controls for string-table extraction and public-surface scans. They are ON standalone and OFF when embedded. The suite exercises `SDL-to-WM_*` translation, message filtering, timers, GDI bounds, resource strings, MCI/MIDI, joystick, and quiet logging.

Test volume did not prevent a cautionary 2026-07-18 sequence. A nine-finding re-audit was immediately followed by two regressions introduced by its changes: a startup abort from reentrant static initialization and real memory growth from bitmap pooling. Users found both, not

the 29-test suite. The episode is a useful reminder that an audit patch changes the system under audit; regression risk belongs inside the review, not after it.

The correct scope statement is thus precise. These libraries reproduce the calls their games and CNA currently need, with unusually visible status metadata and meaningful tests. They are not Wine, a complete Win32 SDK, hardware-accurate DirectX emulation, or a general foundation for arbitrary DirectX 3 software.

Part X

Cross-Platform Engineering

Chapter 61

The Cross-Platform Contract

At `v0.1.0-alpha.1`, “platform” no longer means simply the target OS or “the SDL3 layer.” CNA has five related axes: target OS/toolchain; the selected `CNA::Platform::IPlatform` implementation; the compiled graphics-renderer set and active family; the selected audio implementation; and the XNA-shaped framework surface used by the game. They are composed, not collapsed into one backend name.

61.1 The five-axis model

Axis	Selector or API	Responsibility
Target	CMake toolchain / OS	Object format, native SDKs, architecture and host admission.
Platform implementation	<code>CNA_PLATFORM</code>	Windows, events, input, time, system services, cameras, GL contexts, Vulkan surfaces and CPU-surface presentation.
Graphics	<code>CNA_GRAPHICS_RENDERER(S)</code> and <code>GraphicsRendererSelection</code>	How pixels are produced; a compiled set, configured default, runtime selection and first-device latch.
Audio implementation	<code>CNA_AUDIO_PLATFORM</code>	Playback/capture infrastructure, independent of both the window platform and graphics family.
Game/framework	<code>Microsoft::Xna::Framework</code> and optional CNAEXT	Stable application-facing shape above the three native-service axes.

The separation permits meaningful combinations: HEADLESS platform with NULL audio and HEADLESS graphics for a server; SDL3 platform with NULL audio for a deterministic graphics test; TERMINAL platform with SOFTWARE graphics for a POSIX terminal application; or SDL2 platform/audio while retaining an otherwise current framework. It does not mean every Cartesian product is buildable. Capability and toolchain gates reject combinations that lack a native window, presentation route, SDK, ABI, or common target.

61.2 The platform contract is implemented

`modules/platform` now owns `CNA::Platform`. Its root `IPlatform` is created once by `PlatformFactory` and is not swapped while the process runs. The interface groups:

- refcounted acquisition of video, audio, gamepad, haptic, and sensor subsystems;
- window creation/adoption and native-window identity;
- batched event polling into caller-owned storage;
- monotonic counters, ticks, delay and system services;

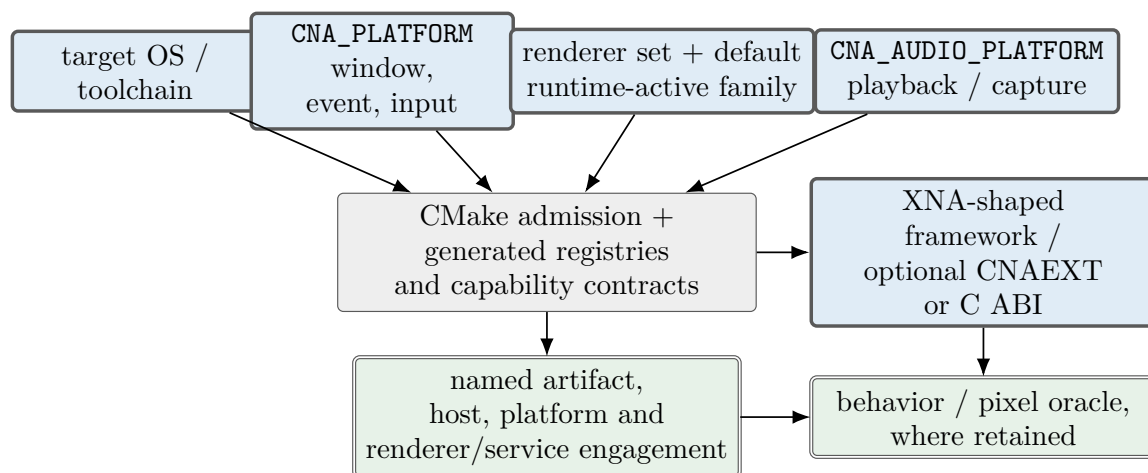


Figure 61.1: Target, platform, graphics, audio and framework are separate axes. Verification belongs to the exact composed route; source or configuration admission alone is not execution evidence.

- keyboard, mouse, text, gamepad, joystick, haptics, sensors and camera services;
- optional GL-context, Vulkan-surface and CPU-surface-presentation interfaces.

This is not a nominal wrapper around SDL types. Public platform headers do not expose SDL; the platform target links the selected toolkit privately on desktop, and a source ratchet rejects new direct SDL ownership outside the admitted boundary. Hot-path lint separately rejects platform calls inside per-pixel, vertex, fragment, sample and event loops. Capability queries are intended to be read once and cached, not dispatched repeatedly in a render loop.

The lifecycle also preserves host ownership. CNA production code acquires individual sub-systems lazily; it does not seize the application's global SDL initialization/shutdown policy. Unpaired release during partial cleanup is tolerated, while successful acquisitions are normally balanced.

61.3 Implemented, host-limited, and reserved choices

SDL3 Default implementation. Owns current SDL3 window, event, input and system service adaptation. Mobile entryptoint glue is an explicit public-build exception.

SDL2 Independent implementation selected and linked only when requested; it is not SDL3 operating in a compatibility mode.

HEADLESS No native windowing dependency. It supplies deterministic platform behavior and remains compiled as a conformance reference even when another implementation is selected.

TERMINAL POSIX `termios/poll` implementation with terminal input and a CPU surface presenter. It is implemented on non-Windows targets only and is not offered on Windows. Compatible renderer identities at the tag are SOFTWARE, SKIA, BLEND2D, PORTABLEGL, HEADLESS, and STUB.

SDL12, WIN32, and EMSCRIPTEN are reserved but unimplemented platform identifiers. On Windows, TERMINAL joins the refused set because no Windows console implementation exists. Selecting any reserved name is a hard configure error naming the implemented alternatives;

there is no silent SDL3 fallback. Targeting Emscripten with an SDL platform is distinct from a future native `CNA_PLATFORM=EMSCRIPTEN` implementation.

61.4 Platform capabilities and renderer needs

A renderer descriptor declares requirements rather than assuming every platform supplies an SDL window. Windowed GPU routes may need a native window system, a GL context or a Vulkan surface. CPU rasterizers may instead hand a finished surface to `IPlatformSurfacePresenter`. `HEADLESS` and `STUB` need neither. The registry/comboination gates evaluate these requirements before factory invocation, so a refused pair is configuration evidence rather than a runtime null dereference.

Runtime renderer selection does not relax those rules. Every family in `CNA_GRAPHICS_RENDERERS` must be buildable for the selected target and platform. Once the first device latches an active renderer, its windows, swapchain or CPU surface and every graphics resource belong to that choice. Fallback may try another compiled descriptor during initial creation only when the application enabled it; it cannot migrate a live resource graph.

61.5 Audio remains orthogonal

The implemented `CNA_AUDIO_PLATFORM` values are `SDL3`, `SDL2` and `NULL`. `OPENAL`, `WASAPI` and `ALSA` are reserved and fail configuration. The axis is deliberately independent: a terminal or headless program may still use `SDL3` audio, and an `SDL3`-windowed program may choose `NULL`. Chapter 45 describes what this changes in the sound route and what remains a shared XNA-shaped contract.

61.6 Target taxonomy is still useful, but different

CNA retains target vocabulary that distinguishes Desktop, Android, iOS and Web, with desktop OS values for Windows, Linux, macOS and Other. Those values answer where the binary is targeted. They do not answer which `IPlatform`, renderer, or audio implementation was composed. For example, Linux is compatible with `SDL3`, `SDL2`, `HEADLESS` or `TERMINAL` platform implementations; “Linux renderer” would still be underspecified.

61.7 A platform claim is an evidence vector

The useful progression remains:

1. a target/platform/renderer/audio source path exists;
2. configuration admits the exact combination;
3. it compiles and links;
4. the artifact executes on the named native, emulated or translated host;
5. the selected platform and intended renderer/service engage; and
6. behavior or pixels meet an identified oracle.

Later steps do not follow from earlier ones. A renderer descriptor may compile while its driver is unavailable; a PE artifact may execute without loading the intended DXVK DLL; a browser test may prove DOM state without compositor pixels; a platform conformance test

can use HEADLESS without exercising SDL. This edition therefore reports each coordinate and preserves unsupported, blocked, and not-attempted states separately.

At the tag, Linux x86_64 has the strongest self-hosted evidence. Windows campaigns largely use MinGW plus Wine/DXVK/vkd3d; browser campaigns use Emscripten and Chromium; Android evidence is emulator-scoped; and Apple routes require their native toolchains and hosts. The new platform architecture makes these routes cleaner to compose and test, but does not retroactively promote historical compile evidence into hardware or oracle evidence.

Chapter 62

Windows I: Cross-Compiling from Linux

CNA’s evidenced Windows engineering loop starts on Linux. CMake selects a MinGW target, builds target-triple-specific SDL dependencies, links a PE executable, stages its runtime DLLs, and teaches CTest to invoke that executable through Wine. Each step has an independent failure mode; a successful compiler probe is not yet a runnable Windows program.

62.1 Two MinGW toolchains

`cmake/toolchains/mingw-w64.cmake` targets Windows `x86_64` when the matching compiler exists and otherwise falls back to the `i686` triple. It sets C, C++, and resource compilers and constrains CMake package, library, and header searches to the cross root while allowing build-host programs to remain native.

```
1 cmake -B build-windows \  
2   -DCMAKE_TOOLCHAIN_FILE=cmake/toolchains/mingw-w64.cmake \  
3   -DCNA_GRAPHICS_RENDERER=SDL_RENDERER \  
4 cmake --build build-windows
```

The fallback is not a fully self-consistent 32-bit identity: the file leaves `CMAKE_SYSTEM_PROCESSOR` at `x86_64` even after selecting the `i686` compiler and search root. It can rescue compiler discovery, but it must not be treated as equivalent to the dedicated `i686` lane when processor-keyed caches or ABI gates matter.

The separate `mingw-w64-i686.cmake` is not merely a convenience alias. GLIDE’s native ABI represents a window handle as a 32-bit `FxU32`, and the available Glide DLL is `x86`. That toolchain fixes the processor and all compiler names to `i686`. As the final section shows, it can validate the renderer boundary but cannot link the whole framework.

There is no CNA-owned Android, Emscripten, or Apple toolchain file. Those paths use the NDK or `emsdk` toolchain externally; Apple requires its native environment. The two checked-in files are specifically the Windows-from-Linux contract.

62.2 Vendored SDL is built for the target, not reused from the host

Before renderer selection, CNA unconditionally calls `cna_configure_vendored_sdl()`. Its helper performs a nested configure, build, and install of `SDL3`, `SDL3_image`, and `SDL3_mixer`, forwarding the parent’s toolchain. A MinGW build therefore receives Windows libraries and DLLs rather than accidentally linking the Linux host’s SDL.

The cache directory is keyed by system and processor:

```

1 .sdl-prebuilt-Linux-x86_64/
2 .sdl-prebuilt-Windows-x86_64/
3 .sdl-prebuilt-Android-aarch64/      # expected by one demo, absent at
   audit
4 .sdl-prebuilt-emscripten/           # absent at audit

```

The audited machine contains the first two only. The Windows cache records the MinGW toolchain and GNU import libraries such as `libSDL3.dll.a`; it has no MSVC `.lib` or `.pdb`. This is direct evidence of cross-compilation, not native Visual Studio output.

Configure-time dependency builds make first configuration heavier and more stateful, but they also keep the target triple explicit. A stale cache from another ABI should not be renamed into place; it must be regenerated under the intended toolchain.

62.3 Runtime staging is part of a successful build

A PE executable that links against an import library still needs the corresponding DLL at runtime. CNA's post-build helpers copy `SDL3`, `SDL3_image`, and `SDL3_mixer` beside Windows targets. MinGW test targets link `libgcc` and `libstdc++` statically, then stage `libwinpthread-1.dll`. Application targets with large RTTI graphs may use the dynamic C++ runtime; a separate helper locates and copies `libgcc_s_seh-1.dll`, `libstdc++-6.dll`, and the threading DLL.

Each helper asks the selected compiler where its runtime file lives and falls back to the compiler's bin directory where appropriate. Failure is a warning that the executable may not run on a clean prefix, rather than a link failure on the build host. Verification must therefore inspect or execute the staged directory, not stop at "target built."

Renderer-specific dependencies add another layer. `DIRECTX8` needs a `D8VK d3d8.dll.a`; `Direct3D` renderers need the corresponding Windows SDK import libraries; `GLIDE` needs an external `glide3x.dll` only at execution. These are not supplied by the generic MinGW runtime helper.

62.4 CTest must know that PE files are not host executables

GoogleTest discovery in `PRE_TEST` mode executes `CnaTests.exe` to enumerate cases. Without an emulator property, Linux attempts to run the PE file natively and discovery fails before any test. CNA sets CMake's `CROSSCOMPILING_EMULATOR` property on `CnaTests` for `DirectX1/2/3/5/6/7/8/9/10/11/12` and `Direct2D`, selecting the matching Wine wrapper.

Discovery and device-free unit tests set each wrapper's documented skip-gate variable. A bare `-gtest_list_tests` call creates no `Direct3D` device and cannot emit a `DXVK` marker; an engagement gate would falsely reject it. Renderer smoke examples instead put the wrapper in the registered command because those executables really do create the device and should prove the runtime engaged.

```

1 # Conceptual CMake behavior
2 set_target_properties(CnaTests PROPERTIES
3   CROSSCOMPILING_EMULATOR
4   "env;CNA_D3D9_SKIP_DXVK_GATE=1;scripts/run-wine-dxvk9.sh")

```

The strict public-API compile check has its own emulator wiring. These two sites are the bridge that makes ordinary `ctest` understand the cross-built artifacts; per-example commands remain explicit.

62.5 Native MSVC is a separate, unproven lane

Two Windows workflows target `windows-latest`, activate MSVC, use Ninja, and include tests. Both are manual `workflow_dispatch` workflows. The repository cites no Actions run ID for them, and CNA production code has essentially no hand-written MSVC conditional logic: its only `_MSC_VER` sites are generated Sokol shader output.

That is not proof that MSVC fails. It means the evidence in this repository establishes MinGW-plus-Wine, not a native Microsoft toolchain or genuine Windows execution. Workflow YAML is executable intent; a recorded run is execution evidence.

62.6 The i686 wall

The 32-bit GLIDE lane reaches farther than a syntax sketch. The renderer passes `-fsyntax-only`, and a fake DLL client verifies all 39 expected exports under Wine. The full CNA executable cannot link because sharp-runtime's `Int128` and `Decimal` require GCC/Clang `__int128`, which `i686-w64-mingw32-g++` does not provide. Sharp-runtime explicitly declines to hand-roll 128-bit arithmetic solely for MSVC/32-bit coverage.

GLIDE therefore has source, configuration, partial cross-compile, and an ABI probe, but no full artifact. The renderer's own syntax can be correct while an unconditional foundation dependency makes the product architecture unreachable. This is exactly why platform status belongs to a layered evidence vector rather than a single build-support checkbox.

Chapter 63

Windows II: Wine and Engagement Gates

Cross-compilation proves that a Windows-shaped program can be produced. CNA's Wine apparatus asks the harder question: when that program runs, did the intended DirectX implementation actually handle it? The answer cannot be inferred from exit code alone, because Wine can fall back to another DLL stack and still draw a plausible image.

63.1 Six prefixes isolate incompatible runtime stacks

The audited machine carries six named Wine prefixes:

- `~/wine` the Direct2D Wine path;
- `~/wine-cna-dx1` shared by DirectX1/2/3/5/6/7 and Wine's builtin DirectDraw;
- `~/wine-cna-dx8` the D8VK path;
- `~/wine-cna-d3d10` the Direct3D 10 path;
- `~/wine-cna-d3d11` shared by D3D9 and D3D11 DXVK lanes; and
- `~/wine-cna-d3d12` the vkd3d-proton lane.

Direct2D's fresh-prefix suite also creates throwaway directories. Other special-purpose prefixes hold Microsoft's `d3dcompiler_47.dll` and a real 32-bit XNA 4.0 installation for the D3D9 oracle. Counting only three because three roles are memorable would lose the actual runtime isolation on disk.

A prefix is mutable system state: DLL overrides, bitness, registry entries, installed runtimes, and first-run initialization all live there. Reusing a personal prefix for an automated experiment can change it irreversibly. Both Proton launchers hard-refuse `~/wine`; that guard was added after a Proton/Wine version mismatch half-upgraded a personal prefix and hung for 9.5 hours installing Mono. Bootstrap steps are now timeout-bounded.

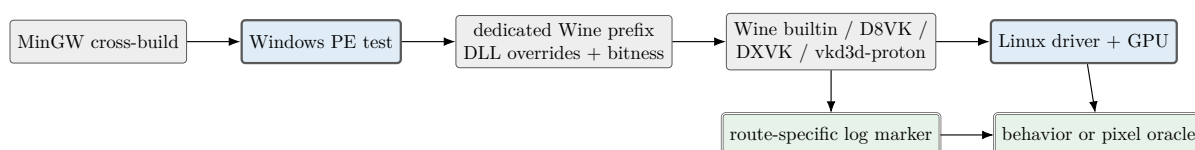


Figure 63.1: The Windows-on-Linux engagement chain. Exit status and plausible pixels are not enough when Wine can silently select a different DLL route.

63.2 Which translation runtime owns each renderer

The renderer name does not uniquely identify the runtime implementation under Linux:

Table 63.1: Windows renderer runtime paths used by the Linux verification loop.

Renderer	Runtime path
DIRECTX1/2/3/5/6/7	Wine builtin <code>ddraw.dll</code> ; DXVK does not translate DirectDraw.
DIRECTX8	D8VK <code>d3d8</code> ; <code>dxgi</code> deliberately remains builtin.
DIRECTX9	DXVK <code>d3d9</code> .
DIRECTX10	Wine <code>d3d10/d3d10_1</code> front, DXVK <code>d3d10core+dxgi</code> .
DIRECTX11	DXVK <code>d3d11+dxgi</code> .
DIRECTX12	<code>vkd3d-proton d3d12/d3d12core</code> .
DIRECT2D	Wine builtin <code>d2d1</code> ; WineD3D or a Proton/DXVK experiment below it.
GDI	Wine’s <code>gdi32</code> ; no dedicated wrapper script.
GLIDE	External x86 <code>glide3x.dll</code> , loaded dynamically.

The same CNA test executable may therefore traverse Wine builtin code, a DXVK component, `vkd3d-proton`, or an external compatibility DLL. A generic “ran under Wine” label hides the very implementation the test is meant to validate.

63.3 An engagement gate turns logs into a verdict

Each DirectDraw launcher greps its captured output for a `:ddraw:` trace. DX8/9/10/11 launchers require a DXVK: marker. D3D12 requires `vkd3d-proton - applicationVersion:.` If the expected marker is absent, the wrapper exits with status 3 and names the likely silent fallback.

```

1  scripts/run-wine-dxvk.sh build-windows/CnaD3D11Tests.exe
2
3  # Device-free discovery cannot emit a marker, so it uses an explicit
4  # narrowly named bypass. Normal smoke/pixel runs must not set this.
5  CNA_D3D11_SKIP_DXVK_GATE=1 \
6    scripts/run-wine-dxvk.sh build-windows/CnaD3D11Tests.exe \
7    --gtest_list_tests

```

Some wrappers also expose an allow-WineD3D diagnostic switch. That is a requested change of oracle, not an equivalent passing configuration. The log gate’s transferable design is simple: identify an implementation-specific positive marker, capture the entire child run, and fail closed when the marker is absent.

Direct2D is the important exception. Its launcher has no `d2d1` engagement marker, and Direct2D CTest registrations disable an inherited DXVK gate. A mis-provisioned prefix could therefore produce a result without proving which Direct2D path ran. GDI has no wrapper at all. The engagement method is strong where implemented, not a universal property of every Windows identity.

63.4 Environment-specific workarounds are part of the evidence

The DX8 lane can force `DXVK_FILTER_DEVICE_NAME=llvmpipe` because the RADV path lost its surface on a second `Present`. Direct3D 10 needs the real `DISPLAY=:0`; its DXGI EDID path crashes under Xvfb. Direct2D has an isolated-Xvfb runner and a fresh-prefix suite. These controls describe the environment in which a result holds. They should not be silently generalized into renderer behavior on genuine Windows.

The hardware-backed DXVK/vkd3d results use an AMD Radeon 780M with Mesa RADV. Logs explicitly skip llvmpipe unless a lane deliberately selects it. Naming the adapter prevents a Vulkan software implementation from being mistaken for hardware proof.

63.5 D3D12 and Proton require matched ownership

Plain Wine's DXGI and a separately overridden vkd3d-proton `d3d12.dll` did not share the command-queue ABI needed by swap-chain creation. The successful lane launches end-to-end through Proton, keeping its Wine, DXGI, and vkd3d pieces matched. It reached a real `CreateSwapChainForHwnd` success on 2026-07-14.

The two Proton scripts do not yet share the same version discipline. Direct2D consults a pin file preferring Proton 9.0 Beta then 8.0 and rejects Experimental unless allowed; the D3D12 script still defaults to `Proton - Experimental`. That moving target is an unresolved reproducibility asymmetry. The Direct2D Proton result also used a fabricated Steam tree and has no surviving prefix, whereas the D3D12 prefix remains.

63.6 The D3D9 oracle controls both sides

One tool runs Microsoft's unmodified stock-effect `.fx` sources through genuine `d3dcompiler_47.dll` under Wine. Sixty-one of 66 resulting variants match FNA's shipped Microsoft bytecode exactly; the five `PixelLighting` vertex differences were narrowed to compiler version rather than optimization flags. Wine's builtin compiler is not an adequate substitute: it fails a real alpha-test ternary and once expanded a roughly 2,160-byte skinning shader to about 1.4 MiB because relative addressing was unavailable.

The pixel oracle renders one declarative scene through two independent programs: real XNA 4.0 inside its dedicated 32-bit prefix and CNA's MinGW D3D9 renderer. Both run through the same DXVK-verified path so the comparison does not accidentally measure two driver stacks. A mutation-tested image comparator proves that changed pixels are actually detected at the configured tolerance.

This loop also exposed a genuine D3D9 restriction. `A8B8G8R8` is legal for textures but not for a primary swap-chain back buffer. CNA substitutes `A8R8G8B8` only at back-buffer creation and retains the natural format for ordinary textures; readback handles both byte orders.

63.7 What translated execution cannot prove

DXVK synthesizes legacy `D3DCAPS9` fields from a modern Vulkan device. CNA's logic can be proven to query and interpret those fields correctly, and its output can match XNA through the same DXVK path, but the returned capability numbers are not observations from period Windows drivers. D3D11 and D3D12 query modern surfaces closer to what native applications use, yet they still execute through translation rather than genuine Windows.

The remaining high-value check requires human hardware: native Windows, authentic device-loss events, fullscreen transitions, real XNA 4.0, and a rerun of the oracle outside Wine. Until then, the strongest precise claim is “pixel-verified against XNA through the same engaged DXVK path on this recorded GPU,” not “validated on Windows hardware.” The engagement gates make the first claim unusually strong by ensuring it is not secretly a result from the wrong runtime.

Chapter 64

Web I: C++23 to WebAssembly

CNA does not have a separate JavaScript implementation. Its Web builds send the same C++23 framework through Emscripten and link it with SDL’s browser port. Configuration admits 24 renderer identities on Emscripten, but five are the web-focused evidence set examined here: `WEBGL1`, `WEBGL2`, `CANVAS`, `HTML_DOM`, and `SVG_DOM`. One configure still packages exactly one selected identity into a WebAssembly application. That compact description hides three independent questions: can the source be translated, can the program survive the browser main-loop hand-off, and has the selected renderer actually been exercised? This chapter answers only the first. Chapters 65 and 66 answer the other two.

64.1 The toolchain changes the execution contract

The root build selects `WEBGL2` by default when `EMSCRIPTEN` is true. This is a public renderer identity backed by the EasyGL family, just as `WEBGL1` selects another configuration of that family. `HTML_DOM`, `SVG_DOM`, and `CANVAS` select distinct implementations. The choice is still exclusive: one configure produces one renderer, not a universal Web binary containing all five.

Exception handling is the first project-wide adjustment. CNA uses C++ exceptions in ordinary control and error paths, so the Web build enables real WebAssembly exceptions with `-fwasm-exceptions`. This is not merely a compiler-acceptance flag. It affects what the generated cleanup machinery catches, and therefore participates directly in the main-loop lifetime defect studied in Chapter 65.

The build remains single-threaded. No Web target enables Emscripten pthread support, and the tree has no corresponding cross-origin isolation setup. One `CnaTests` configuration uses `ASYNCIFY=1` for asynchronous WebSocket/ENet behavior; that transformation is not evidence of a worker-thread architecture. A port that needs concurrent game, networking, or asset work would have to define that architecture and its hosting headers explicitly.

64.2 Assets must cross the virtual-filesystem boundary

A native executable can discover files already present beside its working directory. A browser cannot. Emscripten exposes a virtual filesystem, and a file exists there only if the page creates it or the linker packages it.

At the audited revision, exactly two build sites use `-preload-file`: the 2D graphics demo and the sound demo. There is no project-wide asset manifest, no `-embed-file` policy, and no custom shell file that silently fills the gap. Consequently, successful translation of a target proves nothing about the availability of its content at run time. Each Web application must account for every filename that can reach `TitleContainer` or `ContentManager`.

The same distinction applies to writes. SDL’s preference path resolves inside Emscripten’s

in-memory filesystem, under `/libSDL`. CNA does not mount IDBFS, call `FS.syncfs`, or bridge its storage abstractions to `localStorage`. Saved data therefore survives within the current page instance but not a reload. `StorageDevice` and `GamerServices` persistence on a native platform must be described as volatile on Web until a durable browser store is deliberately connected.

64.3 Networking is a different topology, not a transparent port

Emscripten's socket shim does not report a usable ephemeral bound port, so CNA's ENet host path requests fixed port 61191 instead of `ENET_PORT_ANY`. That host role is meaningful for a Node relay, not an ordinary browser tab: the browser-side client path replaces any listening host with an outbound-only ENet client before connecting. The entire UDP-broadcast discovery implementation is excluded under `__EMSCRIPTEN__`. A Web build can therefore preserve `SystemLink` message transport without preserving native hosting and LAN-discovery topology.

The CnaTests-only `ASYNCIFY=1` setting lets synchronous-looking polling yield to the JS event loop during `WebSocket`/`ENet` tests. It does not add listening sockets, make discovery available, or propagate to ordinary demos. Build evidence for the network module must therefore name the role and host, not merely say that ENet linked.

64.4 What a successful build establishes

A completed Emscripten link is meaningful evidence. It establishes that the selected source closure is accepted by the Web toolchain, that its C++23 surface is compatible with the chosen SDK, and that the requested renderer's symbols can be resolved. It does not establish any of the following:

- that all runtime assets were packaged;
- that save data survives a page reload;
- that a frame was reached after Emscripten took control;
- that generated WebGL shaders compiled in a browser;
- that the page displayed the selected renderer rather than merely loading its module.

This boundary matters because a bare-Node launch is a poor substitute for a browser. DOM renderers need `window` and `document`; SDL-backed renderers need browser host services; WebGL needs a real graphics context. Errors such as `window is not defined` or an SDL initialization failure say that the wrong host was used, not that the renderer's drawing semantics were tested.

64.5 A reproducible build record

Evidence for a Web build should record at least the CNA commit, emsdk version, renderer identity, configure options, named target, and produced `.html`, `.js`, and `.wasm` artifacts. It should also list every preload mapping. Without the mapping, two apparently identical builds may have different runtime content.

For testing, the record should say whether the target was merely linked, launched under Node, or served over HTTP to a named browser. These are ascending evidence levels, not synonyms. The current HTML-DOM workflow, for example, pins emsdk 6.0.3 and serves six pages to

Playwright’s Chromium. That is a browser execution result; a successful build of the other identities is not equivalent to it.

64.6 The portability lesson

WebAssembly preserves much of the language and loses much of the process model. The hard porting work is therefore at the boundaries: ownership across a non-returning host call, assets crossing into a virtual filesystem, persistence crossing into a browser store, and a verdict crossing out of a process whose exit status the harness cannot observe. Treating those boundaries as explicit interfaces keeps “Web build” from becoming a claim broader than the evidence behind it.

Chapter 65

Web II: the Main-Loop Problem

A native `Game::Run()` owns a straightforward loop: construct the game, enter the loop, return when it ends, and then destroy the object. Emscripten inverts that control. The program registers a callback and yields to JavaScript so the browser can schedule frames. CNA once treated that yield like an ordinary returning call, creating a lifetime bug before the first frame was drawn.

65.1 The throw that is not a C++ exception

The Web path calls `emscripten_set_main_loop` with `simulateInfiniteLoop=1`. Emscripten implements the apparent non-return by emitting a raw JavaScript `throw 'unwind'`. With WebAssembly exception handling enabled by `-fwasm-exceptions`, CNA's generated C++ cleanup path intercepted that foreign throw and unwound the current stack.

That interaction made this apparently conventional entry point unsafe:

```
int main() {
    MyGame game;
    game.Run();
}
```

The `unwind` destroyed `game`, while the registered callback still retained pointers into it, including the graphics-device manager. The page could build and start successfully, yet the first scheduled frame dereferenced dead state. This is a useful counterexample to the idea that compilation plus module startup proves a viable Web port.

65.2 Reducing the failure before explaining it

The first hypothesis blamed multiple inheritance and pointer adjustment in the graphics manager. It was plausible: a stale-looking address crossed an interface boundary, and Web builds

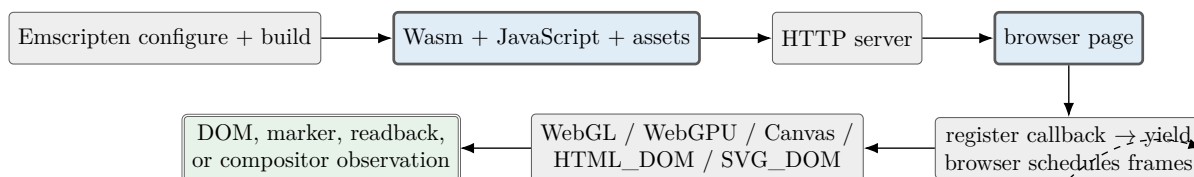


Figure 65.1: Browser/WebAssembly execution. Loading a page, invoking a frame, engaging a renderer, and verifying compositor pixels are separate milestones.

expose layouts differently from familiar native debuggers. Focused reproductions under AddressSanitizer, UndefinedBehaviorSanitizer, vptr checks, and Emscripten `SAFE_HEAP` did not support it.

An isolated lifetime reproduction did. Destruction occurred during the control hand-off, before frame one, and the callback later observed the destroyed object. The decisive evidence was temporal rather than structural: constructor, hand-off, destructor, callback. Recording that sequence ruled out a renderer-specific theory and located the defect in the hosting contract shared by every Web renderer.

65.3 The repair and its boundary

The repaired examples allocate the game for browser lifetime rather than automatic scope:

```
int main() {
    auto* game = new MyGame();
    game->Run();
}
```

Twelve example entry points were changed in commit `b6686bfba`. Intentional process- lifetime ownership is appropriate here because the browser loop does not return to a normal destruction point. A richer host could instead hold the object in explicitly managed state and delete it after cancelling the loop, but stack lifetime cannot cross this hand-off.

The repair was not mechanically complete. The renderer benchmark still constructs its game on the stack at `modules/graphics/examples/graphics_renderer_benchmark.cpp:274`. It is an Emscripten-only target and remains a regression-shaped exception. This matters as much as the twelve repaired sites: a search-based migration must end with a search for survivors, not with a count of edited files.

65.4 The Web loop is not the native loop

Fixing ownership prevents use-after-destruction, but it does not make the two schedulers semantically identical. At the audited revision, the Web path has five observable differences:

1. `SuppressDraw` is ignored;
2. variable time-step mode is ignored and the loop always advances as fixed-step;
3. `IsRunningSlowly` remains false;
4. `EndRun` and `AfterLoop` are never called;
5. elapsed time is clamped to a hard-coded 250 ms rather than the configured `MaxElapsedTime`.

The clock source differs too: the Web route uses `SDL_GetTicks`, whereas the native route uses a performance counter and delay. None of these differences necessarily prevents a demo from drawing. Each can change game behavior, tests, shutdown, or timing-sensitive network code, so they belong in the compatibility contract rather than in an implementation footnote.

65.5 How to test a host-owned loop

A useful regression test cannot stop at “the callback was registered.” It must keep the page alive until at least one frame, assert that the game object is still alive inside that frame,

and communicate the result through a browser-visible channel. It should separately exercise cancellation and shutdown if those semantics are claimed.

The minimum diagnostic trace is equally small: construction, registration, apparent unwind, first callback, and destruction. On a conventional process these events seem obvious; under a foreign scheduler, spelling them out is often the fastest route to the bug. The transferable rule is that a function which syntactically does not return may still run cleanup machinery. Ownership must follow the host's actual control flow, not the C++ surface illusion.

Chapter 66

Web III: Six Renderers, Four Kinds of Evidence

CNA's six web-focused renderer identities do not imply six equally verified implementations: WEBGL1, WEBGL2, CANVAS, HTML_DOM, SVG_DOM and PIXIJS. The tag contains four distinct kinds of evidence: translation, module startup, browser structure, and browser pixels. Keeping those levels separate produces a more useful map than a binary supported/unsupported table.

66.1 An evidence ladder

Build evidence The selected implementation compiled and linked to WebAssembly.

Host-start evidence A JavaScript host loaded the module far enough to execute startup.

Structural evidence A real browser observed renderer-specific DOM or SVG objects and invariants that another renderer could not trivially impersonate.

Pixel evidence A real browser compositor produced sampled colors at expected screen positions.

Higher rungs include more of the delivery chain but do not erase lower-level tests. A pixel sample may prove presentation while missing a malformed object tree; a structural assertion may prove the renderer identity while missing blending. The strongest suite combines both.

66.2 HTML_DOM: automated browser and pixel evidence

HTML-DOM has the only genuine Web workflow at the audited revision. The workflow pins emsdk 6.0.3, installs Playwright with Chromium, builds the renderer, and serves six pages over HTTP. The harness waits for a verdict on `window`, because a WebAssembly process exit code is not observable in the same way as a native child process.

The browser driver also samples the real compositor. It takes a screenshot, transfers it back into an in-page canvas, and calls `getImageData`. Four point samples across its smoke and pixel cases use bounded RGB tolerances. This route is important: inspecting the renderer-owned DOM alone would not prove what Chromium finally composed on screen.

A separate host-integration case rewrites the HTML response with Playwright `page.route()`. It verifies that CNA refuses to seize host-owned DOM and does not create its viewport. Earlier injection through `addInitScript` was measurably too late, so the test modifies the document before module bootstrap. The timing mechanism is part of the proof.

66.3 SVG_DOM: browser harness plus bundle CI

SVG-DOM has a browser driver and 34 checks across three pages. It verifies SVG namespace identity for the root and its flush-slot descendants, keeps the SDL canvas hidden, and performs a four-point scissor-order pixel check. Those namespace assertions are anti-spoofing evidence: an HTML or canvas renderer cannot satisfy them merely by producing a similar picture.

Its shell browser runner still has no direct CI/CTest caller. Alpha.1 does add an Emscripten multi-renderer workflow that compiles and links WEBGL2, Canvas, HTML-DOM and SVG-DOM into one Wasm bundle and checks four generated-registry entries. That job explicitly does not launch a browser. SVG-DOM therefore has a reproducible manual browser route and an automatic bundle/link route, not a continuous SVG compositor test.

66.4 WEBGL2 and WEBGL1: translation without browser execution

Both WebGL identities have reached Emscripten builds, but neither was run in a browser in the audited campaign. A bare-Node attempt stopped at `window is not defined`; that is host-environment evidence only. In particular, generated GLSL for WEBGL1 has not been compiled by an actual WebGL 1 implementation.

This leaves essential questions open: context creation, shader-version compatibility, buffer uploads, presentation, and context loss. The source contains context-loss handling, but source presence is not a recovery test. A browser test must deliberately lose and restore the context and then demand a fresh, visible frame.

66.5 CANVAS: startup failure before renderer evidence

The Canvas renderer also builds, but its bare-Node run fails during SDL initialization. Its manual browser checklist remains unchecked. The result is therefore build evidence, not browser execution evidence, and certainly not pixel evidence. The correct next test is not another Node invocation; it is an HTTP-served browser page with an explicit verdict and a small compositor sample.

66.6 PIXIJS: new source route, browser proof still required

The tag adds the Emscripten-only PIXIJS identity, pinned to PixiJS 7.4.2 with a recorded download hash. It adapts SpriteBatch, SpriteFont, scissor/blend state, textures and one `RenderTarget2D` to Pixi objects while refusing 3D, compiled/custom effects, MRT, cube and volume resources and queries. Neither the HTML-DOM workflow nor the four-renderer bundle workflow selects PIXIJS. Its tag evidence in this edition is source/configuration evidence; an HTTP-served browser test must still prove artifact resolution, object lifetime and compositor pixels.

66.7 The matrix

Identity	Builds	Browser run	Structure	Pixels
HTML_DOM	yes	automated	yes	four samples
SVG_DOM	yes	manual runner	yes	four samples
WEBGL2	yes	no	no	no
WEBGL1	yes	no	no	no
CANVAS	yes	no	no	no
PIXIJS	source route	no automated run	no	no

The table is deliberately asymmetric. “Builds” says nothing about frame one, and four samples say nothing about complete rendering parity. It nevertheless gives each claim a precise meaning and exposes the shortest path to stronger evidence.

66.8 A common browser-verification shape

All six identities can converge on one harness pattern: serve artifacts over loopback HTTP; wait for a named `window` verdict with a timeout; fail on console errors and page exceptions; assert a renderer-specific structural token; sample a few compositor pixels; and archive the page log plus screenshot. Renderer-specific tests then add shader compilation, context restoration, DOM ownership, or canvas behavior.

The central rule is simple: prove that the intended runtime engaged. A green build can be produced without a browser, a green page can be produced before a frame, and a plausible image can be produced by the wrong rendering route. Identity, execution, and visible output are three separate facts, so the evidence should contain all three.

Chapter 67

Android and Apple Platform Evidence

Platform support is not one fact. A source tree may contain platform branches, cross-compile, run tests in an emulator, package an application, draw a frame, and survive real hardware—or stop at any rung in between. Android retains recorded emulator evidence. Alpha.1 contains declared native macOS, iOS device-link and iOS simulator-launch routes, while tvOS remains outside the implemented host. Workflow source establishes those routes; without the tag’s Actions run records this edition does not promote each route to observed success.

Target OS, selected platform implementation and renderer are separate. The Apple and Android routes described here use the SDL3 platform implementation unless configured otherwise; iOS is a target OS, not a fifth `CNA_PLATFORM` value. The iOS smoke selects `SDL_RENDERER`; native `METAL` remains macOS-only at the tag.

67.1 Android application structure

The Devices demo contains a complete Gradle and NDK application rather than a sketch. Its 39 files include a Gradle wrapper, manifest, `SDLActivity` subclass, SDL Java glue, launcher resources, and native build. The JNI-side CMake file uses `add_subdirectory(<cna-root>)`, so producing the APK compiles the CNA library as part of the application. The recorded configuration uses compile SDK 35, NDK 30, minimum SDK 24, and only the `arm64-v8a` ABI.

Android-specific production code is concentrated rather than scattered. The Devices module contains about 2,090 lines of NDK sensor implementation using `ASensorManager`, an `ALooper`, event queues and rollback. Each `AndroidSensorBridge` can own a dedicated polling thread; the Motion backend aggregates six bridge instances, so the single `std::thread` construction site can produce several concurrent workers. CNA itself uses no JNI. Its only Android-specific link adds `android` and `log` to the Devices module; the root CMake file contains no Android branch. Portable sensor mathematics stays outside `__ANDROID__` guards so native tests can exercise it, while the I/O boundary is platform-gated.

67.2 Recorded Android execution

Two emulator campaigns are recorded. On 4 July 2026 an NDK-built `CnaTests` executable was pushed to a Medium Phone AVD. All 230 Net/Gamer/ENet/Packet cases and 1,831 other cases passed before an unrelated `TitleContainer` test reached the no-Activity crash described below; surviving build-system repairs corroborate the run. On 5 July the Devices application produced a 7.3MB APK, installed and started it, captured the emulator screen, and injected sensor data through the emulator console. The commit record says the real UI and its sensor-event flash were visible in that screenshot. That injection exercised SDL’s Accelerometer/Gyroscope

delivery; Compass/Motion’s CNA-owned NDK bridge and six-source fusion were not separately driven.

Those are meaningful execution results, but the repository contains no APK, AAB, logcat log, Android screenshot, or machine-readable test result from them. The strongest honest wording is therefore “recorded emulator execution,” not a continuously reproducible Android gate. No CNA binary has been evidenced on physical Android hardware at the audited revision.

Android graphics is a separate and still open boundary, but “never attempted” is too broad. The July Devices APK’s nested CNA build used Android’s default `SDL_RENDERER`, and its commit records screenshot-confirmed UI presentation. That is historical renderer execution, not a retained pixel oracle: no screenshot survives in the repository, no graphics-specific assertion was made, and the later adaptation was not rebuilt at the audited pin. WebGPU is the only renderer containing an explicit Android native-window surface path, and it has not run there. The strongest claim is thus recorded pre-adaptation SDL-renderer presentation on an AVD, not current Android graphics verification and not evidence for OpenGL or WebGPU.

67.3 Lifecycle coverage is explicit but partial

The platform event contract carries will-enter-background, did-enter-foreground, low-memory and terminating events. `Game` suspends between the first two, dispatches low-memory to content cleanup and exits on termination. That is source-level lifecycle policy plus portable tests; it does not prove every mobile host delivers the events in the expected order or that a game’s GPU resources survive a background transition.

This is best expressed as a state-machine gap. Launch-to-visible behavior covers only the straight path. A useful mobile test must drive background, foreground, low-memory, and process recreation transitions, then verify saved state and graphics-resource reconstruction. An emulator can provide most of this evidence before physical-device testing adds drivers, thermal behavior, and vendor-specific lifecycle timing.

67.4 Assets expose a deeper integration gap

`TitleContainer` first uses `std::filesystem`; on Android it can fall back to `SDL_LoadFile` and wrap the bytes in a memory stream. Historical execution found that fallback could seg-fault without a real Activity/JNI context. More importantly, no production consumer calls `TitleContainer::ContentManager` builds `Content/<asset>` directly from the current working directory.

The fallback is therefore both fragile and unreachable from the main content path. The same CWD-relative content root is a latent issue for Android packages and macOS application bundles. A complete mobile asset contract should choose one route, base it on the package location or platform asset API, and test it through `ContentManager` rather than through an isolated helper.

67.5 macOS: native routes are declared

The Metal renderer keeps Objective-C++ behind a pure-C++ PIMPL boundary. Objective-C++, Metal, QuartzCore, and Foundation are enabled only when the selected identity is `METAL`. This is a sound portability seam, but source isolation is not native validation.

The `Metal` CTest prefix is much broader than three registrations. Portable Metal policy gtests are discovered individually and also collected in `Metal_PortableHelpers`; a recorded

Linux HEADLESS run consequently reports 207/207 for `ctest -R 'Metal'` while executing no Objective-C++, Apple framework, shader compiler, or GPU command. Only the separate `Metal_Smoke` and `Metal_Capabilities` registrations require a `METAL` configuration. The renderer identity of the build must accompany every quoted result.

There is genuine historical Apple evidence: a macOS 14/Xcode 15.4 workflow on the original feature tree ran 143 tests, passing 136 and failing seven. The later adaptation changed interfaces, ownership, transfer policy, registration, and supported behavior. That run is valuable history, not evidence for alpha.1.

At the tag, the dedicated automatic Metal workflow configures `METAL` on macOS 14, builds the native target, enables Metal shader validation/debug-layer variables and runs the Metal CTest prefix plus a renderer-definition control. The separate Apple workflow builds an `SDL_RENDERER` macOS suite and a self-contained `.app`, checks Mach-O dependency paths and launches it for a success marker. These definitions are materially stronger than the old stale-filter route, but this edition did not retrieve workflow-run artifacts and therefore records declared gates rather than a new pixel/Retina/frame-pacing observation.

67.6 iOS: final-link and simulator route; tvOS remains a non-claim

The tag now contains an iOS toolchain, application metadata and a minimal UIKit/SDL-main/Game smoke application. Its workflow matrix final-links `SDL_RENDERER` application bundles for arm64 device and arm64 simulator, inspects Mach-O platform load commands, verifies the SDL entry symbols and absence of build-machine dynamic dependencies, then ad-hoc signs, installs and launches the simulator bundle until a one-frame success marker appears. Tests/examples and Net are deliberately off; no physical device is run. This is an executable route in source, not evidence that the tag's matrix ran successfully unless its Actions record is also retained.

tvOS has no corresponding toolchain/application/workflow route and remains an explicit non-claim. Native `METAL` is also not an iOS claim: the available smoke uses `SDL_RENDERER` over the `SDL3` platform implementation.

The next durable milestone is intentionally narrower than “mobile support”: archive a fresh Android graphics build and emulator frame with logs; retain alpha.1 Metal and Apple workflow artifacts; add a physical iOS-device result; then design, rather than infer, a tvOS host. Each step should state which rung it proves. That makes platform progress cumulative without allowing yesterday's partial result to masquerade as today's complete one.

Part XI

Testing and Verification

Chapter 68

The CnaTests Architecture

CNA’s test system is not a directory of independent unit-test programs. It is a composition of one large GoogleTest binary, hundreds of standalone renderer executables, process-isolation harnesses, negative compilation, and script-based structural gates. Understanding which layer produced a result is a prerequisite for interpreting that result.

68.1 One discovered GoogleTest binary

`cmake/UnitTests.cmake` recursively globs C++ test sources from three roots: non-renderer modules, renderer modules, and the repository-level `tests` directory. The result becomes exactly one executable, `CnaTests`. There is no GoogleTest executable per framework module.

The broad glob is narrowed by nine explicit `list(FILTER ... EXCLUDE REGEX)` operations. They remove standalone Glide ABI programs and minimal-link probes, conditionally remove five FFmpeg video tests, remove Net and GamerServices trees when networking is disabled, exclude POSIX-process tests on Windows, Emscripten, and Android, and select only the configured renderer-specific test directory. One exclusion group even checks itself with a fatal error if Net sources survived a Net-disabled configure. Filtering is therefore part of the build contract, not housekeeping.

GoogleTest cases are exposed to CTest with `gtest_discover_tests(... DISCOVERY_MODE PRE_TEST)`. Discovery runs the built binary at test time, so the complete CTest list does not exist at configure time. Its working directory is pinned to the source root after cases failed to find `tests/assets` when CTest launched them from the runtime-output directory. Both details explain why a source-only count and a `ctest -N` count answer different questions.

68.2 One configure, one renderer set

The ordinary configuration enters one chosen renderer family. Alpha.1 also accepts a compatible semicolon-separated `CNA_GRAPHICS_RENDERERS` set; the selected singular identity remains the default. Renderer-local source filtering tests membership in `CNA_RENDERER_IDENTITIES`, so a multi-renderer `CnaTests` build retains the suites for every compiled family rather than only the default. Metal and Glide still contribute some portable policy suites even when their native implementation is gated. No supported tuple therefore implies “all renderers,” and a result that omits both the compiled set and the active runtime identity cannot be interpreted safely.

There are three layers of renderer gating:

1. source filtering decides which compiled renderer families’ GoogleTests enter `CnaTests`;
2. CMake conditions decide which standalone executables and CTests are registered;
3. runtime pre-flight may return the project-wide skip code 77 when no display is usable.

That last result is deliberately SKIPPED rather than FAILED. It keeps headless machines from turning absence of a display into a renderer defect, but it also means a green CTest summary may contain no executed pixel comparison. Reports must retain skip counts and the display setup.

68.3 Standalone examples are a separate corpus

Files named `*_test.cpp` under renderer `examples` usually do not contain GoogleTest at all. Per-renderer CMake macros build them one executable at a time, each with its own `main()`, then optionally register it with CTest. Many consume the same public fixture from `modules/graphics/examples`, allowing one renderer to expose a defect while the others serve as controls.

Building and registering are independent acts. Some renderers build far more executables than they register; others add Python audit registrations that are not executables. Labels are also uneven: `GraphicsSmoke` is cross-renderer, while several large renderer families label only one or two of hundreds of registrations. A command such as `ctest -L EasyGL` therefore cannot be assumed to select the EasyGL corpus.

68.4 Why some tests need another process

Several failures concern process-global state: SDL shutdown, the audio mixer's global owner, GamerServices dispatch, and network peers. CNA builds small helper executables for these cases and embeds their absolute paths into `CnaTests`. A GoogleTest case spawns the helper and judges its result, gaining a clean address space and making crashes or shutdown ordering observable without poisoning later cases.

Other helpers are diagnostic tools rather than isolation tests: an XNB audio metadata dumper, XWB inspector, FNA-reference dump, and Devices microbenchmark. Their presence beside harnesses does not make them part of every test run. The governing question is always who invokes the target and under what configuration.

68.5 Compilation failure can be the oracle

`cna_strict_xna_api_leak_check` is excluded from the normal build. Its CTest command invokes the build system for that target and is marked `WILL_FAIL TRUE`. The test passes only when the compiler rejects the forbidden API use. This inverted polarity turns a namespace and surface rule into executable evidence: successful compilation would be the regression.

Fourteen module probes apply the complementary idea to linking. Each builds a minimal consumer of one module alias, then a link-closure test inspects the generated `link.txt` against a module-specific forbidden-library expression. The probe asks whether the public target works; the closure gate asks whether it works for the right architectural reason.

68.6 Discovery, selection, execution, verdict

A reliable test report separates four stages:

Discovery Which sources and runtime-expanded GoogleTest cases were present?

Selection Which filter, label, renderer, and feature options chose the cases?

Execution Did the intended executable, display, GPU, browser, or compatibility layer actually engage?

Verdict Which oracle declared pass, fail, or skip?

This model explains most apparent contradictions in CNA's historical test prose. A source can exist but be filtered out; a target can build but not be registered; a CTest can register but skip; a program can pass while running through the wrong graphics translation layer. The rest of this Part examines the numbers and oracles that make those stages auditable.

Chapter 69

Counting Discipline

“How many tests does CNA have?” has no context-free answer. The repository contains source files, GoogleTest definitions, standalone executables, and CTest registrations; a configure selects one renderer or a compatible renderer set; GoogleTest cases are discovered by executing the binary. A useful number must therefore name both its unit and its configuration.

69.1 Four quantities that must not be exchanged

At v0.1.0-alpha.1 @ 1bb2145d, static inspection gives the following representative counts:

Count	Exact definition
568	C++ source files in the three roots matched by the CnaTests glob, before configure-dependent filtering.
8,263	Lexical <code>TEST</code> , <code>TEST_F</code> and <code>TEST_P</code> definitions across those files, before filtering and parameter expansion.
1,720	Direct <code>cna_register_renderer_test</code> call sites in renderer-family <code>CMakeLists.txt</code> files; not active tests in one build.
1,297	C++ files under module <code>examples</code> directories; 1,232 happen to be named <code>*_test.cpp</code> , but almost none are GoogleTest sources.

These figures describe different populations. Adding them is meaningless, and selecting the largest does not make it the most authoritative. A test source can define many cases; a parameterized definition can expand at runtime; one executable can become thousands of CTest entries; one standalone executable can exist without any CTest registration.

69.2 How the static counts are reproduced

The source population is a sorted union, not a sum of unchecked directory totals:

```
{ find modules -path 'modules/*/tests/*' -name '*.cpp' -type f
  find tests -name '*.cpp' -type f; } | sort -u | wc -l
```

Macro counts search only declaration starts, distinguishing `TEST`, `TEST_F`, and `TEST_P`. This is still a source-definition count, not a run count. Nine `INstantiate_Test_Suite_P` sites and platform conditionals can change runtime expansion.

Renderer registrations require another derivation. A lexical search finds 1,720 direct helper call sites under renderer `CMakeLists.txt` files. Helper definitions, wrapper expansion and nested CMake conditions mean that number is deliberately described as call sites, not registered or executed tests.

69.3 Why there is no all-renderer total

Singular builds enter one selected family; multi-renderer builds enter only the explicitly named, compatible set. Platform gates and incompatible native dependency graphs prevent a universal all-family configuration. Summing every family’s registration sites therefore creates a number that no supported build exposes as one CTest suite.

CTest discovery adds a second barrier. `PRE_TEST` obtains case names by running `CnaTests-gtest_list_tests`; configure-time source inspection cannot know the complete expanded list. Build features, conditional compilation, and parameter instantiation all affect it. The authoritative count for a particular build must come from that built artifact and must name its renderer and feature set.

69.4 Identity counts can be settled

Not every disputed number is inherently unknowable. The canonical cache value table contains 50 public renderer identities, while `modules/renderers` contains 46 implementation families after excluding common infrastructure. Four identities share implementations with other public modes. Thus “50 identities, 46 families” resolves the apparent conflict instead of choosing one number and calling the other wrong.

This pattern is widely useful: define the noun before reconciling the count. An identity is a user-selectable contract; a family is a source implementation. They need not be one-to-one.

69.5 Counts that require execution

Static inspection cannot establish:

- the CTest count for a configured and built tree;
- how many parameterized GoogleTests expand or actually run;
- active standalone registrations after nested conditions;
- pass, failure, timing, or skip totals;
- whether any pixel assertion reached a display;
- image-oracle divergence counts;
- line, branch, or function coverage—no coverage tool is configured.

CNA documents several other kinds of “coverage”: API-surface estimates, mappings from source files to suites, and feature checklists. None is measured code coverage. A percentage without that qualifier is more misleading than no percentage at all.

69.6 A reporting grammar

A durable result can fit in one sentence if every noun is present: “On commit X, the Y renderer build with features Z discovered N CTest entries; it ran R, skipped S, and failed F under host H.” A source audit uses a different sentence: “At commit X, this command found N declarations matching definition D.”

Three historical comments claimed approximately 247, 330, and 600 renderer registrations, each using a different unstated meaning; the tag tree contains 1,720 direct helper call sites across configuration-dependent families. Comments rot, while reproducible definitions can be

rerun. The discipline is therefore not to avoid numbers, but to make every number carry its derivation.

Chapter 70

Oracles: What Does “Correct” Mean?

A test input and an observed output do not yield a verdict until something defines the expected result. CNA uses several such oracles: pixels captured from the real XNA runtime, numeric values from FNA, renderer-to-renderer comparisons, independently recomputed mathematics, and looser project-owned golden images. Their authority and tolerances are deliberately different.

70.1 The captured-XNA image corpus

`tools/xna-oracle` contains 39 declarative scenes and the same number of matching 256-by-256 RGBA8 PNGs. The references were captured once by hand from the real XNA 4.0 runtime under Wine. The C# runner source is checked in; its compiled executable, XNA runtime, and private 32-bit Wine prefix are not. Those external pieces are needed to regenerate references, not to compare a current CNA render against the checked-in corpus.

One renderer-agnostic C++ program implements the CNA side and is built for D3D9, EasyGL, OpenGL ES 1, FNA3D, and Skia. The comparator uses absolute per-channel differences. Its default tolerance is zero; it has no PSNR, SSIM, or perceptual metric. Optional RGB and alpha tolerances, a raw-differing-pixel budget, and an allowed rectangle make exceptions explicit rather than globally weakening the corpus.

The governing rule is stricter than “looks similar”: an increased tolerance needs a documented scene-specific reason, because silent tolerance growth turns an authenticity project into a parity project. D3D9’s registered hard gate runs all 39 scenes at tolerance zero. FNA3D’s registered corpus test is presently a render-survival gate, not a pixel-equality gate. EasyGL and OpenGL ES 1 runners are manual measurements rather than CTests.

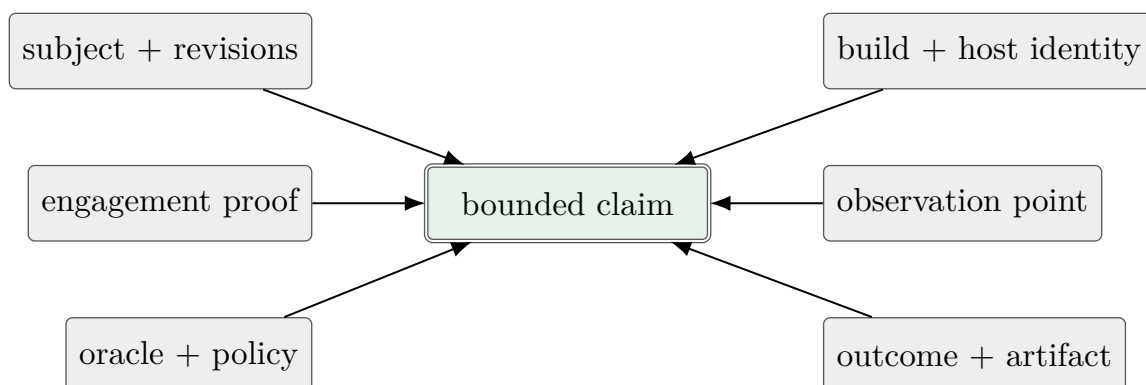


Figure 70.1: The evidence vector behind a durable claim. Source, compile, runtime, renderer, oracle, and pixel labels summarize different populated coordinates.

70.2 A policy table is part of the oracle

Skia runs the nine SpriteBatch-mode scenes under a checked-in per-scene TSV policy. Seven are pixel exact. The two linear-filter scenes permit RGB delta 1, no alpha delta, a bounded number of raw differences, and a specific 80-by-80 rectangle. The runner verifies policy-to-scene and scene-to-policy set equality before comparing images.

This is a strong pattern for necessary relaxation. The tolerance is local, spatially bounded, versioned beside the references, and meta-validated against the corpus. Adding a scene without a policy or leaving a dead policy row both fail. The exception remains reviewable instead of becoming a magic global number.

70.3 Project-owned goldens answer a weaker question

`PixelTestGame` provides single-region samples and full golden comparison for standalone renderer tests. Its comparisons intentionally inspect RGB and ignore alpha. Seventeen 8-by-8 golden PNGs live under `examples/golden`; their names are EasyGL-derived, and OpenGL 2 reuses the same files.

Tolerances range from zero to 60 on an 8-bit channel. At the high end, the test catches gross shape or state regressions rather than subtle shading drift. That does not make it useless; it makes its contract different from the real-XNA corpus. A self-generated golden is a regression oracle for CNA’s chosen behavior, not independent evidence that the behavior matches XNA.

Setting `CNA_UPDATE_GOLDEN` to any non-empty value rewrites the reference from the live render. Updating is therefore a review operation, never a routine way to make a failure green. The change should carry the image diff, renderer, driver, and reason the new output is correct.

70.4 Point samples and the skip boundary

Single-pixel coverage is broader than golden coverage: the audited tree contains 340 `ExpectPixel` calls in 70 files, and 169 renderer example sources include the pixel-test harness. These cases are cheap and precise for clear colors, blend equations, scissor order, or depth outcomes.

Before running, the harness probes SDL video. Failure returns code 77, which CTest interprets as SKIPPED. A green headless run can therefore mean that none of these comparisons executed. A pixel-oracle report must state display and skip evidence, not just the final exit status.

70.5 FNA as a numeric reference

A separate chain runs C# against a live FNA.dll and compares its JSON output with CNA’s C++ reference dumper. It covers enums, state presets, packed-vector types, and viewport cases. Numeric comparison defaults to absolute tolerance 10^{-4} ; non-numeric values must match exactly. It is one-directional: every FNA key must exist in CNA, while CNA-specific extension keys may remain unmatched.

This manual tool found a real `IndexElementSize` disagreement: semantic 16/32 values on one side versus ordinal 0/1 values on the other. The example shows why a second implementation is useful even when neither side produces pixels. It challenges assumptions embedded in the public value surface.

70.6 Independent oracles inside the corpus

Not every expected value comes from an external runtime. CNA uses three recurring patterns:

Golden tables Packed-vector values are derived separately from FNA arithmetic and checked with narrow numeric tolerances.

Recomputation ladders glTF tests rebuild node composition through independent L2, L3, and L4 calculations rather than asking the production routine for its own answer.

Shared oracle headers A single pixel or geometry oracle is consumed by multiple renderer tests so copies cannot drift independently.

LZX decompression adds a byte-exact differential against FNA’s decoder and checked-in reference bytes; it previously exposed a heap-buffer overflow. Direct2D and EasyGL also share a small 64-by-64 diagnostic corpus with a bounded renderer-to-renderer tolerance and an explicit feature whitelist. Excluding unstable filtering cases is preferable to widening the tolerance until everything passes.

70.7 Choose authority before tolerance

The oracle hierarchy can be summarized as follows: real XNA defines authenticity; a live FNA implementation supplies an independent compatibility reference; independent mathematics checks algorithms; cross-renderer controls localize implementation defects; CNA-owned goldens prevent unintended visual change. CNA does not reference the sibling `xna4-decomp` or `xna4-spec` repositories in this chain.

Tolerance belongs to the authority and phenomenon being measured. Zero is appropriate for a byte-exact decode or stable reference image; one-channel rounding may justify a local pixel policy; a tolerance of 60 means only a coarse visual guard. Naming that strength is part of the verdict. “Passed the oracle” is incomplete until the reader knows which oracle, which policy, and whether it actually executed.

Chapter 71

Verification in Hostile Environments

Some test hosts can return success while silently substituting a different runtime. Wine may fall back from DXVK to WineD3D, Proton may resolve an unpinned release, a browser page may load without reaching a frame, and CTest discovery may invoke a graphics binary without opening a device. In these environments the first oracle is not the game output. It is proof that the intended execution route engaged.

71.1 The Wine wrapper as a verdict pipeline

CNA's Wine wrappers follow one recurring sequence: validate arguments, choose an isolated prefix, configure diagnostic variables, refuse a missing prefix, run `wine` through `tee`, preserve Wine's exit status, and inspect the captured log. They do not bootstrap ordinary prefixes. Missing `system.reg` produces setup instructions and a failure rather than silently mutating developer state.

The final inspection is the essential step. D3D9, D3D11, D3D8, and D3D10 require a log line matching `DXVK: <version>`; D3D12 requires `vkd3d-proton - applicationVersion::`; DirectDraw generations require a `:ddraw:` trace. If that token is absent, the wrapper returns code 3 regardless of the program's own exit status. A passing executable through the wrong translation layer is treated as a failed experiment.

The prefix map is part of reproducibility. D3D9 and D3D11 share `~/.wine-cna-d3d11`; DirectX 1, 2, 3, 5, 6, and 7 share `~/.wine-cna-dx1`; D3D12, D3D8, and D3D10 have dedicated prefixes. D3D10 forces `DISPLAY=:0` because of a recorded DXGI/EDID failure under Xvfb, while D3D8 selects `llvmpipe` to avoid a specific surface-loss path. These are measured environment constraints, not renderer semantics.

Direct2D is the dangerous exception: its Wine shim defaults to the developer's ordinary `$HOME/.wine`. A separate fresh-prefix suite demonstrates the safer pattern with a temporary directory and `wineboot -init`. Evidence intended to be reproducible should prefer the disposable route and record the prefix identity.

71.2 Discovery needs a controlled escape hatch

Cross-compiled `CnaTests` uses CMake's `CROSSCOMPILING_EMULATOR` to place the Wine wrapper before the executable. GoogleTest discovery then runs `-gtest_list_tests`. Listing names does not create a graphics device, so no DXVK or DirectDraw token can legitimately appear. Per-wrapper `CNA*_SKIP*_GATE=1` variables bypass only the engagement check for discovery.

This is a narrowly justified exception. The execution CTests must run with the gate enabled, or a green result loses its renderer identity. Recording the bypass in generated test commands makes the distinction reviewable instead of teaching every wrapper to accept missing evidence.

71.3 Proton solves a different reproducibility problem

The Proton wrappers bootstrap their own compatibility prefix rather than requiring an existing Wine prefix. Direct2D resolves the first available runtime from a checked-in priority list, currently Proton 9.0 Beta then Proton 8.0, and rejects directories named Experimental unless an explicit override marks the result non-reproducible. It publishes runtime identity, including the DXVK version extracted from the shipped DLL.

Both Proton paths refuse to operate under the ordinary Wine prefix. That safeguard follows a real incident in which a mixed prefix was partly upgraded and remained hung for hours. Neither path applies the Wine wrappers' log-token gate, but they do not provide equivalent evidence. Direct2D combines the child program's exit status with its emitted, pinned runtime identity. D3D12 defaults to the moving Proton Experimental installation, overlays two vkd3d-proton DLLs, and ultimately forwards only the child's exit status; its diagnostic program must write a file because Proton does not reliably return child output to the shell. A D3D12 Proton pass is thus useful route evidence, but weaker reproducibility evidence than the pinned Direct2D lane. The contrast is instructive: engagement proof must match the host's actual substitution risk rather than be copied mechanically.

71.4 Virtual displays are infrastructure, not evidence

Direct2D's Xvfb wrapper validates display geometry, creates private Xauthority state in a temporary directory, installs cleanup traps, and is reentrant. That makes headless presentation repeatable. It does not prove physical display behavior, vendor drivers, HDR, or desktop composition. The report must still say that Xvfb and a translation layer were used.

Likewise, a native process returning zero under Wine does not become a native-Windows result. The environment belongs in the claim: compiler, executable format, Wine or Proton version, translation backend, GPU or software device, display server, and prefix.

71.5 Browsers need an out-of-process verdict

WebAssembly's exit status is not a reliable observable for the browser harness. HTML-DOM and SVG-DOM therefore publish completion and result fields on `window`. Playwright waits with a bounded timeout, treats page exceptions and failure console lines as errors, and combines that verdict with structural and screenshot evidence.

Serving over loopback HTTP is also part of the environment. The shell wrappers start a local server, poll it for readiness, and terminate it through a cleanup trap. They do not use `file://`, whose origin and asset behavior would differ from a deployed page. Distinct ports permit the six HTML-DOM cases to run without collision.

The compositor check takes a browser screenshot, decodes it into a canvas, and samples pixels with bounded RGB tolerance. This deliberately crosses out of renderer-owned state and back through Chromium's real composition result. SVG-DOM adds namespace and hidden-canvas assertions to prove that the expected implementation, not merely a look-alike image, supplied the frame.

71.6 Hostility is any plausible false green

Wine fallback, stale Proton selection, headless skips, browser startup without a frame, and renderer impersonation look unrelated. They share one failure mode: the program can appear

healthy while the intended experiment never occurred. A robust harness therefore has two verdicts:

1. an engagement verdict proving the named runtime, renderer, device, or browser path;
2. a behavior verdict applying the actual functional oracle.

Only their conjunction is a pass. This pattern is more transferable than any particular grep token: identify how the host can substitute or skip the thing under test, then make evidence of non-substitution a first-class artifact.

Chapter 72

Discipline as a Test

Many CNA regressions would compile, render a plausible frame, and pass ordinary API tests while violating the architecture or historical era a renderer is meant to preserve. The project turns those negative constraints into executable checks: forbidden-token scans, link-closure probes, identity-table comparisons, document-to-code validators, log parsers, and mutation tests. In this layer the gate itself is the principal artifact.

72.1 DirectX eras as executable boundaries

Eight shell gates scan the source and public headers of DirectX 1, 2, 3, 5, 6, 7, 8, and 10. They strip comments, search for forbidden interface names or call shapes, and fail if the source directory is missing. Each forbidden list encodes an architectural statement:

- DirectX 1 permits only first-generation DirectDraw and no Direct3D;
- DirectX 2 forbids execute buffers, while DirectX 3 differs by allowing `IDirectDraw2`;
- DirectX 5 and 6 require version-4 surfaces and reject the bare legacy surface name;
- DirectX 7 forbids viewport objects, texture-handle indirection, and a texture-blend state empirically rejected under Wine;
- DirectX 8 removes DirectDraw and requires modern state names;
- DirectX 10 rejects both the FVF model and D3D9-style render-state calls.

The later scripts use a line-preserving Perl pass to remove both line and block comments. That upgrade became necessary when Doxygen prose began matching forbidden tokens. Preserving line numbers keeps violations actionable. The lesson is not that grep replaces compilation; it is that a focused lexical invariant can reject historically impossible shortcuts that a compiler would happily accept.

72.2 Link closures make module boundaries observable

Fourteen minimal consumers link public module aliases. A companion Python gate reads each generated `link.txt`, rejects forbidden libraries, and can require expected ones. Math, for example, must not acquire a graphics, SDL, ENet, or renderer dependency merely because a transitive umbrella target makes the program link.

The gate returns skip 77 when a generator does not produce `link.txt`. That is honest but weakens the evidence, so reports need generator and skip data. The broader pattern remains valuable: compile proves usability; link inspection proves dependency direction.

Renderer identities receive a similar structural check. An order-sensitive canonical table is compared with the C++ enum and set-wise with the CMake cache choices. This detects missing,

stale, reordered, or unselectable identities before a user discovers the mismatch at configure time.

72.3 Documents can be machine-checked contracts

Six Skia validators connect ledgers and decision records to implementation. They require every renderer method and capability to have a classified parity row; classify every test-matrix entry; hold the accepted 2D-only decision to 37 enumerated 3D requirements; check release-gate row contiguity and Ganesh guards; require successor-contract routes to cover an exact numeric range; and compare a 27-row surface-format table with the live enum and size functions.

These checks turn documentation from commentary into schema-bearing input. They also reveal the maintenance cost. At the audited revision, three registered Skia validators still read paths removed by modularization, and the native HLSL verifier has the same kind of path rot. A gate that aborts before reaching its invariant is not evidence, even if its design is sound.

Every validator should therefore test its own assumptions: required files exist, anchors occur exactly once, parsed sets are non-empty, and fixtures include both accepted and rejected cases. Treating a missing input as success would be the worst possible failure mode.

72.4 Can the green test go red?

Direct2D's mutation checker applies seven pinned edits to the renderer. Every anchor must occur exactly once. In full `-run` mode it rebuilds and runs the named test; if the test still passes, the mutation survived and the checker fails. Source is restored in a `finally` path.

Mutation testing asks a question ordinary execution cannot answer: is the oracle sensitive to the defect it claims to prevent? CI currently performs only the dry-run anchor validation, not the seven rebuild-and-test mutations. That distinction belongs in any claim about mutation coverage.

The Direct2D debug-log verifier applies the same self-suspicion differently. It fails if diagnostics never engaged, the debug layer was off, the report was truncated, severe messages appeared, or unexpected live objects remained. Six checked-in fixtures exercise the parser's pass and failure paths through `-self-test`. A log parser without hostile fixtures is only an untested second program.

72.5 Automation is narrower than apparatus

The tag's script layer is reached through CTest registrations, direct workflow commands and source-only generated-record checks. CTest wiring still matters only when the corresponding configuration is built and run; a Python checker in a workflow is a different population from a renderer executable registered conditionally.

The 21 workflows tell an uneven but broader story. Nineteen have push or pull-request triggers and Direct3D/GDI are manual-only. Multi-renderer, platform/audio, Apple, glTF, TinyGL, NanoVG and C API records now have dedicated routes. The Windows D3D workflow's unrestricted default build includes `CnaTests`, despite a stale header comment saying otherwise. Its renderer-label selectors do not run the general discovered corpus, although the Direct2D leg does run the filtered `Direct2D_Unit` entry through that binary. The GDI workflow instead names 17 standalone targets and never builds `CnaTests`. Metal has a macOS workflow, HTML-DOM has a browser workflow, and Devices has a UBSan path. Conversely, the general

job and two Input legs still use the invalid EASYGL value and never reach compilation; more files do not imply broader executed coverage.

Automation records should therefore distinguish four states: script exists, script is a CTest, workflow references it, and workflow triggers for the change in question. Collapsing them into “covered by CI” hides exactly the gaps the gates were built to expose.

72.6 A gate-quality checklist

A structural gate is trustworthy when it:

1. states one invariant in language a reviewer can compare with the code;
2. proves that its input and engagement conditions were present;
3. fails closed on missing or empty inputs, or reports an explicit skip whose consequence is retained;
4. carries positive and negative fixtures, mutation, or another self-test showing that the verdict can change;
5. records its configuration and is actually wired into the intended automation path;
6. is updated atomically when modularization moves its inputs.

The unifying principle is that a green result that was never produced is worse than a red one. Wine log tokens, compile-must-fail tests, link closures, mutation inversion, diagnostics-engaged checks, and browser verdict channels are independent expressions of that idea. Discipline becomes a test when the project can prove both the rule and the fact that the rule was applied.

Chapter 73

CI Execution and Coverage

A repository can contain excellent tests without running them automatically. CNA’s tree is a sharp example: thousands of assertions, hundreds of renderer programs, structural scripts, and 21 workflow files coexist with dead configuration legs, manual-only GPU jobs, and host boundaries no hosted runner has crossed. This chapter describes the automation at the pinned revision, not the intended future matrix.

73.1 Twenty-one workflows, nineteen automatic triggers

Nineteen workflows react to a push or pull request, often under narrow path filters. Direct3D and GDI remain `workflow_dispatch`-only. The expanded automatic set adds Apple, glTF stride and sanitizer, platform, TinyGL, NanoVG, native and Emscripten multi-renderer, and five C API workflows to the older general, Input, Devices, HTML-DOM, Metal and 32-bit jobs. A workflow file’s presence therefore proves neither that its trigger matched this tag nor that an individual job ran successfully.

The general Linux job configures Debug with tests and examples, builds the default target, then runs unfiltered CTest with a known-failure classifier. The Input matrix builds only `CnaTests` across five renderer/sanitizer combinations and invokes the single `input`-label entry, which repeats 541 selected cases five times in shuffled order: 2,705 case executions when all iterations complete. The workflow’s nearby comment still says “x3,” but the registered CTest command and current Input documentation both say five; executable configuration wins that disagreement. The Devices workflow instead runs filtered GoogleTests directly, following the preset’s warning that per-case CTest discovery can race shared temporary paths.

HTML-DOM is the most complete Web route: pinned emsdk, Playwright Chromium, six HTTP-served pages, browser verdicts, and compositor samples. Metal selects the native renderer on macOS 14, enables Metal validation variables, and runs Metal-named plus renderer-definition tests. The 32-bit job builds three small arithmetic programs with `-m32`; it is a narrow integer-overflow gate, not a 32-bit CNA build.

73.2 New architecture gates

The native multi-renderer workflow configures `HEADLESS;SOFTWARE;STUB`, builds the generated descriptor/registry gate, selects each compiled identity through the public demo, tests the environment-variable route, and retains a singular-build control. A separate Emscripten workflow composes `WEBGL2;CANVAS;HTML_DOM;SVG_DOM`. These are compile and runtime-choice evidence for two compatible sets, not proof that arbitrary families may be combined.

The platform workflow names the graphics, platform and audio values independently. Its matrix exercises SDL3, SDL2 and headless tuples, cross-implementation event oracles, SDL2-native helper executables and a 2D integration demo. Terminal remains covered by source/tests

outside a Windows build; the matrix does not turn its POSIX-only contract into a cross-platform claim.

Five C API workflows check generated surface coverage, pure-C header/toolchain compatibility, the ABI baseline, limitation records and the release decision record. They are substantial structural gates, but their repository snapshots do not prove that the alpha.1 tag’s workflow runs succeeded. Running the tag’s source-only release-record checker reports all ten criteria met and “READY to publish the experimental release.” An exact-tag library build nevertheless fails because the C renderer map has 49 identities while the canonical enumeration has 50. The checker proves that gate definitions exist and records agree; it does not execute the installed consumer or final library build and therefore produces a false artifact-readiness verdict. Its heading also mislabels ABI 0.7.0 as 0.1.0.

73.3 A live configuration defect changes the matrix

At the audited SHA, `general-tests-ci.yml` and two Input legs pass the retired internal token `EASYGL` as the current renderer option’s value. It is an implementation family, not one of the 50 public values accepted by renderer selection. The public native profile is `OPENGLES3` or another named GL identity. Those jobs therefore stop at configure time.

This defect removes more than nominal redundancy. The general job is the only automatic unfiltered suite, and one dead Input leg is the only `AddressSanitizer-plus-UndefinedBehaviorSanitizer` combination in CI. A workflow diagram read without its configure arguments would report both as coverage even though neither reaches compilation.

The correct automation audit starts at the trigger and follows every step: branch/path filter, configure command, named build target, test selector, environment, artifact upload, and failure policy. “Workflow exists” is merely the first row of that trace.

73.4 Windows jobs deliberately run a narrower corpus

The manual Direct3D matrix configures MSVC/Ninja for DirectX 11, DirectX 12, and DirectX 2D, disables examples and networking, then runs renderer-labelled CTests. Direct2D adds debug-layer log verification and a WARP lifetime pass; DirectX 11 invokes the native HLSL verifier. GDI builds 17 named executables and runs its label.

The GDI workflow’s explicit target list excludes `CnaTests`. The D3D workflow is subtler: its unrestricted `cmake -build` builds the default `all` target and therefore includes `CnaTests`, contradicting the workflow header’s claim that it does not. The DirectX 11 and DirectX 12 label runs still omit the unlabelled discovered GoogleTests. Direct2D, however, labels and runs `Direct2D_Unit`, whose command invokes `CnaTests -gtest_filter=Direct2D*`. The same header’s POSIX `setenv` rationale is stale because platform-incompatible process sources are filtered on Windows. Renderer-specific Windows tests remain useful, but the actual build and selector commands—not that comment—define their corpus. The native HLSL verifier also points to a pre-modularization source path and fails before it can import the shader manifest.

Manual dispatch has a further consequence: five of the seven scripts referenced anywhere in CI live only in this D3D workflow. On a routine push, the script layer exercised directly by workflows reduces to the two HTML-DOM runner layers. CTest can invoke many other scripts, but only inside a configure that registers and executes their renderer.

73.5 Sanitizers and validation are targeted

CNA has a root `CNA_SANITIZE` option and separate Devices presets that place raw sanitizer flags in `CMAKE_CXX_FLAGS`. CI contains two sanitizer legs in source: the dead Input ASan+UBSan leg and a Devices UBSan preset. No workflow runs ThreadSanitizer, and no general non-Devices sanitizer matrix exists. Skia tests disable vptr sanitization because the pinned no-RTTI Skia archives cannot supply the required type information.

GPU validation is renderer-specific. Vulkan enables Khronos validation and synchronization validation; three tests assert that the layer actually loaded. D3D11 and Direct2D request the debug device with a missing-SDK fallback. Metal's workflow exports shader and debug-layer variables. WebGPU pushes validation error scopes. These mechanisms are not interchangeable, and a test must prove engagement rather than infer it from an environment variable.

73.6 Fuzzing, properties, and mutation are not continuous campaigns

Four GoogleTest source files contain eight fuzz-named cases for XACT, XNB/LZX, and SDL input. They are deterministic mutation loops inside `CnaTests`; there is no libFuzzer, AFL, or OSS-Fuzz target. One SoundEffect reader file implements a hand-rolled property test. These are valuable adversarial tests, but they do not provide coverage-guided exploration or a persistent crash corpus.

Mutation testing exists for Direct2D only. Its seven mutations can rebuild and invert the expected verdict, but CI runs just `-dry-run`, proving anchors still exist rather than proving the renderer tests kill each mutation. The distinction between mutation machinery and a completed mutation campaign must remain explicit.

73.7 There is no code-coverage measurement

No build file, preset, script, or workflow configures `gcov`, `lcov`, `gcovr`, `llvm-cov`, or compiler coverage flags. Documents named `coverage.md` describe API presence, feature matrices, or source-to-test mappings. Their percentages are not line, branch, or function coverage.

A credible CI summary should therefore report which configured paths executed, their skip and artifact records, and the gaps. At this revision, automatic browser evidence is real, targeted native and sanitizer evidence exists, and much of the renderer apparatus remains manual or configuration-exclusive. The honest result is not “CI complete” or “CI absent,” but a precise map of what ran and what merely exists.

Part XII

Porting, Practice, and the Road Ahead

Chapter 74

Migrating an XNA or FNA Game to CNA

CNA preserves much of XNA’s public shape in C++; it does not make a C# game binary- or source-compatible. A successful migration is a controlled translation with explicit asset, renderer, service, and verification decisions. The steps below use the implementation and evidence boundaries established in the preceding Parts rather than CNA’s stale migration prose.

74.1 Freeze a behavioral baseline first

Before changing language or framework, record the original game at a named commit and capture inputs that matter: deterministic simulation traces, representative screenshots, save files, network packets, audio events, and asset inventories. Record resolution, fixed-step interval, random seeds, locale, and platform. A port without a baseline can become internally consistent while drifting from the game it was meant to preserve.

Classify the starting point. A C# XNA or FNA game requires a language translation; an existing C++ reconstruction may need only its platform-service boundary replaced. Keep these risks separate. Translating game logic and redesigning rendering in one pass destroys the ability to attribute a mismatch.

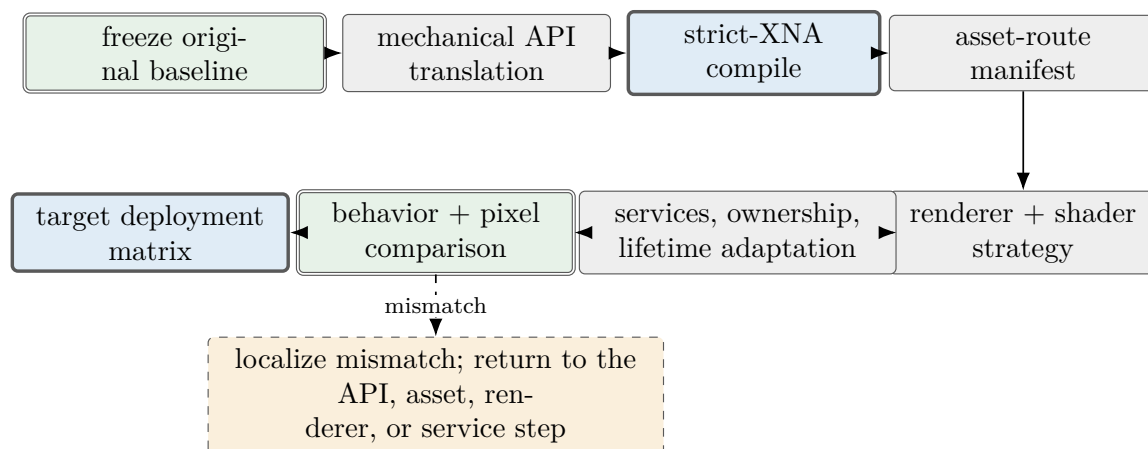


Figure 74.1: A porting workflow that preserves attribution. Translation, asset conversion, renderer choice, and service adaptation are verified as separable changes.

74.2 Translate the API shape mechanically

Namespaces and type/member names generally map directly from dots to C++ scopes. The largest systematic difference is properties: a C# read of `obj.Value` becomes `obj.GetValueProperty()`, and an assignment becomes `obj.SetValueProperty(value)`. Events, nullable values, collections, ownership, and `ref/out` arguments also need explicit C++ idioms.

Do not redesign names while translating. First reach compiling, behaviorally comparable code; then isolate genuinely game-owned abstractions. Compiler errors are useful inventory signals when the translation remains mechanical. Broad wrapper layers that hide every mismatch too early can make an unimplemented CNA behavior look like a game bug.

74.3 Inventory content by actual reader route

CNA can read real XNB files. Its binary stack handles the container, LZX, a registered reader table, shared resources, stock effects, graphics assets, models, and several audio encodings, with 30 checked-in fixtures at the audited revision. Old statements that XNB is entirely unsupported are false.

The boundary is still narrower than XNA's build pipeline. CNA does not produce XNB files, the reader registry is explicit rather than reflective, several reader forms remain unsupported, and LZ4 is refused. The general `EffectReader` accepts XNA/FNA D3D9 Effect Framework bytecode only when the active renderer reports `CompiledEffects`; the five stock-effect XNB readers remain separate. Test every concrete asset against the selected route rather than reasoning from its extension alone.

Loose assets use CNJ envelopes, native image/audio/video formats, or direct glTF/GLB model loading. Runtime glTF and offline `gltf_to_cnj` share one import core but retain material, animation, and multi-group limitations described in Part VI. A migration manifest should map every source asset to one of these routes, its expected reader, sidecars, and verification oracle.

74.4 Treat custom shaders as renderer contracts

`Effect(GraphicsDevice&, byte[])` and the general XNB Effect reader now accept bounded XNA/FNA D3D9 Effect Framework binaries on FNA3D. `SDL_GPU`, `EasyGL` and `Vulkan` expose the same capability only behind their opt-in compiled-effects switches. Inventory the exact byte format and target configuration: `.fx`/HLSL source, DXBC and MonoGame MGFX remain different inputs.

`ShaderEffect` is the separate source/program route, and accepted language varies by renderer: GLSL on several GL paths, HLSL on Direct3D paths, SPIR-V bytes on Vulkan, marker-gated SkSL on Skia, and explicit refusal or null on others. Choose the target renderer before choosing a shader migration strategy. A portable game may need renderer-specific programs or may need to stay within stock effects.

74.5 Audit vertex layouts and model assumptions

The glTF importer recognizes seven packed strides, while the upload path has eight cases because one semantic form expands the dispatch. A custom XNA `VertexDeclaration` is not guaranteed to fit every renderer or importer route. Record element offsets, formats, usages,

stride, index width, primitive topology, and whether data comes through XNB, glTF, CNJ, or direct buffers.

Models have four routes with different evidence: hand-built runtime objects, binary XNB, runtime glTF/GLB, and CNJ/offline import. Skinning also has two ordering systems. Preserve joint, scene-node, and palette index spaces explicitly; do not infer that a model drawing on one renderer proves every route or renderer.

74.6 Select a renderer by evidence, not by name

Renderer selection is a CMake-time choice from 50 public identities backed by 46 families. First filter by platform admissibility and required capabilities: 2D/3D, custom effects, MRT, render targets, texture forms, instancing, readback, and occlusion queries. Then inspect the evidence tier for the exact identity and host.

A useful progression begins with one well-verified desktop renderer, adds a structurally independent control, and only then crosses to Web or mobile. Compile the same game against each identity; do not add renderer-specific conditionals to game logic unless the capability contract requires them. A colored inherited draw fallback or a headless skip is not a parity result.

74.7 Make service differences explicit

List every use of input, audio, media, storage, GamerServices, networking, sensors, and platform dialogs. CNA's implementations are not equally deep: local GamerServices identities are synthetic, Storage containment differs between selectors and files, network Local delivery is not a normal transport, Web discovery and persistence diverge, Android lifecycle coverage is partial, and media depends on the FFmpeg configuration.

Timing deserves its own line item. Set `TargetElapsedTime` from the original simulation instead of accepting CNA's default. Decide how pause, background, slow frames, variable-step mode, and shutdown map on every host; the Emscripten loop does not yet mirror all native semantics.

74.8 Build a verification ladder for the port

Use increasingly independent checks:

1. compile-time surface and negative-compilation checks;
2. deterministic logic tests with no renderer;
3. asset-load tests for every reader form and malformed boundary;
4. single-pixel and small golden checks on the primary renderer;
5. differential screenshots or numeric traces against the frozen original;
6. a second renderer as a localization control;
7. browser, compatibility-layer, emulator, and physical-host engagement evidence.

Record skips and runtime identity. A Wine result must prove DXVK/vkd3d/DirectDraw engagement; a browser result must reach a frame through a window verdict; a mobile result must retain its APK, logs, screenshot, and lifecycle trace. "Built everywhere" and "behaves everywhere" are separate milestones.

74.9 Troubleshoot from the boundary inward

Configure failures usually indicate a wrong public renderer name, a missing sibling dependency, an inadmissible platform/renderer pair, or a feature option that changes the source closure. Link failures often expose module ownership or missing external runtime libraries. Keep the exact configure command and generated renderer definition.

At runtime, classify the first failure before editing: asset resolution, reader registration, window/device creation, frame lifetime, renderer engagement, or behavioral oracle. Log working directory and content root; preserve the original exception; test the smallest asset or draw that reproduces it. The fastest migration is not the one with the fewest red results, but the one in which every red result names the boundary that actually failed.

Chapter 75

Case Study: Speedy Blupi and Planet Blupi

Two legacy Windows games shaped **free-direct**'s scope: *Speedy Blupi* (through the sibling **free-eggbert** repository) and *Planet Blupi*. Both run through the compatibility layer. A separate native-CNA feasibility study remains a proposal. The distinction lets this chapter compare an implemented migration bridge with a source-audited port plan.

75.1 Call-site-driven scope

free-direct's subsystem boundaries come from audits of the two games at named commits. Both use `DirectDraw` and `DirectSound`; only *Free Eggbert* uses `DirectPlay`. *free-eggbert* calls `Blt` four times and `BltFast` six times; *Planet Blupi* calls them three and five times. In each game, one `Blt` performs the once-per-frame back-buffer-to-primary present. This call-site pressure explains why Chapter 60 describes `DirectDraw` as the most developed of the library's three subsystems.

75.1.1 Raw occurrences versus interface calls

A later recount reports eighteen `BltFast` occurrences in *free-eggbert* and seventeen in *Planet Blupi*. That population includes header declarations, definitions of each game's `CPixmap::BltFast` wrapper, and calls to both the wrapper and the `DirectDraw` interface. The six and five counts include only calls that reach `IDirectDrawSurface::BltFast`, the boundary implemented by **free-direct**. The other eight calls per game enter the local wrapper, which then makes one of those interface calls. Counting both levels would count the same traffic twice. Call-site totals therefore need an explicit population definition.

75.2 A pixel-format mismatch that never became a visible bug

Both games request an explicit display depth: *free-eggbert* selects 8 or 16 bits through a runtime flag, while *Planet Blupi* requests 8 bits. **free-direct**'s `SetDisplayMode` ignores that value and creates a 32-bit primary surface. The deviation does not affect these two games because their offscreen surfaces follow a separate, hard-coded 32-bit path. Every observed producer and consumer therefore agrees on the delivered format. This is a verified harmless deviation for the audited callers, not a general guarantee about ignoring pixel-format requests.

75.3 Color keying: implementation broader than observed use

Planet Blupi uses source color-keying at two call sites through one caching helper. Both construct a single-value range and match white through a `GetDC` / `SetPixel` / `Lock` round

trip, not through a fixed palette index. `free-direct` implements low-to-high ranges, but the audited games exercise only the single-value case. A third game would need separate evidence for ranged behavior.

75.4 GetDC as a gameplay dependency

Planet Blupi's `IsIconPixel` performs a per-click gameplay hit test through `GetDC` / `ReleaseDC`. The implementation must handle both a zero-copy 32-bit wrap and an 8-bit surface expanded into a temporary palette-converted buffer. An API that appears to be incidental GDI interoperation therefore sits on an active gameplay path.

75.5 A performance audit adds exact costs to the same call sites

A later performance audit benchmarked the compiled library headlessly with `SDL_VIDEODRIVER=dummy`. `DirectDraw` 3's `BltFast` has no destination-size parameter, so it cannot scale. Every audited `BltFast` interface call in both games (*free-eggbert*: `pixmap.cpp:406,574,580,612,1762,1818`; *Planet Blupi*: `pixmap.cpp:395,401,433,1235,1291`) is consequently 1:1. At the time of measurement, the shared blit engine still performed a per-pixel integer division and bit-depth branch for these copies. It measured **29 times slower than a raw memcpy** of the identical byte count at an optimized build (0.715 ms versus 0.0244 ms for one 640×480 32-bit blit), and **206 times slower** under the then-default unoptimized configuration.

Build	BltFast	memcpy	Ratio
Default (no -O flags)	4.704 ms	0.0229 ms	205.8×
-DCMAKE_BUILD_TYPE=Release	0.715 ms	0.0244 ms	29.4×

Those numbers are discovery evidence, not the current implementation. The pin now defaults an otherwise-unspecified *top-level, single-configuration* free-direct build to Release while leaving a consuming project's choice untouched. It also detects equal source/destination extents at runtime and performs per-row `memcpy` when format and color-key conditions permit, followed by the required opaque-alpha fixup. The recorded follow-up benchmark reduced the same Release case from 0.715 ms to 0.466 ms, or from 29.4 to 14.8 times the raw-copy reference. The remaining gap is mostly the alpha pass; the original 205.8-times row no longer describes a plain standalone configure. The two measurements preserve the before-and-after evidence instead of presenting the discovery result as current performance.

One `Blt`-family call does require scaling: *free-eggbert*'s `CPixmap::Display()` (`pixmap.cpp:1509-1516`) presents the fixed internal resolution into the window client area. The 1:1 fast path therefore compares the two rectangles at runtime. The audit also found dead code on the game side: *free-eggbert*'s `CPixmap::DrawMap(int, RECT, RECT)` (`pixmap.cpp:640`) has exactly one `Blt()` call, but no callers of its three-argument signature in the inspected source.

75.6 Interfaces never exercised at all

Across both games, the only `QueryInterface` call belongs to a `DirectPlay` object. None targets the four `DirectDraw` classes, whose `free-direct` implementations return `DDERR_UNSUPPORTED`. *Planet Blupi* contains no `DirectPlay` use, so the `DirectPlay` scope described in Chapter 60 comes entirely from *free-eggbert*.

75.7 DirectSound structure-layout hazard

Both games populate `PCMWAVEFORMAT`, not the larger `WAVEFORMATEX`. The layouts place `wBitsPerSample` at different offsets; interpreting the caller's pointer as `WAVEFORMATEX` would read padding as the sample depth. `free-direct` therefore stores a raw `void*` and interprets these audited calls as `PCMWAVEFORMAT`. Both games also call `SetPan()` only on mono effects. That explains why the intended contract was documented as mono-only, but it does not make current panning audible. The implementation computes left/right gains and discards them; only the ordinary volume gain reaches SDL. The tests prove clamping, return values, and no-crash behavior, not an audible stereo split.

75.8 Dead and unexercised paths

free-eggbert's alternate BASS-backed sound implementation is behind a compile-time flag fixed to false with no build override. Each game also contains a wave-loading source file that only includes itself. Finally, every inspected call to the sound-playback method passes zero for the looping flags. This caller-side result agrees with Chapter 60's implementation-side conclusion that the two games cannot reach DirectSound looping.

Two narrower searches bounded possible additions. Excluding vendored DirectX 3 SDK headers, neither game calls `GetCurrentPosition`; `free-direct` declares only its `SetCurrentPosition` counterpart. The method remains unimplemented until a target caller requires it. Neither game explicitly requests an 8-bit `CreateSurface` format either. Together with the 32-bit primary path in §75.2, this means the audited games do not exercise `BlitFrom`'s silent mixed-depth fallthrough. That behavior remains a limitation, even though adding a new error route was outside the target-driven scope.

75.9 Native-CNA port feasibility

The preceding evidence concerns the `free-direct` compatibility shell. A later `cna.md` in the inspected *free-eggbert* working tree, dated 2026-07-16, studies a different question: what would it take to port *Free Eggbert* directly onto CNA's `Game / GraphicsDevice / SpriteBatch` surface, replacing `free-direct` (and `free-api` beneath it) rather than merely running through it? That 30,145-byte document was untracked, so it is supplemental evidence outside Appendix D's immutable Git pins; the exact inspected copy has SHA-256 `87b1e3de041cdb2af85c8c4b4faf6e74f0e518f9ff72f851f339e5419ceadb3`. It reframes the compatibility layer as a migration *bridge*, not a migration *destination* — a distinction the analysis itself states plainly: “a CNA application using the `FREEDIRECT` renderer has adopted the XNA-facing API, but it still transitively depends on `free-direct`.”

75.9.1 Scope from a direct line count

Free Eggbert is already C++; the port does not require a language translation. The analysis measures roughly 31,800 lines of C++ source and headers (a later recount in the same document gives 31,773 across `src/*.cpp` and `include/*.hpp`), with more than 21,000 of those lines — the majority — in simulation, world, object, and event logic. It therefore recommends rewriting the platform-services layer (graphics, input, audio, video, files, networking) while leaving the existing simulation/data layer (`decblupi`, `decmove`, `decblock`, `decdesign`, `decio`, and the game's tables) recognizable. Preserving that layer also preserves a direct comparison point for old and new behavior.

75.9.2 Preserving the CPixmap boundary

The analysis recommends retaining `CPixmap`'s public shape across its 98 direct call sites and replacing only its implementation, with `Texture2D` / `RenderTarget2D` / `SpriteBatch` objects standing in for `DirectDraw` surfaces. This separates rendering equivalence from the risk of editing 98 call sites. `SetTransparent2`'s color-key-*range* semantics are singled out as the one piece of that surface CNA's current `.cnj` metadata cannot express directly (it represents a single exact RGB value, not an inclusive range), so a *Free Eggbert*-specific loader would need to read pixels and zero alpha across the requested range by hand via `GetData` / `SetData` — described as “straightforward” precisely because the asset sheets involved are small, not because the technique is trivial in general.

75.9.3 Preserving the 50ms simulation cadence

Free Eggbert updates from a 50ms multimedia timer, a fixed 20Hz simulation cadence. CNA's `Game` defaults to roughly 16.67ms (60Hz). Without an explicit `TargetElapsedTime`, the port would advance the simulation about three times too fast; see Chapter 6 for the timing contract. The existing F5–F8 speed controls add another constraint. The analysis recommends representing them as multiple fixed 50ms simulation steps per rendered frame (2/4/8 steps) instead of shortening `TargetElapsedTime`, preserving fixed-size simulation steps.

75.9.4 MIDI integration gap

The ten inspected music files are Standard MIDI data, and CNA's own `Song` loader does not register `.mid`. The feasibility analysis correctly identifies that public integration gap, but its explanation predates a fuller decoder audit. CNA disables `FluidSynth`, yet its vendored `SDL3_mixer` build leaves the Timidity MIDI decoder enabled; the lower audio layer therefore does have a synthesis route, subject to a usable Timidity instrument configuration. What is missing is a supported `Song`/Content extension mapping, asset/instrument policy, and retained test showing that this route works in CNA rather than merely compiling below it.

The analysis gives three options: pre-render every track to OGG/FLAC against a chosen `SoundFont` ahead of time (lowest implementation risk, but a licensing/distribution question for the rendered assets and the `SoundFont` itself); expose and verify a reusable CNA-level MIDI route over the available decoder (cleaner for future games, but scope beyond what one port should decide unilaterally); or keep a *Free Eggbert*-specific MIDI service bridging through `free-api`'s own existing `TinySoundFont` path while every other subsystem moves to CNA. Its staged recommendation uses the game-specific bridge for an initial, behavior-preserving port, option one (pre-rendered assets) for a final low-maintenance release if asset policy permits it, and option two (a real CNA-level MIDI path) only if neither of the other two is acceptable.

75.9.5 Networking reconstruction defect

`CNetwork::Receive` reads into a local 500-byte stack buffer but never copies the data to the caller's destination. This source-proven defect makes the current decompiled reconstruction an unreliable behavioral specification for networking. Future multiplayer work needs a versioned wire format and two-process tests against intended behavior, not an attempt to preserve this function's omission.

75.9.6 Renderer progression and licensing boundary

The recommended sequence starts on `SDL_RENDERER` for focused portable 2D validation, then builds the same port against `FREEDIRECT` as a differential bridge between the old and new

public APIs. An independent `OPENGLES3` run supplies evidence beyond one renderer. Part [IV](#) describes those identities and their implementation routes. The feasibility document predates the renderer naming migration and calls this identity `EASYGL`; it names the same shared implementation family, not a still-selectable public value. `FREEDIRECT` is a migration tool, not the target architecture: a build linking it has adopted CNA's public API but retains the DirectDraw-era dependency.

Free Eggbert declares GPL-3.0, while CNA declares Ms-PL. Combining and distributing them in one linked program requires a licensing review. Neither the feasibility document nor this book makes that legal determination.

75.10 Porting implications

The call-site method produces a narrower compatibility layer than a speculative full-surface implementation, but one aligned with the two target games. It exposed details that an API-only reading could miss, including `GetDC`'s gameplay role and the `PCMWAVEFORMAT` layout requirement. For a native CNA port, the same method identifies stable seams, timing assumptions, evidence gaps, and licensing questions before code is moved.

Chapter 76

Auditing xna4-spec Compatibility

`xna4-spec` is an offline, machine-readable conversion of Microsoft’s XNA 4.0 documentation. It is useful for asking what the documented API surface contains. It is not part of CNA’s build, test, or oracle chain, and it is not the source of CNA’s dated 227-of-245 FNA type count; that number was produced by diffing FNA’s public type list against CNA headers.

76.1 What the repository contains

At pinned commit `8f61207`, the top-level index references 544 XML type files across 19 documented namespaces. Each file records a kind, syntax, members, overloads, parameters, return text, remarks, and links derived from the former Microsoft documentation archive. The collection includes runtime namespaces and the content-pipeline namespaces CNA does not aim to implement.

The README’s example tree says “550 types,” while its coverage table, index, plan numbering, and files all say 544. This small internal drift is the first rule of using the corpus: treat even a machine-readable conversion as data to validate, not as an infallible executable specification. The repository’s disclaimer also makes clear that the converted documentation remains Microsoft’s content and is offered for informational and educational use.

76.2 Where it sits in CNA’s authority hierarchy

No file in CNA references `xna4-spec`. CNA’s current port checklist instead names the XNA reference-assembly XML as the authority for API surface and FNA as the reference for behavior. Real XNA execution, FNA differential values, and image oracles then settle questions documentation cannot.

`xna4-spec` is therefore a convenient complementary inventory. It can expose a missing property that FNA itself omitted, supply remarks for manual review, and make namespace-wide audits scriptable. It cannot prove CNA behavior, and a clean comparison against it is not an existing CNA gate. A book or report must not promote an offline sibling repository into a runtime authority merely because its name contains “spec.”

76.3 A reproducible surface audit

For one type, a disciplined comparison has six steps:

1. pin the `xna4-spec`, CNA, FNA, and reference-assembly revisions;
2. extract fields, properties, constructors, methods, events, and overload signatures from the XML rather than counting prose mentions;

3. map C# idioms explicitly: properties to getter/setter names, operators to C++ operators, `ref/out` to reference overloads, and iterators to CNAEXT;
4. compare CNA's public header and classify each row as exact surface, accepted language adaptation, marked extension, unexplained extra, or missing;
5. read remarks and implementation for every hand-written accessor or behavior-bearing method;
6. attach an executable oracle for semantics and record cases the XML cannot decide.

The mapping table is part of the result. Otherwise `Equals(object)` looks like a missing member even though a C++ value type has no universal boxed root, and a default constructor or iterator can look like an unmarked XNA extension even when it exists only for the target language's ordinary value/container idiom.

76.4 Auditing Ray

`Microsoft.Xna.Framework/Ray.xml` describes the `Position` and `Direction` fields, the two-vector constructor, equality, hash and string methods, intersection overloads, and equality operators. CNA's `modules/math/.../Ray.hpp` contains both fields, the constructor, value equality, `ToString`, four intersection target types, output-reference forms for box, sphere and plane, and free equality operators.

The comparison also exposes adaptations. CNA adds a default constructor, omits boxed `Equals(Object)`, and returns `std::size_t` from `GetHashCode` rather than C# `int`. Those choices match CNA's documented C++ conventions; they should be classified, not silently counted as either perfect matches or defects.

The converted XML itself needs normalization. Its `Intersects` list contains duplicate output overload records, and some signatures are labelled with the wrong target type even though the surrounding parameter text names `Plane` or `BoundingSphere`. A script that compares signature strings blindly will invent duplicates and gaps. Cross-checking the reference-assembly member IDs or the original documentation is necessary whenever converted fields disagree.

76.5 Auditing Viewport

The Viewport XML lists two non-default constructors, nine properties, and `Project`, `Unproject`, and `ToString`. CNA's header provides those members through a mix of generated field properties and hand-written accessors. It also has a default constructor that produces zero width/height and a 0-to-1 depth range.

The surface comparison looks strong, but `TitleSafeArea` demonstrates why surface is not semantics. CNA returns `Bounds` unchanged and pins that choice with `TitleSafeAreaEqualsBounds`. The documentation describes a subset guaranteed visible on lower-quality displays. A passing test proves CNA's chosen behavior is stable; it does not prove the choice matches XNA on every platform.

The right outcome is a recorded behavioral deviation or a real-XNA experiment, not a surface-coverage percentage. Hand-written properties deserve particular scrutiny because computation and policy can hide behind a signature that compares perfectly.

76.6 What inventories cannot prove

An XML API inventory cannot establish default values produced by a runtime, floating-point edge behavior, exception type and timing, lifetime, GPU state, wire compatibility, content decoding, or platform substitution. It also cannot tell whether a nominally present method is a stub, a safe refusal, or a complete implementation.

Use `xna4-spec` to generate questions and completeness rows. Use CNA's compile-time CNAEXT gate to detect unmarked additions, reference assemblies to settle surface, FNA or real XNA to settle semantics, and the oracle hierarchy from Chapter 70 to produce verdicts. The most trustworthy audit is not the one with the highest percentage, but the one that preserves these authorities instead of collapsing them into one number.

Chapter 77

cna-samples and cna-examples

The two sibling repositories answer different questions. `cna-samples` translates Microsoft’s historical XNA sample programs and exposes compatibility pressure. `cna-examples` is one CNA-native catalog application designed to demonstrate the current API. Neither is a substitute for CNA’s tests, and neither repository’s status prose should be read without its pinned revision and evidence type.

77.1 cna-samples: ports with provenance

At commit [149a19f](#), the repository tracks the official XNA Game Studio 4.0 collection under the Microsoft Permissive License. It expects CNA and sharp-runtime as sibling directories and builds each completed sample as an independent executable.

Its plan accounts for 153 source-archive directories in three populations: 63 completed ports, 23 tracked placeholders, and 67 explicitly ignored entries. The filesystem corroborates the first two numbers: 86 sample directories each carry a `missing.md`, and 63 contain a sample CMake target. The ignored set covers tools, redundant training material, platform/service-bound programs, and other declared non-targets rather than pretending every archive directory is a portable game.

These are catalog statuses, not fresh run results. “Done” means real port source and a build target exist; it does not mean pixel equality, all original scenes, or all current CNA renderers. The plan itself records the instructive SimpleAnimation correction: an entry already marked done later revealed two CNA graphics defects when re-reviewed. A sample is excellent integration pressure precisely because its behavior crosses boundaries that unit tests may isolate away.

77.2 Read each missing file as a dated case record

Every tracked sample’s `missing.md` explains whether it is complete, simplified, blocked, or deliberately scoped. These files record concrete asset substitutions, content-reader gaps, hardware fallbacks, and framework defects found during translation. They are better evidence than a single top-level ratio because they name the affected program and mechanism.

They also age. The pinned top-level plan still says XNB is unsupported, while CNA now has a real XNB stack. Some historical paths point to repositories or layouts that no longer exist. A placeholder must therefore be re-tested against current CNA before it is counted as still blocked. The right workflow is: read the old reason, locate the current API and test, then either port the sample or replace the reason with a new, evidenced boundary.

77.3 Why official samples remain valuable

An official sample combines API calls in the order and lifecycle Microsoft taught users to rely on. That makes it good at finding defects such as default-state mismatch, model winding, component timing, asset transformation, or a renderer path that works in isolation but not in a whole game.

It is not automatically an independent correctness oracle. A C++ translation can misread the C# source, replace assets, skip a scene, or work around a CNA gap. The strongest use pairs the port with the original sample or another implementation, preserves inputs, and compares screenshots or traces. Only two entries in the plan have direct official MonoGame sample counterparts, so that differential opportunity is narrower than the catalog size suggests.

77.4 cna-examples: one navigable CNA-native application

At commit `fa0b49a`, `cna-examples` contains one application with 13 areas, 79 categories, and 249 registered demo screens. A read-only run of its `tools/check_catalog.py` at that pin confirms every screen is defined, registered exactly once, and documented. Areas span framework, math, content, storage, diagnostics, input, audio, devices, networking, media, avatars, and 2D/3D graphics.

The application offers menu navigation, substring search, direct `-demo` selection, scripted actions, bounded frame counts, and screenshots. This makes a screen useful both to a human exploring the API and to a headless sweep. Global state restoration matters because all screens share one process; the Framework sweep explicitly checks that a resolution-changing demo returns the back buffer to its prior size.

77.5 A screenshot sweep needs semantic companions

The repository distinguishes several verification forms:

- real hardware interaction for keyboard, mouse, and much of audio;
- Xvfb screenshot/behavior sweeps for Framework and Media;
- C++ claim programs for Math, CNJ, and XACT statements rendered on screen;
- self-checking 3D screens that turn readback or occlusion results into a visible verdict;
- graceful explanatory screens for unavailable controller, touch, sensor, camera, or dialog hardware.

This separation prevents a caught exception from painting a clean explanatory screen and being mistaken for a successful API demonstration. A screenshot checker can detect blank output or overflow; a claim program or self-check must decide whether the described operation was correct.

77.6 The pinned documentation uses obsolete renderer names

`cna-examples` predates CNA's renderer terminology migration. Its README and build examples use the retired backend-named option with internal value `EASYGL`; current CNA uses `CNA_GRAPHICS_RENDERER` with `OPENGLES3` or another public identity. The repository's recorded 249-of-249 EasyGL sweep refers to the implementation family now selected publicly as an EasyGL- backed identity. It must not be copied as a current configure command.

The SDL-renderer sweep remains conceptually useful: its 2D-only capability causes 3D demos to explain their unavailability rather than throw. Yet the recorded sweep is a dated result, not a run performed during this book audit. Revalidation should update selector names first and retain screenshots plus tool output.

77.7 Use both repositories as triangulation

For learning, start with **cna-examples**: one build, searchable screens, current CNA-shaped code, and explicit evidence helpers. For migration risk, inspect **cna-samples**: original program provenance, per-sample scope records, and failures driven by realistic combinations.

For framework confidence, make the two disagree productively. If a CNA-native screen passes but an official port fails, inspect translation and lifecycle. If both fail across one renderer but pass another, localize the renderer. If both pass yet differ from XNA, bring in a real-XNA or FNA oracle. Examples demonstrate; samples integrate; tests and independent references decide.

Chapter 78

Project Practice and Durable Task Tracking

CNA’s repository is rich in plans, handoffs, audits, remediation ledgers, and integration records. That abundance is valuable and dangerous: an old plan can explain why code exists while being wrong about whether it exists today. Durable practice is therefore not “trust the tracking document.” It is preserving a chain from intent to source, test, artifact, and current status.

78.1 Documents have roles, not equal authority

The root `NEXT.md` is a session and integration handoff. At alpha.1 it opens with the 2026-08-18 merge and compiled-effect follow-up, while thousands of lines of earlier renderer, platform, C API, glTF, and remediation history remain below it. It is operational history, not a normalized single source of current feature truth.

`CHECKLIST.md` defines repeatable port rules: API surface comes from XNA reference assemblies, behavior from FNA, extensions require CNAEXT, and each public member needs tests or an explicit host-dependent exception. It is a policy document; compliance still has to be checked against code and gates.

`AUDIT.md` is a large historical class table with a current C-ABI tracking banner above older per-class rows. It contains valuable provenance and many dated claims; current banner text does not automatically refresh older rows. Treat it as a claim corpus that must be checked against the tag, not a normalized current API inventory. The same warning applies to source-audit files generated before modularization.

78.2 Subsystem plans preserve causality

The CNA root contains 62 `plan_*.md` files at the alpha.1 tag, not a small fixed set. Their numbered tasks record rationale, experiments, owner decisions, defects discovered while doing the work, and follow-up boundaries. A stable identifier such as `GLTF-245` or `REMED-GFX-188` is useful because commits, tests, and later tasks can point back to the same decision.

A done marker is still a claim. Several audits found tasks marked complete because code existed, not because their acceptance case ran; others found plan banners saying nothing was implemented above sections describing completed phases. Before quoting a task, check its named path, test, commit, and present behavior. If any one moved, update the forward pointer rather than erasing the old identifier.

When a task expands across independent renderers or failure modes, split it. The parent becomes a tracking pointer and each child receives its own acceptance evidence. This keeps a partial fix from turning a broad row green and lets controls finish independently.

78.3 Integration records need immutable inputs

The renderer-integration campaign demonstrates a stronger form of handoff. It records every feature lane's exact SHA, derives merge order from path interaction, preserves signed no-fast-forward merges, recomputes registry arithmetic, and names conflicts whose correct result could not be taken from either lane alone. A later reader can reconstruct which trees were combined and why the result differs from each parent.

For work of this scale, a useful checkpoint packet contains:

- base and input SHAs, branch roles, and proof no hidden worktree state was consumed;
- path-overlap and ordering rationale;
- exact configure/build/test commands with renderer and host identity;
- machine-readable results, skip counts, logs, screenshots, and generated manifests;
- newly found defects separated from the requested change;
- a clean-tree/diff check and a handoff naming the next bounded step.

A prose statement that “all lanes passed” cannot replace this packet, especially when one lane was only compile-checked or one renderer ran under a substitute runtime.

78.4 Executable ledgers resist prose drift

The glTF conformance work keeps known defects in both documentation and tests, then a meta-test asserts that the two sets agree. Skia validators compare parity ledgers and decisions with live interfaces and enums. Renderer identity gates compare the CMake and C++ registries. Module probes inspect link closures.

These mechanisms encode a general rule: if a plan row names a finite set that code can derive, derive and compare it. Check row uniqueness, accepted status vocabulary, set equality, path existence, and non-empty parsing. Keep negative fixtures and mutation so the validator can prove it goes red.

Executable documents can rot too. Four wired validators at the pin read paths removed by modularization. A gate that cannot find its input is not current evidence. Self-validation must include its input mapping, and a physical source move should update implementation, tests, validators, and cited docs atomically.

78.5 SemVer and phase tracking coexist

The first tagged release adds a second durable coordinate without replacing task history. Product version authority begins in the root CMake numeric version plus its prerelease field; configuration generates `CNA/Version.hpp`, and release preparation synchronizes Doxygen, the changelog, and the `v0.1.0-alpha.1` tag. `CNA::getVersionString()` and the `CNA_VERSION_*` macros expose that product version to C++ consumers.

The experimental native C package has a separate ABI version in `CNA/C/abi.h`. Its package compatibility and release gate must move according to C ABI rules, not because the C++ product tag changed. A release record is therefore complete only when it names both coordinates where the C surface is in scope. Alpha.1 remains pre-1.0: SemVer identifies the snapshot and communicates prerelease status; it does not promise a stable 1.0 API.

The release checklist in `docs/releasing.md` treats source, changelog, generated version API, tag, and verification evidence as a coordinated operation. Phase/task identifiers remain the causal engineering ledger underneath that snapshot. A task answers *why and how*; the tag answers *which immutable product state*.

78.6 sharp-runtime's database makes scope measurable

The sharp-runtime sibling stores coarse classifications and concrete tickets in `plan.sqlite3`. At pinned commit `f827a6c5`, the `task` table contains exactly 16,201 rows: 1,082 `ported`, 14,979 `ignored`, and 140 `ignore`; 118 rows also carry `outofscope=1`. The `ticket` table contains 2,183 work items with descriptions, acceptance criteria, priorities, estimates, status, and optional validation commands.

The denominator matters more than the apparent percentage. The database deliberately classifies vast areas of .NET that sharp-runtime will never implement. “1,082 ported” means rows with a working C++ counterpart under this schema; it is not a claim of 6.7% behavioral parity with the whole BCL. Doxygen `@note Status:` tags are secondary human hints and can drift from the database.

The query should travel with the number:

```
SELECT status, COUNT(*) FROM task GROUP BY status;
SELECT COUNT(*) FROM ticket;
```

This makes a future change visible and prevents an old book number from becoming an unexplained project constant.

78.7 Separate observations, inferences, and decisions

A robust task record distinguishes three things. An observation is directly reproducible: “this shader compiler path imports a missing module.” An inference connects evidence: “modularization left the workflow validator stale.” A decision sets policy: “the release gate must fail closed on a missing manifest.”

Mixing them causes plans to outlive their premises. A decision can remain valid after the cited bug moves; an observation can become false after a fix; an inference may need revisiting when a new control run appears. Marking the category makes later re-audit cheaper.

78.8 Handoff is part of implementation

An effective handoff states what is complete, what was verified, what remains, which repositories are read-only, and which actions require external approval. It includes failed approaches whose repetition would waste time and identifies known-stale documents that must not be used as status sources. It does not paste an ever-growing transcript into one file.

The final authority order is practical: current source and generated configuration; executable tests and retained artifacts; accepted integration/remediation records; then plans, audits, READMEs, and comments as context. Comments often preserve the best explanation and the worst status. Durable project practice keeps both facts visible by linking explanation to something a future maintainer can rerun.

Chapter 79

Roadmap, Open Gaps, and How to Re-Audit This Book

This roadmap is a snapshot against `v0.1.0-alpha.1 @ 1bb2145d` and the sibling revisions pinned in Appendix D. It deliberately avoids one global completion percentage. API presence, behavior, renderer evidence, platform execution, and automation are different axes, and several CNA documents carrying headline percentages are already stale about XNB, glTF, renderers, or tests.

79.1 Triage correctness before expanding breadth

The ledger now records 60 CNA findings in `cnabugs.md`: the original 54 implementation, capability, gate and documentation findings plus six alpha.1 release/build contradictions. Nine original findings are fixed in tag source, several are narrowed, one is an explicit architecture decision, and the rest remain open. The dispositions are source-derived rather than a 60-case runtime reproduction, so focused reproduction remains the first upstream step.

High-value starting points have small counterexamples and broad consequences: aliased `Matrix::Transpose` corrupts `Plane::Transform`; `StorageContainer` lacks path containment; the CNJ parser has no depth bound; the main CI workflow cannot configure; several capability queries promise operations their renderer refuses. Each should receive a minimal failing test, a scoped fix, controls on unaffected renderers, and a current evidence record.

The audit list must not become a second stale bug tracker. Once triaged upstream, rows need an issue or commit pointer and a status tied to a reproducer. Findings rejected as intentional behavior should move into the documented-deviation contract rather than disappear.

79.2 Complete capability truth before adding renderers

CNA already exposes 50 identities across 46 families. The next renderer milestone is not a larger count; it is truthful answers from the existing contract. Alpha.1 corrected the recorded HEADLESS/SOFTWARE MRT claims, but WebGPU's MRT route and several query reports still disagree with their implementation; WebGPU and bgfx overclaim custom effects; several stencil reports remain inverted; DIRECTX10 inherits an unconditional colored extended-draw downgrade.

Build a table-driven capability conformance suite that pairs every `true` with a usable factory/-draw case and every `false` with a deterministic refusal. Keep capability tests renderer-owned where one configure can exercise them and retain cross-renderer public fixtures as controls. A registry count cannot substitute for this behavioral matrix.

79.3 Close the arbitrary-effect boundary deliberately

The public compiled-effect constructor and general XNB `EffectReader` now accept bounded XNA/FNA Direct3D 9 Effect Framework binaries. FNA3D enables the route normally; SDL_GPU, EasyGL and Vulkan require opt-in MojoShader-backed builds. D3D9's separate stock-source comparison and `ShaderEffect`'s renderer-specific input remain different routes.

Further work is breadth and conformance: more renderer runtimes, hostile-binary hardening, real-game fixture coverage, and renderer-engaged parity for parameters, techniques, passes, samplers, textures, pass state, SpriteBatch and 3D draws. BGFX and many other families still have no compiled-effect runtime; MGFX, DXBC and source compilation remain out of this constructor's contract. Do not hide that matrix behind one Boolean.

79.4 Harden content symmetrically

XNB support is real and substantial. Remaining work includes whole-file size enforcement, named unsupported reader/version policy, LZ4's explicit deferred status, writer/tooling decisions, and continued hostile-input tests. CNJ needs the same depth/size/fuzz discipline already applied to XNB; its current recursive parser is an obvious first target.

The façade has additional contract work: `Unload()` disposes nothing, `CanDeserializeIntoExistingObject` is never consulted, Song/Video can resolve a same-name CNJ they cannot parse, and manifest data is introspection rather than resolution. Fixes should retain the separation between caller-selected asset paths and file-internal reference containment.

79.5 Complete glTF semantics beyond importer coverage

The tag closes the old UV-report, required-extension and PBR morph-stride findings and now exposes losses through `GltfImportReportEXT`. Remaining work is the explicitly bounded GLTF ROBUST matrix: broader renderer consumption of the newer PBR state, real point and spot lights through a shared light ABI, and a wider external-renderer image campaign. KTX2/Basis and WebP without fallback remain unsupported; carried-but-application-owned state and finite vertex-layout ceilings remain documented rather than silently discarded.

Advance one independently generated fixture at a time. Update the manifest, scene digest, semantic golden, runtime/offline parity, and renderer evidence together. Keep the pinned external validator audit distinct from deterministic generator self-checks; alpha.1 contains both, and neither replaces renderer-engaged evidence.

79.6 Turn platform claims into retained artifacts

The adapted Android pin needs a fresh graphics build and emulator frame before it receives a current renderer claim. Retain the APK, configure log, logcat, screenshot, ABI/API/NDK identity, and lifecycle trace. Then cover background, foreground, low-memory, termination, asset access through ContentManager, and physical hardware.

The tag's Apple workflow source contains macOS build/app launch plus iOS device final-link and simulator final-link/install/launch legs with a one-frame marker. Those definitions are stronger than compile-only project files, but this book did not observe a tag run; retain artifacts before promoting them to current execution evidence. Native Metal still needs retained validation, pixels, Retina, and frame-pacing evidence. tvOS remains an explicit non-claim. We-

bGL1, WebGL2, Canvas and PixiJS need HTTP-served browser verdicts, context-loss coverage, and shader/runtime evidence where applicable.

For Windows, retain the MinGW/Wine route while adding genuine native MSVC evidence where the public support table claims it. Every result must name translation runtime, GPU/-software device, display, and prefix. A compatibility-layer pass is valuable and is not a native-Windows pass.

79.7 Repair the verification apparatus itself

Replace obsolete EASYGL workflow selectors with public identities and make the full Linux and sanitizer legs reach compilation. Update the four post-modularization validator paths, run the Direct2D mutation campaign rather than only anchor dry-run, fix Device filter drift, and decide one authoritative direct-binary-versus-CTest policy for the full suite.

Label all renderer registrations consistently or publish generated selectors that do not depend on labels. Make link-closure evidence generator-independent instead of silently skipping under the project's own presets. Bring SVG-DOM's existing browser runner into automation and ensure the release gates, not only their non-release validators, are exercised.

Add code-coverage tooling only with a defined question. Line coverage can identify untouched code but cannot replace oracle authority, mutation sensitivity, or runtime engagement. Keep API-surface "coverage" under a different name so the metrics cannot be confused.

79.8 Extend packaging beyond the native C boundary

Alpha.1's optional C API defines shared/static packaging, a version file, exported C headers, and an out-of-tree `find_package(CNA CONFIG)` consumer gate, but the tag cannot build the library because NanoVG is absent from its C renderer map. First repair that identity closure, add a CI leg that builds the final library and installed consumer, and make the aggregate release checker execute rather than recognize those gates. The broader C++ framework is still designed around sibling `add_subdirectory` composition and has no complete installed module/renderer SDK. A mature C++ consumer path would need installed module targets, public headers, renderer selection, dependency discovery, product-version compatibility, and its own small out-of-tree consumer test. It must not reuse the native C package's ABI 0.7.0 as though that were the product or C++ API version.

Packaging work must preserve the one-SDL-instance invariant and optional-module closure. It should not force every renderer dependency onto a consumer that selects one identity or a small compatible runtime set.

79.9 Reconcile documentation from generated facts

Remove or banner claims that CNA cannot read XNB, that WebGL2 or Android graphics is pixel-verified, that the implementation-family count means public identities, or that old source paths still register gates. Generate renderer identity/family tables, reader inventories, workflow summaries, and test-selection tables from the same sources their validators use.

Dated reports may remain for history if their revision and superseding authority are prominent. Comments should explain why; scripts and tests should establish whether. The fastest way to create new drift is to copy a corrected number into five prose files without a derivation.

79.10 How to re-audit this edition

A future edition should repeat a bounded protocol:

1. pin CNA and every cited sibling SHA; record branch divergence and keep them read-only;
2. enumerate physical modules, public renderer identities/families, build options, workflows, scripts, test registration paths, content readers, and sample/example catalogs mechanically;
3. re-run focused source audits by subsystem and record contradictions separately from defects;
4. verify every explicit path and identifier in the manuscript against the pin;
5. rebuild the complete book, require clean references/index/stale-term checks, render every changed physical page, and inspect high-risk tables and listings at full size;
6. update `PLAN.md` and `NEXT.md` with commands, counts, page ranges, visual findings, fixes, and the next bounded batch.

The edition should change when evidence changes, not merely when prose ages. The final lesson of CNA's own verification culture applies to this book as well: a claim is durable only when a future reader can prove that the intended source, runtime, oracle, and page actually engaged.

Appendix A

Core Framework and Graphics API Reference

This appendix is an orientation map to the public core and graphics surface at the pinned CNA revision. It intentionally groups overload families instead of reproducing declarations: the module-owned headers remain the signature authority, while the cited chapters own behavioral and renderer-specific qualifications. A listed method proves API presence only; it does not imply that every one of the 46 implementation families renders it with the same semantics.

A.1 Math and Geometry Types

Type group	Public shape	Boundary to remember
Vector2, Vector3, Vector4	Public components, constants, arithmetic, distance/dot, interpolation, normalization, reflection, and matrix/quaternion transforms; Vector3 adds cross product.	Value-returning and output-reference overloads are independent API rows and need independent tests.
Matrix	Sixteen public elements, direction/translation properties, decomposition, arithmetic, inversion/transposition, and the XNA creation families.	CNA uses row-vector conventions; ToColumnMajor is a marked bridge for shader upload, not a change of the public convention.
Quaternion	Construction from axis/matrix/yaw-pitch-roll, arithmetic, concatenate, conjugate, inverse, lerp and slerp.	Quaternion multiplication order and normalization are behavioral contracts, not naming details.
Point, Rectangle	Integer screen-space coordinates, containment, intersection/union, offset and inflation.	Rectangle edges and empty/intersection cases use exact XNA/FNA boundary rules.
Color	Packed RGBA value, named colours, vector/float/integer constructors, interpolation and premultiplication helpers.	Packed storage is AABBGRR ; constructor type changes can alter overload and clamping behavior.

Type group	Public shape	Boundary to remember
Plane, Ray, BoundingBox, BoundingSphere, BoundingFrustum	Shape construction, containment, intersection, corners and transform-related queries.	C++ optionals/output references translate nullable and out forms; compare semantics, not signature strings alone.

Chapter 7 covers value semantics and Chapter 7 covers coordinate, matrix, viewport and vertex-layout conventions. The xna4-spec comparison in Chapter 76 shows why generated XML is useful inventory but requires language normalization and behavioral oracles.

A.2 Game Framework and Content

Type	Primary surface	Current qualification
Game	Lifecycle events; components, content, services, window and device; fixed/variable timing; <code>Run</code> , <code>Tick</code> , <code>Exit</code> , and the <code>initialize/load/update/draw</code> hooks.	Host ownership differs on native, Emscripten and mobile paths; the call order is the contract, not merely the presence of virtual methods.
GameTime	Total and elapsed time plus the slow-frame flag.	Mutable by the framework; user code consumes the snapshot.
GameWindow	Bounds, title, <code>resize/orientation/display</code> events and <code>screen-device-change</code> protocol, plus marked <code>SDL/native-window</code> conveniences.	CNA uses one <code>SDL3</code> -backed concrete class; lifecycle/platform behavior remains host-specific.
GraphicsDeviceManager	Preferred <code>backbuffer/depth/profile/fullscreen</code> settings, <code>ApplyChanges</code> , device creation/preparation events, and presentation-mode extensions.	Stored presentation values are requested state, not proof the driver/window accepted them.
ContentManager	Root directory, service provider, generic <code>Load<T></code> , <code>Unload</code> , <code>reader/CNJ</code> registration, manifest and reader-usage extensions.	Resolution is XNB first, then a registered loose reader that may select <code>CNJ</code> or a native format; cache and ownership vary by asset type.

The exact game-loop order is in Chapter 6. Chapters 33 through 36 separate path resolution, XNB container/object graph, and `CNJ`; a successful lookup in one route is not evidence for the

others.

A.3 Graphics Core: Resources and Batching

Type group	Public shape	Evidence boundary
Texture2D	Dimensions/bounds, whole and rectangular SetData/GetData , stream decode, PNG/JPEG save, and marked pixel helpers.	A round trip can observe CNA's CPU shadow rather than sampled GPU memory; mip/subregion behavior is renderer-specific.
Texture3D, TextureCube	Volume or face dimensions and level/subregion upload/readback families.	Construction, format, mip, upload and readback support are separate capabilities; no CPU shadow provides a universal fallback.
RenderTarget2D, RenderTargetCube	Texture surface plus depth/stencil, multisample and usage metadata; cube adds face selection.	Bind, preserve/discard, plural targets, cube unbind and readback have independent renderer paths.
VertexBuffer, IndexBuffer	Counts/usage/declaration plus typed and sliced upload/readback; dynamic subclasses add SetDataOptions .	The constructor's dynamic intent, upload hint, CPU shadow and renderer buffer are four distinct layers.
SpriteBatch	Begin/End , texture Draw overloads and DrawString for strings/StringBuilder.	Sort mode, state objects, custom effect, transform and clipping must reach the selected renderer; a visible sprite alone does not prove every argument.
SpriteFont	Atlas, glyph/cropping/character/kerning arrays, spacing/default character, and measurement.	XNB, CNJ and hand-built fonts enter through different reader/ownership routes.

Chapters 14 and 13 own the detailed behavior. Appendix B gives the implementation-family matrix; do not infer a cross-renderer guarantee from the common C++ type alone.

A.4 Model and Stock Effects

Type group	Public shape	Boundary
<code>Model</code> , <code>ModelBone</code> , <code>ModelMesh</code> , <code>ModelMeshPart</code>	Bone and mesh collections, root/parent hierarchy, transform-copy helpers, part buffers/declaration, effects and draw.	Four loading routes have different provenance; <code>Model::Draw</code> requires matrix-capable effects and uses serial shared scratch state.
<code>BasicEffect</code>	Matrices, texture/vertex colour, material colours, fog and three directional lights.	Renderer-owned shader/fixed-function mapping decides which fields are consumed.
<code>AlphaTestEffect</code>	Texture, vertex colour, fog, alpha comparison and reference value.	A draw can succeed while a fallback loses alpha-test semantics.
<code>DualTextureEffect</code>	Two texture layers, matrices, vertex colour and fog.	The second layer needs a renderer route and compatible packed layout.
<code>EnvironmentMapEffect</code>	Diffuse texture, cube environment map, amount/specular/fresnel, matrices, fog and lights.	Cube construc- tion/sampling and effect dispatch are separate gates.
<code>SkinnedEffect</code>	Matrices/material/texture/fog/lights plus bone palette and 1/2/4 weights per vertex.	Maximum palette/layout and animation binding are runtime contracts; CNA's vertex-colour addition is marked CNAEXT.

Chapter 38 covers runtime ownership, Chapters 39–43 cover glTF/CNJ, and Appendix H keeps importer, runtime and test evidence separate.

A.5 GraphicsDevice and State Objects

`GraphicsDevice` is the renderer-independent dispatch facade. Its public groups are:

- adapter/display/profile/status/presentation identity and disposal state;
- texture, sampler, vertex-buffer, index-buffer and render-target binding;
- blend, depth/stencil, rasterizer, viewport, scissor and related scalar state;
- clear, present, backbuffer readback and screenshot helpers;
- bound-buffer, user-array, indexed and instanced draw families;
- capability queries, unsupported-3D policy, and marked diagnostic/recovery helpers.

The facade stores and returns state even where an implementation family does not consume it. The pin also contains concrete asymmetries in singular/plural bindings, offsets/frequencies, readback source, clear masks, presentation interval/format and capability truth. Therefore use

the operation-level matrices in Chapters 12 and 19; `SupportsCapability(true)` is a preflight hint, not an execution artifact.

The four state objects remain distinct:

Type	State represented
<code>BlendState</code>	colour/alpha factors and equations, write masks, blend factor and presets
<code>DepthStencilState</code>	depth enable/write/compare and front/back stencil operations
<code>RasterizerState</code>	cull/fill, bias, multisampling and scissor-enable intent
<code>SamplerState</code>	addressing, filtering, anisotropy, mip limit and LOD bias

CNA copies assigned state objects by value. Later mutation of the original object does not retroactively modify the device copy. Whether each copied field reaches native state still depends on the renderer family.

A.6 Effect Base, Parameters, and Queries

`Effect` owns parameter and technique collections, a current technique, cloning, and the CNA-added convenience that applies the current pass. Its bytecode constructor accepts the XNA/FNA D3D9 Effect Framework payload supported by the selected renderer; FNA3D is the normal alpha.1 route, while `SDL_GPU`, `EasyGL` and `Vulkan` require their explicit experimental effect options. `EffectParameter` exposes typed scalar/vector/matrix/string/texture setters and getters plus class/type/dimension metadata and nested collections. This is not a source compiler and does not accept `.fx/HLSL`, `DXBC` or `MGFX`. `ShaderEffect` is a distinct renderer-specific source/program route. The existence of a stock effect or parameter setter therefore does not prove compiled-effect execution for a renderer absent the matching capability and factory.

`OcclusionQuery` exposes `Begin`, `End`, completion, and pixel count. Appendix B identifies families with real, conditional, trace-only, null, or misreported query paths. A non-null public wrapper and a true capability bit are insufficient without a usable renderer object and a discriminating sample-count test.

A.7 Reading this appendix safely

For exact declarations, search the selected type under `modules/<owner>/include/Microsoft/Xna/Framework/` at CNA v0.1.0-alpha.1 @ 1bb2145d99ed572dd4eb15009c34e2e5f410fcf0. Then pair the header with its implementation, renderer route, tests and host evidence. This appendix answers “where is the surface?”; the book’s chapters and retained audit reports answer “what does this revision actually do?”

Appendix B

Renderer Registry, Capabilities, and Evidence

CNA exposes **50 public renderer identities / 46 implementation families** at the pinned revision. The four-count difference is exact: five EasyGL profiles share one implementation family, while every other identity has its own family. This appendix derives the registry from `docs/renderer-registry.md` and the source-owned factory paths, then reports a narrow set of capabilities from reachable implementation behavior. It does not copy the older central feature matrix, whose names and cells predate the current registry.

B.1 Canonical public identities

Cluster	Accepted CNA_GRAPHICS_RENDERER values	Count
OpenGL fixed function	OPENGLES1, OPENGL1, TINYGL	3
OpenGL programmable / portable	OPENGLES2, OPENGLES3, OPENGL33, WEBGL1, WEBGL2, OPENGL2, OPENGL4, PORTABLEGL	8
Native modern GPU	VULKAN, WEBGPU, SDL_GPU, METAL, IGL	5
Abstraction layers	BGFX, MAGNUM, LLGL, DILIGENT, SOKOL, WICKED, FNA3D	7
DirectX 1–3 / free-direct	DIRECTX1, DIRECTX2, DIRECTX3, FREEDIRECT	4
Retro ladder	DIRECTX5, DIRECTX6, DIRECTX7, DIRECTX8, GLIDE	5
Modern Direct3D	DIRECTX9, DIRECTX10, DIRECTX11, DIRECTX12	4
Windows 2D	DIRECT2D, GDI	2
2D / vector rasterizers	SDL_RENDERER, SKIA, BLEND2D, OPENVG, NANOVG	5
Web / DOM	CANVAS, HTML_DOM, SVG_DOM, PIXIJS	4
Diagnostic / non-GPU	HEADLESS, SOFTWARE, STUB	3
Total	50 identities / 46 families	50

ASCII is no longer a renderer identity; its reusable work lives in `CNA::Graphics::AsciiPostProcessEffect`. `EASYGL`, `DX3`, `D3D9`, `D3D11`, and `D3D12` are not accepted selectors. `FREEDIRECT` selects the free-direct-backed family; `DIRECTX3` selects the genuine DirectX 3 family. Defaults are `WEBGL2` under Emscripten, `OPENGLES3` on Linux, and `SDL_RENDERER` elsewhere.

B.2 Four implementation questions across all 46 families

The next matrix deliberately asks only four questions that can be answered uniformly from the pinned source. “Native” means the family owns both ordinary effect-aware **Draw*Ex** routes; it does not promise complete stock-effect fidelity. “Fallback–reject” means the shared parameter-dropping fallback reaches an explicit 3D refusal. “Hybrid” means a renderer-owned route still has a bounded coloured fallback. “Conditional” names a real profile/device/extension dependency. “Claims yes” exposes a public capability answer that contradicts the reachable factory or operation.

Family	Public identity / identities	Effect-aware draw	RT2D	MRT	Occlusion query
bgfx	BGFX	native	yes	yes	conditional
blend2d	BLEND2D	fallback–reject	yes	no	no
canvas	CANVAS	fallback–reject	yes	no	no
diligent	DILIGENT	native	yes	yes	conditional
direct2d	DIRECT2D	fallback–reject	yes	no	no
directx1	DIRECTX1	fallback–reject	yes	no	no
directx2	DIRECTX2	native fixed	yes	no	no
directx3	DIRECTX3	native fixed	yes	no	no
directx5	DIRECTX5	native fixed	yes	no	no
directx6	DIRECTX6	native fixed	yes	no	no
directx7	DIRECTX7	native fixed	yes	no	no
directx8	DIRECTX8	native fixed	yes	no	no
directx9	DIRECTX9	native	yes	yes	yes
directx10	DIRECTX10	fallback–coloured	yes	yes	no
directx11	DIRECTX11	native	yes	yes	yes
directx12	DIRECTX12	native	yes	yes	yes
easygl	OPENGLES2, OPENGLES3, OPENGL33, WEBGL1, WEBGL2	native	yes	profile	profile
fna3d	FNA3D	native	yes	yes	yes
freedirect	FREEDIRECT	fallback–reject	yes	no	no
gdi	GDI	reject	yes	no	no
glide	GLIDE	native fixed	throws	no	no
headless	HEADLESS	trace	trace	trace	trace; not a GPU query
html-dom	HTML_DOM	fallback–reject	yes	no	no
igl	IGL	native	yes	yes	no
llgl	LLGL	native	yes	yes	yes
magnum	MAGNUM	native	yes	conditional	yes
metal	METAL	native	yes	no	no
nanovg	NANOVG	fallback–reject	no	no	no
opengl1	OPENGL1	native fixed	conditional	no	conditional
opengl2	OPENGL2	native	yes	conditional	conditional
opengl4	OPENGL4	hybrid	yes	yes	yes
opengles1	OPENGLES1	hybrid fixed	yes	no	no
openvg	OPENVG	fallback–reject	no	no	no
pixijs	PIXIJS	fallback–reject	yes	no	no
portablegl	PORTABLEGL	native CPU	no	no	no
sdl-gpu	SDL_GPU	hybrid	yes	yes	no; claims yes
sdl-renderer	SDL_RENDERER	fallback–reject	yes	no	no
skia	SKIA	reject	yes	no	no
software	SOFTWARE	native CPU	yes	no	no; claims yes
sokol	SOKOL	native	yes	yes	GL build only
stub	STUB	fallback–no-op	no	no	no
svg-dom	SVG_DOM	fallback–reject	yes	no	no
tinygl	TINYGL	native CPU fixed	no	no	no
vulkan	VULKAN	native	yes	yes	yes
webgpu	WEBGPU	hybrid	yes	no; claims yes	no; claims yes
wicked	WICKED	native	yes	yes	yes

The table is a routing audit, not runtime certification. It is mechanically checked to contain

46 families; EasyGL contributes the remaining four public identities. New alpha.1 families IGL, NanoVG, PixiJS, and TinyGL are explicit rows rather than being inferred from their native libraries. Inherited refusal, deliberate no-op, CPU fixed-function, trace-only and hybrid colored-tail routes remain different outcomes even when the public method exists.

B.3 Capability answers require operation-level proof

`GraphicsDevice::SupportsCapability` is a useful preflight query, but its shared default is permissive. At the pin, `WEBGPU` reports MRT while rejecting every multi-target set. `SDL_GPU`, `WEBGPU`, and `SOFTWARE` report occlusion-query support while inheriting a null factory; `HEADLESS` returns a bookkeeping object with a constant pseudo-count rather than a real GPU query. These are tracked defects, not support.

Conversely, a false or absent high-level bit can hide a narrower usable path. Profile-gated GL queries, driver-gated wrapper capabilities, fixed-function rendering, CPU rasterization, and trace validation all need their own labels. The safe decision sequence is:

1. filter by platform/dependency admissibility and the exact public selector;
2. inspect the factory/resource operation and the draw route for the required feature;
3. require renderer-owned tests or a retained host artifact at the needed evidence tier;
4. use a second structurally independent family as a localization control;
5. retain skips, refusals, and capability/reporting contradictions in the result.

Appendix G defines the evidence vocabulary. The renderer chapters own the narrower state, shader, texture, readback, and platform qualifications that cannot fit in this compact matrix.

Appendix C

Glossary of XNA and CNA Terms

ASCII A removed public renderer identity. Its reusable quantization and glyph-atlas logic now lives in the renderer-neutral `CNA::Graphics::AsciiPostProcessEffect` extension under `modules/graphics-ext`; the former renderer’s architecture is retained as history in Chapter 31.

BGFX One of CNA’s 50 public renderer identities; a third-party rendering library integrated directly via its native API. See Chapter 24.

CANVAS CNA’s Emscripten-only, HTML `<canvas>`-based 2D graphics renderer. See Chapter 31.

CNA The project this book is about: a native XNA 4.0 reimplementaion compiled with a C++23 toolchain floor and written in a disciplined C++17/20 idiom, built on SDL3 with a pluggable graphics-renderer layer.

ContentManager The central XNA class resolving an asset name to loaded content. A same-named `.xnb` is checked first; otherwise the registered loose reader can prefer a `.cnj` sidecar before its declared native formats. See Chapter 33.

DIRECTX9 / D3D9 **DIRECTX9** is CNA’s public selector for its Direct3D 9 renderer, and Direct3D 9 is the API this book’s own zero-tolerance oracle methodology (see **Oracle (D3D9)** below) is built around, since a genuine XNA 4.0 runtime itself targets Direct3D 9. See Chapter 28.

DIRECTX11 / D3D11 **DIRECTX11** selects CNA’s Direct3D 11 renderer, sharing a common `D3DCommon` foundation with **D3D12** but implementing mip-chain generation via one native, whole-resource `GenerateMips()` call where **D3D12** cannot. See Chapter 28.

DIRECTX12 / D3D12 **DIRECTX12** selects CNA’s Direct3D 12 renderer; unlike **D3D11** it has no native convenience call for mipmap generation and instead implements it as an explicit, face-scoped CPU readback/downsample/upload loop. See Chapter 28.

Descriptor heap A Direct3D 12-owned table of resource-view descriptions rather than the resources themselves. CNA’s **D3D12** renderer keeps separate RTV, DSV, CBV/SRV/UAV, and sampler heaps, currently using fixed-capacity bump allocation with no slot reclamation during a device lifetime; exhaustion is therefore a CNA implementation limit, not a Direct3D 12 resource-memory limit. See Chapter 28.

FREEDIRECT CNA’s graphics renderer fronting the sibling `free-direct` project; a 2D-only renderer in the same spirit as `SDL_RENDERER`. The former **DX3** selector was renamed; current **DIRECTX3** denotes a different, genuine DirectX 3 implementation. See Chapter 26.

EasyGL profiles `OPENGLES2`, `OPENGLES3`, `OPENGL33`, `WEBGL1`, and `WEBGL2` are distinct public identities sharing CNA’s internal EasyGL factory and the sibling `easy-gl` project. `EASYGL` is not an accepted selector. See Chapter 22.

- Effect** The XNA base class for a shader program bound to a `GraphicsDevice`. CNA ships five stock effects and renderer-specific runtime-compiled custom `ShaderEffect` path — see the `shader/.fx` gap chapter, Chapter 17.
- ENet** The third-party UDP networking library backing CNA’s real Networking namespace implementation, wrapped by an internal `ENetBackend`. The same library `free-direct`’s own optional `DirectPlay` transport is built on. See Chapter 51.
- EXT suffix** A visible naming convention used by many non-XNA methods and types, such as `MinimizeEXT()` / `RestoreEXT()` on `GameWindow`. It is not the enforcement mechanism: the empty `CNAEXT` declaration macro supplies that marker, and an extension can use either naming style only as the source declares it. See Chapter 3.
- FNA** An open-source C# reimplementation of the XNA 4.0 API, maintained independently of CNA. FNA is CNA’s primary behavioral reference and the subject of CNA’s own differential-testing infrastructure — not the same thing as the real Microsoft XNA 4.0 specification, a distinction this book returns to repeatedly (most sharply in the Media chapter’s *Song* finding, Chapter 46).
- Fence (D3D12)** A GPU-completion counter used to tell when submitted Direct3D 12 command work has finished. CNA records one previous fence value for each of its two command-allocator slots and waits before reusing that slot, rather than waiting for the newly submitted frame itself; its simpler off-screen draw helpers intentionally wait synchronously for deterministic readback. See Chapter 28.
- GamerServices** The XNA namespace covering Xbox-Live-era player profile, achievement, leaderboard, and system-UI concepts. CNA implements it with real local persistence where FNA does not implement it at all. See Chapter 49.
- Golden-image test** A captured frame compared with a retained reference image under an explicit tolerance and environment. CNA has renderer-specific golden tests; the reference’s provenance and the fact that the capture reached real pixels remain part of the claim. See Chapters 70 and 72.
- HEADLESS** A CNA graphics renderer that never rasterizes a real pixel at all — it only tracks and reports the last `Clear()` color and similar logic-only state. Used for fast unit testing and unable to produce a real screenshot for that reason. See Chapter 12.
- GraphicsCapability** The small enum `GraphicsDevice::SupportsCapability` takes, letting game code ask whether the active renderer supports a feature (3D rendering, occlusion queries, and so on) before attempting it, rather than attempting it and catching the resulting exception. Not every value is yet wired to answer honestly on every renderer: MRT and occlusion-query false positives are confirmed, and several device-dependent limits cannot be represented by one Boolean. See Chapter 19 and Appendix B for the qualified matrices.
- GraphicsDevice** The central class every rendering operation in CNA passes through; covered in Chapter 12.
- .cnj** CNA’s own JSON-based content envelope. For a registered loose reader it is considered after the same-name `.xnb` route and before that reader’s direct native formats. It is a typed envelope with per-loader payload rules, not one universal object schema. See Chapter 36.
- .xnb** Real XNA’s compiled binary content-pipeline output format. CNA has a substantial, explicit reader registry for XNB containers and object graphs; a same-named XNB is

checked before loose CNJ/native routes. Reader coverage is finite and unsupported reader names or versions must remain explicit. See Chapters 34 and 35, and this book’s migration guide, Chapter 74.

MojoShader A translator used by the FNA/FNA3D stack for compiled Direct3D 9 effect bytecode. CNA’s FNA3D family uses it for the bounded XNA/FNA Effect Framework route reached by the public byte-buffer constructor and XNB `EffectReader`; `SDL_GPU`, `EasyGL` and `Vulkan` expose explicit experimental opt-ins. It is not an `.fx` source, `DXBC` or `MGFX` loader. See Chapters 17 and 25.

MemoryStream sharp-runtime’s in-memory `IO::Stream` implementation, used where a real file path is unavailable (for example content loaded from Android assets). Closing it makes I/O operations throw `ObjectDisposedException`, but deliberately preserves the bytes for post-close `ToArray()` / `GetBuffer()` extraction. Its raw-buffer constructor is a known divergence: it creates a read-only stream where real .NET defaults to writable. See Chapter 56.

CNAEXT An empty declaration marker for non-XNA surface inside the XNA namespace tree. In strict harnesses it becomes a deprecation attribute and deliberate use fails under warnings-as-errors. The `CNA_CNAEXT` CMake option instead enables selected extension products; `CNA::CnaExt` is a link umbrella. These three mechanisms share a name fragment but are not interchangeable. See Chapter 3.

Oracle (D3D9) The pixel-for-pixel verification methodology comparing CNA’s `DIRECTX9` renderer against a genuinely running XNA 4.0 runtime under Wine, at zero tolerance. See Chapter 28, and this book’s Windows/Wine chapter, Chapter 62.

PbrEffect A CNAEXT glTF 2.0-style metallic-roughness material effect with no real-XNA counterpart. Several implementation families carry PBR routes with different evidence; `WebGPU` is one of them and lacks the skinned sibling. See Chapters 15, 39, and 23.

ReflectiveReader<T> Real .NET’s reflection-based automatic content-reader generation mechanism. CNA has no equivalent by permanent, explicit scope decision: only manually registered, explicit C++ `ContentTypeReaders` are supported, so content originally built against an implicit `ReflectiveReader<T>` needs an explicit `ContentTypeWriter`/reader pair rebuilt and registered before it will load. See Chapter 33.

SDL3 The cross-platform windowing/input/audio library CNA is built on. Not itself a graphics renderer — `SDL_RENDERER` is built on SDL3’s own 2D renderer specifically.

SDL_GPU A CNA graphics renderer built directly on SDL3’s own `SDL_GPU` API, using a fixed HLSL-style resource-binding order and a runtime GLSL-to-SPIR-V compile via `libshaderc`. See Chapter 23.28.

SDL_RENDERER CNA’s default renderer outside Linux and Emscripten, selected by `cmake/RendererSelection.cmake`; a thin, 2D-only wrapper over SDL3’s own 2D renderer. See Chapter 30.

SharpRuntime / sharp-runtime The sibling C++ reimplementations of a subset of the .NET base class library that CNA depends on for its non-XNA-specific vocabulary (`System::Object`, `TimeSpan`, `EventHandler<T>`, and more). Covered in full in Part VIII.

SkinnedModelEXT The CNA-original, CNAEXT real-rendering extension for the Avatar system, structurally separate from the faithful, inert base Avatar API. See Chapter 52.

SOFTWARE A real CPU triangle rasterizer (edge functions, perspective-correct interpolation, per-pixel depth test) that never touches SDL video/X11/a GPU at all, making it a renderer that can produce a genuine screenshot in a headless, display-less environment. See Chapter 12.

SpriteBatch The primary 2D rendering API in XNA and CNA alike. See Chapter 13.

Stock effects The five built-in XNA shader effects — `BasicEffect`, `AlphaTestEffect`, `DualTextureEffect`, `EnvironmentMapEffect`, `SkinnedEffect`. See Chapter 15.

Storage The `Microsoft::Xna::Framework::Storage` namespace — a real, if small, XNA namespace for user-data persistence (`StorageDevice`, `StorageContainer`), backed by a straightforward local-filesystem implementation underneath a faithfully-preserved fake-async API shape. See Chapter 50.

VertexDeclaration The renderer-independent description of a vertex layout: a stride plus `VertexElement` records naming offsets, formats, and usages. It is pure data and can be constructed on CNA's 2D-only SDL renderer; an actual 3D draw that uses it is where that renderer correctly throws. See Chapters 7, 15, and 17.

VULKAN A CNA graphics renderer targeting the Vulkan API directly. See Chapter 23.

WEBGPU CNA's experimental graphics renderer targeting native `wgpu-native`. See Chapter 23.18.

XACT Microsoft's Cross-platform Audio Creation Tool format/engine, the format CNA's own Audio namespace parses and plays back through `SDL3_mixer` rather than a FAudio-equivalent engine. See Chapter 45.

xna4-spec A sibling repository containing 544 indexed, machine-readable XML type files converted from XNA 4.0 documentation across 19 documented namespaces. CNA does not reference it in source or CI at the pin, and converted rows contain defects; use it as searchable inventory with normalization and independent oracles, not as a live or infallible ground truth. See Chapter 76.

Appendix D

Repository and Module Map

This appendix records the exact repository and physical-module boundary used for this edition. It is intentionally narrower than an organization-wide project directory: every listed sibling has a pinned revision and a defined role in dependency, verification, examples, or consumption. All sibling trees were read-only during the book audit.

The revisions below are immutable edition pins, not a promise that every sibling’s branch head will remain there. For example, the local `cna-template` next branch advanced to `4d0a2c8a` after this audit; this edition continues to cite and reason from `1d86681`, which remains available in that repository.

D.1 Pinned repository set

Repository	Pin	Role in this edition
<code>cna</code>	<code>1bb2145d</code>	Primary C++23 XNA-style framework at <code>v0.1.0-alpha.1</code> ; source, build, renderer, platform and verification subject of the book.
<code>sharp-runtime</code>	<code>f827a6c5</code>	.NET-shaped C++ foundation consumed by CNA; object/type, collections, IO, threading/task and compatibility evidence.
<code>easy-gl</code>	<code>0b46d35</code>	Shared implementation beneath five public EasyGL renderer identities; owns portable GL objects and draw machinery.
<code>meta-gl</code>	<code>571d3a6</code>	Lower-level typed OpenGL wrapper consumed by <code>easy-gl</code> ; not a separate CNA renderer identity.
<code>free-direct</code>	<code>934f72f</code>	SDL3-based DirectDraw/DirectSound/DirectPlay compatibility layer used by CNA’s FREEDIRECT renderer and the Blupi case study.
<code>free-api</code>	<code>53d7a31</code>	Minimal Win32-style support layer consumed by <code>free-direct</code> ; a second-level dependency, not a direct CNA module.
<code>xna4-spec</code>	<code>8f61207</code>	Converted XNA documentation corpus: 544 indexed XML type files across 19 documented namespaces; useful inventory, not a live CNA gate.
<code>cna-samples</code>	<code>149a19f</code>	Official-sample translation catalog: 63 completed ports, 23 tracked placeholders, and 67 explicitly ignored source-archive entries at the pin.
<code>cna-examples</code>	<code>fa0b49a</code>	One CNA-native catalog application with 13 areas, 79 categories, and 249 mechanically checked demo screens.

Repository	Pin	Role in this edition
<code>cna-template</code>	<code>1d86681</code>	Minimal consumer skeleton; evidence for how a new application composes the framework rather than for framework correctness.
<code>cna-craft</code>	<code>7d7f186</code>	Larger game consumer with desktop/Web pressure; useful integration evidence, not part of CNA's own test suite.
<code>cna-extended</code>	<code>2ff3cff</code>	Extension-library consumer modeled after <code>MonoGame.Extended</code> ; outside the core API but relevant to package/composition boundaries.

The CNA tag also records `sharp-runtime 625476d5b5fff5fa89f392c3c9af8638ff237692` as the revision used for release verification. That is release evidence, not a dependency lock: CNA's CMake consumes a caller-supplied `SHARP_RUNTIME_ROOT` and does not enforce the recorded SHA. The older `f827a6c5` pin in the table is retained as this book's bounded `sharp-runtime` technical audit source; `alpha.1` claims that depend on `sharp-runtime` were checked against the release record separately.

The dependency arrows are not all equivalent:

```

cna ---> sharp-runtime
|----> easy-gl ---> meta-gl      (selected GL family)
'----> free-direct ---> free-api  (selected FREEDIRECT family)

xna4-spec, cna-samples, cna-examples      evidence/catalog peers
cna-template, cna-craft, cna-extended     downstream consumers

```

Renderer-specific third parties fetched or discovered by CNA are not promoted to sibling repositories in this diagram. They remain dependencies of their owning renderer module, with their platform and acquisition gates recorded in Part IV.

D.2 Physical CNA framework modules

The root `modules/CMakeLists.txt` composes fifteen framework/extension modules when Net is enabled, plus the renderer tree; the experimental C adapter is a separate optional module. Directory location is source ownership: production code belongs under the owner's `src/`, public and public-internal contracts under its `include/`, and its tests/examples alongside them. A configure-time validator rejects translation units outside a declared module tree.

Physical module	mod-	Primary ownership
core		CNA helper/marker surface and root framework primitives
math		vectors, matrices, geometry, curves and packed math values
platform		host/window/event abstraction and the selected SDL3, SDL2, headless or POSIX-terminal implementation
graphics		device facade, resources, effects, model runtime and shared renderer contract
input		keyboard, mouse, gamepad, touch and SDL input extensions
audio		SoundEffect/XACT-facing audio objects and SDL3_mixer integration
media		Song/MediaPlayer/Video and FFmpeg-conditioned paths
content		ContentManager, XNB, CNJ, loose readers and glTF import core
storage		StorageDevice/StorageContainer and local-filesystem behavior
runtime		Game, GameWindow, components, services and timing/lifecycle composition
devices		Microsoft/XNA-shaped device and sensor surface
devices-ext		CNA-native camera/dialog/system/clipboard-style host services
graphics-ext		CNA graphics extensions, including the former extension-layer library
gamer-services		local gamer identity, guide, achievements and leaderboards; conditional on networking enablement
net		sessions, gamers, packets and ENet-backed delivery/discovery; conditional
c-api		optional C17 shared-library adapter over the C++ aggregate; separate experimental ABI, enabled by CNA_BUILD_C_API

The interface target **CNA** composes the framework modules, extension umbrellas, shared build flags, sharp-runtime closure, and the selected renderer target set. In a singular build the set has one family; **CNA_GRAPHICS_RENDERERS** can compose several compatible families and a configured default. Individual module aliases support narrower link-closure probes. **CNA::CnaExt** composes **graphics-ext** and **devices-ext**; it is distinct from the empty **CNAEXT** declaration marker and from the **CNA_CNAEXT** build option that enables selected extension products.

D.3 Renderer module tree

modules/renderers/ contains one common-contract area plus 46 implementation families. Only EasyGL maps multiple public identities to one family.

Cluster	Physical family directories
OpenGL	easygl, opengles1, opengl1, opengl2, opengl4, portablegl, tinygl
Native modern	vulkan, webgpu, sdl-gpu, metal
Abstraction layers	bgfx, magnum, llgl, diligent, sokol, wicked, fna3d, igl
DirectX / retro	directx1, directx2, directx3, directx5, directx6, directx7, directx8, directx9, directx10, directx11, directx12, glide, freedirect
2D / vector / Windows	sdl-renderer, skia, blend2d, openvg, nanovg, direct2d, gdi
Web DOM	canvas, html-dom, svg-dom, pixijs
Diagnostic / CPU	headless, software, stub

The physical tree alone does not establish a public selector, platform admissibility, resource factory, or semantic parity. Appendix B joins directory families to all 50 public identities and their narrow implementation behavior.

D.4 How to extend this map without making it stale

For a later edition, pin every repository before counting it; derive module and renderer sets from CMake/source rather than an organization README; distinguish direct dependency, transitive dependency, evidence peer and downstream consumer; and record the command/query behind every count. A local worktree of `cna` is not a sibling project, and an organization repository with no dependency or evidence edge is outside this bounded map.

Appendix E

CNAEXT: Marker and Engine-Layer Catalog

“CNA extension” names three related but non-equivalent mechanisms in the pinned tree. Keeping them separate prevents both API overclaim and build confusion.

Name	Meaning
CNAEXT	Empty marker placed on a declaration inside an XNA-shaped namespace when that declaration is not part of real XNA 4.0. Strict harnesses redefine it as a deprecation attribute and turn use into an error.
CNA::*	Project-owned namespaces for helpers, renderer contracts, input/device services, diagnostics and engine-layer types that do not pretend to be Microsoft XNA declarations.
CNA_CNAEXT / CNA::CnaExt	A CMake option enabling selected extension products, and an interface target composing graphics-ext with devices-ext . Neither is the declaration marker or a promise that every marked method disappears when the option is off.

The retired pre-migration target/library spelling survives only through compatibility wiring; the compatibility label above preserves old book links, but the live manuscript and current target vocabulary use CNAEXT. The authoritative marker inventory is the source plus strict positive/negative compile harnesses, not a hand-copied book list: a whole-token scan at the pin finds 1,876 occurrences across 302 headers, and that count will change as API classification changes.

E.1 Framework and math bridges

Surface	Extension role	Narrative home
GameWindow *EXT members	Raw SDL window access, borderless state and minimize/restore operations absent from the console-era XNA window contract.	Chapter 6

Surface	Extension role	Narrative home
<code>GraphicsDeviceManager</code> presentation modes	Letterbox, overscan, stretch, native-backbuffer and fixed-height/dynamic-width policies layered around XNA's manager.	Chapters 6 and 12
<code>Matrix::ToColumnMajor</code>	Explicit bridge from CNA/XNA row-vector storage to APIs that want column-major uniform bytes.	Chapter 7
Color conveniences	Byte-component construction and convenience operators not present in the reference surface.	Chapter 7
<code>GetType</code> implementations	Project-wide runtime type-name bridge supplied by sharp-runtime conventions, not an XNA member family.	Chapters 3 and 55

E.2 Graphics and content extensions

Surface	Extension role	Boundary
<code>CNA::Graphics::AsciiPostProcessEffect</code>	Reusable quantization/glyph-atlas effect recovered from the removed ASCII renderer identity.	Renderer-neutral extension in graphics-ext ; not a 47th renderer.
<code>PbrEffect</code> , <code>SkinnedPbrEffect</code>	Metallic-roughness material and its skinned sibling for glTF-era content.	Renderer coverage differs; WebGPU lacks the skinned route.
<code>AnimationPlayer</code> , <code>SkinningData</code>	Runtime skeletal system derived from the historical XNA skinned-model sample rather than the framework assembly.	Separate from the Avatar SkinnedModelEXT system.
Morph-target *EXT surface	CPU delta blending, weights and animation attached through model tags.	POSITION/NORMAL only; tangent deltas are out of scope.
<code>ShaderEffect</code>	Renderer-specific source/program custom-shader route.	Does not make a renderer support the separate compiled Effect Framework route; capability and factory must still agree.
Texture/format helpers	Raw pixel/data helpers, DDS cube loading, format-size queries and additional surface-format values.	Presence does not equal support on every renderer.
Graphics diagnostics/recovery	String markers, context-loss simulation/recovery controls, screenshot/readback and headless-related hooks.	Hook presence, renderer override and retained host evidence are separate.

Surface	Extension role	Boundary
CNJ system	JSON envelope, typed loaders and offline model sidecars.	Project-owned format; reader-specific payloads, not a universal schema.
Content manifests and XNB usage summaries	Enumerate discovered assets and reader-name use for audit/tooling.	Introspection does not itself resolve or validate every asset.

The graphics extensions span **graphics**, **content**, and **graphics-ext**; directory ownership and public API classification are different axes. Chapters 15, 17, 38, and 42 carry the operational detail.

E.3 Input, audio, media, and host services

Area	Representative extension surface	Why it is separate
Input	Clipboard, joystick, power, haptics and raw device helpers; scancode/key-name queries; mouse capture/global position; modern gamepad identity, lights, trigger vibration and sensors.	SDL3/modern-controller capabilities exceed XNA's Xbox 360 model.
Audio	Float-buffer submission on DynamicSoundEffectInstance .	Real XNA's streaming surface accepts 16-bit PCM bytes, not this float path.
Media	Video URI and audio/video track selection *EXT methods.	Some originate in FNA extensions, but they remain outside original XNA.
Devices	CNA::Devices camera, dialogs, locale, URL, system/clipboard and power services.	Project namespace and devices-ext module keep them distinct from Microsoft::Devices::Sensors .
Sensors	Synthetic injection, state-symmetry and richer vibration/support helpers marked inside Microsoft-shaped classes.	Useful testing/platform bridges without reclassifying them as XNA.
Storage	Application-name and storage-root helpers.	CNA needs a deterministic local root where XNA selected a storage device through platform UI.
GamerServices	Local overlay rendering/simulation, ranking/re-pagination and persistence hooks.	They make local behavior testable without claiming Xbox LIVE services.

Area	Representative extension surface	Why it is separate
Networking	Public/internal setters and host-delivery controls needed by the ENet-backed implementation.	Restore real session behavior where the reference implementation retains inert or inaccessible state.

Appendix F maps the surrounding faithful APIs. Extension surface still needs the same ownership, failure, hostile-input and host-evidence analysis as an XNA-shaped member; the marker says only “not original XNA”.

E.4 How the strict check proves itself

In normal compilation CNAEXT expands to nothing. Two standalone harness directions give it force:

1. a positive target defines `CNA_STRICT_XNA_API`, enables warnings-as-errors for deprecated declarations, and compiles code restricted to the intended XNA surface;
2. an excluded negative target deliberately calls a marked sensor injection method and is required to fail compilation. Its CTest uses inverted success so a broken guard that lets the leak compile turns the test red.

This pair proves both acceptance and sensitivity for the exercised slice. It does not prove that every project addition has been marked correctly; that still requires comparison with the reference surface and review of the namespace/classification boundary. Appendix G places this negative compile oracle among the other evidence forms.

Appendix F

Ecosystem and Platform Reference

This companion to Appendix A maps the framework-service and sharp-runtime surface used throughout Parts VII–X. It groups stable API roles and puts the current implementation boundary beside each one. Exact signatures belong to the pinned headers; platform delivery, ownership, hostile-input behavior and test evidence belong to the cited chapters.

F.1 Input: Keyboard, Mouse, GamePad, and Touch

Type group	Public shape	Current boundary
Keyboard, KeyboardState	Static state snapshots, key up/down and pressed-key enumeration; marked scancode, modifier and naming helpers.	SDL delivery is real; layout name, physical scancode and XNA key identity must not be collapsed.
Mouse, MouseState	Position, buttons, vertical wheel and desktop capture, relative/global helpers.	Logical-presentation, window, physical-pixel and post-present coordinates diverge by renderer and host.
GamePad, capabilities and state	Player-indexed state, dead-zone modes, vibration, buttons/sticks/triggers; marked identity, light, trigger-rumble and sensor additions.	Device open/hotplug, SDL mapping and modern-controller extensions have separate evidence.
TouchPanel, collection and gesture types	Touch snapshot, display metadata, enabled gestures and gesture dequeue, plus platform-injection plumbing.	Mouse/touch substitution is not physical-touch proof; Web and mobile coordinate/lifecycle paths differ.
CNA : : Input services	Clipboard, joystick, haptics, sensors, power and enumerated input devices outside XNA's surface.	These are CNA extensions and can have platform-specific absence/refusal semantics.

Chapter 44 owns the exact SDL event/state paths and coordinate matrix. A state object being constructible without hardware is not evidence that a host delivered the event or that the selected renderer maps its coordinates back into the same logical space.

F.2 Audio and Media

Type group	Public shape	Current boundary
<code>SoundEffect</code> , <code>SoundEffectInstance</code>	Loaded/raw PCM sound, fire-and-forget or owned instance playback, volume/pitch/pan/loop and listener/emitter <code>Apply3D</code> .	The 3D model is a CNA-side approximation over <code>SDL3_mixer</code> ; source/instance disposal and channel ownership are observable contracts.
<code>DynamicSoundEffectInstance</code>	Buffer-needed event and streamed PCM submission, plus a marked float-buffer path.	<code>Int16</code> and float modes cannot be mixed silently; queue timing and callback thread/order need runtime evidence.
XACT objects	Wave banks, sound banks, cues, categories, variables and RPC/DSP-like authored behavior.	CNA parses and plays a substantial subset through its own engine; unsupported fields and malformed boundaries remain explicit.
<code>Song</code> , <code>MediaPlayer</code>	Song metadata and static queue/playback control, shuffle/repeat/mute/volume, position and state-change events.	Ownership, callback order and track metadata differ between bare files, library scans and active playback. The public <code>Song/Content</code> extension maps omit <code>.mid</code> even though vendored <code>SDL3_mixer</code> retains its Timidity decoder; decoder presence is not a supported asset route.
<code>Video</code> , <code>VideoPlayer</code>	Video metadata, play/pause/resume/stop, frame texture, volume/mute/loop and marked URI/track selection.	FFmpeg availability is platform-conditioned; decoded CPU frame, uploaded texture and synchronized playback are distinct milestones.

Chapters 45 and 46 separate decode, object lifetime, playback and platform codecs. A reader accepting a file is not proof that a mixer/video device played it or that its events arrived on the intended thread.

F.3 Devices and Sensors

Type group	Public shape	Current boundary
Sensor base and readings	Support/state, update interval, current value, start/stop and change events for accelerometer, compass, gyroscope and motion-shaped types.	SDL covers only part of the sensor vocabulary; Android-native seams, permissions and lifecycle must be named.
VibrateController	XNA/WP-shaped vibration plus marked intensity/support/device helpers.	A stored request is not motor engagement; emulator and physical hardware are different evidence tiers.
CNA::Devices	Camera, display/system information, dialogs, URL launch, locale, clipboard, tray and power services.	Optional <code>devices-ext</code> module and host support; many services deliberately fail or return unavailable on headless/unsupported hosts.

Chapter 47 documents event queues, thread safety and the gap between synthetic injection and physical delivery. Chapter 67 records the limited Android/macOS artifacts at the pin; it does not promote source presence to mobile coverage.

F.4 Networking and GamerServices

Type group	Public shape	Current boundary
NetworkSession	Create/find/join/invited families, local/remote/all/previous gamer collections, session state/events, update, host migration and simulated network conditions.	Local discovery and ENet delivery are real; they are not Xbox LIVE matchmaking and do not supply arbitrary Internet traversal.
LocalNetworkGamer, NetworkGamer	Identity/host/local/ready/round-trip state, send/receive, voice-like and session-owned relationships.	Packet framing, peer identity, ordering and leave/disconnect transitions require two-process hostile-input tests.
SignedInGamer and related service types	Local identity, persistence, paging/rating and selected overlay behavior.	Synthetic/local substitutes preserve API shape but do not claim Xbox LIVE service equivalence.

Type group	Public shape	Current boundary
AvatarRenderer and related types	Faithful base Avatar API plus opt-in SkinnedModelEXT real-rendering route.	Base existence and extension rendering are deliberately separate; neither supplies Microsoft's remote avatar service/assets.

Chapters 51, 49, and 52 own the delivery, persistence, hostile-packet, local-substitute, and extension boundaries.

F.5 Storage

Type	Public shape	Current boundary
StorageDevice	Fake-async selector/container open, connected/capacity state, delete-container, and application-root extensions.	Five focused tests exist; selection is a local-filesystem substitute for platform UI.
StorageContainer	Relative directory/file create/delete/enumerate/existence and stream-open operations.	Path containment is absent at the pin and this class has no focused test corpus; caller-controlled paths are a security boundary.

Chapter 50 explains the base-root/container-root asymmetry and fake-async ownership. Do not summarize the namespace as “tested” by transferring StorageDevice evidence to StorageContainer.

F.6 SharpRuntime Core Types Used Everywhere

Type group	Translation role	C++ reality
System::Object and type helpers	.NET-shaped base naming, equality/hash/string and selected runtime identity.	Standard C++ objects/RTTI; no CLR object header, reflection universe, universal boxing or garbage collector.
Primitive aliases and String	Greppable mapping from C# primitive vocabulary into fixed-width C++ types and std::string.	UTF and managed-string semantics require explicit per-operation work; an alias is not a CLR.

Type group	Translation role	C++ reality
<code>TimeSpan</code> , date/time types	Tick-based durations and selected date/time operations.	Value types with explicit arithmetic/ validation; timezone/runtime breadth is bounded.
<code>EventHandler<T></code>	Subscription tokens, add/remove/clear and snapshot-based raise.	Combines C# delegate/event roles explicitly; lifetime and thread safety remain caller/library contracts.
Collections and task/thread primitives	Source-translation-friendly containers, tasks, cancellation and synchronization subsets.	APIs may be implemented, ignored, or intentionally out of scope; the database status is not a behavioral parity percentage.
<code>System::IO::Stream</code> , <code>MemoryStream</code> , readers/writers	Abstract byte I/O, in-memory storage and .NET-shaped binary/text helpers.	Ownership, closed-state behavior, encodings and exception types must be checked per implementation.

Chapters 55–58 separate API-shaped translation help from actual C++ semantics, include closure and verification. The sharp-runtime database at the pin contains 16,201 task rows and 2,183 tickets, but those are workflow classifications with query-defined denominators, not a whole-BCL completion score.

F.7 Reading this appendix safely

Search CNA’s module-owned headers for exact framework declarations and sharp-runtime’s pinned headers/database for foundation details. Then require the appropriate source path, runtime, oracle and host artifact from Appendix G. A common type name across XNA, FNA, CNA and sharp-runtime establishes lineage; it does not by itself establish identical ownership, exceptions, threading, encoding, platform reach, or behavior.

Appendix G

Verification Tiers and Oracles

This appendix is a claim-decoding key for the rest of the book. It does not rank every test on one line from weak to strong: a compile probe, a hostile-input test, a pixel comparison, and a native-host run answer different questions. The useful record is therefore an evidence vector that names the subject, execution environment, observation point, oracle, result, and retained artifact. Chapters 68–73 explain the machinery; the tables below make its vocabulary quick to apply.

G.1 Canonical claim labels

The edition uses a small set of labels consistently. They describe what was established; they are not interchangeable synonyms for “supported.”

Label	Minimum evidence
Source-proven	A pinned implementation, fallback, refusal, or absence was inspected along a reachable source route.
Compile-proven	The named declarations, includes, templates, and link closure compiled in the recorded configuration.
Runtime-observed	The named path executed and yielded the recorded state, value, file, trace, or lifecycle event.
Renderer-engaged	The intended renderer family and native or translation API were identified during execution.
Oracle-compared	An observation was compared with an identified authority under a stated policy.
Pixel-verified	Selected pixels or a complete frame were compared at a named readback boundary, renderer, host, and tolerance.
Manually observed	A human inspected the output without a retained discriminating oracle.
Historically recorded	A dated artifact reports the result, but this edition did not rerun it.
Blocked	A named external precondition prevented the attempted route from reaching its subject.
Unsupported	The pinned implementation has an explicit refusal, absent factory, or documented lack of the named path.
Not attempted	No execution claim is made. Source or compile evidence may still exist.

G.2 Evidence forms are not interchangeable

Evidence form	What a green result establishes	What it does not establish
Header or compile probe	Names, signatures, templates, includes, and link closure are usable in the selected build.	Runtime semantics, resource creation, rendering, or host engagement.
Strict-XNA positive probe	Code restricted to the intended XNA surface compiles with <code>CNA_STRICT_XNA_API</code> .	That the negative guard catches a leaked extension.
Strict-XNA negative probe	A deliberate <code>CNAEXT</code> call fails compilation, proving the guard itself is sensitive.	Coverage of every unmarked or incorrectly classified declaration.
Pure deterministic test	A value, state transition, parser result, or exception matches its asserted oracle on that host.	Renderer, window system, driver, or external runtime engagement.
Hostile-input test	A named malformed boundary rejects, contains, or reports the input as asserted.	Correct behavior for valid data or completeness of the malformed corpus.
Trace / HEADLESS test	Routing, validation, counters, or state order reached the trace implementation.	Rasterization, GPU work, visible pixels, or real occlusion samples.
Source-path audit	At the pinned revision, a reachable implementation, refusal, fallback, or parameter loss is visible in source.	Successful compilation or execution on a required host.
Readback / small-pixel test	The exercised renderer produced the asserted value at the selected readback boundary.	Whole-scene fidelity, a different renderer, or a path bypassed by readback.
Golden-image comparison	A complete captured frame matches a retained reference under the recorded tolerance and environment.	That the reference itself is authoritative or portable.
Differential run	CNA agrees with a named independent implementation or real XNA for the exercised inputs and observation.	General parity beyond that corpus; two implementations can share a defect.
Native-host artifact	The requested executable, API/runtime, device or browser, and observation point all engaged on the named host.	Other hosts, other identities, or long-duration behavior.
CI workflow success	The workflow trigger reached every recorded step and its gates passed for that matrix leg.	Jobs excluded by filters, dispatch-only workflows, dead configure legs, or tests never registered in that configuration.

“Test exists”, “test was registered”, “test ran”, and “oracle discriminated” are four separate statements. A skipped test is not a pass, a dry-run mutation is not a killed mutation, and an uploaded log is useful only when it identifies the binary, configuration, host, and observation that produced it.

G.3 Oracle authority depends on the question

For API shape, CNA’s policy names Microsoft XNA reference assemblies/XML as the target surface; `xna4-spec` is a searchable converted documentation corpus, not a live CNA gate and not an infallible substitute for the original assembly. For behavior, CNA’s checklist names FNA as the ordinary reference, while real XNA, a format specification, or a renderer API can outrank it for a narrower question. A project-authored expected value is an oracle only when its derivation is reviewable and independent of the code under test.

Question	Preferred authority and control
Public API shape	XNA reference assembly/documentation; normalize C#-C++ idioms explicitly, then compile positive and negative probes.
Managed behavior	Real XNA where executable; otherwise pinned FNA plus independently chosen edge cases and documented intentional deviations.
File-format semantics	The owning specification and independently generated fixtures; compare production output with a separate decoder or digest where practical.
GPU/API behavior	The native API contract plus renderer-owned execution evidence; use another renderer only as a localization control, not as the sole truth source.
Project extension	A written CNA contract, negative cases, and a discriminating test; XNA/FNA cannot define semantics for a CNA-only feature.
Historical reconstruction	Original executable/source/data and versioned captures; decompiled code is evidence to audit, not automatically a behavioral specification.

G.4 Renderer and platform claims need a complete vector

A useful renderer result records all of the following fields. Omitting one often changes the meaning of the claim.

Field	Record
Source identity	CNA SHA, sibling SHAs, local-diff status, and generated shader/table revision.
Build identity	Public renderer selector, implementation family/profile, build type, toolchain, feature options, and exact configure/build command.
Host identity	OS/architecture, native API or translation runtime, device/driver, display mode, browser/emulator/physical-hardware identity, and relevant environment variables.
Subject	Exact test, demo, asset, API operation, malformed case, or renderer route.
Engagement proof	A log, marker, API/device identity, browser verdict, screenshot, readback, or lifecycle trace proving the intended path rather than a substitute path ran.
Oracle	Expected value/image/exception and its authority; tolerance and normalization rules.
Outcome	Pass, fail, expected fail, skip, unsupported, blocked, or not attempted—never one overloaded “supported” bit.
Artifact	Retained log, image, trace, manifest, digest, or CI record with a reproducible locator.

This vector distinguishes, for example, a MinGW binary running through Wine and DXVK from a native MSVC/Direct3D run; a browser page loaded from an HTTP server from one opened as a local file; an Android APK that installs and presents a frame from its selected renderer; and an iOS source path from an actual signed device build. Those are all useful milestones, but they are not synonyms.

G.5 Automation at the pinned revision

The tag contains 21 workflow files: 19 react to push or pull-request events, usually under path filters, and the Direct3D/GDI routes are dispatch-only. New source-level gates cover compatible native and Emscripten multi-renderer sets, platform/audio tuples, glTF, TinyGL, NanoVG, Apple and five C API records. The records do not build the final C library; a local exact-tag build reaches the adapter’s 49-versus-50 renderer-identity invariant and stops. Three automatic general/Input legs still stop during configuration because they pass the retired family token `EASYGL` instead of a public selector. Sanitizer, mutation, fuzz-named, and validation mechanisms exist, but their mere presence does not prove a tag workflow ran or enlarge the set of engaged routes. Chapter 73 is the dated path-by-path account.

The safe summary at this pin is therefore neither “CI absent” nor “CI complete”. Automatic browser harnesses, a Devices UBSan route, platform-specific jobs, and several narrow gates are declared; dead configure legs, dispatch-only Windows coverage, no code-coverage measurement, no continuous coverage-guided fuzzer, and a Direct2D mutation dry-run remain real too.

G.6 A minimum claim recipe

Before writing “works”, write a narrower sentence that answers six questions:

1. What exact subject and revision were examined?
2. Which build, renderer, platform, and host actually engaged?
3. Where was behavior observed—return value, trace, readback, frame, file, or peer?
4. What independent authority supplied the expected result?
5. Which negative or perturbed case proves the check can fail?
6. What artifact lets the next auditor reproduce or falsify the claim?

A claim that cannot yet answer all six can still be useful. Label it source-proven, compile-proven, runtime-observed, renderer-engaged, oracle-compared, pixel-verified, manually observed, historically recorded, blocked, unsupported, or not attempted. Do not silently promote one state to behavioral parity.

Appendix H

glTF Importer, Runtime, and Test-Evidence Matrix

This matrix keeps three questions separate: what the shared import core accepts or transforms, what the alpha.1 runtime represents, and how far the tag’s evidence reaches. “Reported” means `GlTFImportReportEXT` makes a loss visible; it does not mean the missing feature was implemented. Chapters 39–43 explain the contracts.

H.1 Containers, scenes, and geometry

glTF feature	Importer / offline route	Runtime representation	Focused evidence
.gltf/.glb	Shared cgltf core; paired generated containers.	Direct <code>Load<Model></code> and offline CNJ.	Manifest, L0/L1, 148 container pairs.
External/data resources	Data URI, buffer view, and contained percent-decoded external paths; absolute/traversal/symlink escapes refused.	In-memory decode and owned textures/buffers.	Structural/URI rejection tests and corpus cases.
Version/extensions	Version 2.0 check, overflow/alignment guards, <code>cgltf_validate</code> ; unsupported required extensions rejected, optional losses warned.	Ordered diagnostics on the model.	Validator audit, extension-registry and rejection fixtures.
Scene graph/instancing	Default scene, parent-before-child graph, node matrix/TRS, shared-mesh placements.	Synthetic root plus one <code>ModelBone</code> per reachable node.	L1–L4 and D1–D3 regressions.
Multiple skins/groups	Every group extracted; converter emits separate outputs.	Direct route retains all groups in one model; <code>SkinsEXT</code> maps palettes to meshes.	Multi-skin, mixed static/skinned, direct/CNJ parity tests.

glTF feature	Importer / offline route	Runtime representation	Focused evidence
Accessors	Offsets, strides, normalization, sparse base/override, safe bounds and index range.	Dense semantic streams and packed buffers.	L2, fuzz/robustness locks, D4 regression.
Vertex layouts	Selection table emits ten glTF strides; source attribute losses counted.	Canonical stride/declaration table reaches renderers.	L5 byte goldens and table validators.
All seven topologies	Points/lines retained; loop closed; strips/fans converted; incomplete tails counted.	Per-part CNAEXT <code>PrimitiveType</code> .	Topology corpus; D5 fixed.
Mirrored placement	Negative determinant detected per instance.	Fact reported; application owns cull-state response.	L3/L4 fixtures and diagnostic lock.

H.2 Materials, scenes, and animation

glTF feature	Importer / offline route	Runtime representation	Focused evidence
Core metallic-roughness	Factors and seven texture slots, alpha/cull state, samplers, colour space, independent transforms and two UV selectors retained.	Rigid/skinned PBR parameter contract; factor-only materials remain PBR.	L3/L5/L6 and L7 material fixtures; D7 fixed.
Vertex colour	<code>COLOR_0</code> multiplied through unlit/basic and PBR layouts.	Strides 24/56/60/80, with renderer capability policy.	Byte and pixel fixtures.
Material variants	Source-order names and sparse primitive mappings retained direct/offline.	Model-wide CNAEXT selection restores full part state or default.	Variant L3–L7 cases.
Extension textures	Specular factors transported; optional maps have named renderer gaps. KTX2/WebP require a PNG/JPEG fallback.	Registry qualifies each claim.	Registry validator, renderer inventories, selected pixels.
Texture transforms/samplers	Independent state per sampled map; up to two source UV sets packed.	Effect parameters plus renderer sampler slots.	L3/L5/L6 and multi-map L7 fixtures.

glTF feature	Importer / offline route	Runtime representation	Focused evidence
Skins	Joint order remapped; ancestry and mesh-node cancellation kept as root prefixes.	All <code>ModelSkinEXT</code> mappings plus first-skin Tag alias.	Reversed order, multi-skin, D8.
Animation	Skeletal clips use palette indices; rigid clips use scene-node indices.	<code>SkinningData</code> or <code>ModelAnimationsEXT</code> ; mixed-carrier drops reported.	LINEAR/STEP/CUBICSPLINE, D6 fixed.
Morphs	Position/normal/tangent targets and weights imported; invalid omissions reported.	CPU blend and buffer re-upload using canonical layout offsets.	Semantic, CNJ parity, runtime tests.
Cameras	Perspective/orthographic projection data and node placement imported.	<code>Model::CamerasEXT</code> .	Camera corpus and model tests.
Punctual lights	Up to three mapped to directional slots; point/spot/range/-cone losses counted.	XNA <code>IEffectLights</code> approximation.	L3/L6/L7 plus report tests.
Draco	Supported only in builds with the decoder; required use otherwise refused.	Ordinary model buffers after decode.	Compressed/uncompressed parity fixtures.

H.3 Route and oracle boundaries

The direct reader and `cna_tool_gltf_to_cnj` share semantic extraction but package the result differently. Direct loading creates one model containing every represented group and fixes unit scale at 1. The converter can apply an explicit scale and emits one named CNJ output per group. A CNJ reload proves the serialized boundary for the fields it carries; it is not a claim that JSON bytes and a live object are identical.

Layer	State	Observation
L0	implemented	corpus, manifest, helper integrity, and suite/rung partition
L1	implemented	container structure, scene and declared relationships
L2	implemented	decoded accessor values
L3	implemented	semantic mesh/material/scene/diagnostic state
L4	implemented	composed scene and skin coordinate spaces
L5	implemented	packed vertex/index bytes; 140 goldens, 8 explicit absences
L6	implemented	bound matrices, materials, maps, samplers, lights, palettes
L7	recorded	four 148-disposition renderer campaigns (140 PNGs + 8 rejections each)

All original D1–D8 records are fixed at alpha.1 and remain in the append-only ledger as regres-

sion history. The broader GLTF ROBUST milestone remains 9/12: two renderer specular-map bindings, native point/spot lighting, and a fourth renderer on the bounded external retake matrix remain the three open rows. The tag therefore supports strong layer-qualified claims, not an unqualified promise about every file and renderer combination.

Appendix I

Experimental Native C API

v0.1.0-alpha.1 contains the source and public headers for an experimental C17 binding over CNA's canonical C++ implementation. It is intended as an optional shared-library surface, not a replacement implementation and not evidence that every C++ expression has a useful C spelling. The tag-level build result is important context for the nominal command:

```
1 cmake -S . -B build-c \  
2   -DCNA_BUILD_C_API=ON \  
3   -DCNA_GRAPHICS_RENDERER=HEADLESS \  
4   -DCNA_PLATFORM=HEADLESS \  
5   -DCNA_AUDIO_PLATFORM=NULL \  
6   -DCNA_ENABLE_NET=ON  
7 cmake --build build-c --target cna_c_api
```

Limitation. This command does not produce the library at the immutable alpha.1 tag. A clean GNU 14.2 build with networking enabled reaches `CnaCapiCoreExt.cpp` and fails its own compile-time identity invariant: the C table and `CNA_GRAPHICS_RENDERER_MAXIMUM` contain 49 identities, while the canonical C++ enumeration contains 50. The missing C identity is `NANOVG`. This is a build-blocked source surface, not a consumable alpha.1 binary artifact. The command is retained so the failure is reproducible and a later tag can be rechecked against the same build contract.

The target definition is `cna_c_api`, exported to CMake consumers as `CNA::Capi`. Its intended normal product is a shared library; the source also defines an explicitly controlled static form for packaging tests. Public headers live under `CNA/C/`, and `CNA/C/cna.h` is the 59-header umbrella. A C consumer includes those headers and links the C API library; it does not include CNA or Sharp Runtime C++ headers. The project builds its C translation units as required C17. The compatibility generator also checks that each public header remains self-contained under C99, C11, C17 and C23 modes where the declared compiler is installed; C99 header compatibility does not lower the project's C17 build standard.

I.1 Two version systems, deliberately independent

Version	At this tag	What it governs
CNA product	0.1.0-alpha.1	Source release, C++ API, tools, modules and documentation. The leading zero and prerelease component mean that compatibility may still change.
Native C ABI	0.7.0	Binary layouts, exported C symbols and the C binding's compatibility rules. <code>CNA_ABI_VERSION</code> encodes it as a 32-bit integer.

The header macros `CNA_ABI_VERSION_MAJOR`, `_MINOR`, `_PATCH` and `CNA_ABI_VERSION` describe the headers. `cna_get_abi_version()` reports the loaded library. Consumers should compare the encoded values and reject an incompatible major. This is separate from the generated C++ product-version API in `CNA/Version.hpp`; 0.7.0 is not CNA product release 0.7.0, and 0.1.0-alpha.1 is not C ABI 0.1.0.

Limitation. The tag’s generated C API release-gate report still labels its header “CNA C ABI 0.1.0”, while `CNA/C/abi.h` and `cna_get_abi_version()` establish 0.7.0. The implementation and public header are the edition authority; the report label is a documentation defect, not a second supported ABI. Its “Ready” verdict is not a build result: the checker confirms that test, install and consumer-gate definitions exist, but does not execute them, and therefore misses the 49-versus-50 compile failure.

I.2 Results, diagnostics, and handles

Every fallible operation returns the fixed-width `CNA_Result`. At the tag the public codes are 0 through 14: success, invalid argument, invalid handle, invalid state, out of memory, I/O, unsupported, platform, thread, callback, overflow, encoding, internal, shutting down, and buffer too small. Failure details are thread-local. Call `cna_error_get_last_info()`, query the UTF-8 message size, then copy the message into caller-owned storage. A successful call can replace the diagnostic, so error retrieval belongs immediately beside the failing call.

Objects cross the boundary as the 64-bit `CNA_Handle`; zero is invalid. The internal registry validates generation, object kind, owning runtime, lifetime and thread affinity before recovering a C++ object. Destroy operations invalidate their handle. A borrowed handle does not transfer ownership, callbacks must obey the route’s re-entry rule, and a handle is not a native pointer that C may dereference. Versioned structures carry `struct_size` and `struct_version`; strings use explicit UTF-8 byte lengths; collections use counts plus indexed access or dedicated enumerator handles.

I.3 Surface and coverage boundary

The headers cover framework/runtime, math and value types, graphics devices and resources, effects and models, input, content, gamer services and networking, storage, audio/XACT, media, sensors, devices, components, windows, and the graphics-device manager. This breadth should not be summarized as “complete C coverage.” The tag’s generated inventory classifies 6,712 public C++ declarations as follows:

Disposition	Declarations
Implemented	6,317
Partially mapped, with an approved limitation	15
Planned / unclassified	0
No C form	380
Total	6,712

The 380 exclusions include deleted operations, iterators, arbitrary C++ templates, protected subclass hooks, Sharp Runtime objects, friendship and internal platform interfaces. The 15 partial entries name their usable subset and alternative route. For example, generic `ContentManager::Load` cannot accept an arbitrary C++ type from C, so the binding supplies typed asset routes. This is reviewed coverage, not one-to-one syntactic transliteration.

The generated limitations prose opens with a stale total of 6,415 even though its own current rows and the coverage inventory sum to 6,712. This edition uses the reproducible inventory and records the prose total as drift.

I.4 Renderer and platform consequences

Once the adapter builds, its C routes create canonical CNA objects, so their capabilities and lifetime rules remain those of the active renderer and selected platform/audio modules. It does not expose an `IGraphicsRenderer` pointer or a second renderer implementation. A multi-renderer build still resolves the runtime renderer through CNA's C++ selection service before the first graphics device; C routes then observe the chosen device and receive `CNA_RESULT_NOT_SUPPORTED` for unavailable operations. Renderer rebuild for a changed multisample request is explicitly resource-destructive and is only safe before GPU resources exist.

This corrects an ambiguity in the tag's C API renderer prose, which says only that the backend is "compiled into CNA." That wording describes a single-renderer build but does not erase the tag's generated multi-renderer registry.

I.5 Verification inventory, not a universal pass claim

Source inspection at the tag finds 74 pure-C test translation units, 82 CTest registrations whose names begin `CApi_`, 176 recorded structure layouts, and 2,861 recorded exports. Five dedicated workflows cover the ABI baseline, compatibility matrix, coverage inventory, limitations matrix, and combined release gate. Their presence proves automation is declared; it does not by itself prove that GitHub executed every cell for this tag. The compatibility matrix describes 23 compiler cells across seven toolchain families, while actual outcomes require workflow-run evidence.

The declared release gates are designed to check that public declarations have a disposition, generated reports are current, headers compile as C and C++, layout/export baselines are reviewable, examples consume the installed package, and selected renderer builds validate the same ABI boundary. At alpha.1, neither the five narrow workflows nor the source-only aggregate checker closes the final-library-build predicate. The missing NanoVG identity, stale ABI label and stale declaration count show why generated records still require a fresh artifact build.

Evidence. The header count, result values, ABI 0.7.0, coverage partition, test-source and registration counts are Source-proven at `v0.1.0-alpha.1`. They are not reported as an executed 82-test run. Two build probes were attempted. With `CNA_ENABLE_NET=OFF`, configuration succeeds but the C adapter cannot see `GamerServices` headers because CMake neither refuses nor repairs the invalid closure. With networking enabled, compilation reaches and fails the 49-versus-50 renderer `static_assert`. The tag-level library status is therefore **Blocked at compile time**, while the generated product-version example remains independently Compile-proven.

Index

- .NET compatibility, 379
- .fx, 131
- ABI version, 549
- Accelerometer, 16, 354
- Activator, 403
- AI-assisted production, vii
- Album, 6, 350
- AlphaTestEffect, 114, 116–118, 121, 250, 283, 514, 524
- Android, 453
- Android emulator, 453
- animation, 317
- AnimationClipEXT, 270
- AnimationPlayer, 120, 191, 302, 303, 305, 317–319, 530
- API audit, 495
- API compatibility, 3
- API parity, 403
- ArgumentException, 64, 318
- ArgumentNullException, 141, 398, 399
- ArgumentOutOfRangeException, 64, 307, 398, 399
- Artist, 6, 350
- ASCII, 521
- AsciiPostProcessEffect, 149, 163
- AudioCategory, 345
- AudioEngine, 345
- audit database, 409
- AvatarAnimation, 373
- AvatarAppearanceEXT, 375
- AvatarDescription, 373
- AvatarRenderer, 373, 374, 536
- AvatarRendererState::Unavailable, 373
- BasicEffect, 59, 89, 114–116, 121, 171, 235, 250, 283, 299, 514, 524
- BGFX, 517, 521
- BgfxRenderer, 165
- BinaryReader, 280
- Blend, 125, 180
- BLEND2D, 517
- BlendFunction, 125, 180, 223, 227
- BlendMorphTargetsEXT, 304
- BlendState, 58, 123, 124, 180, 195, 251, 515
- BlendState::AlphaBlend, 91, 94, 227
- BlendState::Opaque, 227
- BoundingBox, 56, 61, 62, 64, 512
- BoundingBox::CreateFromPoints, 64
- BoundingBox::GetCorners, 63
- BoundingFrustum, 53, 56, 61, 63, 512
- BoundingSphere, 56, 61–64, 512
- BoundingSphere::CreateFromPoints, 63, 64
- BoundingSphere::Transform, 57
- browser main loop, 445
- browser testing, 449
- Buffer, 420
- building CNA, 23
- Buttons, 335
- ButtonState, 333, 335
- C API, 549
- C++23, 15
- Camera, 357, 358
- CANVAS, 517, 521
- case study, 489
- ClearOptions, 83, 125
- Clipboard, 337, 338, 359
- CMake, 23
- CMake component, 387
- CMake target, 65
- CNA, 15, 521, 529
- CNA_AUDIO_PLATFORM, 343
- CNA ecosystem, 9
- CNA_GRAPHICS_RENDERER, 23
- CNA_PLATFORM, 429
- CNA/Version.hpp, 26
- cna-examples, 499
- cna-samples, 499
- CNA::CApi, 549
- CNA::Devices, 70, 357, 531, 535
- CNA::Graphics, 26
- CNA::Graphics::AsciiPostProcessEffect, 261, 517, 521, 530
- CNA::GraphicsRendererSelection, 163
- CNA::Input, 533
- CNA::Internal::Renderers::SdlGpu::SdlGpuRenderer, 196
- CNA::Platform::IPlatform, 65, 343, 429
- CNAEXT, 523
- CnaTests, 459
- CNJ, 287
- .cnj, 522
- CNJ model toolchain, 321
- Collections::Generic::List<T>, 381
- Color, 15, 18, 53, 57, 61, 62, 87, 102, 103, 141, 187, 371, 511, 530
- Color::CornflowerBlue, 33
- ColorWriteChannels, 172
- CompareFunction, 116, 117, 125
- Compass, 354
- conformance testing, 325
- content compression, 275
- content security, 291
- ContentLoadException, 276, 280, 283, 292, 311
- ContentManager, v, 51, 269, 272, 279, 288, 300, 309, 512, 521
- ContentManager::Load, 550
- ContentManager::Load<T>, 291, 292

- ContentManager::Unload(), 302
- ContentReader, 279, 280
- ContentReader::ReadExternalReference<T>, 270
- ContentTypeReader, 523
- ContentTypeReader<T>, 281
- ContentTypeReaderBase, 281
- ContentTypeReaderManager, 279, 280
- continuous integration, 479
- CPixmap, 492
- CPixmap::BlitFast, 489
- CreateFromPoints, 20
- CreateFromSphere, 20
- CreateMerged, 20
- cross-compilation, 433
- cross-platform engineering, 429
- CTest, 459
- CubeMapFace, 104
- Cue, 345
- CullMode, 127
- Curve, 58, 63, 270
- CurveContinuity::Step, 64
- CurveKey, 53, 61, 63
- CurveKeyCollection, 58, 336

- D3D11, 521
- D3D12, 521
- D3D9, 521
- DateTime, 362, 380, 391
- DateTimeKind, 282
- DateTimeOffset, 380, 391
- Decimal, 435
- defect ledger, 325
- Delegate, 391, 404
- Delegate::DynamicInvoke, 407
- dependency acquisition, 73
- dependency cycle, 69
- DepthFormat, 108, 207
- DepthStencilState, 124, 126, 130, 180, 190, 250, 515
- DepthStencilState::Default, 130
- descriptor heap, 521
- developer tools, 73
- Dictionary, 381
- Dictionary<TKey,TValue>, 381
- differential testing, 3, 467
- DILIGENT, 517
- DIRECT2D, 517
- DirectDraw, 423
- DirectPlay, 423
- DirectSound, 423
- DIRECTX1, 517
- DIRECTX10, 517
- DIRECTX11, 517, 521
- DIRECTX12, 517, 521
- DIRECTX2, 517
- DIRECTX3, 517
- DIRECTX5, 517
- DIRECTX6, 517
- DIRECTX7, 517
- DIRECTX8, 517
- DIRECTX9, 517, 521
- DiscardContents, 171
- discipline gate, 475
- DisplayMode, 88
- DisplayOrientation, 125
- disposal
 - graphics resources, 101
- DrawableGameComponent, 41, 45, 47
- DrawableGameComponent::Dispose(bool), 40
- DualTextureEffect, 114, 118, 121, 250, 283, 313, 323, 514, 524
- DXVK, 437
- DynamicIndexBuffer, 109, 110, 142
- DynamicSoundEffectInstance, 343, 344, 531, 534
- DynamicVertexBuffer, 109, 110, 142

- easy-gl, 415
- EasyGL profiles, 521
- easygl::Capabilities::detect_common_features(), 416
- easygl::Device, 415
- easygl::UnsupportedFeatureException, 416
- Effect, 3–5, 91, 113, 114, 131, 135, 137, 143, 211, 249, 254, 270, 283, 297, 300, 515, 522
- Effect::Apply(), 115
- EffectParameter, 113, 114, 515
- EffectPass::Apply(), 113, 115, 131
- EffectReader, 523
- Emscripten, 441
 - main loop, 445
- Encoding, 400
- EndOfStreamException, 272, 275
- ENet, 522
- engagement gate, 437, 471
- EnvironmentMapEffect, 102, 108, 114, 118, 119, 121, 181, 191, 192, 205, 250, 283, 514, 524
- EventHandler<T>, 380, 393, 537
- evidence status, vii
- ExitingEventArgs, 39
- EXT suffix, 522

- fence (D3D12), 522
- FileDialog, 357
- FileNotFoundException, 272, 274
- FileStream, 367
- FillMode, 127, 195
- FNA, 522
- FNA3D, 517
- forbidden-token scan, 475
- Framebuffer, 417, 420
- FrameworkDispatcher, 51, 69, 349, 350
- free-api, 423
- free-direct, 423
- FREEDIRECT, 517, 521
- FriendCollection, 361

- Game, v, 4, 32, 39, 40, 42, 45, 47, 50, 81, 84, 153, 185, 252, 271, 272, 279, 350, 374, 454, 491, 492, 512
- Game lifecycle
 - desktop and browser loops, 39
- Game::BeginDraw(), 49
- Game::Run(), 33
- GameComponent, 39, 41, 45
- GameComponentCollection, 47

- GamePad, 16, 34, 338, 355, 533
- GamePadButtons, 335
- GamePadDeadZone, 334, 335
- GamePadDPad, 335
- GamePadState, 335
- GamePadThumbSticks, 335
- GamePadTriggers, 335
- Gamer, 361, 362
- GamerPresence, 363
- GamerPrivileges, 363
- GamerProfile, 363
- GamerServices, 5, 25, 361, 522
- GamerServicesComponent, 363
- GamerServicesDispatcher, 364
- GameServiceContainer, 39, 47, 48
- GameTime, 39, 44, 45, 381, 512
- GameWindow, 39, 48, 151, 512, 522, 529
- GameWindow::GetNativeSdlWindowEXT(), 151
- GC, 407
- GDI, 517
- Genre, 6, 350
- GestureSample, 336
- GestureSample::Timestamp, 337
- GestureType, 336
- GetGraphicsRendererType(), 165, 188
- GitHub Actions, 479
- GLIDE, 517
- glTF, 309, 523
- glTF conformance, 325
- glTF importer, 309
- gltf_to_cnj, 321
- GltfImportReportEXT, 301, 311, 327, 508, 545
- golden-image test, 467, 522
- GoogleTest, 459
- GpuDrawParams, 82, 86, 132, 134, 143–145, 155, 168, 169, 172, 211, 236
- GraphicsAdapter, 81, 88
- GraphicsCapability, 143, 157, 250, 522
- GraphicsCapability::Instancing, 204
- GraphicsDevice, 3, 4, 32, 69, 81, 82, 87, 89–91, 106, 123, 127, 130, 137, 142, 144, 156, 161, 163, 172, 173, 185, 186, 212, 224, 225, 235, 239, 244, 246, 249, 252, 253, 299, 302, 353, 373, 491, 514, 522
 - SetVertexBuffers, 142
- GraphicsDevice::BlendState, 202
- GraphicsDevice::Clear, 33
- GraphicsDevice::DrawUserPrimitives, 141
- GraphicsDevice::GetGraphicsRendererType(), 164
- GraphicsDevice::Present(), 33
- GraphicsDevice::Reset, 200
- GraphicsDevice::SetRenderTargets(), 202
- GraphicsDevice::SupportsCapability, 519, 522
- GraphicsDeviceManager, v, 32, 40, 49, 81, 84, 153, 200, 235, 512, 530
- GraphicsDeviceManager::Dispose(bool), 40
- GraphicsProfile, 81, 235
- GraphicsProfile.Reach, 234
- GraphicsRendererCreateArgs, 164
- GraphicsRendererSelection, 429
- GraphicsRendererSelection::GetAvailable(), 162
- GraphicsResource, 88, 91, 95, 101, 109, 113, 123, 150
- Guide, 363
- Gyroscope, 354
- HapticDevice, 339
- Haptics, 338, 339
- HashSet, 381
- HEADLESS, 517, 522
- hostile environment, 471
- HTML_DOM, 517
- IAAsyncResult, 373
- IAvatarAnimation, 373
- IDisposable, 280, 405
- IEffectFog, 190
- IEffectLights, 190, 547
- IEffectMatrices, 135, 137, 190, 317
- IEffectRenderer, 133, 134, 243
- IEquatable<T>, 61
- IGameComponent, 45
- IGL, 517
- IGraphicsDeviceManager, 49
- IGraphicsRenderer, 4, 196, 241, 248, 551
- implementation family, 517
- IndexBuffer, 109, 142, 175, 250, 297, 300, 513
- IndexElementSize, 142
- InstancePlayLimitException, 346
- Int128, 435
- InvalidOperationException, 98, 99
- IO::IOException, 399
- IO::RandomAccess, 407
- IO::Stream, 397, 523
- iOS, 453
- IPackedVectorT<UInt32>, 61
- IPlatform, 429, 431
- IPlatformSurfacePresenter, 431
- IRenderTarget, 107
- ITextureCubeRenderer, 192
- ITextureRenderer, 210
- IVertexBufferRenderer
 - SetVertexDeclaration, 142
- IVertexType, 61, 141, 313
- Joysticks, 338, 341
- Keyboard, 34, 331, 332, 533
- KeyboardState, 34, 331, 332, 533
- KeyframeEXT, 319
- KeyNotFoundException, 411
- Keys, 331, 332, 340
- KeyState, 332
- LaunchParameters, 51
- LeaderboardEntry, 362
- LeaderboardReader, 362
- LeaderboardWriter, 361, 362
- link closure, 69, 475
- LinkedList, 381
- LLGL, 517
- LocalNetworkGamer, 535
- macOS, 453
- MAGNUM, 517
- MathHelper, 58
- Matrix, 53, 55, 61, 91, 511

- Matrix::ToColumnMajor, 530
- Media, 6, 349
- MediaLibrary, 6, 349, 350
- MediaPlayer, 69, 349, 350, 534
- MediaQueue, 350
- MemoryStream, 273, 398, 523, 537
- MemoryStream::Write, 411
- MessageBox, 358
- meta-gl, 415
- METAL, 517
- Microphone, 346
- Microsoft.Xna.Framework, 3
- Microsoft::Devices, 70
- Microsoft::Devices::Sensors, 353, 357, 531
- Microsoft::Xna::Framework, 6, 15, 65, 429
- Microsoft::Xna::Framework::Graphics, 19
- Microsoft::Xna::Framework::Input, 331
- Microsoft::Xna::Framework::Net, 369
- Microsoft::Xna::Framework::Storage, 65, 365, 524
- migration, 485
- MinGW-w64, 433
- Model, 188, 211, 246, 270, 297, 300–303, 305, 310–312, 317, 321, 514
- Model.Tag, 300
- Model::Draw, 298, 299
- ModelAnimationsEXT, 319, 547
- ModelBone, 297, 300, 301, 303, 306, 310, 317–319, 514, 545
- ModelBoneCollection, 297, 306
- ModelEffectCollection, 297, 299, 307
- ModelEffectCollection::Add, 299
- ModelMesh, 137, 297, 301, 303, 310, 317, 514
- ModelMesh::Draw, 299
- ModelMeshCollection, 297
- ModelMeshPart, 297, 301, 303, 315, 317, 514
- ModelMeshPart.Tag, 301
- ModelMeshPartCollection, 297, 306
- module monorepo, 65
- MojoShader, 523
- morph targets
 - glTF evidence, 325
- MorphTargetEXT, 303
- Motion, 354
- Mouse, 34, 332, 333, 533
- MouseCursor, 69, 333
- MouseState, 332, 333, 533
- namespace, 15
- NANOVG, 517
- Net, 25
- NetworkGamer, 370, 535
- NetworkSession, 364, 369, 370, 535
- NetworkSession.GamerJoined, 380
- NetworkSessionProperties, 371
- NotImplementedException, 57, 63, 403
- NotSupportedException, 106, 142, 151, 156, 252, 398
- Object, 380, 391, 392
- ObjectDisposedException, 398, 523
- OcclusionQuery, 151, 180, 181, 211, 234, 515
- OcclusionQuery::PixelCount(), 172
- open gap, 507
- OpenGL loader, 415
- OPENGL1, 517
- OPENGL2, 517
- OPENGL33, 517
- OPENGL4, 517
- OPENGLES1, 517
- OPENGLES2, 517
- OPENGLES3, 517
- OPENVG, 517
- oracle (D3D9), 523
- OrderedDictionary, 381
- ownership, 15
- PackedVector, 19
- PacketReader, 371
- PacketWriter, 371
- path containment, 291
- PbrEffect, 190, 205, 323, 523, 530
- permanent deviation, 403
- pixel oracle, 467
- pixel testing
 - oracle scope, 467
- PIXIJS, 517
- Plane, 56, 61, 62, 512
- Plane::Transform, 53, 62
- PlaneIntersectionType::Front, 62
- Planet Blupi, 489
- platform evidence, 429
- PlatformFactory, 429
- PlatformNotSupportedException, 403
- PlayerIndex, 67, 70, 331
- Playlist, 350
- Point, 57, 61, 63, 511
- PORTABLEGL, 517
- porting, 485
- Power, 338, 339, 341
- PowerInfo, 358, 359
- PowerStateEXT, 339
- PresentationMode, 35, 153, 184
- PresentationParameters, 50, 81, 87, 153, 184, 226, 238, 246
- Program, 417
- ProgramPipeline, 417
- Proton, 437
- QualityOfService, 370
- Quaternion, 53, 56, 61, 511
- Query::result_u64(), 421
- Queue, 381
- RasterizerState, 91, 124, 157, 214, 250, 515
- RasterizerState.DepthBias, 181, 199, 209
- RasterizerState::CullCounterClockwise, 130
- RasterizerState::ScissorTestEnable, 82
- RawMesh, 137
- Ray, 12, 56, 61, 512
- re-audit, 507
- reading paths, v
- RecoverableResource, 419, 420
- Rectangle, 28, 37, 53, 57, 61, 92, 93, 511
- Rectangle::Contains, 63
- ReflectiveReader<T>, 523
- renderer identity, 517

- RenderPipelineSettings, 26
- RenderTarget2D, 87, 101, 107, 109, 168, 169, 178, 181, 183, 187, 188, 191–193, 198, 201, 202, 221, 223, 224, 230, 240, 247, 257, 260, 264, 450, 492, 513
- RenderTarget2D::GetData, 85
- RenderTargetBinding, 85, 106, 107
- RenderTargetCube, 104, 106–108, 156, 181, 183, 191–193, 195, 200, 201, 245, 264, 513
- RenderTargetUsage, 107, 252
- resource limit, 291
- ResourceContentManager, 272
- ResourceRegistration, 419, 420
- ResourceRegistry, 419, 420
- revision pin, vii, 503
- roadmap, 507
- runtime substitution, 471

- sample program, 499
- SamplerState, 129, 515
- ScopedBind, 419
- ScopedDebugGroup, 418
- SDL_GPU, 517, 523
- SDL_RENDERER, 517, 523
- SDL3, 523
- SdlGpuEffectRenderer, 197
- SdlGpuRenderTarget2DState, 201
- SdlGpuRenderTargetCubeRenderer, 201
- SdlGpuRenderTargetCubeState, 201
- SdlGpuRenderTargetRenderer, 201
- SdlRenderer, 196
- Semantic Versioning, 26
- SendDataOptions, 370
- SensorBase::TimeBetweenUpdates, 381
- SensorBase<TSensorReading>, 353
- Sensors, 16, 338, 339, 341
- shader, 131
- ShaderEffect, 5, 82, 87, 101, 102, 105, 111, 113, 131, 133–139, 151, 196–198, 205, 243, 256, 508, 515, 530
- sharp-runtime, 379, 523
 - components, 387
 - verification, 409
- SharpRuntime, 523
- sibling repositories, 9
- SignedInGamer, 361, 535
- SKIA, 517
- SkinnedEffect, 17, 114, 115, 119–121, 190, 191, 197, 205, 250, 283, 315, 317, 373, 374, 514, 524
- SkinnedEffect::SetBoneTransforms, 302, 304
- SkinnedEffect::WeightsPerVertex, 17
- SkinnedModelEXT, 300, 301, 317, 323, 373, 374, 523, 530, 536
- SkinnedPbrEffect, 315, 317, 530
- skinning, 317
- SkinningData, 300, 301, 306, 317, 318, 530, 547
- SOFTWARE, 517, 524
- SOKOL, 517
- Song, 6, 270, 273, 349, 350, 492, 522, 534
- SongCollection, 350
- SortedDictionary, 381
- SortedList, 381
- SortedSet, 381
- SoundBank, 345
- SoundEffect, 35, 36, 270, 271, 281, 343, 344, 534
- SoundEffectInstance, 343, 344, 534
- SoundEffectReader, 273, 345
- source audit, 503
- Speedy Blupi, 489
- SpriteBatch, vi, 3, 4, 32, 33, 37, 91, 93, 95, 96, 98, 99, 108, 117, 127, 133, 134, 137, 139, 172, 173, 177, 188, 189, 195, 201, 202, 222, 235, 237, 241–244, 246, 251, 253–255, 363, 491, 492, 513, 524
- SpriteEffect, 91
- SpriteEffects, 91, 92, 94, 96, 251
- SpriteFont, 91, 95, 96, 188, 246, 254, 270, 363, 513
- SpriteSortMode, 91, 94, 173
- SpriteSortMode.Immediate, 234
- SslStream, 405
- Stack, 381
- StencilOperation, 125
- stock effects, 524
- Storage, 524
- StorageContainer, 366, 367, 524, 536
- StorageDevice, 362, 365, 524, 536
- StorageDeviceNotConnectedException, 367
- Stream, 380
- stride ABI, 313
- String, 391, 536
- STUB, 517
- SurfaceFormat, 101, 103, 104, 109, 172, 183, 207, 226, 234
- SurfaceFormat::Color, 101, 103, 104, 109
- SVG_DOM, 517
- Sync, 419
- System, 98
- System::Activator, 383, 384, 394
- System::Delegate, 393, 404
- System::EventHandler<TEventArgs>, 380
- System::Exception, 98, 380, 392, 406
- System::GC, 393, 403
- System::IDisposable, 380, 393
- System::IO::FileShare, 367
- System::IO::MemoryStream, 398
- System::IO::Stream, 366, 537
- System::MulticastAction<Args...>, 380, 393, 404
- System::Object, 380, 391, 536
- System::String, 380, 392, 396
- System::SystemException, 98, 406
- System::Text::StringBuilder, 392
- System::Threading::ReaderWriterLockSlim, 405
- System::TimeSpan, 381
- System::Type, 383, 403
- System::Type::From<T>(), 393
- SystemTray, 357

- Task, 401
- task tracking, 503
- Task::Wait, 382
- Task::WhenAll, 382
- Task::WhenAny, 382
- TaskCanceledException, 382
- TaskCompletionSource<TResult>::TrySetResult, 411

- TaskT<TResult>, 401
- test architecture, 459
- test counting, 463
- test oracle, 467
- test population, 463
- Text::StringBuilder, 396
- Text::UTF8Encoding, 399
- Texture, 101, 105, 107, 417, 420
- Texture2D, 32, 36, 69, 88, 97, 101–104, 107, 109, 150, 173, 187, 189, 191, 194, 203–205, 224, 226, 234, 250, 254, 270, 271, 280, 358, 492, 513
- Texture2D::GetData(), 103
- Texture2D::Width, 37
- Texture3D, 61, 101, 102, 104, 106, 129, 200, 201, 203, 204, 213, 242, 252, 264, 513
- TextureAddressMode, 128, 253
- TextureCollection, 101
- TextureCube, 61, 101, 102, 104–107, 119, 129, 191, 192, 200, 204, 213, 250, 252, 264, 270, 271, 280, 281, 513
- TextureCube::GetData(), 242
- TextureFilter, 128, 251
- Threading::Tasks::Task, 381
- Threading::Thread, 383
- TimeSpan, 34, 381, 537
- TINYGL, 517
- TitleContainer, 51
- TitleContainer::OpenStream, 272
- TitleLocation, 51, 272
- TouchCollection, 336
- TouchLocation, 336
- TouchLocationState, 336
- TouchPanel, 336, 533
- TransformFeedback, 417
- tvOS, 453
- Type, 394, 407
- umbrella target, 69
- UniformCache, 418
- Unsupported3DGraphicsCallBehavior, 144
- UTF8Encoding, 399, 401
- Vector2, 34, 54, 61, 63, 92, 93, 335, 511
- Vector3, 53–55, 61, 97, 511
- Vector3::Backward, 54
- Vector3::Forward, 54
- Vector4, 54, 55, 61, 511
- vertex packing, 313
- VertexArray, 420
- VertexBuffer, 109, 141, 142, 175, 212, 250, 297, 300, 302, 513
- VertexBuffer::SetData, 109, 141
- VertexBufferBinding, 142, 212
- VertexDeclaration, 62, 86, 110, 111, 138, 141, 175, 211, 242, 249, 524
- VertexElement, 142, 249, 524
- VertexElementFormat, 138, 141
- VertexElementUsage, 110
- VertexPositionColor, 86, 89, 109, 238
- VertexPositionColorTexture, 86, 109, 117
- VertexPositionNormalTexture, 86, 109, 137
- VertexPositionTexture, 86, 109
- VibrateController, 355, 535
- Video, 270, 273, 274, 351, 534
- VideoPlayer, 28, 351, 534
- VideoPlayer::Play, 274
- VideoSoundtrackType, 351
- Viewport, 108, 254
- Viewport::Width, 35
- VULKAN, 517, 524
- VulkanRenderer, 196, 198
- VulkanRenderTargetRenderer, 187
- VulkanTextureRenderer, 187
- WaveBank, 345
- web build, 441
- Web renderer evidence, 449
- WebAssembly, 441
- WEBGL1, 517
- WEBGL2, 517
- WEBGPU, 517, 524
- WICKED, 517
- Windows, 433
- Wine, 437
- workflow, 479
- XACT, 524
- XNA, 3
- xna4-spec, 495, 524
- XNB, 275
- .xnb, 522
- XnbReadLimits, 280